达梦数据

全栈数据产品与解决方案提供商

股票代码 688692

达梦数据库管理系统
系统管理员手册



目录

第一部分 数据库体系架构.....1

第 1 章 DM 逻辑结构概述.....1

1.1 数据库和实例.....1

1.1.1 数据库.....1

1.1.2 实例.....1

1.2 DM 逻辑存储结构.....2

1.2.1 数据表空间.....5

1.2.2 日志表空间.....11

第 2 章 DM 物理存储结构.....14

2.1 配置文件.....15

2.1.1 服务器配置文件.....15

2.1.2 客户端配置文件.....254

2.2 控制文件.....289

2.3 数据文件.....290

2.4 重做日志文件.....292

2.5 归档日志文件.....293

2.6 物理逻辑日志文件.....293

2.7 备份文件.....294

2.8 SQL 日志文件.....294

2.9 事件日志文件.....296

第 3 章 DM 内存结构.....298

3.1 内存池.....298
3.1.1 共享内存池.....298
3.1.2 运行时内存池.....299
3.2 缓冲区.....299
3.2.1 数据缓冲区.....299
3.2.2 日志缓冲区.....300
3.2.3 字典缓冲区.....301
3.2.4 SQL 缓冲区.....301
3.3 排序区.....302
3.4 哈希区.....303
第 4 章 管理 DM 线程.....304
4.1 监听线程.....304
4.2 工作线程.....304
4.3 IO 线程.....305
4.4 调度线程.....306
4.5 日志 FLUSH 线程.....306
4.6 日志归档线程.....307
4.7 日志 APPLY 线程.....307
4.8 定时器线程.....307
4.9 MAL 系统相关线程.....308
4.10 其他线程.....308
4.11 线程信息的查看.....308
第二部分 基础数据库管理.....310

第 5 章 DM 系统管理员.....310
5.1 DM 系统管理员的类型.....310
5.2 数据库管理员的任务.....312
5.3 数据库安全员的任务.....314
5.4 数据库审计员的任务.....314
5.5 数据库对象操作员的任务.....315
第 6 章 创建和配置 DM 数据库.....316
6.1 创建 DM 数据库.....316
6.2 使用数据库配置工具创建数据库.....317
6.2.1 启动数据库配置助手.....317
6.2.2 使用数据库配置工具创建数据库.....318
6.3 使用 dminit 创建数据库.....330
6.4 注册数据库服务.....348
6.5 查看数据库信息.....350
6.6 删除数据库.....351
6.7 删除数据库服务.....354
第 7 章 启动和关闭数据库.....359
7.1 启动数据库.....359
7.1.1 Windows 系统.....359
7.1.2 Linux 系统.....363
7.1.3 检查 LICENSE.....364
7.2 数据库状态和模式.....364
7.3 关闭数据库.....366

7.3.1 Windows 系统 366
7.3.2 Linux 系统 367
第 8 章 管理模式对象的空间 368
8.1 设置存储参数 368
8.1.1 普通表和索引 368
8.1.2 堆表 369
8.1.3 HUGE 表 370
8.2 收回多余的空间 371
8.3 用户和表上的空间限制 371
8.3.1 用户的空间限制 371
8.3.2 表对象的空间限制 372
8.4 查看模式对象的空间使用 372
8.4.1 查看用户占用的空间 372
8.4.2 查看表占用的空间 372
8.4.3 查看表使用的页数 372
8.4.4 查看表空闲堆栈占用的空间 373
8.4.5 查看索引占用的空间 373
8.4.6 查看索引使用的页数 373
8.5 数据类型的空间使用 373
第 9 章 管理表 376
9.1 管理表的准则 377
9.1.1 设计表 377
9.1.2 指定表的存储空间上限 377

9.1.3 指定表的存储位置 378
9.2 创建表 378
9.2.1 创建普通表 378
9.2.2 指定表的聚集索引 379
9.2.3 指定表的填充因子 380
9.2.4 查询建表 381
9.2.5 创建临时表 383
9.3 修改表 384
9.4 删除表 385
9.5 清空表 386
9.5.1 使用 DELETE 386
9.5.2 使用 DROP 和 CREATE 386
9.5.3 使用 TRUNCATE 387
9.6 查看表信息 387
9.6.1 查看表定义 387
9.6.2 查看自增列信息 388
9.6.3 查看表的空间使用情况 388
第 10 章 管理索引 390
10.1 索引分类 390
10.1.1 聚集索引和非聚集索引 390
10.1.2 功能索引 392
10.1.3 虚索引和实索引 393
10.1.4 单列索引和复合索引 395

10.1.5 全局索引和局部索引 395
10.2 管理索引的准则 402
10.2.1 在表中插入数据后创建索引 402
10.2.2 索引正确的表和列 402
10.2.3 为性能而安排索引列 403
10.2.4 限制索引的数量 403
10.2.5 估计索引大小和设置存储参数 403
10.2.6 为索引指定表空间 404
10.3 创建索引 404
10.3.1 显式地创建索引 405
10.3.2 隐式地创建索引 410
10.4 如何在 SQL 语句中使用索引 411
10.5 重建索引 413
10.6 删除索引 414
10.7 查看索引信息 415
第 11 章 管理触发器 417
11.1 触发器的使用 418
11.2 表级触发器 421
11.3 事件触发器 422
11.4 时间触发器 423
11.5 触发器总结 424
第 12 章 管理视图、序列和同义词 426
12.1 管理视图 426

12.2 管理序列 427
12.3 管理同义词 428
12.4 查看视图、序列和同义词信息 428
第 13 章 模式对象的常规管理 430
13.1 在单个操作中创建多个模式对象 431
13.2 重命名模式对象 432
13.3 启用和停用触发器 433
13.4 管理完整性约束 434
13.4.1 完整性约束状态 434
13.4.2 定义完整性约束 434
13.4.3 修改或删除现有的完整性约束 435
13.4.4 查看约束信息 435
13.5 管理对象依赖性 436
13.6 管理对象名称解析 436
13.6.1 SQL 语句对象 436
13.6.2 DMSQL 程序对象 437
13.7 显示有关模式对象的信息 438
第三部分 高级数据库管理 439
第 14 章 数据库布局和存储管理 439
14.1 管理表空间 439
14.1.1 创建表空间 439
14.1.2 扩展表空间 439
14.1.3 删除表空间 440

14.1.4 修改表空间名 440
14.1.5 修改表空间状态 440
14.1.6 查询表空间与数据文件对应关系 441
14.1.7 表空间文件失效检查 441
14.1.8 表空间失效文件恢复 441
14.2 管理数据文件 442
14.2.1 添加数据文件 442
14.2.2 删除数据文件 443
14.2.3 扩展数据文件的大小 443
14.2.4 缩减数据文件的大小 443
14.2.5 指定数据文件的扩展属性 443
14.2.6 修改数据文件的路径 444
14.2.7 添加 HUGE 数据文件路径 445
14.3 管理重做日志文件 445
14.3.1 添加重做日志文件 445
14.3.2 扩展重做日志文件 445
14.3.3 删除重做日志文件 446
14.4 管理回滚表空间 446
14.5 管理控制文件 447
第 15 章 管理分区表和分区索引 448
15.1 分区的概念 448
15.2 分区的方法 449
15.3 创建水平分区表 449

15.3.1 创建范围分区表 450
15.3.2 创建 LIST 分区表 452
15.3.3 创建哈希分区表 453
15.3.4 创建多级分区表 455
15.4 在水平分区表上建立索引 457
15.5 维护水平分区表 459
15.5.1 增加分区 459
15.5.2 删除分区 460
15.5.3 交换分区 461
15.5.4 合并分区 462
15.5.5 拆分分区 463
15.6 水平分区表的限制 463
第 16 章 管理列存储表 467
16.1 列存储简介 467
16.2 HUGE 表简介 468
16.3 HUGE 表的分类 470
16.3.1 非事务型 HUGE 表 470
16.3.2 事务型 HUGE 表 472
16.4 创建 HUGE 表 473
16.5 HUGE 表使用说明 476
16.6 查看有关 HUGE 表的信息 478
第 17 章 管理堆表 479
17.1 堆表简介 479

17.2 创建堆表 480
17.3 堆表的限制 481
17.4 维护堆表 482
17.5 查看有关堆表的信息 482
第 18 章 全文检索 483
18.1 全文检索概述 483
18.2 创建全文索引 486
18.3 更新全文索引 486
18.4 执行全文检索 487
18.5 删除全文索引 489
第 19 章 管理事务 490
19.1 事务简介 490
19.2 事务特性 491
19.2.1 原子性 492
19.2.2 一致性 492
19.2.3 隔离性 492
19.2.4 持久性 493
19.3 提交事务 493
19.3.1 自动提交模式 494
19.3.2 手动提交模式 494
19.3.3 隐式提交 494
19.3.4 事务组提交 495
19.4 回滚事务 496

19.4.1 自动回滚 496
19.4.2 手动回滚 497
19.4.3 回滚到保存点 497
19.4.4 语句级回滚 498
19.5 事务锁定 498
19.5.1 锁模式 498
19.5.2 锁粒度 499
19.5.3 查看锁 502
19.6 多版本 502
19.6.1 物理记录格式 503
19.6.2 回滚记录格式 503
19.6.3 可见性原则 503
19.6.4 历史数据获取 504
19.6.5 回滚段自动清理 505
19.7 事务隔离级 505
19.7.1 读提交隔离级 507
19.7.2 串行化隔离级 507
19.7.3 读未提交隔离级 507
19.7.4 只读事务 508
19.8 锁等待与死锁检测 508
19.9 闪回 509
19.10 XA 事务 509
第四部分 故障排除和性能优化 511

第 20 章 问题跟踪和解决 511
20.1 问题分析 511
20.2 监控系统性能(V$) 513
20.3 检查数据物理一致性 513
20.4 调整配置参数 515
20.5 优化数据库布局 518
第 21 章 动态管理和性能视图 520
21.1 理解动态管理视图 520
21.2 使用动态管理视图 520
第 22 章 查询优化 527
22.1 优化目标 527
22.2 查询优化器 527
22.2.1 查询转换 528
22.2.2 估算代价 528
22.2.3 生成计划 529
22.3 数据访问路径 529
22.4 连接 530
22.5 统计信息 532
22.5.1 统计信息简介 533
22.5.2 统计信息用法 534
22.5.3 应用实例 538
22.6 执行计划 545
22.6.1 自适应计划 554

22.7 使用索引 556
22.8 并行查询 557
22.8.1 多机并行查询 557
22.8.2 本地并行查询 557
22.8.3 多机和本地并行查询 563
22.9 查询计划重用 566
22.10 结果集重用 566
第 23 章 SQL 调优 569
23.1 简介 569
23.2 调优目标 569
23.3 确定高负载的 SQL 570
23.4 自动 SQL 调整 570
23.5 开发有效的 SQL 语句 571
23.6 使用优化器提示 574
23.6.1 INI 参数提示 574
23.6.2 索引提示 575
23.6.3 连接方法提示 577
23.6.4 连接顺序提示 589
23.6.5 统计信息提示 591
23.6.6 MPP 本地对象提示 592
23.6.7 忽略重复键值提示 593
23.6.8 禁用计划缓存提示 594
23.6.9 DMDPC 数据分发方式提示 594

23.6.10 分区表遍历操作符 PLL 提示 598
23.6.11 单节点登录提示 599
23.6.12 视图不替换 SQL 记录提示 602
23.6.13 DMDPC 读写分离设置事务的数据一致性级别提示 603
第五部分 数据高可用性 605
第 24 章 故障恢复 605
24.1 简介 605
24.2 REDO 日志 605
24.3 重做日志归档 606
24.3.1 本地归档 607
24.3.2 实时归档 607
24.3.3 即时归档 608
24.3.4 异步归档 608
24.3.5 同步归档 608
24.3.6 远程归档 609
24.3.7 RAFT 归档 609
24.3.8 LEARNER 归档 609
24.4 检查点 610
24.5 回滚段与回滚记录 611
24.6 系统故障恢复 611
24.7 介质故障恢复 612
第 25 章 版本升级 613
25.1 日志格式升级 613

25.1.1 版本说明 613
25.1.2 升级步骤 614
25.2 回滚管理段升级 614
25.2.1 版本说明 615
25.2.2 升级步骤 615
第六部分 附录 617
附录 1 数据字典 617
附录 2 动态性能视图 662
附录 3 V 视图 839
附录 4 执行计划操作符 843
DM 技术支持 877
手册版本 879

第一部分 数据库体系架构

第1章 DM 逻辑结构概述

1.1 数据库和实例

在 DM7 之前版本的 DM 数据库中，“数据库”和“实例”这两个术语经常可以互相替换，意义也很相近。在 DM7 以及之后版本的数据库中，“数据库”和“实例”这两个概念之间有着很大的差别，甚至可以说它们是两个完全不同的实体。

1.1.1 数据库

在有些情况下，数据库的概念包含的内容会很广泛。如在单独提到 DM 数据库时，可能指的是 DM 数据库产品，也有可能是正在运行的 DM 数据库实例，还可能是 DM 数据库运行中所需的一系列物理文件的集合等。但是，当同时出现 DM 数据库和实例时，DM 数据库指的是磁盘上存放在 DM 数据库中的数据的集合，一般包括：数据文件、日志文件、控制文件以及临时数据文件等。

1.1.2 实例

实例一般是由一组正在运行的 DM 后台进程/线程以及一个大型的共享内存组成。简单来说，实例就是操作 DM 数据库的一种手段，是用来访问数据库的内存结构以及后台进程的集合。

DM 数据库存储在服务器的磁盘上，而 DM 实例则存储于服务器的内存中。通过运行 DM 实例，可以操作 DM 数据库中的内容。在任何时候，一个实例只能与一个数据库进行关联（装载、打开或者挂起数据库）。在大多数情况下，一个数据库也只有一个实例对其进行操作。

但是在 DM 数据共享集群（DMDSC）中，多个实例可以同时装载并打开一个数据库（位于一组由多台服务器共享的物理磁盘上）。此时，我们可以同时从多台不同的计算机访问这个数据库。

1.2 DM 逻辑存储结构

逻辑存储结构是数据库向用户展现出的一种抽象的数据存储的结构，用于描述数据的逻辑关系。由于数据在物理磁盘上的实际物理存储是极其复杂的，用户不可能知悉存储的全部位置、大小或结构。为了用户能专注于业务逻辑和数据关系而不必囿于复杂的存储情况，数据库对物理存储结构进行逻辑上的抽象化管理。基于该管理，数据库可以向外界提供一个清晰、简易的数据组织方式和数据关系，也就是逻辑存储结构。它隐藏了数据物理存储结构的复杂性，提供了一个清晰、易于理解和操作的逻辑数据结构，实现了数据独立性，极大地简化了应用程序的开发和管理工作。理解逻辑存储结构是理解数据库如何组织和管理的基石。

DM 数据库的主要存储内容为用户数据、系统数据和对其操作产生的日志信息等，DM 将这些内容存放在磁盘上的物理结构中，即数据库文件（数据文件、日志文件等）。然而，用户并不能直接对数据库文件进行读取操作，而是通过实例对逻辑结构进行操作，进而实现对数据库文件的读写操作。因此引入表空间这一逻辑存储管理单元，作为逻辑与物理的桥梁。用户可以在表空间层面进行各类操作，并最终映射到对应文件中。

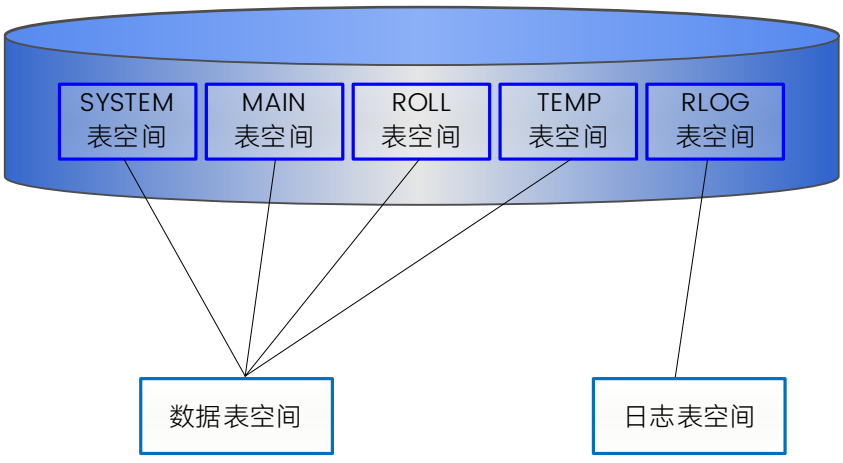


图 1.1 DM 表空间种类

不同表空间存储的内容各不相同，根据其中存储内容种类的不同，表空间划分为不同的种类：

- 存储用户数据、系统数据的表空间为数据表空间。数据表空间又根据其是否能存储 HUGE 表，将其分为了普通表空间和混合表空间：普通表空间不能存储 HUGE 表，而混合表空间可以同时存储普通（非 HUGE）表和 HUGE 表，其中 HUGE 数据文件存储在混合表空间定义中指定的 HUGE 数据文件路径下。可以通过为普通表空间增加指定 HUGE 数据文件路径将普通表空间升级为混合表空间。
- 存储数据库内各类操作产生的日志信息的表空间为日志表空间。

在创建数据库时，系统会自动创建 5 个表空间：SYSTEM 表空间、ROLL 表空间、MAIN 表空间、TEMP 表空间和 RLOG 表空间。其中，SYSTEM 表空间、ROLL 表空间、MAIN 表空间和 TEMP 表空间为数据表空间；RLOG 表空间为日志表空间。以下是五个表空间的详细介绍：

1. SYSTEM 表空间存放 DM 数据库的字典信息，用户不能在 SYSTEM 表空间创建表和索引。SYSTEM 表空间又称为系统表空间。

2. ROLL 表空间完全由 DM 数据库自动维护，用户无需干预。该表空间用来存放事务运行过程中执行 DML(数据操作语言)操作之前的值，从而为访问该表的其他用户提供表数据的读一致性视图。

3. MAIN 表空间在初始化库的时候，就会自动创建一个大小为 128MB 的数据文件 MAIN.DBF，以及一个 HMAIN 目录作为 HUGE 数据文件路径，因此 MAIN 表空间为混合表空间。在创建用户时，如果没有指定默认表空间，则系统自动指定 MAIN 表空间为用户默认的表空间。

4. TEMP 表空间完全由 DM 数据库自动维护。当用户的 SQL 语句需要磁盘空间来完成某个操作时，DM 数据库会从 TEMP 表空间分配临时段。如创建索引、无法在内存中完成的排序操作、SQL 语句中间结果集以及用户创建的临时表等都会使用到 TEMP 表空间。

5. RLOG 表空间称为日志表空间，初始化库时会自动创建 2 个后缀名为 log 的日志文件，用于存放重做日志，用户可主动添加日志文件或者扩展日志文件。日志文件包括当前活动日志文件及非活动日志文件，当前日志文件写满后系统会自动切换到下一个日志文件，日志文件可循环使用。自动创建的日志文件大小受 LOG_SIZE 控制，初始化时可通过调整 LOG_SIZE 大小调整两个日志文件的大小。

每一个用户都有一个默认的表空间。对于 SYS、SYSSSO、SYSAUDITOR 系统用户，默认的用户表空间是 SYSTEM；SYSDBA 系统用户的默认表空间为 MAIN；新创建的用户如果没有指定默认表空间，则系统自动指定 MAIN 表空间为用户默认的表空间。如果用户在创建表的时候，指定了存储表空间 A，并且和当前用户的默认表空间 B 不一致时，表存储在用户指定的表空间 A 中，并且默认情况下，在这张表上面建立的索引也将存储在 A 中，但是用户的默认表空间是不变的，仍为 B。

一般情况下，建议用户自己创建一个表空间来存放业务数据，或者将数据存放在默认的用户表空间 MAIN 中。

用户可以通过执行如下语句来查看 SYSTEM、ROLL、MAIN 以及 TEMP 的表空间相关信息。

```
SELECT * FROM V$TABLESPACE;
```

用户可以通过执行如下语句来查看 RLOG 表空间相关信息。

```
SELECT * FROM V$RLOG;              --查看日志空间信息
SELECT * FROM V$RLOGFILE;          --RLOG 文件信息
SELECT * FROM V$LOG_HISTORY;       --日志文件切换历史
```

结果集中各字段的含义请参考附录部分动态性能视图章节中对各系统视图的介绍，关于

表空间该如何使用，请参考第 14 章 数据库布局和存储管理。

1.2.1 数据表空间

为了进行更加高效的存储管理，同时提高磁盘空间的利用率，数据表空间被进一步划分为段、簇和页（也称块）。数据库中的数据信息（记录）就存储在页中，多个页组合成为簇，多个簇组合成为段。图 1.2 显示了这些数据结构之间的关系

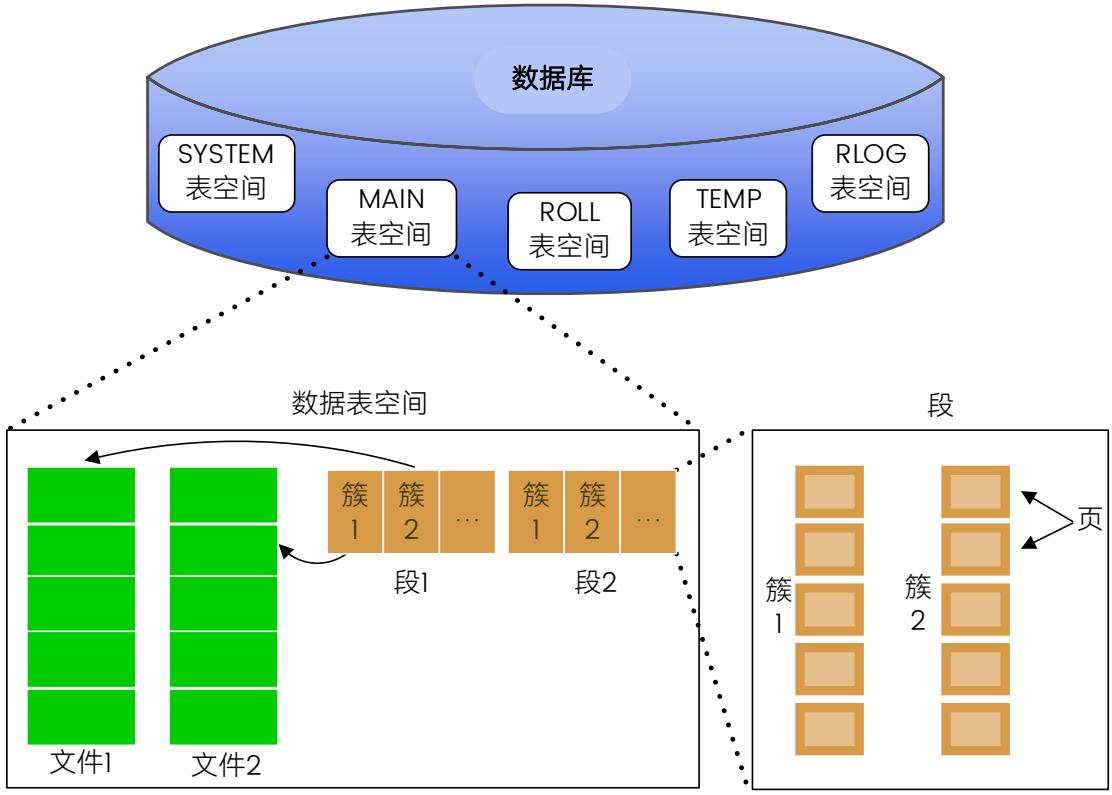


图 1.2 数据表空间、数据文件、段、簇、页的关系

可以看出，在 DM8 中存储的层次结构如下：

1. 数据库由一个或多个表空间组成；
2. 每个表空间对应一个或多个文件；
3. 每个数据文件由一个或多个簇组成；
4. 段是簇的上级逻辑单元，一个段可以跨多个数据文件；
5. 簇由磁盘上连续的页组成，一个簇总是在一个数据文件中；

6. 页是数据库中最小的分配单元，也是数据库中使用的最小的 IO 单元。

1.2.1.1 记录

数据库表中的每一行是一条记录。在 DM 中，除了 HUGE 表，其他的表都是在数据页中按记录存储数据的。也就是说，记录是存储在数据页中的，记录并不是 DM 数据库的存储单位，页才是。由于记录不能跨页存储，这样记录的长度就受到数据页大小的限制。数据页中还包含了页头控制信息等空间，因此 DM 规定每条记录的总长度不能超过页面大小的一半。

1.2.1.2 页

数据页（也称数据块）是 DM 数据库中最小的数据存储单元。页的大小对应物理存储空间上特定数量的存储字节，在 DM 数据库中，页大小可以为 4KB、8KB、16KB 或者 32KB，用户在创建数据库时可以指定，默认大小为 8KB，一旦创建好了数据库，则在该库的整个生命周期内，页大小都不能够改变。图 1.3 显示了 DM 数据库页的典型格式。

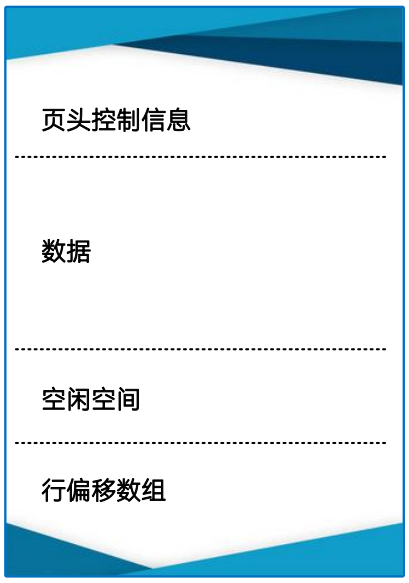


图 1.3 DM 数据页的组成

页头控制信息包含了关于页类型、页地址等信息。页的中部存放数据，为了更好地利用数据页，在数据页的尾部专门留出一部分空间用于存放行偏移数组，行偏移数组用于标识页

上的空间占用情况以便管理数据页自身的空间。

在绝大多数情况下，用户都无需干预 DM 数据库对数据页的管理。但是 DM 数据库还是提供了选项供用户选择，使得在某些情况下能够为用户提供更佳的数据处理性能。

FILLFACTOR 是 DM 数据库提供的一个与性能有关的数据页级存储参数，它指定一个数据页初始化后插入数据时最大可以使用空间的百分比（100），该值在创建表/索引时可以指定。设置 FILLFACTOR 参数的值，是为了指定数据页中的可用空间百分比（FILLFACTOR）和可扩展空间百分比（100-FILLFACTOR）。可用空间用来执行更多的 INSERT 操作，可扩展空间用来为数据页保留一定的空间，以防止在今后的更新操作中增加列或者修改变长列的长度时，引起数据页的频繁分裂。当插入的数据占据的数据页空间百分比低于 FILLFACTOR 时，允许数据插入该页，否则将当前数据页中的数据分为两部分，一部分保留在当前数据页中，另一部分存入一个新页中。

对于 DBA 来说，使用 FILLFACTOR 时应该在空间和性能之间进行权衡。为了充分利用空间，用户可以设置一个很高的 FILLFACTOR 值，如 100，但是这可能会导致在后续更新数据时，频繁引起页分裂，而导致需要大量的 I/O 操作。为了提高更新数据的性能，可以设置一个相对较低（但不是过低）的 FILLFACTOR 值，使得后续执行更新操作时，可以尽量避免数据页的分裂，提升 I/O 性能，不过这是以牺牲空间利用率来换取性能的提高。

1.2.1.3 簇

簇是数据页的上级逻辑单元，由同一个数据文件中 16 个或 32 个或 64 个连续的数据页组成。在 DM 数据库中，簇的大小由用户在创建数据库时指定，默认大小为 16。假定某个数据文件大小为 32MB，页大小为 8KB，则共有 32MB/8KB/16=256 个簇，每个簇的大小为 8KB*16=128KB。和数据页的大小一样，一旦创建好数据库，此后该数据库的簇的大小就不能够改变。

分配数据簇

一般的，当创建一个表/索引的时候，DM 数据库会为表/索引的数据段分配至少一个簇，同时数据库会自动生成对应数量的空闲数据页，供后续操作使用。如果初始分配的簇中所有数据页都已经用完，或者新插入/更新数据需要更多的空间，DM 数据库将自动分配新的簇。在缺省情况下，DM 数据库在创建表/索引时，初始分配 1 个簇，当初始分配的空间用完时，DM 数据库会自动扩展。另外，DM 还提供了延迟段分配的功能，特点为不会在进行创建表/索引时马上为表/索引分配存储结构，具体介绍在 1.2.1.4 段中进行进一步说明。

当 DM 数据库的表空间为新的簇分配空闲空间时，首先在表空间按文件从小到大的顺序在各个数据文件中查找可用的空闲簇，找到后进行分配；如果各数据文件都没有空闲簇，则在各数据文件中查找空闲空间足够的，将需要的空间先进行格式化，然后进行分配；如果各文件的空闲空间也不够，则选一个数据文件进行扩充。

释放数据簇

对于用户数据表空间，在用户将一个数据段对应的表/索引对象 DROP 之前，该表对应的数据段会保留至少 1 个簇不被回收到表空间中。在删除表/索引对象中的记录的时候，DM 数据库通过修改数据文件中的位图来释放簇，释放后的簇被视为空闲簇，可以供其他对象使用。当用户删除了表中所有记录时，DM 数据库仍然会为该表保留 1-2 个簇供后续使用。若用户使用 DROP 语句来删除表/索引对象，则此表/索引对应的段以及段中包含的簇全部收回，并供存储于此表空间的其他模式对象使用。

对于临时表空间，DM 数据库会自动释放在执行 SQL 过程中产生的临时段，并将属于此临时段的簇空间还给临时表空间。

对于回滚表空间，DM 数据库将定期检查回滚段，并确定是否需要从回滚段中释放一个或多个簇。

1.2.1.4 段

段是簇的上级逻辑分区单元，它由一组簇组成。在同一个表空间中，段可以包含来自不

同文件的簇，即一个段可以跨越不同的文件。而一个簇以及该簇所包含的数据页则只能来自一个文件，是连续的 16 或 32 或 64 个数据页。由于簇的数量是按需分配的，数据段中的不同簇在磁盘上不一定连续。

1. 数据段

段可以被定义成特定对象的数据结构，如表数据段或索引数据段。表中的数据以表数据段结构存储，索引中的数据以索引数据段结构存储。DM 以簇为单位给每个数据段分配空间，当数据段的簇空间用完时，DM 数据库就给该段重新分配簇，段的分配和释放完全由 DM 数据库自动完成，可以在创建表/索引时设置存储参数来决定数据段的簇如何分配。

当用户使用 CREATE 语句创建表/索引时，DM 创建相应的数据段。表/索引的存储参数用来决定对应数据段的簇如何被分配，这些参数将会影响与对象相关的数据段的存储与访问效率。对于分区表，每个分区使用单独的数据段来容纳所有数据，对于分区表上的非分区索引，使用一个索引数据段来容纳所有数据，而对于分区索引，每个分区使用一个单独索引数据段来容纳其数据。表的数据段和与其相关的索引段不一定要存储在同一表空间中，用户可以在创建表和索引时，指定不同的表空间存储参数。

2．临时段

在 DM 数据库中，所有的临时段都创建在临时表空间中，这样可以分流磁盘设备的 I/O，也可以减少由于在 SYSTEM 或其他表空间内频繁创建临时数据段而造成的碎片。

当处理一个查询时，经常需要为 SQL 语句的解析与执行的中间结果准备临时空间。DM 数据库会自动地分配临时段的磁盘空间。例如，DM 在进行排序操作时就可能需要使用临时段，当排序操作可以在内存中执行，或设法利用索引就可以执行时，就不必创建临时段。对于临时表及其索引，DM 数据库也会为它们分配临时段。

临时段的分配和释放完全由系统自动控制，用户不能手工进行干预。

3．回滚段

DM 数据库在回滚表空间的回滚段中保存了用于恢复数据库操作的信息。对于未提交事务，当执行回滚语句时，回滚记录被用来做回滚变更。在数据库恢复阶段，回滚记录被用来做所有未提交变更的回滚。在多个并发事务运行期间，回滚段还为用户提供读一致性，所有正在读取受影响行的用户将不会看到行中的任何变动，直到他们事务提交后发出新的查询。

DM 数据库提供了全自动回滚管理机制来管理回滚信息和回滚空间，自动回滚管理消除了管理回滚段的复杂性。此外，系统将尽可能保存回滚信息，来满足用户查询回滚信息的需要。事务被提交后，回滚数据不能再回滚或者恢复，但是从数据读一致性的角度出发，长时间运行查询可能需要这些早期的回滚信息来生成早期的数据页镜像，基于此，数据库需要尽可能长时间的保存回滚信息。DM 数据库会收集回滚信息的使用情况，并根据统计结果对回滚信息保存周期进行调整，数据库将回滚信息保存周期设为比系统中活动的最长的查询时间稍长。

DM 支持了延迟段分配的功能，在创建表或索引时先暂时不申请存储空间，当对表中插入数据时，才尝试申请存储空间。对于已延迟段分配的表创建二级索引时，新建的索引同样也会延迟段分配。（除了全局索引、online 方式创建索引，此时二级索引总是会申请段）。可以通过设置 ini 参数 DEFERRED_SEGMENT_CREATION=1 开启延迟段分配功能，也可以在建表语句中进行指定，具体的段分配情况可以通过查看 V$SEGMENT_INFOS 获取。该功能可以有效改善创建分区表时，分区数较多导致初始为空的分区表分配过多空间的问题。

对表或索引进行延迟段分配，可能存在以下特殊场景：

1） 对已延迟段分配的表进行查询插入，即便查询空表不会插入任何数据，目标表也会申请段。 因此需要注意，在启用 DDL 相关优化如 ALTER_TABLE_OPT 参数时，DM 可能会选择使用查询插入的方式来加速 DDL 执行，此时即便目标表已延迟段分配，该 DDL 的执行会导致目标表申请段。

2） 表指定了自增列起始边界，即便建表时指定了延迟段分配也会被忽略。

3） 重建已延迟段分配的索引将导致该索引申请段。

1.2.2 日志表空间

与数据库中存储的数据不同的是，日志信息是不由用户进行维护的，而是由系统自动产生并进行自动管理，用户仅可查看并应用相关信息。因此，该表空间的存储管理方法略有不同：系统对每一个修改物理数据页的动作生成一个 REDO 记录（RREC）；多个有逻辑关联的 REDO 记录组合成一个物理事务（PTX），与数据库管理系统中事务（Transaction）概念相对应；一个或多个连续的物理事务组合成为一个 REDO 日志包（RLOG_PKG），刷盘后系统将日志包存储到日志表空间中。图 1.4 显示了这些数据结构之间的关系。

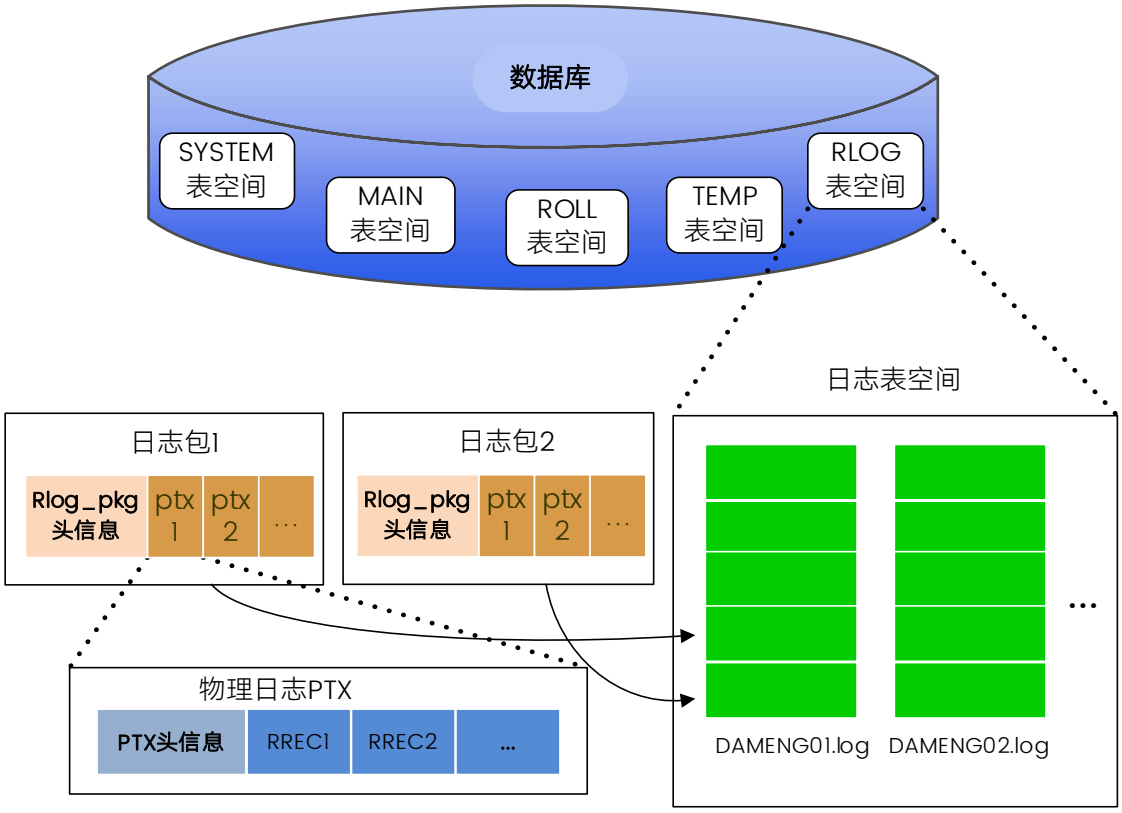


图 1.4 日志表空间、日志文件、日志包、物理事务的关系

1.2.2.1 REDO 记录

REDO 记录（REDO RECORD，简称 RREC）是日志记录过程中的最小单元，每条 REDO 记录都对应一个修改物理数据页的动作，根据记录内容的不同，又分为物理 RREC 和逻辑 RREC。

物理 RREC 记录的是数据页的变化情况，内容包括：操作类型、修改数据页地址、页内偏移、数据页上的修改内容，如果是变长类型的 Redo 记录，在 RREC 记录头之后还会有一个两字节的长度信息。

逻辑 RREC 记录的是一些数据库逻辑操作步骤，主要包括：事务启动、事务提交、事务回滚、字典封锁、事务封锁、B 树封锁、字典淘汰等。逻辑 RREC 记录是专门为数据守护增加的记录类型，用来解决备库重演 Redo 日志与用户访问备库之间的并发冲突，以及主库执行 DDL 后导致的主备数据库字典缓存不一致问题。备库解析到逻辑 RREC 记录时，根据记录内容，生成相应的事务，封锁对应的数据库对象，并从字典缓存中淘汰过期的字典对象。

1.2.2.2 物理事务

物理事务（Physical Transaction，简称 PTX）是数据库内部一系列修改物理数据页操作的集合，与数据库管理系统中事务（Transaction）概念相对应，具有原子性、有序性、无法撤销等特性。对于运行过程中产生的一系列物理事务，系统采用 LSN 进行维护。日志序列值 LSN（Log Sequence Number）是一个 BIGINT 类型数值，具有自动递增、全局唯一特性，每一个 LSN 值代表着 DM 系统内部产生的一个物理事务。

数据库完成一个事务但未提交时，该事务中所有修改物理数据页的操作对应的 RREC，组合成为一个物理事务。系统为这个新生成的 PTX 分配一个 LSN，并存储在 PTX 头信息中。此时由于 PTX 中记录的修改操作尚未刷盘，无法写入日志表空间的联机日志文件中，暂时存储在尚有空间的日志包中，等待合适时机发起日志刷盘请求，写入日志表空间。

1.2.2.3 日志包

Redo 日志包（RLOG_PKG）是 DM 数据库批量保存物理事务产生的 Redo 日志的数据单元，以物理事务 PTX 为单位保存日志，一个日志包内可连续保存一个或多个 PTX。每个日志包都有对应的序号属性，称之为包序号（PKG SEQNO）。单节点数据库中，包序号具有唯一、连续递增的特性，日志包生成时按照序号连续递增，相邻日志包的 LSN 顺序是总体递增的，该特性可用于校验归档日志的连续性。

日志包具有自描述的特性，日志包大小不固定，采用固定包头和可变包头结合的方式，包头记录日志的控制信息，包括类型、长度、包序号、LSN 信息等内容。DM 数据守护系统中，主库以 RLOG_PKG 为最小单位发送 Redo 日志到备库。

日志包生成时，系统为该日志包产生一个包序号，PTX 生成后写入到日志包中，日志包据此修改包头信息。在数据库事务提交或日志包被写满时触发日志刷盘动作，日志刷盘线程负责将日志包写入联机日志文件，即加入日志表空间中。日志包加入后，日志表空间会根据包头信息修改一系列变量，相关信息可以通过 V$RLOG 查看。

第2章 DM 物理存储结构

DM 数据库使用了磁盘上大量的物理存储结构来保存和管理用户数据。典型的物理存储结构包括：用于进行功能设置的配置文件；用于记录文件分布的控制文件；用于保存用户实际数据的数据文件、重做日志文件、归档日志文件、备份文件；用来进行问题跟踪的跟踪日志文件等，如图 2.1 所示。

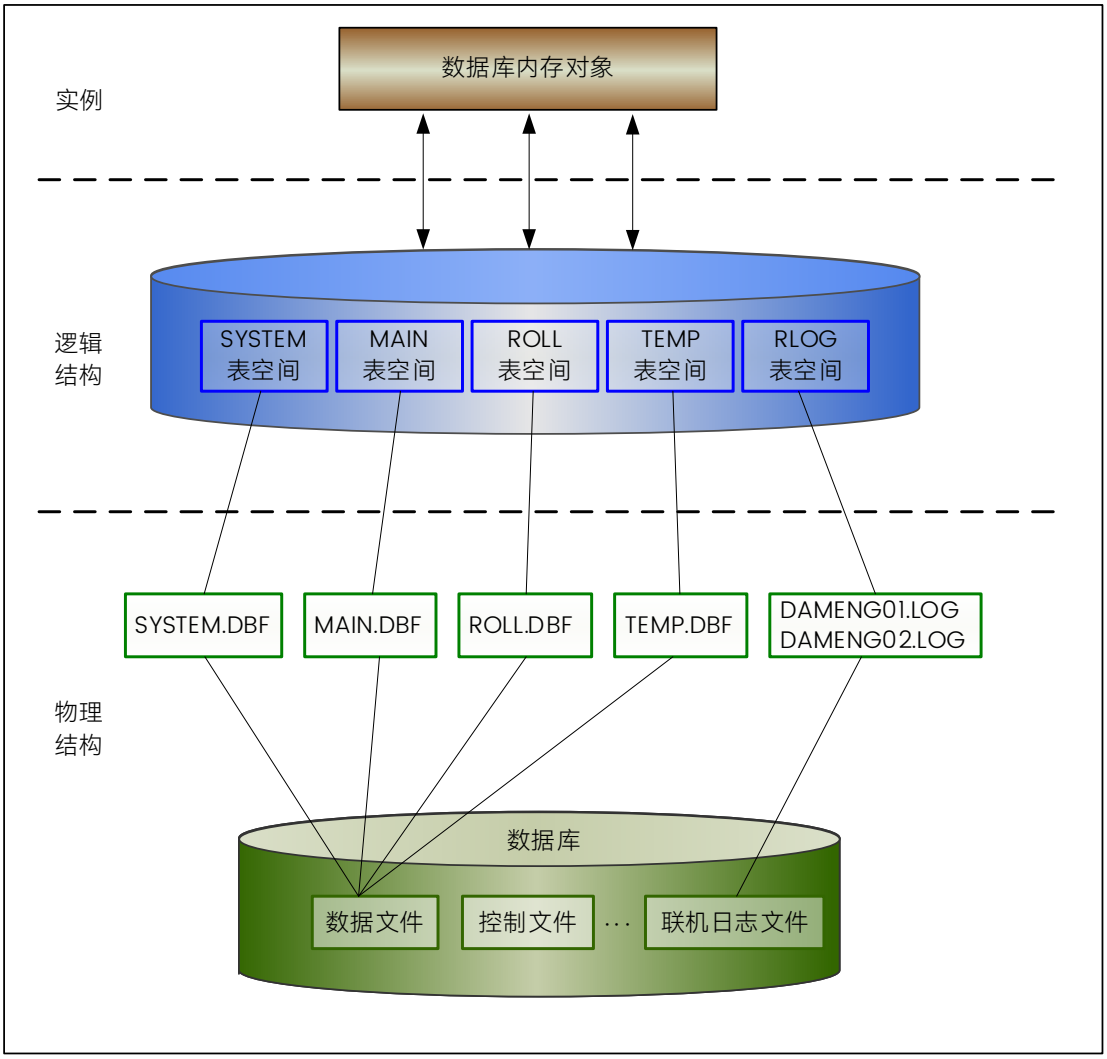


图 2.1 DM 物理存储结构示意图

2.1 配置文件

配置文件是 DM 数据库用来设置功能选项的一些文本文件的集合，分为服务器配置文件和客户端配置文件两类。

服务器配置文件以 INI 为扩展名，它们具有固定的格式，用户可以通过修改其中的某些参数取值来启用/禁用服务器特定功能项，或者针对当前系统运行环境设置更优的参数值以提升系统性能。

客户端配置文件为 dm_svc.conf，用户可以在该文件中为 JDBC 等接口以及 Disql 等工具配置连接参数，或者启用/禁用客户端特定功能项。

2.1.1 服务器配置文件

2.1.1.1 dm.ini

每创建一个 DM 数据库，就会自动生成 dm.ini 参数文件。dm.ini 是 DM 数据库启动所必须的配置文件，通过配置该文件可以设置 DM 数据库服务器的各种功能和性能选项，主要的配置模块包括：控制文件相关、实例名、内存相关、线程相关等。

参数属性分为三种：手动、静态和动态。

手动，不能被动态修改，必须手动修改 dm.ini 参数文件，然后重启才能生效。

静态，可以被动态修改，修改后重启服务器才能生效。

动态，可以被动态修改，修改后即时生效。动态参数又分为会话级和系统级两种。会话级参数被修改后，新参数值只会影响当前会话和新创建的会话，之前创建的会话不受影响；系统级参数的修改则会影响所有的会话。

其中，动态修改是指 DBA 用户可以在数据库服务器运行期间，通过调用系统过程 SP_SET_PARA_VALUE()、SP_SET_PARA_DOUBLE_VALUE() 和

SP_SET_PARA_STRING_VALUE()对参数值进行修改。当动态修改 dm.ini 参数文件时，如果遇到磁盘空间不足导致系统崩溃的情况，需要手动使用 dm.ini 文件所在目录下的 dm.ini.dmbak 文件恢复 dm.ini，并重启服务器。

当手动修改 dm.ini 参数文件且将 dm.ini 中的某参数值设置为非法值时，若设置值与参数类型不兼容，则参数实际取值为默认值；若设置值小于参数取值范围的最小值，则实际取值为最小值；若设置值大于参数取值范围的最大值，则实际取值为最大值。

当动态修改 ini 参数且设置参数为非法值时，包括设置值与参数类型不兼容、设置值小于参数取值范围的最小值、设置值大于参数取值范围的最大值三种情况，均进行报错处理。

需要注意的是，ini 参数的修改不会自动清理之前已生成的计划缓存。当 ini 参数 USE_PLN_POOL 非 0 时，DMServer 会开启计划重用，此时修改 ini 参数，数据库仍优先重用旧计划，可能导致参数变更效果无法体现。建议管理员在修改 ini 参数后，手动清理计划缓存。

2.1.1.1.1 文件路径相关

文件路径相关参数。

表2.1 dm.ini 中控制文件相关参数

参数名	缺省值	属性	说明
CTL_PATH	SYSTEM_PATH\dm.ctl	手动	控制文件路径
CTL_BAK_PATH	SYSTEM_PATH\CTL_BAK	手动	控制文件备份路径。在初始化库或没有配置该项时均指定为缺省路径。 备份文件在初始化库和每次修改控制文件后生成
CTL_BAK_NUM	10	动态，系统级	控制文件备份个数限制，取值范围 1~100，在此限制之外，会再多保留一个备份文件，在生成新的备份文件时，如果当前已存在的备份文件个数大于指定值，则自动删除创建时间最早的备份文件，小于或等于指定值的情况下，不会有删除操作
SYSTEM_PATH	数据库目录\数据库名	手动	系统库目录。其中数据库目录由安装时指定，或者 dminit 工具 PATH 参数指定
CONFIG_PATH	SYSTEM_PATH	手动	指定 DMServer 所读取的配置文件（dmmal.ini, dmarch.ini, dmtimer.ini 等）的路径。不允许指定 ASM 目录
TEMP_PATH	SYSTEM_PATH	手动	临时数据库文件路径
BAK_PATH	SYSTEM_PATH\bak	手动	备份路径
XBOX_DUMP_PATH	空串	手动	DMDPC 环境下专用，XBOX 邮件保存到临时文件的目录。 和 XBOX_DUMP_THRESHOLD 一起设置才生效
PRIKEY_BAK_PATH	SYSTEM_PATH\PRIKEY_BAK	手动	数据库服务器密钥文件备份路径。在初始化库或没有配置该项时均指定为缺省路径。

			备份文件在初始化库和每次修改密钥文件后生成

2.1.1.1.2 实例名

表2.2 dm.ini 中实例名相关参数

参数名	缺省值	属性	说明
INSTANCE_NAME	DMSERVER	静态	实例名。只能由_、$、大写字母 A 至 Z、小写字母 a 至 z、数字 0 至 9 组成，且第一个字符不能是数字。长度不超过 16 个字节

2.1.1.1.3 内存相关

表2.3 dm.ini 中内存相关参数

参数名	缺省值	属性	说明
MEMORY_POOL	500	静态	共享内存池大小，单位MB。共享内存池是由DM管理的内存。取值范围：32位平台为64~2000，64位平台为64~67108864 当建库参数AUTO_ADJ_PARA为1时，本参数实际默认值由系统根据AUTO_ADJ_MEM自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit使用手册》

MEMORY_N_POOLS	1	静态	共享内存池个数，取值范围 0~2048，0 与 1 等价，即共享池不分片。分片时，每池的初始容量为 MEMORY_POOL / MEMORY_N_POOLS，但受最小初始容量为 30M 限制，因此实际分片数可能会小于 MEMORY_N_POOLS，每个池的 TARGET 也按此方式由 MEMORY_TARGET 导出 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》
MEMORY_TARGET	32 位平台为 1000；64 位平台为 15000	动态，系统级	共享内存池在扩充到此大小以上后，空闲时收缩回此指定大小，单位MB，取值范围：32位平台为0~2000，64位平台为0~67108864，0表示不限制 当建库参数AUTO_ADJ_PARA为1时，本参数实际默认值由系统根据AUTO_ADJ_MEM自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit使用手册》
MEMORY_EXTENT_SIZE	32	静态	共享内存池每次扩充的大小，单位 MB，取值范围 1~10240
MEMORY_LEAK_CHECK	0	动态，	是否开启内存泄漏检测。0：否；1：是，此时系统对每一次内存分配都登记到动态视图 V$MEM_REGINFO 中， 并在释放时解除登记

		系统级	
MEMORY_MAGIC_CHECK	1	静态	是否开启对所有内存池的校验。0：不开启；1：开启校验，校验码基于分配出的块地址计算，在被分配空间的头部和尾部写入校验码；2：增强校验，在 1 的基础上，如果是内存池分配的，则对尾部未使用空间也计算校验码，写入未使用空间的头部
HUGEPAGE_THRESHOLD	16	动态，系统级	如果从 OS 申请内存的尺寸大于等于 HUGEPAGE_THRESHOLD * 2MB， 则尝试先申请巨页内存，若失败再申请常规内存； 0 表示不尝试使用巨页内存。取值范围 0~1024 注：本参数仅对 LINUX 操作系统有效，使用巨页内存需要对 OS 进行相关配置
MEMORY_BAK_POOL	4	静态	系统备份内存池大小，单位 MB。系统备份内存池是由 DM 管理的内存。取值范围 2~2048
HUGE_MEMORY_PERCENTAGE	50	静态	指示 HUGE_BUFFER 中可以借用作常规内存分配的空间百分比，取值范围为 0~100。
HUGE_BUFFER	80	静态	HUGE 表使用的缓冲区大小，单位 MB。取值范围 8~1048576
HUGE_BUFFER_POOLS	4	静态	HUGE BUFFER 系统分区数，每个 HUGE BUFFER 分区的大小为 HUGE_BUFFER/HUGE_BUFFER_POOLS。取值范围 1~512

BUFFER	8000	静态	NORMAL 缓冲区大小，单位 MB。推荐值：NORMAL 缓冲区大小为可用物理内存的 60% ~ 80%。取值范围 8~16777216。在 PAGE_SIZE 为 4K 的库中，BUFFER 最大值为 15728640。64 位系统的缺省值为 8000，32 位系统的缺省值为 64。 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
BUFFER_POOLS	19	静态	NORMAL 缓冲区分区数，每个分区的大小为 BUFFER/BUFFER_POOLS。取值范围 1~10000 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
BUFFER_IN_CORE	1	静态	当数据库宕机时，是否将数据缓冲区 BUFFER 的内存转储到 CORE 文件中。0：FALSE；1：TRUE。 注：要求 Linux 内核为 3.4 及以上版本
FAST_POOL_PAGES	3000	静态	快速缓冲区页数。取值范围 0~99999999。 FAST_POOL_PAGES 的值最多不能超过 BUFFER 总页数的 90%，如果超过，系统会自动调整为 BUFFER 总页数的 90%
FAST_ROLL_PAGES	1000	静态	BUFFER 中 FAST POOL 回滚页数量。取值范围 0~9999999，最多不超过 FAST_POOL_PAGES 的 75%，如果超过，

			系统会自动调整。填充规则：将回滚表空间 0 号文件的 0~FAST_ROLL_PAGE-1 号页填入 FAST POOL 中
RECYCLE	300	静态	RECYCLE 缓冲区大小，单位 MB。取值范围 8~16777216 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
RECYCLE_POOLS	19	静态	RECYCLE 缓冲区分区数。取值范围 1~512。一般情况下，每个 RECYCLE 分区的大小为 RECYCLE/RECYCLE_POOLS，但系统也会自动校正 RECYCLE 缓冲区分区数，以保证每个分区中数据页不少于 3000 页 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
MULTI_PAGE_GET_NUM	1	静态	是否开启缓冲区多页读取接口。取值范围 1~64。取值 1 表示关闭多页读接口，取值 2~64 的效果一致，表示启动多页读接口。多页读接口一定由预加载调用，其它情况不会调用。当 ENABLE_FREQROOTS 值非 0 时，为开启动态 FAST POOL 填充模式，此时系统会强制调整 MULTI_PAGE_GET_NUM=1，并关闭预加载功能。 注：当数据库加密时不支持多页读取，此时 DM.INI 中此参数值无效
PRELOAD_SCAN_NUM	0	动	数据页首次预加载时连续扫描的页数，取值范围 0~64。取值为 0 时关闭数据页预加载功能

		态，系统级	注：PRELOAD_SCAN_NUM 实际有效范围为 0 ~ EXTENT_SIZE – 2，当大于 EXTENT_SIZE – 2 时将会间接失效预加载
PRELOAD_EXTENT_NUM	0	动态，系统级	数据页首次预加载的簇数量，取值范围 0~64。取值为 0 时关闭数据页预加载功能
SORT_FLAG	1	动态，会话级	排序机制。 0：原排序机制，普通排序方式，按照排序缓冲区 SORT_BUF_SIZE 大小获取数据，并进行排序； 1：新排序机制，大内存排序方式，将大内存排序缓冲区 SORT_BUF_GLOBAL_SIZE 按照内存片大小 SORT_BLK_SIZE 进行分片，并对每个内存片进行排序，再对整个大内存排序缓冲区的数据进行排序； 2：采用 SORT_FLAG 为 0 的排序方式的同时，增加递增列，当排序列存在相同值时按递增列进行排序； 3：采用 SORT_FLAG 为 1 的排序方式的同时，增加递增列，当排序列存在相同值时按递增列进行排序 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据

			实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
SORT_BUF_SIZE	20	动态，会话级	原排序机制下，排序缓存区最大值，单位 MB。取值范围 1~2048 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
SORT_BUF_GLOBAL_SIZE	1000	动态，系统级	新排序机制下，排序全局内存使用上限，单位 MB。取值范围 10~4294967294。该值须大于 SORT_BUF_SIZE 的值，否则会在重启时被重置为 SORT_BUF_SIZE 两倍和 100 之间的最大值 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
SORT_BLK_SIZE	1	动态，会话级	新排序机制下对数据进行排序的排序缓冲区分片大小，单位 MB。取值范围 1~512。 SORT_BLK_SIZE 一般不需要设置太大，该内存片大小能容纳 1 万至 5 万行待排序的数据即可。该值须小于等于 SORT_BUF_SIZE，否则会在重启时被重置为 SORT_BUF_SIZE。为了保证归并排序的效率，对于大内存排序缓冲区的总的分片个数系统上限为 10000 个，因此，当 SORT_BUF_GLOBAL_SIZE 足够大的时候，为了保证大内存排序缓冲区能容纳足够多的数据，需要适当调大 SORT_BLK_SIZE。例如，当 SORT_BLK_SIZE 为 1 时，SORT_BUF_GLOBAL_SIZE

			的最大可用值为 10000。如果 SORT_BUF_GLOBAL_SIZE 需要 20000M 可用空间排序时，需要设置 SORT_BLK_SIZE 最小为 2 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
SORT_BUF_SINGLE_SIZE	10000	动态，会话级	控制单个排序操作符内存使用上限，大内存排序场景(参数 SORT_FLAG 取值包含 1 时)有效。在允许的取值范围内，内部还将检查其取值是否合理，SORT_BUF_SINGLE_SIZE 的合理取值应该大于或等于 SORT_BLK_SIZE 且小于或等于 SORT_BUF_GLOBAL_SIZE。取值范围 1~4294967294，单位 MB
HAGR_HASH_SIZE	100000	动态，会话级	HAGR 操作时，建立 HASH 表的桶个数。取值范围 10000~100000000
MAL_LEAK_CHECK	0	动态，	是否打开 MAL 内存泄露检查。0：关闭；1：打开。MAL_LEAK_CHECK 为 1 时，可查询 V$MAL_USING_LETTERS 检查 MAL 内存泄露

		系统级	
HJ_BUF_GLOBAL_SIZE	5000	动态，系统级	HASH 连接操作符的数据总缓存大小，单位 MB。取值范围 10~500000。该值须大于 HJ_BUF_SIZE 的值，否则会在尝试使用系统函数修改为非法值或在 DM.INI 中指定为非法值后，被重置为 HJ_BUF_SIZE 两倍和 500 之间的最大值当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
HJ_BUF_SIZE	500	动态，会话级	单个 HASH 连接操作符的数据总缓存大小，单位 MB。取值范围 2~100000。该值须小于 HJ_BUF_GLOBAL_SIZE 的值，否则会在尝试使用系统函数修改为非法值时，被重置为 HJ_BUF_GLOBAL_SIZE/2 与 100000 之间的最小值当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
HJ_BLK_SIZE	2	动态，会话级	HASH 连接操作符每次分配缓存（BLK）大小，单位 MB。取值范围 1~50。该值须小于等于 HJ_BUF_SIZE 的值，否则会在尝试使用系统函数修改为非法值或在 DM.INI 中指定为非法值后，被重置为 HJ_BUF_SIZE 与 50 之间的最小值

HAGR_BUF_GLOBAL_SIZE	5000	动态，系统级	分组、嵌套连接、集合运算、分析函数运算、统计信息、DISTINCT、临时表、TOP、ESEND 操作符的数据总缓存大小，单位 MB。取值范围 10~1000000。该值须大于等于 HAGR_BUF_SIZE，否则会在尝试使用系统函数修改为非法值或在 DM.INI 中指定为非法值后，被重置为两倍 HAGR_BUF_SIZE 和 500 之间的最大值 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
HAGR_BUF_SIZE	500	动态，会话级	单个分组、嵌套连接、集合运算、分析函数运算、统计信息、DISTINCT、临时表、TOP、ESEND 操作符的数据总缓存大小，单位 MB。取值范围 2~500000。该值须小于 HAGR_BUF_GLOBAL_SIZE 的值，否则会在尝试使用系统函数修改为非法值时，被重置为 HAGR_BUF_GLOBAL_SIZE/2 与 500000 之间的最小值 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
HAGR_BLK_SIZE	2	动态，会话级	分组、嵌套连接、集合运算、分析函数运算、统计信息、DISTINCT、临时表、TOP、ESEND 操作符每次分配缓存（BLK）大小，单位 MB。取值范围 1~50。该值须小于等于 HAGR_BUF_SIZE 的值，否则会在尝试使用系统函数修改为非法值或在 DM.INI 中指定为非法值后，被重置为 HAGR_BUF_SIZE 与 50 之间的最小值

MTAB_MEM_SIZE	8	动态，系统级	MTAB 缓存 BDTA 占用内存空间的大小，单位 KB，取值范围 1~1048576
FTAB_MEM_SIZE	0	静态	指定使用 MTAB 机制或 FTAB 机制。取值范围为 0~64 * 1024，单位 KB。0 表示使用 MTAB；大于 0 表示使用 FTAB，此时参数值表示 FTAB 缓存 BDTA 占用内存空间的大小。当取值小于 32 时，按照 32 处理
MMT_SIZE	0	动态，会话级	是否使用 MMT。0：不启用；其他有效值：启用，并确定单个映射文件大小，必须小于 MMT_GLOBAL_SIZE。取值范围 0~64，单位 MB
MMT_GLOBAL_SIZE	4000	动态，系统级	系统总共使用 MMT 的文件总大小，单位 MB，取值范围 10~1000000，仅在 MMT_SIZE 大于 0 时有效

MMT_FLAG	1	动态，会话级	MMT 存储数据方式。1：按页存储；2：BDTA 存储。仅在 MMT_SIZE 大于 0 时有效
DICT_BUF_SIZE	50	静态	字典缓冲区大小，单位 MB，取值范围 1~67108864 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
HFS_CACHE_SIZE	160	动态，系统级	HUGE 表 I/U/D 时 HDTA_BUFFER 缓存池大小，单位 MB。取值范围 160~2000
VM_POOL_SIZE	64	静态	系统执行时虚拟机内存池大小，单位 KB，在执行过程中用到的内存大部分是从这里申请的，它的空间是从操作系统中直接申请的，取值范围 32~1024*1024
VM_POOL_TARGET	16384	静态	虚拟机内存池能扩充到的最大大小，单位 KB，取值范围 0~10*1024*1024，0 表示不限制

			当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
SESS_POOL_SIZE	64	动态，系统级	会话缓冲区大小，单位 KB，取值范围 16~1024*1024。若所申请的内存超过实际能申请的大小，则系统将按 16KB 大小重新申请
SESS_POOL_TARGET	16384	动态，系统级	会话缓冲区能扩充到的最大大小，单位 KB，取值范围 0~10*1024*1024，0 表示不限制 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
RT_HEAP_TARGET	8192	动态，系统级	会话上用于动态对象存储的 RT_HEAP 最大可扩展到的大小，单位 KB，取值范围 8192~10*1024*1024

VM_MEM_HEAP	0	动态，系统级	VM 是否使用 HEAP 分配内存。0：MEMORY POOL 模式；1：HEAP 模式；2：MEMORY POOL 和 HEAP 混合模式
RFIL_RECV_BUF_SIZE	16	静态	控制服务器启动时，进行 REDO 操作过程中，REDO 日志文件恢复时 BUFFER 的大小，单位 MB，取值范围 16~4000
HAGR_DISTINCT_HASH_TABLE_SIZE	10000	动态，会话级	分组 DISTINCT 操作中 HASH 表的大小（桶数）。取值范围为 10000~100000000
CNNTB_HASH_TABLE_SIZE	100	动态，会话级	指定 CNNTB 操作符中创建 HASH 表的大小。取值范围 100~100000000
GLOBAL_RTREE_BUF_SIZE	100	动	R 树全局缓冲区大小，取值范围 0~100000，单位 MB

		态，系统级	
SINGLE_RTREE_BUF_SIZE	10	动态，会话级	单个 R 树的缓冲区大小，单位 MB，必须小于 GLOBAL_RTREE_BUF_SIZE
SORT_OPT_SIZE	0	静态	整型/浮点型数据排序优化辅助空间大小，单位 MB，取值范围 0~1024。取值 0M 时不使用优化；取值 1M 时对应待排序数组中元素的最大最小差值为 262144，超过则不能使用优化
TSORT_OPT	1	动态，系统级	排序优化选项。0：排序操作一律使用估算的行数分配内存（至少 2M）；1：打开优化，排序操作结果行数较少时，使用实际的记录行数分配内存
BIND_PLN_PERCENT	30	静态	固定计划占据计划缓存空间的最大比例，取值范围 0~100

FBACK_HASH_SIZE	10000	动态，系统级	闪回系统中 HASH 表的桶个数。取值范围 10000~1000000
XBOX_MEMORY_TARGET	1024	动态，系统级	DMDPC 环境下专用，XBOX 系统内存在扩充到此大小以上后，空闲时收缩回此指定大小，单位 MB。取值范围 1024~262144
POOL_CLEAR_FLAG	0	静态	内存池释放时是否全部清理，不全部清理时保留 TARGET 的内存。1：是；0：否
REGEX_MEM_LIMIT	20	静态	限制对于正则表达式第三方库单个池允许使用的内存上限，单位 MB，取值范围 0~8102。0 表示不限制，当设置值小于 10 时无效，系统会强制设置为 10
REGEX_MEM_GROUP	8	静态	正则表达式第三方库内存池的数量。取值范围 1-256
IDLE_MEM_THRESHOLD	50	动态，	可用物理内存的报警阈值，单位 MB，取值范围 10~6000

		系统级	
CACHE_POOL_SIZE	100	静态	SQL 缓冲池大小，单位 MB。取值范围：32 位平台下为 1~2048；64 位平台下为 1~67108864 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
BUFFER_MODE	0	静态	BUFFER 管理方法。0：采用 LRU 算法管理 BUFFER，1：使用时钟置换算法管理 BUFFER。
FAST_HOTPAGES	5000	动态，系统级	每次快池填充的最大页数。取值范围 100~1000000
FAST_FILL_INTERVAL	15	动态，系统级	快池填充的时间间隔，单位秒。取值范围 1~864000

FAST_ACCESS_CNT	100	动态，系统级	表示普通池中页至少被命中这么多次，才有资格进入快池。取值范围 10~10000000
ENABLE_SQLVM_CACHE	0	动态，系统级	表示是否重用 SQL 执行节点树。0：不重用； 1：重用；限制： 仅对绑定参数的单条 SQL 有效， 适用于一次准备多次执行的场景。 参数开启后仅首次执行需要创建 SQL 执行树，后续绑定参数执行时重用 SQL 执行树，以降低执行代价
ENABLE_FREQROOTS	0	静态	指定FAST_POOL的管理方式。 0：经典模式，系统启动时装入常用的描述页、索引的根、索引的控制页、指定常驻内存的表以及索引第二层的节点，之后不再变化； 1：动态模式A，系统启动时装入常用的描述页、索引的根、指定常驻内存的表， 后续按负载情况动态装入频繁访问的页； 2：动态模式B， 预装入的页同经典模式，后续按负载情况动态装入频繁访问的页。

			4：不做预装载，后续按负载情况动态装入频繁访问的页。 对于ROLL表空间，最多分配0号文件的前FAST_ROLL_PAGES。 注：DMDSC 仅支持 ENABLE_FREQROOTS 为 0，系统会进行强制设置
REGEXP_COMPILE_CACHE	0	动态，会话级	正则表达式编译结果的缓存数量，取值范围0~64，0表示不缓存

2.1.1.1.4 线程相关

表2.4 dm.ini 中线程相关参数

参数名	缺省值	属性	说明
WORKER_THREADS	16	静态	并发处理会话连接的线程数量；当打开 SESS 线程池（即 STHD_FLAG=2 或 3 时）时，该值为初始化的 SESS 线程池的组数量。取值范围 1~64

			当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_CPUS 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
TASK_THREADS	16	静态	任务线程个数。非 DMDSC 环境取值范围 1~1000；DMDSC 环境取值范围 16~1000 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_CPUS 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
FAST_RW_LOCK	1	静态	快速读写锁标记。0：不启用；1：表示只有 FAST POOL 启用，2：表示所有 POOL 都启用
SPIN_TIME	400 0	静态	线程在不能进入临界区时，自旋的次数。取值范围 0~400000
WORK_THRD_STACK_SIZE	8192	静态	工作线程堆栈大小，单位 KB。取值范围 1024~32768
WORK_THRD_RESERVE_SIZE	512	静态	工作线程预留栈帧大小，单位 KB。取值范围 200~1024。 WORK_THRD_RESERVE_SIZE 受 WORK_THRD_STACK_SIZE 限制，最多预留工作线程堆栈大小一半
WORKER_CPU_PERCENT	0	静态	工作线程占 CPU 的比重，仅非 WINDOWS 下有效。取值范围 0~100。0 表示不限制，相当于 100
STHD_FLAG	0	静态	是否启用 ESESS/SESS 线程池。取值范围 0~3。0：表示关闭；1：表示打开 ESESS 线程池；2：表示打开 SESS 线程池；3：表示同时打开 ESESS 和 SESS 线程池

STHD_THREAD_NUM	8	静态	每个 SESS/ESESS 线程池初始化的线程数量。取值范围 1~10000
STHD_GRP_NUM	8	静态	DMDPC 环境下专用，初始化的 ESESS 线程池的组数量。取值范围 1~64
THRDS_POOL_INIT	50	静态	底层线程池初始化的线程数量。取值范围 1~65535
THRDS_POOL_MAX	0	静态	底层线程池最大的线程数量。取值范围 0~65535。0 表示不限制最大线程数量
PTHD_THRD_POOLS	16	静态	PTHD 线程池的组数量。取值范围 1~1024
MAX_SEC_ASYNC_THREADS	0	动态，系统级	当参数 INDEX_FINS_FLAG 取值包含 1 时，二级索引可能由异步线程维护，该参数用于限制异步处理二级索引使用的线程数量。取值范围 0~65535，0 表示不限制最大线程数量。 需要注意的是，执行中实际可使用的线程数量还受到线程池参数 THRDS_POOL_MAX 的限制
ASYNC_THRD_MAX_PERCENT	80	动态，系统级	全局异步线程数占所有工作线程数的最大百分比，超过时则不再抛出异步任务，报错处理

2.1.1.1.5 查询相关

表2.5 dm.ini 中查询相关参数

参数名	缺省值	属性	说明
USE_PLN_POOL	1	静态	是否重用执行计划。0：禁止执行计划的重用；1：启用执行计划的重用功能 ；2：对不包含显式参数的语句进

			行常量参数化优化；
DYN_SQL_CAN_CACHE	1	动态， 系统级	是否缓存动态语句的执行计划。0：不缓存；1：当 USE_PLN_POOL 不为 0 时，缓存动态语句的执行计划
RS_CAN_CACHE	0	静态	结果集缓存配置。0：禁止重用结果集；1：强制模式，此时默认缓存所有结果集，但可通过 RS_CACHE_TABLES 参数和语句 HINT 进行手动设置；2：手动模式，此时默认不缓存结果集，但可通过语句 HINT 对必要的结果集进行缓存 注：DPC 环境下不支持结果集缓存，故 RS_CAN_CACHE 等结果集缓存相关参数无法在 DPC 环境下开启
RS_CACHE_TABLES	空串	手动	指定可以缓存结果集的基表的清单，当 RS_CAN_CACHE=1 时，只有查询涉及的所有基表全部在此参数指定 范围内，该查询才会缓存结果集。当参数值为空串时，此参数失效。
RS_CACHE_MIN_TIME	0	动态， 系统级	结果集缓存的语句执行时间下限，只有实际执行时间不少于指定时间值的查询，其结果集才会被缓存，仅在 RS_CAN_CACHE=1 时有效。缺省值 0，表示不限制；取值范围 0~4294967294，单位毫秒
RS_BDTA_FLAG	0	静态	是否以 BDTA 形式返回结果集。0：以行为单位返回结果集；2：以 BDTA 形式返回结果集
RS_BDTA_BUF_SIZE	32	静态	配置消息长度，单位 KB；取值范围 8~32768。仅当 RS_BDTA_FLAG=2 时有效
RS_TUPLE_NUM_LIMIT	2000	静态	控制结果集能在内存中缓存的最大行数，取值范围 1~65534。语句块执行时要先填充结果集，填充行数超过该

			值时会刷盘保存结果集
RESULT_SET_LIMIT	10000	动态，会话级	一次请求可以生成的结果集最大个数。取值范围 100~65000
RESULT_SET_FOR_QUERY	0	动态，会话级	是否生成非查询结果集。0：生成；1：不生成
SESSION_RESULT_SET_LIMIT	10000	动态，系统级	会话上结果集个数上限，取值范围 100~65000
BUILD_FORWARD_RS	0	静态	仅向前游标是否生成结果集。0：不生成；1：生成
MAX_OPT_N_TABLES	6	动态，会话级	优化器在处理连接时，一次能优化的最大表连接个数。取值范围 3~8
MAX_N_GRP_PUSH_DOWN	5	动态，会话级	用来控制 GROUP BY 下放优化的最大叶子表节点个数。在 PLACE_GROUP_BY_FLAG 为 2 时生效。取值范围 1~1024
CNNTB_MAX_LEVEL	20000	动态，会话级	层次查询的最大支持层次。取值范围 1~100000

CTE_MAXRECURSION	100	动态， 会话级	递归 WITH CTE 迭代层次。会话级参数。取值范围 1~4294967294
CTE_OPT_FLAG	1	动态， 会话级	递归 WITH CTE 优化选项，表示相关子查询是否转换为 WITH FUCNTION 优化。 取值 0 或 1
BATCH_PARAM_OPT	0	静态	是否启用批量参数优化，默认为不启用。 0：不启用； 1：模式 1，优化幅度大但有诸多限制，如不返回操作影响的行数（批处理执行结果统一返回 0）、不支持批量容错等； 2：模式 2，优化幅度小但没有限制； 3：尽量使用模式 1，如果无法触发模式 1 的优化，则使用模式 2； 4：插入使用模式 1 的优化，更新和删除使用模式 2 的优化，绝大多数情况下返回影响行数，部分特殊情况不返回，例如：使用 IGNORE_ROW_ON_DUPKEY_INDEX 进行插入
CLT_CONST_TO_PARAM	0	静态	是否进行语句的常量参数化优化，0：不进行；1：进行
LIKE_OPT_FLAG	127	动态，	LIKE 查询的优化开关。

		会话级	0：不优化； 1：对于 LIKE 表达式首尾存在通配符的情况，优化为 POSITION()函数；对于 LIKE 表达式首部存在通配符，并且条件列存在 REVERSE()函数索引时，优化为 REVERSE()函数； 2：对于 COL1 LIKE COL2 || '%'的情况，如果 COL1 上没有索引，则优化为 POSITION()函数；如果 COL1 上有索引，则优化为 BETWEEN..AND 条件； 4：对于 COL1 LIKE 'A'||'B%'的情况，优化为 COL1 LIKE 'AB%'； 8：对于可计算的 LIKE 表达式，优化为常量； 16：对于控制函数索引列的 LIKE 表达式，优化为 BETWEEN..AND 条件； 32：将 LIKE ESCAPE 表达式转换为 BETWEEN..AND 条件，尽可能使用索引； 64：在取值为 2 的基础上，对于 COL1 LIKE COL2 || '%'的情况，如果 COL1 上有索引，则优化为 BETWEEN..AND 条件的同时保留 LIKE 表达式； 256：对于条件中 COL1 LIKE 'XXX'||COL2 的情况（其中 COL1 和 COL2 为不同表的列），扩展为 COL1 LIKE 'XXX' || COL2 AND COL1 LIKE 'XXX'||'%' 支持使用上述有效值的组合值，如 31 表示同时进行 1、2、4、8 和 16 的优化

FILTER_PUSH_DOWN	2	动态，会话级	对单表条件是否下放的不同处理方式。 0：表示条件不下放； 1：表示在新优化器下，对单表过滤条件进行下放处理； 2：表示在新优化器下对外连接、半连接进行下放条件优化处理； 4：表示语义分析阶段考虑单表过滤条件的选择率，超过 0.5 则不下放，由后面进行代价计算选择是否下放，参数值 4 仅在参数取值包含 2 时有效，即将参数值设为 6 时有效； 8：表示尝试将包含非相关子查询的布尔表达式进行下放； 16：表示 WHERE 条件尽可能下放，即便 WHERE 之后还有 ROWNUM、ORDER 等操作； 32：将耗时的过滤条件放在 JOIN 连接之后再计算。 支持使用上述有效值的组合值，如 6 表示同时进行 2 和 4 的优化
USE_MCLCT	2	动态，会话级	MPP/LPQ 下，是否替换 MGAT/LGAT 通讯操作符为 MCLCT/LCLCT。0：不替换；1：MPP 下将操作符 MGAT 替换为 MCLCT；2：MPP 下将操作符 MGAT 替换为 MCLCT，或 LPQ 下将操作符 LGAT 替换为 LCLCT
PHF_NTTS_OPT	1	动态，会话级	MPP/LPQ 下，是否进行 NTTS 计划的优化，打开时可能减少计划中的 NTTS 操作符。0：不支持；1：支持

MPP_MOTION_SYNC	200	动态，会话级	通讯操作符同步时认定的邮件堆积数，堆积超过该值则要进行同步检查。取值范围 0~10000，0 表示不进行同步检查
UPD_DEL_OPT	2	动态，会话级	删除更新计划优化方式，取值范围 0、1、2 单节点计划，0：不优化 1，2:都可优化 NTTS MPP 计划，0：不优化； 1：优化删除更新计划，不优化 NTTS；2：优化删除更新计划，同时优化 NTTS
ENABLE_DIST_IN_SUBQUERY_OPT	0	动态，会话级	IN 子查询的优化方式，目前所有优化只支持单列 IN SUBQUERY。 0：不优化；1：对 IN 子查询进行去掉子查询的优化；2：IN 子查询中的等值连接转换为多个 IN；4：IN 子查询中移除与外表相同的内表；8：IN 相关子查询中，将 WHERE 条件的 OR 合并成一个整体（CASE WHEN 形式），期望后面能进行 SEMI 的规则优化 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
MAX_OPT_N_OR_BEXPS	7	动态，会话级	能参与优化的最大 OR 分支个数，超过时布尔表达式仅用于过滤使用。取值范围 7~64
USE_HAGR_FLAG	0	动态，会话级	当带有 DISTINCT 的集函数不能使用 SAGR 操作符时，是否使用 HAGR，0：不使用；1：使用

DTABLE_PULLUP_FLAG	1	动态， 会话级	是否在语法分析阶段对派生表进行上拉优化处理，0：不优化；1：优化
VIEW_PULLUP_FLAG	34	动态， 会话级	是否对视图进行上拉优化，把视图转换为其原始定义，消除视图。 0：不进行视图上拉优化； 1：对不包含别名和同名列的视图进行上拉优化； 2：对包含别名和同名列的视图也进行上拉优化； 4：强制允许带变量的查询语句进行视图上拉优化，有可能造成结果集错误； 8：不对 LEFT JOIN 的右孩子\RIGHT JOIN 的左孩子\FULL JOIN 左右孩子进行上拉； 16：不对 LEFT JOIN 的左孩子进行上拉； 32：视图上拉后，从视图符号表删除不被引用的列 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
GROUP_OPT_FLAG	8252	动态， 会话级	分组项优化参数开关。GROUP_OPT_FLAG=4 须和 INI 参数 PARALLEL_POLICY（非 0）搭配使用。 0：不优化； 1：非 MYSQL 兼容模式下（即 COMPATIBLE_MODE 不等于 4），支持查询项不是 GROUP BY 表达式；

			2：外层分组项下放到内层派生表中提前分组优化； 4：表示对于多级分区，并行下允许尝试不生成多个 AGR 操作符； 8：进行哈希分组时，依赖分组项列中的核心项分组列来分组； 16：位图索引覆盖简单分组查询中的所有列时，允许使用位图索引对查询进行优化； 32：进行哈希分组时，对 TOP 查询进行优化，提前返回指定数量的分组； 64：DMDPC 下，对无分组项且含有 DISTINCT 集函数的查询，允许对集函数参数先进行一次分发处理； 128：尽量使用 SAGR+SORT 计划实现 GROUP； 256：使用新的计算方法计算数据 DISTINCT 值，新计算方法：(i * j)/max(((i - j) * j/i),1)，假设其中 i > j，i、j 为每列 DISTINCT 个数； 512：SAGR 中的 COUNT(DISTINCT)尽量使用优化模式计算； 1024：使用 COST_CPU_CALC_HAGR、COST_CPU_CALC_AAGR 代替 COST_CPU_CALC_ORDER 计算操作符#HAGR、#AAGR 的代价； 2048：LIST 分区只有一个值 HAGR 变 AAGR，只在 DPC 环境有效； 4096：在进行分组时根据 BLANK_PAD_MODE 和 SPACE_COMPARE_MODE 判断是否忽略后缀空格；

			8192：对一级 RANGE 分区的分区首列没有分组时求 MIN/MAX，优化分区裁剪，目前优化暂不支持并行； 16384：对分区表使用 FAGR2 求 COUNT(*)时，不加载字典对象，使用读取系统表直接加载所有叶子表需要的信息优化方式，提升性能； 32768：有分组项和集函数有非分区列的 DISTINCT 也支持两层分组，只在 DPC 环境有效。 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
FROM_OPT_FLAG	0	动态，会话级	控制一些涉及 FROM 项的优化 0：不优化；1：尝试将 FROM 项替换为单个 DUAL 表；2：允许 FROM 项存在同名对象；支持使用上述有效值的组合值，3 表示同时进行 1、2 的优化
HAGR_PARALLEL_OPT_FLAG	4	动态，会话级	MPP、并行下对 GROUP BY、分析函数等的优化开关。 0：不优化； 1：无 DISTINCT 时 HAGR 按照分组列分发； 2：有 DISTINCT 时，HAGR 按照分组列分发； 4：去除多余的通讯操作符； 8：同 4，仅限于外连接操作； 16：分析函数按照 PARTITION BY 列分发；

			32：MPP+LPQ 下，AAGR 的优化处理； 64：始终进行 MPP DISTRIBUTE 分发（取值为 1 时仅在组数大于或等于 1000 时分发）； 128：有 UNION ALL 且存在 MPP DISTRIBUTE 和 MPP GATHER 时，各个站点/线程先处理部分数据后再汇总，此优化必须同时包含取值 1 才有效； 256：始终进行 HAGR->MPP DISTRIBUTE->HAGR 的分发操作； 512：当查询存在集函数时，则将数据汇总到主站点/主线程后再执行 HAGR 计算。其中，带 KEEP 子句的集函数或 FIRST_VALUE 集函数两种情况，不论 HAGR_PARALLEL_OPT_FLAG 的值是否为 512，执行计划都会采用 MPP COLLECT -> HAGR2 这种方式。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
HAGR_DISTINCT_OPT_FLAG	2	动态， 会话级	操作符 HAGR、DISTINCT 的优化开关。此参数仅在参数 USE_HAGR_FLAG = 1 时才有效。 0：不优化； 1：仅 DMMPP 环境下使用。如果分组列是分布列，则进行生成两个 HAGR，顶层 HAGR 去掉 DISTINCT 信息的优化； 2：如果分组列都是字符串类型且总长度小于等于 2，则 HAGR 仅依靠分组列的 HASH 值进行分组。

			支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
REFED_EXISTS_OPT_FLAG	1	动态，会话级	是否把相关 EXISTS 优化为非相关 IN 查询。0：否；1：是；2：特殊场景下，通过将相关表+DISTINCT 构造为派生表添加到子查询中，将相关 EXISTS 转换为非相关 IN；4：特殊场景下，将符合条件的相关 EXISTS 子句拉至上层进行处理，避免不必要笛卡尔积的产生 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
REFED_OPS_SUBQUERY_OPT_FLAG	1	动态，会话级	是否将 OP ALL/SOME/ANY 相关子查询进行转换处理。0：否；1：转换为 EXISTS 相关子查询；2：将包含 PARTITION JOIN 连接的相关子查询转换为 EXISTS 相关子查询（已废弃，不进行转换）；4：转换为包含分析函数的派生表；8：将连接条件中的相关查询表达式转化为派生表的输出列（存在限制：查询项中不能包含*；相关子查询只能引用一张表；其他连接条件不能再包含含有引用表的连接条件等） 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化，当组合值中包含 1 且不包含 4 时，优先按照值为 1 进行转换处理，当组合值中包含 4 时优先按照值为 4 进行转换处理
PARTIAL_JOIN_EVALUATION_FLAG	1	动态，会话级	是否对外连接中的冗余关联进行移除优化；存在去除重复值操作时，是否对下层的连接操作进行转换优化。0：不优化；1：优化
USE_FK_REMOVE_TABLES_FLAG	1	动态，	是否利用外键约束消除冗余表，支持多列主键。0：不启用；1：启用

		会话级	
USE_FJ_REMOVE_TABLE_FLAG	1	动态， 会话级	是否启用使用过滤表消除表及自连接消除优化。0：不启用；1：启用；2：等值条件的左右两边是不同的表时才进行优化
SLCT_ERR_PROCESS_FLAG	0	动态， 会话级	控制如何处理过滤时产生的错误。0：正常处理错误，返回错误码；1：忽略错误，视为数据不匹配
MPP_HASH_LR_RATE	10	动态， 会话级	MPP 下，对 HASH JOIN 节点，可以根据左右儿子行数代价的比值，调整 HASH_JOIN 的左右儿子的通讯操作符添加，从而影响计划。如果行数比值超过此值，则小数据量的一方全部收集到主 EP 来做。取值范围 1~4294967294
LPQ_HASH_LR_RATE	30	动态， 会话级	LPQ 下，对 HASH JOIN 节点，可以根据左右儿子行数代价的比值，调整 HASH_JOIN 的左右儿子的通讯操作符添加，从而影响计划。 如果行数比值超过此值，则小数据量的一方全部收集到主 EP 来做。取值范围 1~4294967294
USE_HTAB	1	动态， 会话级	表示计划中是否能使用 HTAB。0：否；1：是
SEL_ITEM_HTAB_FLAG	0	动态， 会话级	当 USE_HTAB=1 时才有效。 当查询项中有相关子查询时，控制是否做 HTAB 优化及优化的方式。

			0：不生成 HTAB； 1：查询项之后生成 HTAB，需包含单表过滤条件；2：查询项之后生成 HTAB，无需单表过滤条件；4：WHERE 条件之前生成 HTAB（需同时设置参数为 8）；8：包括 GROUP 条件在内，整体生成 HTAB。 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化。
OR_CVT_HTAB_FLAG	1	动态， 会话级	当 USE_HTAB=1 时才有效。 当查询条件 OR 中含有公共因子时，是否允许使用 HTAB 来进行优化。0：不使用；1：使用；2：增强 OR 表达式转换为 HTAB 条件检查，当存在嵌套连接时，不生成 HTAB，以避免缓存过多数据影响性能
CASE_WHEN_CVT_IFUN	9	动态， 会话级	对 CASE WHEN 查询表达式的优化处理。 0：不优化； 1：将 CASE WHEN 查询表达式转换为 IFOPERATOR 函数； 2：将 CASE WHEN 查询表达式转换为 IFOPERATOR 函数，且有限制地进行表达式重用； 4：CASE WHEN 查询表达式在运算符中转换为 OR 进行处理； 8：对于 CASE WHEN 查询表达式不允许 THEN…ELSE 表达式重用； 16：将参数个数少于 30 的 COALESCE 函数转为 CASE WHEN 表达式，提升执行性能；

			32：CASE WHEN 的条件表达式中有恒真或恒假的条件时进行简化； 64：将 CASE VALUE1 WHEN VALUE2 转为 CASE WHEN VALUE1=VALUE2，可配合取值 1 时转为函数进行批量处理，提升性能； 128：允许多分支 CASE WHEN 转为嵌套 IFOPERATOR 函数，需要在 1 开启的基础上使用； 256：允许包含除法运算的 CASE WHEN 转为 IFOPERATOR 函数，但可能会导致报错(由于变函数后参数先计算)，需要在 1 开启的基础上使用。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
OR_NBEXP_CVT_CASE_WHEN_FLAG	0	动态，会话级	是否将OR 转化为CASE WHEN THEN ELSE END语句。0：不转换；1：转换
NONCONST_OR_CVT_IN_LST_FLAG	0	动态，会话级	是否开启 IN LIST 优化：将不含有常量的 OR 表达式转换成 IN LIST。0 表示不开启，1 表示开启
OUTER_CVT_INNER_PULL_UP_COND_FLAG	11	动态，会话级	控制外连接转化内连接的优化，在任意取值下，包含层次查询时禁止外连接转内连接优化。 0：不优化 1：当外连接转化为内连接时，打开连接条件；

			2：转化时探测 OR 布尔表达式是否能优化。即探测 OR 的每个分支中是否隐式包含 IS NOT NULL 的条件，若是则也转为内连接； 4：当过滤条件中包含函数时，禁止外连接转换为内连接； 8：当外连接包含层次查询时，禁止转换为内连接；（已废弃，此种场景下默认禁止转换为内连接，不再通过参数控制） 16：开启旋转功能。即指 LEFT 与 INNER JOIN（哈希内连接、归并内连接、嵌套内连接和索引内连接）操作符同时存在时，让 INNER JOIN 操作符先做的优化。 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
OPT_OR_FOR_HUGE_TABLE_FLAG	0	动态，会话级	是否使用 HFSEK 优化 HUGE 表中列的 OR 过滤条件。0：不使用；1：使用
ORDER_BY_NULLS_FLAG	0	动态，会话级	控制排序时 NULL 值返回的位置，取值 0、1、2、3。0 表示 NULL 值始终在最前面返回；1 表示 ASC 升序排序时 NULL 值在最后返回，DESC 降序排序时 NULL 值在最前面返回，在参数等于 1 的情况下，NULL 值的返回与 ORACLE 保持一致；2 表示 ASC 升序排序时 NULL 值在最前面返回，DESC 降序排序时 NULL 值在最后返回，在参数等于 2 的情况下，NULL 值的返回与 MYSQL 保持一致；3 表示在取值为 1 的基础上，将空串置于 NULL 值和非空值之

			间
SUBQ_CVT_SPL_FLAG	1	动态， 会话级	控制相关子查询的实现方式，0：不优化； 1：使用 SPL2 方式实现相关子查询，由参数 ENABLE_RQ_TO_SPL 取值决定； 2：DBLINK 相关子查询是否转换为函数，由参数 ENABLE_DBLINK_TO_INV 取值决定； 4：将多列 IN 转换为 EXISTS，由参数 MULTI_IN_CVT_EXISTS 取值决定； 8：将引用列转换为变量 VAR； 16：用临时函数替代查询项中的相关查询表达式，由参数 ENABLE_RQ_TO_INV 取值决定； 32：存储过程、语句块中的多列表达式过滤条件含有非相关子查询时转换为连接 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
ENABLE_RQ_TO_SPL	1	动态， 会话级	是否将相关子查询转换为 SPL2 方式，0：不优化；1：优化
MULTI_IN_CVT_EXISTS	1	动态， 会话级	多列 IN 是否转换为等价的 EXISTS 过滤。0：不转换；1：转换
PRJT_REPLACE_NPAR	1	动态，	是否将引用列转换为变量 VAR，并替换查询表达式中的引用列，0：不优化；1：优化

		会话级	
ENABLE_RQ_TO_INV	0	动态， 会话级	相关查询表达式是否转换为函数方式实现。0：不转换；1：转换；2：系统根据实际情况决定是否转换
SUBQ_EXP_CVT_FLAG	193	动态， 会话级	是否将带有集函数且没有 GROUP BY 的相关查询表达式优化为非相关查询表达式。 0：使用普通的去相关性处理； 1：带有集函数且没有 GROUP BY 的相关查询表达式优化为非相关查询表达式； 2：将子查询列均来自上层查询且子查询中不包含层次查询、ROWNUM、TOP 的相关子查询的 FROM 项改写为(SELECT TOP 1 1 FROM ...)派生表，减少中间结果集； 8：将 EXISTS 子查询的查询项转换为常量 0，兼容 ORACLE 处理方式，同时优化子查询对查询项的处理过程。如果子查询为集合查询，则不进行转换； 16：通过原始语句判断查询表达式是否相同； 32：带有集函数且没有 GROUP BY 的相关查询表达式在 OP_NBEXP 中(只有一层)，且集函数没有 COUNT，那么相关拉平可以使用 CROSS JOIN 替换 LEFT JOIN； 64：消除没被使用到的非相关查询表达式或者相关列已在同层 DISTINCT 子句中的相关查询表达式查询项；

			128：对于 DISTINCT 查询表达式，在不影响 DISTINCT 结果情况下，将查询表达式放在 DISTINCT 之后做； 256：尝试将满足条件的 SELECT ITEM 中的子查询与 FROM 项进行左外连接，当遇到多个相似的 SELECT ITEM 时，可避免生成多个 SQL 语句。 512：延迟计算派生表中的相关查询表达式的查询项。 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化。SUBQ_EXP_CVT_FLAG 同时包含 1 和 32 时，1 的优先级更高
USE_REFER_TAB_ONLY	0	动态，会话级	处理相关子查询时是仅将相关的表下放，或者连同上方的 SEMI/HASH JOIN 一起下放。0：一起下放；1：仅下放相关表
REFED_SUBQ_CROSS_FLAG	1	动态，会话级	是否将相关子查询与外层表优化为 CROSS JOIN，0：不优化；1：优化 注：DM 早期版本参数，不再推荐使用
IN_LIST_AS_JOIN_KEY	0	动态，会话级	搜索多表连接方式时，对于索引连接（INDEX JOIN）的探测，NEXP_IN_LST 表达式类型可以作为多表连接的 KEY。 取值： 0：表示搜索多表连接方式时，对于索引连接（INDEX JOIN）的探测，NEXP_IN_LST 不可以作为连接 KEY； 1：表示搜索多表连接方式时，对于索引连接（INDEX JOIN）的探测，把 NEXP_IN_LST 当做普通等值 KEY 的

			处理方式来生成 INDEX JOIN 的连接 KEY。
OUTER_JOIN_FLATING_FLAG	1	动态， 会话级	优化外连接。 0：不启用优化； 1：启用优化，例如，使得类似这样的连接 A LEFT (B CROSS C) ON A.C1=B.C1 优化为 A LEFT (B JOIN (DISTINCT A) ON A.C1=B.C1 CROSS C) ON A.C1=B.C1 ，相当于把 A 和 B 的过滤器平坦化到了下层查询中，使得 B 获取的中间结果较小
TOP_ORDER_OPT_FLAG	69	动态， 会话级	优化带有 TOP 和 ORDER BY 子句的查询，使得 SORT 操作符可以省略。优化的效果是尽量使得 ORDER BY 的排序列所对应的基表可以使用包含排序列的索引，从而可以移除排序 SORT 操作符，减少排序操作。如果排序列不属于同一个基表，或者排序列不是基表列，则无法进行优化。取值： 0：不启用该优化； 1：对最优索引进行优化； 2：优先选择与排序列一致的可以消除排序的索引进行优化； 4：若查询项中包含 TOP 子句和集函数，且查询语句中不存在分组项，则移除 TOP 子句。仅当 TOP 值不为 0 时进行该优化；

			8:当前就有计划是 CSCN 的 TOP_ORDER 语句，根据估算需要 BLKUP 的行数判断是否依然使用 TOP_ORDER 优化； 16:当没有去重操作时，使用堆排序算法优化 TOP N 32:当 ORDER BY 操作符需要的 TOP 个数大于 SELECT 操作符估算出的函数时，直接使用目标索引的全表行数计算代价 64：消除排序时，如果 ORDER 需要孩子的行数小于 TOP 行数，优先使用不消除排序的计划 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
TOP_ORDER_ESTIMATE_CARD	256	动态，会话级	TOP_ORDER_OPT_FLAG=1/2 命中优化时，设置叶子节点的预估扫描行数。下层操作符不是简单的 BT 时，如果优化器预估行数不准，可帮助优化器调整，以便选择最优计划。取值范围（0~4294967294）
TOP_DIS_HASH_FLAG	1	动态，会话级	是否通过禁用 HASH JOIN 方式来优化 TOP 查询，取值： 0：不优化； 1：当 OPTIMIZER_MODE 为 0 时，TOP 下方连接禁用 HASH JOIN；当 OPTIMIZER_MODE 为 1 时，TOP 下方最近的连接倾向于不使用 HASH JOIN； 2：当 OPTIMIZER_MODE 为 0 时，TOP 下方连接禁用 HASH JOIN；当 OPTIMIZER_MODE 为 1 时，TOP 下方

			所有连接都倾向于不使用 HASH JOIN
ENABLE_RQ_TO_NONREF_SPL	1	动态，会话级	相关查询表达式转化为非相关查询表达式，目的在于相关查询表达式的执行处理由之前的平坦化方式转化为一行一行处理。 0：不启用该优化；1：对查询项中出现的相关子查询表达式进行优化处理；2：对查询项和 WHERE 表达式中出现的相关子查询表达式进行优化处理；4：相关查询采用 SPL 方式去相关性后，可以作为单表过滤条件 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
ENABLE_CHOOSE_BY_ESTIMATE_FLAG	0	动态，会话级	通过估算引用表/表连接的结果集行数决定是否进行去相关性优化。 0：不进行行数估算；1：引用单表且 ENABLE_RQ_TO_NONREF_SPL 不为 0 时进行行数估算，若行数过大则不进行 ENABLE_RQ_TO_NONREF_SPL 为对应值时的优化；2：引用单表且 SUBQ_EXP_CVT_FLAG=1 时进行行数估算，若行数过大则不进行 SUBQ_EXP_CVT_FLAG=1 时的优化；4：引用表连接且 ENABLE_RQ_TO_NONREF_SPL 不为 0 时进行行数估算，若行数过大则不进行 ENABLE_RQ_TO_NONREF_SPL 为对应值时的优化。由于需要通过动态采样的方式估算表连接后的结果集行数，因此第一次执行时速度可能较慢 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化

OPTIMIZER_MODE	1	动态，会话级	DM 优化器的模式。 0：老优化器模式，该模式采用后序遍历算法对关系树计算最优计划；1：新优化器模式，该模式采用先序遍历算法对关系树计算最优计划，同时扩展了 NEST LOOP JOIN 等功能。 总体来说，新优化器模式可以替代老优化器的功能，并具有更多的优点。因此建议用户使用新优化器模式
NEW_MOTION	1	动态，会话级	是否使用代价计算来决定使用的通讯操作符，0：否；1：是。 DMDPC 环境下不支持动态修改该参数
LDIS_NEW_FOLD_FUN	0	动态，会话级	是否开启 LOCAL DISTRIBUTE 操作符使用与 MPP DISTRIBUTE 操作符不同的函数来计算哈希值。0：否；1：是
DYNAMIC_CALC_NODES	0	动态，会话级	是否根据数据量设置 LOCAL DISTRIBUTE\ MPP DISTRIBUTE 操作符不同的并行度\节点数。0：否；1：是
OPTIMIZER_MAX_PERM	7200	动态，会话级	控制计划探测过程中的最大排列数，如果大于该阈值，则减少相应的排列，减少探测计划的时间，取值范围 1~4294967294
ENABLE_INDEX_FILTER	1	动态，会话级	索引过滤的优化设置，可取值为 0、1、2、4。 0：不进行优化；

			1：使用索引过滤优化，如果过滤条件涉及的列包含在索引中，那么索引进行 SSEK2 后就可以使用此过滤条件，可以减少中间结果集； 2：在取值为 1 的基础上，将 IN 查询列表转换为 HASH RIGHT SEMI JOIN； 4：将会执行多次且元素较多的 IN_LIST 不作为索引过滤条件。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化。取值 2 和取值 3 效果一样
OPTIMIZER_DYNAMIC_SAMPLING	0	动态， 会话级	当统计信息不可用时是否启用动态统计信息。取值范围：0~12。0：不启用；1~10：启用，采用率 10%~100%； 11：启用，由优化器确定采样率（0.1%~99.9%）；12：同 11，但收集的结果会持久化保存
AUTO_STAT_OBJ	0	动态， 系统级	是否监控增删改导致的修改行数和 TRUNCATE 表操作。0：不监控；1：对所有用户表进行监控；2：只对用户通过 DBMS_STATS. SET_TABLE_PREFS 设置过 STALE_PERCENT 属性的用户表进行监控。 DMDSC 环境下仅允许控制节点动态修改该参数，主备不同步
MONITOR_MODIFICATIONS	0	动态， 系统级	是否监控用户表对象的数据变化情况，包括增删改导致的修改行数、TRUNCATE 表操作以及最新修改时间等。 0：不监控；1：对所有表进行监控；2：只对用户执行过 SET_TABLE_PREFS 的表进行监控。 DMDPC 环境下不支持动态修改该参数，DMDSC 环境下仅允许控制节点动态修改该参数，主备不同步
MON_CHECK_INTERVAL	3600	动态，	将用户表对象的数据变化情况记录到系统表的时间间隔，单位为秒，即每过设置的秒数便将用户表对象的数据

		系统级	变化情况记录到系统表中。仅当 MONITOR_MODIFICATIONS 非 0 时生效
NONREFED_SUBQUERY_AS_CONST	1	动态，会话级	是否将非相关子查询转化为常量处理。0：不进行优化，对非相关子查询使用连接方式处理；1：将非相关子查询转换为常量，作为过滤条件使用
HASH_CMP_OPT_FLAG	0	动态，会话级	是否启用静态哈希表的优化，控制哈希内连接、分组、DISTINCT、包含 DISTINCT 的分组四种操作符的数据比较方式。使用不同哈希算法计算原始数据哈希值，通过对哈希值比较判断数据是否相等。仅适用于对结果正确性不敏感的场景使用。 0：不启用此优化；1：分组中的 DISTINCT 开启此优化；2：对 HASH 连接启用此优化；4：对 HAGR 分组计算启用此优化；8：DISTINCT 操作开启此优化。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
OUTER_OPT_NLO_FLAG	0	动态，会话级	是否进行外连接和内连接的相关优化。0：不优化；1：满足条件时，将外连接转换为嵌套外连接进行优化；2：满足条件时，将内连接转换为嵌套连接+VAR 方式进行优化；4：打开外连接操作符 HLO/HRO/MLO/MRO 只返回不满足条件值的优化。 取值 1、2 仅在 OPTIMIZER_MODE = 0 时生效。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化

DISTINCT_USE_INDEX_SKIP	2	动态，会话级	DISTINCT 列是否使用索引跳跃扫描（单列索引或复合索引）。专门用于 SQL 语句中有 DISTINCT 的场景，DISTINCT 列的查询的一种优化方式，在索引上跳跃着扫描。0：不使用；1：强制使用；2：根据代价选择是否使用；4：多级分区表也尝试使用(所有层次的分区列都需要在 DISTINCT 中)。支持使用上述有效值的组合值，如 7 表示同时进行 1、2、4。该参数只在 OPTIMIZER_MODE=1 时有效
USE_INDEX_SKIP_SCAN	0	动态，会话级	是否使用复合索引跳跃扫描。专门用于 WHERE 子句等值条件的查询列中包含了复合索引的列，但该列又不是复合索引的前导列，这种情况下，选择是否使用复合索引跳跃扫描。和 INDEX_SKIP_SCAN_RATE 搭配使用。 0：不使用； 1：根据代价选择是否使用。通过代价计算选择，倾向于选择前导列 DISTINCT 值少，搜索列 DISTINCT 值较多的索引； 2：强制使用。只要能够找到索引可以使用跳跃扫描，就强制使用
INDEX_SKIP_SCAN_RATE	0.002500	动态，会话级	复合索引跳跃扫描的代价调节开关。当列的 （DISTINCT 数/总行数）比值大于该值时，就不再使用索引跳跃扫描的方式。取值范围 0~1。 和 USE_INDEX_SKIP_SCAN 搭配使用

SPEED_SEMI_JOIN_PLAN	9	动态， 会话级	是否加速半连接的探测过程。 0：不加速； 1：加速计划探测； 2：加速计划探测和执行； 4：调整 DISTINCT 代价估算，期望更接近实际情况，有利于计划选择。4 支持的场景如下： 1)控制半连接转换内连接，生成的 DISTINCT 代价估算； 2)控制普通 DISTINCT 代价计算； 8：SEMI 规则优化增加处理右孩子包含 SLCT 的情况； 16：SEMI 探测消除 SORT 优化时多探测一次 HRS 的情况； 32：仅使用预分析处理好的叶子节点计划进行 SEMI 最优计划探测，可以减少探测路径提升分析速度，但计划可能不是“最优”； 64：半连接执行时对右侧数据进行去重处理，以减少计算连接条件的次数。 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
COMPLEX_VIEW_MERGING	2	动态，	对于复杂视图（一般含有 GROUP 或者集函数等）会执行合并操作，使得 GROUP 分组操作在连接之后才执

		会话级	行。0：不启用；1：对不包含别名和同名列的视图进行合并；2：视图定义包含别名或同名列时也进行合并
ENHANCED_SUBQ_MERGING	3	动态，会话级	是否对子查询进行合并优化，0：不优化；1：对 WHERE 子句和 JOIN ON 子句中存在包含关系的子查询进行合并优化；2：对 UNION 和 INTERSECT 等集合运算中相同的子查询进行合并优化；4：将相同相关列在 OR 集合运算中的多个 EXISTS 子句转换为 UNION ALL 子句 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
HLSM_FLAG	1	动态，会话级	控制多列非相关 NOT IN 的查询实现方式，1：当数据量较大，HASH BUF 放不下时，采用 MTAB 方式处理；2：当数据量较大，HASH BUF 放不下时，采用 B 树方式处理，使用细粒度扫描；3：当数据量较大，HASH BUF 放不下时，采用 B 树方式处理，使用粗粒度扫描
DEL_HP_OPT_FLAG	0	动态，会话级	控制分区表的操作优化，0：不优化； 1：打开分区表 DELETE 优化； 2：控制范围分区表创建的优化处理，转换为数据流方式实现； 4：允许语句块中的间隔分区表自动扩展； 8：开启对 TRUNCATE 分区表的优化处理； 16：完全刷新时删除老数据使用 delete 方式。

			支持使用上述有效值的组合值，如 7 表示同时进行 1、2、4 的优化
OPTIMIZER_OR_NBEXP	29	动态 会话级	OR 表达式的优化方式。0：不优化； 1：生成 UNION_FOR_OR 操作符时，优化为无 KEY 比较方式； 2：OR 表达式优先考虑整体处理方式； 4：相关子查询的 OR 表达式也优先考虑整体处理方式； 8：OR 布尔表达式的范围合并优化； 16：同一列上同时存在常量范围过滤和 IS NULL 过滤时的优化，如 C1 > 5 OR C1 IS NULL。 32：OR 表达式公因子包含子查询时，使用 UNION_OR 方式。避免特定情况过滤条件无法下放。 64：OR 表达式转为 UNION_OR 方式分析时，现在取消了对整个 UNION_OR 添加 AUTOID，使过滤条件尽可能下放，提高性能。该控制位开启时将屏蔽这一优化，仅作为保险措施，正常使用时建议关闭。 支持使用上述有效值的组合值，如 7 表示同时进行 1、2、4 的优化
NPLN_OR_MAX_NODE	20	动态， 会话级	查询优化时，是否展开 OR 表达式的开关。 系统自动将 OR 表达式的分支数和 NPLN_OR_MAX_NODE 进行对比。如果分支数大，则 OR 表达式不予展开，而是当作整体处理，减少内存消耗；如果分支数小，则系统自动决定是否将 OR 表达式展开。

			取值范围：0~20。0 表示 OR 表达式一律不展开。 此参数优化的 SQL 语句是指包含 OR 连接条件但整体不是 OR 的表达式。例如： 语句 SELECT * FROM T01 LEFT JOIN T02 ON (C1<C2 OR C1>C2+2 OR D1<D2 OR D1>D2+5) AND C1>5; 是一个包含了 OR 但整体是 AND 的语句，展开后 OR 分支数为 4，分别为 C1<C2、C1>C2+2、D1<D2 和 D1>D2+5
CNNTB_OPT_FLAG	193	动态， 会话级	是否使用优化的层次查询执行机制。 0：不使用； 1：强制使用； 2：优化器自动决定是否使用； 4：层次查询不支持并行； 8：检查层次查询的表达式是否包含 ROWNUM，若包含则报错； 16：层次查询的条件包含 OR 时仍尝试使用优化，且将 OR 连接的条件表达式当做整体处理； 32：根据 PRIOR 关联列的 DISTINCT 值与统计信息中总行数的比例自适应选择执行机制； 64：对于简单条件（如：DUAL.LEVEL<=10）准确估算层次查询行数； 128：对层次查询的执行计划节点分析时，如果右孩子包含参数无法生成临时表，依然向下分析右孩子的子节

			点能否生成临时表以提升性能； 256：单子树优化模式，仅基于左孩子数据做深搜找到满足条件的路径；旨在减少右孩子的频繁扫描和数据拷贝；仅当 CONNECT BY 条件为非等值条件时生效，发生刷盘时会转换成 CNNTB_OPT_FLAG(0)； 512：针对层次查询的 START_WITH 条件尝试进行外连接转内连接的优化。 1024:DPC 下，CNNTB 支持右边广播的连接方式 支持使用上述有效值的组合值，如 31 表示同时进行 1、2、4、8、16 的优化
ADAPTIVE_NPLN_FLAG	3	动态， 会话级	是否启用自适应计划机制。 0：不启用； 1：仅 OPTIMIZER_MODE=1 时生效，对索引连接、嵌套含 VAR 连接等复杂连接启用自适应计划； 2：仅 OPTIMIZER_MODE=1 时生效，ORDER BY 在 HASH 连接时启用自适应计划； 4：仅 DPC 或新并行时生效，对两层 GROUP/DISTINCT 中的下层 GROUP/DISTINCT 启用自适应，若聚合度/去重率不高则转为管道模式。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化
MULTI_UPD_OPT_FLAG	1	动态，	0：对普通表进行多列更新时，按照语句改写方式实现；对派生表进行多列更新时，利用 SPL 功能实现；

		会话级	1：利用多列 SPL 功能实现多列更新； 2：允许对多表连接视图进行更新； 4：相关查询语句利用多列 SPL 功能实现多列更新； 8：多表连接视图进行更新时不再检查视图的唯一性，而是在 UPDATE 下添加 UFLT，实际执行遇到冲突行时报错。 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
MULTI_UPD_MAX_COL_NUM	128	动态， 会话级	利用多列 SPL 功能以实现多列更新(MULTI_UPD_OPT_FLAG 为 1)时可更新的最大列数，取值范围 2~10000
ENHANCE_BIND_PEEKING	0	动态， 会话级	是否使用自适应的绑定变量窥探开关。0：不使用；1：使用
NBEXP_OPT_FLAG	7	动态， 会话级	控制布尔表达式的一些优化。0：不优化；1：进行根据展开后的项数控制 NOT 是否下放展开的优化；2：进行 AND 分支的 OR 布尔表达式的公因子上拉优化；4：探测移除和派生表/实体视图内层一样的外层过滤条件；8：打开 JSON_VALUE 表达式转换成 JSON TABLE 的优化；16：关闭等值表达式常量替换优化；32：对 WHERE 和 JOIN 条件增强常量替换优化。

			支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
HAGR_HASH_ALGORITHM_FLAG	0	动态，会话级	HAGR 中 HASH 算法选择标记。0：按照移位方式计算；1：按照异或方式计算
DIST_HASH_ALGORITHM_FLAG	0	动态，会话级	DISTINCT 中 HASH 算法选择标记。0：按照移位方式计算；1：按照异或方式计算
VIEW_FILTER_MERGING	138	动态，会话级	是否对视图条件进行优化以及如何优化。 0：不优化； 1：尽可能地进行视图条件合并； 2：自动判断是否进行视图条件合并； 4：视图包含分析函数时，如果分析函数的 PARTITION BY 具有唯一性，则外层过滤条件下推到视图内部； 8：如果派生表存在集函数，则不进行条件下推优化； 16：收集公用表达式相关的所有过滤条件，改写合并所有条件，并下放到公用表达式的内部；32：TOP 信息下放至派生表内部； 64：外层表达式扩展视图内部 IS NOT NULL 过滤条件；

			128：外层 ROWNUM 过滤条件转为内层 TOP 子句； 256：满足条件时，对相关派生表过滤条件进行下推优化； 512：视图已经有过滤条件的，不重复下放，为了后续能使用 HTAB； 1024：包含该取值时，取消下述限制：视图是集合运算时的分支数应小于 10；若视图是集合运算且超过 2 个分支时，过滤条件不能含有复杂函数/查询表达式。 支持使用上述有效值的组合值，如 138 表示同时进行 2、8、128 的优化
OPT_MEM_CHECK	0	动态，会话级	内存紧张时，优化器是否缩减计划探测空间，仅 OPTIMIZER_MODE=0 时生效。0：不缩减计划探测空间；1：缩减计划探测空间
ENABLE_JOIN_FACTORIZATION	1	动态，会话级	是否启用连接分解，即 UNION ALL 分支间存在公共部分时是否进行公因子提取。0：不启用；1：启用
ERROR_COMPATIBLE_FLAG	0	动态、会话级	是否对子查询同名列进行报错。取值 0 或 1。 0 报错，1 不报错
ENABLE_PARTITION_WISE_OPT	1	动态，会话级	是否利用水平分区表的分区信息进行排序、分组的计划优化，即在满足条件的情况下对每个子表单独排序、分组，或者进行子表间的归并排序。0：不启用；1：启用

EXPLAIN_SHOW_FACTOR	1	动态， 会话级	显示执行计划的行数和嵌套层数的基数分别为 1000 和 100，实际显示的行数和嵌套层数为 基数* EXPLAIN_SHOW_FACTOR。取值范围 1~100
HASH_PLL_OPT_FLAG	107	动态， 会话级	当进行 HASH SEMI/INNER 连接且右表为分区表时，控制是否对分区表连接进行优化以及优化的方式。 0：不进行分区表连接优化； 1：允许 HI/HRS 进行分区裁剪优化； 2：单个分区列的一级分区等值查询优化； 4：若为 ANTI SEMI 连接，则不允许使用 INDEX JOIN； 8：索引连接支持非等值 KEY 作为 PLL 的 KEY； 16：分区表根据子表数目粗略调整代价； 32：分析阶段根据 KEY 和扫描方式获得准确数据行数，精准计算 PLL 代价； 64：分区表全局索引扫描支持分区裁剪； 128：DELETE 分区表时计划阶段收集子表信息，执行阶段直接拷贝使用即可，无需再次收集； 256:分区表 INSERT 执行计划不缓存分区子表信息； 512：执行计划不缓存分区信息。

			支持使用上述有效值的组合值，如 107 表示同时进行 1、2、8、32 和 64 的优化
PLACE_GROUP_BY_FLAG	0	动态，会话级	在含有集函数的查询中，是否允许先分组减少数据量后再进行连接。 0：不允许； 1：允许，要求是两个表的连接，且基函数的参数只能来自一张表； 2：允许，2 是 1 的加强，允许有多张表的连接，且基函数的参数可来自多张表，与 MAX_N_GRP_PUSH_DOWN 配合使用。 此参数仅在 OPTIMIZER_MODE=1 时有效
ENABLE_NEST_LOOP_JOIN_CACHE	0	动态，会话级	是否对嵌套循环连接进行优化以及如何优化，0：不优化；1：允许缓存嵌套循环连接的中间结果以加速执行；2：嵌套循环连接执行时优先探测右孩子节点的数据 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
ENABLE_TABLE_EXP_REF_FLAG	1	动态，会话级	是否支持嵌套表表达式或 XML 表表达式与源表的同层引用。0：不支持；1：支持
BIND_PARAM_OPT_FLAG	3	静态	指定绑定参数优化策略。0：不同的绑定参数之间认为不相等；1：根据绑定值判断绑定参数是否相等。确认绑定值之前，认为不同的绑定参数之间相等；确认绑定值之后，根据绑定值判断是否相等；2：优化绑定参数和

			常量之间的比较表达式，例如：针对表达式?=1，若绑定参数的绑定值为 1，则将表达式转换为 TRUE，否则转换为 FALSE。 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
VIEW_OPT_FLAG	33	动态，会话级	视图优化处理参数。0：关闭优化；1：允许不可更新视图输出 ROWID；2：允许对物化视图日志表进行 INSERT、DELETE、UPDATE 以及 TRUNCATE 操作；4：允许对物化视图日志表的基表进行删列、修改列名、修改列类型等操作；8：语义分析阶段移除视图、派生表中未被外层使用的列，提高索引代价估算准确度，选择合适的索引；16：TRUNCATE 基表时，基于主键的日志表按照 PURGE 选项进行清空；32：ON COMMIT 刷新属性方式的物化视图同一时刻最多仅触发一次。支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
ENABLE_DBLINK_TO_INV	0	动态，会话级	DBLINK 相关子查询是否转换为函数。0：不转换；1：转换
ENABLE_BLOB_CMP_FLAG	1	动态，会话级	是否支持大字段类型的比较。0：不支持；1：支持，此时 DISTINCT、ORDER BY、 分析函数和集函数支持对大字段进行处理，且当大字段跟字符串比较时，让大字段转换成字符串类型；2：支持，此时 DISTINCT、ORDER BY、 分析函数和集函数支持对大字段进行处理，且兼容 MYSQL 模式下，大字段跟字符串比较时，让

			字符串转换成大字段类型
CVIEW_STAR_WITH_PREFIX	1	动态，会话级	创建视图获取视图定义语句，展开查询项中的 *时，列名前是否添加表名作为前缀。0：不添加前缀；1：添加前缀
ENABLE_ADJUST_NLI_COST	1	动态，会话级	是否允许系统自动对嵌套连接的代价进行增大调整。0：不允许；1：允许
ENABLE_SEQ_REUSE	0	动态，会话级	控制是否允许重用相同的序列表达式。0：不允许；1：允许。 例如 SELECT SEQ1.NEXTVAL,SEQ1.NEXTVAL FROM DUAL; 当参数取 1：返回值两列相同 当参数取 0：返回值两列不同
DPC_SYNC_STEP	16	动态，会话级	DMDPC 环境下专用，当打开 DMDPC 限流（MPP_MOTION_SYNC 非 0）时使用。 表示检查接收端数据堆积的步长，即每发送 DPC_SYNC_STEP 次 BDTA 后检查一次是否堆积。取值范围（1~5000）
DPC_SYNC_TOTAL	0	动态，	DMDPC 环境下专用，当打开 DMDPC 限流（MPP_MOTION_SYNC 非 0）时使用。

		会话级	XBOX 上所有来源路数（并行线程数）的总限流资源数。取值范围 0~10000
XBOX_DUMP_THRESHOLD	0	动态， 系统级	DMDPC 环境下专用，XBOX 邮件内存阈值，超过该值则保存到文件中。取值范围 0~409600。单位 MB。 和 XBOX_DUMP_PATH 一起设置才生效
XBOX_SPACE_LIMIT	0	动态， 系统级	DMDPC 环境下专用，表示单个 XBOX 的空间限制。取值范围 0~409600。单位 MB。缺省值为 0，表示不作限制。 需要配置了 XBOX_DUMP_THRESHOLD 和 XBOX_DUMP_PATH 后才生效。 可与 XBOXS_SPACE_LIMIT 分别设置，也可以同时设置；同时设置必须满足 XBOXS_SPACE_LIMIT > XBOX_SPACE_LIMIT
XBOXS_SPACE_LIMIT	0	动态， 系统级	DMDPC 环境下专用，表示系统中所有 XBOX 的总空间限制。取值范围 0~1000000。单位 MB。缺省值为 0，表示不作限制。 需要配置了 XBOX_DUMP_THRESHOLD 和 XBOX_DUMP_PATH 后才生效。 可与 XBOX_SPACE_LIMIT 分别设置，也可以同时设置；同时设置必须满足 XBOXS_SPACE_LIMIT > XBOX_SPACE_LIMIT
XBOX_SHORT_MSG_SIZE	1024	动态，	DMDPC 环境下专用，在配置了 XBOX_DUMP_THRESHOLD 后，即使在内存达到阈值的情况下，小于此长度的

		系统级	短消息也不会保存到文件，防止大量的文件读写。取值范围 0~32768。单位 BYTE
STMT_XBOX_REUSE	1	动态， 会话级	DMDPC 环境下专用，设置 DMDPC 会话的语句是否重用 BOX_ID。 0：否，每个语句都请求分配一个 BOX_ID； 1：是
MAX_HEAP_SIZE	0	静态	控制分析阶段允许使用的堆空间大小上限，取值范围 0~4294967294，单位 MB。设置为 0 时，表示不进行限制
ENABLE_ADJUST_DIST_COST	0	动态， 会话级	是否对索引连接的 DISTINCT 值估算行数进行优化调整。0：不优化；1：若索引连接计划中左孩子为 DISTINCT 或 GROUP 类型，则使用左孩子的估算行数和右孩子的估算行数相乘作为连接后的 DISTINCT 行数值；2：索引连接计划中，使用左孩子的估算行数除以 3 和右孩子的估算行数相乘作为连接后的 DISTINCT 值。支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
PLAN_OP_FLAG	0	动态， 会话级	控制执行计划中操作符的使用。0：默认使用所有操作符；1：禁用 SSCN 操作符；2：禁用 SSEK 操作符；4：仅对 BLKUP 操作符的执行进行优化，即在 B 树上搜索到一行记录后，继续尝试在当前页上搜索下一条记录；8：在 4 的基础上，生成计划时，在 BLKUP 操作符下方添加一个 SORT 操作符，使传给 BLKUP 的数据更加紧凑，更能发挥 BLKUP 优化的优势，缺点是 SORT 带来的额外开销；12：在 4 的基础上，对每一批传给 BLKUP

			的数据进行 BDTA 排序，不额外使用新的排序操作符；16：AAGR2 操作符计算 COUNT(COLUMN)集函数时，如果下层操作符为聚集索引扫描（CSCN2），则使用优化方式计算结果；32：MERGE INTO 语句忽略重复键值错误；64：CSCN2、SSCN 使用依照物理存储结构的扫描方式，此操作会 X 锁被扫描的表；128：如果 DISTINCT 中的某些前导列是有序的，那么可以对这些列不同的值进行分组去重。满足该优化条件的情况有 3 种：DISTINCT 下面是 MERGE JOIN、ORDER 或 TABLE 的索引扫描，这三种情况下 DISTINCT 收到的数据是有序的，此时只要 DISTINCT 中某些列是排序列的前缀，就可以按照此前缀进行分组；256：用于控制 UNION 实现方式，参数开启时 UNION 左右孩子增加 SORT 操作并以 MERGE 方式归并数据以保证结果集有序，仅单机环境下有效；512：如果回表查询行数较多，取值 512 增大回表查询代价，倾向于不使用回表查询；1024：控制索引连接输出数据策略：填满 BDTA_SIZE 行后再输出。预期减少索引连接及其父层操作符执行次数，从执行计划看不出差异，可通过 ET 监控 N_ENTER 列观察实际效果；2048：开启外连接的连接条件批量计算优化；4096：开启半连接的连接条件批量计算优化；8192:以批量绑定参数方式执行 MERGE INTO 时禁用 HASH 连接；16384：CSEK 操作符尝试探测连续的记录定位，在部分计划如绑定主键连续定位的场景中减少搜索代价；32768：输出单值的没有非确定性函数的相关子查询产生的 SPL，相同的相关列值只运行一次。 支持使用上述有效值的组合值，如 13 表示同时进行 1 和 12 的操作

DUAL_ENABLE_SELECT	1	动态， 系统级	MOUNT 状态下是否允许查询 DUAL/SYSDUAL/SYS.SYSDUAL2 表。0：不允许，1：允许
LOAD_BINDED_PLN	0	动态， 系统级	是否从系统表 SYSPLNINFO 加载已持久化的绑定计划。1：加载；0：不加载
UNPIVOT_OPT_FLAG	0	动态， 会话级	控制 UNPIVOT 子句转换处理。0：不进行特殊处理，正常转换；1：结果集按照非 UNPIVOT 列排序；2：若 UNPIVOT 子句排除 NULL 值，则在转换的 WITH 视图中添加 NOT NULL 过滤条件，减少中间结果集 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化
USE_DHASH_FLAG	0	动态， 会话级	是否使用动态 HASH。0：不使用；1、2：使用 DHASH2；3：使用 DHASH3，使用动态 HASH 时推荐选择 3
DPC_OPT_FLAG	65407	动态， 会话级	DMDPC 环境下专用，执行计划优化策略。取值 0、1、2、4、8、16、32、64、128、256、512、1024、2048、4096、8192、16384、32768、65536、131072、262144、524288、1048576、2097152 或上述任意取值的组合值。 0：使用默认策略，不进行优化； 1：表示忽略 UNIONALL，将分支的内容直接发给 UNIONALL 之上的 ERECV；

			2：表示根据 HSCN 的使用情况，对 HTAB 的分布信息调整，可以减少 HSCN 之后的数据分发量； 4：表示在通讯符之前将 AFUN 转换成两个，可以减少通讯量； 8：表示不存在分区列过滤条件时，分区遍历操作符提前收集所有子表信息并加以缓存； 16：根据利用统计信息最大、最小值计算相同表不同别名的选择率。例如：T1.C1 + 10 > T1.C1 ==> T1.C1 < MAX(T1.C1) +10； 32：同时开启 SP、MP 和 BP 节点的计划缓存的功能，减少事务密集型应用中计划发送带来的耗时。可通过 V$SQL_PLAN 查询到 BP 端缓存的不完整计划，其 RET_CMD 字段为 0。非 32 的情况下，只有 SP 具有计划缓存功能； 64：多列 PRIMARY KEY 用于连接时的行数估算调整优化； 128：根据估算的行数对连接、分组等使用到的哈希表大小进行缩放优化； 256：哈希连接时，如果连接左边数据来自广播，启用共享哈希表优化减少哈希表构造开销； 512：HUGE 表考虑 HFSEK 与 HFSCN+SLCT 的代价； 1024：计划只从几个子任务开始，而不是所有的叶子节点； 2048：DISTINCT 尝试下放至集合运算的左右侧的子任务先做；

			4096：AFUN/SAGR+SORT+DIS 的 SORT 下放到 DIS 下面，变成 AFUN/SAGR+DIS（归并信息）+SORT。SORT 放到 DIS 下面后，每一路可先做排序，如果子任务并行度比 SAGR/AFUN 的并行度大时能较快的完成排序； 8192：ESEND 发送时对 LINK_ID 进行优化。优化后，消息可以多条链路同时发生，而非一条链路串行； 16384：表示 DMDPC 环境下支持变量跨层引用； 32768：支持 L_FULL 优化。具体为支持 LEFT JOIN 操作符左边使用全部数据，右边使用部分数据的情况； 65536：表示开启子任务为空连接提前终止的优化。若某个子任务的结果为空，上层有连接操作且可以推断出连接的结果也为空时，那直接终止此连接以及此连接的所有子任务。这个过程是可以向上传递的，直到某个子任务结果不为空或者无法推断为空为止； 131072：表示当某任务的子任务都在同一个站点上时，那么系统会优先让这个任务也在此站点做。因为数据在同站点不同线程内传输，相比不同站点会减小数据传输时间； 262144：表示将 GROUP BY 最大值下放到 HTAB；

			524288：表示 SP、BS 节点考虑将接收到的简单 SQL（非分区表查询、包含分区列等值条件过滤的查询、行分区表的值插入等）转发给对应的 BP 节点处理以提升性能；此优化还需要设置 USE_PLN_POOL 非 0 开启计划重用，并与 DPC_OPT_FLAG 取值 32 一起使用； 1048576：当哈希连接的 KEY 包含有大量 NULL 值时，是否启用空值的分发优化。开启后，连接 KEY 包含 NULL 值的记录会根据语义选择自动丢弃或随机选择目标线程； 2097152：两层 GROUP 且中间是 DIS 时，如果下层 HAGR 有冲突，直接将冲突的输出
LIKE_PATTERN_NUM	300	动态， 系统级	LIKE 模式匹配时模糊匹配的长度限制。取值范围 100~10000
FORALL_OPT	1	动态， 系统级	是否进行 FORALL 优化。0：不优化；1：优化；2：仅对 INSERT 语句进行优化。 当 USE_FORALL_ATTR=1 时，系统强制放弃 FORALL 优化
HASH_OPT_FLAG	1	动态， 会话级	是否启用哈希表的普遍优化。0：禁用哈希相关的优化；1：允许进行哈希相同值挂链优化，仅当 HASH_CMP_OPT_FLAG=0 且 USE_DHASH_FLAG=0 时生效；2：针对 DISTINCT、HASH 连接以及 HAGR 分组计算等操作使用优化的 HASH 算法进行计算，仅当 HASH_CMP_OPT_FLAG=0 且 USE_DHASH=0 时生效。 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化。

SFUN_PUSH_DOWN_FLAG	1	动态， 会话级	DMDPC 环境下专用，设置是否将集函数下放到 UNIONALL 的各个分支。 0：否； 1：是。要求外层是简单查询。
MAX_SCAN_PAGES	32	动态， 系统级	用于控制 CSCN、SSCN、CSEK、SSEK 等操作符批量填充记录最大连续扫描数据页数量。取值范围 1~512。CSCN 等操作执行过程中，如果填充记录数达到 BDTA_SIZE 或连续扫描数据页数量达到 MAX_SCAN_PAGES，则结束当前操作符执行。DMDSC 集群最大扫描数据页数量强制调整为 MAX_SCAN_PAGES/HPC_NODE，HPC_NODE 为集群节点个数
ENABLE_PLN_PRESEARCH	0	静态	是否开启预查找执行计划，取值为 0、1、2 或上述取值的组合值。 0：不进行相关优化； 1：在高并发场景下，相同计划仅进行一次硬解析，若有线程报错计划构造等待超时请放大 PLN_BUILD_WAIT_TIME。若计划不可重用不建议开启此参数。该策略也可避免并发时相同 SQL 的计划重复添加缓存，但 DPC 环境下将计划发送到远程节点时没有经过硬解析，远程节点仍有可能重复添加缓存； 2：将计划添加到缓存前检查缓存中是否已存在，是则不再重复添加
DDL_OPT_FLAG	0	动态，	DDL 相关优化标记，取值为 0、1 或这些值的任意组合。

		会话级	0：无优化； 1：移动表空间时，即使目标表空间与源表空间相同，也仍然进行实际移动
DIGIT_AS_NUMBER	0	静态	用于控制常量参数化时是否把数字常量作为 number 类型。0：否；1：是
ENABLE_INJECT_HINT	0	动态，会话级	是否启用 SQL 指定 HINT 的功能，0：不启用；1：启用
STAT_CACHE_CAPACITY	1000	动态，系统级	DMDPC 环境下专用，SP 缓存表行数的缓存容量。当 STAT_CACHE_FLAG=1 时使用。取值范围 100~100000000，单位为缓存表的个数；取值为 0 时，表示不从缓存读写表行数
QUERY_INFO_BITS	0(非 DMDPC)或者 1(DMDPC)	动态，会话级	对应部分系统内部优化器 HINT 的优化功能，部分取值仅用作标识信息，取值 0、1、2、4、8、16、32、64、128、256、512、1024、2048、4096、8192、16384 或上述取值的和，如 5 表示同时进行 1 和 4 的优化。 0：不对计划生成进行任何干预，是否采纳下列各优化点由代价模型决定； 1：SEMI/CROSS JOIN 优化时不生成变量，与 HINT 的 NO_USE_CVT_VAR 效果相同； 2：SEMI/CROSS JOIN 优化时优先考虑生成变量，与 HINT 的 USE_CVT_VAR 效果相同； 4：SEMI JOIN 优化时不生成 CROSS，与 HINT 的 NO_SEMI_GEN_CROSS 效果相同； 8：SEMI JOIN 优化时优先考虑 CROSS，与 HINT 的 SEMI_GEN_CROSS 效果相同； 16：分区表遍历操作符 PLL 提前收集所有叶子表信息，与 HINT 的 COLLECT_ALL_SUB_TAB 效果相同；

			32：分区表遍历操作符 PLL 不提前收集所有叶子表信息，与 HINT 的 NO_COLLECT_ALL_SUB_TAB 效果相同； 64：不生成 MPP 通讯操作符，指定查询以 MPP 本地登录（单节点）方式执行；该值不能设置，仅为展示，对应 HINT 的 MPP_LOCAL； 128：通过插入排序操作符来实现归并连接，与 HINT 的 ENHANCED_MERGE_JOIN 效果相同； 256：查询涉及的表是否均为系统表，该值不可设置，仅为展示； 512：本次查询计划不添加到计划缓存，与 HINT 的 PLAN_NO_CACHE 效果相同； 1024：禁用按列加密明文可见性判断； 2048：DMDPC 下采用数据弱一致性级别，与 HINT 的 READ_POLICY FOLLOWER 效果相同； 4096：DMDPC 下采用数据强一致性级别，与 HINT 的 READ_POLICY LEADER 效果相同； 8192：本次查询在 V$SQL_NODE_HISTORY 中不进行替换，与 HINT 的 SQL_NODE_KEEP_CACHE 效果相同； 16384：DMDPC 查询以本地登录（单节点）方式执行；该值不可设置，仅为展示，对应 HINT 的 DPC_LOCAL 注：仅 DPC 环境下在 DM.INI 中设置该参数值可以达到与优化提示器 HINT 指定相同的效果，非 DPC 环境下文

			件中指定无效。当用户使用 HINT 方式指定相关优化后，会话中参数值会自动被 HINT 值覆盖；如果给定的取值包含了自相矛盾的功能点，优化器会按照取值大小顺序选择较小值确定的选项
FETCH_PACKAGE_SIZE	512	动态，系统级	指定 FETCH 操作时使用的消息包大小，单位 KB，取值范围 32~65536。该参数修改时仅影响新创建的会话
TABLE_STAT_FLAG	0	动态，会话级	控制表统计信息的使用方式。 0：以默认方式使用统计信息，不进行优化； 1：MPP 环境下，对于 RANGE/LIST 水平分区表，估算行数时优先采用统计信息的收集值； 2：当开启对表的数据量监控时，对变化行数达到阈值的表，使其统计信息在分析阶段失效； 4：优先从表统计信息中获取表行数。 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
SORT_ADAPTIVE_FLAG	0	动态，会话级	自适应排序区大小的方式。 0：不进行自适应； 1：SORT_FLAG 为 0、2 时，计算待排序数据的实际排序所需要的内存大小，尽可能地分配所需要的内存，如果申请失败则减半申请，直到申请成功，并按照实际申请的内存大小代替 SORT_BUF_SIZE 大小进行使用。对

			进行过刷盘的数据进行排序时，计算排序所需要的内存，尽可能地放大排序内存，保证一次排序可以结束排序。SORT_FLAG 为 1、3 时，根据 SORT_BUF_GLOBAL_SIZE 和分片内存最大 10000 个的限制，计算 SORT_BLK_SIZE 的最小值，如果 SORT_BLK_SIZE 小于最小值，则实际按照最小值来分片。对整个大内存排序缓冲区的数据进行排序时，计算排序所需要的内存，尽可能地放大排序内存，保证一次排序可以结束排序； 2：SORT_FLAG 为 0、2 时，按照行数估算待排序数据的实际排序所需要的内存大小，找到最适合排序算法的内存大小。当待排序的数据行数超过 50 万时，才开启此优化，数据量太小无需优化； 3：1 和 2 的组合。当 SORT_FLAG 为 0、2 时，计算待排序数据的实际排序所需要的内存大小时，采用 SORT_ADAPTIVE_FLAG 为 2 的按照行数估算待排序所需要内存大小。除此之外的其他情况均保持 SORT_ADAPTIVE_FLAG 为 1 的方式
SLCT_OPT_FLAG	24959	动态， 会话级	简介：对 SLCT 操作符进行优化，可取值为 0、1、2、4、8、16、32、64、128、256、512、1024、2048、4096、8192、16384、32768。支持使用上述有效值的组合值，如 6 表示同时进行 2 和 4 的优化。支持 HINT 指定，（取值 512 不支持 HINT 指定） SLCT_OPT_FLAG 有以下几种具体的优化方式： 1)查询项较多且有单列过滤条件可以过滤掉大部分数据时，通过下放 SLCT 并生成一个 BLKUP 的方式，减少列

			解析，提升查询效率； 2)查询包含 TOP 和 ORDER BY 时，通过语句改写方式推迟不必要的字段解析； 3)查询包含“ROWNUM < 常量”或“ROWNUM <= 常量”过滤条件时，通过将该过滤条件转换为 TOP 子句的方式，减少列解析，提升查询效率； 4)查询包含 ROWNUM 与常量或 NULL 构成的表达式过滤条件时，通过将该过滤条件转换为恒真（TRUE）或恒假（FALSE）过滤条件的方式，减少列解析，提升查询效率； 其中，优化方式 4)的具体转换规则如下： ● 当过滤条件为 ROWNUM > 常量，且常量>=1 时，将该过滤条件转换为 FALSE； ● 当过滤条件为 ROWNUM > 常量，且常量<=0 时，将该过滤条件转换为 TRUE； ● 当过滤条件为 ROWNUM >= 常量，且常量>1 时，将该过滤条件转换为 FALSE； ● 当过滤条件为 ROWNUM >= 常量，且常量<=0 时，将该过滤条件转换为 TRUE； ● 当过滤条件为 ROWNUM < 常量，且常量<=1 时，将该过滤条件转换为 FALSE； ● 当过滤条件为 ROWNUM <= 常量，且常量<1 时，将该过滤条件转换为 FALSE； ● 当过滤条件为 ROWNUM = 常量，且常量<>1 时，将该过滤条件转换为 FALSE；

			● 当过滤条件为 ROWNUM OPS NULL 时，将该过滤条件转换为 FALSE，其中 OPS 表示逻辑比较符，包括>、>=、<、<=、=、<>，例如 ROWNUM > NULL。 5)查询包含形如 C1=(SELECT ... FROM DUAL) 的过滤条件时，支持将派生表优化得到的表达式作为索引 KEY 使用，提升查询效率； 6)查询为 UNION ALL 连接查询时，忽略恒为 FALSE 的查询分支，提升查询效率； 7)过滤条件包含 ROWNUM 时，不禁止批量执行，提升查询效率； 8)根据空间索引和当前最优索引的代价相似度决定是否使用空间索引，代价相差较小情况下，优先使用空间索引； 9)将过滤条件下放到 CSCN 或 HFSCN 操作符中执行； 10)禁止过滤条件字符类型常量等值替换； 11)禁止优化器在分析阶段自动调整过滤条件顺序； 12)在分析过滤条件时，将过滤条件进行优化或者 FROM 项进行裁剪来进行分区裁剪优化； 13)DMDPC 下的 CONTAINS 查询，扫描表时始终为 CONTAINS 生成伪列输出供上层判断，优点是 CONTAINS 计算与全文索引始终在同一站点，避免远程 CONTAINS 网络开销。缺点是需要为每一行都计算 CONTAINS 而

			不能先过滤。由于较少碰见远程 CONTAINS 的情况，默认关闭； 14)利用虚拟列或者 CHECK 约束分区裁剪； 15)当二级索引与聚集索引代价差距较小，但 SIO 要小很多时，优先选择二级索引； 16)如果查询项是相关子查询，且只与嵌套循环连接的其中一张表有相关性，则尝试将相关子查询下放到嵌套循环连接之前进行； SLCT_OPT_FLAG 为 1 时开启优化方式 1)，为 2 时同时开启优化方式 1)和 2)，为 4 时开启优化方式 3)，为 8 时开启优化方式 4)，为 16 时开启优化方式 5)，为 32 时开启优化方式 6)，为 64 时开启优化方式 7)，为 128 时开启优化方式 8)，为 256 时开启优化方式 9)，为 512 时开启优化方式 10)，为 1024 时开启优化方式 11)，为 2048 时开启优化方式 12)，为 4096 时开启优化方式 13)，为 8192 时开启优化方式 14)，为 16384 时开启优化方式 15)，为 32768 时开启优化方式 16)。需要注意的是，SLCT_OPT_FLAG 参数的取值 1、2、4、8、16 在 MPP 环境下无效，取值 32、64、128、256、512、1024、2048、8192、16384、32768 在任何环境下都有效，取值 4096 仅在 DMDPC 环境下有效。 具体的，只有满足以下条件，才会进行 1)的优化： ● 没有可用的索引；

			● 需要查询的单表列在 10 个及以上； ● 过滤条件为单列，且能过滤掉 99%的数据。 只有满足以下条件，才会进行 3)的优化：查询不包含 GROUP BY、ORDER BY、TOP、HAVING、DISTINCT、分析函数以及集函数。 只有满足以下条件，才会进行 5)的优化： ● SELECT … FROM DUAL 中不包含列、查询表达式； ● 非 MPP/LPQ 环境。
UNIONALL_OPT_FLAG	1	动态， 会话级	UNIONALL 优化相关参数，支持 HINT 指定。 0：无优化； 1：当 UNION ALL 的左右孩子都不能消除排序也不能转 HPM 时，强制将 UNION ALL 之上的 ORDER 下推至 UNION ALL 的左右孩子； 2：当 UNION ALL 各分支的 FROM 项存在公因子时，尝试提取各分支 FROM 项公因子和过滤条件公因子，封装为 WITH VIEW 改写 SQL。 支持使用上述有效值的组合值，如 3 表示同时进行 1、2 的优化

DML_TTS_OPT	1	动态， 会话级	是否在查询插入时对临时表进行优化。 0：不优化； 1：优化临时表，需同时满足以下条件：插入表不是堆表或 HUGE 表；插入表与查询表不是同一张表，或者是同一张表时，既不是闪回查询也不存在 SAMPLE 项；在 MPP 环境下，为本地登录或各个站点独自执行 INSERT 操作
BATCH_PARAM_CAN_CACHE	1	动态， 系统级	是否缓存批量绑定参数优化的执行计划。 0：不缓存； 1：当 USE_PLN_POOL 不为 0 时，缓存批量绑定参数优化的执行计划
SMALL_TABLE_THRESHOLD	数据库 启动时 指定	动态， 系统级	在表扫描统计信息收集时，用以区分大表扫描和小表扫描的扫描页数的阈值。扫描页数大于该参数值时，认为进行的是大表扫描，否则认为进行的是小表扫描。取值范围 0~4194304，单位为数据页。指定为 0 时，实际取值为当前 BUFFER 数据页数的 2%。 默认值在数据库启动指定为当前 BUFFER 数据页数的 2%。 动态视图 V$SYSSTAT 的 TABLE SCANS(SHORT TABLES)和 TABLE SCANS(LONG TABLES)属性在 INI 参数 ENABLE_MONITOR=1 时分别统计小表扫描次数和大表扫描次数

以下是一些和代价计算相关的参数，除非得到达梦公司技术服务人员建议，否则不建议用户自行调整，保持默认值即可。

表2.6 dm.ini 中查询相关参数（代价计算相关）

参数名	缺省值	属性	说明
LPQ_MOTION_FACTOR	58(非 DMDPC)或者 2(DMDPC)	动态，会话级	并行及 DMDPC 环境中，计算各分发操作代价的因子。取值范围 1~1073741824
COST_PARALLEL_FACTOR	1	动态，会话级	并行及 DMDPC 环境中，计算一元操作符各分发操作代价的因子。取值范围 0~1073741824
BEXP_CALC_ST_FLAG	128	动态，会话级	针对哪些过滤条件的选择率基于动态采样调整。取值范围 0、1、2、4、8、16、32、64、128 或者上述任意取值的和 0：使用默认策略，不进行调整； 1：同一表不同列之间比较时进行调整； 2：包含函数过滤条件时进行调整； 4：同一表中 AND/OR 过滤条件进行调整； 8：对 CONTAINS 过滤条件进行调整；

			16：单表过滤条件时进行调整； 32：对 LIKE 过滤条件进行调整； 64：列与参数类型值比较的过滤条件根据动态采样调整； 128：列与参数类型值比较的过滤条件根据统计信息调整
COST_RANK_FACTOR	1000	动态，会话级	并行及 DMDPC 环境中，计算各候选分发 KEY 评分的因子。取值范围 1~1073741824
INVOCATION_OPT_FLAG	119	动态，会话级	是否进行函数调用相关优化。 0：不进行优化； 1：对 SUBSTR/SUBSTRING 函数进行优化。在满足条件时，将形如 SUBSTR(COL, 1, N)='ABC'的表达式优化为 COL>='ABC' AND COL<'ABD'。 INVOCATION_OPT_FLAG=1 时的优化存在以下两点限制： 1）表达式必须为等值表达式； 2）SUBSTR/SUBSTRING 函数的第二个参数必须为 1，第三个参数必须等于字符串的长度。 2：对 NVL(E1, E2)函数进行优化。若 E1 为 NULL，则直接使用 E2 代替 NVL 表达式，若 E1 不为 NULL 且可计算，则直接使用 E1 代替 NVL 表达式； 4：对未使用 REVERSE 函数且包含反向索引列的过滤条件表达式进行优化。对于反向索引列 COL，将形如 COL=EXP 的过滤条件表达式优化为 REVERSE(COL)=REVERSE(EXP)，以匹配反向索引；

			8：判断查询语句是否可以使用函数索引时，探测子查询，也尝试探测连接/视图的函数索引覆盖； 16：支持 CAST(MULTISET(SQL_CLAUSE) AS TYPE)函数，将其转换为 WITH FUNCTION 方式实现； 32：在满足条件时，去除集函数参数的 DISTINCT 属性。当集函数中 DISTINCT 对应的参数列具有唯一性约束时，直接去除 DISTINCT 对结果没有影响； 64：当被调用函数存在多个匹配的同名函数时，如果所有同名函数的返回类型一致，则将该被调用函数作为确定函数处理； 128：不确定性自定义函数作为过滤条件时，例如：（COL=FUNC(ARG)）其中参数为常量，在非并行、非 MPP 和 DPC 环境下，放开其作为索引 KEY 限制（即可以使用索引）； 256：计算含 DISTINCT 的 LISTAGG 或 LISTAGG2 集函数时，对只有一行或者两行数据的分组使用优化方式（不需要创建 HASH 表去重。如果是两行则直接比较两行数据是否相同并拼接结果集；如果是一行则直接作为结果输出）快速计算集函数结果； 512：对有 NVL 函数的布尔表达式做优化，可将形如 NVL(C1,CONST1) OPS CONST2 的表达式优化为 C1 OPS CONST2 (OR C1 IS NULL)，亦可将形如 NVL(?, C1) OPS D1 的表达式优化为 ? OPS D1 OR (? IS NULL AND C1 OPS D1)； 1024：将 COL OPS CASE WHEN 转换为 OR，如 COL OPS CASE WHEN B1 THEN E1 WHEN B2 THEN E2 ELSE E3 转化为(B1 AND COL OPS E1) OR (!B1 AND B2 AND COL OPS E2) OR (!B1

			AND !B2 AND COL OPS E3） 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
NBEXP_OR_CVT_IN_MAX_NUM	10000	动态，会话级	控制 OR 表达式转 IN LIST 时，单个列表最大表达式个数。取值范围 10~1000000

2.1.1.1.6 检查点相关

表2.7 dm.ini 中检查点相关参数

参数名	缺省值	属性	说明
CKPT_RLOG_SIZE	1024	动态，系统级	产生多大日志文件后做检查点，单位 MB。取值范围 0~4294967294，为 0 时表示参数不起作用
CKPT_DIRTY_PAGES	0	动态，系统级	产生多少脏页后做检查点，单位为页。取值范围 0~4294967294，为 0 时表示参数不起作用。CKPT_DIRTY_PAGES 仅当参数 FORCE_FLUSH_PAGES 不为 0 时才有效，此时调度线程启动刷脏页流程时，每个 BUFFER POOL 向磁盘写入 FORCE_FLUSH_PAGES 页数的脏页，同时检测 CKPT_DIRTY_PAGES 是否满足触发条件
CKPT_INTERVAL	180	动态，系统级	指定检查点的时间间隔。单位秒。取值范围 0~2147483647，为 0 时表示不自动定时做检查点

CKPT_FLUSH_RATE	5.00	动态，系统级	检查点刷盘比例。取值范围 0~100.00，为 0 时表示参数不起作用
CKPT_FLUSH_PAGES	1000	动态，系统级	检查点刷盘的最小页数。取值范围 1000~100000
CKPT_WAIT_PAGES	1024	动态，系统级	用于控制检查点动态调整间隔。检查点执行过程中，每发起 CKPT_WAIT_PAGES 个数据页的刷盘请求，会等待这些数据页写入磁盘完成，并触发一次检查点调整，释放日志空间。取值范围 1~4294967294
FORCE_FLUSH_PAGES	8	动态，系统级	调度线程启动刷脏页流程时，每个 BUFFER POOL 写入磁盘的脏页数。取值范围 0~1000，为 0 时表示参数不起作用
ENABLE_CKPT_EVENT_TRIG	0	动态，系统级	是否启用检查点事件触发器。0 否，1 是
CKPT_FORCE_WAIT_PAGES	1024	动态，系统级	在日志空间不足时，根据 CKPT_FORCE_WAIT_PAGES 参数控制检查点动态调整间隔。只要满足 CKPT_WAIT_PAGES 或 CKPT_FORCE_WAIT_PAGES 条件，都会触发检查点调整。取值范围 0~65534，0 表示不会根据 CKPT_FORCE_WAIT_PAGES 参数触发检查点调整

2.1.1.1.7 IO 相关

表2.8 dm.ini 中 IO 相关参数

参数名	缺省值	属性	说明
DIRECT_IO	0	静态	非 WINDOWS 下有效，0：使用 OS 文件系统缓存；1：不使用 OS 文件系统缓存，采用系统模拟的异步 IO，异步 IO 的线程数由 IO_THR_GROUP 控制；2：不使用 OS 文件系统缓存，采用系统提供的 NATIVE AIO 机制， 要求 LINUX 内核 2.6 以上，仅特定编译版本支持 注：此参数应根据应用特征谨慎设置，一般保留默认为 0 即可
IO_THR_GROUPS	8	静态	非 WINDOWS 下有效，表示 IO 线程组个数。取值范围 1~512。每个 IO 线程组包含一个读线程和一个写线程，即系统会创建 IO_THR_GROUPS 个读线程和 IO_THR_GROUPS 个写线程，IO 线程总数量为 IO_THR_GROUPS*2。 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_CPUS 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
HIO_THR_GROUPS	2	静态	HUGE 缓冲区 IO 线程组数目。

			取值范围 1~512
FAST_EXTEND_WITH_DS	LINUX/AIX/BSD 缺省为 1，其他缺省 0	动态，系统级	是否按实际磁盘占用大小扩展文件，非 WINDOWS 有效。 0：扩展文件为空洞文件； 1：按 DISK SPACE 扩展文件，文件的逻辑大小与实际磁盘占用大小一致
FIL_CHECK_INTERVAL	0	动态，系统级	指定检查数据文件是否存在的时间间隔，单位秒。取值范围 0~4294967294。0 表示不检查。仅 LINUX 有效
FIL_CHECK_MODE	0	动态，系统级	当系统检测到 SYSTEM、ROLL、TEMP 表空间或联机日志文件缺失时，是否让服务器崩掉。0：否；1：是。非 WINDOWS 有效
IO_TIMEOUT_NO_HALT	1	动态，系统级	当 IO 超时时间超过 IO_TIMEOUT 时，是否开启主动 HALT 机制。 0：开启；1：关闭

2.1.1.1.8 数据库相关

表2.9 dm.ini 中数据库相关参数

参数名	缺省值	属性	说明

MAX_SESSIONS	10000	静态	系统允许同时连接的最大会话数。取值范围 1~150000。 该参数还受到 LICENSE 的限制，取二者中较小者。 当会话数达到 MAX_SESSIONS 值后，为了保证管理员可继续管理数据库，允许 SYSDBA 额外开启 10 个会话连接 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。
MAX_CONCURRENT_TRX	0	动态， 系统级	表示系统支持同时运行事务数的最大值。取值范围 0~1500，0 表示不控制。 注：这个参数仅在需要超大数量连接时才需要设置
CONCURRENT_DELAY	16	动态， 系统级	表示新创建的连接，延迟多少秒之后参与事务限流机制。仅当 MAX_CONCURRENT_TRX 非 0 时生效。取值范围 0~1000， 单位秒。0 表示立即参与
TRX_VIEW_SIZE	512	静态	事务视图中本地事务 ID 的缓存初始化个数。取值范围 16~150000。如果配置了 MAX_CONCURRENT_TRX 参数（MAX_CONCURRENT_TRX>0），那么 TRX_VIEW_SIZE 的取值为 TRX_VIEW_SIZE、MAX_CONCURRENT_TRX 和 MAX_SESSIONS 三者中最小的值
TRX_VIEW_MODE	1	静态	控制 MVCC 事务可见性判断的方式。0：事务启动前收集当前时刻的活动事务 ID 构造可见性视图，以

			确定某一特定事务是否对当前事务可见；1：系统维护事务提交历史，以此判断事务可见性
TRX_CMTARR_SIZE	10	静态	提交状态事务数组的大小，仅当 TRX_VIEW_MODE 为 1 时有效。单位为百万，取值范围 3~1000。此参数设置越大，内存消耗越多，当应用为高并发，单位时间内事务完成数量大时，可将此参数值设大
MAX_SESSION_STATEMENT	10000	动态，系统级	单个会话上允许同时打开的语句句柄最大数，取值范围 64~20480
MAX_SESSION_MEMORY	0	动态，系统级	单个会话的运行内存上限，单位 MB，取值范围 0~262144，0 表示不限制
MAX_CONCURRENT_OLAP_QUERY	0	静态	OLAP 模式下能同时执行的复杂查询个数，超过限制后，复杂查询将在执行阶段进入等待。0 表示不做限制。取值范围 0~100 注：此处的复杂查询是指计划中存在行数超过参数 BIG_TABLE_THRESHHOLD*10000 的节点，含有哈希连接、哈希分组等耗内存操作符，且不涉及到系统表
BIG_TABLE_THRESHHOLD	1000	动态，会话级	OLAP 环境下对复杂查询最大并发数（参数 MAX_CONCURRENT_OLAP_QUERY）进行限制时，认定是否为复杂查询中表的行数下限，单位为万。取值范围 0~4294967294
MAX_EP_SITES	64	静态	MPP 环境下 EP 站点的最大数量，取值范围 2~1024

PORT_NUM	5236	静态	服务器监听通讯端口号，服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配
LISTEN_IP	空串	静态	与PORT_NUM配合使用，监听IP地址。若指定，则连接时必须指定与该值一样的IP，否则创建连接失败
FAST_LOGIN	2	静态	是否在登录时记录登录失败历史信息。0是，同步记录；1否，不记录；2是，异步记录，当同一个用户的登录失败记录数超过100，则清理前面过早的登录失败记录，避免登录相关系统表SYSACCHISTORIES膨胀。在DMDPC环境中，FAST_LOGIN取值固定为2，其他值无效
DDL_AUTO_COMMIT	1	静态	指定 DDL 语句是否自动提交，1：自动；0：手动
COMPRESS_MODE	0	动态，会话级	建表时是否缺省压缩。0：不进行；1：进行
PK_WITH_CLUSTER	0	动态，会话级	在建表语句或增加约束语句中指定主关键字时，是否缺省指定为 CLUSTER，0：不指定；1：指定 注：该参数对水平分区表、列存储表和堆表无效
EXPR_N_LEVEL	200	动态，会话级	表达式最大嵌套层数。取值范围 30~1000
N_PARSE_LEVEL	100	动态，会话级	表示对象 PROC、VIEW、PKG、CLASS 的最大解析层次，如果层次过深则报错返回。取值范围 30~1000

MAX_SQL_LEVEL	500	动态，系统级	指定 DM 虚拟机允许的最大栈帧数。取值范围 100~1000。此参数值设置过大可能会导致内存暴涨，管理员需斟酌设置
BDTA_SIZE	256	静态	BDTA 缓存的记录数。取值范围 1~10000
OLAP_FLAG	2	动态，会话级	启用联机分析处理，0：不启用；1：启用；2：不启用，同时倾向于使用索引范围扫描
JOIN_HASH_SIZE	500000	动态，会话级	HASH JOIN 操作时，HASH 表大小，以 CELL 个数为单位。取值范围 1~250000000
HFILES_OPENED	256	静态	设置可以同时打开的列存储表数据文件个数。取值范围 60~10000
ISO_IGNORE	0	动态，会话级	是否忽略显示设置的事务隔离级别，1 表示忽略，0 不忽略
TEMP_SIZE	4096*PAGE_SIZE/1024	静态	默认创建的临时表空间大小，单位 MB。取值范围 4096 个页大小~2147483647 个页大小，例如：页大小 PAGE_SIZE 为 8KB，则 TEMP_SIZE 的最小值为 4096*8/1024=32MB
TEMP_SPACE_LIMIT	0	动态，系统级	临时表空间大小上限，单位 MB。0 表示不限制临时表空间大小。取值范围 0~4294967294。

			注：TEMP_SPACE_LIMIT 一定要大于等于 TEMP_SIZE
FILE_TRACE	0	静态	日志中是否记录文件操作，0：不记录；1：记录
COMM_TRACE	0	动态， 系统级	服务器日志是否记录通信中产生的警告信息。0：不记录；1：记录
ERROR_TRACE	0	动态， 系统级	服务器日志中是否记录语法分析出错的语句。1：是，0：否
PLN_DICT_HASH_THRESHOLD	20	静态	仅当 CACHE_POOL_SIZE 参数值大于等于此参数值时，才开启缓存计划的字典关联登记。取值范围： 32 位平台下为 1~2048；64 位平台下为 1~67108864。单位 MB
STAT_COLLECT_SIZE	10000	动态， 会话级	统计信息收集时，样本的最小行数。取值范围 0~10000000
STAT_ALL	0	动态， 会话级	在估算分区表行数时，控制一些优化。0：不采样所有分区子表；1：采样所有分区子表；2：优先采用 统计信息中的行数；4：水平分区表不使用主表统计信息而使用叶子表统计信息。 支持使用上述有效值的组合值，如 3 表示优先采用统计信息中的行数，如果没有找到对应统计信息，则 去采样所有分区子表。

PHC_MODE_ENFORCE	0	动态， 会话级	控制连接的实现方式。 0：优化器根据代价情况自由选择连接方式； 1：允许使用 NEST LOOP INNER JOIN； 2：允许使用索引连接； 4：允许使用哈希连接； 8：允许使用归并连接 支持使用上述有效值的组合值，如 6 表示优化器根据代价情况在索引连接和哈希连接间进行选择。 当参数值不为 0 或 15 且包含 2/4/8 其中之一或者它们的组合值时，会将包含的值对应的连接模式参数置为 1，没有包含的值对应的连接模式参数置为 0（2 对应 ENABLE_INDEX_JOIN，4 对应 ENABLE_HASH_JOIN，8 对应 ENABLE_MERGE_JOIN）。例如，取值为 6 时没有包含 8，则会将 ENABLE_INDEX_JOIN 和 ENABLE_HASH_JOIN 置为 1，将 ENABLE_MERGE_JOIN 置为 0（仅强制设置内存中的值，不改变 dm.ini 文件中的值）
ENABLE_HASH_JOIN	1	动态， 会话级	是否允许使用哈希连接，0：不允许；1：允许。

ENABLE_INDEX_JOIN	1	动态，会话级	是否允许使用索引连接，0：不允许；1：允许。
ENABLE_MERGE_JOIN	1	动态，会话级	是否允许使用归并连接，0：不允许；1：允许
MPP_INDEX_JOIN_OPT_FLAG	1	动态，会话级	MPP 下，是否可以使用索引左外连接，0：不使用；1：使用
MPP_NLI_OPT_FLAG	1	动态，会话级	MPP 下嵌套连接的优化方式。 0：嵌套连接操作的左右孩子添加 MPP GATHER，汇总到主 EP 执行， 父节点添加 MPP SCATTER； 1：根据左右孩子的实际情况添加通讯操作符，若当前情况为左孩上的数据为 PART（分布在每个站点上的数据都是部分的），右孩子上的数据为 FULL（数据在每个站点上分布都是完全的），则不添加；若两个孩子均为 PART，则在数据量少的那个孩子上添加 MPP GATHER 和 MPP SCATTER； 2：根据左右孩子的实际情况添加通讯操作符，若当前情况为右孩子为 PART，左孩子为 FULL 则不添加；若两个孩子均为 PART，则在数据量少的那个孩子上添加 MPP GATHER 和 MPP SCATTER； 3：嵌套连接操作的左右孩子都添加 MPP GATHER 和 MPP SCATTER

MAX_PARALLEL_DEGREE	1	动态，会话级	用于设置并行度（即最大并行线程数）。取值范围：1~128，同时要小于等于 PARALLEL_THRD_NUM。 1 表示不并行
PARALLEL_POLICY	0	动态，会话级	用来设置并行策略。取值范围：0、1 和 2。 0：关闭并行； 1：开启自动并行模式，使用 INI 参数 MAX_PARALLEL_DEGREE 作为并行度。如果同时使用了 HINT 参数 PARALLEL 指定并行度，则 HINT 优先； 2：开启手动并行模式，使用 HINT 参数 PARALLEL 指定并行度，不指定表示不并行。在 DMDPC 环境下，取值 2 等同于取值 1。 不支持 HINT 方式指定该参数
PARALLEL_THRD_NUM	10	静态	用来设置并行工作线程个数。取值范围 1~65535
PARALLEL_THRD_TARGET	0	动态，系统级	用于设置并行线程池中保留的线程数上限。取值范围 0~10240，0 表示不设限，系统将根据压力自动控制空闲线程数
PARALLEL_MODE_COMMON_DEGREE	1	动态，会话级	并发模式程度。此值越大，扫描代价将越大，越倾向于使用索引连接而不是哈希连接，需酌情使用。取值范围 1~1024
PUSH_SUBQ	0	动态，会话级	是否下放子查询使先做。0：始终不下放；1：相关子查询且不存在单表过滤条件时下放；2：相关子查询始终下放；4：非相关子查询下放。支持使用上述有效值的组合值，如 3 表示同时进行 1、2

OPTIMIZER_AGGR_GROUPBY_ELIM	1	动态， 会话级	当对派生视图进行分组查询，且分组项是派生视图分组项的子集时，是否允许将两层分组进行合并。 0：不优化；1：将两层分组进行合并
UPD_TAB_INFO	0	静态	系统启动时是否更新表信息（如行数），1：启用更新；0：不更新
ENABLE_IN_VALUE_LIST_OPT	518	动态， 会话级	是否允许 IN LIST 表达式优化。 0：不优化； 1：将 IN LIST 表达式在语义分析阶段优化为 CONST VALUE LIST； 2：将 IN LIST 表达式在代价优化阶段优化为 CONST VALUE LIST； 4：生成传递闭包优化； 16：OR 表达式优化为 LIST IN LIST 表达式时，允许 LIST 中的列来源于不同的表； 32：允许 IN LIST 表达式中的列为分区列时转化为 SEMI JOIN； 64：当 IN LIST 表达式左侧包含多列，并且包含非列类型表达式时，允许将其优化为 CONST VALUE LIST； 128：分析阶段将 IN VALUE LIST 表达式转换为 OR 表达式； 256：使用索引连接时，当连接条件中有多个 IN 表达式时，多个 IN 表达式会先做连接。取该值时，不允

			许分析阶段 IN 连接后的行数超过一万行； 512：语义分析阶段将多列 NOT IN 转换为多个类型相同值的比较，出现 NULL 时认为相同，预期走批量计算，提升执行效率；目前仅支持 NOT IN 为常量值链表时进行转换； 1024：在取值 1 关闭情况下，将表连接(JOIN)的 IN LIST 表达式在语义分析阶段优化为 CONST VALUE LIST； 2048：扩大 IN LIST 作为 JOIN CONDITION 的代价，缩减其转换为 CONST VALUE LIST 的代价； 4096：增强 OR 表达式转换为 IN 表达式（或者 AND 表达式转换为 NOT IN 表达式）的优化，将转换后的 IN 表达式（IN LIST 或 LIST IN LIST）按数据类型（仅限数值和字符类型）进行拆分，避免转换后的不等价引发的报错或结果错误；该增强优化仅在 OR 表达式转换为 IN 表达式（或者 AND 表达式转换为 NOT IN 表达式）的优化生效后有效； 8192：禁止在 PHA 和 PHB 阶段 OR 转换为 IN LST； 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
ENHANCED_BEXP_TRANS_GEN	3	动态， 会话级	0：不允许非等值布尔表达式和外连接 ON 条件中的传递闭包；1：允许非等值布尔表达式和外连接 ON 条件中的传递闭包；2：分析阶段针对表达式通过传递闭包产生的表达式进行去重处理；4：允许将 IN

			LIST 过滤条件传递到非相关子查询内部 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
ENABLE_DIST_VIEW_UPDATE	0	动态， 系统级	是否支持更新含有 DISTINCT 的视图，1：允许；0：不允许，当 COMPATIBLE_MODE=1 时， ENABLE_DIST_VIEW_UPDATE 的实际值均为 1
STAR_TRANSFORMATION_ENABLED	0	动态， 会话级	是否允许对星形模型查询改写以使用位图连接索引。0：不启用；1：启用 此参数仅在 OLAP_FLAG 为 1 时才有效
MONITOR_INDEX_FLAG	0	动态， 会话级	是否对索引进行监控，0：关闭自动监控，可使用 ALTER INDEX 语句启用索引监控；1：打开自动监控， 对用户定义的二级索引进行监控；2：禁止索引监控
AUTO_COMPILE_FLAG	1	动态， 系统级	用于加载失效对象时是否自动编译，0：否；1：是
RAISE_CASE_NOT_FOUND	0	动态， 系统级	CASE 语句 NOT FOUND 是否抛出异常。0：不抛出；1：抛出
FIRST_ROWS	100	动态， 系统级	结果集一个消息中返回的最大行数，取值范围 1~1000000

LIST_TABLE	0	动态，会话级	默认情况下，创建的表是否为堆表，0：否；1：是
ENABLE_SPACELIMIT_CHECK	1	静态	是否启用创建表时定义的空间限制 DISKSPACE LIMIT 功能。1：启用；0：不启用
BUILD_VERTICAL_PK_BTREE	0	静态	HUGE表上创建的主键是否需要创建物理B树。0：不创建；1：创建
BDTA_PACKAGE_COMPRESS	0	动态，会话级	是否启动bdta压缩传递功能， 0：不压缩；1：压缩；2：根据数据特征自适应选择是否压缩，取值2仅限在DMDPC环境下使用
HFINS_PARALLEL_FLAG	0	动态，系统级	是否启动HUGE表查询插入优化。0：否；1：对非HUGE分区表进行优化；2：对HUGE分区表也进行优化
HFINS_MAX_THRD_NUM	100	动态，系统级	HUGE表查询插入优化时并行的最大线程数，取值范围4~200
LINK_CONN_KEEP_TIME	15	静态	DBLINK空闲连接缓存存在时间，单位分钟，为0时表示不缓存DBLINK连接，取值范围0~2147483647。建议在频繁使用有限的几个DBLINK的时候开启该参数并设置较长时间，以便重用连接；在使用的大部分是不同的DBLINK时，单个DBLINK被重用的几率很小，此时可以考虑不设置该参数
MAX_LINK_SESSIONS	10000	静态	用于设置DBLINK连接会话数的上限值，取值范围1~150000。当DBLINK连接会话数达到该上限值时，服务

			器将释放空闲的DBLINK连接，再新建连接会话；若无空闲连接，则拒绝建立新连接
DETERMIN_CACHE_SIZE	5	动态， 系统级	确定性函数值缓冲区大小，单位MB，取值范围1~1000
NTYPE_MAX_OBJ_NUM	1000000	动态， 系统级	复合数据类型中包含的对象或者字符串的总个数，以及查询中包含的变量的总个数，取值范围为 2000~100000000
CTAB_SEL_WITH_CONS	0	动态， 会话级	查询建表时，是否对原始表上的相关信息进行拷贝。0：否（若源列是显式指定了 NOT NULL 约束的非主键列，或者源列是自增列，则拷贝源列的非空信息）；1：对原始表上的约束进行拷贝；2：对原始表上的表结构、分区信息、存储信息以及部分表约束进行拷贝。 其中，CTAB_SEL_WITH_CONS=1 时，不支持 HUGE 表
HLDR_BUF_SIZE	8	动态， 系统级	进行 HUGE 表导入时，缓冲区的大小，单位 MB，取值范围 4~1024
HLDR_BUF_TOTAL_SIZE	4294967294	动态， 系统级	HLDR 资源控制，系统所有 HLDR 使用的 HLDR_BUF 空间的总量，单位 MB。取值范围 100~4294967294
HLDR_REPAIR_FLAG	0	动态，	使用dmfldr装载数据时如发生错误，对错误的处理方式：0：完全回滚方式，数据将回到装载前的状

		系统级	态；1：修复方式，系统将数据修复到最后一个完整装载的数据区。此外，若此参数为1，dmfldr装载数据时不支持对事务型HUGE表二级索引进行维护
HLDR_FORCE_COLUMN_STORAGE	1	动态，系统级	装载最后一个数据区是否强制列存。1：是；0：否
HLDR_FORCE_COLUMN_STORAGE_PERCENT	80	动态，系统级	装载 HUGE 表时最后一个数据区强制列存的最低数据百分比。取值范围 50~100
HLDR_HOLD_RATE	1.5	动态，系统级	HLDR 最多保持重用 HLDR_BUF 的个数与目标表列数的比例。取值范围 1~65535
HLDR_MAX_RATE	2	动态，系统级	HLDR 同时使用的 HLDR_BUF 的最大个数与目标表列数的比例。取值范围 2~65535
HUGE_ACID	0	动态，系统级	是否支持 HUGE 表增删改与查询的并发，取值范围：0、1、2。 0：不支持 HUGE 表增删改与查询的并发； 1：支持 HUGE 表增删改与查询的并发，操作符 HFLKUP 检查数据区是否上锁时使用二分查找； 2：支持 HUGE 表增删改与查询的并发，操作符 HFLKUP 检查数据区是否上锁时使用遍历查找

HUGE_STAT_MODE	2	动态，会话级	设置默认情况下是否计算 WITH DELTA 的 HUGE 表各列的统计信息，取值范围：0、1、2。 0：不计算统计信息； 1：实时计算统计信息； 2：异步计算统计信息
HFS_CHECK_SUM	1	静态	是否为 HUGE 表数据区进行和校验：0：否；1：是
HBUF_DATA_MODE	0	静态	HUGE 表数据缓冲区中缓存数据的格式，取值范围：0、1，缺省值为 0 0：格式与数据文件中的数据块格式一致 1：当数据块是压缩加密时，缓存解密解压后的数据
DBLINK_OPT_FLAG	509	动态，会话级	控制对 DBLINK 的优化。 0：不进行优化； 1：允许 DBLINK 整体优化，采用原始方式； 2：DBLINK 局部优化，采用老方式； 4：DBLINK 局部优化，采用新方式； 8：异构数据库情况下进行兼容性检查，根据表达式类型检查是否可以在异构数据库执行，如果不支

			持，则不对此表达式进行进行收集； 16：采用新方式局部优化时，考虑视图、派生表、子查询、查询表达式是否可以整体优化； 32：采用新方式局部优化时，支持外连接 WHERE 过滤条件的优化； 64：支持 MEMBER_ACCESS 转换为变量的优化方式； 128：增删改支持新方式优化； 256：计划生成阶段增加 DBLINK 优化，支持连接条件转换为变量的优化方式，并对 DBLINK 存在过滤条件的场景进行优化； 512：支持物化视图明细表是基于 DM 到异构数据库 DBLINK 的远程表； 1024：若查询包含 DBLINK，则将 LIKE 表达式的数据类型强制转为 VARCHAR(4000)，防止查询发给 ORACLE 后字符串超长报错； 2048：DBLINK 根据远端的统计信息进行代价计算； 4096：在多次执行的 REMOTE SCAN 操作符前插入 NTTS2 操作符，使用临时表接收远程数据，减少 REMOTE SCAN 执行次数； 8192：如果语句的查询表和过滤条件都来自同一远程连接，将其优化为一张派生表；

			16384:内连接中属于同一个远程连接的多张远程表可以优化为一张派生表，发送到远端执行。 支持使用上述有效值的组合值，如 61 表示同时进行 1、4、8、16、32 的优化。取值 16、32 和 16384 只有和取值 4 组合才有意义
DBLINK_LOB_LEN	8	动态， 系统级	使用 DBLINK 访问大字段类型时，大字段的长度限制。单位 KB，取值范围 4~1024
FILL_COL_DESC_FLAG	0	动态， 系统级	服务器返回给客户端的结果集是否含有列描述信息，0：否；1：是
BTR_SPLIT_MODE	1	动态， 系统级	B 树叶子节点分裂方式，取值范围：0~15，按位标识： 0：按对半分裂，B 树插入满一页后从当前页的中间行分裂成两个页，然后重新定位插入； 1：在插入点进行分裂，插满一页后重新分配一个新页插入新数据； 2：二级索引开启 B 树插入封锁优化，B 树分裂时先尝试 X 封锁叶子页的父亲页，而不是直接 X 封锁整棵 B 树； 4：包含索引键值的聚集索引开启 B 树插入封锁优化，B 树分裂时先尝试 X 封锁叶子页的父亲页，而不是直接 X 封锁整棵 B 树；

			8：不包含索引键值的聚集索引开启 B 树插入封锁优化，B 树分裂时先尝试 X 封锁叶子页的父亲页，而不是直接 X 封锁整棵 B 树。 支持使用上述有效值的组合值，如 3 表示同时进行 1、2
TS_RESERVED_EXTENTS	64	静态	系统为每个表空间提前预留的簇个数，减少系统在执行过程中申请不到空间的情况。取值范围 2 ~ 1024。 注：DMDSC 环境下，该参数所有站点需要保持一致
TS_SAFE_FREE_EXTENTS	512	静态	系统认为安全的 FREE EXTENTS 空间，取值范围 128~65534。 注：DMDSC 环境下，该参数所有站点需要保持一致
TS_MAX_ID	8192	静态	限制系统所支持的最大表空间 ID，取值范围 512~65517， 与系统实际最大 ID 比较，取大。 如果设置为 590，但是系统最大 ID 已经为 1000 了，最后结果是 1000。 注：DMDSC 环境下，该参数所有站点需要保持一致
TS_FIL_MAX_ID	2048	静态	限制每个表空间所支持的最大文件个数，取值范围 64~8191 注：DMDSC 环境下，该参数所有站点需要保持一致

DECIMAL_FIX_STORAGE	0	动态，会话级	是否将长度为 1~18 的 DECIMAL/DEC 类型转换为定长方式存储。0：否；1：是。 若转换为定长存储，则 DECIMAL(P,S)的转换规则为：当 0<P<10 时，存储为 4 字节整型值；当 10<=P<19 时，存储为 8 字节整型值
SAVEPOINT_LIMIT	512	动态，系统级	单个事务中最大可创建的保存点个数。取值范围 0~10000
SQL_SAFE_UPDATE_ROWS	0	动态，系统级	SQL 安全更新控制参数。取值范围：0：关闭 SQL 安全更新；1~2147483647：启用 SQL 安全更新，SQL 语句每次允许影响行数的最大值为指定的参数值
ENABLE_HUGE_SECIND	1	动态，系统级	是否支持事务型 HUGE 表二级索引的维护，0：不支持；1：支持
LOB_MAX_INROW_LEN	900	动态，系统级	指定数据库 LOB 字段的行内数据存放的上限大小，单位为 BYTE，取值范围 900~10240
RS_PRE_FETCH	0	动态，系统级	查询操作结果集返回时是否进行预填充，仅适用于 FORWORD_ONLY 游标的结果集。0：不预填充；1：预填充，即在一批数据返回客户端后会自动进行预取下一批数据；2：在开启消息压缩时，提前压缩结果集，提升消息响应速度

TIMER_TRIG_CHECK_INTERVAL	60	动态，系统级	服务器端检查触发器是否触发的时间间隔，单位为秒，即每过设置的秒数检查触发器是否需要触发，取值范围 1~60
INNER_INDEX_DDL_SHOW	1	动态，会话级	用于控制 DBMS_METADATA 包查询索引的结果。 0：只显示用户显式创建的索引，不显示系统创建的内部索引； 1：显示系统创建的内部索引
HP_STAT_SAMPLE_COUNT	50	动态，系统级	水平分区表采集统计信息时，子表的最大采样数，取值范围 1~65535
MAX_SEC_INDEX_SIZE	16384	静态	控制二级索引记录的最大长度。取值范围 512~32768，单位 BYTE
PWR_FLUSH_PAGES	10000	动态，系统级	控制生成特殊 PWR 记录的频率，每写入 PWR_FLUSH_PAGES 页到磁盘，即生成一条特殊的 PWR 记录，标记之前的 PWR 记录是有效的。取值范围 0~4294967294
REDO_UNTIL_LSN	空串	静态	系统故障重启时，重做 REDO 日志的最大 LSN 值。 使用这个参数要十分慎重，一般只在数据库文件系统损坏，系统无法正常启动情况下使用，通过指定这个参数，将数据库恢复到稍早时间点，让数据库可以正常启动，以便抢救部分数据。 注：使用这个参数后，会截断指定 LSN 之后的所有日志。另外，系统启动后，一定不要忘记将参数值重

			新修改为缺省值
IGNORE_FILE_SYS_CHECK	1	静态	系统重启时，是否检查 SYSTEM/ROLL/MAIN 表空间文件系统。0：检查文件系统；1：不检查文件系统，在实际使用空间比较大情况下，可以考虑关闭文件系统检查，提高系统启动速度；2：检查文件系统，同时检查表空间全局空闲链表是否正常，若异常则截断该空闲链表
FILE_SCAN_PERCENT	100.0	动态，系统级	V$DATAFILE 查询时控制抽样比例，最多抽样 10000 页； 在 50 页以下的不抽样。取值范围 0~100.0，0 相当于 100
STARTUP_CHECKPOINT	0	静态	系统启动时 REDO 后是否强制做完全检查点。 0：不强制；1：强制
CHECK_SVR_VERSION	1	静态	数据库记录的执行码版本比当前 DMServer 版本高时，是否报错。0：不报错；1：报错，服务器不能启动成功
ID_RECYCLE_FLAG	0	静态	是否开启 ID 资源回收功能。 0：不开启。 1：仅回收删除对象的 ID。 2：在 1 的基础上，额外回收分析执行阶段已申请但未被使用且符合内部回收条件的 ID

TRXID_UNCOVERED	0	动态， 会话级	是否禁止二级索引覆盖。0：否，使用二级索引覆盖，当更新不涉及二级索引时，查询最大 TRXID 可能比聚集索引的最大 TRXID 小；1：是
GEN_SQL_MEM_RECLAIM	1	动态， 系统级	语句块执行是否提前回收空间。0：否；1：是，语句块生成各个 SQL 计划后，都需尝试回收部分 PHA 内存空间
ENABLE_PMEM	0	静态	Linux 系统中，是否使用 Intel 提供的动态库（libpmem.so）读写存储在持久性内存上的联机日志文件。libpmem.so 由 INTEL 公司提供，可通过安装 INTEL 硬件持久性内存获取。 0：否，使用系统 IO 接口读写； 1：是，安装了非易失性存储介质并且文件存储于持久性内存才使用 Intel 动态库读写，否则仍然使用系统 IO 接口读写。此种情况不支持 RESIZE LOGFILE 语句和 RENAME LOGFILE 语句
HP_TAB_COUNT_PER_BP	1	动态， 会话级	DMDPC 环境下专用，创建一级、二级哈希分区时，如果分区数指定为 DEFAULT 而不是具体数值，系统会根据本参数值决定每个 BP RAFT 上创建的哈希子表个数。 取值范围 1~1024
SQC_GI_NUM_PER_TAB	1	动态， 会话级	每个表拆分的扫描单元个数。取值范围 1~1024。 扫描表对象时，每个表可以被拆分成多个单元以提升并行性能。参数值表示拆分的单元个数，每个单元

			可以独立用于工作线程的扫描任务。例如：需要扫描一个子表且子表数据量较大，为充分利用工作线程资源，可以调大此参数以使用分区内并行 在非 DMDPC 环境下，若该参数值在 DM.INI 文件中设置为 1（包括缺省情况），系统将会在数据库启动时自动调整为 2，可以在数据库启动后重新手动调整回 1
DBLINK_USER_AS_SCHEMA	1	动态， 会话级	使用 DPI，ODBC、ORACLE 类型的外部链接专用。是否将当前用户名作为远程对象的缺省模式名（或库名）。0：否，匹配第一个满足条件的远程对象；1：是，匹配缺省模式下的远程对象。该参数配置在创建外部链接的服务器上，而非远程对象所在服务器
CTAB_MUST_PART	0	动态， 系统级	用户表是否只能创建为分区表。0：否；1：是
CTAB_WITH_LONG_ROW	0	动态， 系统级	是否尝试在建表时隐式指定 USING LONG ROW 来开启超长记录存储功能。0：否；1：是。 当该参数为 1 时，只有满足附加条件的情况下才会成功开启超长记录存储功能，未满足条件的不能开启。 附加条件： 1. 非 HUGE 表、临时表和外部表

			2. 建表不包含聚集主键约束 3. 如果是分区表，则不包含 ENABLE ROW MOVEMENT 子句 4. 整行记录总长度超长。长度上限可参考《DM8_SQL 语言使用手册》中表 1.4.1。 当 CTAB_WITH_LONG_ROW=1 时，可通过 DISABLE USING LONG ROW 关闭隐式开启的超长记录存储功能
TMP_DEL_OPT	1	动态， 系统级	是否对临时表删除进行优化。0：不优化；1：如果不要求临时表可回滚，则对于无过滤条件的临时表删除语句，在执行删除时直接清除其临时段数据
LOGIN_FAIL_TRIGGER	0	动态， 系统级	登录失败时是否触发 LOGIN/LOGON 触发器。0：不触发；1：触发
INDEX_PARALLEL_FILL	0	动态， 系统级	控制查询插入时二级索引是否启用并行填充。如果表上存在多个二级索引并且没有全文索引，在插入操作符执行时，一旦检测到本次填充的记录行数大于等于此参数指定值，则为每一个二级索引生成一个异步索引填充任务，由任务线程并行执行，以提升查询插入的执行效率。取值范围 0~10000，0 表示关闭并行填充
CIND_CHECK_DUP	0	动态，	指定创建索引时检查索引冗余的方式，以及建表时是否检查重复的 UNIQUE 约束。

		系统级	0：创建索引时已存在同类型且相同索引列的索引则报错，建表时不检查重复的 UNIQUE 约束 1：创建索引时已存在相同索引列的索引则报错，建表时检查重复的 UNIQUE 约束
TSMV_RAFIL_SIZE	64	静态	DMDPC 环境下专用。指定表空间迁移的临时归档文件的最大空间。取值范围 2~1024。单位 MB
TSMV_FILE_COPY_PLL	1	动态， 系统级	DMDPC 环境下专用。表空间迁移时物理文件拷贝并行度。取值范围 1~128
TSMV_WAIT_TIMEOUT	120	动态， 会话级	DMDPC 环境下专用。表空间中表对象封锁超时的时间。取值范围 0~12000，单位秒
TSMV_SEND_BUF_SIZE	4	动态， 系统级	DMDPC 环境下专用。表空间迁移发送文件缓冲区大小。单位 MB，取值范围 1~64。值越小，迁移越慢，但迁移中对并发业务影响越小；值越大迁移速度越快，但迁移中对并发业务影响越大，可结合实际情况设置
HUGE_ENABLE_DEL_UPD	1	动态， 系统级	是否允许对 HUGE 表数据进行更新或删除。0：不允许更新或删除；1：允许更新或删除；2：不允许更新；3：不允许删除
HUGE_DEFAULT_FILE_SIZE	64	动态， 系统级	创建 HUGE 表时，STORAGE 子句中 FILESIZE 的默认值，取值范围 16~1048576

HUGE_DEFAULT_SECTION_SIZE	65536	动态， 系统级	创建 HUGE 表时，STORAGE 子句中 SECTION 的默认值，取值范围 1024~1048576
STAT_OPT_FLAG	0	动态， 会话级	生成统计信息的优化。0：不优化；1：生成统计信息时不存储左孩子的数据，以在某些场景表实现性能的提升
ALLOWED_CLIENT_VERSIONS	空串	静态	控制可连接服务器的客户端版本，最多可配置 10 个支持的版本号，用逗号分隔(不带空格)。默认值空串表示支持所有版本的连接，但参数 IFUN_DATETIME_MODE=1 时可能存在版本连接限制
DPC_TABLESPACE_BALANCE	1	动态， 系统级	DMDPC 环境下专用，当在 DMDPC 下创建分区表，子表个数少于所选择的表空间个数时，是否启用随机选择起始表空间以使得表空间负载均衡。0：不启用；1：启用，从已有子表所使用的表空间中挑选；2：启用，在一级水平分区表新增子表时考虑从所有表空间中挑选
DPC_DCT_REFRESH_POLICY	1	动态， 系统级	DMDPC 环境下专用，MP 广播实例字典信息优化开关。 0：表示关闭，MP 会定时广播实例字典信息； 1：表示打开，MP 只会在每次修改实例字典信息时进行广播
HANDLE_WARN_RATE	75	动态， 系统级	dmserver 进程使用句柄数报警百分比，超过设定值则写日志警告信息，取值范围 0~100

ENET_SESS_CHECK_INTERVAL	10	动态，系统级	DMDPC 环境下专用，当 BP/SP 处于 MP 无效服务状态时，BP/SP 断开会话连接的时间间隔阈值。如果 BP/SP 处于 MP 无效服务状态的时间超过该阈值，则会主动断开所有会话连接。取值范围 0~4294967294，单位秒。0 表示关闭
CYT_CHECK_FLAG	0	静态	是否检查校验加解密过程。0：不检查；1：检查
HUGE_DEFAULT_FILE_INIT_SIZE	1	动态，会话级	指定 HUGE 表数据文件的初始化时的默认大小，单位 MB，取值范围 0~1048576。0 表示关闭，即 HUGE 表数据文件创建时就使用其文件大小创建；非 0 时有效值是 2 的 N 次幂，会自动进行对齐
MAX_UNCOMPRESS_RATIO	100	动态，系统级	UTL_COMPRESS 包中解压相关方法支持的最大解压比例，取值范围 10~1000
MAX_UNCOMPRESS_LENGTH	1024	动态，系统级	UTL_COMPRESS 包中解压相关方法支持的解压后数据的最大长度，单位 MB，取值范围 32~102400
TS_FIL_USE_ABSOLUTE_PATH	0	静态	表空间添加、重命名数据文件是否必须使用绝对路径。0：不强制使用绝对路径；1：必须使用绝对路径
LIST_TABLE_BRANCH	0	动态，系统级	创建堆表的并发分支数，取值范围 0~64。当 LIST_TABLE 开启时参数配置生效
LIST_TABLE_NON_BRANCH	0	动态，	创建堆表的非并发分支数，取值范围 0~64。当 LIST_TABLE 开启时参数配置生效。为 0 时采用系统默

		系统级	认值 1
DHASH3_MAX_CONFLICT	4	动态， 会话级	动态哈希扩展中允许每个槽上的最大冲突数，取值范围 1~100
DHASH3_SIZE_EXTEND_FACTOR	5	动态， 会话级	动态哈希扩展中指定的扩展因子，取值范围 1~100
DB_FILE_NAME_CONVERT	空串	静态	指定还原数据库或备库重演日志时表空间数据文件路径的转换规则。参数值必须为偶数个字符串，以逗号为分隔符。每两个字符串分别表示源库数据文件路径中的待替换字符串和替换后的字符串（PATTERN/REPLACE）。替换后的路径长度不能超过 256 个字节，否则认为当前 PATTERN/REPLACE 无效，匹配下一对 PATTERN/REPLACE。进行字符串替换时，只替换匹配成功的第一对 PATTERN/REPLACE，后续 PATTERN/REPLACE 不再进行替换。未找到匹配项则不进行字符串替换，可能导致重演或还原失败。参数为空串时不进行路径转换，按照 REDOS_FILE_PATH_POLICY 参数规则或者默认规则进行重演或还原
DPC_GUP_SESS_TIMEOUT	180	动态， 会话级	DMDPC 环境下专用，表示 SP 升级时事务、会话超时断开时间。取值范围 60~600。单位秒

TS_AUTO_EXTEND_SIZE	64	静态	表空间自动扩展大小，取值范围 1~1024，单位 MB
INI_SYNC	1	动态， 系统级	用于控制备库是否定期从主库同步 INI 参数及每次同步的时间间隔；取值范围 0~1440，单位分钟；0 表示不开启同步。该参数也用于控制进行主备切换时，新主库是否向备库强制同步 INI 参数，0 表示不强制进行同步
TRUNC_CHECK_MODE	0	动态， 系统级	语句检查并发 TRUNCATE 的方式。 0：分区表根据对应根表的数据版本检查； 1：分区子表各自检查； 2：索引局部进行检查。 注：DM MPP 环境不支持该参数
PAGE_CHECK_POLICY	1	动态， 系统级	建库参数 PAGE_CHECK 非 0 时，若数据页校验失败，对于 INI 参数 PTX_ROLLBACK 为 0 或 PTX_ROLLBACK 为 1 但无法报错处理的情况的处理策略。0：忽略失败；1：系统主动 HALT
FAST_START_MTTR_TARGET	0	动态， 系统级	数据库故障恢复时间，单位秒，取值范围 0~3600，缺省为 0，表示不启用。设置该参数后，系统内部会根据未刷盘日志大小及脏页数量，动态调整 RLOG_RESERVE_THRESHOLD 值 可通过 SP_SET_PARA_VALUE 系统过程设置（最终会转换为 RLOG_RESERVE_THRESHOLD 值）

CSEK2_CHECK_LOCK	0	动态， 系统级	对于简单计划（对主键查询的更新与删除），是否需要在检索时锁行。 0：表示关闭； 1：表示尝试在查询期间提前封锁； 2：表示尝试在查询期间提前修改
CSCN2_CHECK_LOCK	0	动态， 系统级	该参数按位取值，有两层含义 0：表示关闭 1：DSC 集群简单计划（全表更新/删除）在扫描时使用 NBTR_MODIFY_LEAF 模式，借助 PRELOAD 机制提前获取数据页 X 封锁，提升 LBS 封锁效率 2：对于简单计划（全表更新/删除）在扫描时提前修改记录 TID 3：同时启用 1 和 2 功能
STAT_CACHE_FLAG	1	静态	DMDPC 环境下专用，SP 是否缓存表行数。 0：不缓存； 1：SP 缓存表行数，而不是每次都向 MP 请求表行数，从本地缓存中获取表行数以降低 MP 的性能瓶颈。和 STAT_CACHE_CAPACITY 搭配使用

DEFAULT_FILLFACTOR	0	动态， 系统级	设置填充比例的默认值，包括所有可以设置 STORAGE 参数的模块如表、索引、物化视图等。指定插入数据时数据页的充满程度，取值范围 0~100。取缺省值 0 时，等价于 100，表示全满填充，未充满的空间可供页内的数据更新时使用
SHADOW_CHECK_INTERVAL	60	动态， 系统级	用于设置清理影子库的本地归档文件的时间间隔，取值范围 0~3600，单位秒。如果设为 0，则关闭自动清理功能。此参数仅对影子库有用
WM_CONCAT_LOB	1	静态	指定 WM_CONCAT 函数的返回值类型。0：VARCHAR 类型；1：CLOB 类型
CHECK_CONS_NAME	1	动态， 系统级	在创建或重命名约束时，是否对约束名合法性进行检查。0：不检查；1：检查
IFUN_LEN_OPT	3	动态， 系统级	设置如何调整函数返回字符串结果的长度。0：不调整；1：优化部分函数返回字符串结果的最大长度；2：溢出时不报错，截断后返回，取值 2 仅在 COMPATIBLE_MODE 为 2 时生效。 取值为 1 时可以进行优化的函数有： TO_CHAR()、CONCAT()、INITCAP()、REVERSE()、LPAD()、RPAD()、SUBSTRB()、SOUNDEX()、TRANSLATE()、ASCIISTR()、RAWTOHEX()、UNISTR()、NLSSORT()、TO_SINGLE_BYTE()、TO_MULTI_BYTE()、COMPOSE()

			支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的操作。
PKG_BUILD_WAIT_TIME	5	动态， 系统级	等待包对象构建的最长等待时间，取值范围 1~604800，单位秒
PKG_BUILD_WAIT_POLICY	1	动态， 系统级	包编译等待策略 0：总是超时等待 1：对于可能死锁的并发编译串行处理
PTX_ROLLBACK	0	静态	针对磁盘 IO 读请求超时、DMDSC 环境下 LBS 请求 GBS 授权超时场景的处理策略。0：故障退出；1：正常报错。 针对访问数据页损坏的场景（建库参数 PAGE_CHECK 非零可以校验数据页损坏）的处理策略。0：此时处理策略取决于 INI 参数 PAGE_CHECK_POLICY；1：绝大部分情况可以正常报错，当损坏的数据页是文件 0 号页、系统表空间数据页、文件系统 INODE 页或者描述页等时，无法正常报错，此时的处理策略取决于 INI 参数 PAGE_CHECK_POLICY
VM_PINST_COUNT	10	动态， 系统级	可以动态配置栈帧中缓存实例上限，取值范围为 0~20，设置为 0 为不打开。参数打开后，可以缓存最近访问的部分包的实例地址，用于提升后续访问速度

INSTRUCT_COMBINE	0	动态， 系统级	是否尝试进行特定形式表达式的合并指令的生成（例如：将 A.B/A(I).B/A(I)形式的本地变量的读写，改为单独的 DOP_LOAD_SPA/DOP_STORE_SPA 指令执行）；0：否，1：是
MAX_ESESSIONS	100000	动态， 系统级	DMDPC 环境下专用，指定分支 ESESSION 个数上限，取值范围 0~650000，0 表示无限制。ESESSION 为 SP 和其他节点之间的内部会话，SP 上的 ESESSION 为主 ESESSION，其他节点上的 ESESSION 为分支 ESESSION
DEFAULT_HASHPARTMAP	1	动态， 系统级	创建哈希分区表未指定 HASHPARTMAP 选项时的默认值。取值范围 0~3，0 表示 DM 原有的数据定位方式，此时数据分布最均衡，但不能增删分区；1 表示采用新数据定位方式，此时可以增删哈希分区；2 为兼顾数据分布均衡和允许添加分区的数据定位方式；3 采用新的 HASH 算法计算分区列哈希值，分区定位方式与 HASHPARTMAP(1) 保持一致
PL_SQL_STRIP	0	动态， 会话级	指定是否开启语句块/过程/函数/包等脚本中的语句剥离功能，即将其中的 DML 语句改为动态执行。 0：不开启语句剥离功能 1：开启语句块中语句剥离功能 2：开启过程和函数中语句剥离功能 4：开启包和 CLASS 类中语句剥离功能

			8：开启触发器中语句剥离功能 16：表示语句剥离后若不包含 USING 和 INTO，则动态执行直接使用原始 SQL 语句 32：开启 MERGE INTO 语句剥离功能 64：允许 CURSOR 定义或 OPEN 游标指定的查询表达式进行 SQL 剥离。但禁止将静态游标赋值给动态游标；且 OPEN 动态游标时，必须指定 SQL，否则将报错 128：在 ELOG 日志中记录 SQL 剥离转换前后的语句 支持使用上述有效值的组合值，如 6 表示同时进行 2 和 4 的处理
IPV6_LSNR	1	静态	是否监听 IPV6 端口的请求。0：不监听；1：监听
DBLINK_HEALTH_CHECK	1	动态，会话级	指定重用缓存中的 DBLINK 连接时是否检查连接有效性。0：不检查；1：通过接口进行简单检查；2：通过执行查询语句进行严格检查
BTR_FREE_PAGE_STACK	0	静态	指定是否开启 B 树空闲页堆栈。开启 B 树空闲页堆栈后，B 树控制页增加内节点和叶子节点页堆栈，用以缓存空闲的数据页。B 树申请数据页从 B 树空闲页堆栈分配，在 B 树空闲页堆栈分配页前，需检查空闲页堆栈空闲页是否足够，必要时分配数据页填充空闲页堆栈，确保空闲页堆栈一定有足够的空闲页，释放数据页时，优先归还数据页到 B 树空闲页堆栈。0：不开启；1：开启

RANDOM_CRYPTO	空串	静态	指定数据库中使用的随机数算法所在的加密引擎名。未指定或指定的加密引擎不存在，又或者指定的加密引擎中不存在随机数算法，则使用 OPENSSL 提供的软件随机数算法
PTX_BI_PERCENTAGE	50	动态，系统级	RECYCLE POOL 中可用来分配给 PTX 前镜像页的缓冲区 BUFFER 比例。取值范围 1~100
ENABLE_FAST_REFCURSOR	1	动态，系统级	是否支持打开游标时指定 FAST 属性，同时支持 FAST 游标作为输出参数返回上层调用。 0：不支持； 1：支持，即可使用 OPEN WITH FAST……语法打开游标
HUGE_UNIQUE_CHECK	0	动态，会话级	设置 HUGE 表的 UNIQUE 约束或 PRIMARY KEY 约束和 UNIQUE 索引是否检查数据唯一性。0：不检查； 1：检查
DEFERRED_SEGMENT_CREATION	0	动态，系统级	建表时是否启用延迟段分配功能。0：不启用；1：启用。HUGE 表/临时表/物化视图日志表不支持延迟段分配功能
PLN_INDEPENDENT	1	静态	指定构造计划时是否不再级联构造其引用过程/函数/包的计划并添加引用。0：级联构造计划并添加引用；1：不再级联构造计划并添加引用
USE_FORALL_ATTR	0	动态，	是否使用 FORALL 语句的游标属性，0：不使用；1：使用

		会话级	
ALTER_TABLE_OPT	0	动态， 会话级	是否对加列、修改列、删除列操作进行优化，0：全部不优化；1：全部优化；2：打开快速加列，对于删除列和修改列与 1 等效；3：打开快速加列，允许指定快速列默认值，其他功能与 2 时相同 对于 HUGE 表的修改，该设置为 0 或 1 时效果一样，均为采用查询插入实现
MAX_CLTN_MEM_SIZE	4 * 1024	动态， 系统级	设置单个收集类型对象的最大内存空间，单位为 MB，取值范围为 128~4294967294
TMP_OPT_POLICY	1	静态	临时表是否开启簇空间管理优化策略。0：不开启；1：开启
TMP_MAX_RESERVE_EXTENT	50	静态	临时表开启簇空间管理优化策略后，临时表最大的簇预留大小。临时表释放簇时，若全局缓存中预留的簇大小超过该值，后续释放的簇将直接释放回文件系统，而不再预留在缓存中。单位为 MB，取值范围为 0~4096。取值为 0 时表示临时表不设置预留上限，可以预留所有被临时表释放的簇在缓存中。
DPC_CHECK_MP_INI	1	静态	DMDPC 环境下专用，是否检查并修改 MP.INI。 0：不检查；1：检查并修改。 对于数据库迁移场景，将 MP 库物理拷贝或备份还原到另一台机器后，IP 信息会发生改变，MP 启动前需要将此参数配置为 0，避免 MP 使用字典信息中的旧 IP 信息去刷新 MP.INI。正常启动后，用户可以通

			过 SP_MODIFY_DPC_INSTANCE 来更新 IP 等信息，修改完成后再将该参数改回 1
PLN_BUILD_WAIT_TIME	5	动态，系统级	等待计划构建的最长等待时间，仅在 ENABLE_PLN_PRESEARCH=1 时有效，取值范围 1~604800，单位秒
HP_DEF_LOCK_MODE	0	动态，系统级	新建分区表时默认的封锁模式。0：只封锁根表；1：细粒度的封锁模式
INFO5_DEGRADE	0	静态	是否允许集群 INFO5 字段降级和作业系统降级，0：不允许降级；1：允许降级。缺省为 0
PAGE_CHECK_INDEXID	1	动态，系统级	是否校验数据页的 indexid 是否有效，非系统表数据页校验失败时会报错，系统表数据页校验失败时会 halt。0：不检验；1：校验数据页的 indexid 是否有效
CLOB_MAX_CALC_LEN	20480	动态，系统级	CLOB 类型数据计算过程中的最大长度，单位 KB，取值范围 8~102400，作用于如下场景： 1. LIKE 语句中 CLOB 类型的最大长度； 2. 字符串连接运算符 || 处理 CLOB 类型数据时的最大长度； 3. dbg 工具打印 CLOB 数据时，若 TEXT 长度小于 32K，可打印，若 TEXT 长度大于等于 32K，此参数控制最大打印长度； 4. 系统函数中支持的最大 CLOB 长度，目前受影响的函数有: INSTR, TRIM, LPAD, RPAD, UPPER, LOWER,

			CONCAT, SUBSTR, LOCATE, LEFT, RIGHT, CONCAT_WS, OVERLAY, POSITION, REPEAT, FIND_IN_SET,INITCAP, REPLACE, REVERSE, || 连接符, REGEXP_LIKE, REGEXP_COUNT, REGEXP_INSTR, REGEXP_SUBSTR, REGEXP_REPLACE, CFALGORITHMSENCRYPT, INSTRC, LCASE, LEFTSTR, LTRIM, NLS_LOWER, NLS_UPPER, REPEATSTR, RIGHTSTR, RTRIM, STRING_TO_ARRAY, SUBSTRING_INDEX, SUBSTRING, MID, UCASE
ENABLE_CREATE_BM_INDEX_FLAG	1	静态	是否允许创建位图索引。0：不允许创建位图索引；1：允许创建位图索引；2：位图索引作为普通 B 树索引进行创建
ID_RECYCLE_THRESHOLD	60	静态	ID 重用的开启比例。取值范围 0~100。ID 使用量超过此比例时，即当（可重用对象已分配的 ID 数目/非扩展的 ID 数目）*100>=ID_RECYCLE_THRESHOLD 时，开启 ID 重用；小于 ID_RECYCLE_THRESHOLD 时，关闭 ID 重用
IDLE_DISK_THRESHOLD	1000	动态，系统级	磁盘可用空间的报警阈值，单位 MB，取值范围 50~50000
IDLE_SESS_THRESHOLD	5	动态，系统级	有限的会话数阈值，取值范围 1~10
ENABLE_PRISVC	0	静态	是否启用服务优先级的功能，0：不启用；1：启用
CODE_CONVERSE_MODE	1	动态，系统级	DBLINK 对不完整字符的处理。0：丢弃；1：报错

ENABLE_CS_CVT	0	静态	是否直接使用系统函数 CONVERT()。0：否，CONVERT()转换为 CAST()执行；1：是，直接使用系统函数 CONVERT()执行
ENHANCE_RECLAIM	1	动态，系统级	是否采用增强的垃圾回收策略。0：否；1：是
SQLTUNE_CATEGORY	'DEFAULT'	动态，会话级	限制本会话中被允许使用的 SQL 配置（包括为 SQL 注入的 HINT 规则）的生效类别。 当 SQL 配置存在生效类别限制且和本会话的 SQLTUNE_CATEGORY 不一致时，该 SQL 配置在本会话不生效
HFI_HP_MODE	1	静态	事务型分区HUGE表插入时的并发策略。0：封锁子表策略，各子表之间可以并发插入，但可能出现死锁；1：封锁根表策略，各子表之间不能并发，但不会出现死锁
TSMV_FCOPY_STEP	4096	动态，系统级	文件拷贝消息同步步长，用于表空间文件拷贝时流量控制，防止消息堆积耗尽内存，DPC下自动同步到所有节点，取值范围[100，40000]
HNSW_SHARD_SEARCH_PARALLEL_DEGREE	1	动态，会话级	表示HNSW索引分片图并行查询时的分片图数量。取值范围1~128，1表示不开启并行。实际并行度同时受INI参数DBTA_SIZE的影响，例如查询100条结果，DBTA_SIZE设置为300，指定并行度为5，实际并行度可能只有3
PARALLEL_DML	0	动态，	是否使用并行DML以提升性能。0：不启用；1：启用并行插入

		会话级	
SQLERRM_PREFIX	0	动态， 系统级	是否在错误码前添加前缀'DM'。0：不添加；1：添加
SQL_INVOKE_BATCH	0	动态， 系统级	是否开启 SQL 函数批量，取值范围 0-1，默认值 0。设置之后生效。 0表示不开启sql函数批量，1表示开启；开启后仅影响当前生成的sql函数计划，已缓存的sql函数计划不受影响
VECTOR_INDEX_BUILD_PARALLEL_DEGREE	4	动态， 会话级	指定向量索引创建和重建时的并行度，取值范围1~128

2.1.1.1.9 预先装载表相关

表2.10 dm.ini 中预先装载表相关参数

参数名	缺省值	属性	说明
LOAD_TABLE	空串	静态	在服务器启动时预先装载的普通表的完整表名，即"模式名"."表名"，多个表之间用逗号分隔，最多可指定10个表
LOAD_HTABLE	空串	手动	在服务器启动时预先装载的HUGE表的表名。可设置多行，每行指定一个表。格式为 LOAD_HTABLE=<表名> 或 LOAD_HTABLE=<表名><列名>{,<列名>}）。

			例如: LOAD_HTABLE=SYSDBA.T1 LOAD_HTABLE=T2 LOAD_HTABLE=T3（C1,C2）
CLT_CACHE_TABLES	空串	手动	指定可以在客户端缓存的表。表名必须带模式名前缀，如果表名或模式名中包含特殊字符，需要使用双引号包含。如果指定多个缓存表，须以逗号间隔。服务器最多支持指定100个可缓存表。为避免参数值太长导致INI文件分析困难，允许在INI文件中设置多行CLT_CACHE_TABLES参数

2.1.1.1.10 REDO 日志生成相关

表2.11 dm.ini 中 REDO 日志生成相关参数

参数名	缺省值	属性	说明
RLOG_BUF_SIZE	1024	静态	单个日志缓冲区大小，以日志页个数为单位。取值范围 1~32768
RLOG_POOL_SIZE	256	静态	最大日志缓冲区大小，单位 MB。取值范围 1~4096 当建库参数 AUTO_ADJ_PARA 为 1 时，本参数实际默认值由系统根据 AUTO_ADJ_MEM 自动调整，用户也可根据实际需要手动调整。具体可参考《DM8_dminit 使用手册》。

RLOG_PARALLEL_ENABLE	1	静态	是否启动并行日志。0：不启用，但是日志刷盘时可能合并为并行日志包刷盘；1：启用；2：一律不生成并行日志包
RLOG_IGNORE_TABLE_SET	1	动态，系统级	是否开启记录物理逻辑日志功能。1 是，0 否
RLOG_APPEND_LOGIC	0	动态，系统级	是否启用在日志中记录逻辑操作的功能，取值范围 0、1、2、3、4 0：不启用；1、2、3、4 启用。 1：如果有主键列，记录 UPDATE 和 DELETE 操作时只包含主键列信息，若没有主键列则包含所有列信息； 2：不论是否有主键列，记录 UPDATE 和 DELETE 操作时都包含所有列的信息； 3：记录 UPDATE 时包含更新列的信息以及 ROWID，记录 DELETE 时只有 ROWID； 4：只生成事务以及 DDL 相关的逻辑日志 注：DELETE 和 UDPATE 操作日志中描述的“所有列的信息”不包含大字段列（如 BLOB 和 CLOB 类型的列）
RLOG_APPEND_SYSTAB_LOGIC	0	动态，系统级	是否启用在日志中记录系统表逻辑操作的功能。取值范围 0、1、2。 0：不启用； 1：启用。如果有主键列，记录 UPDATE 和 DELETE 操作时只包含主键列信息，若没有主键列则包含所有列信息； 2：启用。不论是否有主键列，记录 UPDATE 和 DELETE 操作时都包含所有列的信息

RLOG_SAFE_SPACE	256	静态	安全的可用日志空间大小，单位 MB。取值范围 0~1024。当系统的可用日志空间小于这个值时，自动触发检查点释放日志空间
RLOG_RESERVE_THRESHOLD	0	动态，系统级	控制是否启动日志空间安全性检查。0：不启动；1~131072：如果系统有效日志量达到设定的值，则启动日志空间预留机制，以提升故障重启和 DSC 集群节点故障处理速度，单位 MB 当配置值大于等于 1 且小于 32 时，dmserver 进程内部会直接使用 32MB 作为日志预留空间门槛进行判断
RLOG_RESERVE_SIZE	8192	静态	控制每次物理事务 PTX 操作需要预留的 REDO 空间大小，单位为日志页个数。取值范围 0~102400，0 表示不再检查剩余 REDO 可用空间是否安全，假定总是有富余。如果初始化库时开启了 HUGE 表日志（RLOG_GEN_FOR_HUGE=1），允许自行修改 RLOG_RESERVE_SIZE 大小，避免生成 HUGE 表日志时 REDO 可用空间不足
RLOG_SEND_APPLY_MON	64	静态	数据守护中，对于主库，用于指定统计最近 N 次主库到每个备库的归档发送时间；对于备库，用于指定统计最近 N 次备库重演日志的时间，N 为此参数设置的值。取值范围 1~1024
RLOG_COMPRESS_LEVEL	0	静态	日志压缩级别，取值范围 0~10。0 表示不压缩；1~9 表示 ZIP 压缩算法，值越大压缩比越高，压缩速度越慢；10 表示 SNAPPY 压缩算法
RLOG_ENC_CMPR_THREAD	4	静态	加密、压缩任务线程数，取值范围 1~64

RLOG_PKG_SEND_ECPR_ONLY	0	静态	用于主备环境下控制加密压缩环境。0：同时发送原始日志数据和加密压缩后的日志到备库；1：只发送加密压缩后的日志到备库
RLOG_HASH_NAME	空串	静态	日志包完整性校验使用的 HASH 算法名称，默认为空，使用 CRC 算法进行校验
DPC_LOG_INTERVAL	0	动态，系统级	DMDPC 环境下专用，MP 定时通知各节点生成相同时间点日志的时间间隔。取值范围 0~86400（24 小时）。单位秒。0 表示关闭定时通知功能。 此参数打开后，可支持使用归档恢复到指定时间点功能，在对系统性能有要求的情况下，可以调大或关闭此参数值
RLOG_RAFT_NEED_WAIT	3	动态，系统级	主库是否进行日志包发送等待控制，仅 RAFT 主库生效。 0：主库不进行日志包发送控制； 1：任意备库存在日志堆积时，根据 RLOG_RAFT_WAIT_TIME 暂缓日志包发送； 2：存在日志堆积的备库个数超过备库总数的半数时，根据 RLOG_RAFT_WAIT_TIME 暂缓日志包发送； 3：任意归档状态有效的备库存在日志堆积，根据 RLOG_RAFT_WAIT_TIME 暂缓日志包发送； 4：存在日志堆积的归档状态有效的备库个数超过备库总数的半数时，根据 RLOG_RAFT_WAIT_TIME 暂缓日志包发送
RLOG_RAFT_WAIT_TIME	1000	动态，系	备库出现堆积后，RAFT 主库延迟发送等待时间，取值范围（0~3600000），单位毫秒。仅 RAFT 主库生效

		统级	
LOG_FILE_POSTFIX_NAME	log	动态，系 统级	归档日志文扩展名，长度不超过 10 个字符。默认与数据库初始化参数 RLOG_POSTFIX_NAME 一致， RLOG_POSTFIX_NAME 默认为 log
RLOG_PKG_PAGE_CRC	0	静态	是否为 REDO 日志包每 4K 数据生成额外的 CRC 校验码。0：否；1：是
RLOG_LLOG_UPD_TO_DEL_INS	0	动态，系 统级	设置当分区表允许行迁移且 UPDATE 操作发生了行迁移时，是否将 UPDATE 逻辑日志更改为 DELETE+INSERT 日志 0：否，保持 UPDATE 逻辑日志不变； 1：是，当分区表发生了行迁移时，将 UPDATE 逻辑日志更改为 DELETE+INSERT 日志
RLOG_LLOG_COMPRESS	0	动态，系 统级	是否开启对 INSERT 逻辑日志的压缩功能。建议插入操作较多导致日志文件过大的情况下使用。0：不开启；1：开启
LOGMNR_DELAY_ANALYZE	0	动态，系 统级	LOGMNR 包是否推迟解析日志的时机。 0：START_LOGMNR 时解析日志；1：查询 V$LOGMNR_CONTENTS 时解析日志
LOGMNR_PARSE_LOB	0	动态，系 统级	LOGMNR 包是否支持挖掘行外大字段逻辑日志。0：不支持；1：支持
LOGMNR_GEN_UNDO	0	动态，系	是否构造 SQL_UNDO 语句。0：否；1：是

		统级	当参数 LOGMNR_GEN_UNDO=1 时，目前仅支持 INSERT/DELETE/UPDATE/FAST INSERT 构造 SQL_UNDO 语句，规则如下： （1）INSERT/FAST INSERT 语句，SQL_UNDO 为 DELETE 语句，DELETE 语句中包含所有 INSERT 日志中的插入列； （2）DELETE 语句，SQL_UNDO 为 INSERT 语句，INSERT 语句中包含所有 DELETE 日志中的列，若 DELETE 日志中不包含列，则 SQL_UNDO 语句为 NULL； （3）UPDATE 语句，SQL_UNDO 为 UPDATE 语句，且 SET 语句中包含所有 UPDATE 日志中的更新列的老值，若 UPDATE 日志中不包含更新列的老值则用“?”问题替代
RLOG_PARALLEL_NUM	16	静态	用于控制日志并行度，此参数仅在 RLOG_PARALLEL_ENABLE 为 1 时才会生效。取值范围 2~8192。
RLOG_FLUSH_WAIT_TIME	0	动态，系统级	日志刷盘负反馈机制，待刷盘日志堆积超过 512M 时，触发日志空间预留等待，用于平滑系统压力。0：不启用；其他值：最长等待时间。取值范围：0~100，单位：毫秒
RAFT_HP_TIMEOUT	60	动态，系统级	DMDPC 的多副本系统的日志包堆积超时时间，单位秒。取值范围 1~36000。 当 DMDPC 的多副本系统主库日志包超过此超时时间没有提交日志包，则会将服务器设置为日志堆积状态，断开所有连接会话，并会将服务器设置为只读模式，后续连接的会话为只读会话。当服务器恢复提交日志包后，服务器会恢复到可写状态，后续连接会话将恢复可写

2.1.1.1.11 REDO 重做/重演相关

本节的 INI 参数为重做或重演所需参数。

其中，以 REDOS_开头的 INI 参数为备库上日志重演专用参数。只需要在备库上设置，主库不需要。

表2.12 dm.ini 中 REDO 重做/重演相关参数

参数名	缺省值	属性	说明
REDO_PWR_OPT	1	静态	指定对于重做 REDO 日志的 PWR 优化方式，PWR 日志优化指跳过已经写入磁盘数据页的 REDO 日志重演。0：不启用 PWR 日志优化；1：生成 PWR 日志，重做日志时启用 PWR 优化；2：生成 PWR 日志，但重做日志时不启用 PWR 优化
REDO_IGNORE_DB_VERSION	0	静态	启动重做 REDO 日志时，是否检查版本信息。0：忽略版本检查，直接使用新版本重做 REDO 日志；1：正常检查版本，不兼容的库会报错，需要使用对应版本启动并正常关闭后，再用新版本执行码启动
REDO_BUF_SIZE	64	静态	日志加载缓存大小，系统故障重启时根据此参数分配内存，并将 REDO 日志从文件读取到此缓存。取值范围 0~512，单位 MB。0 表示无限制
REDOS_BUF_SIZE	1024	动态，系统级	待重演日志堆积的内存限制，堆积的日志缓冲区占用内存超过此限制，则新的日志将会被延迟加入重演队列，等待重演释放部分内存后再加入。单位 MB，取值范围 0~65536。0 表示无内存限制。

			REDOS_BUF_SIZE 和 REDOS_BUF_NUM 同时起作用，只要达到一个条件即会触发延迟处理
REDOS_BUF_NUM	4096	动态， 系统级	待重演日志缓冲区允许堆积的数目限制，超过限制则新的日志将会被延迟加入重演队列，等待堆积数减少后再加入。以个数为单位，取值范围 0~9999。0 表示无限制。 REDOS_BUF_SIZE 和 REDOS_BUF_NUM 同时起作用，只要达到一个条件即会触发延迟处理
REDOS_MAX_DELAY	1800	静态	备库重演日志缓冲区的时间限制，超过此限制则认为重演异常，服务器自动宕机，防止日志堆积、主库不能及时响应用户请求。单位秒，取值范围 0~7200。0 表示无重做时间限制
REDOS_PARALLEL_NUM	1	静态	指定故障重启、还原恢复、备库重演场景下，并行日志重演的线程数。取值范围 1~256。1 表示不进行并行日志重演。建议该参数值取 BUFFER_POOLS / N（N 为整数，且 BUFFER_POOLS / N 的余数为 0，建议结合机器资源情况合理选择 N 值）。当 BUFFER_POOLS 是根据 BUFFER 的配置情况进行自动调整（确保每个 BUFFER POOL 中至少存放 3000 个数据页）时，建议以 BUFFER_POOLS 内存值为准来配置 REDOS_PARALLEL_NUM
REDOS_RPKG_PARSE_NUM	16	静态	表示并行解析日志包的线程数。取值范围 1~256
REDOS_RPKG_FILL_NUM	64	静态	表示重演任务数组中最多允许填充的日志包个数。取值范围 1~320
REDOS_ENABLE_SELECT	1	静态	指定是否允许备库执行查询操作。 0：DMDPC 环境下仅允许备库查询 V$动态视图；其他环境下允许备库查询 V$动态视图、系统表和系统视图，其他的查

			询操作均不支持。如果开启了并行重演，则不需要考虑重演和查询的并发问题，备库会采用完全并行重演方式。 1：允许备库执行所有查询操作，如果开启了并行重演，为了确保查询结果正确，备库在重演到特定日志（X 封锁 B 树根页）时会切换为局部串行重演，特定日志重演结束后再恢复并行重演。 2：允许备库执行所有查询操作，同时会检测实际是否有查询动作在执行，如果开启了并行重演，而实际没有执行查询操作，则仍然采用完全并行重演方式，仅在有查询操作正在执行并且重演到特定日志（X 封锁 B 树根页）时才会切换为局部串行重演。注意：这里的查询检测不包括 V$动态视图、系统表和系统视图的查询。 设置该参数为 0 或 2 时，会在一定程度上提升备库的重演性能
REDOS_FILE_PATH_POLICY	0	静态	备库重演日志时表空间数据文件路径策略。0：数据文件创建到 SYSTEM_PATH 路径下；1：数据文件截掉根路径后，在 SYSTEM_PATH 下生成相同的路径结构
REDOS_PAGE_CRC_CHECK	0	动态， 系统级	是否开启主备页校验功能。0：否；1：是
REDO_PARALLEL_NUM	1	静态	指定服务器故障重启时，重演日志的并行度。取值范围 1~256。当服务器故障重启时，将 MAX(REDOS_PARALLEL_NUM+REDOS_ROLL_PARALL_NUM，REDO_PARALLEL_NUM)作为重演并行度来进行重演日志

2.1.1.1.12 事务相关

表2.13 dm.ini 中事务相关参数

参数名	缺省值	属性	说明
ISOLATION_LEVEL	1	静态	系统默认隔离级别。1：读提交；3：可串行化
DDL_WAIT_TIME	10	动态，会话级	DDL 操作的锁超时时间，单位秒。取值范围 0~604800
BLDR_WAIT_TIME	10	动态，系统级	批量装载时锁超时时间，单位秒。取值范围 0~604800
MPP_WAIT_TIME	10	动态，系统级	设置 MPP 下默认的封锁等待超时，单位秒，取值范围 0~600
FAST_RELEASE_SLOCK	1	动态，系统级	是否启用快速释放 S 锁，0：不启用；1：启用，语句块中所有的 SQL 语句执行完成后快速释放；2：语句块中每条 SQL 都尝试快速释放
SESS_CHECK_INTERVAL	3	动态，	循环检测会话状态的时间间隔，单位秒。取值范围 1~60

		会话级	
LOCK_TID_MODE	1	动态，系统级	SELECT FOR UPDATE 封锁方式。0：结果集记录小于 100 行，直接封锁 TID，否则升级为表锁； 1：不升级表锁,一律使用 TID 锁
LOCK_TID_UPGRADE	0	动态，系统级	0：事务启动时，用自己的事务号生成的TID锁，以X封锁；DELETE/UPDATE记录触发的TID锁，优先使用S封锁，只有在多个事务同时更新同一行记录场景下，升级为X锁。1：所有TID锁以X方式封锁
NOWAIT_WHEN_UNIQUE_CONFLICT	0	静态	插入数据时，如果和未提交数据有 UNIQUE 约束的冲突，是否等待未提交事务结束，0：等待，直至未提交事务结束；1：不等待，立即返回错误
UNDO_EXTENT_NUM	4	静态	指定每个工作线程回滚页的使用上限，超过时启动回收机制，单位为簇的个数。取值范围 1~8192
MAX_DE_TIMEOUT	10	动态，会话级	C、JAVA 外部函数的执行超时时间，单位秒。取值范围 1~3600
TRX_RLOG_WAIT_MODE	0	静态	事务提交时的日志刷盘等待模式。0：保持现有模式；1：主库模式先等待日志刷盘完成，再释放事务上的资源
TRANSACTIONS	75	静态	指定一个会话中可以并发的自治事务数量。取值范围 1~1000
MVCC_RETRY_TIMES	5	静态	指定发生 MVCC 冲突时的最大重试次数。取值范围 0~4294967294

			注：MPP 下此参数无效，发生 MVCC 冲突时将直接报错
MVCC_PAGE_OPT	1	动态， 系统级	控制是否启动数据页可见性优化处理。0：关闭数据页可见性优化；1：启用数据页可见性优化
ENABLE_FLASHBACK	0	动态， 系统级	是否启用闪回查询，0：不启用；1：启用
UNDO_RETENTION	90	动态， 系统级	事务提交后回滚页保持时间，单位秒。取值范围 0~86400 注：类型为 DOUBLE，可支持毫秒
PARALLEL_PURGE_FLAG	0	静态	是否启用并行事务清理，0：不启用；1：启用
PSEG_RECV	2	手动	系统故障重启时，对活动事务和已提交事务的处理方式。 0：跳过回滚活动事务和 PURGE 已经提交事务的步骤。在回滚表空间出现异常、损坏、系统无法正常启动时，可将 PSEG_RECV 设置为 0，让系统启动；但存在一定风险，未提交事务的修改将无法回滚，破坏事务的原子性；另外，已提交未 PURGE 的事务，将导致部分存储空间无法回收； 1：回滚活动事务并 PURGE 已经提交事务； 2：延迟 PURGE 已提交事务，延迟回滚活动事务；

			3：回滚活动事务，延迟 PURGE 已提交事务
ENABLE_IGNORE_PURGE_REC	2	动态，会话级	当返回 EC_RN_NREC_PURGED（-7120）错误（回滚记录版本太旧，无法获取用户记录）时的处理策略；0：报错；1：忽略这一条记录，继续执行；2：报错并生成日志；3：忽略这一条记录，继续执行，并生成日志 注意：配置为 1 或 3 时，忽略了 EC_RN_NREC_PURGED（-7120）错误信息，原本报错的语句会继续执行。为了避免破坏数据一致性，只有只读事务允许忽略 EC_RN_NREC_PURGED 错误。如果当前事务已经修改过数据，直接忽略 ENABLE_IGNORE_PURGE_REC 参数设置，仍然返回 EC_RN_NREC_PURGED 错误；如果当前事务没有修改数据，则忽略 EC_RN_NREC_PURGED 错误，但是会进行延迟检查，一旦后续操作修改数就会触发 EC_RN_NREC_PURGED 错误。 引发数据一致性问题说明：比如执行 UPDATE 语句时，预期更新 2 条记录，但实际只更新了 1 条记录，另外 1 条记录由于忽略了 EC_RN_NREC_PURGED 错误，既没有报错，也没有更新
ROLL_ON_ERR	0	动态，系统级	服务器执行出错时的回滚策略选择，0：回滚当前语句；1：回滚整个事务
XA_TRX_IDLE_TIME	60	动态，	允许未 PRE-COMMITTED 的 XA 事务活动的时间，单位秒。取值范围 30~36000。

		系统级	实际表现可能存在 1-2 秒的误差，属于合理范围
XA_TRX_IDLE_POLICY	0	静态	达到 XA_TRX_IDLE_TIME 超时时间后，是否将 PRE-COMMIT 事务回滚。0：否；1：是
XA_TRX_LIMIT	1024	静态	XA 事务数量上限。取值范围 1024~65535
ENABLE_TMP_TAB_ROLLBACK	1	动态， 系统级	临时表操作是否生成回滚记录，0：不生成；1：生成。 置为 0 时，有三项限制：一是临时表的 DML 操作无法回滚；二是不允许对包含唯一索引或函数索引的临时表进行增删改操作；三是不允许对临时表创建唯一索引、函数索引、主键约束和唯一约束
LOCK_DICT_OPT	2	静态	DICT封锁优化，事务提交或回滚时，不真正释放表上IS锁和IX锁对象。0:不优化；1:单机环境优化，DSC集群环境不优化；2:单机、集群环境都优化
LOCK_DICT_HASH_SIZE	10000	静态	设置封锁系统HASH表大小，取值范围1024~65534
TRX_DICT_LOCK_NUM	64	动态， 系统级	开启LOCK_DICT_OPT优化场景下最多可缓存的IX/IS类型字典锁对象数量。取值范围1~100000
DEADLOCK_CHECK_INTERVAL	1000	动态， 系统级	控制死锁检测间隔时间，单位毫秒，取值范围100~60000
LOB_MVCC	1	动态，	是否支持LOB数据的多版本并发控制访问。0：更新/删除语句会对涉及的大字段进行封锁；1：更新/删除语

		系统级	句不对涉及的大字段进行封锁
COMMIT_WRITE	IMMEDIATE,WAIT	动态，系统级	格式为<IMMEDIATE|BATCH>,<WAIT|NOWAIT|NOTIFY>，第一个参数可以取IMMEDIATE/BATCH，目前仅支持语法解析，无实际作用；第二个参数可以取WAIT/NOWAIT/NOTIFY，WAIT表示事务提交通知日志刷盘并等待日志刷盘成功；NOWAIT表示事务提交不通知日志刷盘、且不等待日志刷盘成功；NOTIFY表示事务提交仅通知日志刷盘、不等待日志刷盘成功。当配置为NOWAIT或NOTIFY时，实例异常退出，可能会导致数据页内容逻辑错误，导致事务不一致的情况，需谨慎配置
COMMIT_BATCH	1	动态，系统级	用于控制是否开启事务组提交（GROUP COMMIT）以及开启后触发事务组提交的事务数。 参数取值范围 1~1024，取值为 1 时表示关闭事务组提交功能，取值大于 1 时表示开启事务组提交功能，此时该参数取值还表示触发事务组提交的事务数。 该参数在 COMMIT_WRITE = IMMEDIATE,WAIT 时生效，COMMIT_WRITE = IMMEDIATE,NOWAIT 时无效
COMMIT_BATCH_TIMEOUT	1000	动态，系统级	事务批量提交超时时间，单位微秒，取值范围 0~1000000。COMMIT_BATCH 大于 1 时生效。事务发起提交请求以后，在 COMMIT_BATCH_TIMEOUT 时间内没有触发合并写入，则强制启动 REDO 日志刷盘。 该参数在 COMMIT_WRITE=IMMEDIATE,WAIT 时生效，COMMIT_WRITE=IMMEDIATE,NOWAIT 时无效。 WINDOWS 环境不支持微秒级超时时间，dmserver 内部会将此参数除以 1000 作为实际等待的毫秒数，若

			参数值小于 1000 则实际等待时间为 0，即事务提交不作等待、直接强制启动 REDO 日志刷盘
DPC_2PC	1	静态	DMDPC 环境下专用，DMDPC 系统是否采用两阶段提交策略及是否使用优化方案。该参数在非 DMDPC 系统中强制设置为 0。 0：关闭两阶段提交，没有故障的情况下直接提交效率更高，但是出现故障就无法保证事务一致性了； 1：启用两阶段提交； 2 或 3：启用两阶段提交，启用 trxid 缓存，启用 snap_seq 缓存方案优化； 4 或 5：启用两阶段提交，启用 trxid 缓存，启用 snap_seq 本地优化； 6 或 7：启用两阶段提交，启用 trxid 缓存，启用 snap_seq 缓存，启用 snap_seq 本地优化
SWITCH_CONN	0	动态，系统级	DMDPC 环境下专用，JDBC 客户端是否进行连接切换，执行过程中发现计划只和一个 BP 相关时，客户端是否切换到该 BP 执行。 0：否；1：是。 此参数需要和 JDBC 驱动/dm_svc.conf 配合使用，只用于 BS 模式
UNDO_BATCH_FAST	0	动态，系统级	是否启用批量插入快速回滚功能，0：不启用； 1：启用

FLDR_LOCK_RETRY_TIMES	0	动态， 系统级	指定快速装载产生死锁后自动重试的次数，取值范围0~4294967294。缺省值0表示不自动重试
DPC_TRX_TIMEOUT	10	动态， 会话级	DMDPC环境下专用，表示事务等待超时时间，单位秒。取值范围0~604800
SESSION_READONLY	0	动态， 系统级	用于设置系统中用户会话SESSION对数据库的读写操作。1：只读；0：可读可写。 对于当前会话，用户还可通过执行SP_SET_SESSION_READONLY (1/0)来重新设置当前会话的读写操作， 1：只读；0：可读可写
SELECT_LOCK_MODE	0	动态， 系统级	语句执行时相关表的上锁方式。0：集中上锁；1：查询在操作符中上锁，相比集中上锁的方式，封锁范围较小、频率较高，适用于期望查询语句不阻塞DDL，且能够接受查询语句被打断的场景 注： 1. 不支持MPP环境； 2. 若不是DPC环境，那么设置PARALLEL_POLICY参数为1或2时本参数将失效； 3. 当本参数设置为1时，若TRUNC_CHECK_MODE参数设置为0，则会被调整为 TRUNC_CHECK_MODE=1；

			4. 如果表上有大字段则禁止SELECT_LOCK_MODE的优化
ENABLE_SEC_RPTR	0	静态	二级索引是否附加回滚记录地址以优化可见性判断。0：不启用；1：启用该优化，缺省值为0。需要注意，启用该参数后新创建的二级索引（视图SYSOBJECTS INFO1字段第八位设置为1）在执行版本降级前需要手动删除
FINS_UNDO_OPT	1	静态	是否支持批量插入回滚的优化处理。0：不支持；1：支持
TRX_VIEW_POLICY	0	动态，系统级	设置事务可见性视图策略。0：脚本中的SQL语句不更新事务可见性视图；1：脚本中的SQL语句更新事务可见性视图
HP_DDL_LOCK_MODE	0	静态	控制分区表DDL封锁策略，是否开启优化，允许部分DDL与DML并发执行。0：不开启优化，与现有策略保持一致；1：开启优化 支持的DDL：新增分区、删除分区、合并分区、拆分分区 限制说明：(1)若DDL与DML操作的子表不存在交集，则允许并发执行；(2)不支持MPP、并行（DPC除外）
UNDO_SPACE	0	动态，系统级	用于限制已提交（COMMIT）未清理（PURGE）事务所占用的回滚表空间的总空间大小，单位GB，取值范围0~4294967294。0表示不限制；非0值表示在回滚表空间使用量达到该值时，即使已提交事务的生存期不足UNDO_RETENTION，系统也会按提交时间先后，强制启动清理（PURGE），直到回滚表空间的使用量

			小于设定值。 注：回滚页使用量的精确统计代价较高，系统使用统计近似值，开启本参数后实际的回滚表空间的占用量和本参数设定值存在一定的误差，这是正常现象
DDL_PURGE_POLICY	0	静态	包含 DDL 相关事务在执行 COMMIT 操作以后是否立即启动 PURGE 选项。 0：不立即启动；1：立即启动。 注：当参数 ENABLE_OBJ_REUSE 为 1 时，强制调整本参数值也为 1
LONG_TRX_VISIBLE	0	动态，会话级	使用 TRX_VIEW_MODE=1 判断事务可见性时遇到 CMTARR 被覆盖而无法确定可见性的处理策略，取值为 0、1、2、4、8、16 或这些值的任意组合。 0：报错“记录可见性无法确定”； 1：当前事务不是长事务时忽略报错； 2：取值包含 1 时是否认为记录不可见，不指定则认为可见； 4：当前事务是长事务时忽略报错； 8：取值包含 4 时是否认为记录不可见，不指定则认为可见。 16：发生表、分区、分区组、表空间迁移导致无法确定可见性时是否禁用强制可见

MAL_WAIT_TIMEOUT	0	动态， 系统级	MPP 下删除更新等待的超时报错时间，单位秒。取值范围为 0~864000，取缺省值 0 时表示关闭超时检测
USER_READ_ONLY_MODE	0	动态， 系统级	是否禁止只读用户设置会话为非只读。 0：否；1：是
ROLL_DEL_OPT	0	动态， 系统级	是否启用全表删除的回滚优化，启用后可以加快全表删除的回滚速度，并且如果执行全表删除的会话已经关闭，则在回滚结束前允许先对同一张表执行 TRUNCATE，TRUNCATE 结束后再继续回滚。 0：不启用，1：启用
DPC_PROXY_TIMEOUT	600	动态， 系统级	DMDPC 环境下专用，允许 MP 代理处理故障 SP 事务的最小故障时间，取值范围 0~4294967294，单位秒。因为存在任务调度时间和节点故障判定时间，SP 故障后至少需要 5-10 秒才会被 MP 代理。
TRX_CR_PAGE_NUM	0	动态， 系统级	当前事务缓存的回滚页数量，MVCC 访问回滚页构造可见版本数据时，在当前事务缓存回滚页信息，消除回滚页访问引发的 BUFFER 冲突等代价，提升并发效率。取值范围 0~10000。0 表示不启用回滚页缓存。
LOB_PURGE_IMMED	0	动态， 系统级	表示是否在事务提交时立即对大字段删除进行 PURGE。用于快速回收 LOB 数据场景，并发场景谨慎使用，可能引起可见性查询失败。 0：事务提交时不立即对大字段删除进行 PURGE；

			1：事务提交时立即对大字段删除进行 PURGE
EGTS_SNAP_COUNT	0	动态， 系统级	DPC_2PC 配置开启 SNAP_SEQ 缓存方案时，设置启动通信任务的频率。 0：引用缓存 SNAP_SEQ 后立即启动通信任务以尽快取回有效 SNAP_SEQ; 非 0：指定缓存 SNAP_SEQ 被引用多少次才需要启动一次通信任务
EGTS_TID_COUNT	0	动态， 系统级	DPC_2PC 配置开启 TRXID 缓存方案时，可以设置服务器的单次通信任务向 MP 申请 TRXID 的数目，多余的 TRXID 会缓存下来供后续事务直接使用。 0：一次性申请 8 次通信任务周期内所需的 TRXID 数; 小于等于 60：一次性申请指定秒数内所需的 TRXID 数; 大于 60：指定一次性申请的具体 TRXID 数
TRIG_DML_LOCK	0	动态， 系统级	ALTER TRIGGER 期间，是否允许对触发器基表进行 DML 操作。 0：不允许；1：允许

2.1.1.1.13 安全相关

表2.14 dm.ini 中安全相关参数

参数名	缺省值	属性	说明
ENABLE_ENCRYPT	0	静态	用于通信加密。基于传输层的 SSL 协议通信加密所采用的方式。取值 0、1、2、4 和 5。 0：不开启 SSL 认证和 SSL 通信加密； 1：开启 SSL 认证和 SSL 通信加密； 2：开启 SSL 认证但不开启 SSL 通信加密； 4：开启 SSL 加密不开启 SSL 认证； 5：开启 SSL 加密和对服务端的 SSL 单向认证。 6：开启商密 TLCP 认证和 TLCP 通信加密。 取值 1、2、4、5 或 6 时，如果服务器 SSL 环境没有配置则服务器无法正常启动，如果客户端 SSL 环境没有配置则无法连接服务器
MIN_SSL_VERSION	771	静态	允许 SSL 连接的最低版本。0：支持所有版本；0X0301：TLSV1；0X0302：TLSV1.1；0X0303：TLSV1.2；0X0304：TLSV1.3。 仅当 ENABLE_ENCRYPT 非 0 时有效
CLIENT_UKEY	0	静态	是否采用 UKEY 身份验证。0：支持所有登录方式，是否采用 UKEY 验证由客户端驱动。当客户端输入

			UKEY 名称和 UKEY_PIN 则采用 UKEY 验证，否则不使用 UKEY 验证；1：客户端强制使用 UKEY 验证
ENABLE_UDP	0	静态	是否支持 UDP 协议。0：否；1：单发单收；2：多发多收
UDP_MAX_IDLE	15	静态	UDP 最大等待时间，单位秒，超过则认为连接断开。取值范围 5~60
UDP_BTU_COUNT	8	静态	UDP 最大不等待连续发送消息块数，当不连续差值大于它时则等待发送。取值范围 4~32
ENABLE_IPC	0	静态	是否支持 IPC 协议。0：否；1：是
AUDIT_FILE_FULL_MODE	3	静态	审计文件存储剩余空间不足时的处理方式： 1：审计文件创建失败，卡住 2：审计文件创建失败，不再审计 3：结合 1 和 2，先尝试删老审计文件，再创建审计文件，失败则不再审计
AUDIT_FLUSH_LEVEL	1	动态， 系统级	审计记录刷盘方式，仅当打开审计开关时有效。打开审计开关的方法请参考《DM8 安全管理》。 0：语句级刷盘，一条语句执行结束进行一次审计记录刷盘操作；1：事务级刷盘，一个事务执行结束执行一次审计日志刷盘操作；2：延迟刷盘，审计记录缓存满时（缓存大小固定为 8MB）自动刷盘
AUDIT_KEEP_DAYS	0	静态	审计文件最小保留天数。取值范围 0~3660，单位天。即使达到审计文件总存储空间大小限制（即 AUDIT_SPACE_LIMIT 设置的值）也不会删除在指定保留天数内的审计文件。只有最老审计日志文件的

			保留天数超过 AUDIT_KEEP_DAYS 才会根据 AUDIT_FILE_FULL_MODE 设定值来处理老的审计文件
AUDIT_SPACE_LIMIT	0	静态	审计文件总存储空间大小限制，单位 MB。取值范围 204800~1024*1024，0 表示不限制
AUDIT_MAX_FILE_SIZE	100	动态， 系统级	单个审计文件的最大大小，单位 MB。取值范围 8~4096
AUDIT_IP_STYLE	0	动态， 系统级	审计记录 IP 字段显示格式。 0：仅显示 IP 1：显示 IP 和主机名
MSG_COMPRESS_TYPE	2	静态	与客户端的通信消息是否进行消息压缩，0：不压缩；1：压缩；2：系统自动决定每条消息是否压缩
LDAP_HOST	空串	静态	LDAP服务器IP
COMM_ENCRYPT_NAME	空串	静态	用于通信加密。基于应用层的消息包加密的开关。 空串：不加密； 算法名：按照指定的算法名进行加密。如果给的加密算法名错误，则使用加密算法AES256_CFB。DM支持的加密算法名可以通过查询动态视图V$CIPHERS获取
COMM_VALIDATE	1	动态，	是否对消息进行校验。0：不检验；1：检验，不记录错误内容；2：检验，记录错误消息内容

		系统级	
MESSAGE_CHECK	0	动态，系统级	是否对通信消息启用消息体校验（只有当消息的发送端和接收端都配置为1才启用通信体校验）。0：不启用；1：启用
EXTERNAL_JFUN_PORT	6363	动态，系统级	服务器执行JAVA外部函数时连接dmagent使用的端口号，dmagent的监听端口，监听方配置此参数，取值范围128~65534，发起连接端的端口在1024~65535之间随机分配
EXTERNAL_AP_PORT	4236	动态，系统级	执行C外部函数或AP备份还原使用的AP的端口号，DMAP的监听端口，监听方配置此参数，取值范围1024~65534，发起连接端的端口在1024~65535之间随机分配
EFC_USE_AP	1	动态，系统级	是否允许结构化的C外部函数在DMServer内部执行。1表示使用常规DMAP方式通信；0表示不使用DMAP，直接在DMServer内部调用执行
ENABLE_PL_SYNONYM	0	动态，系统级	是否可以通过同义词执行非系统用户创建的包或者存储过程。1是，0否。若设置为0，在解析过程/包名时，如果借助了同义词，则这些对象要么是系统内部创建的，或者其创建者必须为系统用户，否则一律报错
FORCE_CERTIFICATE_ENCRYPTION	0	静态	非加密通讯的情况下是否开启用户名密码强制证书解密。1 是，0 否。 私钥证书 DM_LOGIN.PRIKEY 需要提前拷贝到 INI 参数 SYSTEM_PATH 指定的目录下
REGEXP_MATCH_PATTERN	0	动态，会	指定正则表达式的匹配模式。0:支持非贪婪匹配；1:仅支持贪婪匹配

		话级	
UNIX_SOCKET_PATHNAME	空串	静态	LINUX系统中，当服务器与客户端之间使用UNIXSOCKET通信协议时，用于指定客户端连接的socket文件路径
RESOURCE_FLAG	0	静态	设置用户资源限制项 CONNECT_TIME 和 CONNECT_IDLE_TIME 的时间单位。1：秒；0：分钟。 修改该时间单位并不会同步调整限制值，用户需要手动调整已设置的 CONNECT_TIME 和 CONNECT_IDLE_TIME 限制项配置值来保持含义不变，因此建议谨慎调整该 INI 参数
IPV6_ZID_FLAG	WINDOWS 下为 0，LINUX 下为 1	动态，系统级	是否自动获取本地网卡标识，当使用的 IP 地址为不带有网卡标识（ZONE ID）的 IPV6 LINK-LOCAL 地址时，会进行网卡接口自动适配。0：关闭；1：打开。
IPV6_ZID	空串	动态，系统级	允许用户配置使用的网卡标识（ZONE ID），最多可以配置 20 个，最大长度为 128.各网卡接口之间用”,”分隔，读取时会忽略每个网卡标识的前导空格和尾随空格。
GRANT_SCHEMA	0	静态	是否支持授予和回收模式权限功能。 0：不支持； 1：支持模式权限，ALL PRIVILEGES 表示所有模式级别非 ANY 特权； 2：支持模式权限，ALL PRIVILEGES 表示所有模式级别 ANY 特权
AUTH_ENCRYPT_NAME	空串	静态	创建用户时，处理用户口令加密的 HASH 算法名

SSL_SECURITY_LEVEL	0	静态	用于控制数据库OPENSSL的安全等级。 0：不做限制； 1：安全级别80bits； 2：安全级别112bits； 3：安全级别128bits； 4：安全级别192bits； 5：安全级别256bits
ENABLE_RLS	0	静态	是否开启 DBMS_RLS 系统包中的策略管理行级安全访问控制功能。0：不开启；1：开启 注：MPP 环境下，禁用 ENABLE_RLS 参数
RLS_CACHE_SIZE	100000	动态， 系统级	ENABLE_RLS=1 时使用，用于指定策略缓存空间能够存放对象结点数量的最大值。取值范围 1000~1000000。策略缓存中的策略可通过 V$POLICY_CACHE 和 V$POLICY_CACHE_ITEM 进行查看
INVITED_IP	空串	动态， 系统级	数据库级 IP 白名单，仅允许指定的 IP/IP 网段(仅支持 CIDR 格式)发起的连接请求，对该 IP/IP 网段上的 所有用户生效；支持设置多个 IP/IP 网段，使用逗号分隔
EXCLUDED_IP	空串	动态，	数据库级 IP 黑名单，禁止指定的 IP/IP 网段(仅支持 CIDR 格式)发起的连接请求，对该 IP/IP 网段上的所

		系统级	有用户生效；支持设置多个 IP/IP 网段，使用逗号分隔
ENABLE_LOCAL_OSAUTH	0	静态	是否支持基于操作系统的本机身份验证，0：不支持；1：支持
CIPHER_CHECK_INTERVAL	0	动态， 系统级	加密卡自动检测时间间隔，单位秒。取值范围 0~864000，0 表示不进行定期检测。 开启后，若检测失败，会主动 HALT
FILE_ACCESS_DIRECTORY	空串	手动	指定创建和使用 DIRECTORY 对象时可访问的路径。创建 DIRECTORY 的路径，只能在此参数指定的路径下。使用 DIRECTORY 对象时，也会检查其路径，只能在此参数指定的路径下，如果修改了此参数值，则库中已存在的 DIRECTORY 对象的路径可能不符合要求，这些检查不通过的目录路径无法访问。若参数值为空串，即未指定路径，则无访问限制。参数可指定多个路径，用英文逗号进行分隔，其中不合法的路径和不支持的路径将被忽略，若所有路径都被忽略，则创建和使用 DIRECTORY 时不能访问任何路径。 不支持带有英文逗号的路径；检查路径时，会直接检查分隔出的字符串，不除去首尾的空格和\T 等；不支持指定相对路径，只能写绝对路径；不支持使用 LINK 路径

2.1.1.1.14 兼容性相关

表2.15 dm.ini 中兼容性相关参数

参数名	缺省值	属性	说明
BACKSLASH_ESCAPE	0	静态	是否需要对字符串中的反斜杠使用词法分析的反斜杠转义处理。0：否；1：是
STR_LIKE_IGNORE_MATCH_END_SPACE	1	动态，会话级	LIKE 运算中是否忽略匹配串的结尾 0。0：不忽略；1：忽略
EXCLUDE_DB_NAME	空串	静态	服务器可以忽略的数据库名列表，各数据库名以逗号“,”隔开，数据库名不需要加引号
MS_PARSE_PERMIT	0	静态	是否支持 MS SQLSERVER 的语法。 0：不支持；1：支持； 2：在 MS_PARSE_PERMIT =1 的基础上，兼容 MS SQLSERVER 的查询项中支持“标识符=列名”或“@变量名=列名”用法。 注：当 COMPATIBLE_MODE=3 时，若 MS_PARSE_PERMIT 置为 0，系统自动将其实际值改为 2
COMPATIBLE_MODE	0	静态	是否兼容其他数据库模式。0：不兼容，1：兼容 SQL92 标准，2：部分兼容 ORACLE（需要与

			FLOAT_MODE=1 同时开启），3：部分兼容 MS SQL SERVER，4：部分兼容 MYSQL，5：兼容 DM6，6：部分兼容 TERADATA，7：部分兼容 POSTGRES，8：部分兼容 DB2 注：对 COMPATIBLE_MODE 的修改会影响数据存储和操作结果，修改须慎重；具体可咨询达梦技术服务人员
ORA_DATE_FMT	0	静态	指定 DATE 类型格式。 0：DATE 类型使用 NLS_TIMESTAMP_FORMAT 格式，此时动态修改 NLS_DATE_FORMAT 时，实际修改的是 NLS_TIMESTAMP_FORMAT； 1：DATE 类型使用 NLS_DATE_FORMAT 格式。 注：仅当 COMPATIBLE_MODE 为 2 时有效，建议当 COMPATIBLE_MODE 为 2 将 ORA_DATE_FMT 置为 1
ORA_REVERSE_MODE	1	静态	指定 ORACLE 兼容模式下 REVERSE()函数的反转规则。0：按字节反转，与 ORACLE 相同，与旧版本兼容 ORACLE 模式下的规则保持一致；1：按字符反转，与 DM 默认的 REVERSE()规则一致。 注：仅当 COMPATIBLE_MODE 为 2 时有效
JSON_MODE	0	静态	JSON 语法解析兼容模式。0：兼容 ORACLE；1：兼容 PostgreSQL；2：兼容 MYSQL
JSONB_CHECK_MODE	1	动态，系统级	所生成的 JSONB 数据的校验模式。0：不校验；1：CRC32 校验

JSON_OPTION	0	静态	调整 JSON 相关规则，按位赋值，目前仅第一位可用。 0：不做调整; 1：JSON_OBJECT 函数不受 JSON_MODE 参数取值的影响，只支持 JSON_MODE=2 时的规则。 注：此参数的目的是强行修改常规的 JSON 相关规则，用于一些特殊场景，例如需要兼容 V8.1.4.107 之前的版本的 JSON_OBJECT 函数用法。在 V8.1.4.107 之前的版本中，JSON_OBJECT 函数只有一套规则，即当前 JSON_MODE=2 时的规则
ARGUMENT_MATCHING_MODE	1	动态， 系统级	过程/函数参数匹配规则。 0：顺序匹配,但需要保留足够的值能分配给不含默认值的参数; 1：顺序匹配
USE_JSON_DATATYPE	1	静态	控制内建数据类型 JSON 是否启用。0：不启用; 1：启用
DATETIME_FMT_MODE	0	动态， 系统级	是否兼容 ORACLE 的五种日期时间类型的默认日期时间格式。0：不兼容；1：兼容。 注：此参数被动态修改后仅影响新生成的会话，仅在 NLS_XXX 相关参数为默认值时才有效，并且 DATETIME_FMT_MODE 设置时，会忽略其他 ORACLE 兼容参数 COMPATIBLE_MODE 和 ORA_DATE_FMT 的设置。五种日期时间类型的日期时间格式包括 NLS_DATE_FORMAT、NLS_TIME_FORMAT、NLS_TIME_TZ_FORMAT、NLS_TIMESTAMP_FORMAT 和 NLS_TIMESTAMP_TZ_FORMAT。不兼容时，采

			用 DM 的默认日期时间格式；兼容时，采用 ORACLE 的默认日期时间格式，可通过 V$NLS_PARAMETERS 查看对应的默认日期时间格式
DOUBLE_MODE	0	静态	计算 DOUBLE 类型的散列值时，是否只使用 6 个字节。0：否；1：是
CASE_COMPATIBLE_MODE	1	动态，系统级	涉及不同数据类型的 CASE 运算，是否需要兼容 ORACLE 的处理策略。 0：不兼容； 1：兼容，本模式下，当函数 DECODE()中的多个 CASE 类型不一致时，DECODE 会从其中选择一个类型进行匹配； 2：兼容，本模式下，当函数 DECODE()中的多个 CASE 类型不一致时，DECODE 根据第一个 CASE 的类型来决定匹配类型； 4：兼容，本模式下，当 CASE WHEN 分支中的 CASE 类型不一致时，进行报错； 8：在参数取值包含 2 时生效，WHEN 表达式的最终数据类型为 CASE 表达式与首个 WHEN 表达式数据类型之间的强者。 16：兼容 ORACLE，COALESCE 应当包含两个及以上的参数。 支持使用上述有效值的组合值，如 5 表示同时进行 1 和 4 的优化。

			注：取值 1 和 2 是互斥的，取值为组合值 3 时，会根据系统内部逻辑判断处理为 1 或者 2
XA_COMPATIBLE_MODE	0	动态， 系统级	XA事务处理兼容策略。 0：不兼容，返回达梦的错误码； 1：兼容ORACLE的处理方式，返回和ORACLE一样的报错码； 2：兼容MYSQL的XA事务处理策略，取值2时不支持使用DBMS_XA系统包
EXCLUDE_RESERVED_WORDS	空串	静态	语法解析时，需要去除保留字身份的保留字列表，保留字之间用逗号分隔。去除身份的保留字将变成普通字符串 如果去除了保留字身份，可能导致包含相关保留字的 SQL 语句执行失败
COUNT_64BIT	1	动态， 会话级	COUNT 和 SUM 集函数的值是否设置为 BIGINT。0：否；1：是。 当 COUNT_64BIT 取值为 0 时，返回结果超过 INT 范围时会报错
CALC_AS_DECIMAL	0	静态	0：默认值，表示整数类型的除法、整数与字符或 BINARY 串的所有四则运算，结果都处理成整数； 1：表示将整数类型的除法、整数与字符或 BINARY 串的除法转换为 DECIMAL 处理； 2：表示将整数类型的除法、整数与字符或 BINARY 串的所有四则运算都转换为 DECIMAL 处理； 3：所有涉及整型的四则运算全部转换为 DECIMAL 处理。

CMP_AS_DECIMAL	0	静态	0：默认值，表示不对字符串与整型的比较结果类型做任何修改 1：表示字符串与整型转换为 BIGINT 比较数据溢出时，则转换为 DEC 进行比较 2：表示字符串与整型的比较强制转换为 DEC 进行比较
CAST_VARCHAR_MODE	1	动态， 系统级	在字符串向整型转换进行比较时，是否将溢出值转为特殊伪值。0：否；1：是
PL_SQLCODE_COMPATIBLE	0	静态	默认值为 0；如果设置为 1，则 PL 的异常处理中，SQLCODE 的错误码值需要尽量与 ORACLE 一致
LEGACY_SEQUENCE	0	动态， 系统级	序列兼容参数，0：与 ORACLE 兼容；1：与旧版本 DM 兼容，不推荐使用
DM6_TODATE_FMT	0	静态	TO_DATE 中的 HH 格式小时制，0：12 小时制； 1：24 小时制（与 DM6 兼容）
MILLISECOND_FMT	1	动态， 系统级	TO_CHAR 的结果是否显示小数秒值。0：不显示；1：显示。仅在 DATETIME_FMT_MODE 和 NLS_XXX 相关参数均为默认值时有效
NLS_DATE_FORMAT	无	静态	指定 DATE 类型的显示格式，包括 DATE 列 INSERT/SELECT 使用的格式、CAST 转换采用的格式。不设置时格式取决于 DATETIME_FMT_MODE 参数

NLS_TIME_FORMAT	无	静态	指定 TIME 类型的格式，包括 TIME 列 INSERT/SELECT 使用的格式、TO_CHAR 的显示格式、CAST 转换采用的格式。不设置时格式取决于 DATETIME_FMT_MODE 参数
NLS_TIMESTAMP_FORMAT	无	静态	指定 TIMESTAMP 类型的格式，包括 TIMESTAMP 列 INSERT/SELECT 使用的格式、TO_CHAR 的显示格式、CAST 转换采用的格式。不设置时格式取决于 DATETIME_FMT_MODE 参数
NLS_TIME_TZ_FORMAT	无	静态	指定 TIME_TZ 类型的格式，包括 TIME_TZ 列 INSERT/SELECT 使用的格式、TO_CHAR 的显示格式、CAST 转换采用的格式。不设置时格式取决于 DATETIME_FMT_MODE 参数
NLS_TIMESTAMP_TZ_FORMAT	无	静态	指定 TIMESTAMP_TZ 类型的格式，包括 TIMESTAMP_TZ 列 INSERT/SELECT 使用的格式、TO_CHAR 的显示格式、CAST 转换采用的格式。不设置时格式取决于 DATETIME_FMT_MODE 参数
PK_MAP_TO_DIS	0	动态，系统级	专门用于 MPP 环境中，在建表语句中指定了 PK 列，没有指定分布方式时，是否自动将 PK 列作为 HASH 分布列，将整个表转为 HASH 分布。1：是；0：否
MY_STRICT_TABLES	1	静态	仅在 COMPATIBLE_MODE=4 时有效。支持 HINT 指定。每 BIT 控制一种如下情况执行失败时的处理策略：第一位控制数据超长；第二位控制字符类型转换数值类型（包括 INT、SMALLINT、TINYINT、BIGINT、DEC、FLOAT、DOUBLE）；第三位控制除 0。每 BIT 为 0 代表兼容 MYSQL 容错，为 1 代表相应策略报错。第四位控制 CAST AS CHAR，第五位控制 CAST AS VARCHAR。

			0：最大容错，数据超长时自动截断；字符类型转换数值类型失败时，转换为 0；除 0 时返回 NULL； 1：数据超长时报错； 2：字符类型转换数值类型失败时报错； 4：除 0 时报错； 8：使用 CAST AS CHAR(不指定精度)时，变为 CHAR(1),否则兼容 MYSQL 变为 VARCHAR； 16：使用 CAST AS CHAR(N)时，变为 CHAR(N)，否则兼容 MYSQL 变为 VARCHAR(N)； 支持上述有效值的组合值，如 7 表示数据超长时报错、字符类型转换数值类型失败时报错、除 0 时报错。
IN_CONTAINER	0	静态	当前数据库是否运行于容器中，0：否；1：是。若当前数据库运行于容器中，则将根据 CGROUP 的限制规则分配 CPU 和内存资源
NUMBER_MODE	0	静态	控制对 NUMBER、FLOAT、REAL、DOUBLE PRECISION 类型在数据定义（包括表列定义、脚本参数定义等）时的处理方式；对于外部函数的返回值及其参数类型，忽略 NUMBER_MODE 的配置。0：采用 DM 现有的处理方式；1：兼容 ORACLE 的处理方式
NLS_SORT_TYPE	0	动态，会话级	指定自然语言的排序方式。0/6：按默认字符集二进制编码排序；1：按中文拼音排序；2：按中文笔画排序；3：按中文部首排序；4：按泰文排序；5：按韩文排序

			仅字符集为 UTF-8 的数据库支持自然语言按泰文排序
PROXY_PROTOCOL_MODE	0	动态，系统级	控制代理模式的开启，0：不开启；1：开启。 开启代理模式后，服务器才可以解析代理向服务器发送的代理协议消息，从而才可以获得真实客户端 IP， 并且设置限制连接的 IP 后会检查真实客户端 IP 是否有效， 查询 V$SESSIONS 中的 CLNT_IP 时也会显示真实客户端 IP 地址
SPACE_COMPARE_MODE	0	静态	是否在查询语句中比较字符串后缀空格。0：维持建库参数 BLANK_PAD_MODE 指定的策略；1：进行比较
DATETIME_FAST_RESTRICT	1	动态，系统级	TO_DATE(字符串，FAST 格式)或 ALTER SESSION 设置了 DATE 的格式为 FAST 格式后，CAST 字符串 AS DATE 时： 0：字符串可以带时间；1.字符串不允许带时间，带时间会报错。 其中 FAST 格式（标准格式）为：YYYY-MM-DD、 YYYY/MM/DD、YYYY:MM:DD、YYYY.MM.DD
BASE64_LINE_FLAG	1	动态，系统级	BASE64 编码时，是否每满 64 个字节加换行符 ODOA\N\R 两个字节。0：否；1：是
NVARCHAR_LENGTH_IN_CHAR	1	静态	列定义中的 NVARCHAR 和 NVARCHAR2 数据类型是否自动转换为 VARCHAR(N CHAR)数据类型，NCHAR 类型是否自动转换为 CHAR(N CHAR)类型。0：否；1：是

			NVARCHAR_LENGTH_IN_CHAR 与列类型大小有关，建库后不建议修改，错误的修改可能导致后续 MODIFY 列失败
PARAM_DOUBLE_TO_DEC	0	动态，系统级	是否支持将绑定参数中数据类型为 FLOAT/DOUBLE 的输入参数转换为 DEC 精确数据类型，0：不支持转换；1：支持转换，并进行预检查，预检查成功则转换为 DEC，否则保持原始数据类型；2：支持转换，不进行预检查，若转换失败则报错 目前仅支持单行绑定参数转换，多行暂不支持
INSERT_COLUMN_MATCH	0	动态，系统级	指定插入操作的列匹配规则，0：使用 DM 的列匹配规则；1：兼容 ASTER 的列匹配规则；2：兼容 Postgres 的列匹配规则。 值为 1 时，值插入支持列使用逗号占位符，查询插入暂不支持使用逗号占位符。 值为 1 或 2 时，支持尾部列不完全匹配插入。 若参数打开再关闭，如果语句已生成执行计划则需要手动清理，否则当列不匹配时，预期失败实际成功
AUTO_INCREMENT_INCREMENT	1	动态，会话级	用于 AUTO_INCREMENT 自增列中。表示 AUTO_INCREMENT 的步长。取值范围 1~65535
AUTO_INCREMENT_OFFSET	1	动态，	用于 AUTO_INCREMENT 自增列中。表示 AUTO_INCREMENT 的基准偏移。取值范围 1~65535

		会话级	
NO_AUTO_VALUE_ON_ZERO	1	动态， 会话级	用于 AUTO_INCREMENT 自增列中。表示 AUTO_INCREMENT 列插入 0 时，是否自动插入自增的下一个值。取值范围 0、1。0 否，插入 0；1 是，插入自增值
IFUN_DATETIME_MODE	0	静态	指定返回值或参数为 TIMESTAMP/TIMESTAMP WITH TIME ZONE 类型的系统函数的最大精度。0：老版本模式，最大精度为 6；1：新版模式，最大精度为 9 注：IFUN_DATETIME_MODE 为 0 时，毫秒精度最大为 6。为了向下兼容老版本，将字符串转换为时间类型时，当毫秒精度大于 6 且小于等于 9 时直接截断处理，仅保留前 6 位有效精度；若定义时间类型的精度大于 6 且小于等于 9 时，大于 6 的部分用 0 补齐； IFUN_DATETIME_MODE 为 1 时，老版本客户端连接新版本服务器可能会导致版本不匹配报错
CAST_DIGIT_MODE	0	动态， 系统级	当弱数值类型的列与强数值类型的常量比较时，是否将常量的类型转为列的类型。0：否；1：是。该参数支持的数据类型，按由弱到强的顺序依次为 TINYINT、SMALLINT、INT、BIGINT、DECIMAL、FLOAT、DOUBLE、DATE 和 TIMESTAMP
VIEW_ACCESS_MODE	0	静态	指定视图的自主访问控制机制。0：兼容 DM；1：兼容 ORACLE
ENABLE_FULL_WIDTH	1	动态，	是否识别 SQL 语句中的全角中文字符。0：不识别；1：识别

		系统级	
DECIMAL_ENHANCED	1	静态	指定 DECIMAL、DEC、NUMERIC 和 NUMBER 数据类型计算结果的有效精度。0：支持 38 位精度；1：支持 40 位精度
DOUBLE_TO_DEC_MASK	1	静态	指定 DOUBLE/FLOAT 转换 DECIMAL 时使用的格式。 0：统一使用%g 自动格式。%g 自动格式具体分为方式一（%g = (%e +其他操作)）和方式二（%g = (%f + 其他操作)）； 1：将 0 选项中方式一替换成直接使用%e 格式，其他不变； 2：使用浮点数矫正算法，还原成最短且最接近的 DECIMAL
ARG_VARCHAR_ADJUST	0	动态， 系统级	用于控制是否调整过程/包方法的解析策略。 0：保持原有策略； 1：调整过程/包方法解析策略，放大变长和定长字符串参数长度，将变长和定长字符串参数/返回值的长度放大到 32767； 2：兼容 ORACLE 的过程/函数/包方法的 CHAR、VARCHAR、RAW(VARBINARY)类型的定义处理，忽略指定长度，涉及到字符串比较时，需要与 COMPATIBLE_MODE=2 配合使用

CASE_CONVERSION_ENHANCED	1	静态	是否支持多字节字符的大小写转换。0：不支持，表示只支持原有的单字节 ASCII 字符的大小写转换；1：支持，表示增加支持多字节字符的大小写转换
TIME_MODE	1	静态	指定 TIME 解析兼容模式。0：兼容 DM 老版本对于 TIME 的默认解析方式，统一从年开始解析；1：根据参数 COMPATIBLE_MODE 取值处理 TIME 的解析方式，从年或小时开始解析
CAST_CLOB_MODE	1	动态，会话级	对于目标数据类型为 CLOB 的超长字符串"||"连接，是否下放数据类型转换。0：不下放；1：下放
MD5_TYPE	0	静态	用于控制 MD5 函数的返回值类型（MD5 函数用于计算并返回源串的 MD5 散列值，具体可查看《DM8_SQL 语言使用手册》）。0：返回值类型为 VARBINARY；1：返回值类型为 VARCHAR
VIEW_AUTHID_CHECK	1	动态，系统级	用于控制当视图定义包含 AUTHID CURRENT_USER 时是否报错。0：否；1：是
ARG_DECIMAL_ADJUST	0	动态，系统级	专门用于存储过程/存储函数/包中的函数。用于控制是否调整函数输入参数的解析策略。0：否；1：是，系统将 NUMBER 类型的输入参数自动调整为 DEC(0,0)类型，参数允许输入任何有效的 NUMBER 数据，此策略调整不包括返回值数据类型
DEC_BIN_CVT_MODE	0	静态	用于控制 NUMBER 与 BINARY 类型的转换。0：表示原有的处理逻辑，即只支持从 BINARY 向 NUMBER 转换且

			BINARY 数据按 INT64 处理；1：表示新的处理逻辑，支持互相转换，且 BINARY 数据记录的是 NUMBER 类型的内存值。0 和 1 不同取值表示两种不兼容的转换格式，其 BINARY 存储的数据含义不同，使用时应确定某一种，不应该在已有数据的情况下调整配置值
RECORD_COMPARE_MODE	1	静态	是否严格比较记录数据类型的成员，0 表示简单比较，1 表示严格比较。严格比较时，同样的数据类型如字符串，如果定义长度不一致，视为不相同
BINARY_AS_HEX	1	静态	控制 UNHEX 函数的返回值类型。0：VARCHAR；1：VARBINARY
DEFAULT_WEEK_FORMAT	0	动态, 会话级	指定WEEK(DATE)和EXTRACT(WEEK FROM DATE)的规则，规则与WEEK(DATE, MODE)一致，参数可取值以及含义如下: 0：周起始日为周日，返回值范围为0~53，具体规则为本年第一个周日开始为第1周，之前算本年第0周 1：周起始日为周一，返回值范围为0~53，具体规则为本年第一个周一之前如果超过3天则算第1周，否则算第0周 2：周起始日为周日，返回值范围为1~53，具体规则为本年第一个周日开始为第1周，之前算去年第5x周 3：周起始日为周一，返回值范围为1~53，具体规则为本年第一个周一之前如果超过3天则算第1周，否则算去年第5x周；年末不足4天算明年第1周

			4：周起始日为周日，返回值范围为0~53，具体规则为本年第一个周日之前如果超过3天则算第1周，否则算第0周 5：周起始日为周一，返回值范围为0~53，具体规则为本年第一个周一开始为第1周，之前算本年第0周 6：周起始日为周日，返回值范围为1~53，具体规则为本年第一个周日之前如果超过3天则算第1周，否则算去年第5X周；年末不足4天算明年第1周 7：周起始日为周一，返回值范围为1~53，具体规则为本年第一个周一开始为第1周，之前算去年第5x周
LITERAL_PREFIX	1	动态, 会话级	用于设置在对文本串进行语法解析时，是否允许文本串前面出现无意义的前缀信息，0表示不允许，1表示允许
OBJECT_PRIV_MODE	0	静态	是否兼容ORACLE的对象权限授权回收规则。默认：0（否，用DM规则）。1:是。
CASE_TYPE_MODE	0	静态	控制CASE语句中串类型的目标类型是VARCHAR还是TEXT。0：VARCHAR，1：TEXT。
REGISTER_TEXT_IFUN	1	静态	是否注册部分TEXT函数。0：不注册；1：注册。 默认注册，受该参数控制的TEXT函数包括：INITCAP\LEFTSTR\LEFT\REPEATSTR\RIGHTSTR\RIGHT\NLS_UPPER\NLS_LOWER\OVERLAY\CONCAT_WS\REVERSE\CONVERT

BLOCK_ENCRYPTION_MODE	AES-128-ECB	动态, 会话级	指定系统函数 AES_ENCRYPT、AES_DECRYPT 使用的加密算法。参数值书写格式为 AES-keylen-mode，其中 keylen 可取值 128、192、256，mode 可取值 ECB、CBC、CFB、OFB、CFB1、CFB8。参数取值示例 AES-192-CBC
DM_STRICT_TABLES	1	动态, 系统级	是否使用严格的类型检查模式。 0：兼容 DM 老版本的类型检查表现； 1：OUT 及 IN OUT 参数的结果将严格按照绑定变量的类型进行检查。 注：该参数的优先级高于兼容模式参数 COMPATIBLE_MODE。该参数用于兼容 DM 自身老版本的表现，若无特殊需求不建议修改；且该参数涉及到执行计划生成，虽然是系统级参数，但不会自动清理之前已生成的计划缓存，如果需修改该参数，修改完成后需要管理员手动清理计划缓存。
RESERVED_WORDS_MODE	1	静态	控制关键字是否作为保留字。 0：存在歧义时间日期的函数名作为保留字：SYSDATE、SYSTIMESTAMP、CURRENT_DATE、CURRENT_TIMESTAMP、LOCALTIMESTAMP、DBTIMEZONE、SESSIONTIMEZONE。如果只需要部分存在歧义日期时间的函数名作为保留字，可配合 EXCLUDE_RESERVED_WORDS 剔除其余单词； 1：存在歧义的时间日期函数名作为非保留字；

			2：WINDOW 关键字作为非保留字； 4：MODEL、DIMENSION 关键字作为非保留字。 支持使用上述有效值的组合值，如 5 表示同时控制 1 和 4 的关键字作为非保留字
ROWCOUNT_FLAG	0	动态，会话级	控制所有返回消息的表现。 0：批量执行返回最后一行 DML 的影响行数； 1：返回所有 DML 的累计影响行数
DATETIME_STRICT_FLAG	1	静态	时间容错开关，优先于 COMPATIBLE_MODE。0：DM 规则（宽松）；1：兼容其他数据库规则（严格）。在 DM 规则下支持以下场景的使用： ● 对于 12 小时制，源串中小时取值为 00 有效； ● 源串中时间数字之间可以不带分隔符； ● COMPATIBLE_MODE=2，TO_DATE 指定的日期格式不带 FF 时，允许源串中带毫秒； ● COMPATIBLE_MODE=7，日期格式中日和时之间有分隔符，但源串中对应部分可以没有分隔符； ● COMPATIBLE_MODE=7，日期格式中的分隔符（非限定、限定）的字节数只需大于等于源串中对应位置的字符数即可，不需要严格遵循分隔符匹配规则，后缀空白符不参与比较，前缀空白符参与比较。

NESTED_C_STYLE_COMMENT	0	动态, 系统级	是否支持 C 风格的嵌套注释。0：不支持；1：支持
DEC_CAST_FORMAT	1	静态	纯小数 DEC/DECMAIL/NUMBER 类型转为字符类型时，是否忽略前导 0。 0：否； 1：当 COMPATIBLE_MODE=2，兼容 ORACLE 模式时，忽略前导 0

2.1.1.1.15 用户请求跟踪相关

表2.16 dm.ini 中用户请求跟踪相关参数

参数名	缺省值	属性	说明
SVR_LOG	0	动态，系统级	是否打开 SQL 日志功能，0：关闭；1：打开，并按照 SQLLOG.INI 中的配置来记录 SQL 日志；2：打开，按文件中记录数量切换日志文件，日志记录为详细模式；3：打开，不切换日志文件，日志记录为简单模式，只记录时间和原始语句
SVR_LOG_NAME	SLOG_ALL	动态，系统级	SQLLOG.INI 中预设模式的名称。支持指定多个预设模式名，限制如下: 1、最多指定 10 个预设模式名； 2、预设模式名之间使用逗号分隔，不允许换行；

			3、每个预设模式名长度不能超过 128 个字节，否则该模式名无效； 4、dm.ini 中 SVR_LOG_NAME 参数总长度不能超过 256 个字节，否则使用 SLOG_ALL 模式。动态修改 SVR_LOG_NAME 时，若指定的参数值超过 256 字节则报错； 5、指定多个预设模式名时，若其中某个模式名不在 SQLLOG.INI 文件中，则该模式名无效。若指定的多个预设模式名均不在 SQLLOG.INI 文件中，则使用<DEFAULT>模式 例如：SVR_LOG_NAME = SLOG_ALL,SLOG_LOGIN
SVR_LOG_PLN_STR	0	动态，会话级	控制 SQL 日志中是否打印 SQL 计划 0 表示不打印 1：表示打印计划 2：表示打印动态语句的计划 4：记录 SQL 操作符真实返回行数。 取值非 0 则会开启 1 的功能，取值 2 和 4 的功能按位判断是否开启
ELOG_REPORT_LINK_SQL	0	动态，会话级	是否记录 DBLINK 执行的 SQL 到服务器日志文件中。0：不记录；1：记录

2.1.1.1.16 系统跟踪相关

表2.17 dm.ini 中系统跟踪相关参数

参数名	缺省值	属性	说明
GDB_THREAD_INFO	0	静态	系统强制 HALT 时，是否打印线程堆栈信息到日志文件中。0：不打印；1：打印
TRACE_PATH	SYSTEM_PATH	静态	存放系统 TRACE 文件的路径。不允许指定 ASM 路径。默认的 TRACE_PATH 是 SYSTEM_PATH；如果 SYSTEM_PATH 保存在 ASM 上，则../CONFIG_PATH/TRACE 作为 TRACE_PATH
SVR_OUTPUT	0	动态，系统级	print 指令是否在服务器端打印，打印信息会记录到 log 目录下的 DmServerSerivce.log 文件中。1 是，0 否
SVR_ELOG_FREQ	0	动态，系统级	指定 ELOG 日志文件归档时的切换频率，满足条件后系统会自动将当前的 ELOG 日志文件更名后归档。0：按月，格式为 DM_实例名_年月.log；1：按天，格式为 DM_实例名_年月日.log；2：按小时，格式为 DM_实例名_年月日_小时.log；3：按文件体积（需配置 INI 参数 ELOG_SWITCH_LIMIT），当达到设置的体积上限时进行切换，格式为 DM_实例名_年

			月日_小时_分钟.log
ELOG_FLAG	0	手动	指定当前的 ELOG 日志文件名是否带有时间后缀。 0：是，格式为 dm_实例名_年月.log； 1：否，格式为 dm_实例名.log
ENABLE_OSLOG	0	静态	是否启用操作系统日志，启用后会在/var/log/secaudit/db 目录下生成名为 audit_yyyymmdd.log 的操作系统日志文件，仅对非 WINDOWS 系统有效。0：禁用；1：启用
LOG_IN_SHOW_DEBUG	2147483647	动态， 系统级	发版为DEBUG版本时，控制RLOG模块日志输出。取值范围0~2147483647。 LOG_IN_SHOW_DEBUG 转换为二进制后，最低位为 1，表示开启 RLOG 模块日志输出；0 表示关闭 RLOG 模块日志输出。 其他位，暂时没有用到
TRC_LOG	0	动态， 系统级	TRACE 日志开关。需和 INI 参数 TRC_LOG_MODULE 配合使用。 0：表示关闭； 非 0：为功能组合值，组合规则为： SWITCH_MODE*4 + ASYN_FLAG*2 + 1。其中，<SWITCH_MODE>日志文件切换方式： 0 不切换；1 按文件记录

			行数，满 10000 行切换；2 按文件大小进行切换，满 128M 切换； 3 按时间间隔进行切换，每小时切换。 <asyn_flag>日志文件是否采用异步方式：0：否，采用同步方式；1：是，采用异步方式 。 TRACE 日志位于安装目录下的/log 中，名称为 dmtrc_instname.log（不含切换方式）或 dmtrc_instname_日期时间.log（含有切换方式）
TRC_LOG_MODULE	ALL	动态， 系统级	当打开 TRC_LOG 开关时，用于配置 TRACE 日志中记录的内容。该参数为字符类型，最大长度 128。 书写格式为：TRC_LOG_MODULE=<服务器记录项>,<DMDPC 记录项>。 服务器记录项和 DMDPC 记录项设置方法相同。缺省表示不记录该模块中的任何内容；若配置时省略逗号','，则表示服务器与 DMDPC 的记录项配置相同。 具体记录项和代号：内存 MEM（0）、任务线程 TSK（1）、站点信息 SITE（2）、事务 TRX（3）、通信消息 MSG（4）、会话 SESS（5）、计划 PLN_GEN（6）、HUGE 文件系统 HFS（7）、LOCK 相关(8)、快速装载服务器端模块 BLDR(9)、字典 DICT(10)、错误堆栈 ESTK(11)、用户自定义向日志写入内容(12)、DSC 相关(13)、计划缓存失效相关(14)、0~14 全部记录(ALL)。多个记录项之间使用冒号(:)分割。 例如： 1）记录服务器模块与 DMDPC 模块 0~12 项所有的信息，需设置 TRC_LOG_MODULE = ALL

			2）只服务器模块记录 HFS 和 TRX，DMDPC 模块不记录，需设置 TRC_LOG_MODULE= 3:7, 3）只 DMDPC 模块记录 MSG，服务器模块不记录，需设置 TRC_LOG_MODULE=,4 4）若服务器记录 HFS 和 TRX，DMDPC 记录 MSG，需设置 TRC_LOG_MODULE=3:7,4
ELOG_SWITCH_LIMIT	0	动态， 系统级	事件日志文件体积上限，单位 MB，取值范围 0~4294967294。只在 INI 参数 SVR_ELOG_FREQ 设置为 3 时生效；配置为 0 时不限制日志文件体积
ELOG_ERR_ARR	空串	静态	指定需要生成 ELOG 日志的错误码，最多可以指定 10 个错误码，各错误码之间使用“,”进行分隔
ELOG_LANGUAGE_FLAG	0	静态	用于设置服务器启动和退出时的 ELOG 语言的标记。0：英文，1：中文
CORE_DUMP_FLAG	1	动态， 系统级	在系统崩溃时，是否 DUMP 线程信息和 SQL 语句。若打开该参数，则在系统崩溃时，将线程信息和 SQL 语句导出到系统 LOG 目录下的 DMHALT_<实例名>_<进程号>_年月日_小时分钟秒.DUMP 文件中。1：是；0：否

2.1.1.1.17 MONITOR 监控相关

表2.18 dm.ini 中 MONITOR 监控相关参数

参数名	缺省值	属性	说明
ENABLE_MONITOR	1	动态，	用于打开或者关闭系统的监控功能。1：打开；0：关闭

		系统级	
MONITOR_SQL_EXEC	0	动态， 会话级	操作符、虚拟机栈帧、执行计划节点的监控开关。该监控项的生效必须是在 ENABLE_MONITOR 打开的情况下。0：关闭监控；1：打开监控；2：打开监控，在 1 的基础上，又增加了表达式运行时操作符的统计。该参数涉及的动态视图为 V$STKFRM、V$SQL_PLAN_NODE、V$SQL_NODE_HISTORY
LARGE_MEM_THRESHOLD	10000	动态， 系统级	大内存监控阈值。单位 KB，取值范围 0~10000000。其中 0~100 关闭统计，100 以上才统计。 一条 SQL 语句使用的内存值超过这个值，就认为是使用了大内存，此时开启大内存监控。使用了大内存的 SQL 语句记录在 V$LARGE_MEM_SQLS， V$SYSTEM_LARGE_MEM_SQLS 视图中
ENABLE_MONITOR_DMSQL	1	动态， 会话级	启用动态监控 SQL 执行时间功能标记， 0：不启用；1：监控。只监控多条或复杂 SQL，例如包含引用包、嵌套子过程、子方法、动态 SQL 的 SQL 语句。监控的记录在 V$DMSQL_EXEC_TIME 视图中
ENABLE_TIMER_TRIG_LOG	0	动态， 系统级	是否启动作业时间触发器日志功能，0：不启动；1 启动
IO_TIMEOUT	300	静态	设置 IO 超时时间。单位秒，取值范围 0 或 60~3600。0 表示关闭 IO 超时检测。若设置大于 0，调度线程每分钟检测系统内部是否存在 IO 超时（读磁盘或写磁盘）的数据页信息或者 REDO 日志刷盘超时信息，如果存在则将信息写入日

			志文件。 IO 读超时受 INI 参数 PTX_ROLLBACK 控制，若 PTX_ROLLBACK=0，则当 IO 读时间大于 IO_TIMEOUT 设定值时，系统自动 HALT；若 PTX_ROLLBACK=1，则当 IO 读时间大于 IO_TIMEOUT 设定值的一半时，系统报错处理。IO 写超时不受 INI 参数 PTX_ROLLBACK 控制，当 IO 写时间大于 IO_TIMEOUT 设定值时，系统自动 HALT。 REDO 日志刷盘超时检测在不同环境中应用不同的策略，在单机环境下，当 REDO 日志刷盘时间大于 IO_TIMEOUT 设定值时，系统自动 HALT；在 DSC 集群环境下，当 REDO 日志刷盘时间大于 IO_TIMEOUT 设定值的一半或 DSC_SLOT_WAIT_TIMEOUT 设定值的一半时，系统自动 HALT
ENABLE_MONITOR_BP	1	动态，系统级	是否监控绑定参数相关信息。该监控项的生效必须是在 ENABLE_MONITOR 打开的情况下。0：不监控；1：监控
SQL_HISTORY_CNT	10000	动态，系统级	动态视图 V$SQL_HISTORY 的记录数上限，取值范围 1000~100000
LONG_EXEC_SQLS_CNT	1000	动态，系统级	动态视图 V$LONG_EXEC_SQLS 的记录数上限，取值范围 1000~1000000
SYSTEM_LONG_EXEC_SQLS_CNT	300	动态，	动态视图 V$SYSTEM_LONG_EXEC_SQLS 的记录数上限，取值范围 10~1000

		系统级	
BP_ITEM_MAXSIZE	50	动态， 系统级	动态视图 V$SQL_BINDDATA_HISTORY 的 BINDDATA 列的最大内存空间。取值范围 1~500，单位 KB
MONITOR_SQL_PARSE	0	动态， 会话级	查询分析过程信息的监控开关。该监控项须在参数 ENABLE_MONITOR=1 的条件下生效。取值：0：关闭监控；1：打开监控。可在动态视图 V$SQL_PARSE_HISTORY 中查看监控到的查询分析过程信息
RT_ERR_HISTORY_CNT	5000	动态， 系统级	动态视图 V$RUNTIME_ERR_HISTORY 的记录数上限，取值范围 1000~100000
ENABLE_MONITOR_PLNHIST	0	动态， 会话级	是否开启历史计划信息持久化功能。0：否；1：是。当 USE_PLN_POOL!= 0 时有效
DMSQL_ET_CNT	10000	动态， 系统级	指定 V$DMSQL_EXEC_TIME 的最大记录数，用于控制 V$DMSQL_EXEC_TIME 的容量。取值范围 1~100000
DMSQL_EXEC_THRESHOLD	0	动态， 会话级	用于提供监控 DMSQL 执行的时间阈值,当 DMSQL 中执行时间超过阈值的语句会被记录。取值范围 0~4294967294，单位毫秒，其中 0 表示不进行监控
AP_PORT_NUM	0	静态	分布式环境下协同工作的监听端口，SP/BP 本地设置的 AP_PORT_NUM 如果和自身在 MP 上注册的

			AP_PORT_NUM 不一致，按照在 MP 上实际注册的为准，启动时会自动覆盖
BINDDATA_COLTYPE	0	静态	用于设置动态视图 V$SQL_BINDDATA_HISTORY 的 BINDDATA 列的数据类型。0：VARBINARY；1：BLOB
CACHEITEM_ALLOC_MONITOR	0	动态，系统级	用于打开或者关闭不在 SQL 缓冲池中但未释放的缓存项的内存分配监控功能。0：关闭；1：监控未释放计划；2：监控未释放包信息；4：监控未释放结果集。支持使用上述有效值的组合值，如 3 表示同时开启 1 和 2 的功能，监控未释放的计划和包信息
MONITOR_PLNHIST_THRESHOLD	1000	动态，会话级	历史执行计划的单次处理上限。在 ENABLE_MONITOR_PLNHIST 开启时有效。取值范围 1000~10000。当待处理的历史执行计划数量超过该数值时，用新产生的历史执行计划信息替换其中旧的历史执行计划信息

2.1.1.1.18 数据守护相关

表2.19 dm.ini 中数据守护相关参数

参数名	缺省值	属性	说明
DW_MAX_SVR_WAIT_TIME	0	动态，系统级	数据库等待守护进程启动的最大时间，取值范围 0~65534，单位 S。如果设定时间内，守护进程没有启动，数据库实例强制退出。0 表示不检测
DW_INACTIVE_INTERVAL	60	动态，	服务器认定守护进程未启动的时间，取值范围 0~1800，单位秒。

		系统级	如果服务器距离上次收到守护进程消息的时间间隔在设定的时间范围内，则认为守护进程处于活动状态，此时，不允许手工执行修改服务器模式、状态的 SQL 语句； 如果超过设定时间仍没有收到守护进程消息，则认为守护进程未启动，此时，允许手工方式执行这类 SQL 语句
DW_PORT	0	静态	服务器监听守护进程连接请求的端口，服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配。 数据守护 V3.0 版本中，此参数仅作为单节点库的兼容参数保留。 对于新配置的数据守护系统，如果是单节点库，建议改用 DMMAL.INI 中的 MAL_INST_DW_PORT 参数进行配置，如果是 DMDSC 集群，此参数不再有用，必须使用 DMMAL.INI 中的 MAL_INST_DW_PORT 参数进行配置
ALTER_MODE_STATUS	1	动态， 系统级	是否允许手工修改服务器的模式、状态和 OGUID。2 表示允许，1 表示在本地守护进程故障时允许，0 表示不允许。 注：数据守护环境下建议配置为 0，实例处于主机或备机模式后，不允许用户直接通过 SQL 语句修改服务器的模式、状态和 OGUID
ENABLE_OFFLINE_TS	1	动态， 系统级	是否允许 OFFLINE 表空间，0：不允许；1：允许；2：备库不允许。 注：数据守护环境下建议配置为 2
SESS_FREE_IN_SUSPEND	60	动态，	到备库的归档失败会导致主库挂起，为了防止应用的连接一直挂住不切换到新主库，设置该参数，表示归档失败挂起后隔

		系统级	一段时间自动断开所有连接。取值范围 0~1800，单位秒，0 表示不断开
SUSPEND_WORKER_TIMEOUT	180	动态， 系统级	在一些ALTER DATABASE等操作过程中，需要先暂停所有工作线程，如果超过此设置时间，仍有部分工作线程在执行中，则会报错，并终止当前操作。取值范围60~3600，单位秒
DW_CONSISTENCY_CHECK	0	动态， 系统级	备库是否检测主库到备库的连接丢失或归档失效，并在检测到上述情况时停止提供只读服务。0：不检测；1：检测
DW_ARCH_SPACE_CHECK	0	动态， 系统级	备库是否检测本地归档磁盘空间不足，并在磁盘空间不足时主动关闭服务，同时将本地守护进程切换为SHUTDOWN状态。 0：不检测；1：检测。 此参数目前仅支持单节点备库的检测
RLOG_PKG_SEND_NUM	100	动态， 系统级	主库启动异步恢复时，连续发送设定值个日志包以后需要等待备库响应。取值范围1~200。缺省值100表示每发送100个日志包就要等待一次响应。如果主备库之间存在比较大的网络延迟，可以通过调大此参数以提升主库的日志发送速度
DW_SUSPEND_TIME	60	动态， 系统级	设置主库在 SUSPEND 状态下最多保持的时长，单位秒，取值范围 0~86400，允许动态修改。 主备进入异步恢复流程时，为了确保备库能够和主库的数据追平，在异步恢复的最后阶段会将主库切换为 SUSPEND。此参数开启时，会在切换 SUSPEND 之前估算剩余待同步的日志量在 DW_SUSPEND_TIME 时长范围内是否能够同步完成，如果可以，再将主库切换到 SUSPEND 状态，否则主库仍然保持 OPEN 状态。

			当此参数配置为大于 0 的值时，需要与 dmwatcher.ini 中的 DW_SUSPEND_CHECK 参数配置使用。只有在 DW_SUSPEND_CHECK 参数也配置为 1 时，守护进程才会在异步恢复切换成 SUSPEND 之前检查主库的剩余日志是否满足 DW_SUSPEND_TIME 的要求；否则只要主库累计 5 次发送归档后就会被守护进程直接切换成 SUSPEND 状态
RLOG_SYNC_MODE	0	静态	设置主库的日志同步模式。0：主库向备库发送日志失败后，需要切换至 SUSPEND 状态，主库守护进程进入 Failover 故障处理流程，设置归档失效后再 OPEN；1：主库向备库发送日志失败后，直接将备库归档失效，不再切换至 SUSPEND 状态，主库守护进程不再进入 Failover 故障处理流程。 参数取值 1 仅在非自动切换模式下有效
RLOG_PKG_SEND_TIME	0	动态， 系统级	当此参数配置在异步备库的源库上时，用于设置源库到异步备库的日志同步检测时间（仅检测异步备库）。取值范围 0~3600，单位秒，0 表示关闭检测。若超出该参数指定的时间还未收到异步备库的响应消息，则源库认为通信出现异常并主动断开连接，等网络恢复正常后源库会自动重建连接并再次启动日志同步。 当此参数配置在异步备库上时，用于设置异步备库的日志接收检测时间。取值范围 0~3600，单位秒，0 表示关闭检测。若异步备库发现距离上一次接收到来自源库的日志包的时间间隔超过此参数配置的值，则认为通信出现异常并主动断开到源库的连接，等待网络恢复正常后源库会自动重建连接并再次启动日志同步
DW_ARCH_HANG_CHECK	0	动态，	主库是否检测本地归档磁盘空间不足，并在磁盘空间不足时主动退出。0：不检测；1：检测。此参数目前仅支持单节点主库

		系统级	的检测
ARCH_SPACE_CHECK_INTERVAL	0	动态，系统级	定期检查归档磁盘空间的时间间隔。单位秒，取值范围 0~86400；取值为 0 时表示不进行该检查。 与参数 ARCH_SPACE_FREE_THRESHOLD 配合使用，当检查到归档磁盘剩余空间小于归档磁盘剩余空间阈值时，打印警告日志；仅当所述两个参数均不为 0 时，此功能才开启
ARCH_SPACE_FREE_THRESHOLD	0	动态，系统级	设置定期检查的归档磁盘的剩余空间阈值。单位 MB，取值范围 0 或 1024~2147483647；取值为 0 时表示不进行该检查。 与参数 ARCH_SPACE_CHECK_INTERVAL 配合使用，当检查到归档磁盘剩余空间小于归档磁盘剩余空间阈值时，打印警告日志；仅当所述两个参数均不为 0 时，此功能才开启
ELOG_ARCH_TIMEOUT	0	动态，系统级	本地归档等待处理超时时间。当归档任务处理耗时或归档发送时间超过该参数设定的值，则打印一条超时日志，单位 MS，取值范围 100~3600000，允许配置为 0；缺省值为 0，表示不检查是否超时。 此参数仅在设置 ENABLE_MONITOR=1 时才生效
REDOS_RECV_PLL_NUM	0	静态	备库在 Recovery 阶段要调整的重演并行度。取值范围 0~256，0 表示不启用。参数开启后，备库在 recovery 期间取 MAX(REDOS_PARALLEL_NUM，REDOS_RECV_PLL_NUM)作为真正的重演并行度使用，recovery 结束后，并行度重新恢复为 REDOS_PARALLEL_NUM，如果备库 REDOS_ENABLE_SELECT=1 或 2 开启查询，则在 recovery 期间会强制关闭查询服务，recovery 结束后再恢复查询，以便备库能快速追赶上主库。此参数仅对 REALTIME 和 TIMELY 备库生效，建议结合机

			器资源情况合理设置参数值。
RPKG_PROCESS_TIMEOUT	0	动态， 系统级	日志包处理超时记录时间。单位毫秒，取值范围 0~3600000。 当取值为 0 时，表示不检查日志包发送接收处理是否超时。 当主库发送时间超过设定的时间，生成一条记录，记录到动态视图 V$RPKG_WARN_SEND_INFO。 当备库接收日志包到发送回复完成时间超过此参数记录到动态视图 V$RPKG_WARN_RECV_INFO
ELOG_ARCH_ASYNC	1	动态， 系统级	控制源库向异步备库发送归档产生的运行日志的级别。 0：仅打印失败或特殊情况的日志同步信息；1：打印详细的日志同步信息

2.1.1.1.19 全文索引相关

表2.20 dm.ini 中全文索引相关参数

参数名	缺省值	属性	说明
CTI_HASH_SIZE	100000	动态, 会话级	使用全文索引查询时，用于设置关键字匹配的哈希表大小。取值范围1000~10000000

CTI_HASH_BUF_SIZE	50	动态, 会话级	使用全文索引查询时，用于设置哈希缓存内存大小，单位MB。取值范围1~4000

2.1.1.1.20 配置文件相关

表2.21 dm.ini 中配置文件相关参数

参数名	缺省值	属性	说明
MAL_INI	0	动态，系统级	是否启用 MAL 系统，0：不启用；1：启用
ARCH_INI	0	动态，系统级	是否启用归档，0：不启用；1：启用
TIMER_INI	0	动态，系统级	是否启用定时器，0：不启用；1：启用
MPP_INI	0	静态	是否启用 MPP 系统，0：不启用；1：启用
DMTHRD_INI	0	动态，系统	是否启用线程绑核功能，0：不启用；1：启用

		级	
DMAP_INI	0	静态	是否启用 DMAP.INI 来控制服务器的备份还原日志切换规则，0：不启用；1：启用

2.1.1.1.21 备份还原相关

表2.22 dm.ini 中备份还原相关参数

参数名	缺省值	属性	说明
BAK_USE_AP	1	动态，会话级	备份还原实现策略。 1：DMAP 辅助进程方式，要求必须启动 dmap 服务，可支持第三方备份。DMAP 插件执行，改造了备份还原任务子系统，大幅提升了备份还原的效率，特别是加密、压缩的处理效率。 2：无辅助进程方式，不依赖 DMAP，由主进程 dmserver 自身执行备份还原，但不支持第三方备份
BAK_DIRECT_IO	0	动态，系统级	备份还原是否启用 DIRECT_IO 参数。0：不启用；1：启用
BAK_SAFE_CHECK	7	动态，系统级	备份的安全检查策略。按位取值，10 进制取值如下： 0：不开启备份安全检查；

			1：数据文件处在 CORRUPT 状态时报错； 2：检查 HUGE 文件错误； 4：增量备份时检查备份集链表中必须包含完全备份集； 8：校验备份集时是否进行页校验； 支持使用上述有效值的组合值，如 3 表示表示同时启用 1 和 2 对应的这两个策略。 缺省不开启备份集页校验，其余选项均开启
BAK_TIMEOUT	30	动态， 系统级	备份过程中阻塞操作的超时间隔，单位秒，取值范围 10~3600

2.1.1.1.22 快速插入

表2.23 dm.ini 中快速插入相关参数

参数名	缺省值	属性	说明
BATCH_INSERT_ROWS	10	动态，系统级	开启批量插入的记录行数阈值。当查询插入或批量绑定参数插入的记录行数大于等于 BATCH_INSERT_ROWS、且满足批量插入优化条件时，采用批量方式进行插入，否则采用逐行插入。取值范围 2~65536

INDEX_FINS_FLAG	0	动态，系统级	用于控制索引增删改的批量处理优化。当表满足批量处理的条件下，且当前语句的处理行数大于 BATCH_INSERT_ROWS 时可选的批量优化选项: 0：不启用相关优化; 1：异步方式处理二级索引; 2：启用 UNIQUE 索引批量处理; 4：启用普通二级索引批量处理; 8：插入或删除前尝试先对每批数据排序; 16：当异步任务堆积时，是否使用会话线程协助处理; 32：尝试页内比较、插入或删除; 64：尝试复用 B 树历史搜索路径; 256：启用聚集索引批量插入优化; 上述取值可组合使用，但启用 8、32、64 优化的前提是开启 2/4/256 以支持对应索引。部分优化仅数据有序时有意义，启用 8 提前排序时可以应用相关优化，但也需要考虑排序代价。 参数取值包含 1 时，异步处理二级索引的线程个数受参数 MAX_SEC_ASYNC_THREADS 限制，可通过查看

			V$THRD_POOL_GRP 获取实际执行的使用情况，即 TYPE 为 PTHD_POOL 的记录

2.1.1.1.23 DMDSC 相关

表2.24 dm.ini 中 DMDSC 相关参数

参数名	缺省值	属性	说明
DSC_N_CTLS	1028096	静态	LBS/GBS 控制页数目。取值范围 4096~4294967294
DSC_N_POOLS	19	静态	LBS/GBS 池数目。取值范围 2~1024
DSC_USE_SBT	1	静态	是否使用辅助的平衡二叉树。0：否；1：是
DSC_TRX_CMT_LSN_SYNC	3	动态，系统级	事务提交时 LSN 同步方式。取值范围 0~3600，0 表示提交时不主动同步 LSN；99 表示每次提交时强制节点间 LSN 同步；其他值表示每间隔多少秒同步节点间 LSN 值
DSC_ENABLE_MONITOR	1	动态，系统级	是否启用 DMDSC 请求时间监控，0：不启用；1：启用。监控的内容参见 V$DSC_REQUEST_STATISTIC
DSC_TRX_VIEW_SYNC	1	动态，系统级	节点间活动事务同步策略。0：无需等待响应；1：需要等待其他节点响应；2 表示采用定时同步策略，每隔 1 秒同步一次

DSC_TRX_VIEW_BRO_INTERVAL	1000	动态，系统级	控制 TRX VIEW 广播间隔时间，单位毫秒，取值范围 1~60000
DSC_REMOTE_READ_MODE	1	静态	DSC 环境下 REMOTE READ 优化模式。0：不优化；1：发出日志刷盘请求后不等待，立即响应，请求节点获取数据页前等待；2：发出日志刷盘请求后不等待，立即响应，请求节点日志写文件前等待。支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化
DSC_RESERVE_PERCENT	0.08	动态，系统级	计算启动 GBS CTL RESERVE 的阈值，当空闲的 GBS_CTL 数量小于 DSC_RESERVE_PERCENT%比例后，提前启动 GBS CTL 回收流程。取值范围 0~10，取值为 0 时不提前回收 GBS_CTL
DSC_TABLESPACE_BALANCE	0	动态，会话级	是否开启 DSC 服务器端基于表空间的负载均衡。0：不开启；1：开启。开启时，DSC 环境下服务器自动检查执行的查询语句是否满足特定条件，满足时系统会自动重连至指定的优化节点执行，且自动重连前服务器会返回运行时错误 EC_SWITCH_CONN_NODE(-613)
HA_INST_CHECK_IP	需要在 HA 场景下指定	静态	HA 实例启动检测 IP，是指 HA 系统中另外一台机器的 IP 地址，考虑到多网卡情况，最多允许配置 5 个 IP。如果需要配置多个 IP，则编辑多个配置项即可，比如： HA_INST_CHECK_IP = 192.168.0.8 HA_INST_CHECK_IP = 192.168.0.9

HA_INST_CHECK_PORT	65534	静态	HA 实例监听端口，数据库实例启动后，监听此端口的连接请求，并使用此端口号连接 HA_INST_CHECK_IP 判断另外一个 HA 实例是否已经启动。 只允许配置一个监听端口，所有 HA_INST_CHECK_IP 使用同一个端口进行检测，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配
DSC_INSERT_LOCK_ROWS	0	动态， 系统级	DSC 集群插入操作符单次插入记录数大于等于此参数指定值时，增加额外的封锁，将多个节点针对同一张表的插入操作串行执行，以减少节点间冲突，提升并发插入性能。取值范围 0~10000，0 表示不进行优化处理。需要注意的是，开启此参数后，DSC 集群单个节点的并发插入性能会有一定程度下降
DSC_CRASH_RECV_POLICY	0	静态	故障处理策略。0：故障处理时重演所有节点 REDO 日志，且日志重演期间服务挂起；1，故障处理时仅重演故障节点 REDO 日志，日志重演期间服务挂起；2，故障处理时仅重演故障节点日志，且日志重演期间正常提供服务。需要注意，故障处理策略 1 和 2 不适用于备库，且仅适用于单节点故障，不满足约束条件时将自动切换到故障处理策略 0
DSC_LBS_REVOKE_DELAY	0	动态， 系统级	DSC 集群优先满足本地 LBS 请求，延迟响应 LBS REVOKE 请求的时间。取值范围 0~1000，单位毫秒。数据页节点间冲突比较严重场景，可以开启此 LBS REVOKE 延迟响应机制，确保冲突数据页在一段时间内保持在某个节点运行，以提升整体性能
DSC_REQUEST_TIMEOUT	3600	动态，	DSC 环境下 LBS 请求 GBS 授权的超时时间，当请求时间超过设定值时，若 INI 参数 PTX_ROLLBACK 为 0 则实例故

		系统级	障退出，若 PTX_ROLLBACK 为 1 则报错。单位秒，取值范围 1~3600
DSC_QUOTA_RELOAD_INTERVAL	5	动态， 系统级	DSC 环境下重新加载表空间配额的时间间隔，单位秒，取值范围 1~60
DSC_SLOT_WAIT_TIMEOUT	0	动态， 系统级	DSC 集群全局 LBS 封锁超时 HALT 时间。系统运行过程中，获取数据页权限长时间得不到请求，当时间超过设置值的十分之一和 5 之间的较小值后则会在运行日志登记警告信息。单位秒，取值范围 0 或 10~3600。取值为 0 时表示关闭超时 HALT 机制
DSC_IGNORE_INI_CHECK	0	静态	DSC 环境下是否忽略节点间的 INI 参数一致性校验。0：不忽略，1：忽略
DSC_DV_PAGE_ACCESS_MODE	1	动态， 系统级	DSC 集群 V$表 0 号页访问模式，0：标准数据页访问 0 号页，1：访问本地 BUFFER 读取 0 号页相关信息
DSC_LBS_OPT_FLAG	1	动态， 系统级	LBS 封锁优化开关。 0：不优化； 1：页读完成后，提前请求下一页的 LBS 封锁； 2：LBS 请求授权时，尝试并行本地读该页。 支持使用上述有效值的组合值，如 3 表示同时进行 1 和 2 的优化

ASM_EXTENT_INIT_CHECK	0	手动	访问到 ASM 文件未格式化的簇时是否 HALT。 0：忽略； 1：访问到 ASM 文件未格式化的簇时主动 HALT

2.1.1.1.24 RDMA 通讯

表2.25 dm.ini 中 RDMA 通讯相关参数

参数名	缺省值	属性	说明
USE_RDMA	0	手动	LINUX 环境下，服务器是否支持 RDMA 通讯方式。 0：否，使用 TCP 方式；1：是，使用 RDMA 方式。 当 USE_RDMA=1 时，DPI 可以使用 RDMA 方式登录服务器。使用 RDMA 之前需先安装好迈络思 OFED 驱动（RDMA 协议的基础库 LIBIBVERBS.SO 和 LIBRDMACM.SO）
MAX_SEND_WR	128	静态	QP 发送队列中最大发送的工作请求(WR)个数，取值范围 1~4294967294。MAX_SEND_WR 参数值不能大于等于 MAX_RECV_WR，若大于等于，会调整为 MAX_RECV_WR 值一半
MAX_RECV_WR	512	静态	QP 接收队列中最大接收的工作请求(WR)个数，取值范围 1~4294967294。需要注意，若 MAL 开启 MAL_USE_RDMA，MAL 会从 SHARE POOL 内存池中为每个连接预分配 MAX_RECV_WR 个接收消息

			的缓存，每个缓存大小不超过 32KB，因此配置 MAX_RECV_WR 时需要结合服务器内存大小适当分配
CQ_MOD	100	静态	QP 发送工作请求时，每隔多少个工作请求(WR)生成一个信号来通知完成队列（CQ）生成相应的事件(CQE)响应，取值范围 1~4294967294。CQ_MOD 取值不能超过 MAX_SEND_WR，若超过，MAX_SEND_WR>1 时会调整 CQ_MOD 为（MAX_SEND_WR-1）， MAX_SEND_WR=1 时调整 CQ_MOD 为 1。

2.1.1.1.25 删除表相关

表2.26 dm.ini dm.ini 中删除表相关相关参数

参数名	缺省值	属性	说明
DROP_CASCADE_VIEW	0	动态，会话级	删除表或者视图的时候级联删除视图，0：将待删除的表或视图与其上的视图的依赖关系进行剥离，只删除表或者视图；1：将表或视图作为其上建立的视图的依赖项，删除表或者视图时根据语句的 CASCADE 选项进行相应处理。当 COMPATIBLE_MODE=1 时，DROP_CASCADE_VIEW 的实际值为 1

2.1.1.1.26 未列在 dm.ini 中的 INI 配置项

下表中的 INI 参数没有包含在 dm.ini 文件中，用户可以通过调用系统过程动态修改参数值，也可直接在 dm.ini 文件中添加参数并指定参数值。

表2.27 不在 dm.ini 中的 INI 配置项一

参数名	缺省值	属性	说明
HFS_UNDO_ERR_POLICY	0	静态	在系统启动过程中，如果 HUGE 表 UNDO 操作出错，是否将系统直接 HALT。0：是；1：否，忽略该错误
STARTUP_TIMEOUT	5	动态，系统级	握手消息的等待超时，取值范围 5-10000
HAVING_RATE_FACTOR	1	动态，会话级	HAVING 过滤行数修正因子，当 INI 参数 OLAP_FLAG>0 有效，支持 HINT; 计算公式：HAVING 估算行数 / HAVING_RATE_FACTOR
INTER_RATE_FACTOR	1	动态，会话级	集合查询 INTERSECT、INTERSECT ALL 行数修正因子，支持 HINT; 计算公式：MIN(左孩子行数, 右孩子行数) / INTER_RATE_FACTOR
DPC_CONN_POLICY	0	动态，系统级	DMDPC 环境下专用，设置 DMDPC 系统链路的管理策略，按位取值，10 进制取值如下：0：无特殊策略；1：DPC AP 监听端口如果长时间没有监听到连接，则重建监听端口；2：阻塞模式的链接对于 EINPROGRESS 错误进行检查并重试。

			调用方法动态调整时不会将调整后的值保存至配置文件中，INI 文件中的参数值必须通过手动修改

下表中的 INI 参数没有包含在 dm.ini 文件中，用户只能通过调用系统过程动态修改参数值，不支持在 dm.ini 文件中指定参数值。

表2.28 不在 dm.ini 中的 INI 配置项二

参数名	缺省值	属性	说明
PWD_POLICY	15	动态，系统级	设置系统缺省口令策略。取值：15、31。 15：禁止与用户名相同、口令长度需大于等于 INI 参数 PWD_MIN_LEN 设置的值、须同时包含大写字母(A~Z)和小写字母(a-z)、至少包含一个数字(0-9)； 31：除了取值 15 的策略外，还要求至少包含一个标点符号(英文输入法状态下，除“ 和空格外的所有符号)。 DMINIT 工具中 SYSDBA_PWD、SYSAUDITOR_PWD、SYSSSO_PWD、SYSDBO_PWD 使用的口令策略，请参考 PWD_POLICY 为 15
PWD_MIN_LEN	8	动态，系统级	设置用户口令的最小长度，取值范围 8~48。配合 INI 参数 PWD_POLICY 使用

ENABLE_OBJ_REUSE	0	静态	是否支持客体重用，0：不支持；1：支持。 注：修改此参数后，必须正常退出数据库并重启后才能生效
ENABLE_REMOTE_OSAUTH	0	静态	是否支持基于操作系统的远程身份验证，0：不支持；1：支持。 注：该参数设置仅安全版有效。修改此参数后，必须正常退出数据库并重启后才能生效
MAC_LABEL_OPTION	1	动态，系统级	用于控制 SP_MAC_LABEL_FROM_CHAR 过程的使用范围。 0：只有 SSO 可以调用； 1：所有用户都可以调用； 2：所有用户可以调用，但是非 SSO 用户不会主动创建新的 LABEL。 注：该参数设置仅安全版有效
ENABLE_EXTERNAL_CALL	0	静态	是否允许创建或执行外部函数。0：不允许；1：允许。 注：修改此参数后，必须正常退出数据库并重启后才能生效
ENABLE_DDL_ANY_PRIV	0	动态，系统级	是否可以授予和回收DDL相关的ANY系统特权。0：否；1：是
SEC_PRIV_MODE	0	静态	表示权限管理模式。 0：表示传统模式；

			1：表示专用机模式； 2：表示 EVAL 测评模式； 3：表示 ZB 模式； 4：表示 ZBMM 模式。 注：该参数设置仅安全版有效。修改此参数后，必须正常退出数据库并重启后才能生效
PASSWORD_VERIFICATION	0	动态，系统级	用户修改数据库身份验证模式下自己的口令时，是否强制需要提供原口令。0：不需要；1：需要。 注：修改此参数后，必须正常退出数据库并重启后才能生效

下表中的 INI 参数没有包含在 dm.ini 文件中，用户可以直接在 dm.ini 文件中添加参数并指定参数值，不支持动态修改。

表2.29 不在 dm.ini 中的 INI 配置项三

参数名	缺省值	属性	说明
RAFT_SHADOW	0	手动	是否是一个多副本影子库。0：否；1：是。 该参数仅能通过备份还原指令设置，不能通过系统函数动态修改，且不是建库参数
ENABLE_AUDIT	0	手动	控制审计功能的开启。

			0：不开启审计； 1：开启普通审计； 2：开启普通审计和实时审计
RLOG_ENC_ID	0	手动	表示加密算法 ID。当使用 DMINIT 的 RLOG_ENCRYPT_NAME 参数设置联机日志文件和归档日志文件的加密算法时进行加密，若未设置则不加密。 RLOG_ENCRYPT_NAME 在数据库创建成功后无法修改，可通过查询 V$PARAMETER 或 V$DM_INI 中的 RLOG_ENC_ID 参数名查看 RLOG_ENCRYPT_NAME 设置的加密算法 ID，还可以通过使用系统函数获取此参数设置的加密算法 ID：SELECT SF_GET_PARA_VALUE(2,'RLOG_ENC_ID')；

INI 参数 ENABLE_MAC 较为特殊，用户只能在初始化数据库时指定该参数值，初始化完成后无法修改，仅可使用 V$DM_INI 视图查询该参数值。

表2.30 不在 dm.ini 中的 INI 配置项四

参数名	缺省值	属性	说明
ENABLE_MAC	2	不可	是否打开强制访问控制功能，仅对安全版有效。开启成功后后续启动无需再添加

		修改	ENABLE_MAC 参数

2.1.1.1.27 主备环境 INI 同步规则

在主备环境中，备库上部分 INI 参数（来自 dm.ini 或未列在 dm.ini 中的配置项）须和主库保持一致。DM 提供自动同步和手动同步两种方法。版本升降级期间，由于不同版本的 INI 参数可能不一致，因此不同版本的主备库不会进行 INI 同步。

- **自动同步**

一 开启INI_SYNC参数同步后， 备库的部分INI参数会定时与主库同步到一致。同步原则为备库文件值与主库同步至一致，备库内存值与主库同步至一致。若备库为DMDSC集群，集群内节点均会进行同步。不同参数类型的同步规则如下：

1. 静态 INI 参数：仅同步 dm.ini 文件值，不同步内存值，备库需要重启才能使用新的参数值；
2. 系统级动态 INI 参数：同时同步 dm.ini 文件值与内存值；
3. 会话级动态 INI 参数：同步全局内存值，不同步会话上的 INI 参数值；
4. 若是在 INI 同步改进前后的两个版本间执行，不会真正同步，而是直接返回成功；
5. INI_FILE 类型的参数，只会写入 INI 文件，需要重启才能生效；
6. 不论是否开启 INI_SYNC 参数，对于参数 ENABLE_AUDIT、PWD_POLICY、PWD_MIN_LEN、MAC_LABEL_OPTION、ENABLE_DDL_ANY_PRIV，

主库修改后，备库会随重演同步调整 INI 参数值；

7. DMDSC 下启动时，非控制节点会从控制节点同步一次 INI 参数，此轮同步会登记同步信息到 V$INI_SYNC_INFO，此轮同步不受 INI_SYNC 参数控制。

二 主备切换时，当新主库开启了 INI_SYNC 参数时，新主库会主动向所有备库归档目标同步一次参数，通知备库将参数与新主库同步到一致，对于 DMDSC 备库，归档目标为控制节点，仅会向控制节点同步参数；若新主库未开启 INI_SYNC 参数，则新主库不主动向备库同步参数。

- **手动同步**

除了自动同步外，还可以通过 SF_SYNC_INI 或者 SF_DSC_SYNC_INI 函数进行手动同步，函数的具体使用方法可以参考《DM8_SQL 语言使用手册》。

2.1.1.2 dmmal.ini

dmmal.ini 是 MAL 系统的配置文件。dmmal.ini 的配置项见下表。需要用到 MAL 环境的实例，所有站点 dmmal.ini 需要保证严格一致。

表2.31 dmmal.ini 的配置项

项目	项目意义	字段	字段意义

MAL_CHECK_INTERVAL	MAL 链路检测时间间隔，取值范围 0~1800，单位秒（s），缺省为 30s，配置为 0 表示不进行 MAL 链路检测	无	无
MAL_CONN_FAIL_INTERVAL	判定实例之间 MAL 链路断开的时间，取值范围 2~1800，单位秒（s），缺省为 10s。仅当 MAL_CHECK_INTERVAL 不为 0 时有效	无	无
MAL_CHECK_TIMEOUT	单个实例的 MAL 网络最大延迟时间，单位毫秒（ms），取值范围 0~65534，缺省为 0，表示不进行 MAL 网络延迟时间检测。需要和 MAL_CHECK_IP 配合使用。仅当 MAL_CHECK_INTERVAL 不为 0 时有效	无	无
MAL_CHECK_IP	第三方确认机器的 IP，用于检测各个实例的 MAL 网络延迟时间。需要和 MAL_CHECK_TIMEOUT 配合使用。确认机器为 MAL 链路所有实例均可访问到的独立外网机器，MAL 链路实例包括 DMServer、DMASMSVR（非 DMASM 镜像）和 DMASMSVRM（DMASM 镜像）。	无	无

	实例每隔 MAL_CHECK_INTERVAL 时间 ping 一次 MAL_CHECK_IP，若连接耗时超过 MAL_CHECK_TIMEOUT 指定时间，则该实例主动 HALT。 开启 MAL 网络延迟时间检测后，需要为 MAL 链路所有实例 赋予 ping 权限，具体方法可以参考《DM8 数据共享集群》		
MAL_SEND_THRESHOLD	最大待发送邮件数，用于控制待发送邮件总数，防止发送邮件堆积。取值范围 1024~10000，缺省为 2048	无	无
MAL_LEAK_CHECK	是否打开 MAL 内存泄露检查。0：关闭；1：打开。缺省为 0	无	无
MAL_LOGIN_TIMEOUT	MPP/DBLINK 等实例间登录时的超时检测间隔 3~1800，单位秒，缺省为 15s	无	无
MAL_BUF_SIZE	单个 MAL 缓存大小限制，单位 MB。当此 MAL 的缓存邮件超过此大小，则会将邮件存储到文件中。取值范围 0~500000，缺省为 100	无	无
MAL_SYS_BUF_SIZE	MAL 系统总内存大小限制，单位 MB。取值范围 0~500000，	无	无

	缺省为 0，表示 MAL 系统无总内存限制		
MAL_VPOOL_SIZE	MAL 系统使用的内存初始化大小，单位 MB。取值范围 1~500000，缺省为 10，此值一般要设置的比 MAL_BUF_SIZE 大一些	无	无
MAL_COMPRESS_LEVEL	MAL 消息压缩等级，取值范围 0~10。0 代表不进行消息压缩，为默认值 1~9 表示采用 zip 算法，从 1 到 9 表示压缩速度依次递减，压缩率依次递增。 10 表示采用 snappy 算法，压缩速度高于 zip 算法，压缩率相对低	无	无
MAL_TEMP_PATH	指定临时文件的目录。当邮件使用的内存超过 MAL_BUF_SIZE或者MAL_SYS_BUF_SIZE时，将新产生的邮件保存到临时文件中。如果缺省，则新产生的邮件保存到TEMP.DBF文件中	无	无

MAL_COMBIN_BUF_SIZE	指定 MAL 消息合并发送的缓冲区大小，单位 KB。MAL 发送消息时进行合并，减少网络发送次数。取值范围 0~131072。0 表示不进行消息合并，为缺省值。最小缓冲区大小 128KB，当设置值大于 0 小于 128 时，自动调整为 128	无	无
MAL_MESSAGE_CHECK	是否对 MAL 通信消息启用消息体校验（只有当消息的发送端和接收端都配置为 1 才启用通信体校验）。0：不启用；1：启用。缺省为 1	无	无
MAL_MSG_ENCRYPT	MAL 系统是否启用 SSL 通讯加密。0：不启用；1：启用。缺省为 0	无	无
MAL_USE_RDMA	LINUX 环境下，MAL 是否使用 RDMA 通讯方式。 0：否，使用 TCP 方式；1：是，使用 RDMA 方式。 当 MAL_USE_RDMA=1 时，MAL_HOST 必须是 RDMA 网卡所绑定的 IP。使用 RDMA 之前需先安装好迈络思 OFED 驱动（RDMA 协议的基础库 LIBIBVERBS.SO 和	无	无

	LIBRDMACM.SO）		
[MAL_INST1]	实例配置	MAL_INST_NAME	实例名。MAL 系统中的各数据库实例不允许同名。 此参数应与 DM.INI 中的 INSTANCE_NAME 保持一致
		MAL_HOST	MAL IP 地址，使用 MAL_HOST + MAL_PORT 创建 MAL 链路。 对于 IPV6 地址，若当前环境存在多块网卡，需要用%号指定具体有效的网卡序号或网卡名称；若只有一块网卡或者已配置默认网卡，则可以不指定序号或名称。 例如：MAL_HOST = FE80::610E:9715:5EC6:4EA8%ENS01，其中 ENS01 为当前环境使用的网卡名称
		MAL_PORT	MAL 监听端口，用于数据守护、DSC、MPP 等环境中各节点实例之间 MAL 链路配置，监听端端口配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间

			随机分配
		MAL_INST_PORT	MAL_INST_NAME 实例服务器监听通讯端口号，服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配。 此参数应与 DM.INI 中的 PORT_NUM 保持一致
		MAL_INST_HOST	连接数据库服务器使用的 IP 地址。（在 MPP 下以及读写分离的即时归档主备系统下一定要配置）
		MAL_DW_PORT	MAL_INST_NAME 实例守护进程的监听端口，其他守护进程或监视器使用 MAL_HOST + MAL_DW_PORT 创建与该实例守护进程的 TCP 连接，监听端配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配。缺省为 0，表示不配置端口号，无法监听
		MAL_INST_DW_PORT	节点实例监听守护进程的端口，实例服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之

			间随机分配。缺省为 0，表示不配置端口号，无法监听
		MAL_LINK_MAGIC	MAL 链路网段标识，取值范围 0~65534，缺省为 0。设置了此参数后，对于设置的 MAL_LINK_MAGIC 值不同的实例，DMWATCHER 之间和 DMServer 之间的通信使用配置的 MAL_INST_HOST；对于设置的 MAL_LINK_MAGIC 值相同的实例，DMWATCHER 之间和 DMServer 之间的通信使用配置的 MAL_HOST。 搭建跨网段的 DBLINK 时，如果内外网不通，则必须配置此参数；如果内外网互通，则可以不配
		MAL_RDMA_MAGIC	MAL 链路通信方式标识，取值范围 0~65534，缺省为 0，属于为每个实例配置的局部变量，为了支持 RDMA/TCP 混合通信。在 MAL_USE_RDMA 开启的情况下，配置相同 MAL_RDMA_MAGIC 的节点间使用 RDMA 通信，不同则使用 TCP 通信。若不配置 MAL_RDMA_MAGIC，缺省为 0，

			可以和 MAL_RDMA_MAGIC 为 0 的节点进行 RDMA 通信。

注意：dmmal.ini 和 dm.ini 中都可配置 MAL_LEAK_CHECK，启动时以 dmmal.ini 中设置为准，如果 dmmal.ini 中没有配置，则以 dm.ini 中配置为准，两个都没配置时，则默认 0。此参数在 dm.ini 中可动态更改。

2.1.1.3 dmarch.ini

dmarch.ini 用于配置归档。dmarch.ini 的配置项见下表。

表2.32 dmarch.ini 的配置项

项目	项目意义	字段	字段意义
全局配置项		ARCH_WAIT_APPLY	备库收到 Redo 日志后，是否需要重演完成后再响应主库。0 表示收到马上响应（高性能模式），1 表示重演完成后响应（事务一致模式）。配置为即时归档时，缺省值为 1；配置为实时归档时，缺省值为 0。 实时读写分离的事务一致模式，仅在数据守护配置为自动切换模式时才会生效；如果数据守护配置为手动切换模式时进行实时归档，ARCH_WAIT_APPLY=1 不会生效。

			针对配置实时归档的 DMDSC 数据守护集群，不支持将 ARCH_WAIT_APPLY 配置为 1
		ARCH_RESERVE_TIME	归档日志保留时间，单位分钟，取值范围 0~2147483647。只对本地归档有效。服务器每隔 5 分钟检查是否存在超过保留时间的归档并删除。缺省为 0，表示不删除归档
		ARCH_LOCAL_SHARE	DMDSC 集群本地归档是否共享给远程节点作为远程归档。 注：自 V8.1.3.61 版本起废弃该参数，始终共享
		ARCH_LOCAL_SHARE_CHECK	是否进行 LOCAL 归档和 REMOTE 归档路径的一致性校验。 注：自 V8.1.3.61 版本起废弃该参数，始终校验
		ARCH_SEND_POLICY	主库发送 REALTIME 归档的策略，取值 0 或 1，缺省为 0。0：主库发送 REALTIME 归档后立即等待备库响应消息；1：主库发送 REALTIME 归档后不立即等待备库的响应消息，而是先将当前日志包写入联机日志文件，然后再等待备库响应消息。 当此参数配置为 1 时，日志包刷盘与等待备库响应的动作等效于并发执行，相比于配置为 0 的策略而言，能够减少主库日志刷盘线程因等待备库响应消息而阻塞的时间，降低 REALTIME 备库对主库性能的影响。 TIMELY 主备不支持此参数；DMDSC 集群中也不支持配置此参数，但允许在 DMDSC 主备中的单机库上配置此参数。

			注意：如果主库将 ARCH_SEND_POLICY 配置为 1，且守护进程配置为自动接管模式，则备库自动接管可能会有集群分裂的风险，需要谨慎使用此配置方式
		XMAL_HB_INTERVAL	节点间通信检测间隔。单位 S，取值范围 1~600。该参数仅在 DMDPC 的多副本系统中有效，各副本需配置相同
		RAFT_HB_INTERVAL	主库广播心跳消息的间隔时间。单位 MS，取值范围 5~1800。该参数仅在 DMDPC 的多副本系统中有效，各副本需配置相同
		RAFT_VOTE_INTERVAL	选举超时时间。单位 MS，取值范围 10~3600000。这个配置值需要保证至少是 RAFT_HB_INTERVAL 的 2 倍大小，确保在选举超时时间内能够收到主库的心跳消息，避免误判主库故障发起无效选举。建议各实例配置为不同的值，以便能够快速选举出主库，配置的值越小，被选举为主库的优先级越高。该参数仅在 DMDPC 的多副本系统中使用，并且仅在开启选举开关的实例上有效
		XMAL_IP	当前副本 XMAL 通信 IP。DMDPC 环境下无需在 DMARCH.INI 文件中配置此项
		XMAL_PORT	当前副本 XMAL 通信端口。DMDPC 环境下无需在 DMARCH.INI 文件中配置此项
		RAFT_SELF_ID	副本号，取值范围 0~31。该参数仅在 DMDPC 的多副本系统中有效，用于标识 RAFT 组中不同的节点
		RAFT_LEARNER	标识当前副本是否为 LEARNER 角色。1：是，0：否。缺省为 0。该参数仅在 DMDPC 的多副本系统中有效，非

			LEARNER 节点可不配置此参数
		RAFT_PKG_INTERVAL	备库接收主库日志包的超时时间。当备库超过该时间还未接收到主库日志包，则备库主动断开与主库连接。单位 S，取值范围 60~36000。缺省为不配置该参数。该参数仅在 DMDPC 的多副本系统中有效
[ARCHIVE_LOCAL1]	本地归档配置	ARCH_TYPE	归档类型
		ARCH_DEST	归档路径。需要配置在 ARCH_TYPE 之后
		ARCH_FILE_SIZE	单个归档文件大小，单位 MB，取值范围 64~2048，缺省为 1024MB，即 1GB
		ARCH_SPACE_LIMIT	本地归档文件空间限制。当同一节点号的本地归档文件达到限制值时，系统自动删除最早生成的本地归档日志文件。0 表示无空间限制，取值范围 1024~2147483647，缺省为 0，单位 MB
		ARCH_FLUSH_BUF_SIZE	归档合并刷盘缓存大小，单位 MB，取值范围 0~128。0 表示不使用归档刷盘缓存机制。缺省为 2
		ARCH_HANG_FLAG	本地归档写入失败时系统是否挂起。取值范围 0~2；缺省为 1。 0：写入失败后不挂起，直接报错返回，不再尝试写入； 1：写入失败后挂起，反复尝试写入归档，直到写入成功； 2：写入失败后先删除最老的归档文件，再重新尝试写入，反复进行该过程直到归档写入成功，或者所有归档文件都已经被删除；若所有归档文件都已经被删除的情况下仍然写入失败，则直接报错返回，不再尝试写

			入。 需要注意的是，若当前库配置的所有本地归档 ARCH_HANG_FLAG 均为非 1 值，则写入归档失败后可能会出现归档日志缺失的情况，进而可能产生特定情况下备份还原无法正常进行等问题
ARCHIVE_REALTIME	实时归档配置	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标实例名。需要配置在 ARCH_TYPE 之后
		WAIT_APPLY	若归档目标未配置 WAIT_APPLY，则以全局 ARCH_WAIT_APPLY 为准；若某一个归档目标配置了 WAIT_APPLY，则对应归档目标是否是事务一致由配置的 WAIT_APPLY 为准。取值：0：表示高性能模式；1：表示事务一致模式。 实时读写分离的事务一致模式，仅在数据守护配置为自动切换模式时才会生效；如果数据守护配置为手动切换模式时进行实时归档，WAIT_APPLY=1 不会生效。 针对配置实时归档的 DMDSC 数据守护集群，不支持将 WAIT_APPLY 配置为 1
		ARCH_STANDBY_APPLY	是否根据归档目标重演情况确定自身本地归档可删除。取值 0：否；1：是。默认值为 0
ARCHIVE_ASYNC	异步归	ARCH_TYPE	归档类型

	档	ARCH_DEST	归档目标实例名。需要配置在 ARCH_TYPE 之后
		ARCH_TIMER_NAME	定时器名称。需要配置在 ARCH_TYPE 之后
		ARCH_SEND_DELAY	指定源库到异步备库的归档延时发送时间，单位分钟，取值范围 0~1440，缺省为 0，表示不启用归档延时发送功能。 注：仅在异步归档类型中支持此配置项，其他归档类型不支持。如果源库是 DSC 集群，建议用户配置时保证各节点上配置的值是一致的，并保证各节点所在机器的时钟一致，避免控制节点发生切换后计算出的归档延迟发送时间不一致
		ARCH_STANDBY_APPLY	是否根据归档目标重演情况确定自身本地归档可删除。取值：0：否；1：是。默认值为 0
ARCHIVE_SYNC	同步归档	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标实例名。需要配置在 ARCH_TYPE 之后
		ARCH_RECOVER_TIME	主库向同步备库发起异步恢复的时间间隔，单位秒，取值范围 1~86400，缺省为 60，表示主库每隔 60 秒检查一次同步备库的归档状态，若为 INVALID 且和同步备库通信正常，则发起异步恢复。 注：仅在同步归档类型中支持配置此项，其他归档类型不支持
		ARCH_STANDBY_APPLY	是否根据归档目标重演情况确定自身本地归档可删除。取值：0：否；1：是。默认值为 0

ARCHIVE_TIMELY	即时归档	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标实例名。需要配置在 ARCH_TYPE 之后
		WAIT_APPLY	若归档目标未配置 WAIT_APPLY，则以全局 ARCH_WAIT_APPLY 为准；若某一个归档目标配置了 WAIT_APPLY，则对应归档目标是否是事务一致由配置的 WAIT_APPLY 为准。取值：0：表示高性能模式；1：表示事务一致模式。 实时读写分离的事务一致模式，仅在数据守护配置为自动切换模式时才会生效；如果数据守护配置为手动切换模式时进行实时归档，WAIT_APPLY=1 不会生效。 针对配置实时归档的 DMDSC 数据守护集群，不支持将 WAIT_APPLY 配置为 1
		ARCH_STANDBY_APPLY	是否根据归档目标重演情况确定自身本地归档可删除。取值：0：否；1：是。默认值为 0
[ARCHIVE_RAFT1]	RAFT 归档	ARCH_TYPE	归档类型。ARCH_TYPE 支持配置为 RAFT 和 LEARNER。其中 LEARNER 归档仅在动态增删节点时使用
		ARCH_DEST	归档目标实例名。需要配置在 ARCH_TYPE 之后
		ARCH_DEST_IP	归档目标 XMAL 通信 IP。DMDPC 环境下无需在 DMARCH.INI 文件中配置此项
		ARCH_DEST_PORT	归档目标 XMAL 通信端口。DMDPC 环境下无需在 DMARCH.INI 文件中配置此项
		ARCH_DEST_ID	归档目标多副本节点编号，用于标识多副本 RAFT 组中不同的节点。取值范围 0~31

		ARCH_STANDBY_APPLY	是否根据归档目标重演情况确定自身本地归档可删除。取值：0：否；1：是。默认值为 0
REMOTE	远程归档配置	ARCH_TYPE	归档类型
		ARCH_DEST	归档目标实例名。需要配置在 ARCH_TYPE 之后
		ARCH_INCOMING_PATH	指定远程归档目标 ARCH_DEST 的本地归档日志文件存放路径。需要配置在 ARCH_TYPE 之后
		ARCH_RECV_PATH	本地的归档备份路径。当 DMDSC 集群中出现故障节点时，控制节点会将联机日志检查点偏移后的所有有效日志备份到 ARCH_RECV_PATH 路径下，即便后续 ASM 文件系统损坏，也能保证 LOCAL 归档与 ARCH_RECV_PATH 路径下的归档合并起来是完整的。该参数没有默认值，可不配置

相关说明

1. 归档类型 ARCH_TYPE 有以下几种：

   本地归档　　LOCAL（一个主库最多配 8 个）

   远程实时归档　　REALTIME（一个主库最多配 8 个）

   远程异步归档　　ASYNC（一个主库最多配 8 个）

   同步归档　　SYNC（一个主库最多配 8 个）

   即时归档　　TIMELY（一个主库最多配 8 个）

远程归档　　REMOTE（一个主库最多配 8 个）

RAFT 归档　　RAFT（一个主库最多配 8 个）

LEARNER 归档　　LEARNER（一个主库最多配 9 个）

2. 配置名[ARCHIVE_*]表示归档名，在配置文件中必须唯一。
3. 不能存在相同实例名的不同归档。
4. 不能存在 DEST 相同的不同归档实例。
5. ARCH_TIMER_NAME 为定制的定时器名称，定时器配置见 dmtimer.ini。
6. 对于一个归档配置，ARCH_TYPE 需要配置在 ARCH_DEST、ARCH_TIMER_NAME、ARCH_INCOMING_PATH 之前。
7. ARCH_STANDBY_APPLY 参数支持 REALTIME/TIMELY/SYNC/ASYNC/RAFT 归档，如果多个归档目标都配置了此字段，当前库会根据重已重演 LSN 值最小的备库重演信息确定自身本地归档是否可以删除。当主库配置了本地归档文件空间限制参数 ARCH_SPACE_LIMIT，同一节点号的本地归档文件已经达到空间上限，但归档又不满足删除条件的情况下，仍然正常创建新文件，不再做挂起处理，优先保证主库服务不中断。如果备库一直故障，主库从未获取过此备库的 apply 信息，则不允许删除任何日志；如果曾经获取过此备库的 apply 信息，则以此历史信息为准判断是否删除。不要求备库一定和主库配置一致，备库切换主库后，将按照新主库的配置情况来决定如何删除归档，主备切换后，有可能还是会发生归档已经被清理的情况。对于异步备库的级联配置情况，异步备库会根据级联的备库重演情况决定自身的本地归档是否可删除。没有级联配置的备

库，不受此参数控制。

8. 关于 DSC 环境的本地归档清理规则说明如下：

（1）查询 V$ARCH_FILE 时，DSC 环境不仅会有本地归档还会有远程归档，若是 ASM 环境，还可能查出 DSC 集群其他节点的本地归档和远程归档。本地归档空间上限和远程归档空间上限分开计算。

（2）配置 ARCH_STANDBY_APPLY=1 后，DSC 为备库时，主库归档空间达到上限后，按照 DSC 控制节点的重演情况清理本地归档。

（3）当主库是 DSC 集群，备库生成本地归档时会按照收到日志包的节点号分开保存归档（对于主库是 DSC2 节点的场景，备库收到 0 号节点发送过来的日志包产生的归档文件加入到 0 号归档链表中，备库收到 1 号节点发送过来的日志包产生的归档文件加入到 1 号归档链表中）。两个归档文件链表的空间上限分开计算。若此后又发生主备切换，新主库为单机库的场景，主库生成的新归档文件仅会加入到 0 号归档链表中。

2.1.1.4 sqllog.ini

sqllog.ini用于SQL日志的配置，当且仅当INI参数SVR_LOG=1时使用。

2.1.1.4.1 sqllog.ini 配置项

sqllog.ini的详细配置请参考下表。

表2.33 sqllog.ini 的配置项

参数名	缺省值	属性	说明
SQL_TRACE_MASK	1	动态，系统级	指定 SQL 日志中需要被记录的语句类型。为了避免日志量过大，建议用户根据实际需要有选择地指定，而非一律全部指定。 指定方式为 SQL_TRACE_MASK=位号:位号:位号……。例如：3:5:7 表示第 3，第 5，第 7 位号代表的类型需要被记录在 SQL 日志中。 以下位号均可以单独使用，也可以搭配使用，其中位号 2~17 与 23、24、25、26、28 搭配使用时，表示取其交集。 例如：SQL_TRACE_MASK=2 表示记录 DML 语句；SQL_TRACE_MASK=24 表示记录执行语句；SQL_TRACE_MASK=29 表示记录事务相关语句；SQL_TRACE_MASK=2:3:24:29 表示记录 DML 和 DDL 的执行语句以及事务相关语句。 位号的含义如下所示: 1 全部记录（等同于同时设置 4~31） 2 DML 类型相关语句（等同于同时设置 4~10） 3 DDL 类型相关语句（等同于同时设置 11~17） 4 UPDATE 类型语句（更新） 5 DELETE 类型语句（删除）

			6 INSERT 类型语句（插入） 7 SELECT 类型语句（查询） 8 COMMIT 类型语句（提交） 9 ROLLBACK 类型语句（回滚） 10 CALL 类型语句（过程调用） 11 BACKUP 类型语句（备份） 12 RESTORE 类型语句（恢复） 13 创建对象操作（CREATE DDL） 14 修改对象操作（ALTER DDL） 15 删除对象操作（DROP DDL） 16 授权操作（GRANT DDL） 17 回收操作（REVOKE DDL） 22 记录绑定参数的行数 23 记录存在错误的语句（语法错误，语义分析错误等）

			24　记录执行语句 25　记录执行语句、执行语句的时间、语句的影响行数（只有增删改查有行数，其它语句无行数） 26　记录执行语句的时间、语句的影响行数（只有增删改查有行数，其它语句无行数）。25 和 26 二者只能选择其一。25 和 26 同时存在时，只有 25 有效 27　记录原始语句（服务器从客户端收到的未加分析的语句） 28　记录参数信息，包括参数的序号、数据类型和值 29　记录事务相关事件，包括锁类型、锁等待时间等 30　记录 XA 事务 31　记录数据库登录操作，包括登录成功、登录失败、退出登录
FILE_NUM	5	动态， 系统级	总共记录的符合当前配置命名规则的日志文件数量，当日志文件达到这个设定值以后，再生成新的文件时，会删除最早的那个日志文件。取值范围 2~1024。日志文件名称中将包含日期时间信息，关于日志文件命名格式的详细介绍请参考 2.8 SQL 日志文件
SWITCH_MODE	2	动态， 系统级	表示 SQL 日志文件切换的模式： 0：不切换 1：按文件中记录数量切换

			2：按文件大小切换 3：按时间间隔切换
SWITCH_LIMIT	与切换模式有关	动态，系统级	不同切换模式 SWITCH_MODE 下，意义不同： ◆ 按数量切换时，一个日志文件中的 SQL 记录条数达到多少条之后，系统自动将日志切换到另一个文件中。取值范围 1000~10000000，缺省为 100000 ◆ 按文件大小切换时，一个日志文件达到该大小后，系统自动将日志切换到另一个文件中，单位 MB。取值范围 1~2000，缺省为 128 ◆ 按时间间隔切换时，每隔指定的时间间隔，系统自动将日志切换到另一个文件中，单位分钟。取值范围 1~30000，缺省为 60
ASYNC_FLUSH	1	动态，系统级	是否打开 SQL 日志异步刷盘功能。0：否，采用实时刷盘；1：是，采用异步刷盘
MIN_EXEC_TIME	0	动态，系统级	记录的最小语句执行时间，单位根据 EXECTIME_PREC_FLAG 进行判断，当 EXECTIME_PREC_FLAG 等于 0 或 1 时，其单位为毫秒 MS；当 EXECTIME_PREC_FLAG 等于 2 时，其单位为微秒 US。执行时间小于该值的语句不记录在日志文件中。取值范围 0~9223372036854775807
FILE_PATH	../LOG	动态，	SQL 日志文件所在的文件夹路径。缺省生成在 DM 安装目录的 LOG 子目录下面

		系统级	
BUF_TOTAL_SIZE	10240	动态， 系统级	SQL 日志 BUFFER 占用空间的上限，单位 KB，取值范围 1024~1024000
BUF_SIZE	1024	动态， 系统级	一块 SQL 日志 BUFFER 的空间大小，单位 KB，取值范围 50~102400
BUF_KEEP_CNT	6	动态， 系统级	系统保留的 SQL 日志缓存的个数，取值范围 1~100。当 PART_STOR=1 时，建议取值稍大于用户数；当 PART_STOR=2 时，建议取值大于 11
PART_STOR	0	动态， 系统级	SQL 日志分区存储，表示 SQL 日志进行分区存储的划分条件。 0 表示不划分；1 表示 USER：根据不同用户分布存储
ITEMS	0	动态， 系统级	指定一条 SQL 日志中应包含的内容。指定方式为 ITEMS=位号:位号:位号……。 例如：ITEMS=3:5:7。表示应包含第 3、第 5、第 7 位代表的内容。 0 表示记录除 CLIENT_INFO 外的所有的列（等同于同时设置 1~12，默认不设置 13） 1 TIME 执行的时间 2 SEQNO 服务器的站点号

			3 SESS 操作的 SESS 地址 4 USER 执行的用户 5 TRXID 事务 ID 6 STMT 语句地址 7 APPNAME 客户端工具 8 IP 客户端 IP 9 STMT_TYPE 语句类型。分别为[ORA]表示原始语句（服务器从客户端收到的未加分析的语句）、[DDL]表示 DDL 语句、[INS]表示 INSERT 语句、[DML]表示 DML 语句、[CAL]表示 CALL 语句、[UPD] 表示 UPDATE 语句、[DEL] 表示 DELETE 语句、[SEL]表示 SELECT 语句、[LGN]表示登录登出语句 10 INFO 记录内容，记录当前执行的 SQL 语句 11 RESULT 运行结果，包括运行用时、影响行数和 EXEC_ID（可能没有） 12 THRD 线程地址 13 CLIENT_INFO 客户端主机名（不能与 0 同时使用）
USER_MODE	0	动态，	SQL 日志按用户过滤时的过滤模式，取值

		系统级	0：关闭用户过滤 1：白名单模式，只记录列出的用户操作的 SQL 日志 2：黑名单模式，列出的用户不记录 SQL 日志
USERS	空串	动态， 系统级	打开 USER_MODE 时指定的用户列表。格式为：用户名：用户名：用户名
EXECTIME_PREC_FLAG	0	动态， 系统级	设置 SQL 日志中执行时间 EXECTIME 的时间单位。 0：单位为毫秒 MS；1：单位为微秒 US；2：表示 MIN_EXEC_TIME 的过滤单位为微秒 US
CLOSE_IF_WRITE_ERR OR	1	动态， 系统级	当 SQL 日志遇到因空间不足导致的写入失败时，是否关闭 SQL 日志。0：否，继续尝试写入；1：是，关闭 SQL 日志
FILE_NAME_FLAG	1	动态， 系统级	当前活跃的日志文件是否带有时间后缀。0：否，SWITCH_MODE 打开时仍然按照原有文件命名规则定期做归档；1：是

2.1.1.4.2 sqllog.ini 用法

sqllog.ini 在 INI 参数 SVR_LOG=1 时启用。

如果在服务器启动过程中，修改了 sqllog.ini 文件，须调用过程 SP_REFRESH_SVR_LOG_CONFIG()才会生效。需要注意的是，sqllog.ini 文件的修改可能导致日志文件的命名规则发生变化，但旧配置产生的日志文件不会自动删除。S

sqllog.ini 配置文件的内容分为全局配置区和模式配置区。

全局配置区为 sqllog.ini 文件头中无模式的配置区。全局配置区内允许配置所有的配置项，包括全局配置项（参数 BUF_TOTAL_SIZE、BUF_SIZE、BUF_KEEP_CNT、CLOSE_IF_WRITE_ERROR、FILE_NAME_FLAG）和模式内配置项（除全局配置项外的其他参数）。其中，全局配置项对全部模式生效，配置的参数的优先级为：模式配置区配置的值>全局配置区配置的值>缺省值。

模式配置区位于全局配置区后面，以“[模式名]”开头，可配置表 2.33 sqllog.ini 的配置项中除了全局配置项之外的所有参数，只对本模式生效。

下面以一个简单的 sqllog.ini 为例。配置好 sqllog.ini 后，需同时开启 INI 参数 SVR_LOG=1，SVR_LOG_NAME=SLOG_ALL 之后，sqllog.ini 和 SLOG_ALL 模式参数才能生效。

```
#   以#开头的行表示是注释
#   全局配置区
BUF_TOTAL_SIZE              = 10240          #SQLs Log Buffer Total Size(K)(1024~1024000)
BUF_SIZE                    = 1024           #SQLs Log Buffer Size(K)(50~102400)
BUF_KEEP_CNT                = 6              #SQLs Log buffer keeped count(1~100)
```

```
#   模式配置区。此处模式名为 SLOG_ALL
[SLOG_ALL]
    FILE_PATH      = ../log
    PART_STOR      = 0
    SWITCH_MODE    = 2
    SWITCH_LIMIT     = 128
    ASYNC_FLUSH     = 1
    FILE_NUM = 5
    ITEMS      = 0
    SQL_TRACE_MASK    = 1
    MIN_EXEC_TIME = 0
    USER_MODE     = 0
    USERS =
```

当 SVR_LOG 为 1 时，但 sqllog.ini 不存在，或 sqllog.ini 存在但是配置的多个模式名均不在 INI 参数 SVR_LOG_NAME 中的情况下，按照各配置项的系统默认值记录 SQL 日志。在系统默认值中，模式名为<DEFAULT>。

通过动态视图 V$DM_SQLLOG_INI 和 V$DM_SQLLOG_CONFIG 可以分别查询 sqllog.ini 文件和内存中的 SQL 日志配置参数，由于视图中使用<PUBLIC>来标记全局配置参数，并且使用<DEFAULT>来标记系统默认配置，因此 sqllog.ini 中不允许指定模式名为<PUBLIC>或<DEFAULT>。

sqllog.ini 中可以配置多个模式配置区。不同模式配置区中的配置项（数量或取值）可以不同。例如：记录过滤掩码，刷盘方式，文件切换方式等。当各模式的 items 设置不同时，使用其并集。仅当配置的模式名存在于 INI 参数 SVR_LOG_NAME 中时，该模式名对应配置才会生效。当配置为多模式时，会在日志文件名中的实例名后面加上该日志文件对应的模式名。

例 在 INI 参数 SVR_LOG=1 时，sqllog.ini 文件中配置了两个模式，模式名分别是 SLOG_ALL 和 SLOG_LOGIN，SLOG_ALL 模式指定日志中记录全部记录，SLOG_LOGIN 模式指定日志中仅记录数据库登录操作。

```
BUF_TOTAL_SIZE              = 10240          #SQLs Log Buffer Total Size(K)(1024~1024000)
BUF_SIZE                    = 1024           #SQLs Log Buffer Size(K)(50~102400)
BUF_KEEP_CNT                = 6              #SQLs Log buffer keeped count(1~100)

[SLOG_ALL]
```

```
    FILE_PATH      = ../log
    PART_STOR      = 0
    SWITCH_MODE    = 2
    SWITCH_LIMIT     = 128
    ASYNC_FLUSH     = 1
    FILE_NUM = 5
    ITEMS      = 0
    SQL_TRACE_MASK    = 1
    MIN_EXEC_TIME = 0
    USER_MODE     = 0
    USERS =


[SLOG_LOGIN]
    FILE_PATH      = ../log
```

```
PART_STOR      = 0

SWITCH_MODE    = 2

SWITCH_LIMIT     = 128

FILE_NUM = 5

ITEMS      = 0

SQL_TRACE_MASK    = 31
```

2.1.1.5 dmthrd.ini

dmthrd.ini 用于为线程类型绑定 CPU 核心，当且仅当 INI 参数 DMTHRD_INI=1 时使用。

2.1.1.5.1 dmthrd.ini 配置项

dmthrd.ini 支持的线程类型请参考下表。

表2.34 dmthrd.ini 支持的线程类型

线程类型	说明
DM_AUDIT_THD	审计日志刷盘线程
DM_TIMWRK_THD	定时器线程
DM_RAPPLY_THD	日志重演工作线程，备用节点从主节点获取重演日志
DM_RAPREDO_THD	日志并行重演线程
DM_IOCP_THD	SESS 线程池打开时的任务监控线程
DM_WRKGRP_THD	会话连接并发处理线程
DM_SQLLOG_THD	DMServer 的 DMSQL 写线程
DM_RTRD_THD	高可用集群远程读线程
DM_GBSWRK_THD	全局闩服务工作线程
DM_GLSWRK_THD	全局锁服务工作线程
DM_GTVWRK_THD	全局事务视图工作线程
DM_AUX_THD	全局事务视图 AUX 系统线程

DM_HIO_THD	HUGE 文件系统 IO 线程
DM_OSIO_THD	系统 IO 线程
DM_PURGE_THD	清除线程
DM_LBSWRK_THD	本地闩服务工作线程
DM_LLSWRK_THD	本地锁服务工作线程
DM_LTVWRK_THD	本地事务视图工作线程
DM_REDOLOG_THD	日志重做线程，日志刷盘
RLOG4_ENC_CMPR_THD	日志加密压缩线程
DM_ASM_IO_THD	DMASM 系统 IO 线程
DM_ASYNC_SEND_THD	异步归档发送线程
DM_RSYSWRK_THD	异步归档工作线程
DM_LLOG_THD	逻辑日志线程
DM_SCHED_THD	调度线程，接管系统中所有需要定时调度的任务

DM_PWR_THD	PWR 日志刷盘线程
DM_CHKPNT_THD	检查点刷盘线程
DM_TRXBRO_THD	活动事务视图广播线程
DM_LPQ_THD	本地并行工作线程
DM_TSKWRK_THD	解析执行 SQL 语句的任务工作线程
DM_SQL_THD	会话工作线程

2.1.1.5.2 dmthrd.ini 用法

dmthrd.ini 在 INI 参数 DMTHRD_INI=1 时启用。

dmthrd.ini 中为线程类型绑定 CPU 核心的语法如下:

```
<线程类型名> = [<最小值>, <最大值>]

或

<线程类型名> = [<绑定值>]
```

<线程类型名>：指定线程类型，支持的线程类型请参考 2.1.1.5.1 dmthrd.ini 配置项。

[<最小值>,<最大值>]：非负整数，指定 CPU 核心的绑定范围。例如设置 DM_AUDIT_THD=[0,2]，表示创建审计日志刷盘线程时会循环绑定编号为 0、1、2 的三个 CPU 核心。

[<绑定值>]：非负整数，表示仅绑定一个 CPU 核心，例如设置 DM_AUDIT_THD=[2]，表示仅绑定编号值为 2 的 CPU 核心，等同于 DM_AUDIT_THD=[2,2]。

线程类型绑定 CPU 核心的注意事项如下：

1. <最小值>或者<最大值>缺省时，默认值为 0。例如 DM_AUDIT_THD=[,1]等同于 DM_AUDIT_THD=[0,1]；

2. <最小值>不能大于<最大值>，否则生成错误日志，并将<最大值>作为<最小值>。例如 DM_AUDIT_THD=[2,1]等同于 DM_AUDIT_THD=[1,1]；

3. <最大值>或<绑定值>不能大于当前机器的 CPU 核心数量，否则生成错误日志，并将其调整为 CPU 核心数量减 1；

4. 如果对同一个线程类型多次绑定 CPU 核心，则以最后一次绑定范围或绑定值为准；

5. 所有线程类型默认不绑定 CPU 核心。

下面以一个简单的 dmthrd.ini 为例。配置好 dmthrd.ini 后，需开启 INI 参数 DMTHRD_INI=1，dmthrd.ini 才能生效。

```
#   以#开头表示是注释

DM_TSKWRK_THD  = [0,15]          #创建 DM_TSKWRK_THD 类型线程时循环绑定编号在 0~15 范围内的 CPU 核心

DM_SQL_THD = [16,31]             #创建 DM_SQL_THD 类型线程时循环绑定编号在 16~31 范围内的 CPU 核心
```

2.1.1.6 dmtimer.ini

dmtimer.ini 用于配置定时器，用于数据守护中记录异步备库的定时器信息中记录异步复制的定时器信息。dmtimer.ini 的配置项见下表。

表2.35 dmtimer.ini 的配置项

配置项	配置含义
[TIMER_NAME1]	定时器名称。例如，名为 RT 的定时器，名称书写方式为[RT]
TYPE	定时器调度类型: 1：执行一次。时间上只需要指定 DURING_START_DATE 即可，不需要指定 START_TIME、END_TIME 和 DURING_END_DATE 2：按日执行 3：按周执行 4：按月执行的第几天 5：按月执行的第一周 6：按月执行的第二周 7：按月执行的第三周

	8：按月执行的第四周 9：按月执行的最后一周 10：按 REPEAT_INTERVAL 指定的日历表达式的频率进行。不再需要通过 START_TIME、END_TIME、DURING_START_DATE 和 DURING_END_DATE 指定时间
FREQ_MONTH_WEEK_INTERVAL	间隔月或周数。这个列的可选值与上面的不同调度类型有关。表示执行的频率。说明如下: 当 TYPE=1 时，这个值无效，系统不会做检查。 当 TYPE=2 时，表示每隔几天执行，值的有效范围为 1 到 100。 当 TYPE=3 时，表示的是每隔几个星期执行，值的有效范围为 1 到 100。 当 TYPE=4 时，表示每几个月中的某一天执行，值的有效范围为 1 到 100。 当 TYPE 为 5、6、7、8 或 9 时，都表示每几个月的某一周执行，值的有效范围为 1 到 100
FREQ_SUB_INTERVAL	间隔天数。这个列的可选值与上面的不同调度类型有关。表示执行的频率，在 FREQ_MONTH_WEEK _INTERVAL 基础上，继续指定更为精细的频率。说明如下: 当 TYPE=1 或 2 时，这个值无效，系统不会做检查。 当 TYPE=3 时，表示的是某一个星期的星期几执行，可以同时选中七天中的任意几天。取值范围 1~127。具体如何取值，请用户参考如下规则。

	因为每周有七天，所以 DM 用七位二进制来表示选中的日子。从最低位开始算起，依次表示周日、周一...周五、周六。选中周几，就将该位置 1，否则 0。例如：选中周二和周六，7 位二进制就是 1000100，转化成十进制就是 68，所以 FREQ_SUB_INTERVAL 就取值 68。 当 TYPE=4 时，表示将在一个月的第几天执行，此时这个列的取值范围为 1 到 31 当 TYPE 为 5、6、7、8 或 9 时，都表示将在某一周内第几天执行，有效值为 1 到 7，分别表示从周一到周日
FREQ_MINUTE_INTERVAL	间隔分钟数。当 TYPE 为 2、3、4、5、6、7、8 或 9 时，在 FREQ_MONTH_WEEK_INTERVALHE 和 FREQ_SUB_INTERVAL 的基础上，继续指定更为精细的频率。 表示一天内每隔多少分钟执行一次。取值范围 0~1440，0 表示一天内执行一次
REPEAT_INTERVAL	当 TYPE 为 10 时使用，用来指定的日历表达式。语法详见《DM8 系统包使用手册》手册 DBMS_SCHEDULER 包中关于 REPEAT_INTERVAL 的介绍
START_TIME	定时器开始时间，TIME 类型。例如：20:59:59.999999
END_TIME	定时器结束时间，TIME 类型
DURING_START_DATE	指定定时器的有效日期范围的起始日期，必须是有效的日期字符串，不可以为空。TIMESTAMP 类型。例如 2021-12-15 10:00:00
DURING_END_DATE	指定定时器的有效日期范围的结束日期，当 NO_END_DATE_FLAG=1 时可以为空。当 NO_END_DATE_FLAG=0 时不可以为空，必须是有效的日期字符串，同时必须晚于 DURING_START_DATE 日期。TIMESTAMP 类型

NO_END_DATE_FLAG	是否无结束日期标记。1 是，0 否
DESCRIBE	定时器描述
IS_VALID	定时器有效标记，缺省为 0。 0：表示已关闭定时器 1：表示已启用定时器

2.1.2 客户端配置文件

dm_svc.conf 是一个客户端配置文件，它包含了 DM 各接口和客户端工具所需要配置的一些参数。它必须和接口/客户端工具位于同一台机器上才能生效。

初始 dm_svc.conf 文件在 DM 安装时自动生成。不同平台的生成目录有所不同。

1. 32 位的 DM 安装在 Win32 操作平台下，此文件位于%SystemRoot%\system32 目录；
2. 64 位的 DM 安装在 Win64 操作平台下，此文件位于%SystemRoot%\system32 目录；
3. 32 位的 DM 安装在 Win64 操作平台下，此文件位于%SystemRoot%\SysWOW64 目录；
4. 在 Linux 平台下，此文件位于/etc 目录。

可以通过设置操作系统环境变量 DM_SVC_PATH 来修改 dm_svc.conf 文件所在目录路径。

2.1.2.1 dm_svc.conf 配置项

2.1.2.1.1 JDBC/.NET PROVIDER/NODE JS/Go 接口

下表中列举了 DM 对 JDBC/.NET PROVIDER/NODE JS/Go 接口配置项的支持情况，其中✓表示支持，×表示不支持。

表2.36 JDBC/.NET PROVIDER/NODE JS/Go 接口配置项介绍

配置项	缺省值	简述	JDBC	.NET PROVIDER	NODE JS	GO
服务名	无	连接服务名，参数值格式为 ip[:port],ip[:port],.....	√	√	√	√
ADDRESS_REMAP	无	库地址重定向，格式 ADDRESS_REMAP =(IP:PORT, IP:PORT)，多个: ADDRESS_REMAP =(IP:PORT, IP:PORT) ADDRESS_REMAP =(IP:PORT, IP:PORT)	√	×	√	√
ALWAYS_ALLOW_COMMIT	TRUE	在自动提交开关打开时，是否允许手动提交回滚；取值 1/0 或 true/false；默认 true	√	×	√	√
APP_NAME	无	客户端应用程序名称	√	√	√	√
AUTO_COMMIT	TRUE	是否自动提交；取值 1/0 或 true/false；默认 true	√	×	√	√
EP_SELECTOR	0	连接数据库时采用何种模型建立连接。在 JDBC 中，该配置项的取值为大于或等于 0 的整数，0：依次选取列表中的不同节点建立连接，	√	√	√	√

		使得所有连接均匀地分布在各个节点上；其余大于 0 整数表示选择列表中第几个节点建立连接，只有当前节点无法建立连接时才会选择下一个节点进行连接，若配置的值超过服务名列表的节点数，则连接第（epSelector % 服务名列表节点数）个节点； 在.NET PROVIDER、NODE.JS、GO 中，该配置项的取值为 0 或 1， 0：依次选取列表中的不同节点建立连接，使得所有连接均匀地分布在各个节点上；1：选择列表中最前面的节点建立连接，只有当前节点无法建立连接时才会选择下一个节点进行连接				
AUTO_RECONNECT （DO_SWITCH 别名）	0	连接发生异常或一些特殊场景下连接处理策略。0：关闭连接；1：当连接发生异常时自动切换到其他库，无论切换成功还是失败都会抛一个 SQLEXCEPTION，用于通知上层应用进行事务执行失败时的相关处理；2：配合 EP_SELECTOR=1 使用，如果服务名列表前面的节点恢复了，将当前连接切换到前面的节点上；4：保持各节点会话动态均	✓	×	✓	✓

		衡，通过后台线程检测节点及会话数变化，并切换连接使之保持均衡。 也可以将 AUTO_RECONNECT 置为上述几个值的组合值，表示同时进行多项配置，如置为 3 表示同时配置 1 和 2				
BATCH_ALLOW_MAX_ERRORS	0	开启批量执行出错时继续时，批量执行最大可容错行数。达到上限则停止执行，但不报错。如设置 0 则无限制。默认 0	✓	✓	✓	✓
BATCH_CONTINUE_ON_ERROR（CONTINUE_BATCH_ON_ERROR 别名）	FALSE	批量执行出错时是否继续执行；缺省为 false；取值 true/True，false/False；true/True：是，false/False：否	✓	✓	✓	✓
BATCH_NOT_ON_CALL	FALSE	执行 SQL 为存储过程时，是否禁止批量执行。此处的批量执行是指允许一次执行多行参数。TRUE 是，采用非批量，FALSE 否，采用批量。缺省为 FALSE	✓	✓	✓	✓
BATCH_TYPE	1	批处理模式；1 表示批量绑定执行　2 表示一行一行执行；默认 1	✓	×	✓	✓

BUF_PREFETCH	0	结果集 fetch 预取消息 buffer 大小；单位 KB，有效值范围 32~65535。默认 0 表示按服务器配置，若结果集上指定了 fetchSize 会自动预估大小	√	×	√	√
BUFFER_TYPE	0	消息缓存类型。取值 0：byte 数值；1：java 堆内存 buffer；2：直接内存 buffer	√	×	×	×
CIPHER_PATH	无	第三方加密算法的路径	√	×	×	√
CLOB_AS_STRING	FALSE	clob 类型列调用 resultSetMetaData 的 getColumnType()映射为 Types.VARCHAR 类型；取值 1/0 或 true/false；默认 false	√	×	×	×
CLUSTER	无	连接的集群类型；DSC 表示数据共享集群	√	×	√	√
COLUMN_NAME_CASE	无	列名大小写统一；取值 upper：全部大写；lower：全部小写	√	×	√	√
COLUMN_NAME_UPPER_CASE	FALSE	列名转换为大写字母；取值 1/0 或 true/false；默认 false	√	×	√	√
COMPATIBLE_MODE	无	兼容其他数据库；取值为数据库名称：oracle 表示兼容 oracle，mysql 表示兼容 mysql	√	×	√	√

COMPRESS（COMPRESS_MSG 别名）	0	是否压缩消息。取值范围 0~2。0 表示不压缩；1 表示完全压缩，采用第三方压缩包进行压缩；2 表示优化的压缩，只有当消息长度超过 8192 字节并且非本地连接，才会采用第三方压缩包进行压缩，其他情况不压缩。缺省为 0。采用的第三方压缩包类型由 compressID 指定	✓	×	✓	✓
COMPRESS_ID	0	消息压缩算法标识，最终与服务器支持情况协商决定。取值 0 和 1。缺省为 0。 0 指定 zip 算法，zip 算法由第三方压缩包 jzlib-X.X.X.jar 提供；1 指定 snappy 算法，snappy 算法由第三方压缩包 snappy-java-X.X.X.jar 提供。X.X.X 表示版本号。 指定了 compressID 参数后，用户 JAVA 代码中须引用相应的第三方压缩包。第三方压缩包位于 DM 安装目录 \tool\dropins\com.dameng\plugins\com.dameng.third 中	✓	×	✓	✓

COMPRESS_MSG（别名 COMPRESS）	0	是否启用消息压缩。0：不启用；1：启用；2：自适应压缩。即自动选取长度较长的通信消息进行压缩；短消息不需要压缩，直接传输效率更高	√	×	√	√
CONNECT_TIMEOUT	5000	连接超时时间，单位为毫秒。0 表示无限制	√	×	√	√
CONTINUE_BATCH_ON_ERROR	FALSE	批量执行出错时是否继续执行；默认 false；取值（true/True，false/False）	√	×	√	√
DATABASE_PRODUCT_NAME	无	数据库产品名称	√	×	×	×
DB_ALIVE_CHECK_FREQ	0	检测数据库是否存活的频率；单位 ms，有效值范围 0~2147483647，0 表示不检测；默认 0	√	×	×	×
DBMD_CHK_PRV	TRUE	编目函数是否进行权限检测；取值 1/0 或 true/false；默认 true	√	×	×	×
DO_SWITCH	0	连接发生异常或一些特殊场景下连接处理策略；0 表示关闭连接；1 表示当连接发生异常时自动切换到其他库，无论切换成功还是失败都会抛一个 SQLException，用于通知上层应用进行事务执行失败时的	√	×	√	√

		相关处理； 2 包含 1 的功能，并配合 epSelector 使用，检测故障库的恢复，恢复后将当前连接切换回前面的数据库				
ENABLE_RS_CACHE	0	是否进行客户端结果集缓存。0：不进行；1：进行	✓	×	✓	✓
ESCAPE_PROCESS	TRUE	是否进行语法转义处理；取值 1/0 或 true/false；默认 true	✓	✓	✓	✓
HOST	localhost	主库地址，包括 IP 地址、localhost 或者配置文件中主库地址列表对应的变量名	✓	✓	×	×
IGNORE_CASE	TRUE	结果集列名是否忽略大小写；取值 1/0 或 true/false；默认 true	✓	×	×	×
IS_BDTA_RS	TRUE	是否使用列模式结果集，需同步服务器开启该功能（即设置 INI 参数 RS_BDTA_FLAG 取值为 2）；取值 1/0 或 true/false；默认 true	✓	✓	✓	✓
KERBEROS_LOGIN_CONF_PATH	无	用户名加前缀"///"标识开启 Kerberos 认证，该参数指定 Kerberos 认证登录配置文件路径	✓	×	×	×
KEY_WORDS（KEYWORDS 别名）	无	标识用户关键字，所有在列表中的字符串，如果以单词的形式出现在 sql 语句中，则这个单词会被加上双引号；默认为空串	✓	✓	✓	✓

KEYWORDS	无	标识用户关键字，所有在列表中的字符串，如果以单词的形式出现在 sql 语句中，则这个单词会被加上双引号。该参数主要用来解决用户需要使用 DM8 中的保留字作为对象名使用的状况	✓	✓	✓	✓
LANGUAGE	操作系统语言	当前数据库服务器使用的语言，会影响帮助信息错误和提示信息。支持的选项为：CN（表示中文）和 EN（表示英文）。可以不指定，若不指定，系统会读取操作系统信息获得语言信息，建议有需要才指定	✓	✓	✓	✓
LOB_MODE	1	大字段数据获取模式；1 表示 get 数据时从服务器段获取，2 表示结果集生成时将大字段数据完整缓存到本地；默认 1	✓	×	✓	✓
LOG_DIR	无	日志等其他一些 JDBC 过程文件生成目录，默认为 jvm 当前工作目录	✓	✓	✓	✓
LOG_FLUSH_FREQ	60	日志刷盘频率；单位 s，有效值范围 0~2147483647。JDBC 默认 30；NODE JS 默认 30；Go 默认 10	✓	×	✓	✓
LOG_LEVEL	off	生成日志的级别，日志级别按从低到高依次如下，高级别同时记录低级别的信息，缺省为 off	✓	✓	✓	✓

		off：不记录；assert：断言日志信息（仅 JDBC 支持）；error：只记录错误日志；warn：记录警告信息（仅 JDBC、NODE JS、Go 支持）；sql：记录 sql 执行信息；info：记录全部执行信息；DEBUG：记录 debug 信息（仅 JDBC 和 NODE JS 支持）；all：记录全部（仅 JDBC、NODE JS、Go 支持）				
LOGIN_CERTIFICATE	无	指定登录加密用户名密码公钥所在的路径，一旦配置即认为开启了客户端的证书加密用户名密码模式	✓	×	✓	×
LOGIN_DSC_CTRL	0	服务名连接数据库时是否只选择 dsc control 节点的库。0：否；1：是 注：不建议与 EP_SELECTOR 同时使用	✓	×	✓	✓
LOGIN_MODE	4	指定优先登录的服务器模式。0：优先连接 Primary 模式的库，Normal 模式次之，最后选择 Standby 模式；1：只连接主库；2：只连接备库；3：优先连接 Standby 模式的库，Primary 模式次之，最后	✓	✓	✓	✓

		选择 Normal 模式；4：优先连接 Normal 模式的库，Primary 模式次之，最后选择 Standby 模式。 当启用读写分离时（即 rw_separate 不为 0），login_mode 不生效				
LOGIN_STATUS	0	连接时只连状态匹配的库； 0 表示不限制；3 表示 mount 状态；4 表示 open 状态；5 表示 suspend 状态；默认 0	✓	×	✓	✓
MARK_CONF	无	网络 M 标配置文件路径	✓	×	×	×
MAX_ROWS	0	结果集行数限制，超过上限结果集截断；有效值范围 0~2147483647，0 表示无限制；默认 0	✓	×	✓	✓
MPP_LOCAL	FALSE	是否 MPP 本地连接；取值 1/0 或 true/false；默认 false	✓	×	✓	✓
MPP_OPT	0	Mpp 集群批量插入数据的优化处理；范围 0~1；默认 0	✓	×	×	×
OS_AUTH_TYPE	0	指定操作系统认证用户类型，开启操作系统认证时，用户名使用系统用户名。0 表示关闭；1 表示 DBA；2 表示 SSO； 3 表示 AUDITOR；4 表示自适应。默认 0	✓	×	✓	×

OS_NAME	操作系统名称	操作系统名称	√	√	√	√
PASSWORD	无	登录密码	√	√	×	×
PORT	5236	端口号，服务器登录端口号	√	√	×	×
PSTMT_POOL_SIZE	0	prepare 语句句柄池大小；有效值范围 0~2147483647，0 表示关闭；默认 0	√	√	×	×
PSTMT_POOL_VALID_TIME	0	prepare 语句缓存的有效时间；单位 ms， 有效值范围 0~2147483647，0 表示无限制；默认 0	√	×	×	×
RESULT_SET_TYPE	ResultSet.TYPE_FORWARD_ONLY	指定默认创建结果集类型；取值为 java 标准中的 ResultSet.TYPE_FORWARD_ONLY，ResultSet.TYPE_SCROLL_INSENSITIVE，ResultSet.TYPE_SCROLL_SENSITIVE，默认 ResultSet.TYPE_FORWARD_ONLY	√	×	×	×

RS_CACHE_SIZE	20	设置结果集缓冲区大小，以 MB 为单位。NODE JS 接口的取值范围为 1~65536，其他接口的取值范围为 1~65535。该值如果设置太大，可能导致空间分配失败，进而使缓存失效	√	√	√	√
RS_REFRESH_FREQ	10	结果集缓存检查更新的频率，以秒为单位，有效值为 0~10000，如果设置为 0，则不需检查更新	√	×	√	√
RW_AUTO_DISTRIBUTE	TRUE	读写分离系统事务分发是否由驱动自动管理，取值范围 true/false，GO 和 JDBC 驱动都支持。false：事务分发由用户管理，用户可通过设置连接上的 readOnly 属性标记事务为只读事务	√	×	×	√
RW_HA	FALSE	是否开启读写分离系统高可用；取值 1/0 或 true/false；默认 false	√	×	√	√
RW_IGNORE_SQL	FALSE	读写分离 sql 语句不做类型校验的优化，所有 sql 都可能发到备库执行，开启优化检测 sql 开头 2 个单词判断语句类型，写操作不发往备库	×	×	√	√
RW_PERCENT	25	读写分离分发比例，有效值范围 0~100	√	√	√	√

RW_SEPARATE	0	是否启用读写分离。0：不启用；1：启用，备库由系统自动决定；2：启用，备库由客户端进行选择，且只会选择服务名中配置的节点；3：启用，备库由服务器通过消息告知；4：启用，备库由客户端进行选择，只连接事务一致性备库；5：启用，目前仅 JDBC 提供支持，为 GATEWAY 准备，连接 EPLIST 列表内第一个可用的备库	√	√	√	√
RW_STANDBY_RECOVER_TIME	60000	读写分离系统备库故障恢复检测间隔，单位 ms，有效值范围 0~2147483647，0 表示不回复；默认 60000	√	×	√	√
SCHEMA	登录用户默认模式	指定用户登录后的当前模式，默认为用户的默认模式	√	×	√	√
SESS_ENCODE	无	会话通信字符编码，默认使用库编码	√	×	√	√
SESSION_TIMEOUT	0	会话超时时间；单位 s，有效值范围 0~2147483647，0 表示无限制；默认 0	√	×	√	√
SSL_FILES_PATH	无	数据库端开启 ssl 通信加密，该参数指定 ssl 加密文件的路径	√	×	√	√
SSL_KEYSTORE_PASS	无	数据库端开启 ssl 通信加密，该参数指定 ssl 加密文件的指令	√	×	×	×

STAT_DIR	无	状态监控信息以文本文件形式输出的目录，默认为 jvm 当前工作目录	√	×	×	√
STAT_ENABLE	FALSE	是否启用状态监控；取值 1/0 或 true/false；默认 false	√	×	×	√
STAT_FLUSH_FREQ	10	状态监控统计信息写文件刷盘频率；单位 s，有效值范围 0~2147483647；0 表示不写文件；默认 10	√	×	×	√
STAT_HIGH_FREQ_SQL_COUNT	100	统计高频 sql top 行数；有效值范围 0~1000；默认 100	√	×	×	√
STAT_SLOW_SQL_COUNT	100	统计慢 sql top 行数；有效值范围 0~1000；默认 100	√	×	×	√
STAT_SQL_MAX_COUNT	1000	状态监控可以统计不同 sql 的个数；有效值范围 0~100000；默认 100000	√	×	×	√
STAT_SQL_REMOVE_MODE	eldest	执行的不同 sql 个数超过 statSqlMaxCount 时使用的淘汰方式；取值 latest/eldest；latest 表示淘汰最近执行的 sql，eldest 表示淘汰最老的 sql；默认 eldest	√	×	×	√
STMT_POOL_SIZE	15	语句句柄池大小；有效值范围 0~2147483647，0 表示关闭；默认 15	√	√	√	×
SWITCH_INTERVAL	200	在服务器之间切换的时间间隔，单位为毫秒，有效值范围 1~2147483647	√	√	√	√

SWITCH_TIMES	1	服务名连接数据库时，若未找到符合条件的库成功建立连接，将尝试遍历服务名中库列表的次数。有效值范围 1~9223372036854775807	√	√	√	√
TIME_ZONE	操作系统时区	指明客户端的默认时区，设置范围为：-779~840m，如 60 对应+1:00 时区	√	√	√	√
UKEY_NAME	无	Ukey 的用户名	√	×	×	×
UKEY_PIN	无	Ukey 的口令	√	×	×	×
UNIX_SOCKET_FILE	无	UNIX 系统与服务器使用本地 unix socket 方式通信使用的.sock 文件路径	√	×	×	×
USER	无	登录用户	√	√	×	×
LOG_SIZE	100 * 1024 * 1024	单个日志文件大小，单位字节。默认大小 100MB	×	√	×	×
SERVER	LOCALHOST	主库地址，包括 IP 地址、localhost 或者配置文件中主库地址列表对应的变量名	×	√	×	×

COMMAND_TIMEOUT	30	SQL 执行超时，单位 ms，有效值范围 0~2147483647	×	√	×	×
STMT_POOLING	1	是否启用句柄池，0 不启用，1 启用。默认启用	×	√	×	×
PSTMT_POOLING	0	是否启用 prepare 句柄池，0 不启用，1 启用。默认不启用	×	√	×	×
CONN_POOLING	0	是否启用连接池，0 不启用，1 启用。默认不启用	×	√	×	×
CONN_POOLSIZE	100	连接池大小，有效值范围 0~2147483647。默认值 100	×	√	×	×
CONN_POOL_CHECK	0	是否启用连接有效性检查，0 不启用，1 启用。默认不启用	×	√	×	×
TRACE	NONE	是否启用.NET Provider 的 TRACE 功能。NONE：不启用；DEBUG：打印到控制台；NORMAL：打印到执行目录下的“PROVIERTRACE.TXT”文件中；TRACE：打印到执行目录下的“PROVIERTRACE.TXT”文件中，比 NORMAL 内容要更详细一些；THREAD：每个线程的 TRACE 分别打印到执行目录下的“PROVIERTRACE 线程号.TXT”文件中	×	√	×	×
AFTER_GET_MORE_RESULTS	1	调用 JDBC 标准接口 getMoreResults()后是否关闭之前的结果集；取值 1/2/3；缺省为 1；1：关闭当前一个结果集；2：保持当前结果集打开；3：关闭当前所有结果集	√	×	×	×

ALLOW_RANGE	5	允许动态负载均衡误差范围的百分比，百分比越大表示允许的误差范围越大，取值范围 0~50	√	×	×	×
CHECK_EXEC_TYPE	FALSE	是否检查 executeXXX 接口执行的 SQL 语句类型与接口是否匹配；取值 1/0 或 true/false；1/true：是，0/false：否	√	×	×	×
CHECK_FREQ	300000	服务名连接数据库时，循环检测连接是否需要重置的时间间隔。即每间隔设定时间检测连接对象是否发生改变，若连接对象发生改变，JDBC 连接会自动重置到新对象。单位 ms，取值范围 0~2147483647	√	×	×	×
DB_ALIVE_CHECK_TIMEOUT	10000	检测数据库是否存活的连接超时时间，如果该时间内未连接成功即认为数据库故障；单位 ms，取值范围 1~2147483647	√	×	×	×
DMSVCCONF	无	指定 url 属性配置文件所在路径	√	×	×	×
ERR_MAP	无	自定义 DM 错误码和 Oracle 错误码的映射关系。 格式：errMap=(dmCode:oracleCode, ..., dmCode:oracleCode) 例如：errMap=(-2223:955, -2207:2289)	√	×	×	×

GEN_KEY_NAME_CASE	upper	调用 preparedStatement(String sql, String[] columnName)接口 prepare sql 时可同时指定列名，用于在表数据增删改后返回指定的列值。该参数用于指定这里的列名的大小写。取值 0/1/2 或 none/upper/lower。取值 0 或 none:保持用户输入不变；1 或 upper:将用户输入转为大写；2 或 lower：将用户输入转为小写	√	×	×	×
IGNORE_WARNINGS	默认为空	用于设置忽略指定的 SQLWarnings，取值与含义如下: all：忽略所有警告； sqlCode：忽略指定的警告； op sqlCode：忽略满足条件的警告，op 可以为符号>、<、>=、<=、!; (sqlCode,sqlCode)：忽略指定的多个警告，默认为空	√	×	×	×
injectArray	FALSE	绑定参数为数组时，是否直接注入到 SQL 语句中，非数组类型的参数不受影响，仅支持 select、insert、update、delete 语句；缺省为 false，取值范围（true，false）	×	×	√	×

		例如：conn.execute("select * from test where id in (?);", [[1,2,3]]) 实际执行的 SQL 语句会被改写为：select * from test where id in (1, 2, 3)				
LOCAL_ENCRYPT	FALSE	是否启用用户名密码本地加密；取值 1/0 或 true/false；1/true：是，0/false：否	✓	×	×	×
LOCAL_ENCRYPT_ENGINE	无	本地加密引擎名，不指定时使用系统内置的加密引擎，用户可指定自定义的加密引擎	✓	×	×	×
logBufferSize	32768	日志缓冲区大小	×	×	×	✓
logFlusherQueueSize	100	日志刷盘线程中等待刷盘的日志缓冲区队列大小	×	×	×	✓
PREPARE_OPTIMIZE	FALSE	是否对预编译 SQL 做优化；取值 1/0 或 true/false；1/true：是，0/false：否	✓	×	×	×
QUOTE_REPLACE	FALSE	是否将 SQL 语句中的双引号改成单引号；取值 1/0 或 true/false；1/true：是，0/false：否	✓	×	×	×

socketTimeout	0	网络通信链路超时时间；单位 ms，有效值范围 0~2147483647，0 表示无限制	√	√	√	√
sslCertPath	无	指定 ssl 加密证书文件的路径	×	×	×	√
sslKeyPath	无	指定 ssl 加密密钥文件的路径	×	×	×	√
ZIPKIN_SERVICE_NAME	dmService	zipkin 服务名，用于 zipkin 链路追踪，必须和 sql 拦截器 customFilter =com.dameng.zipkin.DmZipkinCustom 一起使用 例如：jdbc:dm://localhost:5236?customFilter=com.dameng.zipkin.DmZipkinCustom&zipkinServiceName=myDatabaseService	√	×	×	×
reconnectErrors	reconnectErrors=(-6001)	用于指定需要重连的异常，即当用服务名进行连接时，若发生指定的异常时将继续找下一个节点进行连接，发生其他异常则会抛出异常停止连接。 配置格式：reconnectErrors=(errorCode,errorCode,.....)。 errorCode 为数据库错误码（其中，超过了最大连接限制错误码-	√	×	×	×

		6001 不论设置与否，都会继续重连）。 例如：reconnectErrors=(-14009,-13217)				
KeepCursorAlive	FALSE	CallableStatement 关闭时，是否保持游标结果集开启。TRUE 是； FALSE 否	✓	×	×	×

2.1.2.1.2 DPI 接口

表2.37 DPI 接口配置项介绍

配置项	缺省值	简述
服务名	无	连接服务名，参数值格式为 ip[:port],ip[:port],.....
ADDRESS_REMAP	无	库地址重定向，格式 ADDRESS_REMAP =(IP:PORT, IP:PORT)，多个: ADDRESS_REMAP =(IP:PORT, IP:PORT) ADDRESS_REMAP =(IP:PORT, IP:PORT)
EP_SELECTOR	0	连接数据库时采用何种模型建立连接。0：依次选取列表中的不同节点建立连接，使得所有连接均匀地分布在各个节点上；其余大于 0 整数表示选择列表中第几个节点建立连

		接，只有当前节点无法建立连接时才会选择下一个节点进行连接，若配置的值超过服务名列表的节点数，则连接第（epSelector % 服务名列表节点数）个节点
AUTO_RECONNECT	0	连接发生异常或一些特殊场景下连接处理策略。0：关闭连接；1：当连接发生异常时自动切换到其他库，无论切换成功还是失败都会重置一下链路切换事件；
COMPRESS_MSG	0	是否启用消息压缩。0：不启用；1：启用；2：自适应压缩。即自动选取长度较长的通信消息进行压缩；短消息不需要压缩，直接传输效率更高
CONNECT_TIMEOUT	5000	连接超时时间，单位为毫秒。0 表示无限制
DEC2DOUB	0	指明在 DPI、DMODBC、DCI、DMPHP 和 DM PRO*C 中，是否将 DEC 类型转换为 DOUBLE 类型。dec(0,0)的类型描述为 double。0：不转换；1：转换
ENABLE_RS_CACHE	0	是否进行客户端结果集缓存。0：不进行；1：进行
KEYWORDS	无	标识用户关键字，所有在列表中的字符串，如果以单词的形式出现在 sql 语句中，则这个单词会被加上双引号。该参数主要用来解决用户需要使用 DM8 中的保留字作为对象名使用的状况

LANGUAGE	操作系统语言	当前数据库服务器使用的语言，会影响帮助信息错误和提示信息。支持的选项为：CN（表示中文）和 EN（表示英文）。可以不指定，若不指定，系统会读取操作系统信息获得语言信息，建议有需要才指定
LOGIN_CERTIFICATE	无	指定登录加密用户名密码公钥所在的路径，一旦配置即认为开启了客户端的证书加密用户名密码模式
LOGIN_DSC_CTRL	0	服务名连接数据库时是否只选择 dsc control 节点的库。0：否；1：是 注：不建议与 EP_SELECTOR 同时使用
LOGIN_MODE	4	指定优先登录的服务器模式。0：优先连接 Primary 模式的库，Normal 模式次之，最后选择 Standby 模式；1：只连接主库；2：只连接备库；3：优先连接 Standby 模式的库，Primary 模式次之，最后选择 Normal 模式；4：优先连接 Normal 模式的库，Primary 模式次之，最后选择 Standby 模式
PROTOCOL_TYPE	TCP	网络协议类型。可选 TCP/UDP/IPC/RDMA
RS_CACHE_SIZE	20	设置结果集缓冲区大小，以 MB 为单位。有效值为 1~65535，如果设置太大，可能导致空间分配失败，进而使缓存失效

RS_REFRESH_FREQ	10	结果集缓存检查更新的频率，以秒为单位，有效值为 0~10000，如果设置为 0，则不需检查更新
RW_PERCENT	25	读写分离分发比例，有效值范围 0~100
RW_SEPARATE	0	是否启用读写分离。0：不启用；1：启用；4：启用，备库由客户端进行选择，只连接事务一致性备库
SSL_CONFIG	无	指定用户的 SSL 信息，在配置 SSL_CONFIG 时，参数信息需要在同一行输入，输入信息中存在换行会导致初始化 SSL 环境失败。参数格式为：(USER=(用户名 1) SSL_PATH=(SSL 路径 1) SSL_PWD=(SSL key1))...
SWITCH_INTERVAL	200	在服务器之间切换的时间间隔，单位为毫秒，有效值范围 1~9223372036854775807
SWITCH_TIMES	1	服务名连接数据库时，若未找到符合条件的库成功建立连接，将尝试遍历服务名中库列表的次数。有效值范围 1~9223372036854775807
TIME_ZONE	操作系统时区	指明客户端的默认时区，设置范围为：-779~840m，如 60 对应+1:00 时区
WALLET_LOCATION	无	wallet 文件的路径。支持 disql、dmfldr、dimp/dexp、备份还原等客户端工具使用。wallet 文件用法请参考《DM8 安全管理》手册。

		该配置项仅支持全局配置，不支持服务名配置
DPI_TCP_DETECT	1	用于创建 TCP 的链路异常检测。1：创建；0：不创建
DETECT_TIMEOUT	0	用于设置链路异常检测超时时间，当链路检测时间超过设定时间没有响应，判定为链路异常，主动断开链路。取值范围：0~4294967294，0 表示没有超时，单位秒
RECONNECTERRORS	RECONNECTERRORS=(-6001)	用于指定需要重连的异常，即当用服务名进行连接时，若发生指定的异常时将继续找下一个节点进行连接，发生其他异常则会抛出异常停止连接。 配置格式：RECONNECTERRORS=(errorCode,errorCode,.....)。errorCode 为数据库错误码（其中，超过了最大连接限制错误码-6001 不论设置与否，都会继续重连）。 例如：RECONNECTERRORS=(-14009,-13217)
VARCHAR_MAXSZ_DESC	无	用于指定 VARCHAR 类型的最大描述长度。取值范围 1~32767，当服务器返回的 VARCHAR 定义长度超过设定值时，按照设定值返回描述长度。若配置的参数值超出有效取值范围，则忽略该配置。 建议谨慎使用该配置项，使用时必须保证设定值适应实际数据长度，否则存在数据获取不完整的风险

UKEY_LOGIN	0	配置服务名是否必须使用 UKEY 登录。0：非必须；1：必须使用 ukey 登录
MSG_VALIDATE	1	表明 dpi 接口解析获取服务器消息时是否开启消息有效性校验，对消息的部分字段长度和消息总长度进行有效性检验，检验失败报错返回。MSG_VALIDATE=(1)开启，MSG_VALIDATE=(0)关闭
LIBSSL_LOAD_PATH	无	用于指定 ssl 动态库和加密库的加载路径。 该配置项仅支持全局配置，不支持服务名配置
USER_ENCRYPT	1	表明 dpi 接口加密时是否加密用户名。0：不加密；1：加密

2.1.2.1.3 CYT 安全接口

表2.38 CYT 安全接口配置项介绍

配置项	缺省值	简述
CIPHER_PATH	无	第三方加密算法的路径

2.1.2.1.4 DIMP/DEXP 接口

表2.39 DIMP/DEXP 工具配置项介绍

配置项	缺省值	简述
DIRECT	Y	DIMP 专用，作用相当于 DIMP 工具的 FAST_LOAD 参数。表示是否使用快速装载。y：使用；n：不使用
DUMMY	P	DEXP 和 DIMP 均可使用。用于设置交互信息处理。P：提供交互界面。当导出文件已存在的时候，提供是否覆盖交互界面。Y：不提供交互界面，所有交互都按 YES 处理。N：不提供交互界面，所有交互都按 NO 处理。在使用 dexpdp/ dimpdp 的时候，此参数无需设置，不提供交互信息，全部按 YES 处理
WALLET_LOCATION	无	wallet 文件的路径。支持 disql、dmfldr、dimp/dexp、备份还原等客户端工具使用。wallet 文件用法请参考《DM8 安全管理》手册。 该配置项仅支持全局配置，不支持服务名配置
DUMP_CONN_STR	NONE	DEXP 和 DIMP 均可使用。指定导入导出默认连接串。DEXP 或 DIMP 没有指定 USERID 连接串时，使用该连接串
DUMP_PASSWORD	NONE	DEXP 和 DIMP 均可使用。指定导入导出默认连接密码。DEXP 或 DIMP 的 USERID 参数缺省密码时，使用该密码

2.1.2.1.5 DPC 接口

表2.40 DPC 接口配置项介绍

配置项	缺省值	简述
ENABLE_SSL	0	是否启用 ssl。1/Y/y：启用 SSL；其他值：不启用
SSL_CONFIG	无	在启用 SSL 的前提下，设置 SSL 值。在配置 SSL_CONFIG 时，参数信息需要在同一行输入，输入信息中存在换行会导致初始化 SSL 环境失败。格式如下：SSL_CONFIG=((USER=(用户名 1) SSL_PATH=(SSL 路径 1) SSL_PWD=(SSL key1)) (USER=(用户名 2) SSL_PATH=(SSL 路径 2) SSL_PWD=(SSL key2)))

2.1.2.1.6 Disql 工具

表2.41 Disql 工具配置项介绍

配置项	缺省值	简述
AUTO_LOGIN	无	DIsql 读取的用于自动登录的连接串。例如：AUTO_LOGIN=(SYSDBA/DMsys_123@192.168.6.7:5236)。DIsql 会优先连接 AUTO_LOGIN 中的服务器

WALLET_LOCATION	无	wallet 文件的路径。支持 Disql、dmfldr、dimp/dexp、备份还原等客户端工具使用。wallet 文件用法请参考《DM8 安全管理》手册。 该配置项仅支持全局配置，不支持服务名配置

2.1.2.1.7 DCI 接口

表2.42 DCI 接口配置项介绍

配置项	缺省值	简述
DCI_VERSION	0	控制 DCI 接口是否返回 DM 的版本信息。可取值 0/1，0 表示返回 DM 的版本信息，1 表示返回 Oracle 的版本信息。 该配置项仅支持全局配置，不支持服务名配置

2.1.2.2 dm_svc.conf 用法

2.1.2.2.1 简介

dm_svc.conf 配置文件的内容分为全局配置区和服务配置区。全局配置区在前；服务配置区在后，以“[服务名]”开头。其中，服务名配置项可以在全局配置区，也可以在此服务名对应的服务配置区之前，即在其对应的“[服务名]”之前。

dm_svc.conf 格式如下：

```
#  全局配置区
参数配置……


#  服务配置区
[服务名 1]
参数配置……


#  服务配置区
[服务名 2]
参数配置……
```

服务配置区中的配置优先级高于全局配置区。全局配置区的配置项影响所有会话，除非会话所属服务配置区中单独进行了配置，因此对于全局配置区配置项的设置修改需要慎重。

如果对 dm_svc.conf 的配置项进行了修改，需要重启客户端工具，修改的配置才能生效。另外，如果 dm_svc.conf 配置文件中包含中文，则必须保证该配置文件的编码与客户端编码一致。

2.1.2.2.2 应用场景

场景一 配置常规环境的 dm_svc.conf。此处常规环境是指服务名中的 IP 之间互不相关（既不是主备关系、也不在一个集群中）。

例 NORMAL 中的 192.168.0.1 和 192.168.0.2 是两个互不相关的 IP。

```
#  以#开头的行表示是注释
#  全局配置区
NORMAL=(192.168.0.1:5000,192.168.0.2:5236)
TIME_ZONE=(480)        #表示+8:00 时区
DIRECT=(Y)


#  服务配置区
#  常规环境，两个没有关系的 IP
[NORMAL]
TIME_ZONE=(540)        #表示+9:00 时区
LOGIN_MODE=(4)
SWITCH_TIMES=(3)
SWITCH_INTERVAL=(100)
```

场景二 配置集群环境的 dm_svc.conf。一个服务名中的所有 IP 均来自于同一个集群。

例 以 DMDSC 集群为例。

```
#  以#开头的行表示是注释
#  全局配置区
DMDSC1=(192.168.1.1:5236,192.168.1.3:5236)
DMDSC2=(192.168.1.5:5236,192.168.1.7:5236)
```

```
TIME_ZONE=(480)        #表示+8:00 时区


#DMDSC1 服务配置区
#以下配置目标连接 DMDSC1 服务名的第一个服务（192.168.1.1:5236），以间隔 1000 毫秒的节奏尝试遍历服务名中的各服务进行连接，若在遍历中成功连上符合要求的服务则立即返回；若没有，则继续。一共尝试 60 次遍历，若 60 次尝试后依然未连接上目标服务，则根据 LOGIN_MODE 的配置进行下一种连接的尝试。若连接建立时连上的是 2 号服务，而一段时间后 1 号服务可以正常连接了，当前连接也不会自动切换到 1 号服务。
[DMDSC1]
LOGIN_MODE=(4) #若是 DSC 主+单机备，建议这里配置成 0
SWITCH_TIMES=(60)
SWITCH_INTERVAL=(1000)
EP_SELECTOR=(1)
AUTO_RECONNECT=(1)


#DMDSC2 服务配置区
#以下配置目标连接 DMDSC2 服务名的第一个服务（192.168.1.5:5236），以间隔 1000 毫秒的节奏尝试遍历服务名中的各服务进行连接，若在遍历中成功连上符合要求的服务则立即返回；若没有，则继续。一共尝试 60 次遍历，若 60 次尝试后依然未连接上目标服务，则根据 LOGIN_MODE 的配置进行下一种连接的尝试。假设 2 号服务先连接成功，由于 AUTO_RECONNECT=(2)，因此当 1 号服务可以正常连接后当前连接会切换到 1 号服务。
[DMDSC2]
CLUSTER=(DSC)
LOGIN_MODE=(4) #若是 DSC 主+单机备，建议这里配置成 0
```

```
SWITCH_TIMES=(60)

SWITCH_INTERVAL=(1000)

EP_SELECTOR=(1)

AUTO_RECONNECT=(2)
```

场景三 配置常规环境和集群环境公用的 dm_svc.conf。

例 [NORMAL]为常规环境，[Data_Watch]为数据守护环境。服务名 NORMAL 中的 IP 互不相关，服务名 Data_Watch 中的 IP 为同一个数据守护环境中的 IP。

```
#  以#开头的行表示是注释

#  全局配置区

NORMAL=(192.168.0.1:5000,192.168.0.2:5236)

Data_Watch=(192.168.0.3:5236,192.168.0.4:4350)

TIME_ZONE=(480)       #表示+8:00 时区

DIRECT=(Y)



#  服务配置区

#  常规环境，两个没有关系的 IP

[NORMAL]

TIME_ZONE=(540)       #表示+9:00 时区

LOGIN_MODE=(4)

SWITCH_TIMES=(3)

SWITCH_INTERVAL=(100)



#  服务配置区

#  数据守护环境，一主一备，只连备库
```

```
[Data_Watch]
TIME_ZONE=(540)        #表示+9:00 时区
LOGIN_MODE=(2)
SWITCH_TIMES=(3)
SWITCH_INTERVAL=(100)
```

2.1.2.2.3 使用服务名

客户端工具可通过服务名连接上符合条件的服务器。至于连接上哪一台服务器，则由全局配置区和服务配置区中的参数共同决定。

例 DIsql 使用 dm_svc.conf 中的服务名 NORMAL，即可连接上 NORMAL=(192.168.0.1:5000,192.168.0.2:5236)中的某一个服务器。假设登录用户名和密码为 SYSDBA/DMsys_123。

```
disql SYSDBA/DMsys_123@NORMAL
```

2.2 控制文件

每个 DM 数据库都拥有两个控制文件，即 dm.ctl 和 dmtemp.ctl 控制文件，控制文件均为二进制文件。创建数据库时按照 CTL_PATH 参数自动创建 dm.ctl 控制文件，首次启动数据库时自动在 dm.ctl 所在目录下创建 dmtemp.ctl 控制文件。

dm.ctl 用于记录数据库必要的初始信息，主要包含以下内容：

1. 数据库名称；
2. 数据库服务器模式；
3. OGUID 唯一标识；
4. 数据库服务器版本；
5. 数据文件版本；
6. 数据库的启动次数；
7. 数据库最近一次启动时间；
8. 表空间信息，包括表空间名，表空间物理文件路径等，记录了所有数据库中使用的表空间，数组的方式保存起来；
9. 控制文件校验码，校验码由数据库服务器在每次修改控制文件后计算生成，保证控制文件合法性，防止文件损坏及手工修改。

dmtemp.ctl 用于记录临时表空间相关信息，主要包括临时表空间名以及临时表空间物理文件路径等。

在服务器运行期间，执行表空间的 DDL 等操作后，服务器内部需要同步修改控制文件内容。如果在修改过程中服务器故障，可能会导致控制文件损坏，为了避免出现这种情况，在修改控制文件时系统内部会执行备份操作。dm.ctl 备份文件命名格式为“dm_年月日时分秒_毫秒.ctl”，dmtemp.ctl 备份文件命名格式为“dmtemp_年月日时分秒_毫秒.ctl”。

备份策略如下：

◆ 策略一 在修改控制文件之前，先执行一次备份，确定控制文件修改成功后，再将备份删除，如果控制文件修改失败或中途出现故障，则保留备份文件。

◆ 策略二 在修改控制文件成功之后，根据 dm.ini 中指定的 CTL_BAK_PATH/CTL_BAK_NUM 对最新的控制文件执行备份，如果用户指定的 CTL_BAK_PATH 是非法路径，则不再生成备份文件，在路径有效的情况下，生成备份文件时根据指定的 CTL_BAK_NUM 判断是否删除老的备份文件。

注意:

- 如果控制文件存放在裸设备上，则【策略一】不会生效。
- 如果指定的 CTL_BAK_PATH 是无效路径，则【策略二】也不会生效。
- 如果【策略一】和【策略二】的条件都满足，则都会生效执行，否则只执行满足条件的备份策略，如果都不满足，则不会再生成备份文件。
- 如果是初始化新库，在初始化完成后，会在“SYSTEM_PATH/CTL_BAK”路径下对原始的控制文件执行一次备份。

2.3 数据文件

数据文件以 DBF 为扩展名，它是数据库中最重要的文件类型，一个 DM 数据文件对应磁盘上的一个物理文件或者达梦分布式数据库中的一个逻辑文件，数据文件是真实数据存储的地方，每个数据库至少有一个与之相关的数据文件。在实际应用中，通常有多个数据文件。

当 DM 的数据文件空间用完时，它可以自动扩展。可以在创建数据文件时通过 MAXSIZE 参数限制其扩展量，当然，也可以不限制。但是，数据文件的大小最终会受物理磁盘大小的限制。在实际使用中，一般不建议使用单个巨大的数据文件，为一个表空间创建多个较小的数据文件是更好的选择。

数据文件在物理上按照页、簇和段的方式进行管理，详细结构请参考第一章的内容。

数据文件按数据组织形式，可以分为如下几种:

1. B 树数据

行存储数据，也是应用最广泛的存储形式，其数据是按 B 树索引组织的。普通表、分区表、B 树索引的物理存储格式都是 B 树。

一个 B 树包含两个段，一个内节点段，存放内节点数据；一个叶子段，存放叶子节点数据。其 B 树的逻辑关系由段内页面上的记录，通过文件指针来完成。

当表上没有指定聚集索引时，系统会自动产生一个唯一标识 ROWID 作为 B 树的 key 来唯一标识一行。

2. 堆表数据

堆表的数据是以挂链形式存储的，一般情况下，支持最多 128 个链表，一个链表在物理上就是一个段，堆表采用的是物理 ROWID，在插入过程中，ROWID 在事先已确定，并保证其唯一性，所以可以并发插入，插入效率很高，且由于 ROWID 是即时生成，无需保存在物理磁盘上，也节省了空间。

3. 列存储数据

数据按列方式组织存储，包含每个列对应的存放列数据的一系列数据文件，以及存放列数据控制信息的辅助表。读取列数据时，只需要顺序扫描列数据文件和辅助表数据。在某些特殊应用场景下，其效率要远远高于行存储。

4. 位图索引

位图索引与 B 树索引不同，每个索引条目不是指向一行数据，而是指向多行数据。每个索引项保存的是一定范围内所有行与当前索引键值映射关系的位图。

数据文件中还有两类特殊的数据文件：ROLL 和 TEMP 文件。

1). ROLL 文件

ROLL 表空间的 DBF 文件，称为 ROLL 文件。ROLL 文件用于保存系统的回滚记录，提供事务回滚时的信息。回滚文件可被分为若干回滚段，每个事务的回滚页在回滚段中各自挂链，页内则顺序存放回滚记录。

2). TEMP 文件

TEMP.DBF 临时数据文件，临时文件可以在 dm.ini 中通过 TEMP_SIZE 配置大小。

当数据库查询的临时结果集过大，缓存已经不够用时，临时结果集就可以保存在 TEMP.DBF 文件中，供后续运算使用。系统中用户创建的临时表也存储在临时文件中。

2.4 重做日志文件

重做日志（即 REDO 日志）指在 DM 数据库中添加、删除、修改对象，或者改变数据，DM 都会按照特定的格式，将这些操作执行的结果写入到当前的重做日志文件中。重做日志文件默认以 log 为扩展名，可在初始化数据库时使用建库参数 RLOG_POSTFIX_NAME 指定重做日志文件的扩展名。每个 DM 数据库实例必须至少有 2 个重做日志文件，默认两个日志文件为 DAMENG01.log、DAMENG02.log，这两个文件循环使用。

重做日志文件因为是数据库正在使用的日志文件，因此被称为联机日志文件。

重做日志文件主要用于数据库的备份与恢复。理想情况下，数据库系统不会用到重做日志文件中的信息。然而现实世界总是充满了各种意外，比如电源故障、系统故障、介质故障，或者数据库实例进程被强制终止等，数据库缓冲区中的数据页会来不及写入数据文件。这样，在重启 DM 实例时，通过重做日志文件中的信息，就可以将数据库的状态恢复到发生意外时的状态。

重做日志文件对于数据库是至关重要的。它们用于存储数据库的事务日志，以便系统在出现系统故障和介质故障时能够进行故障恢复。在 DM 数据库运行过程中，任何修改数据库的操作都会产生重做日志，例如，当一条元组插入到一个表中的时候，插入的结果写入了重做日志，当删除一条元组时，删除该元组的事实也被写了进去，这样，当系统出现故障时，通过分析日志可以知道在故障发生前系统做了哪些动作，并可以重做这些动作使系统恢复到故障之前的状态。

2.5 归档日志文件

日志文件分为联机日志文件和归档日志文件。DM 数据库可以在归档模式和非归档模式下运行。非归档模式下，数据库会只将重做日志写入联机日志文件中进行存储；归档模式下，数据库会同时将重做日志写入联机日志文件和归档日志文件中分别进行存储。

联机日志文件指的是系统当前正在使用的日志文件。创建数据库时，联机日志文件通常被扩展至一定长度，其内容则被初始化为空，当系统运行时，该文件逐渐被产生的日志所填充。对日志文件的写入是顺序连续的。然而系统磁盘空间总是有限，系统必须能够循环利用日志文件的空间，为了做到这一点，当所有日志文件空间被占满时，系统需要清空一部分日志以便重用日志文件的空间，为了保证被清空的日志所“保护”的数据在磁盘上是安全的，这里需要引入一个关键的数据库概念——检查点。当系统中产生检查点时，系统会自动将日志缓冲区中的日志和数据缓冲区中的脏数据页全部写入磁盘，以保证当前日志所“保护”的数据页都已安全写入磁盘，这样日志文件即可被安全重用。

归档日志文件，就是在归档模式下，重做日志被连续写入到归档日志后，所生成了归档日志文件。归档日志文件以归档时间命名，扩展名默认与初始化数据库时指定的参数 RLOG_POSTFIX_NAME 一致，也可通过 INI 参数 LOG_FILE_POSTFIX_NAME 指定归档日志文件的扩展名。只有在归档模式下运行时，DM 数据库才会将重做日志写入到归档日志文件中。采用归档模式会对系统的性能产生影响，然而系统在归档模式下运行会更安全，当出现故障时其丢失数据的可能性更小，这是因为一旦出现介质故障，如磁盘损坏时，利用归档日志，系统可被恢复至故障发生的前一刻，也可以还原到指定的时间点，而如果没有归档日志文件，则只能利用备份进行恢复。

2.6 物理逻辑日志文件

物理逻辑日志，是按照特定的格式存储的服务器的逻辑操作，专门用于 DBMS_LOGMNR

包挖掘获取数据库系统的历史执行语句。当开启记录物理逻辑日志的功能时，这部分日志内容会被存储在重做日志文件中。

要开启物理逻辑日志的功能，需要满足下面两个条件:

首先，要设置 RLOG_APPEND_LOGIC 为 1、2、3 或者 4;

其次，通过设置参数 RLOG_IGNORE_TABLE_SET=1 或者建表（或修改表）时指定 ADD LOGIC LOG 开启。如果需要记录所有表的物理逻辑日志，设置 INI 参数 RLOG_IGNORE_TABLE_SET 为 1 即可；如果只需要记录某些表的物理逻辑日志，设置 INI 参数 RLOG_IGNORE_TABLE_SET 为 0，并在建表或者修改表的语法中使用 ADD LOGIC LOG。

2.7 备份文件

备份文件是对数据库中的数据等信息进行备份生成的文件，用于在数据丢失、损坏或系统故障时将数据库还原至正常状态。DM 数据库的备份包括物理备份和逻辑备份两种类型。

物理备份是对数据库物理文件（如数据文件、控制文件和日志文件等）的备份。物理备份成功后会在指定目录（缺省为 SYSTEM_PATH 下的 bak 目录）中生成备份文件（即备份集）。备份集包括备份元数据文件（后缀.meta）和备份片文件（后缀.bak）。物理备份的详情请参考《DM8 备份与还原》。

逻辑备份是对数据库逻辑组件（如表、视图和存储过程等数据库对象）的备份。逻辑备份成功后会在指定目录（缺省为 dexp 工具所在路径）中生成备份文件（后缀.dmp）。逻辑备份详情请参考《DM8_dexp 和 dimp 使用手册》。

用户可根据需要将备份文件拷贝至任何地方存放，并不会影响系统的运行。

2.8 SQL 日志文件

用户在 dm.ini 中配置 SVR_LOG 参数后就会打开 SQL 日志。

SQL 日志文件是一个纯文本文件。命名格式为“dmsql_实例名[_模式名][_用户名][_

日期_时间].log”。当 sqllog.ini 中存在多个有效模式名时，日志文件名中会添加“_模式名”，否则不添加；当 sqllog.ini 参数 PART_STOR=1 时，日志文件名中会添加“_用户名”，否则不添加；当 sqllog.ini 参数 SWITCH_MODE 不为 0 时，日志文件名中会添加“_日期_时间”，否则不添加。

SQL 日志文件缺省生成在 DM 安装目录的 log 子目录下面，管理员可通过 sqllog.ini 参数 FILE_PATH 设置其生成路径。

SQL 日志内容包含系统各会话上用户执行的 SQL 语句、参数信息、错误信息等。跟踪日志主要用于分析错误和分析性能问题,基于跟踪日志可以对系统运行状态有一个分析,比如,可以挑出系统现在执行速度较慢的 SQL 语句，进而对其进行优化。

系统中 SQL 日志的缓存是分块循环使用，管理员可根据系统执行的语句情况及压力情况设置恰当的日志缓存块大小及预留的缓冲块个数。当预留块不足以记录系统产生的任务时，系统会分配新的用后即弃的缓存块，但是总的空间大小由 sqllog.ini 参数 BUF_TOTAL_SIZE 控制，管理员可根据实际情况进行设置。

打开 SQL 日志会影响系统的性能，因此一般在需要查错和调优的时候才会打开。缺省情况下系统是关闭 SQL 日志的。若用户需要打开 SQL 日志，同时对日志的实时性并没有严格的要求，又希望系统性能好，此时可以从两方面进行改进：一设置 sqllog.ini 参数 SQL_TRACE_MASK 和 MIN_EXEC_TIME 只记录关注的相关记录，减少日志总量；二设置 sqllog.ini 参数 ASYNC_FLUSH 打开 SQL 日志异步刷盘提高系统性能。

用户可以调用系统过程 SP_SET_SQLLOG_INI() 或 SP_DELETE_SQLLOG_INI_MODE()来动态修改 sqllog.ini 文件的内容，修改后可以调用系统过程 SP_REFRESH_SVR_LOG_CONFIG()更新内存中对应的参数值，以使所做的修改生效。需要注意的是，sqllog.ini 文件的修改可能导致日志文件的命名规则发生变化，但旧配置产生的日志文件不会自动删除。

用户可以利用动态视图 V$DM_SQLLOG_INI 可以查询 sqllog.ini 文件中的 SQL 日志配置参数，利用动态视图 V$DM_SQLLOG_CONFIG 可以查询内存中的 SQL 日志配置参数。

2.9 事件日志文件

事件日志文件记录了 DM 数据库运行时的关键事件。例如：系统启动、关闭、内存申请失败、IO 错误等一些致命错误；数据库运行过程中的日志信息；备份还原过程中备份还原操作的阶段性信息等。

事件日志文件主要用于系统出现严重错误时进行查看并定位问题。事件日志简称 ELOG。事件日志文件随着 DM 数据库服务的运行一直存在。

事件日志文件打印的是中间步骤的信息，所以出现部分缺失属于正常现象。事件日志信息格式为：时间+日志类型（INFO/WARNING/ERROR/FATAL）+进程(database)+进程 ID(P 开头)+线程(dm_sql_thd/main_thread 等)+日志内容。

初始化过程中产生的 ELOG 会保存在 ELOG_PATH 参数指定的目录下，名称为“dminit+日期+时间”。例如：dminit20220914090452.log。

系统启动和运行过程中产生的 ELOG 保存在系统 log 目录下。

系统启动时，如遇到 dm.key、dm.ini 不存在等原因无法正常启动，因尚未获取到实例名，因此将生成的事件日志文件命名为“dm_unknown_年月”。例如：dm_unknown_202212。

当 ELOG_FLAG 设为 1 时，数据库运行过程中的事件日志都记录在名为“dm_实例名”的日志文件中，如 dm_DMSERVER.log。

当 ELOG_FLAG 设为 0 时，事件日志会被拆分记录到多个带有时间后缀的日志文件中。图 2.2 展示了 ELOG_FLAG 设为 0 时，9 月份的月度日志和分时日志。运行过程中，ELOG 日志文件的生成分两步完成：

第一步，系统将所有 ELOG 内容生成到当月的月度 ELOG 日志文件中，命名为“dm_实

例名_年月”。例如：9 月的月度 ELOG 日志文件为 dm_DMSERVER_202209.log。

第二步，当符合 SVR_ELOG_FREQ 切换频率的切换点来临时，系统将月度 ELOG 日志文件中最近生成的（1 月/1 日/1 时）日志，剪切并归档到新的（分月/分日/分时）ELOG 日志文件中，命名为“dm_实例名_日期_整点”，其中后缀中的整点为该日志首条记录的时间。不论切换频率为月、日或时，切换的时间点均为某时 0 分。例如：设置 SVR_ELOG_FREQ=2 按小时进行切换，则归档后的分时 ELOG 日志文件名为 dm_DMSERVER_20220914_09.log。

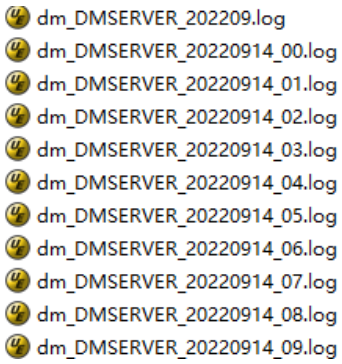


图 2.2 ELOG_FLAG 为 0 时的 ELOG 日志文件

通过查询动态视图 V$ELOG_FILE 可以获得当前实例的日志文件的信息。可以使用系统过程 SP_ELOG_FILE_DELETE 删除指定文件名的 ELOG 日志文件。

第3章 DM 内存结构

数据库管理系统是一种对内存申请和释放操作频率很高的软件，如果每次对内存的使用都使用操作系统函数来申请和释放，效率会比较低，加入自己的内存管理是 DBMS 系统所必须的。通常内存管理系统会带来以下好处：

1. 申请、释放内存效率更高；
2. 能够有效地了解内存的使用情况；
3. 易于发现内存泄露和内存写越界的问题。

DM 数据库管理系统的内存结构主要包括内存池、缓冲区、排序区、哈希区等。根据系统中子模块的不同功能，对内存进行了上述划分，并采用了不同的管理模式。

3.1 内存池

DMServer 的内存池包括共享内存池和其他一些运行时内存池。

动态视图 V$MEM_POOL 详细记录了当前系统中所有的内存池的状态，可通过查询这个动态视图掌握 DMServer 的内存使用情况。

3.1.1 共享内存池

共享内存池是 DMServer 在启动时从操作系统申请的一大片内存。

在 DMServer 的运行期间，经常会申请与释放小片内存，而直接向操作系统申请和释放内存时需要发出系统调用，此时可能会引起线程切换，降低系统运行效率。于是 DM 采用共享内存池的方式：一次向操作系统申请一片较大内存，作为共享内存池。当系统在运行过程中需要申请小片内存时，可在共享内存池内进行申请，当用完该内存时，再释放掉，即归还给共享内存池。

DM 系统管理员可以通过 DMServer 的配置文件（dm.ini）来对共享内存池的大小进行

设置，共享池大小的参数为 MEMORY_POOL，缺省大小为 500MB。如果在运行时所需内存大于配置值，共享内存池也可进行自动扩展，INI 参数 MEMORY_EXTENT_SIZE 指定了共享内存池每次扩展的大小，参数 MEMORY_TARGET 则指定了共享内存池扩展到超过该值后，空闲时会收缩到的大小。

3.1.2 运行时内存池

除了共享内存池，DMServer 的一些功能模块在运行时还会使用自己的运行时内存池。这些运行时内存池是从操作系统申请一片内存作为本功能模块的内存池来使用，如会话内存池、虚拟机内存池等。

3.2 缓冲区

3.2.1 数据缓冲区

数据缓冲区是 DMServer 在将数据页写入磁盘之前以及从磁盘上读取数据页之后，数据页所存储的地方。这是 DMServer 至关重要的内存区域之一，将其设定得太小，会导致缓冲页命中率低，磁盘 IO 频繁；将其设定得太大，又会导致操作系统内存本身不够用。

系统启动时，首先根据配置的数据缓冲区大小向操作系统申请一片连续内存并将其按数据页大小进行格式化，并置入自由链中。数据缓冲区存在三条链来管理被缓冲的数据页，一条是自由链，用于存放目前尚未使用的内存数据页，一条是 LRU 链，用于存放已被使用的内存数据页(包括未修改和已修改)，还有一条即为脏链，用于存放已被修改过的内存数据页。

LRU 链对系统当前使用的页按其最近是否被使用的顺序进行了排序。这样当数据缓冲区中的自由链被用完时，从 LRU 链中淘汰部分最近未使用的数据页，能够较大程度地保证被淘汰的数据页在最近不会被用到，减少 IO。

在系统运行过程中，通常存在一部分“非常热”（反复被访问）的数据页，将它们一直留在

缓冲区中，对系统性能会有好处。对于这部分数据页，数据缓冲区开辟了一个特定的区域用于存放它们，以保证这些页不参与一般的淘汰机制，可以一直留在数据缓冲区中。

1） 类别

DMServer 中有三种类型的数据缓冲区，分别是 NORMAL、FAST 和 RECYCLE。NORMAL 缓冲区也称通用数据缓冲区，主要提供给系统处理的一些数据页；RECYCLE 缓冲区供临时表空间使用；FAST 缓冲区根据用户指定的 FAST_POOL_PAGES 大小由系统自动进行管理。

DMServer 提供了可以更改这些缓冲区大小的参数，用户可以根据自己应用需求情况，指定 dm.ini 文件中 BUFFER(8000MB)、RECYCLE(300MB)、FAST_POOL_PAGES(3000)值（括号中为默认值），这些值分别对应是 NORMAL 缓冲区大小、RECYCLE 缓冲区大小、FAST 缓冲区数据页总数。

2） 读多页

在需要进行大量 I/O 的应用当中，DM 之前版本的策略是每次只读取一页。如果知道用户需要读取表的大量数据，当读取到第一页时，可以猜测用户可能需要读取这页的下一页，在这种情况下，一次性读取多页就可以减少 I/O 次数，从而提高了数据的查询、修改效率。

DMServer 提供了可以读取多页的参数，用户可以指定这些参数来调整数据库运行效率的最佳状态。在 DM 配置文件 dm.ini 中，可以指定参数 MULTI_PAGE_GET_NUM 大小（默认值为 1 页），来控制每次读取的页数。

如果用户没有设置较适合的参数 MULTI_PAGE_GET_NUM 值大小，有时可能会给用户带来更差的效果。如果 MULTI_PAGE_GET_NUM 太大，每次读取的页可能大多都不是以后所用到的数据页，这样不仅会增加 I/O 的读取，而且每次都会做一些无用的 I/O，所以系统管理员需要衡量好自己应用需求，给出最佳方案。

3.2.2 日志缓冲区

日志缓冲区是用于存放重做日志的内存缓冲区。为了避免由于直接的磁盘 IO 而使系统

性能受到影响，系统在运行过程中产生的日志并不会立即被写入磁盘，而是和数据页一样，先将其放置到日志缓冲区中。那么为何不在数据缓冲区中缓存重做日志而要单独设立日志缓冲区呢？主要是基于以下原因：

1. 重做日志的格式同数据页完全不一样，无法进行统一管理；
2. 重做日志具备连续写的特点；
3. 在逻辑上，写重做日志比数据页 IO 优先级更高。

DMServer 提供了参数 RLOG_BUF_SIZE 对日志缓冲区大小进行控制，日志缓冲区所占用的内存是从共享内存池中申请的，单位为页数量。

3.2.3 字典缓冲区

字典缓冲区主要存储一些数据字典信息，如模式信息、表信息、列信息、触发器信息等。每次对数据库的操作都会涉及到数据字典信息，访问数据字典信息的效率直接影响到相应的操作效率，如进行查询语句，就需要相应的表信息、列信息等，这些字典信息如果都在缓冲区里，则直接从缓冲区中获取即可，否则需要 I/O 才能读取到这些信息。

DM8 采用的是将部分数据字典信息加载到缓冲区中，并采用 LRU 算法进行字典信息的控制。缓冲区大小设置问题，如果太大，会浪费宝贵的内存空间，如果太小，可能会频繁的进行淘汰，该缓冲区配置参数为 DICT_BUF_SIZE，默认的配置大小为 50MB。

DM8 采用缓冲部分字典对象，那会影响效率吗？数据字典信息访问存在热点现象，并不是所有的字典信息都会被频繁的访问，所以按需加载字典信息并不会影响到实际的运行效率。

但是如果在实际应用中涉及对分区数较多的水平分区表访问，例如上千个分区，那么就需要适当调大 DICT_BUF_SIZE 参数值。

3.2.4 SQL 缓冲区

SQL 缓冲区提供在执行 SQL 语句过程中所需要的内存，包括计划、SQL 语句和结果集缓

存。

很多应用当中都存在反复执行相同 SQL 语句的情况，此时可以使用缓冲区保存这些语句和它们的执行计划，这就是计划重用。这样带来的好处是加快了 SQL 语句执行效率，但同时给内存也增加了压力。

DMServer 在配置文件 dm.ini 提供了参数来支持是否需要计划重用，参数为 USE_PLN_POOL，当指定为非 0 时，则启动计划重用；为 0 时禁止计划重用。DM 同时还提供了参数 CACHE_POOL_SIZE（单位为 MB），来改变 SQL 缓冲区大小，系统管理员可以设置该值以满足应用需求，默认值为 100MB。

结果集缓存包括 SQL 查询结果集缓存和 DMSQL 程序函数结果集缓存，在 INI 参数文件中同时设置参数 RS_CAN_CACHE=1 且 USE_PLN_POOL 非 0 时 DM 服务器才会缓存结果集。DM 还提供了一些手动设置结果集缓存的方法，具体可参看 22.10 结果集重用。

客户端结果集也可以缓存，但需要在配置文件 dm_svc.conf 中设置参数：

```
ENABLE_RS_CACHE = (1)     //表示启用缓存;
RS_CACHE_SIZE = (100)     //表示缓存区的大小为 100MB
RS_REFRESH_FREQ = (30)    //表示每 30 秒检查缓存的有效性，如果失效，自动重查； 0 表示不检查。
```

同时在服务器端使用 INI 参数文件中的 CLT_CACHE_TABLES 参数设置哪些表的结果集需要缓存。另外，FIRST_ROWS 参数表示当查询的结果达到该行数时，就返回结果，不再继续查询，除非用户向服务器发一个 FETCH 命令。这个参数也用于客户端缓存的配置，仅当结果集的行数不超过 FIRST_ROWS 时，该结果集才可能被客户端缓存。

3.3 排序区

排序缓冲区提供数据排序所需要的内存空间。当用户执行 SQL 语句时，常常需要进行排序，所使用的内存就是排序缓冲区提供的。在每次排序过程中，都首先申请内存，排序结束后再释放内存。

DMServer 提供了参数来指定排序缓冲区的大小，参数 SORT_BUF_SIZE 在 DM 配置文件 dm.ini 中，系统管理员可以设置其大小以满足需求，由于该值是由系统内部排序算法和排序数据结构决定，建议使用默认值 20MB。

3.4 哈希区

DM8 提供了为哈希连接而设定的缓冲区，不过该缓冲区是个虚拟缓冲区。之所以说是虚拟缓冲，是因为系统没有真正创建特定属于哈希缓冲区的内存，而是在进行哈希连接时，对哈希连接的数据量进行了计算。如果计算出的数据量大小超过了哈希缓冲区的大小，则使用 DM8 创新的外存哈希方式；如果没有超过哈希缓冲区的大小，实际上还是使用内存池来进行哈希操作。

DMServer 在 dm.ini 中提供了参数 HJ_BUF_SIZE 来进行控制，由于该值的大小可能会限制哈希连接的效率，所以建议保持默认值，或设置为更大的值。

除了提供 HJ_BUF_SIZE 参数外，DMServer 还提供了创建哈希表个数的初始化参数，其中，HAGR_HASH_SIZE 表示处理集函数时创建哈希表的个数，建议保持默认值 100000。

第4章 管理 DM 线程

DM 服务器使用“对称服务器构架”的单进程、多线程结构。这种对称服务器构架在有效地利用了系统资源的同时又提供了较高的可伸缩性能，这里所指的线程即为操作系统的线程。服务器在运行时由各种内存数据结构和一系列的线程组成，线程分为多种类型，不同类型的线程完成不同的任务。线程通过一定的同步机制对数据结构进行并发访问和处理，以完成客户提交的各种任务。DM 数据库服务器是共享的服务器，允许多个用户连接到同一个服务器上，服务器进程称为共享服务器进程。

DM 进程中主要包括监听线程、IO 线程、工作线程、调度线程、日志线程等，以下分别对它们进行介绍。

4.1 监听线程

监听线程主要的任务是在服务器端口上进行循环监听，一旦有来自客户的连接请求，监听线程被唤醒并生成一个会话申请任务，加入工作线程的任务队列，等待工作线程进行处理。它在系统启动完成后才启动，并且在系统关闭时首先被关闭。为了保证在处理大量客户连接时系统具有较短的响应时间，监听线程比普通线程优先级更高。

DM 服务器所有配置端口的范围为 1024-65534。当客户端工具发起连接时，由操作系统为客户端工具自动分配一个端口用于与 DM 服务器进行通信。对于数据守护、DMDSC、DM MPP 和 DMDPC 等分布式数据库中各实例节点之间的通信，发起连接的节点也由操作系统自动分配端口，所以配置主备/DMDSC/DM MPP/DMDPC 时，除了各实例指定的端口外，发起连接的实例也会有操作系统自动分配的用于和其他实例进行通信的端口。

4.2 工作线程

工作线程是 DM 服务器的核心线程，负责服务器各种类型任务的处理及所有实际的数据

相关操作。工作线程包括任务工作线程和会话工作线程等。

任务工作线程负责处理服务器内部生成并加入对应任务队列的各项任务，如调度线程生成的清理缓存任务、异步执行的任务等，其初始个数由 INI 参数 TASK_THREADS 指定。任务工作线程空闲时从任务队列依次摘取任务并进行处理，当任务队列中的任务堆积时，系统会增加任务工作线程数量以减少任务堆积。

会话工作线程负责处理会话的请求任务，分为非线程池模式和线程池模式，由 INI 参数 STHD_FLAG 控制。非线程池模式下每一个会话对应一个会话工作线程，一个会话上的任务全部由同一个工作线程完成，这样减少了线程切换的代价，提高了系统效率。线程池模式初始工作线程个数由 INI 参数 WORKER_THREADS*STHD_THREAD_NUM 指定，由会话轮询线程接收所有用户请求，加入会话任务队列，会话工作线程从会话任务队列依次摘取请求任务进行处理。

4.3 IO 线程

在数据库活动中，IO 操作历来都是最为耗时的操作之一。当事务需要的数据页不在缓冲区中时，如果在工作线程中直接对那些数据页进行读写，将会使系统性能变得非常糟糕，而把 IO 操作从工作线程中分离出来则是明智的做法。IO 线程的职责就是处理这些 IO 操作。

通常情况下，DMServer 需要进行 IO 操作的时机主要有以下三种：

1. 需要处理的数据页不在缓冲区中，此时需要将相关数据页读入缓冲区；
2. 缓冲区满或系统关闭时，此时需要将部分脏数据页写入磁盘；
3. 检查点到来时，需要将所有脏数据页写入磁盘。

IO 线程在启动后，通常都处于睡眠状态，当系统需要进行 IO 时，只需要发出一个 IO 请求，此时 IO 线程被唤醒以处理该请求，在完成该 IO 操作后继续进入睡眠状态。

IO 线程的个数是可配置的，可以通过设置 dm.ini 文件中的 IO_THR_GROUPS 参数来设置，默认情况下，IO 线程的个数是 8 个。同时，IO 线程处理 IO 的策略根据操作系统平台

的不同会有很大差别，一般情况下，IO 线程使用异步的 IO 将数据页写入磁盘，此时，系统将所有的 IO 请求直接递交给操作系统，操作系统在完成这些请求后才通知 IO 线程，这种异步 IO 的方式使得 IO 线程需要直接处理的任务很简单，即完成 IO 后的一些收尾处理并发出 IO 完成通知，如果操作系统不支持异步 IO，此时 IO 线程就需要完成实际的 IO 操作。

4.4 调度线程

调度线程用于接管系统中所有需要定时调度的任务。调度线程每秒钟轮询一次，负责的任务有以下一些：

1. 检查系统级的时间触发器，如果满足触发条件则生成任务加到任务工作线程的任务队列由任务工作线程执行；
2. 清理 SQL 缓存、计划缓存中失效的项，或者超出缓存限制后淘汰不常用的缓存项；
3. 执行动态缓冲区检查。根据需要动态扩展或动态收缩系统缓冲池；
4. 自动执行检查点。为了保证日志的及时刷盘，减少系统故障时恢复时间，根据 INI 参数设置的自动检查点执行间隔定期执行检查点操作；
5. 会话超时检测。当客户连接设置了连接超时时，定期检测是否超时，如果超时则自动断开连接；
6. 必要时执行数据更新页刷盘；
7. 唤醒等待的工作线程。

4.5 日志 FLUSH 线程

任何数据库的修改，都会产生重做 REDO 日志，为了保证数据故障恢复的一致性，REDO 日志的刷盘必须在数据页刷盘之前进行。事务运行时，会把生成的 REDO 日志保留在日志缓冲区中，当事务提交或者执行检查点时，会通知 FLUSH 线程进行日志刷盘。由于日志具备顺序写入的特点，比数据页分散 IO 写入效率更高。日志 FLUSH 线程和 IO 线程分开，能获得更

快的响应速度，保证整体的性能。DM8 的日志 FLUSH 线程进行了优化，在刷盘之前，对不同缓冲区内的日志进行合并，减少了 IO 次数，进一步提高了性能。

如果系统配置了实时归档，在 FLUSH 线程日志刷盘前，会直接将日志通过网络发送到实时备库。如果配置了本地归档，则生成归档任务，通过日志归档线程完成。

4.6 日志归档线程

日志归档线程包含异步归档线程，负责远程异步归档任务。如果配置了非实时归档，由日志 FLUSH 线程产生的任务会分别加入日志归档线程，日志归档线程负责从任务队列中取出任务，按照归档类型做相应归档处理。

将日志 FLUSH 线程和日志归档线程分开的目的是为了减少不必要的效率损失，除了远程实时归档外，本地归档、远程异步归档都可以脱离 FLUSH 线程来做，如果放在 FLUSH 线程中一起做会严重影响系统性能。

4.7 日志 APPLY 线程

在配置了数据守护的系统中，创建了一个日志 APPLY 线程。当服务器作为备库时，每次接收到主库的物理 REDO 日志生成一个 APPLY 任务加入到任务队列，APPLY 线程从任务队列中取出一个任务在备库上将日志重做，并生成自己的日志，保持和主库数据的同步或一致，作为主库的一个镜像。备库数据对用户只读，可承担报表、查询等任务，均衡主库的负载。

4.8 定时器线程

在数据库的各种活动中，用户常常需要数据库完成在某个时间点开始进行某种操作，如备份；或者是在某个时间段内反复进行某种操作等。定时器线程就是为这种需求而设计的。

通常情况下，DMServer 需要进行定时操作的事件主要有以下几种：

1. 逻辑日志异步归档；

2. 异步归档日志发送（只有在 PRIMARY 模式下，且是 OPEN 状态下）；

3. 作业调度。

定时器线程启动之后，每秒检测一次定时器链表，查看当前的定时器是否满足触发条件，如果满足，则把执行权交给设置好的任务，如逻辑日志异步归档等。

默认情况下，达梦服务器启动的时候，定时器线程是不启动的。用户可以设置 dm.ini 中的 TIMER _INI 参数为 1 来设置定时器线程在系统启动时启动。

4.9 MAL 系统相关线程

MAL 系统是 DM 内部高速通信系统，基于 TCP/IP 协议实现。服务器的很多重要功能都是通过 MAL 系统实现通信的，例如数据守护、MPP、远程日志归档等。MAL 系统内部包含一系列线程，有 MAL 监听线程、MAL 发送工作线程、MAL 接收工作线程等。

4.10 其他线程

事实上，DM 数据库系统中还不止以上这些线程，在一些特定的功能中会有不同的线程，例如回滚段清理 PURGE 线程、审计写文件线程等，这里不一一列出。

4.11 线程信息的查看

为了增加用户对 DM 数据库内部信息的了解，以及方便数据库管理员对数据库的维护，DM 提供了很多动态性能视图，通过它们用户可以直观地了解当前系统中有哪些线程在工作，以及线程的相关信息。相关动态视图见表 4.1。

表 4.1 DM 线程相关的动态视图

名称	说明
V$THREADS	记录当前系统中活动线程的信息
V$PROCESS	记录服务器进程信息

以上视图的详细定义参考本手册中附录 2 动态性能视图。

第二部分 基础数据库管理

第5章 DM 系统管理员

为了保证数据库系统的安全性，DM 数据库采用“三权分立”或“四权分立”的安全机制，“三权分立”时系统内置三种系统管理员，包括数据库管理员、数据库安全员和数据库审计员，“四权分立”时新增了一类用户，称为数据库对象操作员。它们各司其职，互相制约，有效地避免了将所有权限集中于一人的风险，保证了系统的安全性。

5.1 DM 系统管理员的类型

在现实生活中，任何一个系统如果将所有的权利都赋予给某一个人，而不加以监督和控制，势必会产生权利滥用的风险。从数据库安全角度出发，一个大型的数据库系统有必要将数据库系统的权限分配给不同的角色来管理，并且各自偏重于不同的工作职责，使之能够互相限制和监督，从而有效保证系统的整体安全。

DM 数据库实现了 B1 级安全特性。“三权分立”的安全机制，将系统管理员分为数据库管理员、数据库安全员和数据库审计员三类。在安装过程中，DM 数据库会预设数据库管理员账号 SYSDBA、数据库安全员账号 SYSSSO 和数据库审计员账号 SYSAUDITOR，其口令均由用户自己设置。“四权分立”的安全机制，将系统管理员分数据库管理员、数据库对象操作员、数据库安全员和数据库审计员四类，在“三权分立”的基础上，新增数据库对象操作员账户 SYSDBO，其口令由用户自己设置。

1． 数据库管理员（DBA）

“三权分立”的安全机制，每个 DM 数据库至少需要一个数据库管理员来管理，负责评估数据库运行所需的软、硬件环境、安装和升级 DM 数据库、配置 DM 数据库参数、创建主要的

数据库存储结构（表空间）和对象（如表、视图、索引、角色、用户等）、监控和优化数据库性能、数据导入导出以及数据库的备份和恢复等。

“三权分立”时数据库管理员既可进行系统管理和维护工作，也可对数据内容进行增删查改动作。根据国产数据库军事使用要求，数据库管理员只能进行系统管理和维护工作，不能对数据内容进行增删查改，数据库应用人员则可操作数据内容，而不能管理和维护系统。

“四权分立”的安全机制，在原有“三权分立”基础上调整自主访问控制权限，只具有“三权分立”中 DBA 角色预设的一部分与数据库管理相关的明确的数据库权限，如数据库创建、备份、还原和校验等，具体见《DM8 安全管理》。

2．数据库安全员（SSO）

对于很多对安全性要求不高的系统来说，C2 级安全特性已经能够工作得很好，此时不需要考虑通过数据库安全员来进一步加强系统的安全机制。但是在很多大型的系统中，安全性还是至关重要的，有必要由安全员来制定安全策略，强化系统安全机制，此时数据库安全员的主要任务就是制定安全策略，定义新的数据库安全员，设置系统的安全等级、范围和组，并为主、客体定义安全标记，从而全面提升系统安全性。

3．数据库审计员（AUDITOR）

数据库审计员可以设置要审计的对象和操作、定义新的数据库审计员、查看和分析审计记录。通过设置审计，几乎可以跟踪任何人在系统内执行的任何操作，为事后追查提供便利。

4．数据库对象操作员（DBO）

数据库对象操作员是“四权分立”新增加的一类用户，可以创建数据库对象，并对自己拥有的数据库对象（表、视图、存储过程、序列、包、外部链接）具有所有的对象权限并可以授出与回收，但其无法管理与维护数据库对象。

需要说明的是，在 DM 数据库中有一个特殊的预定义用户“SYS”，此用户仅用于保存系统内部对象，并非数据库管理员，也无法进行登录。

5.2 数据库管理员的任务

每个数据库至少需要一个 DBA 来管理，DBA 可能是一个团队，也可能是一个人。在不同的公司，数据库管理员的职责可能也会有比较大的区别，总体而言，数据库管理员的职责主要包括以下任务。

1． 评估数据库服务器所需的软、硬件运行环境

通常情况下，DBA 不会直接配置除数据库之外的软、硬件环境，不过在某些情况下还是需要。即便不用负责配置其他软、硬件，还是应该负责为软、硬件的选型提供指南和规范，以便最终配置能够提供满足业务需要的最优级别的性能、可靠性和可扩展性，同时保证总体费用在预算范围内。

作为 DBA，需要结合实际应用负载、总体费用、性能预期目标等来评估可供选择的软、硬件运行环境。其中实际应用负载和总体费用基本上都是确定的，DBA 的目标就是保证在负载和费用基本不变的情况下，实现系统总体性能的最优化。DBA 主要考虑的因素包括：

1） 操作系统/中间件等通用软件的稳定性、性能、安全性；

2） 处理器的个数和性能；

3） 内存容量和性能；

4） 网络带宽；

5） 存储容量和读写性能；

6） HBA 卡传输性能。

2． 安装和升级 DM 服务器

作为数据库管理员，应负责安装 DM 数据库服务器软件。具体的安装配置步骤请参考《DM8 安装手册》。需要注意的是，和所有软件一样，应先在测试系统上完成安装，这样可以确保不会影响到生产系统。

随着 DM 数据库的发展，会在 DM 数据库发布后提供服务包和安全更新。正确地安装服

务包和安全更新是至关重要的任务，它影响到系统的稳定性、性能和安全。DBA 需要清楚地了解服务包和安全更新对现有系统的影响，然后考虑是否进行升级。是在生产系统上应用新的升级包之前，必须在测试环境下进行确认。

3．数据库结构设计

在安装 DM 数据库之后，DBA 就可以开始进行数据库结构设计。数据库的结构设计直接影响系统的综合性能，是 DBA 最为重要的工作任务之一。

DBA 可以依次参考第 6 章 创建和配置 DM 数据库来创建和配置 DM 数据库，参考第 7 章 启动和关闭数据库来启动和关闭数据库，参考第 8 章 管理模式对象的空间和第 13 章 模式对象的常规管理来进行模式对象的空间布局和实操管理，参考第 14 章 数据库布局和存储管理来实施数据库整体布局和存储管理，参考第 9 章 管理表-第 12 章 管理视图、序列和同义词、第 15 章 管理分区表和分区索引-第 18 章 全文检索以进行各数据库对象及其存储特性的设计。通过在创建对象之前规划每个对象和它的物理存储之间的关系，可以直接影响数据库的性能。

4．监控和优化数据库的性能

监控和优化数据库的性能是数据库管理员的核心职责之一。对系统进行监控有助于在问题发生时识别它们，性能优化则有助于消除那些问题并防止它们发生。监视长时间运行的查询有助于发现可以改进的领域，如增加索引、重写查询、改进应用程序逻辑等，还可以发现由于缺少内存/内存分配不合理而导致的性能降低，以便调整 DM 数据库的内存使用参数，或为服务器增加更多的内存。DBA 需要提供对于各种问题的解决方案，努力优化 DM 数据库的各个方面，必要时，可以直接拨打 DM 数据库产品售后服务热线来寻求帮助。

5．计划和实施备份与故障恢复

即便是最好、最稳定的系统也会存在意外，如断电、硬件故障等。DBA 必须对各种可能的故障和计划中的停机提供解决方案。一般来说，系统的可靠性与成本之间存在着类似线性

的关系，越是可靠的解决方案，其整体成本必然也会更高，DBA 必须在企业限定的成本内制定可行的可靠性解决方案。

DM 数据库提供了最基本的故障恢复、备份与还原功能，另外还提供了数据守护、数据共享集群等高可靠、高可用解决方案。DBA 可参考本书第五部分、《DM8 备份与还原》、《DM8 数据守护与读写分离集群 V4.0》和《DM8 数据共享集群》来选择合适的解决方案。

5.3 数据库安全员的任务

有些应用对于安全性有着很高的要求，传统的由 DBA 一人拥有所有权限并且承担所有职责的安全机制可能无法满足企业实际需要，此时数据库安全员和数据库审计员两类角色和用户就显得异常重要，它们对于限制和监控数据库管理员的所有行为都起着至关重要的作用。

数据库安全员的主要职责就是制定并应用安全策略，强化系统安全机制。数据库安全员 SYSSSO 是 DM 数据库初始化的时候就已经创建好的，可以以该用户登录到 DM 数据库来创建新的数据库安全员。SYSSSO 或者新的数据库安全员都可以制定自己的安全策略，在安全策略中定义安全级别、范围和组，然后基于定义的安全级别、范围和组来创建安全标记，并将安全标记分别应用到主体（用户）和客体（表），以便启用强制访问控制功能。数据库安全员不能对用户数据进行增、删、改、查，也不能执行普通的 DDL 操作，他们只负责制定安全机制，将合适的安全标记应用到主体和客体，通过这种方式可以有效的对 DBA 的权限进行限制，DBA 此后就不能直接访问添加有安全标记的数据，除非安全员给 DBA 也设定了与之匹配的安全标记，DBA 的权限受到了有效的约束，详细的安全特性介绍请参考《DM8 安全管理》。

5.4 数据库审计员的任务

我们可以想象一下，某个企业内部 DBA 非常熟悉公司内部 ERP 系统的数据库设计，该系统包括了员工工资表，里面记录了所有员工的工资，公司的出纳通过查询系统内部员工工

资表来发放工资。传统的 DBA 集所有权利于一身，可以很容易修改工资表，从而导致公司工资账务错乱。为了预防该问题，可以采用前面数据库安全员制定安全策略的方法，避免 DBA 或者其他数据库用户具有访问该表的权限。为了能够及时找到 DBA 或者其他用户的非法操作，在 DM 数据库中还可以在系统建设初期，由数据库审计员（SYSAUDITOR 或者其他由 SYSAUDITOR 创建的审计员）来设置审计策略（包括审计对象和操作），在需要时，数据库审计员可以查看审计记录，及时分析并查找出幕后真凶。

从上面的介绍我们也可以看到，在 DM 数据库中，审计员的主要职责就是创建和删除数据库审计员，设置/取消对数据库对象和操作的审计设置，查看和分析审计记录等。关于审计的详细介绍，请参考《DM8 安全管理》。

5.5 数据库对象操作员的任务

在一些对数据安全要求较高的数据库应用系统中，数据库管理须遵循最小权限原则和职责分离原则，因此，DM 引入了四权分立，新增数据库对象操作员。在四权分立模式下，对数据库管理员的权限进行了重新分配，部分数据库管理员的权限被划分给了新引入的数据库对象操作员，具体划分权限为:

1． 创建数据库对象并进行操作

可以创建各类数据库对象如表、视图、存储过程、序列、包、外部链接等，也可对对象进行各类增删改查操作。对各数据库对象的操作语法可以参考《DM8_SQL 语言使用手册》。

2． 对自己拥有对象的所有权限的授权回收

数据库对象操作员可以对自己所拥有对象的所有权限进行授权与回收。授权回收操作可以查看《DM8 安全管理》。

由上述介绍可以看出，在数据安全要求较高的数据库应用中，可引入数据库对象操作员对应负责数据库中数据录入、更新等日常数据操作的角色，以实现操作与管理职责的分离。

第6章 创建和配置 DM 数据库

DM 数据库可以在安装 DM 软件时创建，也可以在安装 DM 软件之后，通过数据库配置工具或 dminit 来手工创建数据库，创建数据库时要使用初始化参数。

建库完成后，在 DM 服务器运行期间，可以通过查询动态视图或使用系统函数的方式获取建库参数的具体信息，具体的获取方式可以查看《DM8_dminit 使用手册》。

6.1 创建 DM 数据库

用户创建数据库之前，需要规划数据库，如数据库名、实例名、端口、文件路径、簇大小、页大小、日志文件大小、SYSDBA 和 SYSAUDITOR 等系统用户的密码等，然后可以使用图形化界面或者 dminit 创建数据库。用户可以在安装 DM 数据库软件时创建数据库，也可以在安装之后创建数据库。

在创建数据库之前需要做如下准备工作:

1. 规划数据库表和索引，并估算它们所需的空间大小;
2. 确定字符集。所有字符集数据，包括数据字典中的数据，都被存储在数据库字符集中，用户在创建数据库时可以指定数据库字符集，如不指定则使用默认字符集 GB18030;
3. 规划数据库文件的存储路径，可以指定数据库存储路径、控制文件存放路径、日志文件存放路径等，应注意在指定的路径或文件名中尽量不要包含中文字符，否则可能由于数据库与操作系统编码方式不一致导致不可预期的问题;
4. 配置数据库时区，如中国是+8:00 时区;
5. 设置数据库簇大小、页大小、日志文件大小，在数据库创建时由 EXTENT_SIZE、PAGE_SIZE、LOG_SIZE 初始化参数来指定，并且在数据库创建完成之后不能修改此参数。

6. 为系统用户指定新的口令。为安全起见，应当为数据库中的 SYSDBA 和 SYSAUDITOR 系统用户指定新的口令密码，在数据库创建时由 SYSDBA_PWD 和 SYSAUDITOR_PWD 初始化参数来指定，创建后可修改。

如果没有对数据库进行合理规划，可能造成各种不可预知的后果，影响数据库的正常使用。例如，当磁盘空间有限时，如果不适当通过设置 LOG_SIZE 取值缩小日志文件大小，可能因系统创建的两个默认日志文件过大，而无法成功创建数据库。

创建数据库之前，必须满足以下必要条件：

1. 安装必需的 DM 软件，包括为操作系统设置各种环境变量，并为软件和数据库文件建立目录结构；
2. 必须有足够的内存来启动 DM 数据库实例；
3. 在执行 DM 的计算机上要有足够的磁盘存储空间来容纳规划的数据库。

6.2 使用数据库配置工具创建数据库

数据库配置工具提供了一个图形化界面来引导用户创建和配置数据库。使用数据库配置工具来创建数据库是用户的首选，因为数据库配置工具更趋于自动化，当使用数据库配置工具安装完成时，数据库也做好了使用准备工作。数据库配置工具可以通过 DM 数据库安装之后作为一个独立的工具来启动。

6.2.1 启动数据库配置助手

数据库配置工具可以通过 DM 支持的模板或用户自定义的模板来创建数据库。

通过下列步骤来启动数据库配置助手（以 Windows 操作系统为例）：

1. 使用通过验证的具有管理权限组的成员登录到计算机上，安装 DM 数据库软件，并创建和运行数据库；
2. 在 Windows 操作系统中启动数据库配置助手。选择“开始”→“程序”→“达梦数据库”→

“客户端”→“DM 数据库配置助手”，双击启动数据库配置助手。

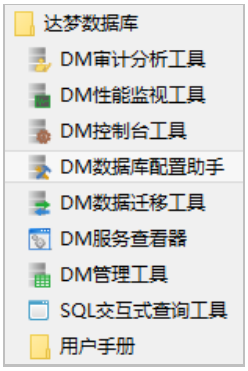


图 6.1 启动数据库配置工具

6.2.2 使用数据库配置工具创建数据库

在数据库配置工具操作窗口中，选择“创建数据库”选项启动能够创建和配置一个数据库的向导，如图 6.2 所示。这个向导引导用户完成以下操作：

1. 数据库模板；
2. 数据库目录；
3. 数据库标识；
4. 数据库文件；
5. 初始化参数；
6. 口令管理；
7. 创建示例库；
8. 创建摘要；
9. 创建数据库。

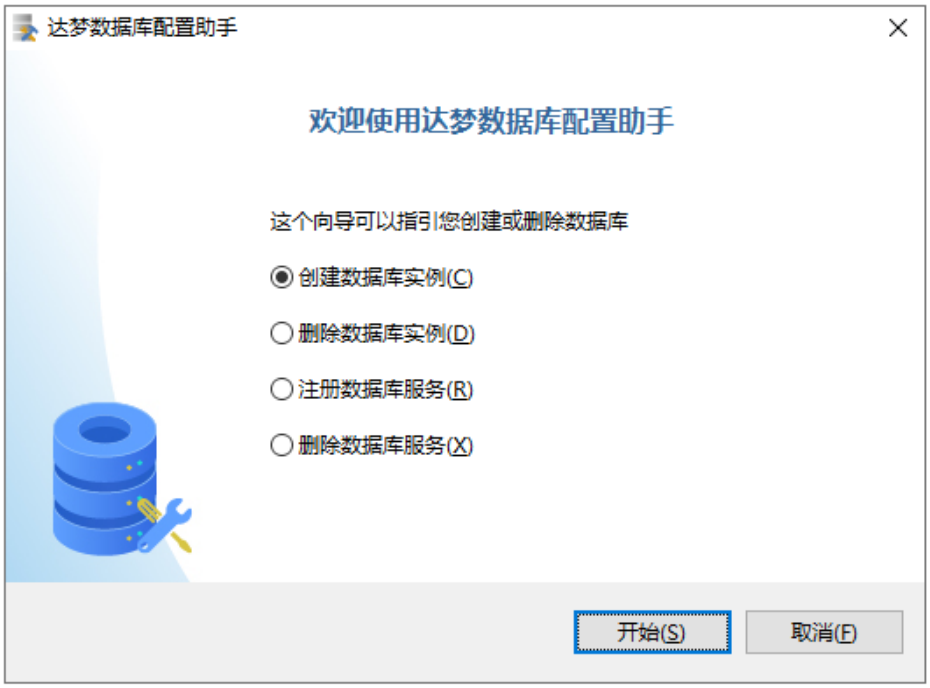


图 6.2 数据库配置工具操作界面

6.2.2.1 数据库模板

在这个窗口中可以选择需要创建数据库的类型，DM 预定义了一些模板，如一般用途、联机分析处理模板或联机事务处理模板。如图 6.3 所示。

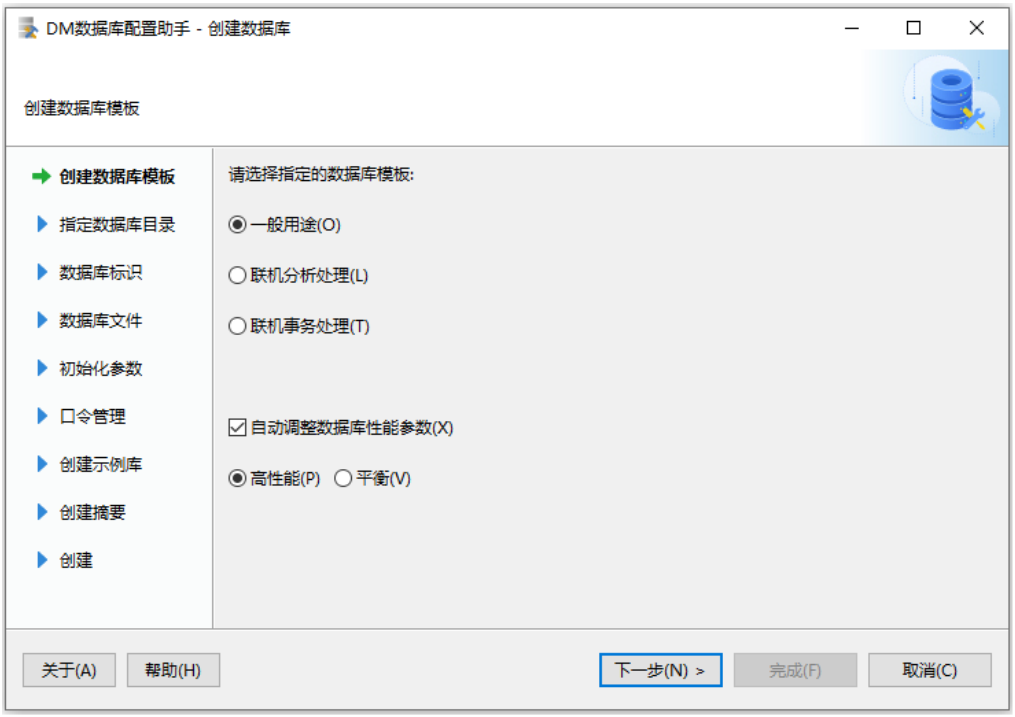


图 6.3 数据库模板选择

6.2.2.2 数据库目录

指定数据库目录。如图 6.4 所示。

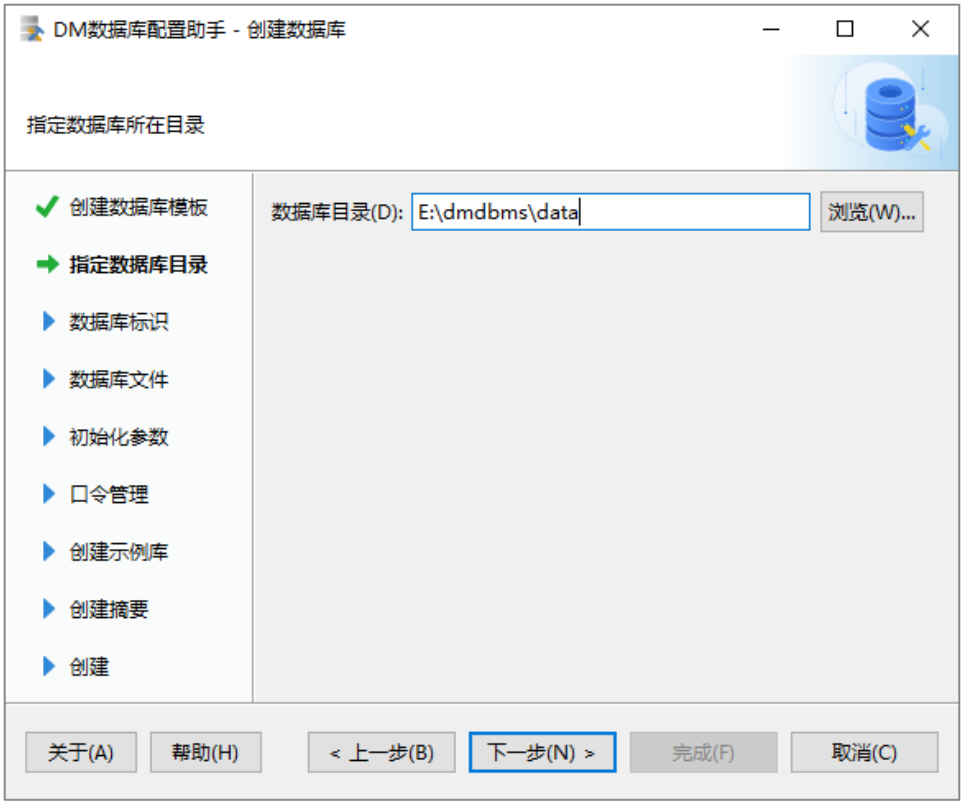


图 6.4 指定数据库目录

6.2.2.3 数据库标识

在“数据库名（D）”文本框中，输入数据库名；在“实例名（I）”文本框中输入数据库实例名；在“端口号（P）”文本框中，输入端口号。如图 6.5 所示。

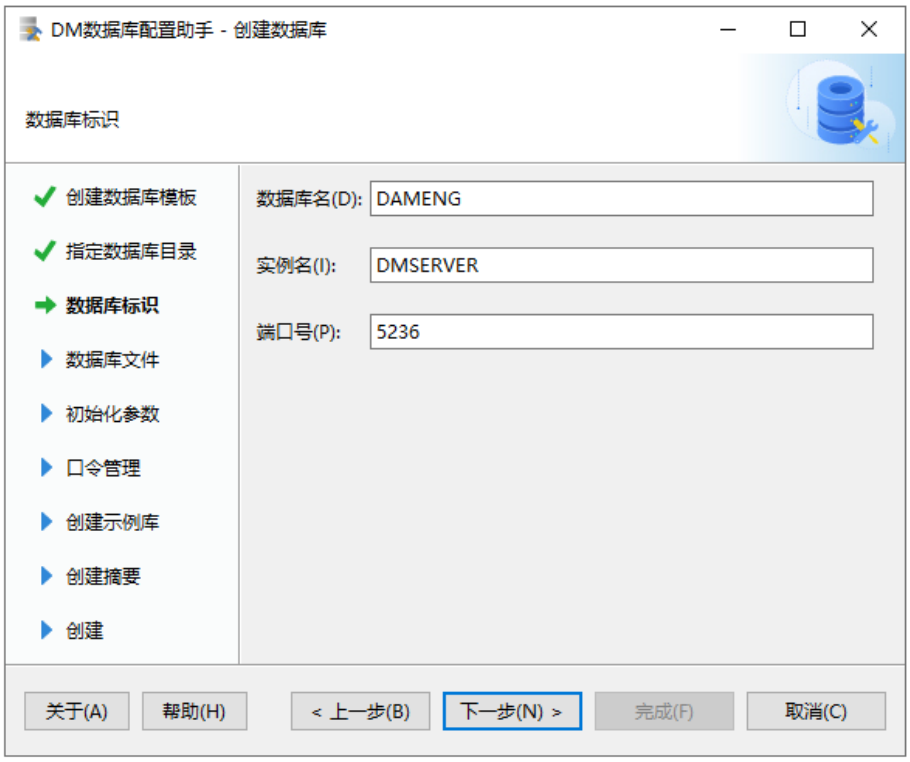


图 6.5 设置数据库标识

6.2.2.4 数据库文件

如图 6.6 所示，此界面包含四个选项卡：“控制文件”、“数据文件”、“日志文件”和“初始化日志”，可以通过双击路径来更改文件路径。

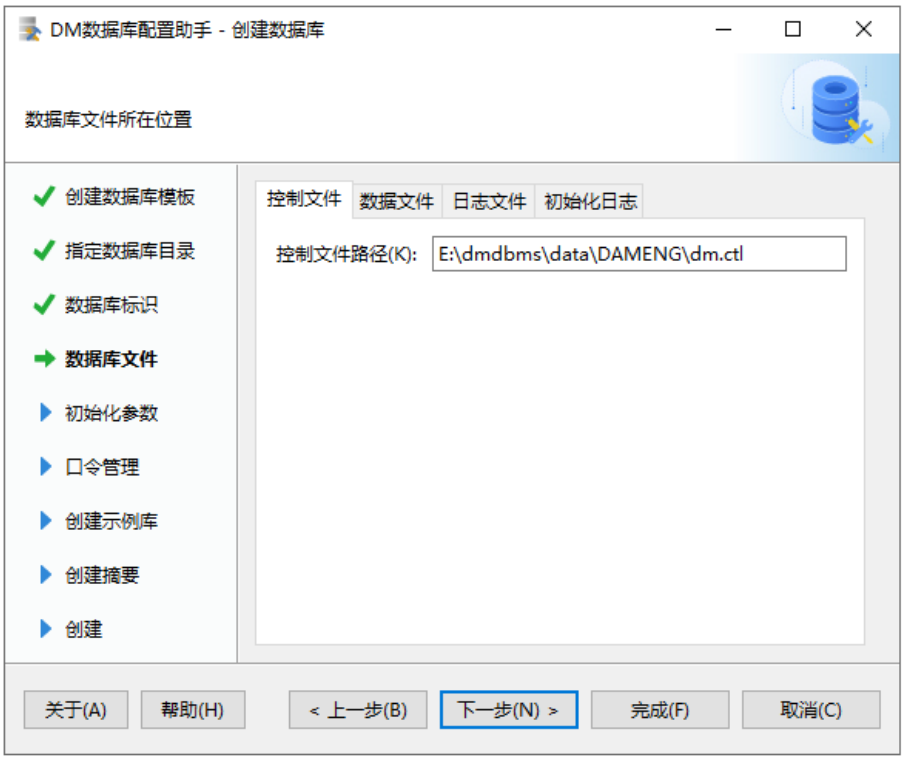


图 6.6 设置数据库文件路径

控制文件

与配置文件类似，控制文件对系统的运行及性能有很大的影响，但不同的是，配置文件中的配置项可以随意更改，而控制文件中的控制信息一般在系统第一次创建完毕后就不能随意更改，所以控制文件不是文本文件，而是二进制文件。

数据文件

“数据文件”选项卡用来指定 SYSTEM 表空间路径、MAIN 表空间路径、ROLL 表空间路径和 TEMP 表空间路径。同时还可以指定 SYSTEM 表空间镜像、MAIN 表空间镜像、ROLL 表空间镜像路径。三个镜像文件是分别和 SYSTEM 表空间、MAIN 表空间、ROLL 表空间一模一样的文件，当 SYSTEM 表空间、MAIN 表空间、ROLL 表空间文件损坏时，就可以使用相应的镜像文件来替换。

数据文件是数据库中最重要的文件类型之一，这是数据最终要存储的地方，每个数据库至少有一个与之相关的数据文件，通常情况下，将会有多个。为了理解 DMServer 如何组织这些文件，以及数据在它们内部是如何组织的，必须理解数据页和簇的概念，它们都是

DMServer 用于保存数据库对象的分配单元。

数据页是系统进行磁盘 IO 和缓冲区调度的单元,其大小在数据库创建时就固定下来了,而且一旦固定就不可更改，它们的容量也都是相同的。所有数据页的格式大致相同。

簇是数据文件中一个连续的分配空间，簇由多个数量固定的数据页组成。数据文件对空间的标识都以簇为单位，每个数据文件都维护着两条链，一条为半空簇的链，另一条为自由簇的链，其中半空链用于标识文件中所有被用过一部分的簇，自由链则标识文件中所有未被用过的簇。通常情况下，系统在分配空间时，以簇为单位分配会更有效。

日志文件

重做日志文件对于 DMServer 是至关重要的。它们用于存储数据库的事务日志，以便系统在出现系统故障和介质故障时能够进行故障恢复。在 DMServer 中，任何修改数据库的操作都会产生重做日志，这样，当系统出现故障时，通过分析日志可以知道在故障发生前系统做了哪些动作，并可以重做这些动作使系统恢复到故障之前的状态。

初始化日志

初始化日志用来指定初始化过程中生成的日志文件所在路径。

6.2.2.5 初始化参数

数据文件使用的簇大小，即每次分配新的段空间时连续的页数，只能是 16 页或 32 页或 64 页，缺省使用 16 页。

数据文件使用的页大小，可以为 4KB、8KB、16KB 或 32KB，选择的页大小越大，则 DM 支持的元组长度也越大，但同时空间利用率可能下降，缺省使用 8KB。

日志文件使用的大小，32 位版本时，取值为 256 和 2048 之间的整数，单位为 MB，缺省值为 2048MB；64 位版本时，取值为 256 和 8192 之间的整数，单位为 MB，缺省值为 4096MB。

时区设置，缺省使用+08:00，范围为-12:59 和+14:00 之间。

页校验，缺省为默认，选项包括不启用、CRC 算法、HASH 算法、快速 CRC 算法、默认，其中选择默认时采用快速 CRC 算法。

页校验 HASH 算法，当页校验中选择 HASH 算法时，可以对使用的 HASH 算法进行选择，系统内置的 HASH 算法包括 MD5、SHA1、SHA224、SHA256、SHA384、SHA512，缺省使用 MD5。此外，用户也可选择 DM 提供的第三方散列算法，具体可参考《DM8 安全管理》。

字符集，缺省使用 GB18030，选项包括 GB18030（支持 GB18030-2022 版本）、UTF_8（支持 UNICODE 12.0 版本）、EUC-KR。

页加密分片大小，缺省使用 4096B，选项包括 0B、512B、4096B。

勾选启用全库加密时，可以选择使用的全库加密算法。

勾选库默认加密设置时，可以选择配置使用的加密算法、HASH 算法和加密引擎

以上可在数据库配置助手的初始化栏进行设置，如图 6.7 所示。



图 6.7 数据库初始化参数

6.2.2.6 口令管理

为了数据库管理安全，提供了为数据库的 SYSDBA 和 SYSAUDITOR 系统用户指定口令的功能，如果安装版本为安全版，将会增加为 SYSSSO 和 SYSDBO 系统用户指定口令的功能。用户可以选择为每个系统用户设置不同口令，也可以为所有系统用户设置同一口令。如图 6.8 所示。

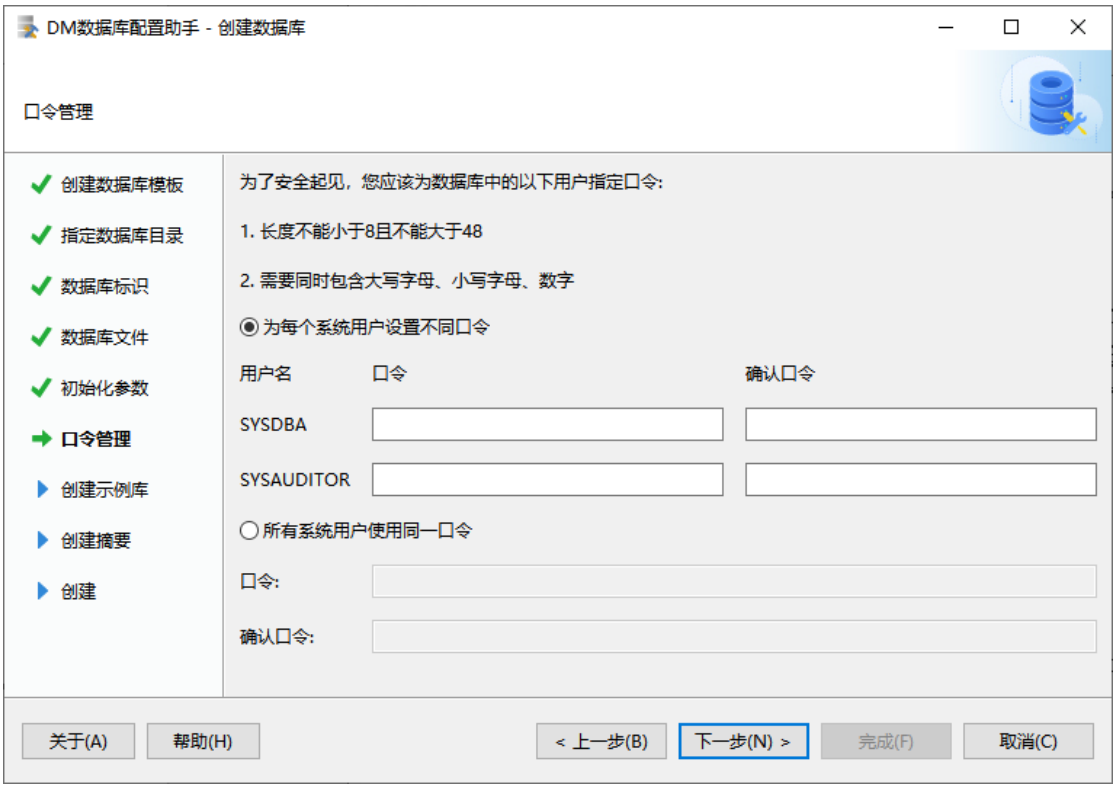


图 6.8 口令管理

6.2.2.7 创建示例库

示例库 BOOKSHOP 模拟武汉代理图书的某销售公司，该公司欲建立在线购物平台来拓展其代理产品的销售渠道。该示例在 DM 各演示程序中使用。安装该示例后，将在数据库中创建 BOOKSHOP 表空间，同时创建 RESOURCES、PERSON、PRODUCTION、PURCHASING、SALES、OTHER 这 6 个模式和相关的表。

示例库 DMHR 模拟武汉达梦数据库股份有限公司人力资源管理系统。安装完该示例库，将创建一个模式 DMHR 和一个表空间 DMHR，在 DMHR 模式下创建 REGION、CITY、LOCATION、DEPARTMENT、JOB、EMPLOYEE、JOB_HISTORY 等 7 张表，并插入数据。

如图 6.9 所示。



图 6.9 创建示例库

6.2.2.8 创建摘要

列举创建数据库纲要，会列举创建时指定的数据库名、示例名、数据库目录、端口、控制文件路径、数据文件路径、日志文件路径、簇大小、页大小、日志文件大小、大小写是否敏感、是否使用 unicode 等信息，方便用户确认创建信息是否符合自己的需求，及时返回修改。如图 6.10 所示。

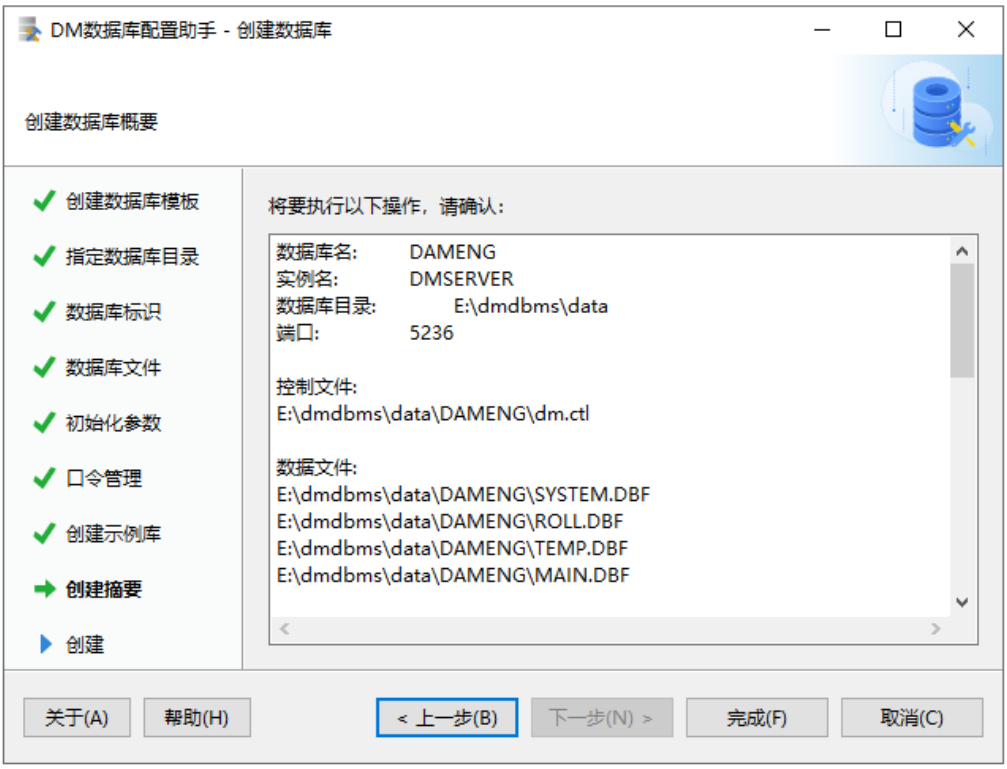


图 6.10 创建摘要

6.2.2.9 创建数据库

核对完创建信息后，开始创建数据库、创建并启动实例、创建示例库。如图 6.11 所示。

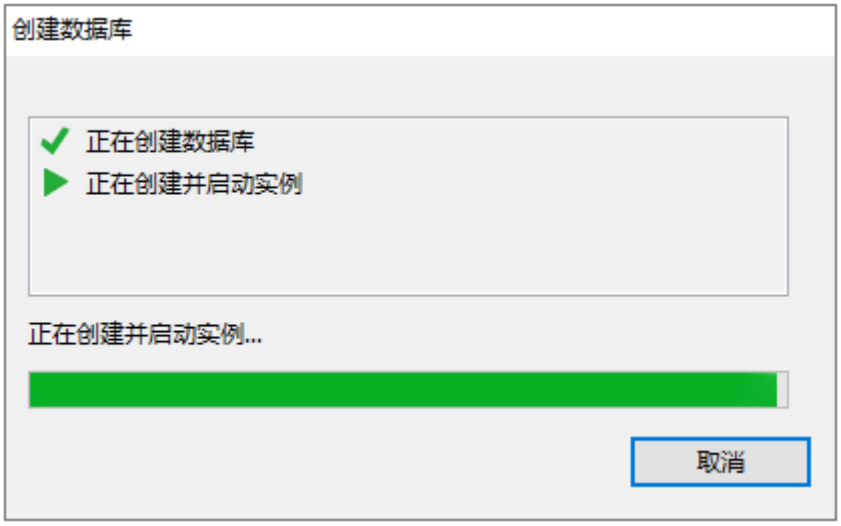


图 6.11 创建数据库

安装完成之后将显示对话框，提示安装完成或错误反馈信息，如图 6.12 所示。



图 6.12 创建数据库完成

如果数据库配置工具运行在 Linux、Solaris、AIX 和 HP-UNIX 系统中，使用非 root 系统用户在创建数据库完成时，将弹出提示框，提示应以 root 系统用户执行以下命令，用来创建数据库的随机启动服务，如图 6.13 所示。



图 6.13 执行命令提示

6.3 使用 dminit 创建数据库

在安装 DM 的过程中，用户可以选择是否创建初始数据库，如果当时没有创建，也可以在完成安装后，利用初始化库工具 dminit 来创建。系统管理员可以利用该工具提供的各种参数，设置数据库存放路径、段页大小、是否对大小写敏感以及是否使用 unicode，创建出满足用户需要的初始数据库。该工具位于安装目录的 bin\目录下。更多详细的功能请参考《DM8_dminit 使用手册》。

在 Windows“命令提示符”窗口中输入带参数的 dminit 命令启动 dminit 工具，命令格式为 dminit [para=value][para=value]……。para 为参数，value 为参数取值。dminit 工具的部分参数说明见表 6.1。

表 6.1 dminit 工具的部分参数说明

参数	含义	取值	建库后是否可修改
INI_FILE	已有 INI 文件的路径，此 INI 文件用于将其所有参数值作为当前新生成 INI 文件的参数值	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名	/
PATH	初始数据库存放的路径	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名，默认路径为	/

		dminit.exe 当前所在的工作目录	
CTL_PATH	初始数据库控制文件的路径	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名，缺省值 Windows 下是 **PATH\DB_NAME**\dm.ctl，linux 下是 **/PATH/DB_NAME**/dm.ctl（粗体表示前面设置的参数）	/
LOG_PATH	初始数据库日志文件的路径	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名。日	可修改

		志文件路径个数不超过 10 个，缺省值 Windows 下是 **PATH\DB_NAME**\DB_NAME01.log 和 **PATH\DB_NAME**\DB_NAME02.log，Linux 下是 **PATH/DB_NAME**/DB_NAME01.log 和 **PATH/DB_NAME**/DB_NAME02.log（粗体表示前面设置的参数）	
EXTENT_SIZE	数据文件使用的簇大小，即每次分配新的段空间时连续的页数	只能是 16 或 32 或 64 之一，单位为	不可修改

		页，缺省值为 16	
PAGE_SIZE	数据文件使用的页大小，可以为 4KB、8KB、16KB 或 32KB 之一，选择的页大小越大，则 DM 支持的元组长度也越大，但同时空间利用率可能下降	只能是 4、8、16 或 32 之一，单位为 KB，缺省值为 8	不可修改
LOG_SIZE	单个日志文件使用的大小，以 MB 为单位	32 位版本时，取值为 256 和 2048 之间的整数，单位为 MB，缺省值为 2048MB； 64 位版本时，取值为 256 和 8192 之间的整数，单位为 MB，缺省值为 4096MB	可修改
CASE_SENSITIVE	大小写是否敏感。Y、y、1 表示敏感，N、n、0 表示不敏感。当大小写敏感时，小写的标识符应用双引号括起，否则被转换为大写；当大小写不敏感时，	只能是 Y, y, N, n,1,0 之一，缺省值为 Y	不可修改

	系统不自动转换字符的大小写，系统比较函数会将大写字母全部转为小写字母再进行比较		
CHARSET/UNICODE_FLAG	字符集选项。0 代表 GB18030（支持 GB18030-2022 版本）；1 代表 UTF-8（支持 UNICODE 12.0 版本）；2 代表韩文字符集 EUC-KR	取值 0、1 或 2 之一。缺省值为 0	不可修改
SEC_PRIV_MODE	权限管理模式。0：表示传统模式；1：表示专用机模式；2：表示 EVAL 测评模式；3：表示 ZB 模式；4：表示 ZBMM 模式。 此参数设置仅安全版有效	取值 0、1、2、3、4 之一。缺省值为 0	可修改
SYSDBA_PWD	初始化时设置 SYSDBA 的密码	符合口令策略的字符串，详情请参考 INI 参数 PWD_POLICY	可修改
SYSAUDITOR_PWD	初始化时设置 SYSAUDITOR 的密码	符合口令策略的字符串，详情请参考 INI 参数 PWD_POLICY	可修改

DB_NAME	初始化数据库名字。只能由_、$、大写字母 A 至 Z、小写字母 a 至 z、数字 0 至 9 组成，且第一个字符不能是数字。	有效的字符串，不超过 128 个字符，缺省是 DAMENG	不可修改
INSTANCE_NAME	初始化数据库实例名字。只能由_、$、大写字母 A 至 Z、小写字母 a 至 z、数字 0 至 9 组成，且第一个字符不能是数字	有效的字符串，不超过 16 个字符，缺省是 DMSERVER	可修改
PORT_NUM	初始化时设置 dm.ini 中的 PORT_NUM	取值范围：1024~65534，缺省为 5236	可修改
BUFFER	初始化时设置 NORMAL 缓冲区大小，单位为 MB	取值范围：8~16777216。在 PAGE_SIZE 为 4K 的库中，BUFFER 最大值为 15728640。64 位系统的缺省值为 8000，32 位系统的缺省值为 64	可修改

TIME_ZONE	初始化时区	格式为[正负号]小时[:分钟]（正负号和分钟为可选）。时区设置范围为：-12:59~+14:00 缺省为东八区	不可修改
PAGE_CHECK	页检查模式。0：禁用页校验；1：开启页校验并使用 CRC 校验（CRC32 标准）；2：开启页校验并使用指定的 HASH 算法进行校验，和 PAGE_HASH_NAME 配合使用；3：开启页校验并使用快速 CRC 校验（CRC32C 标准）	取值范围：0~3。缺省值为 3	不可修改
PAGE_HASH_NAME	设置页检查 HASH 算法。当且仅当 PAGE_CHECK=2 时，使用 PAGE_HASH_NAME 来指定用于页检查的 HASH 算法	有效的字符串，不超过 128 个字符	不可修改
EXTERNAL_CIPHER_NAME	设置默认加密算法	有效的字符串，不超过 128 个字符	不可修改

EXTERNAL_HASH_NAME	设置默认 HASH 算法	有效的字符串，不超过 128 个字符	不可修改
EXTERNAL_CRYPTO_NAME	设置根密钥加密引擎	有效的字符串，不超过 128 个字符	不可修改
RLOG_ENCRYPT_NAME	设置联机日志文件和归档日志文件加密算法	有效的字符串，不超过 128 个字符	不可修改
RLOG_POSTFIX_NAME	设置联机日志文件扩展名，长度不超过 10。	有效字符串，不超过 10 个字符，缺省为 log，例如：DAMENG01.log	不可修改
USBKEY_PIN	USBKEY PIN，用于加密服务器根密钥	合法的字符串，最长为 48 个字节	不可修改
PAGE_ENC_SLICE_SIZE	数据页加密分片大小	可配置大小为 0、512 或 4096，单位为 BYTE。缺省值为 4096。其	不可修改

		中，0 表示不按分片进行加解密，即按照页大小进行加解密，后续查询参数时，将显示为页大小。	
ENCRYPT_NAME	全库加密使用的算法。算法可以是 DM 内部支持的加密算法，或者是第三方的加密算法。不指定时，不进行全库加密	合法的字符串，最长为 128 个字节	不可修改
BLANK_PAD_MODE	设置字符串比较时，结尾空格填充模式是否兼容 ORACLE。兼容后在变量赋值、列插入、列向变量赋值的情况下会进行填充，填充规则为： 如果目标类型为 CHAR，则按定义长度填充空格。 如果目标类型为 VARCHAR2，则既不填充空格也不截断结尾空格。	取值 0 或 1。0 不兼容，1 兼容。缺省为 0	不可修改
SYSTEM_MIRROR_PATH	指定 system.dbf 文件的镜像路径	绝对路径，缺省为空	不可修改
MAIN_MIRROR_PATH	指定 main.dbf 文件的镜像路径	绝对路径，缺省为空	不可修改

ROLL_MIRROR_PATH	指定 roll.dbf 文件的镜像路径	绝对路径，缺省为空	不可修改
MAL_FLAG	初始化时设置 dm.ini 中的 MAL_INI	取值 0 或 1，缺省 0	可修改
ARCH_FLAG	初始化时设置 dm.ini 中的 ARCH_INI	取值 0 或 1，缺省 0	可修改
MPP_FLAG	Mpp 系统内的库初始化时设置 dm.ini 中的 MPP_INI	取值 0 或 1，缺省 0	可修改
CONTROL	指定初始化配置文件路径。初始化配置文件是一个保存了各数据文件路径设置等信息的文本。使用 control 初始化时，若文件已存在，系统会屏幕打印提示，然后直接覆盖	主要用于将数据文件放在裸设备或 DSC 环境下	/
AUTO_OVERWRITE	0 不覆盖，表示建库目录下如果没有同名文件，直接创建。如果遇到同名文件时，屏幕提示是否需要覆盖，由用户手动输入是与否（y/n，1/0），若选择不进行覆盖则初始化失败； 1 部分覆盖，表示覆盖建库目录下所有同名文件； 2 完全覆盖，表示先清理掉建库目录下所有文件再重新创建	只能是 0,1,2 之一，缺省值为 0	/

USE_NEW_HASH	字符类型在计算 HASH 值时所采用的 HASH 算法类别。0：原始 HASH 算法；1：改进的 HASH 算法	取值 0 或 1，缺省值为 1	不可修改
ELOG_PATH	指定初始化过程中生成的事件日志文件所在路径	合法的路径。文件路径长度最大为 257(含结束符)，不包括文件名	/
AP_PORT_NUM	分布式环境下协同工作的监听端口	手动配置的取值范围为 1024~65534。缺省值为 0，表示未配置	可修改
HUGE_WITH_DELTA	是否仅支持创建事务型 HUGE 表	取值：1 是；0 否。缺省值为 1	不可修改
RLOG_GEN_FOR_HUGE	是否生成 HUGE 表 REDO 日志	取值：1 是；0 否。缺省值为 1	不可修改
PSEG_MGR_FLAG	是否仅使用管理段记录事务信息	取值：1 是；0 否。缺省值为 0	不可修改
CHAR_FIX_STORAGE	定长字符（CHAR）是否按定长存储	取值：Y/1：表示定长字符（CHAR）	不可修改

		按定长存储，即不允许 CHAR 变化为 VARCHAR2 进行存储；N/0：表示定长字符（CHAR）按变长存储，即允许 CHAR 变化为 VARCHAR2 进行存储。缺省值为 N	
SQL_LOG_FORBID	是否禁止打开 SQL 日志	取值：Y/1：是；N/0：否。缺省值为 N	不可修改
DPC_MODE	DMDPC 专用参数。指定 DMDPC 集群中的实例角色	取值：0：无；1/MP：MP；2/BP：BP；3/SP：SP。缺省值为 0	不可修改

SYSSSO_PWD	初始化时设置 SYSSSO 的密码，仅在安全版本下可见和可设置	符合口令策略的字符串，详情请参考 INI 参数 PWD_POLICY	可修改
SYSDBO_PWD	初始化时设置 SYSDBO 的密码，仅在安全版本且 PRIV_FLAG=1 时可见和可设置。	符合口令策略的字符串，详情请参考 INI 参数 PWD_POLICY	可修改
PRIV_FLAG	是否是四权分立。四权分立的具体权限见《DM8 安全管理》。默认情况下，使用三权分立。 仅在安全版本下可见和可设置	取值：1 是，0 否。缺省值为 0	不可修改
ENABLE_MAC	是否打开强制访问控制功能，仅对安全版有效。开启成功后后续启动无需再添加 enable_mac 参数	取值：1 启用，0 不启用。缺省值为 0。取值为 1 时，参数 ENABLE_MAC 在系统中被赋值为 1；取值为 0 时，参数	不可修改

		ENABLE_MAC 在系统中被赋值为 2	
MAIN_DBF_PATH	MAIN 数据文件存放路径	合法的路径。文件路径长度最大为 257(含结束符)，包括文件名。路径不允许是 HUGE_PATH 路径及其子目录 缺省值 Windows 下是 **PATH\DB_NAME**\MAIN.DBF，linux 下是 **/PATH/DB_NAME**/MAIN.DBF（粗体表示前面设置的参数）	不可修改

SYSTEM_DBF_PATH	SYSTEM 数据文件存放路径	合法的路径。 文件路径长度最大为 257(含结束符)，包括文件名。 路径不允许是 HUGE_PATH 路径及其子目录缺省值 Windows 下是 **PATH\DB_NAME**\SYSTEM.DBF，linux 下是 **/PATH/DB_NAME**/SYSTEM.DBF（粗体表示前面设置的参数）	不可修改
ROLL_DBF_PATH	ROLL 数据文件存放路径	合法的路径。 文件路径长度最大为 257(含结束符)，包括文件名。	不可修改

		路径不允许是 HUGE_PATH 路径及其子目录 缺省值 Windows 下是 **PATH\DB_NAME**\ROLL.DBF，linux 下是 **/PATH/DB_NAME**/ROLL.DBF（粗体表示前面设置的参数）	
TEMP_DBF_PATH	TEMP 数据文件存放路径	合法的路径。 文件路径长度最大为 257(含结束符)，包括文件名 路径不允许是 HUGE_PATH 路径及其子目录	可修改

		缺省值 Windows 下是 **PATH\DB_NAME**\TEMP.DBF，linux 下是 **/PATH/DB_NAME**/TEMP.DBF（粗体表示前面设置的参数）	
USE_DB_NAME	初始化数据库的路径是否拼接 DB_NAME	取值：Y、y、1：是，N、n、0：否。缺省值为 1	不可修改
RANDOM_CRYPTO	指定数据库中使用的随机数算法所在的加密引擎名。未指定或指定的加密引擎不存在，又或者指定的加密引擎中不存在随机数算法，则使用 OPENSSL 提供的软件随机数算法	支持的加密引擎名可以通过查询视图 V$EXTERNAL_CIPHER_LIBS 的 NAME 列获取	可修改

AUTO_ADJ_PARA	是否对 CPU 和内存相关 INI 参数进行自动调优。为 1 时 AUTO_ADJ_CPUS、AUTO_ADJ_MEM 才生效。	取值：1：是；0：否，缺省值为 1	不可修改
AUTO_ADJ_CPUS	指定数据库可用 CPU 核数进行 INI 参数自动调优，AUTO_ADJ_PARA 打开才生效。默认根据当前所有 CPU 对于相关 INI 进行调优。具体请参考《DM8_dminit 使用手册》。	取值范围 0~64，缺省值为 0	不可修改
AUTO_ADJ_MEM	指定数据库可用内存大小进行 INI 参数自动调优，AUTO_ADJ_PARA 打开才生效，默认使用当前机器 80%的内存对于相关 INI 进行调优。具体请参考《DM8_dminit 使用手册》。	最小为 0，最大可设置为最大八位整数，单位：MB，缺省值为 0	不可修改
HELP	显示帮助信息		

说明：dminit 一般是要有参数的，如果没有带参数，系统就会引导用户设置。另外，参数、等号和值之间不能有空格。Help 参数的后面不用添加'='号。

例如：

```
dminit PATH=c:\dmdata PAGE_SIZE=16 SYSDBA_PWD=DMdba_123 SYSAUDITOR_PWD=DMauditor_123
```

如果创建成功，则屏幕显示如图 6.14 所示。此时在 c 盘根目录下出现 dmData 文件夹，内容包含初始数据库 DAMENG 的相关文件和初始化文件 dm.ini。将 dm.ini 文件拷贝到 DM 安装目录的 bin 下，DM 服务器就可以启动该初始数据库了。

```
initdb V8
db version: 0x7000a
file dm.key not found, use default license!
License will expire on 2021-06-01

 log file path: c:\dmdata\DAMENG\DAMENG01.log


 log file path: c:\dmdata\DAMENG\DAMENG02.log

write to dir [c:\dmdata\DAMENG].
create dm database success. 2020-07-08 16:34:12
```

图 6.14 创建成功后的屏幕显示

6.4 注册数据库服务

注册数据库服务，该功能用于将使用命令行工具生成的数据库，重新注册成系统服务，方便用户管理与控制。

用户注册数据库服务可以通过图形化界面和 shell 脚本两种方式，本节讲述的是通过数据库配置工具注册数据库服务，通过 shell 脚本注册数据库服务详细操作请参见《DM8_Linux 服务脚本手册》。如图 6.16 所示。

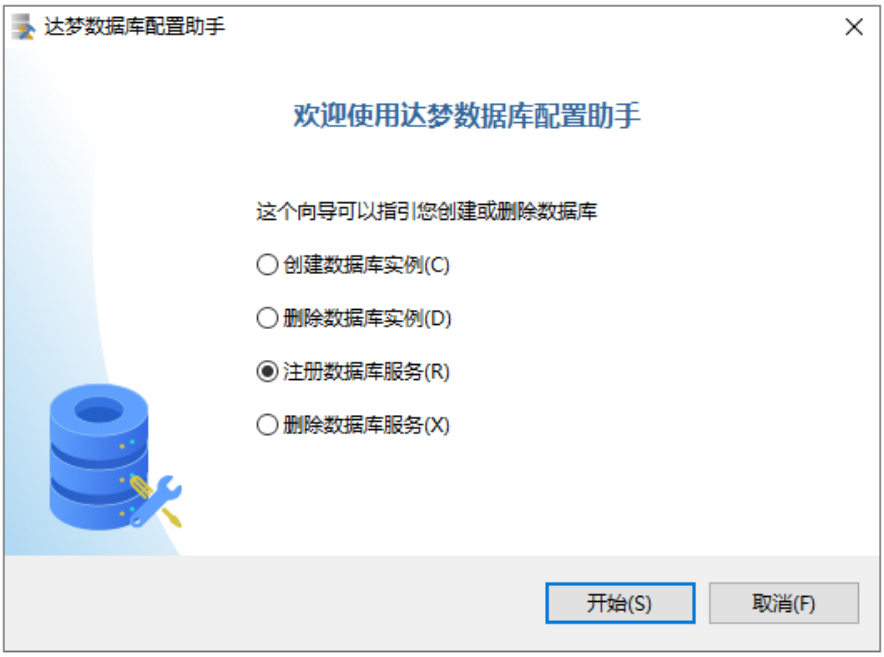


图 6.16 选择操作方式

用户选择 dm.ini 文件来注册相应的数据库，并可修改相应的端口号和实例名，也可以选

择是否以配置状态启动数据库，如图 6.17 所示。

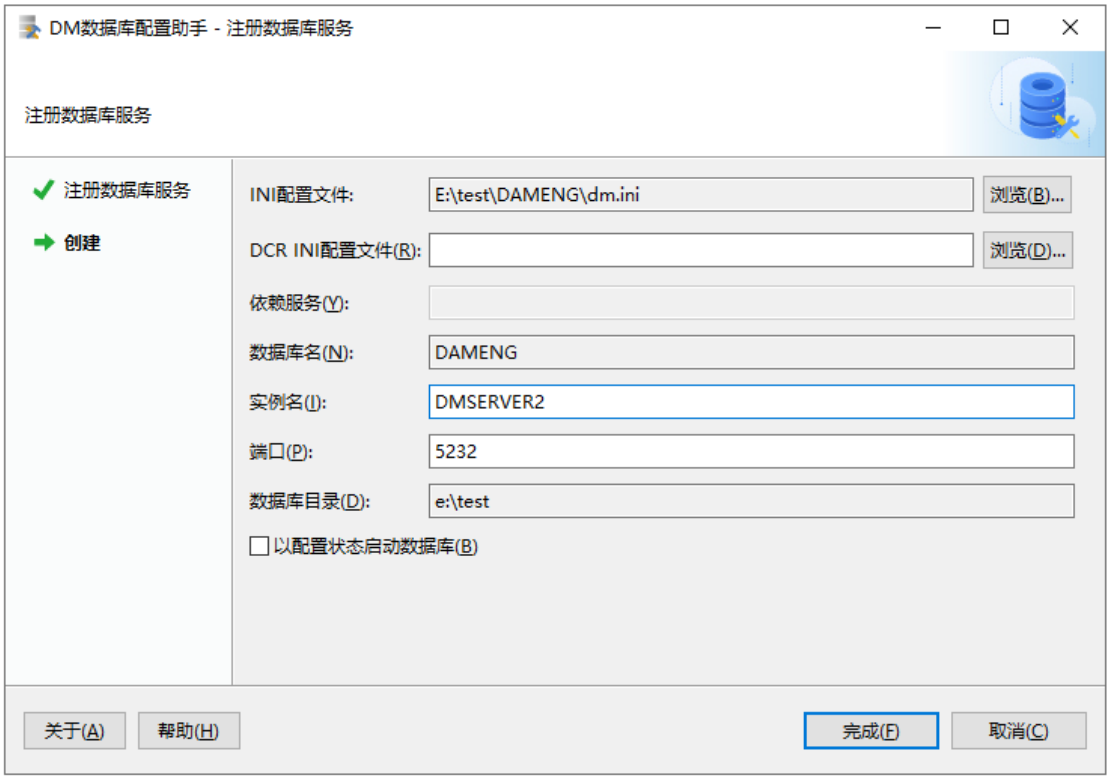


图 6.17 注册数据库服务

正在注册数据库服务，如图 6.18 所示。

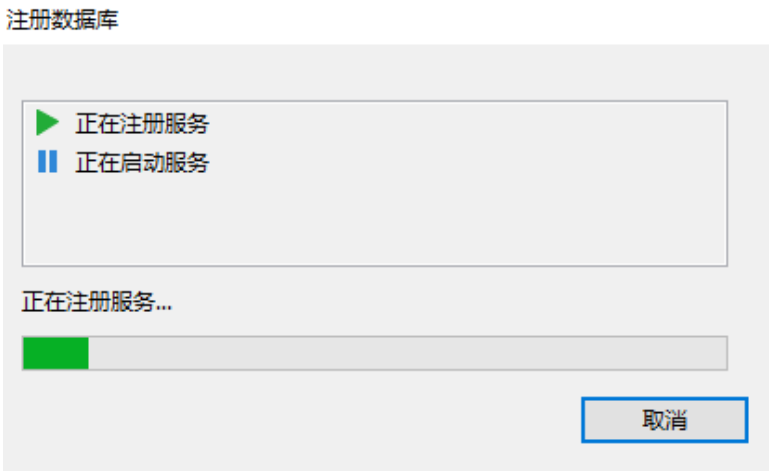


图 6.18 注册数据库服务

注册完成之后将显示对话框，提示完成或错误反馈信息，如图 6.19 所示。



图 6.19 注册数据库完成

6.5 查看数据库信息

登录 DM 管理工具，通过表空间属性，可以查看数据文件路径、总空间大小、空闲空间大小、使用率。如图 6.20 所示。

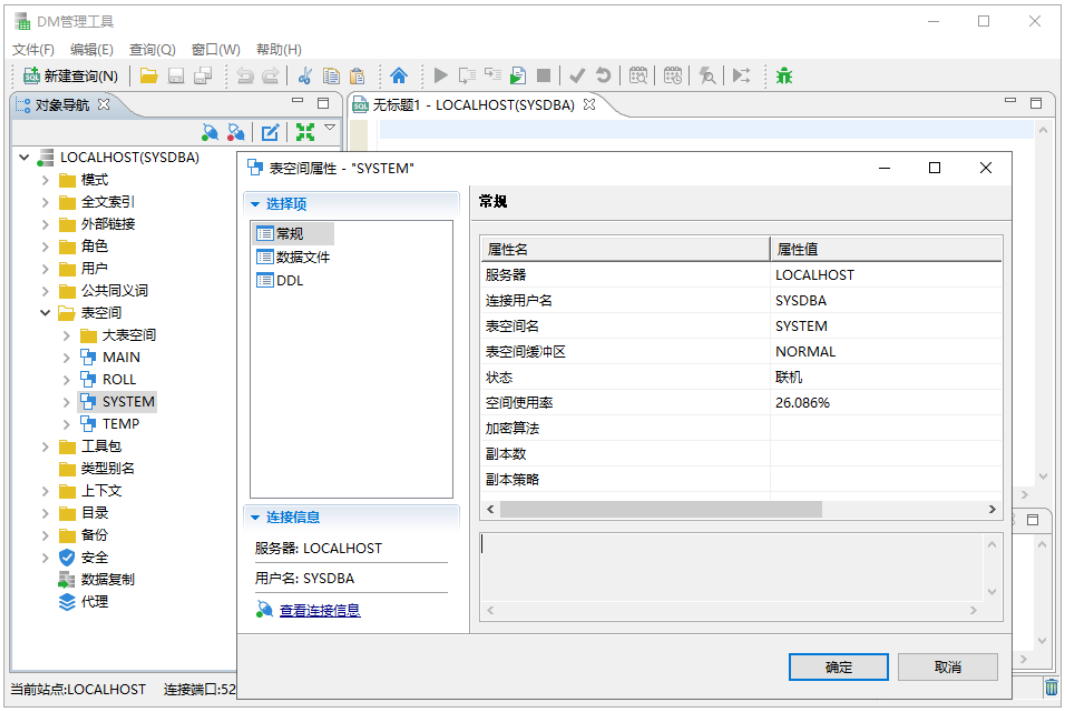


图 6.20 使用 DM 管理工具查看数据库信息

通过 DM 控制台工具，查看实例配置属性，如图 6.21 所示，可以查看到如下信息：控制文件、实例名、内存、线程、查询、数据库、检查点、IO、REDO 日志、事务、安全、兼容性、监控、数据守护、全文索引、配置文件和 DSC 等相关参数。

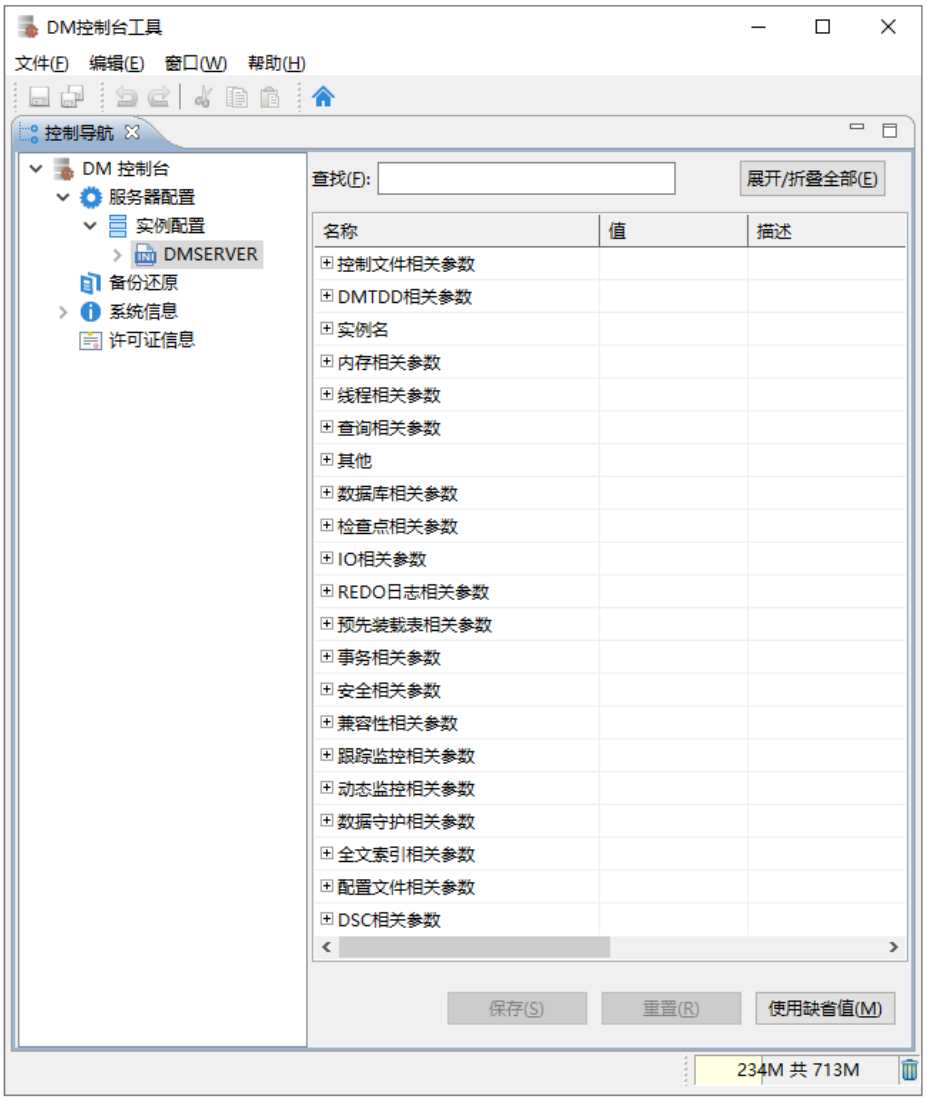


图 6.21 使用 DM 控制台工具查看数据库信息

6.6 删除数据库

删除数据库，包括删除数据库的数据文件、日志文件、控制文件和初始化参数文件。为了保证删除数据库成功，必须保证 DMServer 已关闭。

可以使用数据库配置工具来删除数据库。如图 6.22 所示。

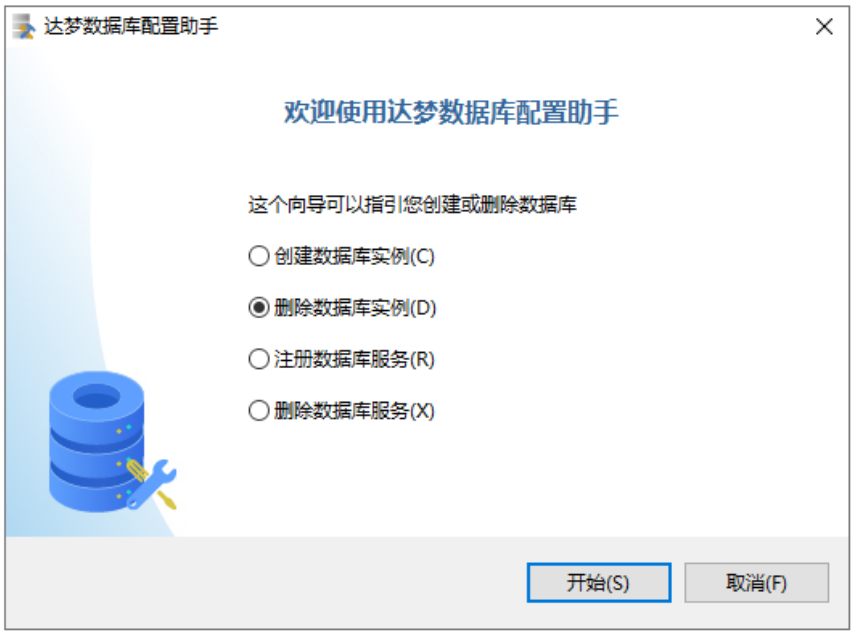


图 6.22 选择操作方式

根据数据库实例名称，选择要删除的数据库，如图 6.23 所示，也可以通过指定数据库配置文件删除数据库。

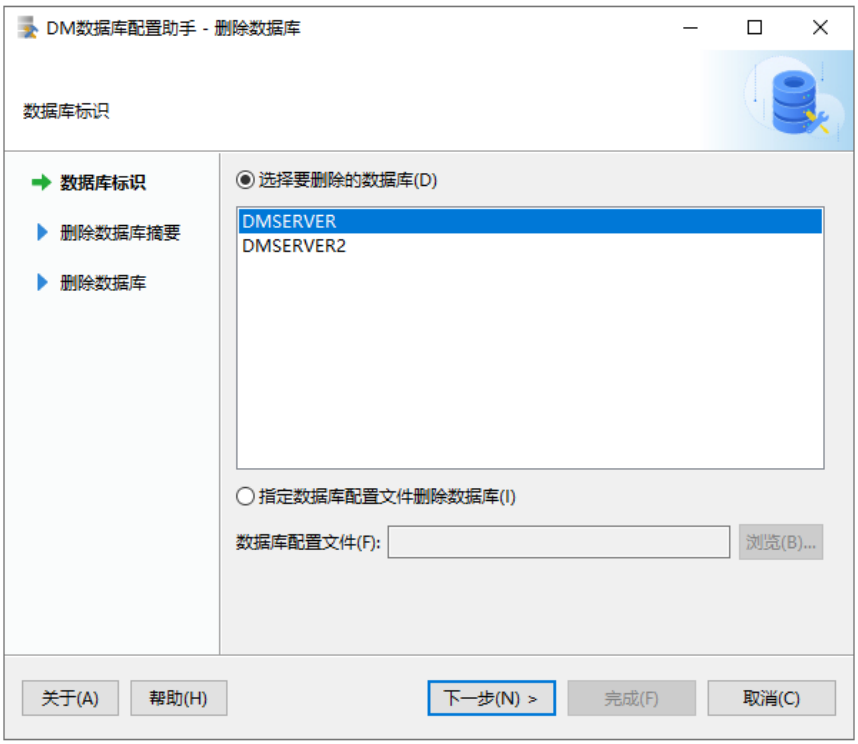


图 6.23 要删除的数据库

确认将删除的数据库名、实例名、数据库目录，如图 6.24 所示。

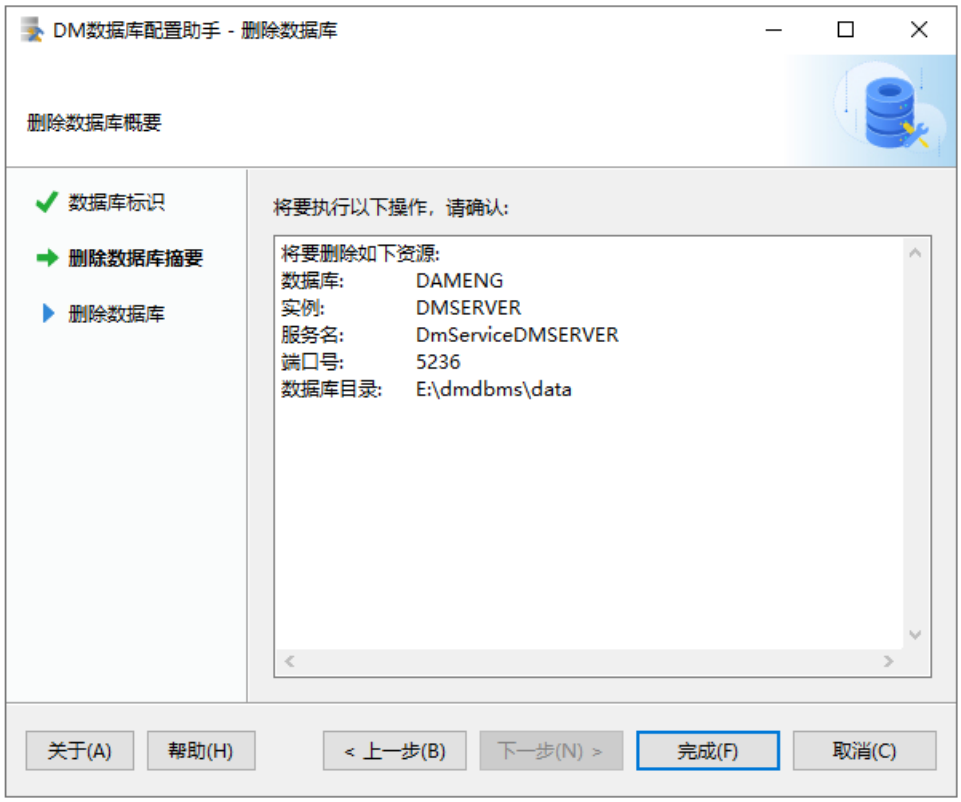


图 6.24 删除数据库摘要

首先停止实例，然后删除实例，最后删除数据库，如图 6.25 所示。

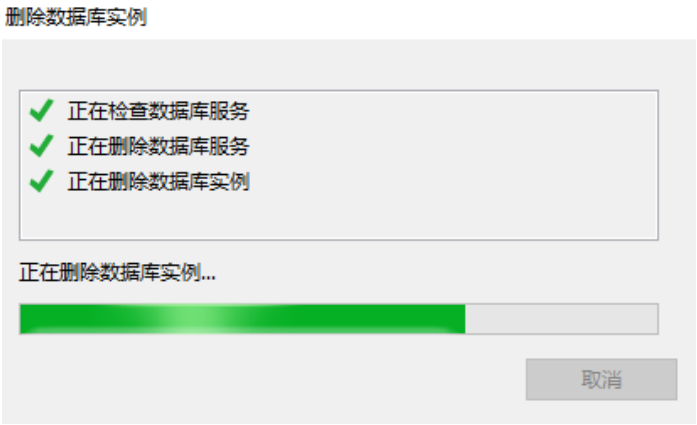


图 6.25 删除数据库实例

删除完成之后将显示对话框，提示完成或错误反馈信息，如图 6.26 所示。



图 6.26 删除数据库完成

如果数据库配置工具运行在 Linux、Solaris、AIX 和 HP-UNIX 系统中，使用非 root 系统用户在删除数据库完成时，将弹出提示框，提示应以 root 系统用户执行以下命令，用来删除数据库的随机启动服务，如图 6.27 所示。

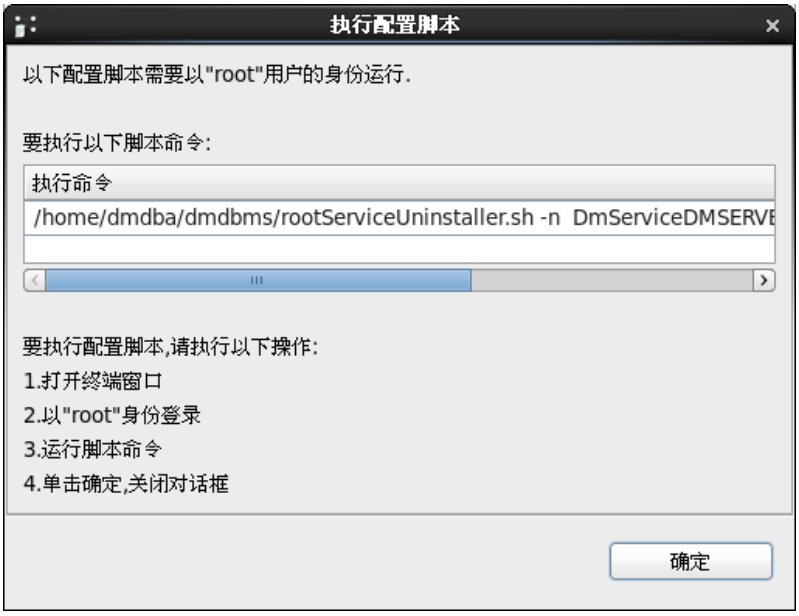


图 6.27 执行命令提示

6.7 删除数据库服务

删除数据库服务，只删除用于启动和停止数据库的服务文件，不会删除数据库的数据文

件、日志文件、控制文件和初始化参数文件。

用户删除数据库服务可以通过图形化界面和 shell 脚本两种方式，本节讲述的是通过数据库配置工具删除数据库服务，通过 shell 脚本删除数据库服务详细操作请参考《DM8_Linux 服务脚本使用手册》。如图 6.28 所示。

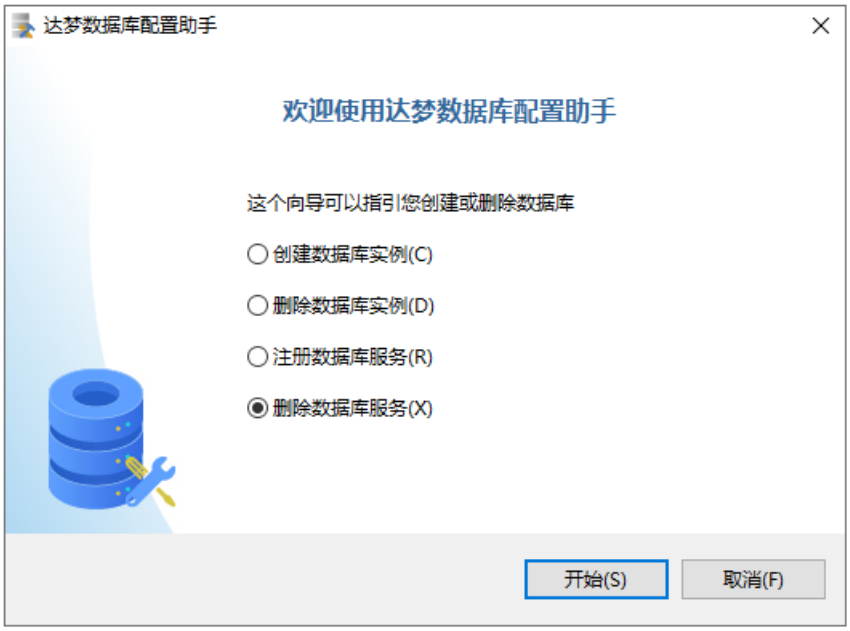


图 6.28 选择操作方式

根据数据库服务名称，选择要删除的数据库服务，如图 6.29 所示，也可以通过指定数据库配置文件删除数据库服务。

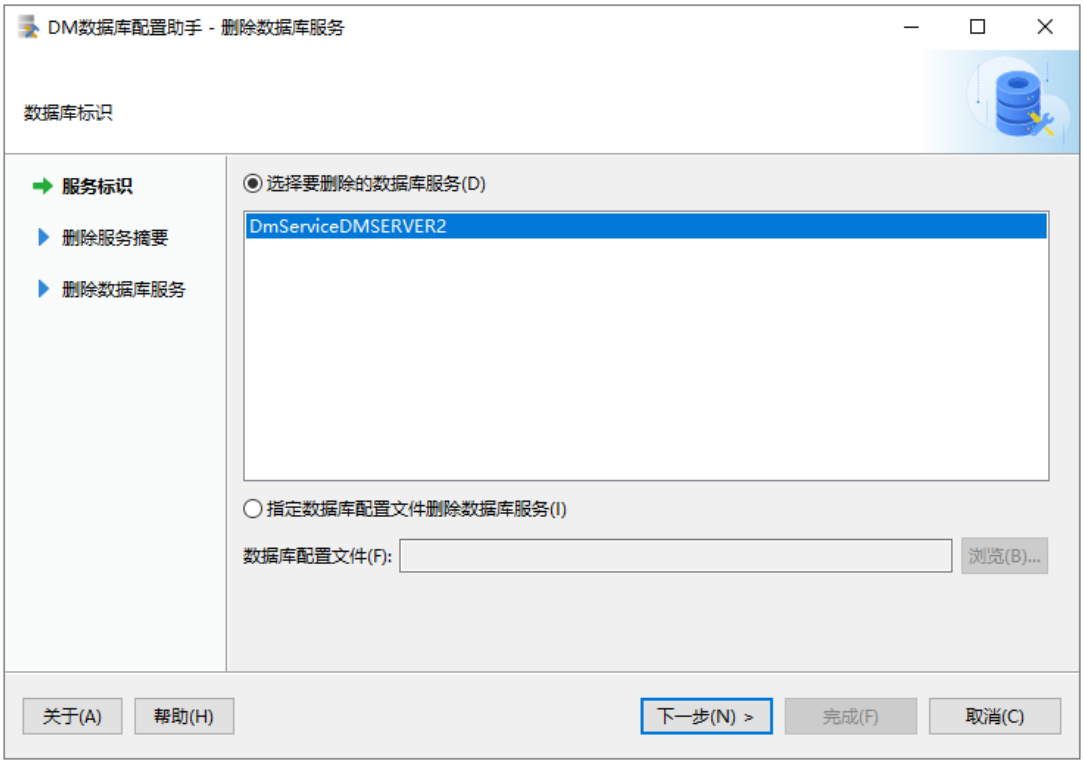


图 6.29 要删除的数据库服务

确认将删除的数据库名、实例名、数据库服务名、数据库目录，如图 6.30 所示。

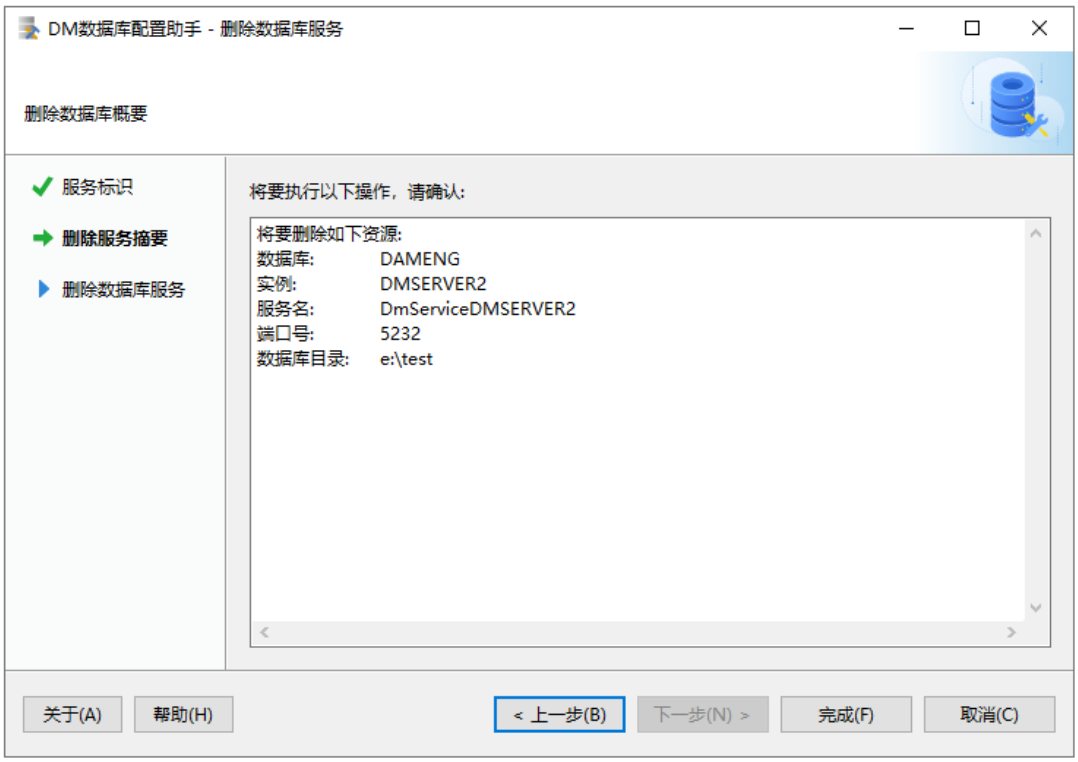


图 6.30 删除服务摘要

首先检查数据库服务，然后删除数据库服务，如图 6.31 所示。

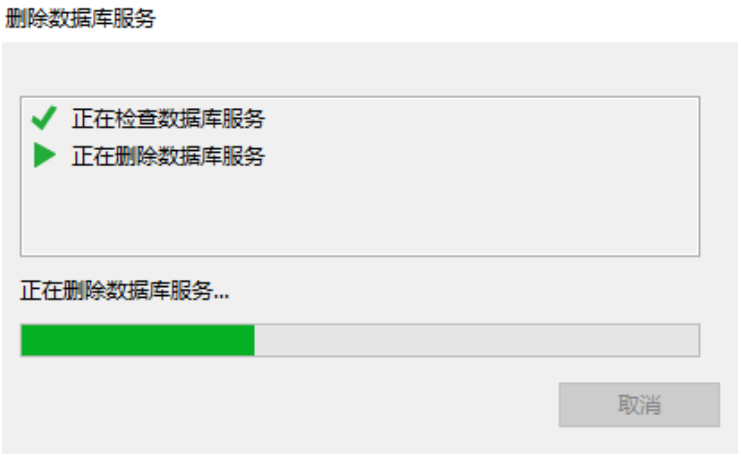


图 6.31 删除数据库服务

删除完成之后将显示对话框，提示完成或错误反馈信息，如图 6.32 所示。



图 6.32 删除数据库服务完成

如果数据库配置工具运行在 Linux、Solaris、AIX 和 HP-UNIX 系统中，使用非 root 系统用户在删除数据库服务完成时，将弹出提示框，提示应以 root 系统用户执行以下命令，用来删除数据库的随机启动服务，如图 6.33 所示。

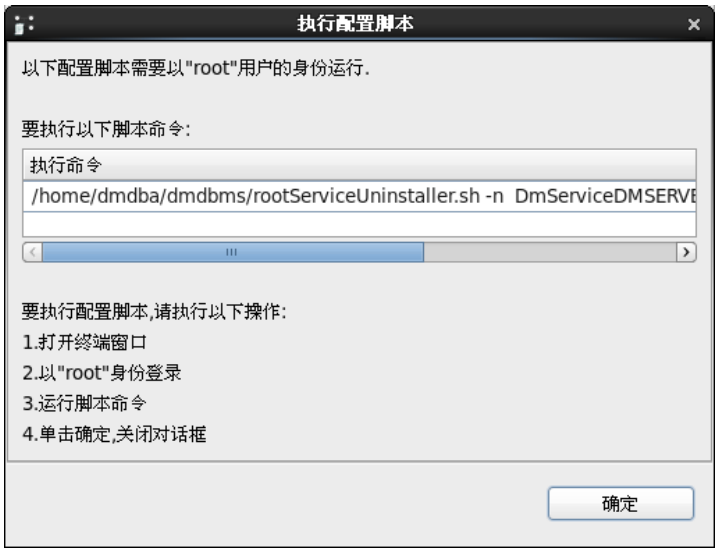

执行配置脚本
以下配置脚本需要以"root"用户的身份运行.
要执行以下脚本命令:
执行命令
/home/dmdba/dmdbms/rootServiceUninstaller.sh -n DmServiceDMSERVE
要执行配置脚本,请执行以下操作:
1.打开终端窗口
2.以"root"身份登录
3.运行脚本命令
4.单击确定,关闭对话框
确定

图 6.33 执行命令提示

第7章 启动和关闭数据库

DM 支持多种操作系统。常用的操作系统基本上可以划分为 Windows 操作系统和 Linux 操作系统。下面分别介绍这两种操作系统下启动和关闭 DM 数据库的操作。

7.1 启动数据库

7.1.1 Windows 系统

1. 菜单方式

安装 DM 数据库后(默认情况下安装成功后 DM 服务会自动启动)，在 Windows 的开始菜单选项中选择如图 7.1 所示的菜单项中的 DM 服务查看器可以启动 DM 数据库。



图 7.1 DM 数据库菜单方式启动 1

点击 DM 服务查看器选项后，会弹出如图 7.2 所示的界面:



图 7.2 DM 数据库菜单方式启动 2

在弹出界面中选中所要启动的数据库，点击鼠标右键，在菜单栏中选择启动。

2. Windows 服务方式

安装 DM 数据库并且新建一个 DM 实例后。Windows 的服务中会自动增加一项和该实例名对应的服务。例如：新建一个实例名为 DMSERVER 的 DM 数据库，Windows 的服务中会增加一项名称为“DmServiceDMSERVER”的服务。打开 Windows 的管理工具，选择“服务”，打开 Windows 服务控制台，如图 7.3 所示，选择“DmServiceDMSERVER”，用鼠标在工具栏点击启动按钮或者点击鼠标右键，在菜单栏中选择“启动”，启动 DM 数据库。

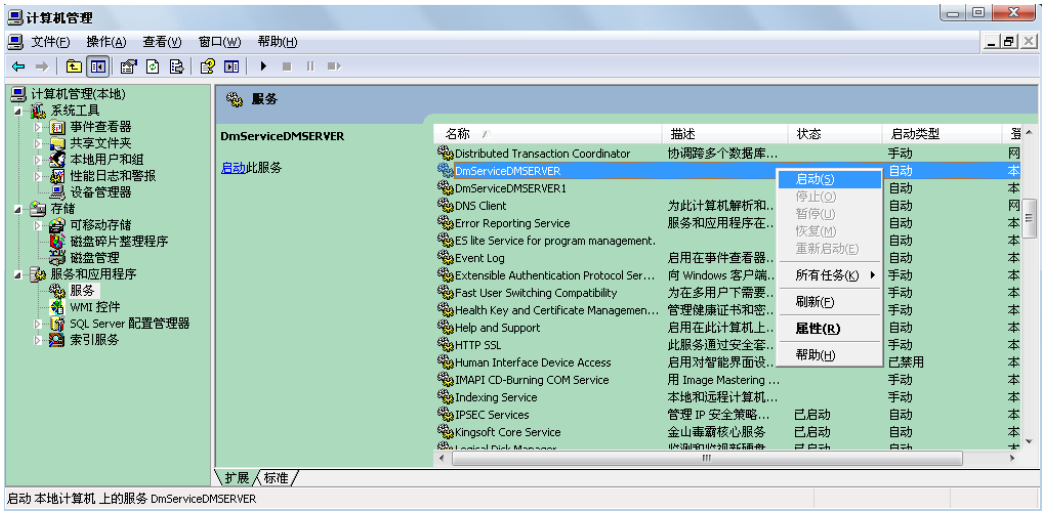


图 7.3 DM 数据库服务方式启动

3. 命令行方式

进入 DM 安装目录下的 bin 目录，直接打开应用程序 dmserver 就可以启动 DM 数据库。或者先打开 Windows 命令提示符工具，在命令工具中执行命令进入 DM 服务器的目录，再执行 dmserver 的命令启动 DM 数据库，如图 7.4 所示。

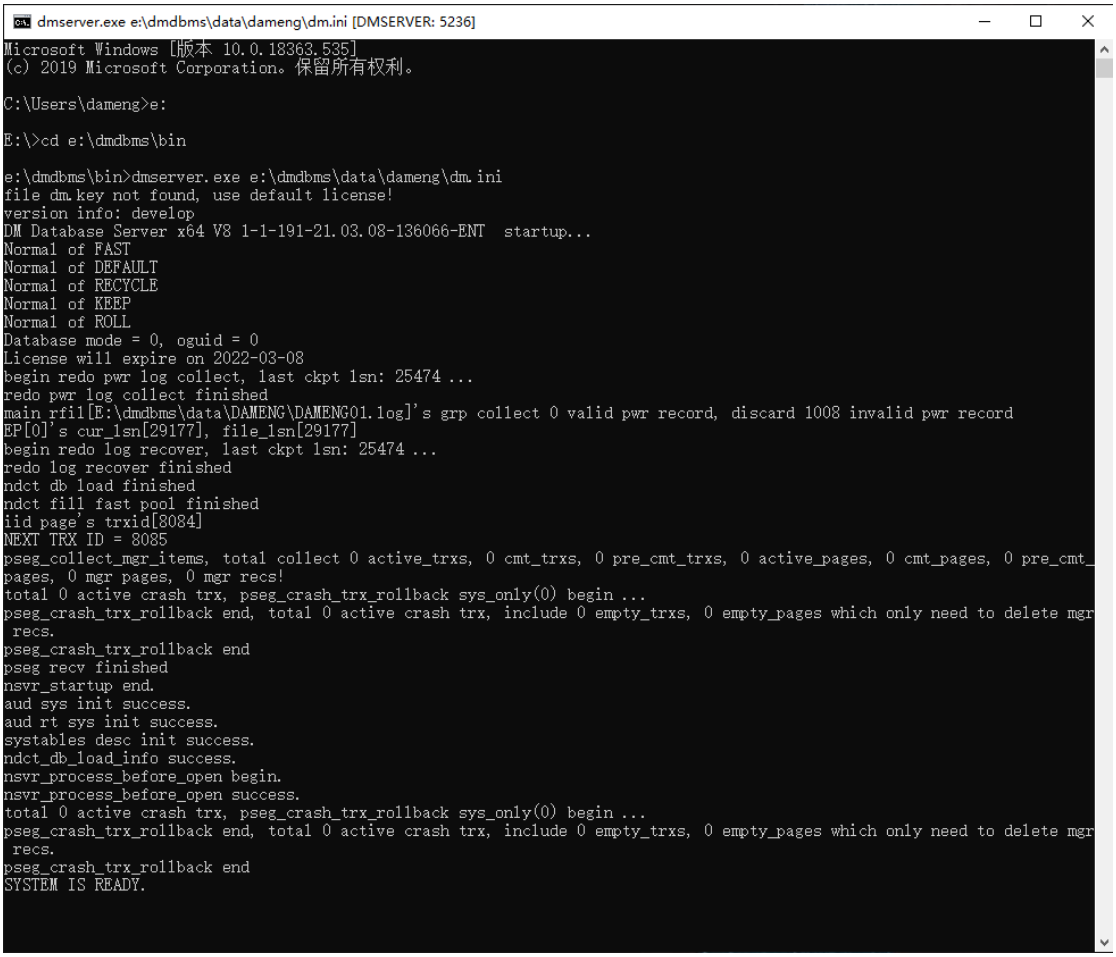


图 7.4 DM 数据库命令行方式启动

启动 DM 实例时，可指定 dm.ini 文件的路径、非控制台方式启动、指定数据库是否以 MOUNT 状态启动等启动参数。DM 实例启动时也可不指定任何参数，缺省使用当前目录下的 dm.ini 文件，如果当前目录不存在 dm.ini 文件，则无法启动。

命令行方式启动参数如下：

```
dmserver.exe [[path=]ini_file_path][dcr_ini=dcr_path][-
noconsole][mount][dpc_mode=mode][upd_lic=value][recover_check=value][enable_mac
][listen_ipv6=value]
```

参数说明：

path：dm.ini 绝对路径。缺省为 dmserver 当前目录的 dm.ini；

dcr_ini：如果使用 css 集群环境，指定 dmdcr.ini 文件路径；

-noconsole：以服务方式启动。如果以此方式启动，则无法通过在控制台中输入服务器命令；

mount：以 MOUNT 配置状态启动，关于更多数据库状态见 7.2 数据库状态和模式介绍；

dpc_mode：指定 DPC 中的实例角色，缺省为 0。0：无、1：MP、2：BP、3：SP、4：BS，取值 1/2/3/4 时也可以用 MP/BP/SP/BS 代替；

upd_lic：是否升级服务器安全版本信息。0：否；1：自动升级；2：强制升级。缺省为 0。一般情况下无需使用该参数。取值 1 和 2 专门用于将非安全版的系统升级为安全版的特殊场景：在当前非安全系统中放入安全版 dm.key 之后，以 upd_lic=1 方式启动服务器，系统会检查当前的版本信息，并将非安全版的系统升级为安全版，而对于已经是安全版的情况，检查版本标记后不会进行升级操作；以 upd_lic=2 方式启动服务器，不管之前是否是安全版本，均会强制进行升级的相关操作，主要用于处理之前升级失败但却设置了版本标记的特殊错误情况。升级之后再次启动 DM 实例时无需再指定 upd_lic 参数；

recover_check：是否检查数据库启动前有没有执行更新 DB_MAGIC（即检查数据恢复状态，具体说明可以查看《DM8 备份与还原》）。0：不检查；1：检查；缺省为 1。当该参数设置为 0 时，允许在执行更新 DB_MAGIC 之前以备库配置状态启动数据库，并进行只读操作；

enable_mac：是否开启强制访问控制功能，仅在当前库是安全版时或者非安全版通过 upd_lic 升级为安全版时有效。取值为 1 表示启用，此时参数 ENABLE_MAC 在系统中被赋值为 1；取值为 0 表示不启用，此时参数 ENABLE_MAC 在系统中被赋值为 2。缺省值为 0。开启成功后后续启动无需再添加 enable_mac 参数；

listen_ipv6：是否监听 IPV6。0：否；1：是。缺省为 1；

help：打印帮助信息。

当不确定启动参数的使用方法时，可以使用 help 参数，将打印出格式、参数说明和使用

示例。使用方法如下:

```
dmserver help
```

当以控制台方式启动 DMServer 时，用户可以在控制台输入一些命令，服务器将在控制台打印出相关信息或执行相关操作。支持的命令见下表。

表 7.1 DMServer 控制台支持的命令

命令	操作
EXIT	退出服务器
LOCK	打印锁系统信息
TRX	打印等待事务信息
CKPT	设置检查点
BUF	打印内存池中缓冲区的信息
MEM	打印服务器占用内存大小
SESSION	打印连接个数
DEBUG	打开 DEBUG 模式

7.1.2 Linux 系统

1. 菜单方式

安装 DM 数据库后(默认情况下安装成功后 DM 服务会自动启动)，在 Linux 的开始菜单选项中选择启动服务器菜单项可以启动 DM 数据库。启动方式类似 Windows。

2. Linux 服务方式

安装 DM 数据库后，在/etc/rc.d/init.d 中有名称为 DmService 开头的文件，文件全名为 DmService+实例名（例如：如果实例名为 DMSERVER，则相对应的服务文件为 DmServiceDMSERVER)。以实例名为 DMSERVER 为例,在终端输入./DmServiceDMSERVER

start 或者 service DmServiceDMSERVER start 即可启动 DM 数据库。

3. 命令行方式

在终端进入 DM 安装目录下的 bin 目录，执行./dmserver 启动 DM 数据库，参数选项同 Windows。

7.1.3 检查 LICENSE

无论是在何种操作系统下运行，DM 数据库在启动时都会进行 LICENSE 检查。

LICENSE 内容包括许可证版本号、LICENSE 文件序列号、服务器版本类型、服务器发布类型、服务器版本号、有效日期等。可通过查看 V$LICENSE 了解所安装的 DM 数据库的 LICENSE 信息。

LICENSE 文件为 dm.key。LICENSE 过期会导致 DM 服务器会强制退出。

LICENSE 类型须与 DM 软件类型相匹配。例如安全版的 DM 安装软件须使用安全版的 LICENSE(即安全版 dm.key)。

当服务器以 UPD_LIC=1 或 UPD_LIC=2 参数启动服务器,如果之前是非安全版的环境,当前的 dm.key 是安全版的 KEY，服务器将会从非安全版环境升级为安全版。

7.2 数据库状态和模式

DM 数据库包含以下几种状态:

1. 启动状态（STARTUP）：数据库刚启动时为 STARTUP 状态。

2. 日志重做完成状态（AFTER REDO）：数据库启动过程中联机日志重做完成后，回滚活动事务前为 AFTER REDO 状态。非 STANDBY 模式的数据库在将系统设置为 OPEN 状态前，也会先将系统设置为 AFTER REDO 状态。

3. 配置状态 (MOUNT)：不允许访问数据库对象，只能进行控制文件维护、归档配置、

数据库模式修改等操作。

4. 打开状态（OPEN）：不能进行控制文件维护、归档配置（不包括增加/删除同步归档）等操作，可以访问数据库对象以及增加/删除同步归档，对外提供正常的数据库服务。

5. 挂起状态（SUSPEND）：与 OPEN 状态的唯一区别就是，限制磁盘写入功能；一旦修改了数据页，触发 REDO 日志、数据页刷盘，当前用户将被挂起。

6. 关闭状态（SHUTDOWN）：数据库正常关闭时为 SHUTDOWN 状态。

OPEN 状态与 MOUNT 和 SUSPEND 能相互转换，但是 MOUNT 和 SUSPEND 之间不能相互转换。STARTUP、AFTER REDO 和 SHUTDOWN 状态为系统内部状态，用户不能主动干预。

DM 数据库包含以下几种模式：

1. 普通模式（NORMAL）：用户可以正常访问数据库，操作没有限制；

2. 主库模式（PRIMARY）：用户可以正常访问数据库，所有对数据库对象的修改强制生成 REDO 日志，在归档有效时，发送 REDO 日志到备库；

3. 备库模式（STANDBY）：接收主库发送过来的 REDO 日志并重做。数据对用户只读。

NORMAL、PRIMARY、STANDBY 三种数据库模式切换在 MOUNT 状态下可以直接执行，在 OPEN 状态下需要加 FORCE 关键字强制执行（OPEN 状态下的模式切换仅在单机和 DSC 环境下有效）。

对于新初始化的库，首次启动不允许使用 MOUNT 方式，需要先正常启动并正常退出，然后才允许 MOUNT 方式启动。

一般情况下，数据库为 NORMAL 模式，如果不指定 MOUNT 状态启动，则自动启动到 OPEN 状态。

在需要对数据库配置时（如配置数据守护时），服务器需要指定 MOUNT 状态启动。当数据库模式为非 NORMAL 模式（PRIMARY、STANDBY 模式），无论是否指定启动状态，服务器启动时自动启动到 MOUNT 状态。

7.3 关闭数据库

7.3.1 Windows 系统

1. 菜单方式

在 Windows 的开始->程序菜单中选择达梦数据库->DM 服务查看器，在弹出的界面中，选中要关闭的数据库，点击鼠标右键，在菜单栏中选择停止。

2. Windows 服务方式

安装 DM 数据库并且新建一个 DM 实例后。Windows 的服务中会自动增加一项和该实例名对应的服务。例如：新建一个实例名为 DMSERVER1 的 DM 数据库，Windows 的服务中会增加一项名称为“DmServiceDMSERVER1”的服务。打开 Windows 的管理工具，选择服务，打开 Windows 服务控制台，如图 7.5 所示，选择“DmServiceDMSERVER1”，用鼠标在工具栏点击停止按钮或者点击鼠标右键，在菜单栏中选择停止，关闭 DM 数据库。

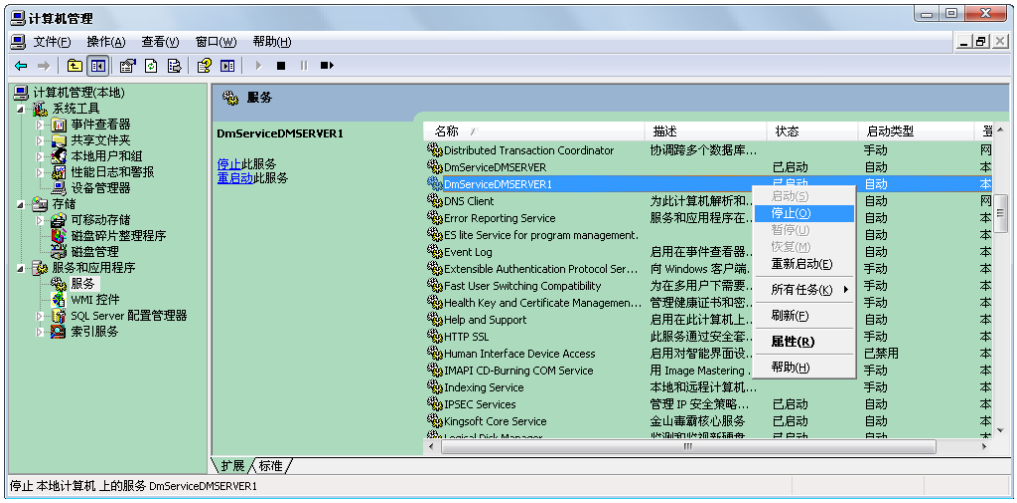


图 7.5 DM 数据库服务方式停止

3. 命令行方式

在启动数据库的命令工具中输入 exit，然后回车，关闭 DM 数据库。如图 7.6 所示。

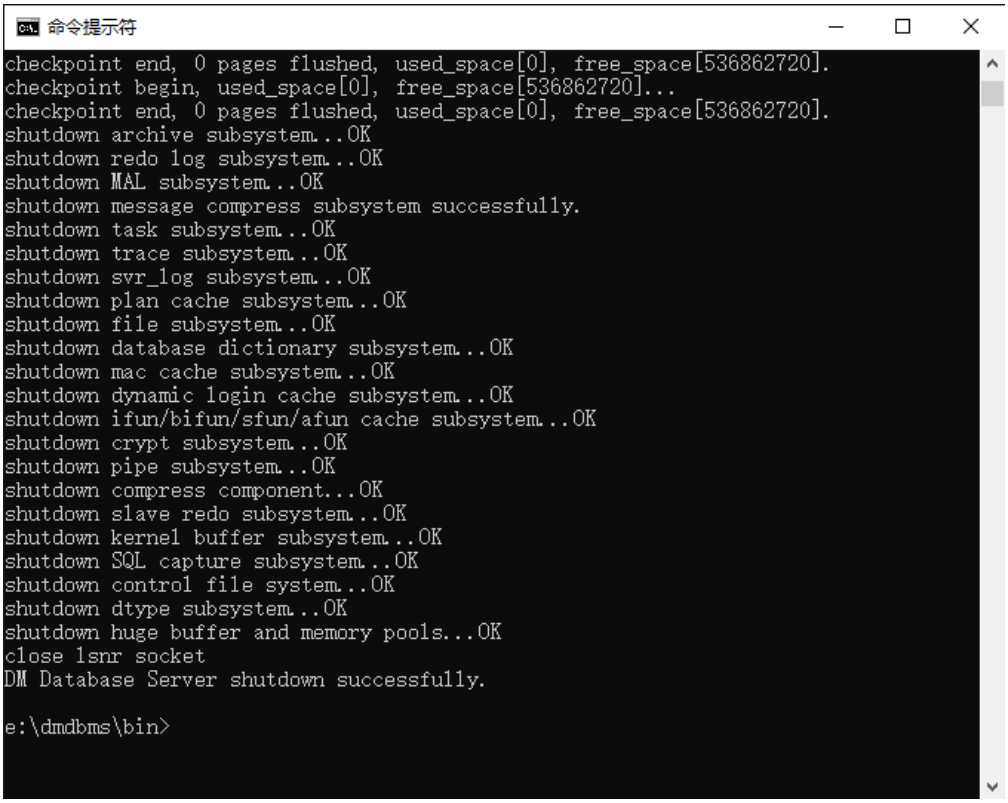


图 7.6 DM 数据库命令行方式停止

7.3.2 Linux 系统

1. 菜单方式

在 Linux 的开始菜单中选择关闭数据库的菜单项可以关闭 DM 数据库。关闭方式类似 Windows。

2. Linux 服务方式

进入/etc/rc.d/init.d，以实例名为 DMSERVER 为例，在命令行工具中输入./DmServiceDMSERVER stop 即可关闭 DM 数据库。

3. 命令行方式

在启动数据库的命令工具中输入 exit，然后回车，退出 DM 数据库。关闭方式类似 Windows。

第8章 管理模式对象的空间

模式对象的空间管理关系到空间的有效使用和数据的合理分布，本章介绍了模式对象相关的存储参数、多余空间的回收机制、模式对象上的空间限制、如何查看已使用的存储空间以及各种数据类型实际使用的空间大小。

8.1 设置存储参数

8.1.1 普通表和索引

对于普通表和索引，DM8 提供了以下的存储参数：

1. 初始簇数目 INITIAL：指建立表时分配的簇个数，必须为整数，最小值为 1，最大值为 256，缺省为 1；
2. 下次分配簇数目 NEXT：指当表空间不够时，从数据文件中分配的簇个数，必须为整数，最小值为 1，最大值为 256，缺省为 1；
3. 最小保留簇数目 MINEXTENTS：当删除表中的记录后，如果表使用的簇数目小于这个值，就不再释放表空间，必须为整数，最小值为 1，最大值为 256，缺省为 1；
4. 填充比例 FILLFACTOR：指定插入数据时数据页的充满程度，取值范围从 0 到 100。默认值为 0，等价于 100，表示全满填充，未充满的空间可供页内的数据更新时使用。插入数据时填充比例的值越低，可由新数据使用的空间就越多；更新数据时填充比例的值越大，更新导致出现的页分裂的几率越大；
5. 表空间名：在指定的表空间上建表或索引，表空间必须已存在，默认为用户缺省的表空间。

例 表 PERSON 建立在表空间 TS_PERSON 中，初始簇大小为 5，最小保留簇数目为 5，下次分配簇数目为 2，填充比例为 85。

```
CREATE TABLE PERSON.PERSON
( PERSONID  INT IDENTITY(1,1) CLUSTER PRIMARY KEY,
SEX CHAR(1) NOT NULL,
NAME VARCHAR(50) NOT NULL,
EMAIL VARCHAR(50),
PHONE VARCHAR(25))
STORAGE
      ( INITIAL     5,  MINEXTENTS    5,
NEXT      2,  ON    TS_PERSON,    FILLFACTOR     85);
```

也可以在分区表上指定某个分区的存储参数，如下面的建表语句指定了 PAR2 分区存储在 TS_PAR2 表空间上。

```
CREATE TABLE PARTITION_TABLE
(C1 INT,
C2 INT)
PARTITION BY RANGE(C1)
 (PARTITION PAR1 VALUES LESS THAN(5),
PARTITION PAR2 VALUES LESS THAN(100) STORAGE (ON TS_PAR2));
```

8.1.2 堆表

对于堆表可以指定并发分支 BRANCH 和非并发分支 NOBRANCH 的数目，其范围是（1=<BRANCH <= 64，1<=NOBRANCH<=64），堆表最多支持 128 个链表，具体可参考第 17 章 管理堆表。

例 下例创建的 LIST_TABLE 表有并发分支 2 个，非并发分支 4 个。

```
CREATE TABLE LIST_TABLE(C1 INT) STORAGE(BRANCH (2,4));
```

8.1.3 HUGE 表

HUGE 表需要建立在混合表空间上。建立 HUGE 表如果不使用默认的混合表空间 MAIN，则必须要先创建一个混合表空间。除去需要建立在混合表空间之上的要求外，关于 HUGE 表的更多信息请参考第 16 章 管理列存储表。

例 建立一个名称 TS1 的混合表空间，指定 HUGE 数据文件路径为 d:\dmdbms\data\DAMENG\TS1\HUGE1。

```
CREATE TABLESPACE TS1 DATAFILE 'd:\dmdbms\data\DAMENG\TS1\TS1.dbf' SIZE 128 WITH HUGE PATH 'd:\dmdbms\data\DAMENG\TS1\HUGE1';
```

对于 HUGE 表可以指定如下参数：

1. 区大小（一个区的数据行数）。区大小可以通过设置表的存储属性来指定，区的大小必须是 2 的 n 次方，如果不是则向上对齐。取值范围：1024 行~1024*1024 行。默认值为 65536 行。
2. 是否记录区统计信息，即在修改时是否做数据的统计。
3. 所属的表空间。创建 HUGE 表，需要通过存储属性指定其所在的表空间，不指定则存储于默认表空间 MAIN 中。HUGE 表指定的表空间只能是混合表空间。
4. 文件大小。创建 HUGE 表时还可以指定单个文件的大小，通过表的存储属性来指定，取值范围为 16M~1024*1024MB。不指定则默认为 64MB。文件大小必须是 2 的 n 次方，如果不是则向上对齐。
5. 日志属性。1）LOG NONE：不做镜像；2）LOG LAST：做部分镜像；3）LOG ALL：全部做镜像。

例如下面的建表语句：STUDENT 表的区大小为 65536 行，文件大小为 64MB，指定所在的混合表空间为 TS1，做完整镜像，S_COMMENT 列指定的区大小为不做统计信息，其它列（默认）都做统计信息。

```
CREATE HUGE TABLE STUDENT
(
    s_no                INT,
    s_class             VARCHAR,
    s_comment               VARCHAR(79) STORAGE(stat none)
)STORAGE(SECTION(65536) ,FILESIZE(64), on TS1) LOG ALL;
```

8.2 收回多余的空间

DM8 中表和索引对象的所占用的簇要么是全满的状态要么是半满的状态, 空闲的簇会被系统自动回收。

8.3 用户和表上的空间限制

8.3.1 用户的空间限制

用户占用的空间是其下所有用户表对象占用空间的总和。可以通过设置表空间配额来限制用户使用的表空间大小，当用户执行创建表、创建索引或者插入更新数据等操作时，如果超过了指定的表空间配额，会报超出配额的错误。如创建用户 TEST_USER 时可指定该用户可以使用的 TEST_TABLESPACE 表空间大小为 50MB。

```
CREATE USER TEST_USER IDENTIFIED BY TEST_password_123 QUOTA 50M ON TEST_TABLESPACE;
```

对用户的表空间配额限制也可进行更改，如修改用户 TEST_USER 可以使用的 TEST_TABLESPACE 表空间大小为 100M。

```
ALTER USER TEST_USER QUOTA 100M ON TEST_TABLESPACE;
```

8.3.2 表对象的空间限制

表对象占用的空间是其上所有索引占用空间的总和。可以限制表对象使用的磁盘空间大小，当在表对象上创建索引或者插入更新数据超过了指定的空间限制时，会报空间不足的错误。如创建表 TEST 时可指定该表对象可使用的最大磁盘空间为 500M。

```
CREATE TABLE TEST (SNO INT, MYINFO VARCHAR) DISKSPACE LIMIT 500;
```

对表对象空间的限制也可进行更改，如修改表 TEST 的磁盘空间限制为 50M。

```
ALTER TABLE TEST MODIFY DISKSPACE LIMIT 50;
```

8.4 查看模式对象的空间使用

8.4.1 查看用户占用的空间

可以使用系统函数 USER_USED_SPACE 得到用户占用空间的大小，函数参数为用户名，返回值为占用的页的数目。

```
SELECT USER_USED_SPACE('TEST_USER');
```

8.4.2 查看表占用的空间

可以使用系统函数 TABLE_USED_SPACE 得到表对象占用空间的大小，函数参数为模式名和表名，返回值为占用的页的数目。

```
SELECT TABLE_USED_SPACE('SYSDBA', 'TEST');
```

8.4.3 查看表使用的页数

可以使用系统函数 TABLE_USED_PAGES 得到表对象实际使用页的数目，函数参数为模式名和表名，返回值为实际使用页的数目。

```
SELECT TABLE_USED_PAGES('SYSDBA', 'TEST');
```

8.4.4 查看表空闲堆栈占用的空间

可以使用系统函数 TABLE_FREE_PAGE_STACK_USED_SPACE 得到表空闲页堆栈占用空间的大小，函数参数为模式名和表名，返回值为占用的页的数目。

```
SELECT TABLE_FREE_PAGE_STACK_USED_SPACE ('SYSDBA', 'TEST');
```

8.4.5 查看索引占用的空间

可以使用系统函数 INDEX_USED_SPACE 得到索引占用空间的大小，函数参数为索引 ID，返回值为占用的页的数目。

```
SELECT INDEX_USED_SPACE(33555463);
```

8.4.6 查看索引使用的页数

可以使用系统函数 INDEX_USED_PAGES 得到索引实际使用页的数目，函数参数为索引 ID，返回值为实际使用页的数目。

```
SELECT INDEX_USED_PAGES(33555463);
```

8.5 数据类型的空间使用

各种数据类型占用的空间是不同的，下表 8.1 列出了主要数据类型所需要的空间。

表 8.1 主要数据类型所需的空间

数据类型	所需空间
CHAR	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制
VARCHAR	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制

TINYINT BIT BYTE	需要 1 个字节
SMALLINT	需要 2 个字节
INT	需要 4 个字节
BIGINT	需要 8 个字节
REAL	需要 4 个字节
FLOAT	需要 8 个字节
DOUBLE DOUBLE PRECISION	需要 8 个字节
DEC DECIMAL NUMERIC	SIZE 为 1~22 个字节
BINARY	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制
VARBINARY	SIZE 为 1~8188 字节， 具体情况受到页面大小和记录大小的共同限制
DATE	需要 3 个字节
TIME	需要 5 个字节
TIMESTAMP DATETIME	需要 8 个字节
TIME WITH TIME ZONE	需要 7 个字节
TIMESTAMP WITH TIME ZONE	需要 10 个字节

INTERVAL YEAR INTERVAL MONTH INTERVAL YEAR TO MONTH	需要 12 个字节
INTERVAL DAY INTERVAL HOUR INTERVAL MINUTE INTERVAL SECOND INTERVAL DAY TO HOUR INTERVAL DAY TO MINUTE INTERVAL DAY TO SECOND INTERVAL HOUR TO MINUTE INTERVAL HOUR TO SECOND INTERVAL MINUTE TO SECOND	需要 24 个字节
BLOB IMAGE LONGVARBINARY	SIZE 为 1~(100G-1)字节
CLOB TEXT LONGVARCHAR	SIZE 为 1~(100G-1)字节

第9章 管理表

表是数据库中数据存储的基本单元，是对用户数据进行读和操纵的逻辑实体。表由列和行组成，每一行代表一个单独的记录。表中包含一组固定的列，表中的列描述该表所跟踪的实体的属性，每个列都有一个名字及各自的特性。

列的特性由两部分组成: 数据类型（dataType）和长度（length）。对于 NUMERIC、DECIMAL 以及那些包含秒的时间间隔类型来说，可以指定列的小数位及精度特性。在 DM 系统中，CHAR、CHARACTER、VARCHAR 数据类型的最大长度由数据库页面大小决定，数据库页面大小在初始化数据库时指定。DM 系统具有 SQL-92 的绝大部分数据类型，以及部分 SQL-99、Oracle 和 SQL Server 的数据类型。

为了确保数据库中数据的一致性和完整性，在创建表时可以定义表的实体完整性、域完整性和参考完整性。实体完整性定义表中的所有行能唯一地标识，一般用主键、唯一索引、UNIQUE 关键字及 IDENTITY 属性来定义；域完整性通常指数据的有效性，限制数据类型、缺省值、规则、约束、是否可以为空等条件，域完整性可以确保不会输入无效的值；参考完整性维护表间数据的有效性、完整性，通常通过建立外键联系另一表的主键来实现。

如果用户在创建表时没有定义表的完整性和一致性约束条件，那么用户可以利用 DM 所提供的基表修改语句来进行补充或修改。DM 系统提供基表修改语句，可对基表的结构进行全面的修改，包括修改基表名、列名、增加列、删除列、修改列类型、增加表级约束、删除表级约束、设置列缺省值、设置触发器状态等一系列修改功能。

本章描述管理表的几个方面，包括以下内容:

1. 管理表的准则;
2. 创建表;
3. 修改表;

4. 删除表;
5. 清空表;
6. 查看表信息。

9.1 管理表的准则

9.1.1 设计表

表是数据库设计过程中的基本构件，基于来自应用开发者的有关应用如何运作和所期望的数据类型，数据库管理员应与应用开发者一起工作，并认真规划每个表，具体需要做到以下几点:

1. 规范化表，估算并校正表结构，使数据冗余达到最小;
2. 为每个列选择合适的数据类型，是否允许为空等，并根据实际情况判断是否需要对列进行加密或压缩处理;
3. 建立合适的完整性约束，管理约束可查看 13.4 管理完整性约束的内容;
4. 建立合适的聚集索引。每个表（列存储表，堆表除外）都含一个聚集索引，默认以 ROWID 建立，而建立合适的聚集索引，可以有效加快表的检索效率;
5. 根据实际需要，建立合适类型的表。DM 支持的表类型包括普通表、临时表、水平分区表、堆表和列存储表。本章只介绍普通表和临时表，其他类型表将在其他章节中重点介绍。

9.1.2 指定表的存储空间上限

在创建表时指定 SPACE LIMIT 子句，可以对表的存储空间指定上限。DM 支持对表的存储空间指定大小，单位是 MB，即表的大小可由管理员指定，便于表的规模管理。当表的所有索引所占用的存储空间超过指定大小时，表将不能再新增数据。

9.1.3 指定表的存储位置

创建表时，在 STORAGE 子句中，可对表指定存储的表空间。如果没有指定，则该表将创建在用户的默认表空间中。

在创建表时指定合适的表空间，有以下优点：

1. 提高数据库系统的性能，因为不同的数据库表可能对应不同的数据文件，可减少对相同文件的竞争；
2. 减少数据库管理的时间，数据库表分布在不同的表空间中，即使一个表空间损坏，也不影响其他表空间上数据库表的正常访问。

9.2 创建表

9.2.1 创建普通表

如果要在所属模式中创建新表，需要有 CREATE TABLE 数据库权限；而要在其他用户的模式中创建新表，则需要有 CREATE ANY TABLE 数据库权限。创建表时，应当为表指定一个表空间，否则，表将在 MAIN 创建。下面给出一个创建一个简单表的例子。

```
CREATE  TABLE  employee (
empno   INT                 PRIMARY KEY,
ename   VARCHAR(15)         NOT NULL,
job     VARCHAR(10),
mgr             INT
CONSTRAINT emp_fkey REFERENCES employee(empno),
hiredate  DATE              DEFAULT (curdate),
salary          FLOAT,
deptno  TINYINT             NOT NULL
```

```
CONSTRAINT dept_fkey REFERENCES dept(deptno))
STORAGE (
INITIAL          50,
NEXT             50,
MINEXTENTS       10,
FILLFACTOR       80,
ON               users);
```

在上述 CREATE TABLE 语句中，在 users 表空间上建立了 employee 表，并有几个完整性约束，其中包含定义在不同列上的一个主键和外键。约束将在 15 章中讨论。

创建表之后，可以使用 INSERT 命令插入数据或使用达梦数据导入导出工具装载数据，还可以直接使用 CREATE TABLE AS SELECT 创建一个表。

9.2.2 指定表的聚集索引

表（列存储表和堆表除外）都是使用 B 树(以下简称 B 树)索引结构管理的，每一个普通表都有一个聚集索引，数据通过聚集索引键排序，根据聚集索引键可以快速查询任何记录。

当建表语句未指定聚集索引键, DM 的默认聚集索引键是 ROWID, 即记录默认以 ROWID 在页面中排序。ROWID 是 B 树为记录生成的逻辑递增序号，表上不同记录的 ROWID 是不一样的，并且最新插入的记录 ROWID 最大。很多情况下，以 ROWID 建的默认聚集索引并不能提高查询速度，因为实际情况下很少人根据 ROWID 来查找数据。

因此，DM 提供三种方式供用户指定聚集索引键:

1. CLUSTER PRIMARY KEY: 指定列为聚集索引键, 并同时指定为主键, 称为聚集主键;
2. CLUSTER KEY：指定列为聚集索引键，但是是非唯一的;
3. CLUSTER UNIQUE KEY：指定列为聚集索引键，并且是唯一的。

例 创建 student 表，指定 stu_no 为聚集主键。

```
CREATE  TABLE  STUDENT(
        STUNO       INT                 CLUSTER PRIMARY KEY,
        STUNAME     VARCHAR(15)         NOT NULL,
        TEANO       INT,
        CLASSID     INT
);
```

指定聚集索引键后，如果查询条件中含有聚集索引键，可以定位记录在 B 树上的位置，使查询性能大大提高。然而，插入记录也需要根据聚集索引键定位插入位置，有可能导致页面的分裂而影响插入性能。

在 dm.ini 配置文件中，可以通过指定 PK_WITH_CLUSTER 使表中的主键自动转化为聚集主键。默认情况下，PK_WITH_CLUSTER 为 0，即建表时指定的主键不会自动转化为聚集主键；若为 1，则主键自动变为聚集主键。PK_WITH_CLUSTER 对水平分区表、列存储表和堆表无效。

9.2.3 指定表的填充因子

上文提到，每个普通表都含有一个聚集索引，指定表的填充因子，即指定聚集索引的填充因子。索引的填充因子指在新建和重组索引时，页面记录存储空间占页面总大小的百分比。而这部分预留空间是为更新字段时使用的，一个有效的填充因子可以大大减少由更新记录导致的页面拆分。

例如，上述的创建 employee 表语句中的 STORAGE 子句，指定了 FILLFACTOR 为 80，即填充因子为 80%。

当填充因子取值低，则需要更多的页来存储数据，因而读取范围大，会影响性能。而当填充因子取值高，则在更新数据时可能造成大量的页拆分，页拆分需要消耗较多 CPU 和 I/O 资源，同样会影响性能。原则上，在只读表上应该设置填充因子为较高的值，而有大量更新

的表上应该设置较低的值。默认情况下，DM 新建的表和索引的填充因子是 100，可根据实际情况设置合适的填充因子大小。

9.2.4 查询建表

为了创建一个与已有表相同的新表，或者为了创建一个只包含另一个表的一些行和列的新表，可以使用 CREATE TABLE AS SELECT(CTAS)命令。使用该命令，可以通过使用 WHERE 条件将已有表中的一部分数据装载到一个新表中，或者可以通过 SELECT * FROM 子句将已有表的所有数据装载到创建的表中，如以下程序片段所示:

```
CREATE TABLE NEW_EMP
AS
SELECT * FROM EMPLOYEE;
```

如果用户通过单表的全表查询进行建表操作，因 INI 参数 CTAB_SEL_WITH_CONS 默认为 0，若源列是显式指定了 NOT NULL 约束的非主键列，或者源列是自增列，则拷贝源列的非空信息；否则都不拷贝其非空信息。

用户可以通过将 INI 参数 CTAB_SEL_WITH_CONS 置为 1 进行原始表上约束的拷贝，列上能拷贝的约束包括默认值属性、自增属性、非空属性以及加密属性，表上能拷贝的约束包括唯一约束、PK 约束以及 CHECK 约束。如果拷贝的唯一约束和 PK 约束不是聚集索引键，则不拷贝聚集索引；若源表包含虚拟列，则将虚拟列变为普通列。需要注意的是，该参数取值为 1 的情况不支持 HUGE 表。

用户也可将 CTAB_SEL_WITH_CONS 置为 2 进行原始表上表结构、分区信息、存储信息以及表约束的拷贝（仅限 huge 表与 huge 表之间或行表与行表之间进行拷贝），其中各列属性完全拷贝，表约束包括唯一约束、PK 约束以及 CHECK 约束，并拷贝聚集索引和系统创建的实索引，若源表包含虚拟列，则拷贝此虚拟列；若源表是系统表，且存储在 SYSTEM 表空间，则目标表调整为使用默认表空间 MAIN。需要注意，只有在全表查询时才会进行表

约束和相关信息的拷贝，需保证查询项与源表列顺序与列名完全一致。

对于查询建表列非空属性的处理规则如下：

1. 计算列、集合查询列允许为 NULL；

2. 若源列允许为 NULL，则新表列允许为 NULL；

3. 源列具有 NOT NULL 属性，需要考虑外连接和过滤条件，规则如下：

1） 源列作为 LEFT JOIN 左表列/RIGHT JOIN 右表列时，新表列为 NOT NULL；

2） 源列作为 LEFT JOIN 右表列/RIGHT JOIN 左表列/FULL JOIN 表列时，若存在 IS NOT NULL 过滤条件，则新表列为 NOT NULL；否则新表列允许为 NULL。

DM 也支持在查询建表时指定列定义，但在此场景下有诸多限制。对于查询建表指定列定义的规则如下：

1. 查询建表的目标表的后面括号中可以指定新表的列名、列的 DEFAULT 值、列约束、表约束。列名数需要与源表列数完全一致；列的 DEFAULT 值和列约束需紧跟在列名后定义，DEFAULT 与列约束的顺序可互换；

2. DEFAULT 值与 DEFAULT 子句不同，不支持 ON NULL 选项；

3. 列约束包含[NOT] NULL、NOT VISIBLE、unique、primary key、check，不支持外键约束和聚集索引；

4. 表约束包含 unique、primary key、check，不支持外键约束和聚集索引；

5. 查询建表指定列定义时，不支持虚拟列、指定列数据类型、自增列等上述未描述过的语法。此外，当查询建表指定列定义时，忽略 CTAB_SEL_WITH_CONS，即仅根据查询建表列定义建表，源表中的列约束（包括 CTAB_SEL_WITH_CONS=0 时拷贝显式 NOT NULL）、表约束均不拷贝。

如表 T 有两列 C1、C2，均为 int 列，则可以指定列定义进行查询建表，为新表指定列定义，列名调整为 ID1、ID2，在 ID1 列通过列定义创建 CHECK 约束并指定 default 值，在 ID2

列通过列定义指定 NOT NULL，并通过表定义指定 PK 约束，如以下程序片段所示：

```
CREATE TABLE NEW_T(ID1 DEFAULT 1 CHECK(ID1 > 0), ID2 NOT NULL, PRIMARY KEY(ID2))

AS

SELECT * FROM T;
```

9.2.5 创建临时表

当处理复杂的查询或事务时，由于在数据写入永久表之前需要暂时存储一些行信息或需要保存查询的中间结果，可能需要一些表来临时存储这些数据。DM 允许创建临时表来保存会话甚至事务中的数据。在会话或事务结束时，这些表上的数据将会被自动清除。

临时表中的数据不能像在其它永久表中的数据那样进行备份，当事务结束或会话断开时，数据就会被清空。在临时表创建过程中，不会像永久表和索引那样自动分配数据段，而是仅当第一次执行 DML 语句时，才会为临时表在临时表空间中分配空间。并且，对于不同的会话，临时表上的数据是独享的，不会互相干扰，即会话 A 不能访问会话 B 临时表上的数据。

对复杂查询的传统响应方式之一是使用一个视图，使复杂查询更易于操作。但是，视图在每次访问时都需要执行，因而大大降低了性能。而通过 AS SELECT 子句建立的临时表是将复杂查询的结果通过临时 B 树记录了下来，下次访问不用重新执行查询就可以获得数据，并且会话或事务结束后数据将自动删除，是复杂查询的一个优秀的解决方案，且提高了性能。

DM 临时表支持以下功能：

1. 在临时表中，会话可以像普通永久表一样更新、插入和删除数据；
2. 临时表的 DML 操作产生较少的 REDO 日志；
3. 临时表支持建索引，以提高查询性能；
4. 在一个会话或事务结束后，数据将自动从临时表中删除；
5. 不同用户可以访问相同的临时表，每个用户只能看到自己的数据；
6. 临时表的数据量很少，意味着更高效的查询效率；

7. 临时表的表结构在数据删除后仍然存在，便于以后的使用；

8. 临时表的权限管理跟普通表一致。

临时表 ON COMMIT 关键词指定表中的数据是事务级还是会话级的，默认情况下是事务级的。

1. ON COMMIT DELETE ROWS：指定临时表是事务级的，每次事务提交或回滚之后，表中所有数据都被删除；

2. ON COMMIT PRESERVE ROWS：指定临时表是会话级的，会话结束时才清空表，并释放临时 B 树。

例 创建一个事务级的临时表

```
CREATE GLOBAL TEMPORARY TABLE TMP_EMP(
        EMPNO   INT          PRIMARY KEY,
        ENAME   VARCHAR(15)      NOT NULL,
        JOB     VARCHAR(10))
ON COMMIT DELETE ROWS;
```

9.3 修改表

想更改的表如果在所属的模式中，用户必须具有 ALTER TABLE 数据库权限；若在其他模式中，用户必须有 ALTER ANY TABLE 的数据库权限。

通过修改表，用户可以对数据库中的表作如下修改:

1. 添加或删除列，或修改现有的列的定义（列名、数据类型、长度、默认值）。其中，对于添加列，当设置 INI 参数 ALTER_TABLE_OPT 为 1 时，添加列采用查询插入实现，可能会导致 ROWID 的改变；ALTER_TABLE_OPT 为 2 时，系统开启快速加列功能，对于没有默认值或者默认值为 NULL 的新列，系统内部会标记为附加列，能够达到瞬间加列的效果，此时记录 ROWID 不会改变，若有默认值且默认值不为 NULL，

则默认值的存储长度不能超过 4000 字节，此时仍旧采取查询插入实现；ALTER_TABLE_OPT 为 3 时，系统会开启快速加列功能，允许指定新增列的默认值，系统会为该列设置附加列标记，查询表中已存在的数据时，会自动为记录设置追加列默认值，此时记录 ROWID 不会改变。特殊的是，对于 HUGE 表的修改，参数 ALTER_TABLE_OPT 设置为 0 或 1 时效果一样，均为采用查询插入实现；

2. 添加、修改或删除与表相关的完整性约束；
3. 重命名一个表；
4. 启动或停用与表相关的完整性约束；
5. 启动或停用与表相关的触发器；
6. 修改表的 SPACE LIMIT；
7. 增删自增列。

9.4 删除表

当一个表不再使用时，可以将其删除。删除表时，将产生以下结果：

1. 表的结构信息从数据字典中删除，表中的数据不可访问；
2. 表上的所有索引和触发器被一起清除；
3. 所有建立在该表上的同义词、视图和存储过程变为无效；
4. 所有分配给表的簇标记为空闲，可被分配给其他的数据库对象。

一般情况下，普通用户只能删除自己模式下的表。若要删除其他模式下的表，则必须具有 DROP ANY TABLE 数据库权限。

以下语句可删除 employee 表：

```
DROP TABLE employee;
```

删除不存在的表会报错。若指定 IF EXISTS 关键字，删除不存在的表，不会报错，如：

```
DROP TABLE IF EXISTS employee;
```

如果要删除的表被其他表引用，即其他表的外键引用了表的任何主键或唯一键，则需要在 DROP TABLE 语句中包含 CASCADE 选项，如：

```
DROP TABLE employee CASCADE;
```

9.5 清空表

有些情况下，当表的数据不再使用时，需要删除表的所有行，即清空该表。DM8 支持以下方式来删除表中的所有的行：

1. 使用 DELETE 语句；
2. 使用 DROP 和 CREATE 语句；
3. 使用 TRUNCATE 语句。

9.5.1 使用 DELETE

使用 DELETE 语句能删除表中的行。

例 下面的语句删除 employee 表中的所有行

```
DELETE FROM employee;
```

但是，使用 DELETE 清空表，当表有很多行时，会消耗很多系统资源。因为，DELETE 操作需要 CPU 时间，并且会产生大量的 REDO 日志和 UNDO 记录。另外，如果表上关联了元组级触发器，每删除一行，就会启动一次触发器。这都需要大量的系统资源。

9.5.2 使用 DROP 和 CREATE

使用 DROP 删除一个表，然后创建一个同名的表，也可以达到清空表的效果。

例 下面的语句先删除 employee 表，之后再重新创建

```
DROP TABLE employee;

CREATE TABLE employee(…);
```

当删除和重新创建表时，所有与之相关联的索引、完整性约束和触发器也被删除。同样，所有针对被删除表的授权也会被删除。

9.5.3 使用 TRUNCATE

使用 TRUNCATE 语句能删除表中的所有行。

例 下面的语句清空 employee 表

```
TRUNCATE TABLE EMPLOYEE;
```

TRUNCATE 语句为我们提供了一种快速、有效地删除表所有行的方法。并且 TRUNCATE 是一个 DDL 语句，不会产生任何回滚信息。执行 TRUNCATE 会立即提交，而且不能回滚。

TRUNCATE 语句并不影响与被删除的表相关联的任何结构、约束、触发器或者授权。 另外，DM 数据库 TRUNCATE 表后，原来分配给该表的空间会被释放，供其他数据库对象使用，大大提高空间的利用效率。

一般情况下，普通用户只能 TRUNCATE 自己模式下的表。若要 TRUNCATE 其他模式下的表，则必须具有 DROP ANY TABLE 数据库权限。

如果要清空的表被其他表引用，即其他表的外键引用了表的任何主键或唯一键，并且子表不为空或子表的外键约束未被禁用，则不能 TRUNCATE 该表。

9.6 查看表信息

9.6.1 查看表定义

创建表后，可以通过 SP_TABLEDEF 系统过程查看表的定义。

例 下面的语句查看 SYSDBA 模式下的 EMPLOYEE 表的定义语句

```
CALL SP_TABLEDEF('SYSDBA', 'EMPLOYEE');
```

DM 通过提供的 SP_TABLEDEF 系统过程来显示当前表的定义。当表多次进行 ALTER

TABLE 后，显示的表定义将是最后一次修改后的建表语句。关于 SP_TABLEDEF 系统过程的详细介绍请参考《DM8_SQL 语言使用手册》。

9.6.2 查看自增列信息

DM 支持 INT、BIGINT 和 DEC(n, 0)三种数据类型的 IDENTITY 自增列，并提供以下函数查看表上自增列的当前值、种子和增量等信息：

1. IDENT_CURRENT：获得表上自增列的当前值；
2. IDENT_SEED：获得表上自增列的种子信息；
3. IDENT_INCR：获得表上自增列的增量信息。

```
CREATE TABLE IDENT_TABLE (
    C1              INT         IDENTITY(100, 100),
    C2              INT
);
SELECT ident_current('SYSDBA.IDENT_TABLE');
SELECT ident_seed('SYSDBA.IDENT_TABLE');
SELECT ident_incr('SYSDBA.IDENT_TABLE');
```

9.6.3 查看表的空间使用情况

DM 使用段、簇和页实现数据的物理组织。DM 支持查看表的空间使用情况，包括：

1. TABLE_USED_SPACE：已分配给表的页面数；
2. TABLE_USED_PAGES：表已使用的页面数；
3. TABLE_FREE_PAGE_STACK_USED_SPACE：表空闲页堆栈拥有的页面数。

```
CREATE TABLE SPACE_TABLE (
    C1              INT,
```

```
    C2              INT
);
SELECT table_used_space('SYSDBA','SPACE_TABLE');
SELECT table_used_pages('SYSDBA','SPACE_TABLE');
SELECT table_free_page_stack_used_space ('SYSDBA','SPACE_TABLE');
```

第10章 管理索引

索引是为了快速检索和定位数据行而创建的一种数据结构。索引是由表中索引列数据进行排序后的集合和指向这些值的物理标识（例如：ROWID 等聚集索引键）共同组成。在 DM 中，除了位图索引、位图连接索引、全文索引外，索引数据都采用 B+树结构进行存储，在 DM 手册的其余地方都简称为 B 树。索引和表一样需要存储空间。

索引相当于一本书的目录，根据目录中的页码标识快速检索并定位到的查找内容。

索引可使 SQL 语句快速地定位到相关数据记录上。除了系统自动创建的聚集索引以外，其它索引均为可选项。

创建或删除一个索引，不会影响基表、数据库应用或其它索引。当插入、更改和删除相关表的行时，DM8 会自动管理索引。如果删除索引，所有的应用仍继续工作，但访问以前被索引了的数据时速度可能会变慢。

10.1 索引分类

可以从五个角度对索引进行分类，下面分别进行介绍。

10.1.1 聚集索引和非聚集索引

从物理存储角度进行分类，可分为聚集索引和非聚集索引。

- 聚集索引（又称为一级索引、主索引）：聚集索引就是按照聚集索引键构造一棵 B 树，表数据存储在 B 树叶子节点上，通过定位索引可直接在 B 树中找到数据。每一个表有且只有一个聚集索引。
- 非聚集索引（又称为二级索引、辅助索引）：将二级索引列和聚集索引列共同存储在 B 树叶子节点上。如果查找非聚集索引键值或聚集索引键值可直接在 B 树中找到；如果查找索引键值以外的数据，则需要回到一级索引中进行查找。每一个表可以有多个非聚集索引。

下面用一个具体的示例展示聚集索引和非聚集索引的用法。检索过程如图 10.1 所示。

```
//创建表格 T1，并插入数据。
CREATE TABLE T1 (ID int,NAME varchar(20),DEPARTMENT varchar(20));
INSERT INTO T1 VALUES(1, 'Zhang San', 'A 部');
INSERT INTO T1 VALUES(2, 'Li Si', 'B 部');
INSERT INTO T1 VALUES(3, 'Wang Wu', 'C 部');
INSERT INTO T1 VALUES(4, 'Chen Liu', 'D 部');
//创建聚集索引，索引键为 ID。
CREATE CLUSTER INDEX C1 on T1(ID);
//创建非聚集索引，索引键为 NAME。
CREATE INDEX S1 on T1(NAME);
//使用聚集索引进行查找
EXPLAIN SELECT * FROM  T1 WHERE ID=2;
1   #NSET2: [1, 1, 112]
2     #PRJT2: [1, 1, 112]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3       #CSEK2: [1, 1, 112]; scan_type(ASC), C1(T1), scan_range[2,2]
//使用非聚集索引进行查找
EXPLAIN SELECT ID FROM  T1 WHERE NAME='Zhang San';
1   #NSET2: [1, 1, 64]
2     #PRJT2: [1, 1, 64]; exp_num(2), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3       #SSEK2: [1, 1, 64]; scan_type(ASC), S1(T1), scan_range['Zhang San','Zhang San'],
is_global(0)
//先使用非聚集索引进行查找，再通过非聚集索引关联上聚集索引进行二次查找
EXPLAIN SELECT DEPARTMENT FROM  T1 WHERE NAME='Chen Liu';
```

```
1    #NSET2: [1, 1, 96]
2        #PRJT2: [1, 1, 96]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3            #BLKUP2: [1, 1, 96]; SI(T1); use_clu_addr(0)
4                #SSEK2: [1, 1, 96]; scan_type(ASC), SI(T1), scan_range['Chen Liu','Chen Liu'],
is_global(0)
```

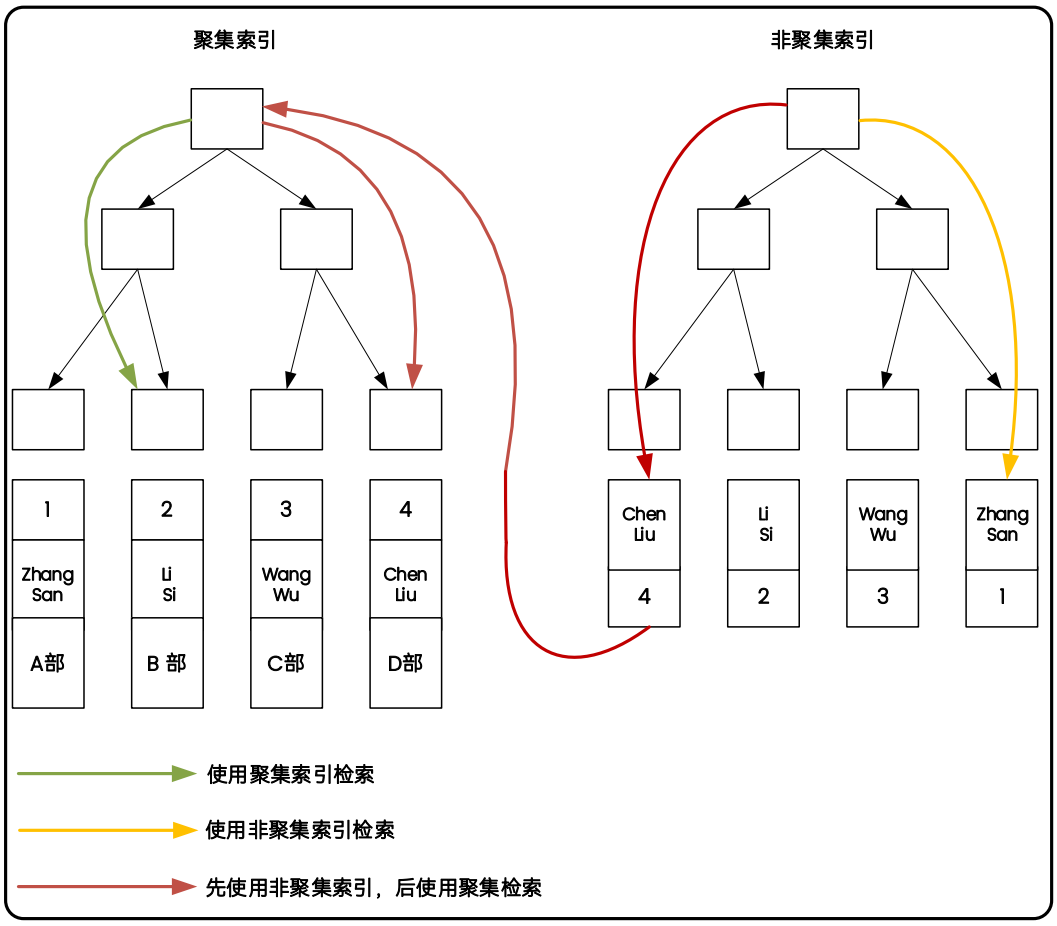


图 10.1 聚集索引和非聚集索引的检索流程

10.1.2 功能索引

从索引功能角度进行分类，可分为唯一索引、函数索引、位图索引、位图连接索引、全文索引、数组索引、普通索引。

- 唯一索引：索引数据根据索引键唯一；
- 函数索引：包含函数/表达式的预先计算的值；

- 位图索引：对低基数的列创建位图索引；
- 位图连接索引：针对两个或者多个表连接的位图索引，主要用于数据仓库中；
- 全文索引：在表的文本列上而建的索引。具体内容请参考第 18 章；
- 数组索引：在一个只包含单个数组成员的对象列上创建的索引；
- 普通索引：除了唯一索引、函数索引、位图索引、位图连接索引、全文索引、数组索引以外的索引，均为普通索引。

10.1.3 虚索引和实索引

从是否存储数据的角度进行分类，可分为虚索引和实索引。

- 实索引：指存储数据的索引。虚索引以外的索引均为实索引。
- 虚索引：指不存储数据的索引。虚索引由系统自动并隐式地创建，目的是保证约束的正确性。

系统在下述情景中会自动创建一个虚索引：

情景 1 为普通表（非 HUGE 表）创建非自引用约束的外键的引用约束且未使用 WITH INDEX 时，系统自动创建一个不含数据的虚索引。

情景 2 为非事务型 HUGE 表创建 PRIMARY KEY 主键约束或 UNIQUE 唯一约束时，系统自动创建一个不含数据的虚索引。此处需要说明一点，系统自动为 UNIQUE 约束创建的 UNIQUE 索引为虚索引，而用户手动通过 CREATE UNIQUE INDEX 语法创建的 UNIQUE 索引为实索引，同为 UNIQUE 索引但二者类型不同。

情景 3 当 DM.INI 参数 HUGE_UNIQUE_CHECK=0，为事务型 HUGE 表创建 PRIMARY KEY 主键约束或 UNIQUE 唯一约束时，系统自动创建一个不含数据的虚索引。

下面用具体的示例展示系统创建的虚索引。

例 1 展示引用约束对应的虚索引。

```
CREATE  TABLE T1 (C1 INT ,C2 INT UNIQUE);
```

```
CREATE  TABLE T2(C1 INT,C2 INT REFERENCES T1(C2));
//查看约束对应的索引 ID(INDEXID)，假设为 33556046
SELECT INDEXID FROM SYSCONS WHERE TABLEID=(SELECT ID FROM SYSOBJECTS WHERE NAME = 'T2');
行号          INDEXID
---------- -----------
1             33556046
//查看该索引是否为虚索引。3 表示 1+2。其中 2 代表虚索引。
SELECT FLAG FROM SYSINDEXES WHERE ID= 33556046;
行号          FLAG
---------- -----------
1             3
```

例 2 展示 HUGE 表主键约束对应的虚索引。

```
//初始化一个 HUGE_WITH_DELTA =0，RLOG_GEN_FOR_HUGE =0 的库。
dminit      path=d:\dmdbms\data      HUGE_WITH_DELTA=0      RLOG_GEN_FOR_HUGE=0
SYSDBA_PWD=DMdba_123 SYSAUDITOR_PWD=DMauditor_123
//连接库后，创建一个包含主键约束的非事务型 HUGE 表。
CREATE HUGE  TABLE T2(C1 INT,C2 INT PRIMARY KEY)STORAGE(WITHOUT DELTA);
//查看约束对应的索引 ID(INDEXID)，假设为 33555472
SELECT INDEXID FROM SYSCONS WHERE TABLEID=(SELECT ID FROM SYSOBJECTS WHERE NAME = 'T2');
行号          INDEXID
---------- -----------
1             33555472
//查看该索引是否为虚索引。7 表示 1+2+4。其中 2 代表虚索引。
SELECT FLAG FROM SYSINDEXES WHERE ID=33555472;
```

```
行号          FLAG

---------- -----------

1             7
```

10.1.4 单列索引和复合索引

从索引键值的个数进行分类，可分为单列索引和复合索引。

- 单列索引：只有一个索引键的索引。
- 复合索引：含有多个索引键的索引。

10.1.5 全局索引和局部索引

从分区的角度进行分类，可分为全局索引和局部索引。全局索引和局部索引均为二级索引，专门用于水平分区表中。

当分区子表个数较多、索引列的选择率较好、没有使用分区裁剪优化等因素时，执行计划会倾向于使用全局索引进行查询。但由于全局索引的日常维护成本高于局部索引。因此用户可根据实际应用场景权衡利弊之后再在全局索引和局部索引中进行选择。

- 全局索引：全局索引是以整张表的数据为对象而建立的索引。

指定 GLOBAL 关键字创建的索引即为全局索引。创建全局索引时，会在水平分区表的主表上创建全局索引和在每个子表上创建全局本地索引。索引数据存储在全局索引上。全局本地索引的命名规则为：INDEX+全局本地索引 ID_全局索引 ID。例如：全局索引 IDX1 的 ID 为 3504，全局本地索引的 ID 为 3506，那么全局本地索引的名称为 INDEX3506_3504。

全局索引具体分为两种：全局非分区索引和全局分区索引。指定了<PARTITION 子句>的索引即为分区索引。只有 DMDPC 功能支持分区索引。

全局非分区索引可用于除 DMDPC 以外的任何环境中。

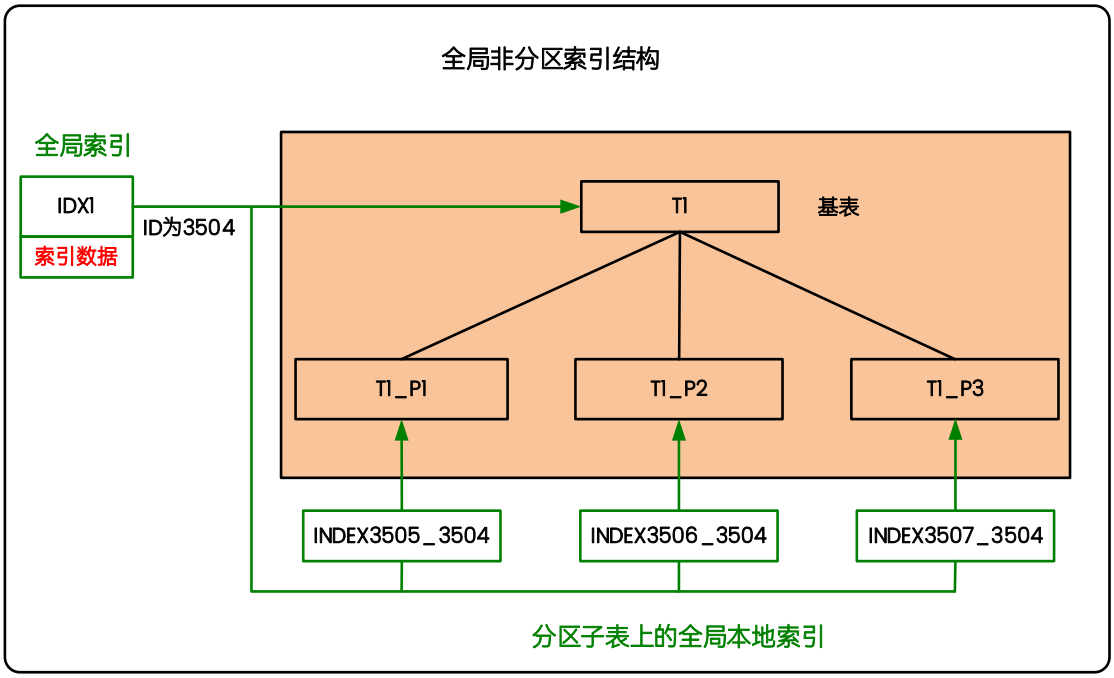


图 10.2 全局非分区索引

例 1 创建全局非分区索引

```
//创建表
drop table t1;
create table t1(c1 int, c2 int, c3 int)  partition by range(c1)
(
    partition p1 values less than(100),
    partition p2 values less than(200),
    partition p3 values less than(maxvalue)
);
//创建全局非分区索引
create index idx1 on t1(c2) global;
//在 WHERE 中指定索引列，可使用索引 idx1 查询
explain select * from t1 where c2 = 130;
1   #NSET2: [1, 1, 24]
```

```
2       #PRJT2: [1, 1, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3         #PARALLEL: [1, 1, 24]; scan_type(FULL), key_num(0, 0, 0), simple(0), range_sfun_opt(0)
4           #BLKUP2: [1, 1, 24]; IDX1(T1); use_clu_addr(0)
5             #SSEK2: [1, 1, 24]; scan_type(ASC), IDX1(T1), scan_range[130,130], is_global(1)
//直接对分区子表查询，则使用全局本地索引
explain select * from t1_p1 where c2 = 130;
1    #NSET2: [1, 1, 24]
2       #PRJT2: [1, 1, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3         #BLKUP2: [1, 1, 24]; INDEX33555521_33555520(T1_P1); use_clu_addr(0)
4                   #SSEK2: [1, 1, 24]; scan_type(ASC), INDEX33555521_33555520(T1_P1),
scan_range[130,130], is_global(1)
```

全局分区索引专门用于 DMDPC 环境中。索引经<PARTITION 子句>分区之后，全局索引便有了分区子索引。索引数据存储在分区子索引上。

分区子索引的命名规则为：全局索引名_索引分区名。例如：全局索引名称为 IDX1，索引分区名为 P1、P2，那么分区子索引名称为 IDX1_P1 和 IDX1_P2。

分区子索引是全局分区索引的组成部分，使用全局索引即可使用到分区子索引。不支持单独使用分区子索引进行查询。

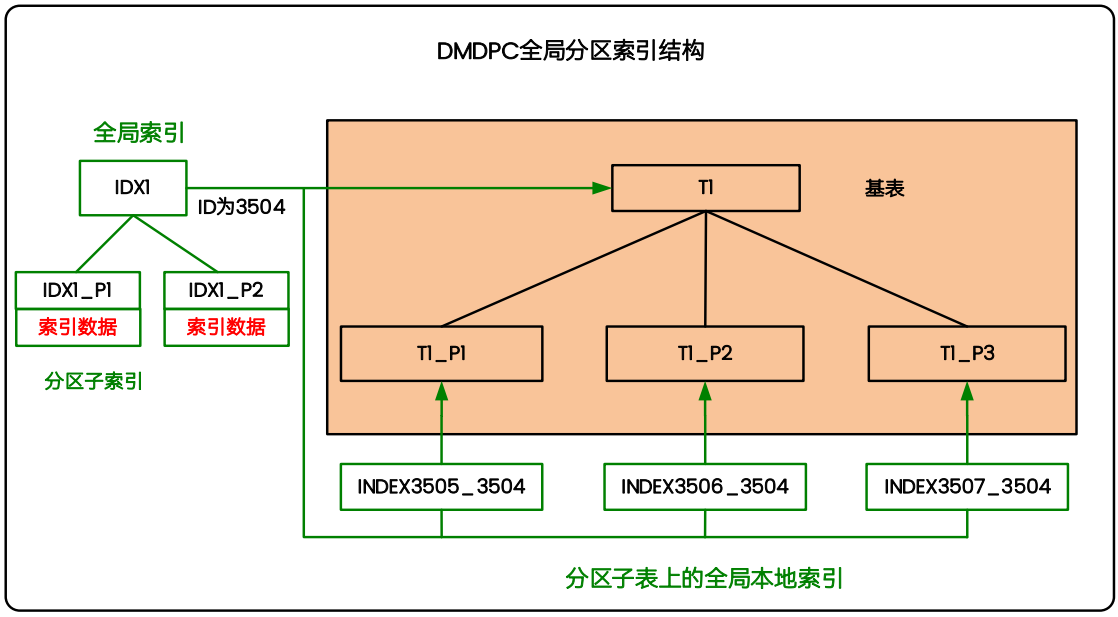


图 10.3 全局分区索引

例 2 在 DMDPC 环境中创建全局分区索引

```
//创建表空间
create tablespace ts1 datafile 'opt/ts/ts01.dbf' size 128 storage (on raft_1);
create tablespace ts2 datafile 'opt/ts/ts02.dbf' size 128 storage (on raft_2);
//创建表
drop table t1;
create table t1(c1 int, c2 int, c3 int)   partition by range(c1)
(
      partition p1 values less than(100),
      partition p2 values less than(200),
      partition p3 values less than(maxvalue)
);
//创建全局分区索引
drop index idx1;
create index idx1 on t1(c2) global partition by range(c2)
```

```
(
  partition p1 values less than(1000) storage(on ts2) ,
  partition p2 values less than(maxvalue) storage(on ts1)
);
//使用索引 idx1 查询
explain select * from t1 where c2 = 130;  //当数据量很大时
explain select * from t1 index idx1 where c2 = 130;  //或直接指定全局索引
1   #NSET2: [2, 1, 24]
2     #ERECV: [2, 1, 24]; stask_no(-1), l_stask_no(1), n_key(0), trig(0)
3       #ESEND: [2, 1, 24]; stask_no(1), type(DIRECT), sites(1:1,2:1), sql_invoke(0), pwj_opt(0),
table(-); INFO_BITS(0x8)
4         #PRJT2: [2, 1, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
5           #BLKUP2: [2, 1, 24]; IDX1(T1); use_clu_addr(0)
6             #ERECV: [2, 1, 24]; stask_no(1), l_stask_no(0), n_key(0), trig(0)
7               #ESEND: [2, 1, 24]; stask_no(0), type(ROWID), sites(2:1), sql_invoke(0),
pwj_opt(0), table(T1) empty_type(ERROR); INFO_BITS(0xc)
8                 #GI: [2, 1, 24]; policy(PART_UNIT), gi_unit[0..0], scan_type[0](EQU)
9                   #SSEK2: [2, 1, 24]; scan_type(ASC), IDX1(T1), scan_range[130,130],
is_global(1)
//直接对分区子表查询，则使用全局本地索引
explain select * from t1_p1 where c2 = 130;
1   #NSET2: [1, 1, 24]
2     #ERECV: [1, 1, 24]; stask_no(-1), l_stask_no(1), n_key(0), trig(0)
3       #ESEND: [1, 1, 24]; stask_no(1), type(DIRECT), sites(1:1), sql_invoke(0), pwj_opt(0),
```

```
table(-); INFO_BITS(0x8)
4            #PRJT2: [1, 1, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
5              #BLKUP2: [1, 1, 24]; INDEX33555508_33555505(T1_P1); use_clu_addr(0)
6                #ERECV: [1, 1, 24]; stask_no(1), l_stask_no(0), n_key(0), trig(0)
7                    #ESEND: [1, 1, 24]; stask_no(0), type(DIRECT), sites(2:1,1:1), sql_invoke(0),
pwj_opt(0), table(-); INFO_BITS(0xc)
8                    #GI: [1, 1, 24]; policy(RANDOM), gi_unit[0..0], scan_type[0](FULL)
9                        #SSEK2: [1, 1, 24]; scan_type(ASC), INDEX33555508_33555505(T1_P1),
scan_range[130,130], is_global(1)
```

- 局部索引：局部索引是在分区表的每个分区上创建的索引。

未指定 GLOBAL 关键字创建的索引即为局部索引。局部索引暂时不支持分区。创建局部索引时，会在水平分区表的主表上创建局部索引和在每个子表上创建一个子表局部索引。索引数据存储在子表局部索引上。

子表局部索引的命名规则为：INDEX+子表局部索引 ID_局部索引 ID。例如：局部索引 IDX1 的 ID 为 3504，子表局部索引的 ID 为 3506，那么子表局部索引的名称为 INDEX3506_3504。

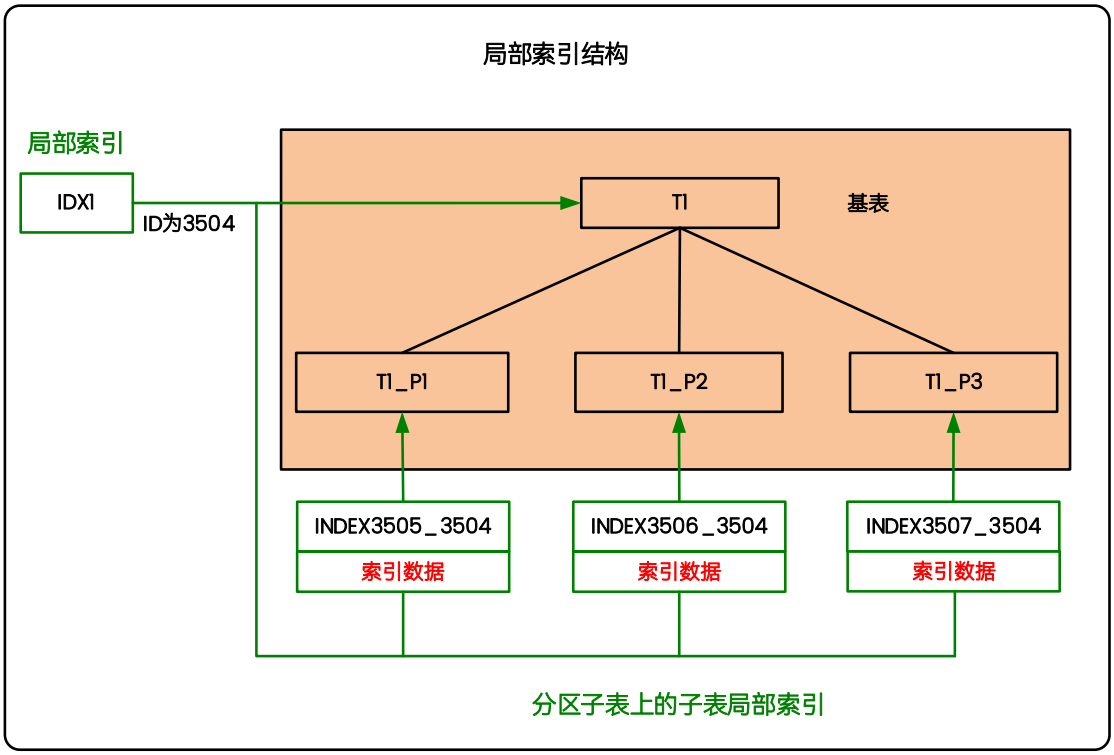


图 10.4 局部索引

例 3 单机环境创建局部索引

```
//创建表
drop table t1;
create table t1(c1 int, c2 int, c3 int)  partition by range(c1)
(
    partition p1 values less than(100),
    partition p2 values less than(200),
    partition p3 values less than(maxvalue)
);
//创建局部索引，必须缺省 GLOBAL 关键字
create index idx1 on t1(c2);
//在 WHERE 中指定索引列，可使用索引 idx1 查询
explain select * from t1 where c2 = 130;
```

```
1    #NSET2: [1, 1, 24]
2        #PRJT2: [1, 1, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3            #PARALLEL: [1, 1, 24]; scan_type(FULL), key_num(0, 0, 0), simple(0), range_sfun_opt(0)
4                #BLKUP2: [1, 1, 24]; IDX1(T1); use_clu_addr(0)
5                    #SSEK2: [1, 1, 24]; scan_type(ASC), IDX1(T1), scan_range[130,130], is_global(0)
//直接对分区子表查询，则使用子表局部索引
explain select * from t1_p1 where c2 = 130;
1    #NSET2: [1, 1, 24]
2        #PRJT2: [1, 1, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3            #BLKUP2: [1, 1, 24]; INDEX33555529_33555528(T1_P1); use_clu_addr(0)
4                        #SSEK2: [1, 1, 24]; scan_type(ASC), INDEX33555529_33555528(T1_P1),
scan_range[130,130], is_global(0)
```

10.2 管理索引的准则

根据准则管理和使用索引，可以大大提高查询效率。

10.2.1 在表中插入数据后创建索引

一般情况下，在插入或装载了数据后，为表创建索引会更加有效率。如果在装载数据之前创建了一个或多个索引，那么在插入每行时 DM8 都必须更改和维护每个索引，使得插入效率降低。

10.2.2 索引正确的表和列

使用下面的准则来决定何时创建索引：

1. 如果需要经常地检索大表中的少量的行，就为查询键创建索引；

2. 为了改善多个表的连接的性能，可为连接列创建索引；

3. 主键和唯一键自动具有索引，在外键上很多情况下也创建索引；

4. 小表不需要索引。

选取表中的索引列时可以考虑以下几点：

1. 列中的值相对比较唯一 ；

2. 取值范围大，适合建立索引；

3. CLOB 和 TEXT 只能建立全文索引、BLOB 不能建立任何索引。

10.2.3 为性能而安排索引列

在 CREATE INDEX 语句中列的排序会影响查询的性能。通常，将最常用的列放在最前面。

如果查询中有多个字段组合定位，则不应为每个字段单独创建索引，而应该创建一个组合索引。当两个或多个字段都是等值查询时，组合索引中各个列的前后关系是无关紧要的。但是如果是非等值查询时，要想有效利用组合索引，则应该按等值字段在前，非等值字段在后的原则创建组合索引，查询时只能利用一个非等值的字段。

10.2.4 限制索引的数量

一个表可以有任意数量的索引。但是，索引越多，修改表数据的开销就越大。当插入或删除行时，表上的所有索引也要被更改；更改一个列时，包含该列的所有索引也要被更改。因此，在从表中检索数据的速度和更新表的速度之间有一个折衷。例如，如果一个表主要仅用于读，则索引多就有好处；如果一个表经常被更新，则索引不宜多建。

10.2.5 估计索引大小和设置存储参数

创建索引之前先估计索引的大小能更好地促进规划和管理磁盘空间。可以用索引以及回滚段、重做日志文件的组合估计的大小来决定支持所期望的数据库所需的磁盘空间的大小。

通过这些估计，就可以购买合适的硬件和做出其他正确的决定。

用单个索引估计的大小能更好地管理索引使用的磁盘空间。创建索引时，可以设置适当的存储参数，并改善使用该索引的应用的 I/O 性能。例如，假设在创建索引之前估计索引的最大大小，之后就可以在创建该索引时设置适当的存储参数，就能很少为表的数据段分配簇。并且，所有的该索引的数据都被保存在相对连续的磁盘空间扇区中，这就减少了使用该索引的磁盘 I/O 操作所需的时间。

10.2.6 为索引指定表空间

可以在除临时表空间、日志表空间和回滚段表空间外的其他任何表空间中创建索引，也可以在其索引的表的相同或不同的表空间中创建索引。如果表及其索引使用相同的表空间 能更方便地对数据库进行管理（如表空间或文件备份）或保证应用的可用性，因为所有有关的数据总是在一起联机。然而，将表及其索引放在不同的表空间（在不同磁盘上）产生的性能比放在相同的表空间更好，因为这样做减少了磁盘竞争。但是将表及其索引放在不同的表空间时，如果一个表上某索引所在的表空间脱机了，则涉及这张表的 SQL 语句可能由于执行计划仍旧需要使用被脱机的索引而不能成功执行。

10.3 创建索引

本节描述如何创建索引。要在用户自己的模式中创建索引，至少要满足如下条件之一:

1. 要被索引的表是在自己的模式中 ；
2. 在要被索引的表上有 CREATE INDEX 权限;
3. 具有 CREATE ANY INDEX 数据库权限。

要在其他模式中创建索引，用户必须具有 CREATE ANY INDEX 数据库权限。

10.3.1 显式地创建索引

可以用 CREATE INDEX 语句显式地创建索引。

例 在 emp 表的 ename 列上创建一个名为 emp_ename 的索引，该索引使用表空间 users

```
CREATE INDEX emp_ename ON emp(ename)
STORAGE (
INITIAL     50,
NEXT        50,
ON          users);
```

上述语句同时为该索引显式地指定了几个存储设置和一个表空间。如果没有给索引指定存储选项，则 INITIAL 和 NEXT 等存储选项会自动使用表空间的默认存储选项。

下面介绍几种常见索引的显式创建方法。

10.3.1.1 创建聚集索引

DM8 中表（列存储表和堆表除外）都是使用 B 树索引结构管理的，每一个普通表都有且仅有一个聚集索引，数据都通过聚集索引键排序，根据聚集索引键可以快速查询任何记录。

当建表语句未指定聚集索引键时，DM8 的默认聚集索引键是 ROWID。若指定索引键，表中数据都会根据指定索引键排序。

建表后，DM8 也可以用创建新聚集索引的方式来重建表数据，并按新的聚集索引排序。

例 可以对 emp 表以 ename 列新建聚集索引

```
CREATE CLUSTER INDEX clu_emp_name ON emp(ename);
```

新建聚集索引会重建这个表以及其所有索引，包括二级索引、函数索引，是一个代价非常大的操作。因此，最好在建表时就确定聚集索引键，或在表中数据比较少时新建聚集索引，

而尽量不要对数据量非常大的表建立聚集索引。

创建聚集索引的约束条件:

1. 每张表中只允许有一个聚集索引，如果之前已经指定过 CLUSTER INDEX 或者指定了 CLUSTER PK，则用户新建立 CLUSTER INDEX 时系统会自动删除原先的聚集索引。但如果新建聚集索引时指定的创建方式（列，顺序）和之前的聚集索引一样，则会报错;
2. 指定 CLUSTER INDEX 操作需要重建表上的所有索引，包括 PK 索引;
3. 删除聚集索引时，缺省以 ROWID 排序，自动重建所有索引;
4. 若聚集索引是默认的 ROWID 索引，不允许删除;
5. 聚集索引不能应用到函数索引中;
6. 不能在列存储表和堆表上新建聚集索引;
7. 建聚集索引语句不能含有 partition_clause 子句;
8. 在临时表上增删索引会使当前会话上临时 b 树数据丢失;
9. 不支持在含有多媒体类型的表上新建聚集索引。在此类表上新建聚集索引时，系统不会直接报错，而是将其创建为非聚集索引。

10.3.1.2 创建唯一索引

索引可以是唯一的或非唯一的。唯一索引可以保证表上不会有两行数据在键列上具有相同的值，非唯一索引不会在键列上施加这个限制。

可用 CREATE UNIQUE INDEX 语句来创建唯一索引。

例 创建一个唯一索引

```
CREATE UNIQUE INDEX dept_unique_index ON dept (dname)
STORAGE (ON users);
```

用户可以在希望的列上定义 UNIQUE 完整性约束, DM8 通过自动地在唯一键上定义一个

唯一索引来保证 UNIQUE 完整性约束。

10.3.1.3 创建基于函数的索引

基于函数的索引促进了限定函数或表达式的返回值的查询，该函数或表达式的值被预先计算出来并存储在索引中。正确使用函数索引，可以带来以下好处：

1. 创建更强有力的分类，例如可以用 UPPER 和 LOWER 函数执行区分大小写的分类；
2. 预先计算出计算密集的函数的值，并在索引中将其分类。可以在索引中存储要经常访问的计算密集的函数，当需要访问值时，该值已经计算出来了。因此，极大地改善了查询的执行性能；
3. 增加了优化器执行范围扫描而不是全表扫描的情况的数量。

例 考虑如下 WHERE 子句中的表达式

```
CREATE INDEX idx ON example_tab(column_a + column_b);

SELECT * FROM example_tab WHERE column_a + column_b < 10;
```

因为该索引是建立在 column_a + column_b 之上的，所以优化器可以为该查询使用范围扫描。优化器根据该索引计算查询代价，如果代价最少，优化器就会选择该函数索引，column_a + column_b 就不会重复计算。

创建函数索引有以下约束条件：

1. 表达式不允许为时间间隔类型；
2. 表达式中不允许出现半透明加密列；
3. 函数索引表达式的长度理论值不能超过 816 个字符(包括生成后的指令和字符串)；
4. 函数索引不能为 CLUSTER 或 PRIMARY KEY 类型；
5. 表达式不支持集函数和不确定函数，不确定函数为每次执行得到的结果不确定，系统中不确定函数包括：RAND、CURDATE、CURTIME、CURRENT_DATE、CURRENT_TIME、CURRENT_TIMESTAMP、GETDATE、NOW、SYSDATE、

CUR_DATABASE、DBID、EXTENT、PAGE、SESSID、UID、USER、VSIZE、SET_TABLE_OPTION、SET_INDEX_OPTION、UNLOCK_LOGIN、CHECK_LOGIN、GET_AUDIT、CFALGORITHMSENCRYPT、SF_MAC_LABEL_TO_CHAR、CFALGORITHMSDECRYPT、BFALGORITHMSENCRYPT、SF_MAC_LABEL_FROM_CHAR、BFALGORITHMSDECRYPT、SF_MAC_LABEL_CMP;

6. 快速装载不支持含有函数索引的表;
7. 若函数索引中要使用用户自定义的函数,则函数必须是指定了 DETERMINISTIC 属性的确定性函数;
8. 若函数索引中使用的确定性函数发生了变更或删除,用户需手动重建函数索引;
9. 若函数索引中使用的确定性函数内有不确定因素,会导致前后计算结果不同的情况。在查询使用函数索引时,使用数据插入函数索引时的计算结果为 KEY 值;修改时可能会导致在使用函数索引过程中出现根据聚集索引无法在函数索引中找到相应记录的情况,对此进行报错处理。

10.3.1.4 创建位图索引

位图索引主要针对含有大量相同值的列而创建。位图索引被广泛引用到数据仓库中,创建方式和普通索引一致,对低基数(不同的值很少)的列创建位图索引,能够有效提高基于该列的查询效率。且执行查询语句的 where 子句中带有 AND 和 OR 谓词时,效率更加明显。

例 创建一个位图索引

```
CREATE BITMAP INDEX S1 ON PURCHASING.VENDOR (VENDORID);
```

位图索引具有以下约束:

1. 支持普通表、堆表和水平分区表创建位图索引;
2. 不支持对大字段创建位图索引;

3. 不支持对计算表达式列创建位图索引；
4. 不支持在 UNIQUE 列和 PRIMARY KEY 上创建位图索引；
5. 不支持对存在 CLUSTER KEY 的表创建位图索引；
6. 仅支持单列或者不超过 63 个组合列上创建位图索引；
7. MPP 环境下不支持位图索引的创建；
8. 不支持全局位图索引；
9. 包含位图索引的表不支持并发的插入、删除和更新操作；
10. 不支持在间隔分区表上创建位图索引。

10.3.1.5 创建位图连接索引

位图连接索引是一种提高通过连接实现海量数据查询效率的有效方式，主要用于数据仓库环境中。区别于上一节所说的建立在单表上的位图索引，位图连接索引是针对两个或者多个表的连接而建立的位图索引，同时保存了连接的位图结果。对于索引列中的每一个值，位图连接索引在索引表中保存了对应行的 ROWID。

例 创建一个位图连接索引

```
CREATE BITMAP INDEX SALES_CUSTOMER_NAME_IDX
ON SALES.SALESORDER_HEADER(SALES.CUSTOMER.PERSONID)
FROM SALES.CUSTOMER, SALES.SALESORDER_HEADER
WHERE SALES.CUSTOMER.CUSTOMERID = SALES.SALESORDER_HEADER.CUSTOMERID;
```

使用说明

1. 适用于常规索引的基本限制也适用于位图连接索引；
2. 用于连接的列必须是维度表中的主键或存在唯一约束；如果是复合主键，则必须使用复合主键中的所有列；
3. 当多个事务同时使用位图连接索引时，同一时间只允许更新一个表；

4. 连接索引创建时，基表只允许出现一次；
5. 不允许对存在 cluster key 的表创建位图连接索引；
6. 位图连接索引表（内部辅助表，命名为 BMJ$_索引名）仅支持 select 操作，其他操作都不支持：如 insert、delete、update、alter、drop 和建索引等；
7. 不支持对位图连接索引所在事实表和维度表的备份还原，不支持位图连接索引表的表级备份还原；
8. 不支持位图连接索引表、位图连接索引以及虚索引的导出导入；
9. 位图连接索引及其相关表不支持快速装载；
10. 位图连接索引名称的长度限制为：事实表名的长度+索引名称长度+6<128；
11. 仅支持普通表、堆表和 HUGE 表；
12. WHERE 条件只能是列与列之间的等值连接，并且必须含有所有表；
13. 事实表上聚集索引和位图连接索引不能同时存在；
14. 不支持对含有位图连接索引的表中的数据执行 DML，如需要执行 DML,则先删除该索引；
15. 含有位图连接索引的表不支持下列 DDL 操作：删除、修改表约束，删除、修改列，修改表名。另外，含位图连接索引的堆表不支持添加列操作；
16. 不允许对含有位图连接索引的表并发操作；
17. 创建位图连接索引时，在存储参数中可指定存储位图的字节数，有效值为：4~2048，服务器自动校正为 4 的倍数，默认值为 128。如 STORAGE(SECTION(4))，表示使用 4 个字节存储位图信息。

10.3.2 隐式地创建索引

隐式创建方式只支持创建唯一索引。

DM8 是通过在唯一键或主键上创建一个唯一索引来在表上实施 UNIQUE KEY 或

PRIMARY KEY 完整性约束。因此当创建 UNIQUE KEY 或 PRIMARY KEY 约束时，DM8 会隐式地自动创建唯一索引。此处隐式创建的唯一索引是包含索引数据的实索引。

例 创建 PRIMARY KEY 约束时，系统会隐式地自动在表 T1 的 C1 列上创建一个唯一索引

```
DROP TABLE T1;
CREATE TABLE T1(C1 INT, C2 INT, C3 INT);
//创建一个名为 PK_EMP_NAME 的 PRIMARY KEY 约束
ALTER TABLE T1 ADD CONSTRAINT PK_EMP_NAME PRIMARY KEY (C1);
//经查看，为约束创建了一个名为 INDEX33555548 的索引
SELECT INDEX_NAME FROM ALL_CONSTRAINTS WHERE CONSTRAINT_NAME='PK_EMP_NAME';
行号          INDEX_NAME
---------- -------------
1             INDEX33555548
//继续查看，INDEX33555548 索引为唯一索引
SELECT UNIQUENESS FROM ALL_INDEXES WHERE INDEX_NAME='INDEX33555548';
行号          UNIQUENESS
---------- ---------- ----------
1               UNIQUE
```

10.4 如何在 SQL 语句中使用索引

如果 SQL 语句想使用某一个索引，则 SQL 语句的查询项和 WHERE 子句的过滤项中必须包含该索引的全部索引键。当同时包含了多个索引的索引键时，系统会选择一个最优的索引进行查询。例如：SQL 语句“SELECT A FROM T WHERE B=1; ”会使用建立在（A,B）列上的索引。

总体来说，最少的查找即可得到所需数据，即为最优索引。

下面用具体的示例进行介绍。

数据准备:

```
CREATE TABLE T2 (ID int,NAME varchar(20),DEPARTMENT varchar(20),SALARY INT);
CREATE CLUSTER INDEX S21 on T2(ID); //聚集索引
CREATE INDEX S22 on T2(NAME,DEPARTMENT);
CREATE INDEX S23 on T2(NAME);
CREATE INDEX S24 on T2(DEPARTMENT);
CREATE INDEX S25 on T2(SALARY);
```

例 1 使用聚集索引 S21

```
EXPLAIN SELECT SALARY FROM T2 WHERE ID=10;

1    #NSET2: [1, 1, 8]
2      #PRJT2: [1, 1, 8]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3        #CSEK2: [1, 1, 8]; scan_type(ASC), S21(T2), scan_range[10,10]]
```

例 2 使用非聚集索引 S22

查询语句想使用索引 S22，则 SELECT 查询项和 WHERE 子句过滤项需包含 S22 的全部索引键 NAME 和 DEPARTMENT。

```
EXPLAIN SELECT DEPARTMENT FROM T2 WHERE NAME LIKE'%Zhang' AND DEPARTMENT='A 部门';
1    #NSET2: [1, 1, 96]
2      #PRJT2: [1, 1, 96]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3            #SLCT2: [1, 1, 96]; (T2.DEPARTMENT = 'A 部门' AND T2.NAME LIKE '%Zhang');
slct_pushdown(0); spl_info(NULL); flt_batch_exec(1)
4          #SSCN: [1, 1, 96]; S22(T2); btr_scan(1); is_global(0)
```

```
Predicate Information (identified by operation id):

---------------------------------------------------

3 - filter((T2.DEPARTMENT = 'A 部门' AND T2.NAME LIKE '%Zhang'))

或

EXPLAIN SELECT DEPARTMENT FROM T2 WHERE NAME='Zhang San';

1    #NSET2: [1, 1, 96]

2       #PRJT2: [1, 1, 96]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3          #SSEK2: [1, 1, 96]; scan_type(ASC), S22(T2), scan_range[('Zhang San',min),('Zhang

San',max)), is_global(0)
```

例 3 先使用非聚集索引 S23，再使用聚集索引

首先，在 SSEK 中使用 S23 非聚集索引进行扫描定位，得到 NAME；

其次，因为 SELECT * 中查询项太多，需要在 BLKUP2 中通过 S23 回到聚集索引上进行二次查找，得到*中的其它数据。

```
EXPLAIN SELECT * FROM T2 WHERE NAME='Zhang San';

1    #NSET2: [1, 1, 116]

2       #PRJT2: [1, 1, 116]; exp_num(5), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3          #BLKUP2: [1, 1, 116]; S23(T2); use_clu_addr(0)

4             #SSEK2: [1, 1, 116]; scan_type(ASC), S23(T2), scan_range['Zhang San','Zhang San'],

is_global(0)
```

10.5 重建索引

当一个表经过大量的增删改操作后，表的数据在物理文件中可能存在大量碎片，从而影响访问速度。另外，当删除表的大量数据后，若不再对表执行插入操作，索引所处的段可能占用了大量并不使用的簇，从而浪费了存储空间。

可以使用重建索引来对索引的数据进行重组，使数据更加紧凑，并释放不需要的空间，从而提高访问效率和空间效率。DM8 提供的重建索引的系统函数为：

```
SP_REBUILD_INDEX(SCHNAME varchar(128), INDEXID int);
```

SCHNAME 为索引所在的模式名，INDEXID 为索引 ID。

使用说明：

1. 水平分区子表，临时表和系统表上建的索引不支持重建

2. 虚索引和聚集索引不支持重建

例 需要重建索引 emp_name，假设其索引 ID 为 1547892，那么使用以下语句重建索引：

```
SP_REBUILD_INDEX('SYSDBA', 1547892);
```

10.6 删除索引

用户可能出于以下某项原因需要删除一个索引：

1. 不再需要该索引；
2. 该索引没有为针对其相关的表所发布的查询提供所期望的性能改善。例如，表可能很小，或者尽管表中有许多行但只有很少的索引项；
3. 应用没有用该索引来查询数据。

要想删除索引，则该索引必须包含在用户的模式中或用户必须具有 DROP ANY INDEX 数据库权限。索引删除之后，该索引的段的所有簇都返回给包含它的表空间，并可用于表空间中的其他对象。

如何删除索引，取决于该索引是否是通过 CREATE INDEX 语句明确创建的，是则可以用 DROP INDEX 语句删除该索引。

例 1 下面的语句删除 emp_ename 索引

```
DROP INDEX emp_ename;
```

删除不存在的索引会报错。若指定 IF EXISTS 关键字，删除不存在的索引，不会报错，例如：

```
DROP INDEX IF EXISTS emp_ename;
```

然而，不能直接删除与已启用的 UNIQUE KEY 键或 PRIMARY KEY 键约束相关的索引。要删除一个与约束相关的索引，必须停用或删除该约束本身。

例 2 下面的语句删除主键约束 pk_emp_name，同时删除其对应的索引

```
ALTER TABLE emp DROP CONSTRAINT pk_emp_name;
```

除了删除普通索引，DM8 还提供删除聚集索引，只要其聚集索引是通过 CREATE CLUSTER INDEX 明确建立的。

例 3 下面的语句删除 emp 表的聚集索引 clu_emp_name

```
DROP INDEX clu_emp_name;
```

删除聚集索引其实是使用 ROWID 作为索引列重建聚集索引，即跟新建聚集索引一样会重建这个表以及其所有索引。

删除表就自动删除了所有与其相关的索引。

10.7 查看索引信息

创建索引后，可以通过 INDEXDEF 系统函数查看索引的定义。

```
INDEXDEF(INDEXID int, PREFLAG int);
```

INDEXID 为索引 ID, PREFLAG 表示返回信息中是否增加模式名前缀, 1 表示导出模式名; 0 表示只导出对象名。

例 需要查看索引 emp_name 的定义，假设其索引 ID 为 1547892，那么使用以下语句查看索引定义

```
SELECT INDEXDEF(1547892, 0);
```

或

```
SELECT INDEXDEF(1547892, 1);
```

第11章 管理触发器

DM 是一个具有主动特征的数据库管理系统，其主动特征包括约束机制和触发器机制。约束机制主要用于对某些列进行有效性和完整性验证；触发器（TRIGGER）定义当某些与数据库有关的事件发生时，数据库应该采取的操作。通过触发器机制，用户可以定义、删除和修改触发器。DM 自动管理和运行这些触发器，从而体现系统的主动性，方便用户使用。

触发器是一种特殊的存储过程，它在创建后就存储在数据库中。触发器的特殊性在于它是建立在某个具体的表或视图之上的，或者是建立在各种事件前后的，而且是自动激发执行的，如果用户在这个表上执行了某个 DML 操作（INSERT、DELETE、UPDATE），触发器就被激发执行。

触发器常用于自动完成一些数据库的维护工作。例如，触发器可以具有以下功能：

1. 可以对表自动进行复杂的安全性、完整性检查；
2. 可以在对表进行 DML 操作之前或者之后进行其它处理；
3. 进行审计，可以对表上的操作进行跟踪；
4. 实现不同节点间数据库的同步更新。

触发器与存储模块类似，都是在服务器上保存并执行的一段 DMSQL 程序语句。不同的是，存储模块必须被显式地调用执行，而触发器是在相关的事件发生时由服务器自动隐式地激发。触发器是激发它们的语句的一个组成部分，即直到一个语句激发的所有触发器执行完成之后该语句才结束，而其中任何一个触发器执行的失败都将导致该语句的失败，触发器所做的任何工作都属于激发该触发器的语句。

触发器为用户提供了一种自己扩展数据库功能的方法。可以使用触发器来扩充引用完整性，实施附加的安全性或增强可用的审计选项。关于触发器应用的例子有：

1. 利用触发器实现表约束机制（如：PRIMARY KEY、FOREIGN KEY、CHECK 等）无法

实现的复杂的引用完整性;

2. 利用触发器实现复杂的事务规则（如：想确保薪水增加量不超过 25%）;

3. 利用触发器维护复杂的缺省值（如：条件缺省）;

4. 利用触发器实现复杂的审计功能;

5. 利用触发器防止非法的操作。

触发器是应用程序分割技术的一个基本组成部分，它将事务规则从应用程序的代码中移到数据库中，从而可确保加强这些事务规则并提高它们的性能。

DM 提供了三种类型的触发器:

1. 表级触发器：基于表中的数据进行触发;

2. 事件触发器：基于特定系统事件进行触发;

3. 时间触发器：基于时间而进行触发。

11.1 触发器的使用

触发器是依附于某个具体的表或视图的特殊存储过程，它在某个 DML 操作的激发下自动执行。在创建触发器时应该仔细考虑它的相关信息。具体来说，应该考虑以下几个方面的问题:

1. 触发器应该建立在哪个表/视图之上;

2. 触发器应该对什么样的 DML 操作进行响应;

3. 触发器在指定的 DML 操作之前激发还是在之后激发;

4. 对每次 DML 响应一次，还是对受 DML 操作影响的每一行数据都响应一次。

在确定了触发器的实现细节后，现在就可以创建触发器了，创建触发器的语法格式如下所示，更详细的语法格式请参考《DM8_SQL 语言使用手册》。

```
CREATE [OR REPLACE] TRIGGER 触发器名[WITH ENCRYPTION]
BEFORE|AFTER|INSTEAD OF
```

```
DELETE|INSERT|UPDATE [OF 列名]
ON 表名
[FOR EACH ROW [WHEN 条件]]
BEGIN
    DMSQL 程序语句
END;
```

用户如果要在自己的模式中创建触发器，需要具有 CREATE TRIGGER 数据库权限。如果希望能够在其它用户的模式中创建触发器，需要具有 CREATE ANY TRIGGER 数据库权限。

在创建触发器的语法结构中，用方括号限定的部分是可选的，可以根据需要选用。创建触发器的命令是 CREATE TRIGGER，根据指定的名字创建一个触发器。OR REPLACE 子句的作用是如果已经存在同名的触发器，则删除它，并重新创建。

触发器可以是前激发的（BEFORE），也可以是后激发的（AFTER）。如果是前激发的，则触发器在 DML 语句执行之前激发。如果是后激发的，则触发器在 DML 语句执行之后激发。用 BEFORE 关键字创建的触发器是前激发的，用 AFTER 关键字创建的触发器是后激发的，这两个关键字只能使用其一。INSTEAD OF 子句仅用于视图上的触发器，表示用触发器体内定义的操作代替原操作。

触发器可以被任何 DML 命令激发，包括 INSERT、DELETE、UPDATE。如果希望其中的一种、两种或者三种命令能够激发该触发器，则可以指定它们之间的任意组合，两种不同的命令之间用 OR 分开。如果指定了 UPDATE 命令，还可以进一步指定当表中的哪个列受到 UPDATE 命令的影响时激发该触发器。

FOR EACH ROW 子句的作用是指定创建的触发器为元组级触发器。如果没有这样的子句，则创建触发器为语句级触发器。INSTEAD OF 触发器固定为元组级触发器。

由关键字 BEGIN 和 END 限定的部分是触发器的代码，也就是触发器被激发时所执行的

代码。代码的编写方法与普通的 DMSQL 语句块的编写方法相同。

在触发器中可以定义变量，但必须以 DECLARE 开头。触发器也可以进行异常处理，如果发生异常，就执行相应的异常处理程序。

例 下面创建的触发器是为了监视用户对表 emp 中的数据所进行的删除操作。如果有这样的访问，则打印相应的信息。

```
CREATE OR REPLACE TRIGGER DEL_TRG
BEFORE DELETE
ON emp
BEGIN
    PRINT '您正在对表 emp 进行删除操作';
END;
/
```

如果在表 EMP 上进行 DELETE 操作，则激发这个触发器，例如：

```
SQL> DELETE FROM EMP;
您正在对表 EMP 进行删除操作
```

从触发器的执行情况可以看出，无论用户通过 DELETE 命令删除 0 行、1 行或者多行数据，这个触发器只对每次 DELETE 操作激发一次，所以这是一个典型的语句级触发器。

如果一个触发器不再使用，那么可以删除它。删除触发器的语法为：

```
DROP TRIGGER [IF EXISTS] 触发器名;
```

例 要删除刚才创建的触发器 DEL_TRG，使用语句为：

```
SQL> DROP TRIGGER DEL_TRG;
```

触发器的创建者和数据库管理员可以使触发器失效。触发器失效后将暂时不起作用，直到再次使它有效，使触发器失效的命令格式为：

```
ALTER TRIGGER 触发器名 DISABLE;
```

触发器失效后只是暂时不起作用，它仍然存在于数据库中，使用命令可以使它再次起作用，使触发器再次有效的命令格式为：

```
ALTER TRIGGER 触发器名 ENABLE;
```

例 下面的两条命令先使触发器 DEL_TRG 失效，然后使其再次有效：

```
SQL> ALTER TRIGGER DEL_TRG DISABLE;

SQL> ALTER TRIGGER DEL_TRG ENABLE;
```

11.2 表级触发器

表级触发器都是基于表中数据的触发器，它通过针对相应表对象的插入/删除/修改等 DML 语句触发。

- 触发动作

激发表级触发器的触发动作是三种数据操作命令，即 INSERT、DELETE 和 UPDATE 操作。在触发器定义语句中用关键字 INSERT、DELETE 和 UPDATE 指明构成一个触发器事件的数据操作的类型，其中 UPDATE 触发器会依赖于所修改的列，在定义中可通过 UPDATE OF <触发列清单>的形式来指定所修改的列，<触发列清单>指定的字段数不能超过 128 个。

- 触发级别

根据触发器的级别可分为元组级（也称行级）和语句级。

元组级触发器，对触发命令所影响的每一条记录都激发一次。假如一个 DELETE 命令从表中删除了 1000 行记录，那么这个表上的元组级 DELETE 触发器将被执行 1000 次。元组级触发器常用于数据审计、完整性检查等应用中。元组级触发器是在触发器定义语句中通过 FOR EACH ROW 子句创建的。对于元组级触发器，可以用一个 WHEN 子句来限制针对当前记录是否执行该触发器。WHEN 子句包含一条布尔表达式，当它的值为 TRUE 时，执行触发器；否则，跳过该触发器。

语句级触发器，对每个触发命令执行一次。例如:对于一条将 500 行记录插入表 TABLE_1

中的 INSERT 语句，这个表上的语句级 INSERT 触发器只执行一次。语句级触发器一般用于对表上执行的操作类型引入附加的安全措施。语句级触发器是在触发器定义语句中通过 FOR EACH STATEMENT 子句创建的，该子句可缺省。

- 触发时机

触发时机通过两种方式指定。一是通过指定 BEFORE 或 AFTER 关键字，选择在触发动作之前或之后运行触发器；二是通过指定 INSTEAD OF 关键字，选择在动作触发的时候，替换原始操作，INSTEAD OF 允许建立在视图上，并且只支持元组级触发。

在元组级触发器中可以引用当前修改的记录在修改前后的值，修改前的值称为旧值，修改后的值称为新值。对于插入操作不存在旧值，而对于删除操作则不存在新值。

对于新、旧值的访问请求常常决定一个触发器是 BEFORE 类型还是 AFTER 类型。如果需要通过触发器对插入的行设置列值，那么为了能设置新值，需要使用一个 BEFORE 触发器，因为在 AFTER 触发器中不允许用户设置已插入的值。在审计应用中则经常使用 AFTER 触发器，因为元组修改成功后才有必要运行触发器，而成功地完成修改意味着成功地通过了该表的引用完整性约束。

11.3 事件触发器

前面表级触发器都是基于表中数据的触发器，它通过针对相应表对象的插入/删除/修改等 DML 语句触发。DM 还支持事件触发器，包括库级和模式级触发器，这类触发器并不依赖于某个表，而是基于特定系统事件触发的，通过指定 DATABASE 或某个 SCHEMA 来表示事件触发器的作用区域。创建事件触发器的用户需要拥有 CREATE_TRIGGER 或 CREATE_ANY_TRIGGER 的权限。

可以触发的事件包含以下两类：

1. DDL 事件，包括 CREATE、ALTER、DROP、GRANT、REVOKE 以及 TRUNCATE；
2. 系统事件，包括 LOGIN/LOGON、LOGOUT/LOGOFF、AUDIT、NOAUDIT、BACKUP

DATABASE、RESTORE DATABASE、TIMER、STARTUP、SHUTDOWN 以及 SERERR（即执行错误事件）。

所有 DDL 事件触发器都可以设置 BEFORE 或 AFTER 的触发时机，但系统事件中 LOGOUT，SHUTDOWN 仅能设置为 BEFORE，而其它则只能设置为 AFTER。模式级触发器不能是 LOGIN/LOGON、LOGOUT/LOGOFF、SERERR、BACKUP DATABASE、RESTORE DATABASE、STARTUP 和 SHUTDOWN 事件触发器。

与数据触发器不同，事件触发器不能影响对应触发事件的执行。它的主要作用是帮助管理员监控系统运行发生的各类事件，进行一定程度的审计和监视工作。

11.4 时间触发器

时间触发器是一种特殊的事件触发器。时间触发器的特点是用户可以定义在任何时间点、时间区域、每隔多长时间等等的方式来激发触发器，而不是通过数据库中的某些操作包括 DML、DDL 操作等来激发，它的最小时间精度为秒。

时间触发器与其它触发器的不同只是在触发事件上，在 DMSQL 语句块（BEGIN 和 END 之间的语句）的定义是完全相同的。

时间触发器的创建语句如下：

```
CREATE [OR REPLACE] TRIGGER <触发器名> WITH ENCRYPTION
AFTER TIMER ON DATABASE
{<时间定义语句>}
BEGIN
    <执行语句>
END;
```

时间定义语句的语法规则较多，具体可参考《DM8_SQL 语言使用手册》中时间触发器相关章节。

下面的例子在每个月的第 28 天，从早上 9 点开始到晚上 18 点之间，每隔一分钟就打印一个字符串“Hello World”。

```
CREATE OR REPLACE TRIGGER timer2
AFTER TIMER ON DATABASE
FOR EACH 1 MONTH DAY 28
FROM TIME '09:00' TO TIME '18:00' FOR EACH 1 MINUTE
DECLARE
str VARCHAR;
BEGIN
    PRINT 'HELLO WORLD';
END;
/
```

时间触发器实用性很强，如在凌晨（此时服务器的负荷比较轻）做一些数据的备份操作，对数据库中表的统计信息的更新操作等类似的事情。同时也可以作为定时器通知一些用户在未来的某些时间要做哪些事情。

11.5 触发器总结

表级触发器的触发事件包括某个基表上的 INSERT、DELETE 和 UPDATE 操作，无论对于哪种操作，都能够为其创建 BEFORE 触发器和 AFTER 触发器。如果触发器的动作代码不取决于受影响的数据，语句级触发器就非常有用。例如，可以在表上创建一个 BEFORE INSERT 语句触发器，以防止在某些特定期限以外的时间对一个表进行插入。

每张基表上可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。创建触发器的用户必须是基表的创建者，或者拥有 DBA 权限。注意，不存在触发器的执行权限，因为用户不能主动“调用”某个触发器，是否激发一个

触发器是由系统来决定的。

对于语句级和元组级的触发器来说，都是在 DML 语句运行时激发的。在执行 DML 语句的过程中，基表上所创建的触发器按照下面的次序依次执行：

1. 如果有语句级前触发器的话，先运行该触发器；
2. 对于受语句影响每一行：
   1） 如果有元组级前触发器的话，运行该触发器；
   2） 执行该语句本身；
   3） 如果有元组级后触发器的话，运行该触发器；
3. 如果有语句级后触发器的话，运行该触发器。

需要注意的是，同类触发器的激发顺序没有明确的定义。如果顺序非常重要的话，可以使用 FOLLOW 和 PRECEDES 来指定，具体可参考《DM8_SQL 语言使用手册》。触发器功能强大，但需要谨慎使用，过多的触发器或复杂触发器过程脚本会降低数据库的运行效率。

还需要注意的是，在 DM 的数据守护环境下，备库上定义的触发器是不会被触发的。

第12章 管理视图、序列和同义词

视图、序列和同义词是数据库中比较重要的模式对象概念，达梦数据库支持这些模式对象的管理。

12.1 管理视图

视图是关系数据库系统提供给用户以多种角度观察数据库中数据的重要机制，它简化了用户数据模型，提供了逻辑数据独立性，实现了数据共享和数据的安全保密。视图是数据库技术中一个十分重要的功能。从系统实现的角度讲，视图是从一个或几个基表（或视图）导出的表，但它是一个虚表，即数据字典中只存放视图的定义（由视图名和查询语句组成），而不存放对应的数据，这些数据仍存放在原来的基表中。当对一个视图进行查询时，视图将查询其对应的基表，并且将所查询的结果以视图所规定的格式和次序进行返回。因此当基表中的数据发生变化时，从视图中查询出的数据也随之改变了。从用户的角度来讲，视图就像一个窗口，透过它可以看到数据库中用户感兴趣的数据和变化。当用户所需的数据是一张表的部分列、或部分行，或者数据是分散在多个表中，那么就可以创建视图来将这些满足条件的行和列组织到一个表，而不需要修改表的属性、甚至创建新的表。这样不仅简化了用户的操作，还可以提高数据的逻辑独立性，实现数据的共享和保密。

例 为了防止非公司人员查询供应商的具体信息，包括 email、phone 等，这时可以建立视图，针对不用的用户建立查询供应商信息的视图

```
CREATE VIEW normal_view AS SELECT name FROM person;

CREATE VIEW special_view AS SELECT name, sex, email, phone FROM person;
```

然后把视图 normal_view 的权限授予普通用户，只允许他们查询供应商的姓名信息，把 special_view 视图的权限授予高级用户，允许他们查看供应商的 email、电话等信息，这样就较好地实现了供应商信息的安全与保密。

关于视图的定义、删除、查询和修改可以参考《DM8_SQL 语言使用手册》。

用户也可以对那些经常进行的查询建立相应视图，如果经常使用的查询语句较庞大，这样可以节省繁琐语句的书写。

由于视图没有直接相关的物理数据，所以不能被索引。

DM 提供了一些以"V$"开头的视图来供用户了解当前服务器的使用情况。这些视图的数据在服务器运行期间一直变化，并且数据反映的主要是系统的性能信息，因此它们被称为动态性能视图。用户对动态性能视图只能进行查询操作，动态视图的具体内容可参考附录 2。

12.2 管理序列

序列（sequence）是 DM 数据库中的数据库实体之一。通过使用序列，多个用户可以产生和使用一组不重复的有序整数值。比如可以用序列来自动地生成主关键字值。序列通过提供唯一数值的顺序表来简化程序设计工作。

当一个序列第一次被查询调用时，它将返回一个预定值，该预定值就是在创建序列时所指定的初始值。默认情况下，对于升序序列，序列的缺省初始值为序列的最小值，对于降序序列，缺省初始值为序列的最大值。可以指定序列能生成的最大值，默认情况下，降序序列的最大值缺省为-1，升序序列的最大值缺省为 2^{63}-1；也可以指定序列能生成的最小值，默认情况下，升序序列的最小值缺省为 1，降序序列的最小值缺省为-2^{63}。序列的最大值和最小值可以指定为 bigint（8 个字节）所能表示的最大和最小有符号整数。

在随后的每一次查询中，序列将产生一个按其指定的增量增长的值。增量可以是任意的正整数或负整数，但不能为 0。如果此值为负，序列是下降的，如果此值为正，序列是上升的。默认情况下，增加缺省为 1。

一旦序列生成，用户就可以在 SQL 语句中用以下伪列来存取序列的值：

1. CURRVAL 返回当前的序列值；
2. NEXTVAL 如果为升序序列，序列值增加并返回增加后的值；如果为降序序列，序列

值减少并返回减少后的值。

序列可以是循环的，当序列的值达到最大值/最小值时，序列将从最小值/最大值计数。使用一个序列时，不保证将生成一串连续不断递增的值。例如，如果查询一个序列的下一个值供 insert 使用，则该查询是能使用这个序列值的唯一会话。如果未能提交事务处理，则序列值就不被插入表中，以后的 insert 将继续使用该序列随后的值。

序列在对编号的使用上具有很大用处，如果想对表建立一个列专门用来表示编号，如订单号，这样就可以使用序列，依次递增生成，用户不需进行特殊管理，这给用户带来了很大方便。如果用户需要间隔的编号，创建序列时指定 INCREMENT，就可以生成用户需要的编号。关于序列的具体使用，请参考《DM8_SQL 语言使用手册》。

12.3 管理同义词

同义词相当于模式对象的别名，起着连接数据库模式对象和应用程序的作用。假如模式对象需要更换或者修改，则不用修改应用程序而直接修改同义词就可以了。

同义词是用来实现下列用途的数据库对象：

1. 为可以存在于本地或远程服务器上的其他数据库对象（称为基础对象）提供备用名称；
2. 提供抽象层以免客户端应用程序对基础对象的名称或位置进行更改。

同义词的好处在于用户可能需要某些对象在不同的场合采用不用的名字，使其适合不同人群的应用环境。例如，创建表 product，如果客户不认识这个英文词，这时可以增加同义词，命名“产品”，这样客户就有较直观的观念，一目了然。

关于同义词的具体使用，请参考《DM8_SQL 语言使用手册》。

12.4 查看视图、序列和同义词信息

视图、序列以及同义词的定义信息可以通过查看系统表 SYSOBJECTS 和 SYSTEXTS 得

到，如查看普通视图 view_1 的信息，可以执行：

```
SELECT b.* FROM    SYS.SYSOBJECTS a, SYS.SYSTEXTS b WHERE a.ID = b.ID and a.NAME LIKE 'VIEW_1%';
```

可以得到：

```
ID                  SEQNO               TXT

1          16778216          0          create view view_1 as select id from sysobjects;

2          16778216          1          SELECT ID FROM SYSOBJECTS
```

这样可以很明确知道 VIEW_1 视图的定义。

视图的定义信息也可以通过 SP_VIEWDEF 系统过程来查看。对于物化视图而言，其定义只有通过该系统函数才能完整地获取。

```
CALL    SP_VIEWDEF('SYSDBA', 'VIEW1');
```

第13章 模式对象的常规管理

用户的模式(SCHEMA) 指的是用户账号拥有的对象集，在概念上可将其看作是包含表、视图、索引和权限定义的对象。在 DM 中，一个用户可以创建多个模式，一个模式中的对象（表、视图等）可以被多个用户使用。模式不是严格分离的，一个用户可以访问他所连接的数据库中有权限访问的任意模式中的对象。

系统为每一个用户自动建立了一个与用户名同名的模式作为其默认模式，用户还可以用模式定义语句建立其它模式。

采用模式的原因有几点：

1. 允许多个用户使用一个数据库而不会干扰其它用户；
2. 把数据库对象组织成逻辑组，让它们更便于管理；
3. 第三方的应用可以放在不同的模式中，这样可以避免和其它对象的名字冲突。模式类似于操作系统层次的目录，只不过模式不能嵌套。

DM 模式可以通过 SQL 语句进行操作。DM 模式主要包含以下的模式对象：

1. 表；
2. 视图；
3. 索引；
4. 触发器；
5. 存储过程/函数；
6. 序列；
7. 全文索引；
8. 包；
9. 同义词；

10. 类;

11. 外部链接。

在引用模式对象的时候，一般要在模式对象名前面加上模式名。具体格式如下:

[模式名].对象名

当然，在当前模式和要引用的模式对象所属的模式相同时，可以省略模式名。如果我们访问一个表时，没有指明该表属于哪一个模式，系统就会自动给我们在表前加上缺省的模式名。类似地，如果我们在创建对象时不指定该对象的模式，则该对象的模式为用户的缺省模式。

模式对象之外的其他对象统一称为非模式对象，非模式对象主要包括以下几种对象:

1. 用户;

2. 角色;

3. 权限;

4. 表空间。

13.1 在单个操作中创建多个模式对象

在 DM 数据库中，使用 CREATE SCHEMA 语句就可以创建一个空的模式对象，也可以在创建模式时，同时创建多个模式对象。CREATE SCHEMA 语句的语法说明参见《DM8_SQL 语言使用手册》。

下面的语句在创建模式 TEST 的同时，创建了属于模式 TEST 的一个序列和两张表。

```
CREATE SCHEMA TEST
    CREATE SEQUENCE ADDRESS_SEQ INCREMENT BY 1
    CREATE TABLE ADDRESS(
            ADDRESSID INT,
                ADDRESS1 VARCHAR(60) NOT NULL,
```

```
        ADDRESS2 VARCHAR(60),
        CITY VARCHAR(30) NOT NULL,
        POSTALCODE VARCHAR(15) NOT NULL)
CREATE TABLE ADDRESS_TYPE(
    ADDRESS_TYPEID INT PRIMARY KEY,
        NAME VARCHAR(50) NOT NULL);
```

可以用下面的格式来引用 TEST 模式中的 ADDRESS 表：TEST.ADDRESS，如向 ADDRESS 表中插入一条数据。

```
INSERT INTO TEST.ADDRESS VALUES(1,'武汉市关山一路特 1 号光谷软件园 C6 栋 5 层','上海市闸北区江场三路 28 号 301 室','上海市','200436');
```

13.2 重命名模式对象

要重命名一个模式对象，那么这个模式对象必须在指定的模式里面。用户可以采用下面几种方式来重命名模式对象：

1. 删除原有的模式对象，重新创建；
2. 使用 ALTER … RENAME 语句（TABLE）。

如果采用的是删除并重建模式对象的方式来重命名模式对象，那么所有的基于这个模式对象的授权都将失效。在模式对象重建之后，基于原对象的权限和角色必须重新授予。

如果使用的是 ALTER … RENAME 语句来重命名模式对象，那么所有基于原模式对象的权限将转移到重命名后的模式对象上，不需要重新授权。如重命名 TEST 上的 ADDRESS 表：

```
ALTER TABLE TEST.ADDRESS RENAME TO ADDRESS1;
```

如果一个用户 USER1 在 ADDRESS 上拥有查询、插入的权限，那么重命名之后，USER1 在 ADDRESS1 上同样拥有查询、插入权限。

在重命名一个模式对象之前，需要考虑以下影响：

1. 所有基于重命名模式对象的视图和 DMSQL 语句块都将失效，在使用之前，必须重新编译；
2. 所有基于重命名模式对象的同义词在使用的时候都会报错。

13.3 启用和停用触发器

触发器（TRIGGER）定义当某些与数据库有关的事件发生时，数据库应该采取的操作。触发器是在相关的事件发生时由服务器自动地隐式激发的。触发器是激发它们的语句的一个组成部分，即直到一个语句激发的所有触发器执行完成之后该语句才结束，而其中任何一个触发器执行的失败都将导致该语句的失败，触发器所做的任何工作都属于激发该触发器的语句。

一个触发器有两种不同的状态：

1. 启用（ENABLED）：处于开启状态的触发器在触发条件满足时，执行触发体。缺省状态下，新创建的触发器都处于开启状态；
2. 禁止（DISABLED）：处于禁止状态的触发器在触发条件满足时，也不会执行触发体。

要设置触发器的状态，可以通过 ALTER TRIGGER 语句来完成。用户必须满足以下条件：

1） 拥有该触发器；

2） 有 ALTER ANY TRIGGER 权限。

用户可以通过使用带有 ENABLED/DISABLED 选项的 ALTER TRIGGER 语句来启用/禁止触发器。

例 1 启用 addr_trig 触发器：

```
ALTER TRIGGER addr_trig ENABLE;
```

例 2 禁用 addr_trig 触发器：

```
ALTER TRIGGER addr_trig DISABLE;
```

13.4 管理完整性约束

完整性约束规则，限制表中一个或者多个列的值。约束子句可以出现在 CREATE TABLE 或者 ALTER TABLE 语句中，确定约束条件并指定受到约束的列。

13.4.1 完整性约束状态

用户可以指定一个约束是启用（ENABLE）或禁止（DISABLED）。如果启用一个约束，那么在插入数据或者是更新数据时会对数据进行检查，不符合约束的数据被阻止进入。如果约束是禁止（DISABLED）的，不符合约束的数据被允许进入数据库。

1. 禁止约束

要执行完整性约束定义的规则，约束应当设置为开启状态。但是，在下面的情况下，从性能的角度考虑，可以暂时将完整性约束禁用。

1） 导入大量的数据到一张表中；

2） 做批处理操作并对一张表做大规模修改时；

3） 导入导出一张表。

在上面三种情形下，暂时禁用完整性约束能提高系统的性能，特别是在数据仓库的情况下。在禁用约束的情况下，可以将违反约束规则的数据插入到表中，因此，用户应当在上面列表中的操作结束之后启用约束。

2. 启用约束

在启用完整性约束的情况下，不满足约束规则的行是不能插入到表中的。

13.4.2 定义完整性约束

下面的 CREATE TABLE 和 ALTER TABLE 语句在定义的时候就启用完整性约束：

```
CREATE TABLE t_con (
```

```
id NUMBER(5) CONSTRAINT t_con_pk PRIMARY KEY);

ALTER TABLE t_con

ADD CONSTRAINT t_con_pk PRIMARY KEY (id);
```

使用 ALTER TABLE 语句来启用完整性约束，可能会失败，这是因为表中已存在的数据可能违反完整性约束条件。

当启用 UNIQUE 或者 PRIMARY KEY 约束时，系统就会创建一个对应的索引。

13.4.3 修改或删除现有的完整性约束

用户可以使用 ALTER TABLE 语句来启用、禁止、删除一个约束。当用户使用 UNIQUE 或者 PRIMARY KEY 约束时，系统会创建对应的索引；当这个约束被删除或者是被禁用，索引就会被删除。

1. 禁用已被启用的约束

下面的 ALTER TABLE 语句可以禁止完整性约束：

```
ALTER TABLE t_con DISABLE CONSTRAINT t_con_pk;
```

当有外键引用 UNIQUE 或者 PRIMARY KEY 列时，用户不能禁用 UNIQUE 或者 PRIMARY KEY 约束。

2. 删除约束

用户可以通过 ALTER TABLE 语句和 DROP CONSTRAINT 参数来删除完整性约束。如：

```
ALTER TABLE t_con DROP CONSTRAINT t_con_pk;
```

如果有外键引用约束(UNIQUE 或者 PRIMARY KEY)时，在删除约束时必须加上 CASCADE 参数，否则不能删除。

13.4.4 查看约束信息

用户可以在系统表 SYSOBJECTS 和 SYSCONS 中查询除 NOT NULL 约束、NOT VISIBLE

约束以及 CLUSTER KEY 约束之外的其余完整性约束信息。

例 在 SYSOBJECTS 系统表中查找约束名为 T_CON_PK 的信息。

```
SELECT * FROM SYSOBJECTS WHERE NAME='T_CON_PK';
```

查找所有约束的信息：

```
select * FROM SYSOBJECTS WHERE SUBTYPE$='CONS';
```

系统表 SYSOBJECTS 和 SYSCONS 的详细描述详见附录数据字典部分。

13.5 管理对象依赖性

在实际应用中，许多模式对象需要引用其他的对象。例如，一个视图中包含一个查询，其中引用了其他的表或视图。一个模式对象引用了别的对象被称为依赖对象，被引用的对象称为被依赖对象。

达梦数据库在对象被调用的时候自动重新编译，确保对象有效。

13.6 管理对象名称解析

DM 数据库对象名字是由两部分组成，之间用点号隔开。例如：TEST.ADDRESS。

对象分为两大类：一是 SQL 语句中的对象；二是 DMSQL 程序中的对象。两者解析方式不同。例如：SQL 语言中 A.B 的解析会优先解析为“模式.对象”，例如：模式 A 表 B。DMSQL 程序中 A.B 的解析会优先解析为“本模式下包.对象”，如本模式下的包 A.函数 B。

13.6.1 SQL 语句对象

在 SQL 语句中，对象名字的解析规则如下：

1. 如果只有一个名字，而没有点号。

在当前模式下寻找是否存在相同名字的对象，如果找到，则返回；否则报错。

2. 如果有点号，首先检测对象名的第一部分，例如：在 TEST.SCHOOL.CLASS 中，TEST

就是第一部分。具体步骤下面详细介绍：

1） 寻找哪一个模式的名字和第一部分相同，如果找到，以此模式进行步骤 2)；否则报错。

2） 在模式中需找是否有与对象名第二部分同名的对象，如果找到且待解析对象名只有两个部分，则返回；如果找到但待解析对象名多于两个部分，则转步骤 3)；否则报错。例如 TEST.SCHOOL.CLASS，若 SCHOOL 表对象匹配成功，进入步骤 3）；

3） 在上次匹配成功的对象中按层次依次查找待解析对象名的剩余部分，如果所有待解析对象名全部匹配成功，则返回；否则报错。例如 TEST.SCHOOL.CLASS，在 TEST 模式下中的 SCHOOL 表对象中寻找 CLASS 列对象，若找到，则返回。

13.6.2 DMSQL 程序对象

在 DMSQL 程序中，对象名字的解析规则如下：

1. 如果只有一个名字，而没有点号。

在当前模式下寻找是否存在相同名字的对象，如果找到，则返回；否则报错。这一点与 SQL 语句对象一致。

2. 如果有点号，首先检测对象名的第一部分，如在 TEST.ADDRESS 中，TEST 就是第一部分。具体步骤下面详细介绍：

1）寻找当前模式下是否存在包名与第一部分相同，如果找到，进行步骤 2），否则进行步骤 4）。

2）在步骤 1 的包中查找是否有与对象名第二部分同名的对象，若找到且待解析对象名只有两个部分，则返回。若找到但待解析的对象名多于两个，则进行步骤 3）。若包中不存在与第二部分同名的对象则报错。

3）查找包含在前一个对象中并且和待解析对象名当前解析部分相同名字的对象，如果找到，循环步骤 3)，直至所有的部分检测结束后再返回；如果没有找到，则直接报错。

4）查找是否存在模式名与对象名的第一部分相同，如果找到，以此模式进行步骤 5），否则以当前模式进行步骤 5）。

5）在模式中需找是否有与对象名第二部分同名的对象，如果找到且待解析对象名只有两个部分，则返回；如果找到但待解析对象名多于两个部分，则转步骤 6)；否则报错。

6）在模式中查找包含在前一个对象中并且和待解析对象名当前解析部分相同名字的对象，如果找到，循环步骤 6)，直至所有的部分检测结束后再返回；如果没有找到，则直接报错。如 TEST.SCHOOL.CLASS.STUDENT，在 TEST 模式下中的 SCHOOL 对象中寻找 CLASS 的对象，然后在 CLASS 对象中寻找 STUDENT 对象。

13.7 显示有关模式对象的信息

在 DM 数据库系统中，模式对象的信息主要记录在 SYSOBJECTS 系统表中。可以通过下面的语句来查询模式对象的相关信息。

```
SELECT * FROM SYSOBJECTS WHERE TYPE$ = 'SCHOBJ' OR TYPE$ = 'TABOBJ';
```

但是，SYSOBJECTS 系统表并不能将模式对象的所有信息存储起来。用户应该和其它系统表一起获得更加详细的信息。

如果用户想获得索引更详细的信息，可以查询 SYSINDEXES 表；如果用户想了解存储过程、函数等的定义信息，可以查询 SYSTEXTS 表；如果用户想了解更详细的约束的相关信息，可以查询 SYSCONS 表。（系统表的详细介绍参考附录 1。）

第三部分 高级数据库管理

第14章 数据库布局和存储管理

规划数据库结构时需要考虑如何管理数据库中的相关文件，每个表空间存储什么数据，在表空间中创建几个多大的数据文件，以及数据文件存储的位置等。本章介绍了表空间的管理、数据文件的管理、重做日志文件的管理、回滚空间的管理和控制文件的管理。

14.1 管理表空间

表空间的管理操作需要 DM 服务器处于打开状态下。

14.1.1 创建表空间

创建表空间时需要指定表空间名和其拥有的数据文件列表。比如创建名为 bookshop 的表空间，并指定该空间上拥有 2 个数据文件，每个数据文件的大小为 128MB。

```
CREATE TABLESPACE bookshop DATAFILE 'd:\bookshop1.dbf' SIZE 128, 'd:\bookshop2.dbf' SIZE 128;
```

理论上最多允许有 65535 个表空间，但用户允许创建的表空间 ID 取值范围为 0~32767，超过 32767 的只允许系统使用，ID 由系统自动分配，ID 不能重复使用，即使删除掉已有表空间，也无法重复使用已用 ID 号，也就是说只要创建 32768 次表空间后，用户将无法再创建表空间。

14.1.2 扩展表空间

表空间通过数据文件来扩展，表空间的大小等于构成该表空间的所有数据文件的大小之和。所以要扩展表空间可以通过添加新的数据文件或者扩展表空间中已有的数据文件完成。数据文件的添加和扩展可见下一节。

14.1.3 删除表空间

只可以删除用户创建的表空间并且只能删除未使用过的表空间。删除表空间时会删除其拥有的所有数据文件。

例 删除 bookshop 表空间

```
DROP TABLESPACE bookshop;
```

14.1.4 修改表空间名

可修改已存在的由用户创建的表空间的名称。比如可修改 bookshop 表空间名为 books。

```
ALTER TABLESPACE bookshop RENAME TO books;
```

14.1.5 修改表空间状态

用户表空间有联机和脱机两种状态。系统（SYSTEM）表空间、回滚（ROLL）表空间、日志表空间（RLOG）和临时（TEMP）表空间不允许脱机。设置表空间状态为脱机状态时，如果该表空间有未提交的事务，则脱机失败报错。脱机后可对表空间的数据进行备份。

例 1 修改 bookshop 表空间状态为脱机

```
ALTER TABLESPACE bookshop OFFLINE;
```

例 2 修改 bookshop 表空间状态为联机

```
ALTER TABLESPACE bookshop ONLINE;
```

注意：DM MPP 环境下，可能发现节点间的表空间不一致情况，例如：EP01 为 ONLINE 状态，EP02 为 OFFLINE 状态，这个时候，无论执行 ONLINE 还是 OFFLINE 都是报错。需要用户介入，才可以解决问题。用户 LOCAL 方式登录实例，并执行 SP_SET_SESSION_LOCAL_TYPE(1)，使得该会话可以执行 DDL 操作，再执行 ONLINE 或

者 OFFLINE 即可。

14.1.6 查询表空间与数据文件对应关系

可以通过查询动态视图 V$TABLESPACE 得到系统中除 RLOG 表空间外其他所有表空间的信息，通过查询动态视图 V$DATAFILE 得到系统中除 RLOG 表空间外所有数据文件的信息。将两个动态视图以表空间 ID 为连接条件，通过以下查询可以得到表空间上对应的数据文件。

```
SELECT ts.NAME, df.PATH FROM V$TABLESPACE AS ts, V$DATAFILE AS df WHERE ts.ID =
df.GROUP_ID;
```

14.1.7 表空间文件失效检查

LINUX 操作系统中，被进程打开的文件仍可以在 OS 系统中被删除，因此存在 DM 数据文件可能被误删的风险。如果数据文件被删除，DM 系统能够及时检测出来，并立刻停止对其继续使用并通知用户。

在 dm.ini 中参数 FIL_CHECK_INTERVAL 的值指定 DM 系统检查数据文件是否仍存在的时间间隔。将其设为 0 表示不进行检查。

也可以通过系统过程 SP_FILE_SYS_CHECK()来手动的进行检查。

系统一旦检测出某个表空间内的数据文件被删除，则与该表空间所有的操作都将会失败，并报错该表空间内有数据文件被删除。

14.1.8 表空间失效文件恢复

LINUX 系统中被删除的文件，只要其句柄没有被关闭，可以在/proc/<pid>/fd 中找到其对应的文件副本。其中<pid>指打开该文件的进程 id。

利用该方法，结合 OS 命令，DM 提供失效文件的恢复方案如下：

1、 调用系统过程 SP_TABLESPACE_PREPARE_RECOVER(表空间名称)准备进行恢复;

2、 如果使用过程中 DM 报错表空间数据文件被删除，通过操作系统的 ps 命令找到当前 DMServer 的 PID：ps -ef|grep dmserver;

3、 使用操作系统 ll 命令查看被删除文件对应的副本：ll /proc/<PID>/fd，会发现被删除的文件后有(deleted)字样;

4、 使用操作系统的 cp 命令将文件复制到原位置：cp 源路径 目的路径;

5、 复制成功后，调用系统过程 SP_TABLESPACE_RECOVER(表空间名称)完成表空间失效文件的恢复。

注意，要保证数据文件正确修复，需要保证在 SP_TABLESPACE_PREPARE_RECOVER 后进行数据文件的复制。

14.2 管理数据文件

管理数据文件的操作需要 DM 服务器处于打开状态下进行。其中，移动数据文件的操作在 DM 服务器处于打开或配置状态下都可以进行。

14.2.1 添加数据文件

可以在用户表空间中添加数据文件。添加的数据文件大小最小为 4096*页大小，如页大小为 8KB，则可添加的文件最小值为 4096*8kB=32MB。比如在 bookshop 表空间中添加大小为 64MB 的数据文件。

```
ALTER TABLESPACE bookshop ADD DATAFILE 'd:\book.dbf' SIZE 64;
```

一个表空间中，数据文件和镜像文件的个数一起不能超过 INI 参数 TS_FIL_MAX_ID 设置的值。例如，如果创建表空间的时候已经指定了 1 个数据文件，那么添加数据文件的时候，最多只能添加 TS_FIL_MAX_ID-1 个文件了。

14.2.2 删除数据文件

可以删除用户表空间中的数据文件。比如删除 bookshop 表空间中的一个名为 book.dbf 的数据文件。

```
ALTER TABLESPACE bookshop DROP DATAFILE 'd:\book.dbf';
```

删除表空间中数据文件时必须遵守文件 ID 从大到小的原则。先从具有最大文件 ID 的数据文件开始，且表空间中文件 ID 为零的数据文件不能被删除。

14.2.3 扩展数据文件的大小

可以扩展用户表空间中已存在的数据文件的大小。比如扩展 bookshop 表空间中数据文件 book.dbf 大小至 128MB。

```
ALTER TABLESPACE bookshop RESIZE DATAFILE 'd:\book.dbf' TO 128;
```

14.2.4 缩减数据文件的大小

可以缩减用户表空间中已存在的数据文件的大小。比如将 bookshop 表空间中数据文件 book.dbf 大小缩减至 32MB。

```
ALTER TABLESPACE bookshop RESIZE DATAFILE 'd:\book.dbf' TO 32;
```

14.2.5 指定数据文件的扩展属性

可以指定数据文件是否可以扩展，每次扩展的空间大小以及数据文件可扩展到的最大空间大小，子句的语法为：

```
AUTOEXTEND OFF|ON [NEXT <文件扩展大小>] [MAXSIZE <文件限制大小>]
```

OFF 表示文件不可扩展，ON 表示文件可扩展。文件扩展大小表示当需要扩展文件时，文件一次增大的空间大小，取值范围是 1~2048，单位是 MB。实际的文件扩展大小由指定的

值与 INI 参数 TS_AUTO_EXTEND_SIZE 共同决定，为大于等于 TS_AUTO_EXTEND_SIZE 且是指定的文件扩展大小的整数倍的值。若未指定文件扩展大小，则扩展大小默认为 INI 参数 TS_AUTO_EXTEND_SIZE 所指定的大小，后续不随该 INI 参数的变化而动态调整。另外，TEMP 表空间文件在 V$DATAFILE 中显示的扩展大小始终为 0，但其实际扩展大小由 INI 参数 TS_AUTO_EXTEND_SIZE 控制，会随着该参数值的变化而自动调整；使用 V8.1.5.13 之前的版本创建的扩展大小为 0 的表空间文件在 V$DATAFILE 中显示的扩展大小为 0 时，其实际扩展大小由 INI 参数 TS_AUTO_EXTEND_SIZE 控制，会随着该参数值的变化而自动调整。

文件限制大小表示文件可扩展的最大空间大小，为 0 或者 UNLIMITED 表示无限制，单位是 MB，缺省为无限制。

创建表空间时可指定文件的扩展属性。如创建表空间时指定数据文件的扩展属性为可自动扩展，每次扩展大小为 10MB，最大可扩展到 100MB：

```
CREATE TABLESPACE bookshop DATAFILE 'd:\book.dbf' SIZE 32 AUTOEXTEND ON NEXT 10 MAXSIZE 100;
```

在表空间中添加文件时可指定文件的扩展属性。如添加数据文件时指定扩展属性为不可自动扩展：

```
ALTER TABLESPACE bookshop ADD DATAFILE 'd:\book.dbf' SIZE 1024 AUTOEXTEND OFF;
```

可修改表空间中已存在的数据文件的扩展属性。如修改数据文件的扩展属性为可自动扩展：

```
ALTER TABLESPACE bookshop DATAFILE 'd:\book.dbf'  AUTOEXTEND ON;
```

14.2.6 修改数据文件的路径

可以修改用户表空间中已存在数据文件的路径，待修改的数据文件所在表空间必须处于脱机状态并且只可修改用户创建的表空间中文件的路径。如修改 bookshop 表空间中文件

book.dbf 的路径为 e:\ book.dbf。

```
ALTER TABLESPACE bookshop RENAME DATAFILE 'd:\book.dbf' TO 'e:\book.dbf';
```

14.2.7 添加 HUGE 数据文件路径

可以为普通表空间添加 HUGE 数据文件路径将其升级为混合表空间，也可以为混合表空间添加 HUGE 数据文件路径，一个混合表空间最多可以添加 127 个 HUGE 数据文件路径。如为普通表空间 bookshop 添加 HUGE 数据文件路径 e:\HUGE，将 bookshop 升级为混合表空间。

```
ALTER TABLESPACE bookshop ADD HUGE PATH 'e:\HUGE';
```

14.3 管理重做日志文件

14.3.1 添加重做日志文件

在服务器打开状态下，可以添加新的重做日志文件。在 32 位版本下，文件大小的范围是 256 和 2048 之间的整数，单位 MB；在 64 位版本下，文件大小的范围是 256 和 8192 之间的整数，单位 MB。如添加重做大小为 256MB 的重做日志文件 DAMENG03.log。

```
ALTER DATABASE ADD LOGFILE 'd:\DAMENG03.log' size 256;
```

14.3.2 扩展重做日志文件

在服务器打开状态下，可以扩展已有的重做日志文件的大小。如扩展重做日志文件 DAMENG03.log 到 300MB。

```
ALTER DATABASE RESIZE LOGFILE 'd:\DAMENG03.log' to 300;
```

14.3.3 删除重做日志文件

在服务器打开状态下，可以删除已有的重做日志文件。如删除重做日志文件 DAMENG03.log。

```
ALTER DATABASE DROP LOGFILE 'd:\DAMENG03.log';
```

在删除重做日志文件时，指定的文件需要满足以下的一些条件才能够成功删除：

1. 指定文件必须是当前节点自身的重做日志文件。

2. 指定文件不是当前正在写的重做日志文件，并且该文件上不存在有效的重做日志。

3. 指定文件必须是 RLOG 表空间中 FILE_ID 最大的那个文件。

4. 指定文件不能是 0 号或 1 号重做日志文件。

5. 指定文件删除前后的日志空间都应该是安全的。即当前已使用的日志空间不应该超过 RLOG_RESERVE_THRESHOLD，并且删除该文件不会导致剩余日志空间过小。

14.4 管理回滚表空间

回滚表空间的管理和用户表空间的管理基本是一样的，区别是回滚空间的空间名固定为 ROLL，不可修改。可增加、扩展和缩减回滚空间中的回滚文件，设置回滚空间的扩展属性，相关操作可参考 14.2 管理数据文件中的说明。需要注意的是，回滚表空间文件的缩减操作比较特殊。由于执行缩减语句的事务本身需要分配新的回滚页，而每个用户会话的事务所新分配出的回滚页，其页号分布是不相同的。因此，当指定的目标缩减偏移量较小时，即使不存在其他已使用且未释放的回滚页，也无法保证能够一次性缩减成功，这种情况下可以通过开启多个会话反复尝试。

回滚文件的路径记录在控制文件里面，可以使用 dmctlcvt 工具在 DM 服务器关闭的状态下对控制文件进行修改。使用 dmctlcvt 工具将控制文件转换为文本文件，编辑文本文件中要修改的文件的路径后再使用 dmctlcvt 工具将文本文件转换为控制文件即可。

首先转换控制文件到文本文件：

```
dmctlcvt type=1 src=D:\dm.ctl   dest=D:\ctl.txt
```

编辑 ctl.txt 文本文件中 fil_path=d:\roll.dbf 为 fil_path=e:\roll.dbf，保存文本文件。复制 d:\roll.dbf 文件为 e:\roll.dbf。

最后转换文本文件到控制文件：

```
dmctlcvt type=2 src= D:\ctl.txt   dest= D:\dm.ctl
```

这种修改文件路径的方法也可用于重做日志文件，SYSTEM 表空间文件等路径的修改。

14.5 管理控制文件

可以在 dm.ini 中通过设置 CTL_PATH 配置参数的值来指定控制文件的路径，缺省控制文件在数据目录下。例如可以把控制文件复制到 D 盘的 control_path 目录下，同时修改 dm.ini 中 CTL_PATH = D:\control_path\dm.ctl。

第15章 管理分区表和分区索引

在大型的企业应用或企业级的数据库应用中，要处理的数据量通常达到 TB 级，对于这样的大型表执行全表扫描或者 DML 操作时，效率是非常低的。

为了提高数据库在大数据量读写操作和查询时的效率，达梦数据库提供了对表和索引进行分区的技术，把表和索引等数据库对象中的数据分割成小的单位，分别存放在一个个单独的段中，用户对表的访问转化为对较小段的访问，以改善大型应用系统的性能。

DM 提供了水平分区方式。水平分区包括范围、哈希和列表三种方式，企业可以使用合适的分区方法，如日期（范围）、区域（列表），对大量数据进行分区。由于 DM 划分的分区是相互独立且可以存储于不同的存储介质上的，完全可满足企业高可用性、均衡 IO、降低维护成本、提高查询性能的要求。

15.1 分区的概念

分区是指将表、索引等数据库对象划分为较小的可管理片段的技术，每一个片段称为分区子表或分区索引。一个表被分区后，对表的查询操作可以局限于某个分区进行，而不是整个表，这样可以大大提高查询速度。

DM 支持对大表进行水平分区。例如，通讯公司将用户通话记录保存在一张表中，一年这个表产生 40GB 的数据。假设要对用户的通话信息按照季度进行统计，那么这样的统计需要在全表范围内进行。如果对表按季度进行水平分区，那么每个分区的大小平均为 10GB 左右，这样在进行统计时，只需在 10GB 的范围内进行即可。

DM 采用子表方式创建分区表，分区表作为分区主表，而每一个分区以一个子表实体存在，即每一个分区都是一个完整的表，一般命名为主表名 _ 分区名。对于水平分区，子表跟主表具有相同的逻辑结构，即分区子表与分区主表有相同的列定义和约束定义。在 DM 分区表中，主表本身不存储数据，所有数据只存储在子表中，从而实现不同分区的完全独立性。

水平分区子表删除后，会将子表上的数据一起删除。

由于每一个分区都以一个子表作为实体，那么不同分区可以存储于相同表空间，也可以位于不同的表空间中。将这些分区放在不同的表空间中具有以下的好处：

1. 减少所有数据都损坏的可能性，一个表空间损坏不影响其他表空间，提高可用性；
2. 恢复时间大大减少；
3. 可以将同一个表中的数据分布在不同的磁盘上，从而均衡磁盘上的 I/O 操作；
4. 提高了表的可管理性、可利用性和访问效率。

分区操作对现存的应用和运行在分区表上的标准 DML 语句来说是透明的。但是，可以通过在 DML 中使用分区子表名字来对应用进行编程，使其充分利用分区的优点。

15.2 分区的方法

达梦数据库 DM 支持对表进行水平分区。对于水平分区，提供以下分区方式：

1. 范围(range)水平分区：对表中的某些列上值的范围进行分区，根据某个值的范围，决定将该数据存储在哪个分区上；
2. 哈希（hash）水平分区：通过指定分区编号来均匀分布数据的一种分区类型，通过在 I/O 设备上进行散列分区，使得这些分区大小基本一致；
3. 列表（list）水平分区：通过指定表中的某个列的离散值集，来确定应当存储在一起的数据。例如，可以对表上的 status 列的值在('A'，'H'，'O')放在一个分区，值在('B'，'I'，'P')放在另一个分区，以此类推；
4. 多级分区表：按上述三种分区方法进行任意组合，将表进行多次分区，称为多级分区表。

15.3 创建水平分区表

在创建表的语法中，使用 partition 子句指定分区方式和分区列，以及分区的名字等信

息，即可创建分区表。而分区子表可以指定 storage 子句，设置子表的存储属性，如所属表空间等；如果不指定，则继承分区主表的存储特性及表的其他属性。同时，支持多级分区表。

水平分区表的 ROWID 与其主表属性一致：LIST 表的水平分区表的 ROWID 是物理的；普通表的水平分区表的 ROWID 是逻辑的，且每个子表的 ROWID 都是从 1 开始增长，但是最终返回前，ROWID 的高字节会补充上子表序号。

15.3.1 创建范围分区表

范围分区是按照某个列或几个列的值的范围来创建分区，当用户向表中写入数据时，数据库服务器将按照这些列上的值进行判断，将数据写入相应的分区中。

在创建范围分区时，首先要指定分区列，即按照哪些列进行分区，然后为每个分区指定数据范围。范围分区支持 MAXVALUE 范围值的使用，MAXVALUE 相当于一个比任何值都大的值。范围分区非常适用于数据按时间范围组织的表，不同的时间段的数据属于不同的分区。

例如，以下语句创建一个范围分区表 callinfo，用来记录用户的 2010 年的电话通讯信息，包括主叫号码、被叫号码、通话时间和时长，并且根据季度进行分区。

```
CREATE  TABLE  callinfo(
caller     CHAR(15),
callee     CHAR(15),
time       DATETIME,
duration  INT
)
PARTITION BY RANGE(time)(
PARTITION p1 VALUES LESS THAN ('2010-04-01'),
PARTITION p2 VALUES LESS THAN ('2010-07-01'),
PARTITION p3 VALUES LESS THAN ('2010-10-01'),
```

```
PARTITION p4 VALUES EQU OR LESS THAN ('2010-12-31')  //'2010-12-31'也可替换为 MAXVALUE
);
```

值得注意的是，MAXVALUE 之间无法比较大小。如下所示:

```
create table callinfo (
caller      CHAR(15),
callee      CHAR(15),
time        DATETIME,
duration  INT
)
partition by range(caller, callee)
            (
            partition p1 values less than ('a', 'b'),
            partition p2 values less than (maxvalue, 'd'),
            partition p3 values less than (maxvalue,maxvalue)
                  );     //报"范围分区值非递增"错误，建分区表失败
```

在创建分区表时，首先通过"PARTITION BY RANGE"子句指定分区的类型为范围分区，然后在这个子句之后指定一个或多个列作为分区列，如 callinfo 的 time 字段。

表中的每个分区都可以通过"PARTITION"子句指定一个名称。并且每一个分区都有一个范围，通过"VALUES LESS THAN"子句可以指定上界，而它的下界是前一个分区的上界。如分区 p2 的 time 字段取值范围是['2010-04-01', '2010-07-01')。如果通过"VALUES EQU OR LESS THAN"指定上界，即该分区包含上界值，如分区 p4 的 time 字段取值范围是['2010-10-01', '2010-12-31']。另外，可以对每一个分区指定 storage 子句，不同分区可存储在不同表空间中。

在创建范围分区表时，可通过“INTERVAL <间隔表达式>”子句将该范围分区表指定为间隔分区表。当对间隔分区表中的数据进行插入或更新操作时，若新的数据无法匹配现有的分区子表，则系统将自动以用户指定的现有分区的末尾临界值为起始值，以<间隔表达式>指定的值为间隔值创建一个可以匹配新数据的间隔分区。用户可通过查看系统表 SYSHPARTTABLEINFO 来获取新建分区的分区表 ID 以及分区名等信息。该功能可方便数据库管理员对分区表的管理。

如果分区表包含多个分区列，采用多列比较方式定位匹配分区。首先，比较第一个分区列值，如果第一列值在范围之内，就以第一列为依据进行分区；如果第一列值处于边界值，那么需要比较第二列的值，根据第二列为依据进行分区；如果第二列的值也处于边界值，需要继续比较后续分区列值，以此类推，直到确定目标分区为止。匹配过程参看表 15.1。

表 15.1 多分区列匹配

插入记录	分区范围值		
	(10,10,10)	(20,20,20)	(30,30,30)
(5,100,200)	满足		
(10,10,11)		满足	
(31,1,1)	不满足	不满足	不满足

当在分区表中执行 DML 操作时，实际上是在各个分区子表上透明地修改数据。当执行 SELECT 命令时，可以指定查询某个分区上的数据。

例 以下语句查询 callinfo 表中分区 p1 的数据

```
SELECT * FROM callinfo PARTITION (p1);
```

15.3.2 创建 LIST 分区表

范围分区是按照某个列上的数据范围进行分区的，如果某个列上的数据无法通过划分范

围的方法进行分区，并且该列上的数据是相对固定的一些值，可以考虑使用 LIST 分区。一般来说，对于数字型或者日期型的数据，适合采用范围分区的方法；而对于字符型数据，取值比较固定的，则适合于采用 LIST 分区的方法。

例 创建一个产品销售记录表 sales，记录产品的销量情况。由于产品只在几个固定的城市销售，所以可以按照销售城市对该表进行分区。

```
CREATE  TABLE  sales(
sales_id INT,
saleman CHAR(20),
saledate DATETIME,
city        CHAR(10)
)
PARTITION BY LIST(city)(
PARTITION p1 VALUES ('北京', '天津'),
PARTITION p2 VALUES ('上海', '南京', '杭州'),
PARTITION p3 VALUES ('武汉', '长沙'),
PARTITION p4 VALUES ('广州', '深圳')
);
```

在创建 LIST 分区时，通过“PARTITION BY LIST”子句指定对表进行 LIST 分区，然后在每个分区中分区列的取值通过 VALUES 子句指定。当用户向表插入数据时，只要分区列的数据与 VALUES 子句指定的数据之一相等，该行数据便会写入相应的分区子表中。

注意的是，LIST 分区的分区键必须唯一。

15.3.3 创建哈希分区表

在很多情况下，用户无法预测某个列上的数据变化范围，因而无法实现创建固定数量的

范围分区或 LIST 分区。

在这种情况下，DM 哈希分区提供了一种在指定数量的分区中均等地划分数据的方法，基于分区键的散列值将行映射到分区中。当用户向表中写入数据时，数据库服务器将根据一个哈希函数对数据进行计算，把数据均匀地分布在各个分区中。在哈希分区中，用户无法预测数据将被写入哪个分区中。

现在重新考虑产品销售表的例子。如果销售城市不是相对固定的，而是遍布全国各地，这时很难对表进行 LIST 分区。如果为该表进行哈希分区，可以很好地解决这个问题。

```
CREATE  TABLE  sales01(
sales_id INT,
saleman CHAR(20),
saledate DATETIME,
city       CHAR(10)
)
PARTITION BY HASH(city)(
PARTITION p1,
PARTITION p2,
PARTITION p3,
PARTITION p4
);
```

如果不需指定分区表名，可以通过指定哈希分区个数来建立哈希分区表。

```
CREATE  TABLE  sales02(
sales_id INT,
saleman CHAR(20),
saledate DATETIME,
```

```
city        CHAR(10)
)
PARTITION BY HASH(city)
PARTITIONS 4 STORE IN (ts1, ts2, ts3, ts4);
```

PARTITIONS 后的数字表示哈希分区的分区数，STORE IN 子句中指定了哈希分区依次使用的表空间。使用这种方式建立的哈希分区表分区名是匿名的，DM 统一使用 DMHASHPART+分区号（从 0 开始）作为分区名。例如，需要查询 sales 第一个分区的数据，可执行以下语句：

```
SELECT * FROM sales02 PARTITION (dmhashpart0);
```

15.3.4 创建多级分区表

在很多情况下，经过一次分区并不能精确地对数据进分类，这时需要多级分区表。

例 创建一个产品销售记录表 sales，记录产品的销量情况。由于产品需要按地点和销售时间进行统计，则可以对该表进行 LIST-RANGE 分区。

```
DROP TABLE SALES;
CREATE  TABLE  SALES(
SALES_ID  INT,
SALEMAN CHAR(20),
SALEDATE DATETIME,
CITY        CHAR(10)
)
PARTITION BY LIST(CITY)
 SUBPARTITION BY RANGE(SALEDATE) SUBPARTITION TEMPLATE(
     SUBPARTITION P11 VALUES LESS THAN ('2012-04-01'),
```

```
    SUBPARTITION P12 VALUES LESS THAN ('2012-07-01'),
    SUBPARTITION P13 VALUES LESS THAN ('2012-10-01'),
    SUBPARTITION P14 VALUES EQU OR LESS THAN (MAXVALUE))
(
    PARTITION P1 VALUES ('北京', '天津')
    (
      SUBPARTITION P11_1 VALUES LESS THAN ('2012-10-01'),
      SUBPARTITION P11_2 VALUES EQU OR LESS THAN (MAXVALUE)
    ),
    PARTITION P2 VALUES ('上海', '南京', '杭州'),
    PARTITION P3 VALUES (DEFAULT)
);
```

在创建多级分区表时，指定了子分区模板，同时子分区 P1 自定义了子分区描述 P11_1 和 P11_2。P1 有两个子分区 P11_1 和 P11_2。而子分区 P2 和 P3 有四个子分区 P11、P12、P13 和 P14。

DM 支持最多八层多级分区。

下面给出一个三级分区的例子，更多级别的分区表的建表语句语法类推。

```
CREATE TABLE STUDENT(NAME VARCHAR(20), AGE INT, SEX VARCHAR(10) CHECK (SEX IN ('MALE','FEMALE')), GRADE INT CHECK (GRADE IN (7,8,9)))
PARTITION BY LIST(GRADE)
  SUBPARTITION BY LIST(SEX) SUBPARTITION TEMPLATE
  (
    SUBPARTITION Q1 VALUES('MALE'),
```

```
    SUBPARTITION Q2 VALUES('FEMALE')
  ),
  SUBPARTITION BY RANGE(AGE) SUBPARTITION TEMPLATE
  (
    SUBPARTITION R1 VALUES LESS THAN (12),
    SUBPARTITION R2 VALUES LESS THAN (15),
    SUBPARTITION R3 VALUES LESS THAN (MAXVALUE)
   )
(
  PARTITION P1 VALUES (7),
  PARTITION P2 VALUES (8),
  PARTITION P3 VALUES (9)
);
```

15.4 在水平分区表上建立索引

DM 支持在水平分区表上建立普通索引、唯一索引、聚集索引、函数索引、位图索引。

此外，只支持在水平分区表上创建局部聚集索引、不支持全局聚集索引、不支持局部唯一函数索引。HUGE 水平分区表不支持全局索引。

在水平分区表上创建索引的方式分为两种：隐式和显式。下面分别进行介绍：

一 隐式地创建索引

创建水平分区表时，若使用了 PRIMARY KEY 或 UNIQUE 关键字，无需用户显式指定，系统会隐式创建全局索引或局部索引。

创建水平分区表时，若表的 PRIMARY KEY 或 UNIQUE 列未包含所有分区列，系统会自动创建全局索引，否则自动创建局部索引。创建全局索引时，在水平分区表的主表创建全局

索引，每个分区子表的数据都被索引在同一个 B 树中，并在每个子表创建全局本地索引，与主表共用一个 B 树。

例 1 下面的语句在创建水平分区表 test 时会自动创建一个全局索引。

```
CREATE TABLE test(c1 INT, c2 INT PRIMARY KEY) PARTITION BY HASH(c1) PARTITIONS 2;
```

例 2 下面的语句在创建水平分区表 test2 时会自动创建一个局部索引(PRIMARY KEY 对应）和一个全局索引（UNIQUE 对应）。

```
CREATE TABLE test2(c1 INT, c2 INT primary key,c3 int unique) PARTITION BY HASH(c2) PARTITIONS 2;
```

二 显式地创建索引

当使用创建索引语句在水平分区表上创建索引时，若指定 GLOBAL 关键字，创建全局索引，否则创建为局部索引。

例 通过 CREATE INDEX 语句显式地创建全局和局部索引

```
//首先，创建表 t1
create table t1(c1 int, c2 int, c3 int)  partition by range(c1)
(
    partition p1 values less than(100),
    partition p2 values less than(200),
    PARTITION p3 VALUES EQU OR LESS THAN (MAXVALUE)
);
//其次，指定 GLOBAL 创建全局索引 idx1
create index idx1 on t1(c2) GLOBAL;
//再次，缺省 GLOBAL 创建局部索引 idx2
create index idx2 on t1(c3);
```

在水平分区表上创建全局唯一索引时，对索引键是否全部包含分区键没有使用限制，若

要创建局部唯一索引，则要求索引键必须包含所有的分区键。这是因为对于局部索引，每一个分区子表都会建立一个索引分区，负责索引分区子表的数据。由于每个索引分区只负责索引本分区上的数据，其他分区上的数据无法维护，只有当分区键都包含在索引键中时，才能对分区主表保证索引键唯一。

例 在 t1 表上的 c1 列上建立局部唯一索引，必须包含全部的分区键 C1

```
CREATE UNIQUE INDEX ind_c1 ON t1(c1);

CREATE UNIQUE INDEX ind_c13 ON t1(c1,c3);
```

15.5 维护水平分区表

创建水平分区表后，DM 提供了对分区表的修改，功能包括：

1. 增加分区：建立水平分区表后，可根据实际需要新增一个分区；
2. 删除分区：建立水平分区表后，可根据实际需要删除一个分区；
3. 合并分区：将相邻的两个范围分区合并为一个分区。合并分区通过指定两个分区名进行，将相邻的两个分区的数据进行合并，构建新的大分区。只能在范围和 LIST 分区上进行合并分区；
4. 拆分分区：将某一个范围分区拆分为相邻的两个分区。拆分分区时指定的常量表达式值必须是原范围分区的有效范围值。只能在范围和 LIST 分区上进行拆分分区；
5. 交换分区：将分区数据跟普通表数据进行交换，交换的两张表需要具有相同的结构。仅范围分区和 LIST 分区支持交换分区。不支持含有加密列的分区表交换分区。

在 DM 中，由于局部索引反映基础表的结构，因此当对表的分区和子分区进行修改操作时，会自动地对局部索引进行相应的修改。

15.5.1 增加分区

DM 支持用 ALTER TABLE ADD PARTITION 语句将新分区增加到最后一个现存分区的后

面。

例 范围分区表 callinfo 现需要记录用户的 2011 年的第一季度的通讯信息，那么，需要为 2011 年第一季度增加一个分区，并将其存储在表空间 ts5 中

```
ALTER TABLE callinfo
ADD PARTITION p5 VALUES LESS THAN ('2011-4-1') STORAGE (ON ts5);
```

对于范围分区，增加分区必须在最后一个分区范围值的后面添加，要想在表的开始范围或中间增加分区，应使用 SPLIT PARTITION 语句。

对于 LIST 分区，增加分区包含的离散值不能已存在于某个分区中。

例 为 LIST 分区表 sales 添加一个分区管理拉萨和呼和浩特的销售情况

```
ALTER TABLE sales
ADD PARTITION p5 VALUES ('拉萨', '呼和浩特') STORAGE (ON ts5);
```

可以对范围分区和 LIST 分区增加分区，存储选项 HASHPARTMAP 为 1 的 HASH 分区表也支持增加分区。增加分区不会影响分区索引，因为分区索引只是局部索引，新增分区仅是新增分区子表，并更新分区主表的分区信息，其他分区并不发生改变。

15.5.2 删除分区

DM 支持用 ALTER TABLE DROP PARTITION 语句将分区删除。

例 范围分区表 callinfo 现需要删除记录用户的 2011 年的第一季度的通讯信息，那么，只需删除 callinfo 的分区 p1 即可。

```
ALTER TABLE callinfo DROP PARTITION p1;
```

只能对范围分区和 LIST 分区进行删除分区，哈希分区不支持删除分区。跟增加分区一样，删除分区不会影响分区索引，因为分区索引只是局部索引，删除分区仅是删除分区子表，并更新分区主表的分区信息，其他分区并不发生改变。

15.5.3 交换分区

DM 支持用 ALTER TABLE EXCHANGE PARTITION 语句将分区数据跟普通表数据进行交换，交换的两张表需要具有相同的结构。仅范围分区和 LIST 分区支持交换分区。进行交换的两张表，如果包含加密列，对应的加密列要求加密信息完全一致。

假设上文提到的 callinfo 表是用于维护最近 12 个月的用户通话信息，超过 12 个月的订单需要迁移到该季度的通话信息历史表中，并且每一个季度都有一个相应的历史表。如果没有使用水平分区，需要较多的删除和插入操作，并产生大量的 redo 和 undo 日志。

如果使用分区表，如上文提到的 callinfo，只需使用交换分区即可完成以上功能。

例如，2011 年第二季度已到了，需删除 2010 年第二季度的通话记录，因此，可通过以下脚本来实现：

```
CREATE  TABLE  callinfo_2011Q2(
caller     CHAR(15),
callee     CHAR(15),
time       DATETIME,
duration  INT
);
//交换分区
ALTER TABLE callinfo EXCHANGE PARTITION p2 WITH TABLE callinfo_2011Q2;
//删除原分区
ALTER TABLE callinfo DROP PARTITION p2;
//新增分区，记录 2011 年第二季度通话记录
ALTER TABLE callinfo
ADD PARTITION p6 VALUES LESS THAN ('2011-7-1') STORAGE (ON ts2);
```

通过交换分区实现分区 p2 和新建表 callinfo_2011Q2 的数据交换，表 callinfo_2011Q2 将得到 2010 年第二季度的通话记录，而分区 p2 数据将被清空。交换分区采用数据字典信息交换的技术，几乎不涉及 IO 操作，因此效率非常高。

15.5.4 合并分区

要想将两个范围分区的内容融合到一个分区，就要使用 ALTER TABLE MERGE PARTITION 语句。如果分区的数据很少，或相对其他分区某些分区的数据量较少，导致 I/O 不均衡，就可以考虑使用合并分区。

例 可将 callinfo 的 2010 第 3 季度和第 4 季度合并成一个分区

```
ALTER TABLE callinfo MERGE PARTITIONS p3, p4 into partition p3_4;
```

仅范围分区表和 LIST 分区表支持合并分区。其中，合并的 RANGE 分区必须是范围相邻的两分区。

多级分区表进行 MERGE 合并的注意事项：

1. 仅支持一级子表类型为 RANGE、LIST；

2. 合并多级分区表中的一级子表时，该一级子表下的二级及以上层次子表按照级别分别判断：在创建分区表时当前层次是否指定了模板，若指定了模板，则按照模板创建当前层次子表；若未指定模板，则由系统自动合并为一个子表，子表名称为系统内部设置。RANGE 类型范围值为 MAXVALUE；LIST 类型范围值为 DEFAULT；

3. 不允许自定义二级及以上层次子表；

4. 不允许直接合并二级及以上层次子表。

合并分区会导致数据的重组和分区索引的重建，因此，合并分区可能会比较耗时，所需时间取决于分区数据量的大小。

15.5.5 拆分分区

ALTER TABLE 语句的 SPLIT PARTITION 子句被用于将一分区中的内容重新划分成两个新的分区。当一个分区变得太大以至于要用很长时间才能完成备份、恢复或维护操作时，就应考虑做分割分区的工作，还可以用 SPLIT PARTITION 子句来重新划分 I/O 负载。

例 将合并后的 p3_4 拆分为原两分区 p3 和 p4，分别记录 2010 年第三和第四季度的通话记录

```
ALTER TABLE callinfo SPLIT PARTITION p3_4 AT ('2010-9-30') INTO (PARTITION p3, PARTITION p4);
```

仅范围分区表和 LIST 分区表支持拆分分区。拆分分区另一个重要用途是作为新增分区的补充。通过拆分分区，可以对范围分区表的开始或中间范围添加分区。

多级分区表进行拆分 SPLIT 注意事项：

1. 仅支持一级子表类型为 RANGE、LIST；

2. 支持 SPLIT 为 2 个或多个子表；

3. 不允许自定义二级及以上层次子表；

4. SPLIT 产生的新分区二级及以上层次子表结构与被分隔子表保持一致，名称由系统内部定义。

拆分分区会导致数据的重组和分区索引的重建，因此，拆分分区可能会比较耗时，所需时间取决于分区数据量的大小。

15.6 水平分区表的限制

DM 水平分区表有如下限制条件：

1. 分区列类型必须是数值类型、字符类型、一般日期时间数据类型或标准时区类型；

2. 范围分区和哈希分区的分区键可以多个，每一层最多不超过 16 列；LIST 分区的分区键必须唯一；

3. 只有当分区键都包含在索引键中时，才能创建非全局唯一索引；
4. 水平分区表不支持临时表；
5. 不能在水平分区表上建立自引用约束；
6. 普通环境中，水平分区表的各级分区数的总和上限是 65535；MPP 环境下，水平分区表的各级分区总数上限取决于 INI 参数 MAX_EP_SITES，上限为 2 ^(16 - $\log_2$MAX_EP_SITES)。比如：当 MAX_EP_SITES 为默认值 64 时，分区总数上限为 1024；
7. 不允许对分区子表执行任何 DDL 操作；
8. 哈希分区支持重命名、增加/删除约束、设置触发器是否启用的修改操作；
9. 范围分区支持分区合并、拆分、增加、删除、交换、重命名、增加/删除约束、设置触发器是否生效操作；
10. LIST 分区支持分区合并、拆分、增加、删除、交换、重命名、增加/删除约束、设置触发器是否生效操作；
11. LIST 分区子表范围值个数与数据页大小和相关系统表列长度相关，存在以下限制：
    1） 4K 页，单个子表最多支持 120 个范围值；
    2） 8K 页，单个子表最多支持 254 个范围值；
    3） 16K\32K 页，单个子表最多支持 270 个范围值；
12. 对范围分区增加分区值必须是递增的，即只能在最后一个分区后添加分区。LIST 分区增加分区值不能存在于其他已存在分区；
13. 当分区数仅剩一个时，不允许删除分区；
14. 仅能对相邻的范围分区进行合并，合并后的分区名可为高分区名或新分区名；
15. 拆分分区的分区值必须在原分区范围中，并且分区名不能跟已有分区名相同；
16. 与分区进行分区交换的普通表，必须与分区表拥有相同的列及索引，但交换分区并

不会对数据进行校验，即交换后的数据并不能保证数据完整性，如 CHECK 约束；分区表与普通表创建的索引顺序要求一致；

17. 不能对水平分区表建立全局聚集索引或局部唯一函数索引；
18. 不能对分区子表单独建立索引；
19. 在未指定 ENABLE ROW MOVEMENT 的分区表上执行更新分区键，不允许更新后数据发生跨分区的移动，即不能有行迁移；
20. 不允许引用水平分区子表作为外键约束；
21. 多级分区表最多支持八层；
22. 多级分区表支持下列修改表操作：新增分区、新增列、删除列、删除表级约束、修改表名、设置与删除列的默认值、设置列 NULL 属性、设置列可见性、设置行迁移属性、启用超长记录、with delta、新增子分区、删除子分区、修改二级分区模板信息；
23. 水平分区表支持的列修改操作除了多级分区表支持的操作外，还支持：设置触发器生效/失效、修改列名、修改列属性、增加表级主键约束、删除分区、SPLIT/MERGE 分区和交换分区；
24. 间隔分区表的限制说明：
    1） 仅支持一级范围分区创建间隔分区；
    2） 只能有一个分区列，且分区列类型为数值类型、一般日期时间数据类型或标准时区类型；
    3） 对间隔分区进行 SPLIT，只能在间隔范围内进行操作；
    4） 被 SPLIT/MERGE 的分区，其左侧分区不再进行自动创建；
    5） 不相邻的间隔的分区，不能 MERGE；
    6） 表定义不能包含 MAXVALUE 分区；
    7） 不允许新增分区；

8）不能删除起始间隔分区；

9）间隔分区表定义语句显示到起始间隔分区为止；

10）自动生成的间隔分区，均不包含边界值；

11）间隔表达式只能为常量或日期间隔函数。日期间隔函数为：NUMTOYMINTERVAL、NUMTODSINTERVAL；数值常量可以为整型、DEC 类型；

12）MPP 下不支持间隔分区表；

13）当间隔分区的“间隔”设置得过大时，可能导致触发扩展分区时计算次数超过上限，新增间隔分区失败；

14）不支持创建 HUGE 间隔分区表。

第16章 管理列存储表

16.1 列存储简介

达梦数据库中，表的数据存储方式分为行存储和列存储。行存储是以记录为单位进行存储的，数据页面中存储的是完整的若干条记录；列存储是以列为单位进行存储的，每一个列的所有行数据都存储在一起，而且一个指定的页面中存储的都是某一个列的连续数据。

下面以基表（表 16.1）为例，对行、列存储进行对比。

表 16.1 基表

NAME 列	ID 列
SYSCLASSES	268435457
SYSCOLUMNS	268435458
SYSCONS	268435459
SYSDUAL	268435460
SYSGRANTS	268435461

下图展示了相同的表数据在行存储表与列存储表中的不同存储方式。

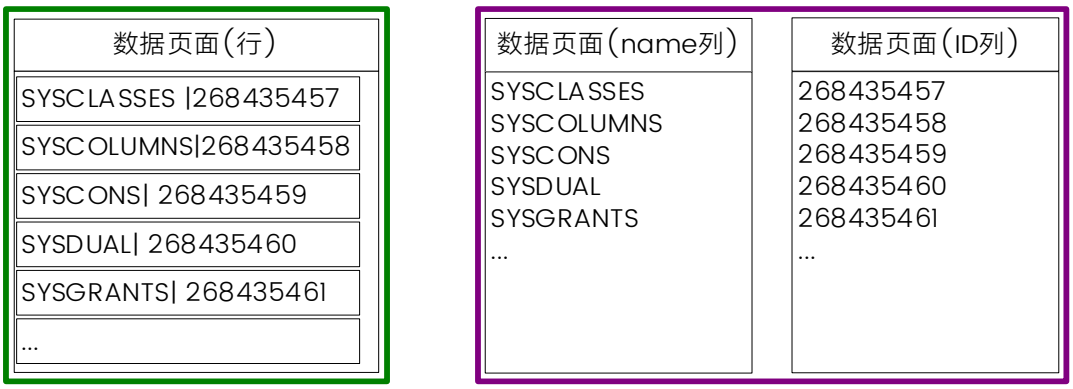


图 16.1 行存储（左）和列存储对比图（右）

Huge File System（简称 HFS）是达梦数据库实现的，针对海量数据进行分析的一种高效、简单的列存储机制。列存储表（也称为 HUGE 表）就是建立在 HFS 存储机制上的一种

表。

16.2 HUGE 表简介

HUGE 表是建立在混合表空间上的。HUGE 数据文件存储在混合表空间定义中指定的 HUGE 数据文件路径下，一个混合表空间最多可以添加 127 个 HUGE 数据文件路径，混合表空间的 HUGE 数据文件路径相关信息存储在动态视图 V$HUGE_TABLESPACE 中。

普通的表空间，数据是通过段、簇、页来管理的，并且以固定大小（4KB、8KB、16KB、32KB）的页面为管理单位；而混合表空间存储 HUGE 表是通过 HFS 存储机制来管理的，它相当于一个文件系统。为混合表空间指定一个 HUGE 数据文件路径，其实就是创建一个空的 HUGE 数据文件目录（系统中有一个默认的混合表空间 MAIN，其 HUGE 数据文件目录名为 HMAIN）。在创建一个 HUGE 表并插入数据时，数据库会在指定的混合表空间的 HUGE 数据文件目录下创建一系列的目录及文件。HFS 文件系统结构如下图所示：

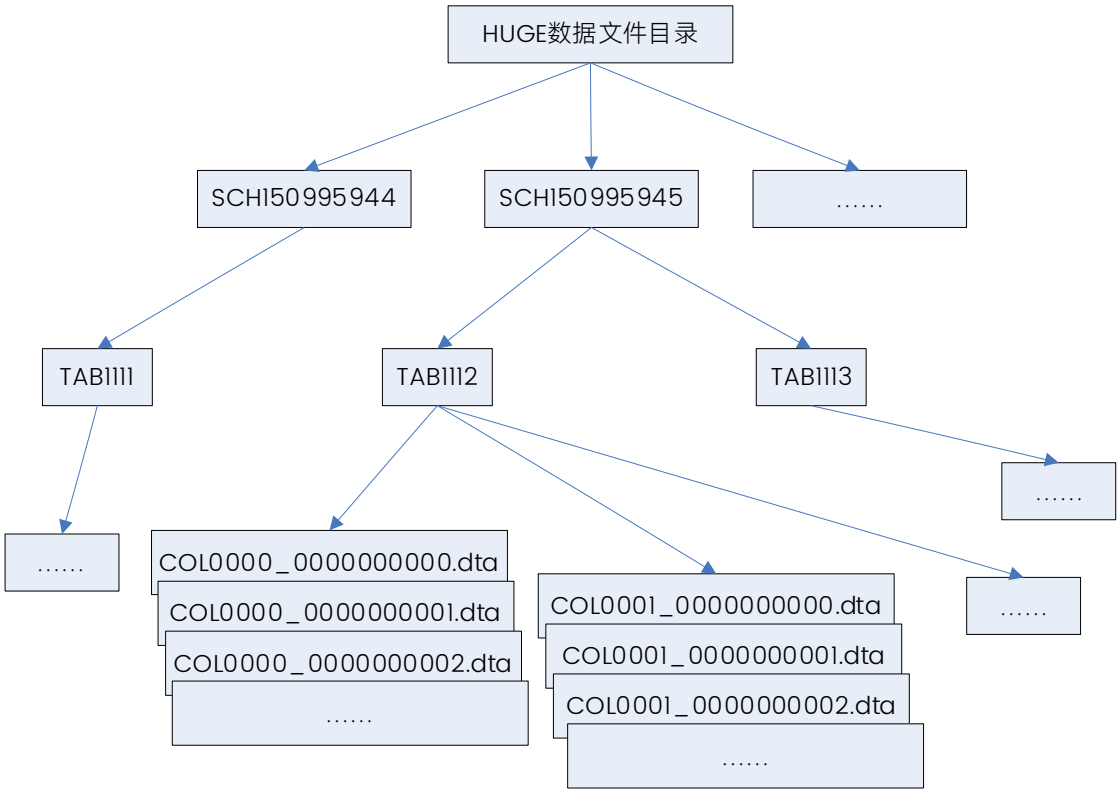


图 16.2 HFS 结构示意图

从上图可以看出，在 HUGE 数据文件目录下成功创建 HUGE 表，系统内部需要经过以下步骤：

首先，在 HUGE 数据文件目录下创建这个表对应的模式目录。目录名为“SCH+长度为 9 的 ID 号”组成的字符串，比如：SCH150995944。创建时如果这个目录已经存在，则无需重新创建。

其次，在模式目录下创建对应的表目录。表目录也是同样的道理，表目录名为“TAB+长度为4的ID号”组成的字符串，比如：TAB1112。表目录中存放的是这个表中所有的文件。

再次，在新创建表后插入数据时，每一个列对应一个以dta为后缀的文件，文件大小可以在建表时指定，默认为64MB，文件名为“COL+长度为4的列号+_+长度为10的文件号”。例如，在图16.2中，列号0000表示第1列，列号0001表示第2列……；0000000000表示第1个文件，0000000001表示第2个文件……。最初一个列只有一个文件即可，当随着数据量不断增长，一个文件已放不下之后，系统会自动创建新的文件来存储不断增长的数据。

对于一个文件，其内部存储是按照区来管理的，区是文件内部数据管理的最小单位，也是唯一的单位。一个区中，可以存储单列数据的行数是在创建表时指定的，一经指定，在这个表的生命过程就不能再修改。所以，对于定长数据，一个区的大小是固定的；而对于变长数据，一般情况下区大小都是不相同的。每一个区的开始位置及长度在文件内都是 4KB 对齐的。

HUGE 表的存储方式有以下几个优点：

1．同一个列的数据都是连续存储的，可以加快某一个列的数据查询速度；

2．连续存储的列数据，具有更大的压缩单元和数据相似性，可以获得远优于行存储的压缩效率，压缩的单位是区；

3．条件扫描借助数据区的统计信息进行精确过滤，可以进一步减少 IO，提高扫描效率；

4．允许建立二级索引；

5．支持以 ALTER TABLE 的方式添加或者删除 PK 和 UNIQUE 约束。

16.3 HUGE 表的分类

DM 支持两种类型的 HUGE 表：非事务型 HUGE 表和事务型 HUGE 表。本节将分别介绍上述两种 HUGE 表。

16.3.1 非事务型 HUGE 表

对非事务型 HUGE 表的增、删、改是直接对 HUGE 表进行写操作的，不写 UNDO 日志，不通过 BUFFER 缓存，直接操纵文件，速度快，但也因此导致不支持事务。另外，非事务型 HUGE 表中的 ROWID 是不固定的。

当非事务型 HUGE 表在操作过程中出现系统崩溃或者断电等问题时，因为修改时采取的是直接写的策略，所以有可能会出现数据不一致的问题。为了保证数据的一致性，在操作时可以适当地做一些日志来保证数据的完整性，完整性保证策略主要是通过数据的镜像来实现的，镜像的不同程度可以实现不同程度的完整性恢复。镜像文件是放在表目录中的以.mir 为扩展名的文件。DM 对于镜像提供三种方案：

1. LOG NONE：不做镜像。相当于不做数据一致性的保证，如果出错只能手动通过系统函数来修复表数据，当然速度是最快的，不需要额外的 IO，如果用户明确知道自己的环境不会出现问题可以采用该方案，效率最高。
2. LOG LAST：做部分镜像。但是在任何时候都只对当前操作的区做镜像，如果当前区的操作完成了，那么这个镜像也就失效了，可能会被下一个被操作区覆盖。这样做的好处是镜像文件不会太大，同时也可以保证数据是完整的。但有可能遇到的问题是：一次操作很多的情况下，有可能一部分数据做镜像已经完成，另一部分数据还没有来得及开始做的问题。如果用户能接受这个问题的话这个选择不失为最佳选

择，这也是系统默认的选择。

3. LOG ALL：全部做镜像。在操作过程中，所有被修改的区都会被记录下来，当一次操作修改的数据过多时，镜像文件有可能会很大，但好处是，能够保证操作完整性。比如，在操作过程中失败了，那么这个操作会完整的撤消，不存在 LOG LAST 方案中一部分修改部分还没修改的问题。

需要注意的是，若创建数据库时使用参数 HUGE_WITH_DELTA 的缺省值 1 ，则不支持创建非事务型 HUGE 表。

16.3.1.1 AUX 辅助表

对于每个 HUGE 表，相应地配备一个 AUX 辅助表来管理其数据。因为在 HUGE 数据文件（.dta 文件）中只存储了数据，辅助表用来管理以及辅助系统用户操作这些数据，AUX 辅助表是在创建 HUGE 表时系统自动创建的，表名为“表名$AUX”，如果该 HUGE 表为分区表，则针对每一个分区子表，其辅助表名为“子表名$AUX”。辅助表的表名长度不能大于 128 个字节。AUX 辅助表中每一条记录对应文件中的一个数据区，包括下面 15 列：

1．COLID：表示当前这条记录对应的区所在的列的列 ID 号；

2．SEC_ID：表示当前这个记录对应的区的区 ID 号，每一个区都有一个 ID 号，并且唯一；

3．FILE_ID：表示这个区的数据所在的文件号；

4．OFFSET：表示这个区的数据在文件中的偏移位置，4K 对齐；

5．COUNT：表示这个区中存储的数据总数（有可能包括被删除的数据）；

6．ACOUNT：表示这个区中存储的实际数据行数；

7．N_LEN：表示这个区中存储的数据在文件中的长度，4K 对齐的；

8．N_NULL：表示这个区中的数据中包括的 NULL 值的行数；

9．N_DIST：保留字段，无意义；

10．CPR_FLAG：表示这个区是否压缩；

11．ENC_FLAG：表示这个区是否加密；

12．CHKSUM：用来存储标记位；

13．MAX_VAL：表示这个区中的最大值，精确值；

14．MIN_VAL：表示这个区中的最小值，精确值；

15．SUM_VAL：表示这个区中所有值的和，精确值。

前面 7 列是用来控制数据存取的，根据这些信息就可以知道这个区的具体存储位置、长度及基本信息。后面 8 列都是用来对这个区进行统计分析的。其中，COLID 和 SEC_ID 的组合键为辅助表的聚集关键字。

16.3.2 事务型 HUGE 表

非事务型 HUGE 表在进行增、删、改时直接对 HUGE 表进行写操作，每次写操作需要至少对一个区进行 IO，导致 IO 量较大，且并发性能不高。

为此，DM 推出了事务型 HUGE 表，通过增加 RAUX、DAUX 和 UAUX 行辅助表，减少了事务型 HUGE 表增、删、改操作的 IO，提高效率，同时提高并行性能。事务型 HUGE 表支持 UNDO 日志，实现了事务特性。

16.3.2.1 RAUX 行辅助表

RAUX 行辅助表用于缓存向 HUGE 表插入的数据，表名为“HUGE 表名$RAUX”。如果该 HUGE 表为分区表，则针对每一个分区子表，其辅助表名为“子表名$RAUX”。辅助表的表名长度不能大于 128 个字节。

RAUX 行辅助表中内容对应于 HUGE 表中的最后一部分记录（不够存满一个数据区的）。当 RAUX 表中数据达到区大小时，将 RAUX 表中数据以列为单位写入各列对应的文件中，并清空 RAUX 表。RAUX 表与 HUGE 表结构相同，不论数据在哪个表中，每一行数据的 ROWID

固定不变。

16.3.2.2 DAUX 行辅助表

DAUX 行辅助表记录 HUGE 数据文件中被删除的数据，表名为“HUGE 表名$DAUX”。如果该 HUGE 表为分区表，则针对每一个分区子表，其辅助表名为“子表名$DAUX”。辅助表的表名长度不能大于 128 个字节。

DAUX 行辅助表中的一行数据对应 HUGE 表中一条或一段连续的被删除的数据，记录了被删除数据的起始行号和被删除数据的行数。删除 HUGE 表数据时，向 DAUX 行辅助表中插入对应删除数据的记录。

16.3.2.3 UAUX 行辅助表

UAUX 行辅助表记录 HUGE 表被更新的数据的新值，表名为“HUGE 表名$UAUX”。如果该 HUGE 表为分区表，则针对每一个分区子表，其辅助表名为“子表名$UAUX”。辅助表的表名长度不能大于 128 个字节。

UAUX 行辅助表中一条记录对应 HUGE 表中更新列的一个值，记录了更新的列号、行号和更新后的值的二进制格式。更新 HUGE 表数据时，操作转换为对 UAUX 表的插入操作或更新操作。

16.4 创建 HUGE 表

当用户确定自己要使用 HUGE 表的时候，首先需要在模式中创建新表，创建一个 HUGE 表需要有 CREATE TABLE 数据库权限，要想在其他用户的模式中创建新表需要有 CREATE ANY TABLE 数据库权限。

但是创建一个 HUGE 表时，如果不使用默认的表空间，则必须要先创建一个混合表空间，创建混合表空间以及 HUGE 表的语法请参考《DM8_SQL 语言使用手册》。

在创建 HUGE 表时，根据 WITH|WITHOUT DELTA 区分创建非事务型 HUGE 表还是事务型 HUGE 表。指定 WITH DELTA，创建事务型 HUGE 表；指定 WITHOUT DELTA，则创建非事务型 HUGE 表，缺省为 WITH DELTA。若需要创建非事务型 HUGE 表，在创建数据库时需要将参数 HUGE_WITH_DELTA 和 RLOG_GEN_FOR_HUGE 置为 0。

例 1 创建非事务型 HUGE 表 T1

```
CREATE HUGE TABLE T1 (A INT, B INT) STORAGE(WITHOUT DELTA);
```

例 2 创建事务型 HUGE 表 T2

```
CREATE HUGE TABLE T2 (A INT, B INT) STORAGE(WITH DELTA);
```

需要注意的是，当指定创建事务型 HUGE 表时，指定 HUGE 表镜像文件方案的选项 LOG NONE|LOG LAST|LOG ALL 失效。

另外，在创建表 HUGE 表时，可以指定表的存储属性，存储属性包括如下几个方面：

1. 区大小（一个区的数据行数）

区大小可以通过设置表的存储属性来指定，区的大小必须是 2 的 n 次方，如果不是则向上对齐。取值范围：1024 行~1024*1024 行。默认值为 65536 行。

例 1 创建 HUGE 表 test，指定区的大小为 2048，其它默认

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE(SECTION (2048));
```

例 2 创建 HUGE 表 test，所有列都采用默认值

```
CREATE HUGE TABLE test(name VARCHAR, sno INT);
```

2. 是否记录区统计信息，即在插入时是否记下其最大值最小值

关于这一点有一个原则，如果这个列经常用作查询条件，并且数据不是很均匀，或者基本是有序的，那么做统计信息是非常有用的，反之则可以不做统计。缺省情况下，为记录区统计信息。如果想不记录，可通过设置 STAT NONE 实现。

例 1 创建 HUGE 表 test，通过列存储属性指定统计信息属性（不记录区统计信息）

```
CREATE HUGE TABLE test(name VARCHAR STORAGE (STAT NONE), sno INT);
```

例 2 创建 HUGE 表 test，通过表存储属性指定统计信息属性（不记录区统计信息）

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE (STAT NONE);
```

3. 所属的表空间

创建 HUGE 表，需要通过存储属性指定其所在的表空间，不指定则存储于默认表空间 MAIN 中。HUGE 表指定的表空间只能是混合表空间，例如：TS1 为指定的混合表空间。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE (ON TS1);
```

4. 文件大小

创建 HUGE 表时还可以指定单个文件的大小，通过表的存储属性来指定，取值范围为 16M~1024*1024MB。不指定则默认为 64MB。文件大小必须是 2 的 n 次方，如果不是则向上对齐。

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) STORAGE (filesize(64));
```

5. 指定压缩级别

为特定列指定压缩级别，取值范围 0~10，分别代表不同的算法和级别。有两种压缩算法：SNAPPY 和 ZIP。10 采用 SNAPPY 算法轻量级方式压缩。2~9 采用 ZIP 算法压缩，2~9 代表压缩级别，值越小表示压缩比越低、压缩速率越快；值越大表示压缩比越高、压缩速度越慢。0 和 1 为快捷使用，默认值为 0。0 等价于 LEVEL 2；1 等价于 LEVEL 9。

例 创建 HUGE 表 test，指定 sno 列按照最大压缩比压缩

```
CREATE HUGE TABLE test(name VARCHAR, sno INT) COMPRESS LEVEL 1 (sno);
```

下面是一个综合的创建 HUGE 表的例子：

```
CREATE HUGE TABLE orders
    (
        o_orderkey              INT,
        o_custkey               INT,
        o_orderstatus               CHAR(1),
```

```
        o_totalprice        FLOAT,
        o_orderdate         DATE,
        o_orderpriority         CHAR(15),
        o_clerk             CHAR(15),
        o_shippriority      INT,
        o_comment               VARCHAR(79) STORAGE(stat none)
    )STORAGE(SECTION(65536) , FILESIZE(64), WITH DELTA, ON ts1) COMPRESS LEVEL 9 FOR
'QUERY HIGH'  (o_comment);
```

这个例子创建了一个名为 orders 的事务型 HUGE 表，orders 表的区大小为 65536 行，文件大小为 64MB，指定所在的混合表空间为 ts1，o_comment 列指定的区大小为不做统计信息，其它列（默认）都做统计信息，指定列 o_comment 列压缩类型为查询高压缩率，压缩级别为 9。

16.5 HUGE 表使用说明

HUGE 表与普通行表一样，可以进行增、删、改操作，操作方式也是一样的。使用 HUGE 表时应注意存在以下一些限制：

1．非事务型 HUGE 表的插入、删除与更新操作处理都不能进行回滚；

2．建 HUGE 表时仅支持定义 NULL、NOT NULL、UNIQUE 约束以及 PRIMARY KEY。后两种约束也可以通过 ALTER TABLE 的方式添加，但这两种约束不检查唯一性，用户需要确保实际数据符合约束，否则相关操作的结果可能不符合预期；

3．HUGE 不允许建立聚集索引，允许建立二级索引，不支持建位图索引，DPC 环境下不支持建立位图连接索引，其中 UNIQUE 索引不检查唯一性；

4．不支持 SPACE LIMIT（空间限制）；

5．不支持建立全文索引；

6．不支持使用自定义类型；

7．不支持引用约束；

8．不支持 IDENTITY 自增列；

9．不支持大字段列；

10．不支持建触发器；

11．不支持显式游标的修改操作；

12．非事务型 HUGE 表 PRIMARY KEY 主键约束或 UNIQUE 唯一约束不检查唯一性，对应的索引都为虚索引。事务型 HUGE 表的 PRIMARY KEY 主键约束或 UNIQUE 唯一约束的唯一性检查由 DM.INI 参数 HUGE_UNIQUE_CHECK 决定，当 HUGE_UNIQUE_CHECK=1 时，PRIMARY KEY 主键约束或 UNIQUE 唯一约束对应的索引均为实索引；HUGE_UNIQUE_CHECK=0 时，创建索引规则仍然与非事务型 HUGE 表保持一致；

13．不允许对分区子表设置 SECTION 和 WITH/WITHOUT DELTA；

14．对于非事务型 HUGE 表，若指定记录区统计信息，可能因为统计信息超长造成记录插入或更新失败；

15．当事务型 HUGE 表进行了较多增删改操作时，应对其进行数据重整操作，以提高性能；

16．HUGE 表不支持删除 NOT NULL 约束，删除 PK 约束时，会保留 NOT NULL 约束；

17．HUGE 表不支持查询 TRXID 和 PHYROWID 列；

18．DPC 环境下，HUGE 表加列时不支持以序列作为默认值；

19．在 DAUX 和 RAUX 表有数据的情况下，HUGE 表不允许修改 NULL 列为 NOT NULL 列，若要修改，需要先重整数据。

16.6 查看有关 HUGE 表的信息

1. 表定义

用户可以通过 SP_TABLEDEF 系统过程查看 HUGE 表的定义语句，更多介绍请参考 9.6.1 查看表定义。

2. 数据存储情况

HUGE 表有一个很好的特点就是有 AUX 辅助表，其中用户可以利用的信息很多，因为每一条记录对应一个区，所以可以查看每一个区的存储情况，每一个列的存储情况及每一个列中具有相同区 ID 的所有数据的情况等，还包括了很精确的统计信息，用户可以通过观察 AUX 辅助表中的信息对表进行一些相应的操作。

第17章 管理堆表

17.1 堆表简介

普通表都是以 B 树形式存放的，ROWID 都是逻辑的 ROWID，即从 1 一直增长下去。在并发情况下，每次插入过程中都需要逻辑生成 ROWID，这样影响了插入数据的效率；对于每一条数据都需要存储 ROWID 值，也会花费较大的存储空间。堆表就是基于上述两个理由而提出的。

简单地说，堆表是指采用了物理 ROWID 形式的表，即使用文件号、页号和页内偏移而得到 ROWID 值，这样就不需要存储 ROWID 值，可以节省空间。逻辑 ROWID 在插入或修改过程中，为了确保 ROWID 的唯一性，需要依次累加而得到值，这样就影响了效率，而堆表只需根据自己的文件号、页号和页内偏移就可以得到 ROWID，提高了效率。

普通表都是以 B 树形式存储在物理磁盘上，而堆表则采用一种“扁平 B 树”方式存储，结构如下图所示。

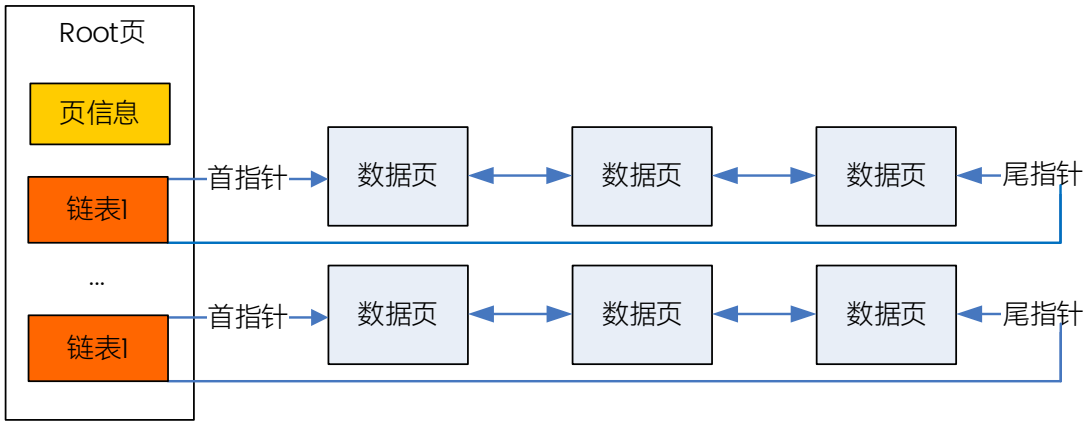


图 17.1 堆表存储方式

采用了物理 ROWID 形式的堆表，DM 服务器内部对聚集索引进行了调整，没有采用传统 B 树结构，取而代之的是“扁平 B 树”，数据页都是通过链表形式存储。为支持并发插入，扁平 B 树可以支持最多 128 个数据页链表(最多 64 个并发分支和最多 64 个非并发分支)，

在 B 树的控制页中记录了所有链表的首、尾页地址。

针对多个分支数，即存在多个链表时，并发分支和非并发分支的处理情况有所不同。对于并发分支，则各用户登录系统之后，会依据其事务 ID 号，随机选择一条链表来对堆表进行插入操作。因为是并发，所以可能出现多个用户选择了同一条分支的情况，此时用户需要排队，等待前一个用户插入结束并释放锁之后才能进行插入。在并发情况下，不同用户可以在不同的链表上进行插入，效率得到较大提升。对于非并发分支，则各用户按分支号从小到大选择一个可用的分支来进行插入。如果不存在空闲分支则需要等待其他用户释放锁之后才能进行插入。

17.2 创建堆表

堆表的创建有两种方式，一种是采用在配置文件 dm.ini 中设置参数，一种是在建表语句中显式指定堆表选项。

1. INI 参数方式

用户可以在配置文件中，添加 LIST_TABLE 参数：

1) 如果 LIST_TABLE = 1，则在未显式指定表是否为堆表或非堆表时，默认情况下创建的表为堆表；

2) 如果 LIST_TABLE = 0，则在未显式指定表是否为堆表或非堆表时，默认情况下创建的表为普通表形式。

用户可以在配置文件中，添加 LIST_TABLE_BRANCH 参数和 LIST_TABLE_NON_BRANCH 参数：

1) 如果 LIST_TABLE_BRANCH = 0，模式为 NOBRANCH；

2) 如果 LIST_TABLE_NON_BRANCH = 0 且 LIST_TABLE_BRANCH = N(N 非 0)，模式为 BRANCH N；

3) 如果 LIST_TABLE_NON_BRANCH = M(M 非 0)且 LIST_TABLE_BRANCH = N(N

非 0），模式为 BRANCH(N,M)。

2. SQL 语句显式指定

不管参数 LIST_TABLE 设置为何值，创建表时可以在 STORAGE 选项中指定需要创建的表形式， 与堆表创建形式相关的关键字有三个，分别是 NOBRANCH、BRANCH、CLUSTERBTR。

分支数目主要用于插入场景中。适当提高分支数量可有效地提升并发数据处理效率，但是分支过多又会加重扫描的负担，因此，合适的<BRANCH 数>和<NOBRANCH 数>值需用户根据实际情况来决定。<BRANCH 数>主要在 FLDR 导入和查询插入等并发、批量插入场景中使用；<NOBRANCH 数>在非并发、单行插入场景中使用。

详细语法形式参见《DM8_SQL 语言使用手册》。

1) NOBRANCH：如果指定为 NOBRANCH，则创建的表为堆表，并发分支个数为 0，非并发分支个数为 1；

2) BRANCH(N,M)：如果为该形式，则创建的表为堆表，并发分支个数为 N，非并发个数为 M；

3) BRANCH N：指定创建的表为堆表，并发分支个数为 N，非并发分支个数为 1；

4) CLUSTERBTR：创建的表为非堆表，即普通 B 树表。

例 下例创建的 LIST_TABLE 表有并发分支 2 个，非并发分支 4 个

```
CREATE TABLE LIST_TABLE(C1 INT) STORAGE(BRANCH (2,4));
```

17.3 堆表的限制

堆表由于其自身的特性，与普通表相比，也为自己增添了一些限制。堆表不支持以下功能：

1. 聚集索引

堆表采用了物理 ROWID，即通过文件号、页号和页内偏移直接生成该值。这样如果我

们知道了 ROWID 值，也就知道文件号、页号和页内偏移这些变量，就可以直接定位到某条记录，所以没有必要再为堆表创建聚集索引了。在创建堆表时，系统会默认创建聚集索引，该索引只是一个根页信息。显式建立聚集索引是不允许的，如果用户需要借助聚集索引主键对数据进行排序则不推荐使用堆表。

2. 列存储

由于列存储采用了不同方式对表进行物理存储， DM 服务器暂时不支持堆表的列存储。

17.4 维护堆表

堆表在进行数据扫描过程中，有着其先天的优势。如果知道了数据记录的 ROWID，则直接可以对 ROWID 进行解码，得到该记录的文件号、页号和页内偏移，也就得到了该记录。所以建议在经常查询的列上建立二级索引，这样在进行操作中，先通过二级索引找到记录 ROWID，就可以直接找到数据，效率有较大提高。

堆表虽然支持表的 ALTER 操作，但是建议轻易不要进行此类操作。对表进行 ALTER 操作，数据记录的 ROWID 有可能发生改变，这样每次进行 ALTER 操作，都可能进行索引的重建，需要花费较多的时间。

达梦服务器支持对堆表的备份与还原操作。还原数据时，B 树数据和二级索引可以同时被还原。

17.5 查看有关堆表的信息

用户可以通过 SP_TABLEDEF 系统过程查看堆表的定义语句，更多介绍请参考 9.6.1 查看表定义。

第18章 全文检索

现有的数据库系统，绝大多数是以结构化数据为检索的主要目标，因此实现相对简单。比如数值检索，可以建立一张排序好的索引表，这样速度可以得到提高。但对于非结构化数据，即全文数据，要想实现检索，一般都是采用模糊查询的方式实现的。这种方式不仅速度慢，而且容易将汉字错误切分，于是产生了全文检索技术。

全文检索技术是智能信息管理的关键技术之一，其主要目的就是实现对大容量的非结构化数据的快速查找。DM 实现了全文检索功能，并将其作为 DM 服务器的一个较独立的组件，提供更加准确的全文检索功能，较好地解决了模糊查询方式带来的问题。

18.1 全文检索概述

DM 全文检索根据已有词库建立全文索引，文本查询完全在索引上进行。全文索引为在字符串数据中进行复杂的词搜索提供了有效支持。

用户可以在指定表的文本列上创建和删除全文索引。创建全文索引后全文索引未插入任何索引信息。当用户填充全文索引时，系统才将定义了全文索引的文本列的内容进行分词，并根据分词结果填充索引。用户可以在进行全文索引填充的列上使用 CONTAINS 谓词进行全文检索。

DM 全文索引改进了原有的分词算法，为全文检索提供了更好的基础。在创建全文索引成功后，假设索引名为 INDEX_NAME，则系统会自动产生如下相关的辅助表（后面简称 I 表、P 表、N 表、D 表）：CTI$INDEX_NAME$I、CTI$INDEX_NAME$P、CTI$INDEX_NAME$N 和 CTI$INDEX_NAME$D，其表结构如表 18.1~18.4 所示。I 表主键为（WORD、FIRSTID、WID），用于保存分词结果，记录词的基本信息，通过该信息就可以快速地定位到该词的基表记录；P 表主键为（PND_DOCID），用于保存基表发生的增量数据变化，用于修改全文索引时的增量填充。N 表主键为（N_DOCID），用于保存原表记录 rowid 和新词条记录的

docid 的映射关系，N_DOCID 是 unique 的；D 表主键为（DOCID），保存了所有将被删除的 docid，被删除的 docid 即将不能通过全文索引查询到。另外，如果原表有自定义聚集主键，那么 P 表和 N 表会将该自定义聚集主键列"复制"到各自表中。

表 18.1 CTI$INDEX_NAME$I 表结构

序号	字段名	类型	长度	精度	刻度	说明
1	WID	BIGINT	8	19	0	词 ID
2	WORD	VARCHAR	64	64	0	词文本，相同的词重复存储
3	TYPE	SMALLINT	2	5	0	词类型
4	FIRSTID	BIGINT	8	19	0	开始 ROWID，用于范围查找
5	LASTID	BIGINT	8	19	0	结束 ROWID
6	COUNT	INTEGER	4	10	0	词所在的文档数（即 ROWID 的个数）
7	ID_INFO	VARBINARY	4000	4000	0	保存所在文档所有的 ROWID，连续存放

表 18.2 CTI$INDEX_NAME$P 表结构

序号	字段名	类型	长度	精度	刻度	说明
1	PND_ROWID	BIGINT	8	19	0	原表记录的 ROWID
2	PND_TYPE	SMALLINT	2	5	0	INS/UPD/DEL 类型
	cluster key	...	...	...	...	原表上的自定义聚集主键

表 18.3 CTI$INDEX_NAME$N 表结构

序号	字段名	类型	长度	精度	刻度	说明
1	N_DOCID	BIGINT	8	19	0	与原表 ROWID
2	N_ROWID	BIGINT	8	19	0	原表记录 ROWID
	cluster key	...	...	...	...	原表上的自定义聚集主键

表 18.4 CTI$INDEX_NAME$D 表结构

序号	字段名	类型	长度	精度	刻度	说明
1	DOCID	BIGINT	8	19	0	待删除词的 DOCID

例 对示例库 bookshop 中的 address 表的 address1 列创建全文索引，创建的 SQL 语句如下：

```
CREATE CONTEXT INDEX cti_address ON person.address (address1) LEXER DEFAULT_LEXER;
```

执行成功之后，该索引信息会保存到 ctisys 模式下的 SYSCONTEXTINDEXES 系统表中，4 个自动生成的辅助表 CTI$CTI_ADDRESS$I 、 CTI$CTI_ADDRESS$P 、CTI$CTI_ADDRESS$N 和 CTI$CTI_ADDRESS$D 信息保存在 SYSOBJECTS 系统表中。这四张表由服务器自动维护，用户可以查询，但不可以直接修改表内容。

DM 全文索引创建的过程中，用户可以为分词器定义分词参数，即控制分词器分词的数量，包括 5 种分词参数：

1. CHINESE_LEXER，中文最少分词；
2. CHINESE_VGRAM_LEXER，机械双字分词；
3. CHINESE_FP_LEXER，中文最多分词；
4. ENGLISH_LEXER，英文分词；
5. DEFAULT_LEXER，默认分词，为中文最少分词。

指定中文分词参数可以切分英文，但是指定英文分词参数不可以切分中文，所以使用英文分词算法对中文文本进行分词时，分词结果将为空。

中文最多分词可以将存在二义性的词划分出来，例如：“和服装”会生成“和”、“和服”、“服”、“服装”和“装”。那么在查询其中任何一个词的时候都可以检索到该文本。与之相对应的是最少分词，其消除了存在二义性的词，即前面举例的文本分词的结果为“和”、“服装”两个词，这样可以减少大量冗余词的存储，能进行更准确的匹配。英文分词即根据英文分隔符的分词，对

超过 32 个字节的英文单词进行了拆分处理。

全文检索的中文分词依赖系统词库，该词库是只读的，不允许修改。

18.2 创建全文索引

在 DM 中，全文索引必须在基表上定义，而不能在系统表、视图、临时表、列存储表、外部表上定义。同一列只能创建一个全文索引。DM 定义全文索引时，不需要在表上先建立主关键字索引。创建全文索引的列类型可为 CHAR、CHARACTER、VARCHAR、VARCHAR2、LONGVARCHAR、TEXT 或 CLOB，例如上节对 address 表创建的全文索引列为 VARCHAR 类型。DM 的全文索引现只支持简体中文、英文语言。较为详细的语法介绍可参考《DM8_SQL 语言使用手册》。

18.3 更新全文索引

全文索引本质是借助辅助关系表存储索引数据。当对基表执行数据更新操作后，数据并不会立刻填充到辅助表中，需要用户主动同步数据用以更新全文索引，未同步到辅助表中的记录将无法通过全文索引查询到结果。

全文索引的更新包括两种方式：完全更新和增量更新。

1. **完全更新**

删除原有的全文索引，对基表进行全表扫描，逐一重构索引信息。在创建全文索引成功后，需完完全更新全文索引才可以执行有效的全文检索。完全更新全文索引没有次数限制，用户可以根据需要在增量更新或者是完全更新失败以及发生系统故障后都可以执行完全更新全文索引。另外，完全更新由于完全丢弃辅助表已有数据，重新开始对基表数据进行分词并填充到辅助表，因此服务器允许这种情况下更改分词算法。

例 执行如下 SQL 可以对 address 表的全文索引进行完全更新：

```
ALTER CONTEXT INDEX cti_address ON person.address REBUILD;
```

2. 增量更新

在上一次完全更新之后，表数据发生了变化，就可以使用增量更新来更新索引。增量更新只对新发生改变的数据进行重新分词生成索引信息，所以其代价是较小的。此时，也可以使用完全更新达到同样的更新索引的效果，但代价较大。

对全文索引的基表进行数据的增删改操作时，会将该数据的操作类型和 ROWID 信息保存到 CTI$INDEX_NAME$P 表中。

例 执行如下 SQL 可以对 address 表的全文索引进行增量更新：

```
ALTER CONTEXT INDEX cti_address ON person.address INCREMENT;
```

DM 在执行更新全文索引的过程中如果在更新过程中发生任何错误，服务器将回滚到最近一次事务提交成功时的状态。

无论是完全更新还是增量更新，在 DMDSC 环境下更新全文索引前，需要手动将执行码目录下的 SYSWORD.UTF8.LIB 文件复制到库所在的 DMASM 路径下，否则会导致全文索引词库加载错误。

增量更新还有一种特殊情况，实时自动增量更新，即在创建全文索引时指定同步方式为"SYNC TRANSACTION"，此种方式不需要用户手动更新全文索引，而是在每次事务提交时由服务器自动完成增量更新动作。

18.4 执行全文检索

全文检索采用的分词算法与全文索引数据同步时采用的分词算法保持一致，用户可以通过系统表"CTISYS.SYSCONTEXTINDEXES"查询"WSEG_TYPE"得知全文索引的分词算法。全文检索首先对查询关键字或者句子进行分词，并将分词结果在 I 表中进行匹配，然后将匹配结果返回。DM8 支持对中英文关键字或者句子的布尔查询。

全文检索支持的检索类型有：

1. 支持英文单词的检索（单词不区分大小写）；
2. 支持全角英文的检索；
3. 支持简体中文词语的检索；
4. 支持常见单个汉字词的检索；
5. 支持简体中文长句子的检索；
6. 支持中英文混合的检索。

全文检索支持的检索方式有：

1. 在 CONTAINS 谓词内支持 AND、OR 和 AND NOT 的短语查询组合条件。

   例 查询 address 表中地址在洪山区太阳城的记录：

```
SELECT * FROM person.address WHERE CONTAINS(address1, '洪山区' AND '太阳城');
```

2. 全文检索支持单词或者句子的检索。

   例 查询 address 表中地址在江汉区发展大道的记录：

```
SELECT * FROM person.address WHERE CONTAINS(address1, '江汉区发展大道');
```

3. 不支持模糊方式的全文检索，例如查询“江汉区*”。
4. 检索条件子句可以和其他子句共同组成 WHERE 的检索条件。

   例 查询 address 表中地址在武汉市洪山区的记录

```
SELECT * FROM person.address WHERE CONTAINS(address1, '洪山区') AND city LIKE '%武汉市%';
```

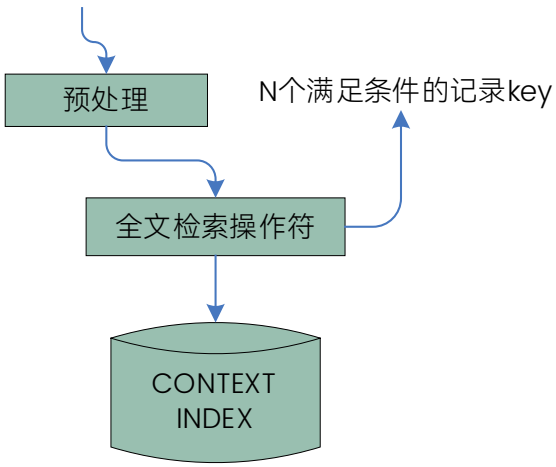


图 18.1 全文查询

如上图所示，DM 全文检索时对查询的文本做了预处理，首先根据词文本进行最少分词，并依据词频信息找出查询关键字，然后对这些关键字进行优先级排序，这样可以在第一次检索时找出所需文本的最小结果集，然后在该结果集的基础上再进行依次筛选，最终获得查询文本，这样能较大地提高查询性能。

18.5 删除全文索引

删除全文索引时，数据字典中相应的索引信息和全文索引内容都会被删除。删除索引有两种方式：使用语句删除和当模式对象发生改变时系统自动删除。第一种方式仅删除指定的基表的全文索引信息；第二种方式会删除模式对象（如数据库、基表等）上的所有全文索引，也包括与全文索引相关的动态表。全文索引一旦删除就不能回滚，基于该表的全文检索就会失败，只能通过重新构建获得新的全文索引。

例 删除 address 表，则会将表的全文索引 cti_address 也删除，包括与全文索引相关的动态表。

```
DROP CONTEXT INDEX cti_address ON person.address;
```

删除成功后，查询 ctisys 模式下的 SYSCONTEXTINDEXES 系统表会发现 cti_address 全文索引已删除。

第19章 管理事务

数据库是一个共享资源，可以被大量应用程序所共享。这些应用程序可以串行运行，但在绝大多数情况下，为了有效地利用数据库资源，多个应用程序会并发地访问数据库，这就是数据库的并发操作。此时，如果不对并发操作进行控制，则会存取不正确的数据，或破坏数据库数据的一致性。DM 数据库通过事务管理相关技术，有效解决了上述问题。本章阐述了事务的定义，并讲解如何使用事务来管理 DM 数据库。

19.1 事务简介

数据库事务是指作为单个逻辑工作单元的一系列操作的集合。一个典型的事务由应用程序中的一组操作序列组成，对于 DM 数据库来说，第一次执行 SQL 语句时，隐式地启动一个事务，以 COMMIT 或 ROLLBACK 语句/方法显式地结束事务。另外，在执行 DDL 前，DM 数据库会自动把前面的操作进行提交，DDL 前面的操作作为一个完整的事务结束，DDL 语句本身所属事务则根据“DDL_AUTO_COMMIT”配置参数决定是否隐式地提交（注意，无论 DDL_AUTO_COMMIT 参数如何设置，ALTER TABLESPACE 和 ALTER USER 操作总是自动提交的）。

COMMIT 操作会将该语句所对应事务对数据库的所有更新持久化（即写入磁盘），数据库此时进入一个新的一致性状态，同时该事务成功地结束。ROLLBACK 操作将该语句所对应事务对数据库的所有更新全部撤销，把数据库恢复到该事务初启动前的一致性状态。

我们以一个模拟的银行转账业务为例，假设一个银行客户 A 需要转 5000 元给 B，其具体业务步骤如下：

1. 从 A（ID 为 5236）的储蓄账户扣除 5000 元；

```
UPDATE account SET balance=balance-5000 WHERE id=5236;
```

2. 将 B（ID 为 5237）的储蓄账户增加 5000 元；

```
UPDATE account SET balance = balance +5000 WHERE id=5237;
```

3. 在业务日志中记录此次业务;

```
INSERT INTO trans_log VALUES(log_seq.NEXTVAL, 5236, 5237, 5000);
```

4. 提交事务。

```
COMMIT;
```

在上面的例子中，需要考虑两种情况：如果三条 SQL 语句全部正常执行，使账户间的平衡得以保证,那么此事务中对数据的修改就可以应用到数据库中;如果发生诸如资金不足、账号错误、硬件故障等问题，导致事务中一条或多条 SQL 语句不能执行，那么整个事务必须被回滚掉才能保证账户间的平衡。

DM 数据库提供了足够的事务管理机制来保证上面的事务要么成功执行，所有的更新都会写入磁盘，要么所有的更新都被回滚，数据恢复到执行该事务前的状态。无论是提交还是回滚，DM 保证数据库在每个事务开始前、结束后是一致的。

为了提高事务管理的灵活性，DM 数据库还提供了设置保存点（SAVEPOINT）和回滚到保存点的功能。保存点提供了一种灵活的回滚，事务在执行中可以回滚到某个保存点，在该保存点以前的操作有效，而以后的操作被回滚掉。可以使用 SAVEPOINT SAVEPOINT_NAME 命令创建保存点，使用 ROLLBACK TO SAVEPOINT SAVEPOINT_NAME 命令来回滚到保存点 SAVEPOINT_NAME。

19.2 事务特性

事务必须具备什么属性才是一个有效的事务呢？一个逻辑工作单元必须表现出四种属性，即原子性、一致性、隔离性和持久性，这样才能成为一个有效的事务。DM 数据库提供了各种机制以保证事务满足以上各种要求。

19.2.1 原子性

事务的原子性保证事务包含的一组更新操作是原子不可分的，也就是说这些更新操作是一个整体，对数据库而言全做或者全不做，不能部分地完成。这一性质即使在系统崩溃之后仍能得到保证，在系统崩溃之后将进行数据库恢复，用来恢复和撤销系统崩溃时处于活动状态的事务对数据库的影响，从而保证事务的原子性。系统对磁盘上的任何实际数据的修改之前都会将修改操作本身的信息记录到磁盘上。当发生崩溃时，系统能根据这些操作记录当时该事务处于何种状态，以此确定是撤销该事务所做出的所有修改操作，还是将修改的操作重新执行。

19.2.2 一致性

数据一致性是指表示客观世界同一事务状态的数据，不管出现在何时何处都是一致的、正确的、完整的。换句话说，数据一致性是任何点上保证数据以及内部数据结构的完整性，如 B 树索引的正确性。

一致性要求事务执行完成后，将数据库从一个一致状态转变到另一个一致状态。它是一种以一致性规则为基础的逻辑属性，例如在转账的操作中，各账户金额必须平衡，这一条规则对于程序员而言是一个强制的规定。事务的一致性属性要求事务在并发执行的情况下事务的一致性仍然满足。

19.2.3 隔离性

事务是隔离的，意味着每个事务的执行效果与系统中只有该事务的执行效果一样，也就是说，某个并发事务所做的修改必须与任何其他的并发事务所做的修改相互隔离。这样，只有当某个值被一个事务修改完并提交后才会影响到另一个事务。事务只会识别另一并发事务修改之前或者修改完成之后的数据，不会识别处于这中间状态的数据。事务的隔离行为依赖

于指定的隔离级别。隔离级别的介绍请参考 19.7 事务隔离级。

19.2.4 持久性

持久性是指一个事务一旦被提交，它对数据库中数据的改变就是永久性的，接下来的其他操作和数据库故障不应该对其有任何影响。即一旦一个事务提交，DBMS 保证它对数据库中数据的改变应该是永久性的。如果 DM 数据库或者操作系统出现故障，那么在 DM 数据库重启的时候，数据库会自动恢复。如果某个数据驱动器出现故障，并且数据丢失或者被损坏，可以通过备份和联机重做日志来恢复数据库。需要注意的是，如果备份驱动器也出现故障，且系统没有准备其他的可靠性解决措施，备份就会丢失，那么就无法恢复数据库了。

19.3 提交事务

提交事务就是提交事务对数据库所做的修改，将从事务开始的所有更新保存到数据库中，任何更改的记录都被写入日志文件并最终写入到数据文件，同时提交事务还会释放由事务占用的资源，如锁。如果提交时数据还没有写入到数据文件，DM 数据库后台线程会在适当时机（如检查点、缓冲区满）将它们写入。

具体说来，在一个修改了数据的事务被提交之前，DM 数据库进行了以下操作：

1. 生成回滚记录，回滚记录包含了事务中各 SQL 语句所修改的数据的原始值；
2. 在系统的重做日志缓冲区中生成重做日志记录，重做日志记录包含了对数据页和回滚页所进行的修改，这些记录可能在事务提交之前被写入磁盘；
3. 对数据的修改已经被写入数据缓冲区，这些修改也可能在事务提交之前被写入磁盘。

已提交事务中对数据的修改被存储在数据库的缓冲区中，它们不一定被立即写入数据文件内。DM 数据库自动选择适当的时机进行写操作以保证系统的效率。因此写操作既可能发生在事务提交之前，也可能在提交之后。

当事务被提交之后，DM 数据库进行以下操作：

1. 将事务所有更改的记录写入日志文件并最终写入到数据文件；

2. 释放事务上的所有锁，将事务标记为完成；

3. 返回提交成功消息给请求者。

在 DM 数据库中还存在有 3 种事务模式：自动提交模式、手动提交模式和隐式提交模式。

除了以上三种事务提交模式之外，事务提交还存在一种优化策略：事务组提交（GROUP COMMIT）。

19.3.1 自动提交模式

除了命令行交互式工具 Disql 外，DM 数据库缺省都采用自动提交模式。用户通过 DM 数据库的其他管理工具、编程接口访问 DM 数据库时，如果不手动/编程设置提交模式，所有的 SQL 语句都会在执行结束后提交，或者在执行失败时回滚，此时每个事务都只有一条 SQL 语句。

在 Disql 中，用户也可以通过执行如下语句来设置当前会话为自动提交模式：

```
SET AUTOCOMMIT ON;
```

19.3.2 手动提交模式

在手动提交模式下，DM 数据库用户或者应用开发人员明确定义事务的开始和结束，这些事务也被称为显式事务。在 Disql 中，没有设置自动提交时，就是处于手动提交模式，此时 Disql 连接到服务器后第一条 SQL 语句或者事务结束后的第一条语句就标记着事务的开始，可以执行 COMMIT 或者 ROLLBACK 来提交或者回滚事务。

19.3.3 隐式提交

在手动提交模式下，当遇到 DDL 语句时，DM 数据库会在执行 DDL 语句之前自动提交

前面的事务，然后开始一个新的事务执行 DDL 语句，这种事务提交被称为隐式提交。需要注意的是，如果 DDL 语句解析失败，则不会自动提交前面的事务，但在 DMDPC 环境下，节点会将 DDL 语句转发给 MP 进行解析执行，在转发之前便会自动提交前面的事务，因此在 DMDPC 环境下，DDL 语句解析失败与否不会影响前面事务的提交。

DM 数据库在遇到以下 SQL 语句时自动提交前面的事务：

1. CREATE；
2. ALTER；
3. TRUNCATE；
4. DROP；
5. GRANT；
6. REVOKE；
7. 审计设置语句。

19.3.4 事务组提交

事务组提交（GROUP COMMIT）是一种数据优化策略，它允许数据库在一次磁盘操作中提交多个事务，而不是为每一个事务单独执行一次磁盘 I/O 操作。

COMMIT_WRITE=IMMEDIATE,WAIT 时，事务提交动作响应用户之前要求事务修改产生的 REDO 日志写入联机日志文件。在高并发的短事务提交场景，会频繁触发 REDO 日志写入动作，高频向联机日志文件写入较小数据量的 REDO 日志，影响系统整体性能。事务组提交功能通过将多个事务的提交操作组合在一起，减少联机日志的写入次数，从而提升数据库整体性能。

DM 可通过 INI 参数 COMMIT_BATCH 控制是否开启事务组提交以及触发一次写盘动作提交的事务数。一个事务提交不会马上触发 REDO 日志写盘动作，而是等待其他事务提交后合并写入。当连续提交的事务数量达到 COMMIT_BATCH 的配置值时，可以触发联机日

志写入动作。

事务组提交还可以通过设置 COMMIT_BATCH_TIMEOUT 的取值在事务提交数量长时间未达到 COMMIT_BATCH 的设定值的情况下进行超时强制写盘操作。

19.4 回滚事务

回滚事务是撤消该事务所做的任何更改。回滚有两种形式：DM 数据库自动回滚，或者通过程序/ROLLBACK 命令手动回滚。除此之外，DM 还支持回滚到保存点和语句级回滚。

在事务回滚过程中如果访问到损坏的数据页，则会将当前事务信息登记到系统表 SYSCRASHTRXS 中，SYSCRASHTRXS 中的事务会被当做活动事务，保证其他事务不会访问到故障事务修改后的记录。

事务回滚过程可能失败，比如遇到存储空间不足、对象已经被修改、表空间被删除等，事务回滚失败，不再继续回滚。此时会将事务状态调整为回滚挂起，事务保留在系统中，事务持有的锁等资源也保留，可能会导致后续对回滚挂起事务中涉及的表的修改操作因表被封锁导致失败报错锁超时，通过 V$TRX 中 status 字段可以查询到此类事务。系统定时每 5 分钟发起任务尝试对这些回滚挂起事务进行自动回滚操作，也可以通过 SUSPEND_TRX_PROCESS(trxid)手动继续回滚，回滚成功后即可恢复正常操作。

19.4.1 自动回滚

若事务运行期间出现连接断开，DM 数据库都会自动回滚该连接所产生的事务。回滚会撤消事务执行的所有数据库更改，并释放此事务使用的所有数据库资源。DM 数据库在恢复时也会使用自动回滚。例如在运行事务时服务器突然断电，接着系统重新启动，DM 数据库就会在重启时执行自动恢复。自动恢复要从事务重做日志中读取信息以重新执行没有写入磁盘的已提交事务，或者回滚断电时还没有来得及提交的事务。

19.4.2 手动回滚

一般来说，在实际应用中，当某条 SQL 语句执行失败时，用户会主动使用 ROLLBACK 语句或者编程接口提供的回滚函数来回滚整个事务，避免不合逻辑的事务污染数据库，导致数据不一致。如果发生错误后确实只用回滚事务中的一部分，则需要用到回滚到保存点的功能。

19.4.3 回滚到保存点

除了回滚整个事务之外，DM 数据库的用户还可以部分回滚未提交事务，即从事务的最末端回滚到事务中任意一个被称为保存点的标记处。用户在事务内可以声明多个被称为保存点的标记，将一个大事务划分为几个较小的片断。之后用户在对事务进行回滚操作时，就可以选择从当前执行位置回滚到事务内的任意一个保存点。例如用户可以在一系列复杂的更新操作之间插入保存点，如果执行过程中一个语句出现错误，用户可以回滚到错误之前的某个保存点，而不必重新提交所有的语句。当事务被回滚到某个保存点后，DM 数据库将释放被回滚语句中使用的锁。其他等待“被锁资源”的事务就可以继续执行，需要更新“被锁数据行”的事务也可以继续执行。

将事务回滚到某个保存点的过程如下：

1. 只回滚保存点之后的语句；
2. 保留该保存点，其后创建的保存点都被清除；
3. 释放此保存点之后获得的所有锁，保留该保存点之前的锁。

被部分回滚的事务依然处于活动状态，可以继续执行。DM 数据库用户可以使用 SAVEPOINT SAVEPOINT_NAME 命令创建保存点，使用 ROLLBACK TO SAVEPOINT SAVEPOINT_NAME 命令来回滚到保存点 SAVEPOINT_NAME。

19.4.4 语句级回滚

如果在一个 SQL 语句执行过程中发生了错误，此语句对数据库产生的影响将被回滚，回滚后就如同此语句从未执行过，这种操作被称为语句级回滚。语句级回滚只会使此语句所做的数据修改无效，不会影响此语句之前所做的数据修改。

当 INI 参数 ROLL_ON_ERR 为缺省值 0 时，在 SQL 语句执行过程中发生的错误，将会导致语句级回滚，例如违反唯一性、死锁（访问相同数据而产生的竞争）、运算溢出等。在 SQL 语句解析的过程中发生错误（例如语法错误），由于未对数据产生任何影响，因此不会产生语句级回滚。

19.5 事务锁定

DM 数据库支持多用户并发访问、修改数据，有可能出现多个事务同时访问、修改相同数据的情况。若对并发操作不加控制，就可能会访问到不正确的数据，破坏数据的一致性和正确性。

封锁机制是实现数据库并发控制的一个非常重要的技术。当事务在对某个数据库对象进行操作前，需要先对其封锁。封锁后事务就对该数据库对象有了一定的控制，在该事务释放锁之前，其他的事务不能对此数据库对象进行相应操作。

本节将详细介绍 DM 数据库中的事务锁定相关功能，包括锁的分类以及如何查看锁等。

19.5.1 锁模式

锁模式指定并发用户如何访问锁定资源。DM 数据库使用四种不同的锁模式：共享锁、排他锁、意向共享锁和意向排他锁。

1. **共享锁**

共享锁（Share Lock，简称 S 锁）用于读操作，防止其他事务修改正在访问的对象。这

种封锁模式允许多个事务同时并发读取相同的资源，但是不允许任何事务修改这个资源。

2. 排他锁

排他锁（Exclusive Lock，简称 X 锁）用于写操作，以独占的方式访问对象，不允许任何其他事务访问被封锁对象；防止多个事务同时修改相同的数据，避免引发数据错误；防止访问一个正在被修改的对象，避免引发数据不一致。一般在修改对象定义时使用。

3. 意向锁

意向锁（Intent Lock）用于读取或修改被访问对象数据时使用，多个事务可以同时对相同对象上意向锁，DM 支持两种意向锁：

1） 意向共享锁（Intent Share Lock，简称 IS 锁）：一般在只读访问对象时使用；

2） 意向排他锁（Intent Exclusive Lock，简称 IX 锁）：一般在修改对象数据时使用。

四种锁模式的相容矩阵如下表所示，其中“Y”表示相容；“N”表示不相容。如表中第二行第二列为“Y”，表示如果某个事务已经加了 IS 锁时，其他事务还可以继续添加 IS 锁，第二行第五列为“N”，表示如果某个事务已经加了 IS 锁时，其他事务不能添加 X 锁。

表 19.1 不同事务之间锁的相容矩阵

已加 \ 待加	IS	IX	S	X
IS	Y	Y	Y	N
IX	Y	Y	N	N
S	Y	N	Y	N
X	N	N	N	N

19.5.2 锁粒度

按照封锁对象的不同，锁可以分为 TID 锁和对象锁。

1. TID 锁

TID 锁以事务号为封锁对象，为每个活动事务生成一把 TID 锁，代替了其他数据库行锁的功能，防止多个事务同时修改同一行记录。DM 实现的是行级多版本，每一行记录隐含一个 TID 字段，用于事务可见性判断。

执行 INSERT、DELETE、UPDATE 操作时，设置事务号到 TID 字段。这相当于隐式地对记录上了一把 TID 锁，INSERT、DELETE、UPDATE 操作不再需要额外的行锁，避免了大量行锁对系统资源的消耗。只有多个事务同时修改同一行记录时，才会产生新的 TID 锁。例如，当事务 T1（事务号为 TID1）试图修改某行数据，而该行数据正在被另一个事务 T2（事务号为 TID2）修改，此时事务 T1 会生成一个新的 TID 锁，其锁对象为事务号 TID2，而非事务 T2。

同时多版本写不阻塞读的特性，SELECT 操作已经消除了行锁，因此 DM 中不再有行锁的概念。

2. 对象锁

对象锁是 DM 新引入的一种锁，通过统一的对象 ID 进行封锁，将对数据字典的封锁和表锁合并为对象锁，以达到减少封锁冲突、提升系统并发性能的目的。我们先看一下通常数据字典锁和表锁各自应承担的功能：

1） 数据字典锁：用来保护数据字典对象的并发访问，解决 DDL 并发和 DDL/DML 并发问题，防止多个事务同时修改同一个对象的字典定义，确保对同一个对象的 DDL 操作是串行执行的。并防止一个事务在修改字典定义的同时，另外一个事务修改对应表的数据。

2） 表锁：表锁用来保护表数据的完整性，防止多个事务同时采用批量方式插入、更新一张表，防止向正在使用 FAST LOADER 工具装载数据的表中插入数据等，保证这些优化后数据操作的正确性。此外，表锁还有一个作用，避免对存在未提交修改的表执行 ALTER TABLE、TRUNCATE TABLE 操作。

为了实现与数据字典锁和表锁相同的封锁效果，从逻辑上将对象锁的封锁动作分为四类：

a）独占访问（EXCLUSIVE ACCESS），不允许其他事务修改对象，不允许其他事务访问对象，使用 X 方式封锁

b）独占修改（EXCLUSIVE MODIFY），不允许其他事务修改对象，允许其他事务共享访问对象，使用 S + IX 方式封锁

c）共享修改（SHARE MODIFY），允许其他事务共享修改对象，允许其他事务共享访问对象，使用 IX 方式封锁

d）共享访问（SHARE ACCESS），允许其他事务共享修改对象，允许其他事务共享访问对象，使用 IS 方式封锁

3. 显式锁定表

用户可以根据自己的需要显式的对表对象或视图进行封锁。显式锁定表的语法如下:

```
LOCK TABLE <table_name|view_name> IN <lock_mode> MODE [NOWAIT];
```

lock_mode 是锁定的模式，可以选择的模式有 INTENT SHARE（意向共享）、INTENT EXCLUSIVE（意向排他）、SHARE（共享）和 EXCLUSIVE（排他），其含义分别如下:

1） 意向共享: 不允许其他事务独占修改该表。意向共享锁定后, 不同事务可以同时增、删、改、查该表的数据，也支持在该表上创建索引，但不支持修改该表的定义;

2） 意向排他：不允许其他事务独占访问和独占修改该表。被意向排他后，不同事务可以同时增、删、改、查该表的数据，不支持在该表上创建索引，也不支持修改该表定义;

3） 共享：只允许其他事务共享访问该表，仅允许其他事务查询表中的数据，但不允许增、删、改该表的数据;

4） 排他：以独占访问方式锁定整个表，不允许其他事务访问该表，是封锁力度最大的一种封锁方式。

当使用 NOWAIT 时，若不能立即上锁成功则立刻返回报错信息，不再等待。

目前 LOCK TABLE 不支持多表关联的视图和物化视图。

LOCK TABLE 封锁视图时仅封锁视图对应的基表。

19.5.3 查看锁

为了方便用户查看当前系统中锁的状态，DM 数据库专门提供了一个 V$LOCK 动态视图。通过该视图，用户可以查看到系统当前所有锁的详细信息，如锁的内存地址、所属事务 ID、锁类型、锁模式等。用户可以通过执行如下语句查看锁信息：

```
SELECT ADDR,LTYPE,LMODE FROM V$LOCK;
```

其结果看起来和下面类似：

```
行号          ADDR                    LTYPE    LMODE

---------- -------------------- ------ -----

1            140365054396096        OBJECT IS

2             140365054396248        OBJECT IS

3             140364897457408        OBJECT IX
```

19.6 多版本

在多版本控制以前，数据库仅通过锁机制来实现并发控制。数据库对读操作上共享锁，写操作上排他锁，这种锁机制虽然解决了并发问题，但影响了并发性。例如，当对一个事务对表进行查询时，另一个对表更新的事务就必须等待。DM 数据库的多版本实现完全消除了行锁对系统资源的消耗，查询永远不会被阻塞也不需要上行锁，并通过 TID 锁机制消除了插入、删除、更新操作的行锁。数据库的读操作与写操作不会相互阻塞，并发度大幅度提高。

DM 数据库基于物理记录和回滚记录实现行级多版本支持，数据页中只保留物理记录的最新版本，通过回滚记录维护历史版本，所有事务针对特定的版本进行操作。

19.6.1 物理记录格式

为了适应多版本机制，高效地获取历史记录，每一条物理记录中包含了两个字段：TID 和 RPTR。TID 保存修改记录的事务号，RPTR 保存回滚段中上一个版本回滚记录的物理地址。插入、删除和更新物理记录时，RPTR 指向操作生成的回滚记录的物理地址。物理记录格式如下：



新物理记录的 RPTR 指向当前回滚记录的物理地址。

19.6.2 回滚记录格式

回滚记录与物理记录一样，增加了两个字段：TID 和 RPTR。TID 保存回滚记录对应的事务号，RPTR 保存回滚段中上一个版本回滚记录的物理地址。



插入物理记录时，由于没有更老的版本数据，回滚记录的 RPTR 值为 NULL；更新和删除物理记录时，RPTR 指向原始物理记录的 RPTR。

19.6.3 可见性原则

实现多版本控制的关键是可见性判断，找到对当前事务可见的特定版本数据。DM 通过活动事务表，确定事务的可见性。根据事务隔离级的不同，在事务启动时（串行化），或者语句执行时（读提交），收集这一时刻所有活动事务，并记录系统中即将产生的事务号 NEXT_TID。DM 多版本可见性原则：

1. 物理记录的 TRXID 等于当前事务号，说明是本事务修改的物理记录，物理记录可见；
2. 物理记录的 TRXID 不在活动事务表中，并且 TRXID 小于 NEXT_TID，物理记录可

见；

3. 物理记录的 TRXID 包含在活动事务表中，或者 TRXID 大于等于 NEXT_TID，物理记录不可见。

在 DM 数据库运行过程中，由于 DM 多版本并发控制实现策略的缘故，可能会出现“-6527 记录可见性无法确定”报错。出现这个报错说明操作的执行时间过长了，报错相关操作执行前启动的某个事务 A 在这个操作的执行过程中进行了提交，且系统提交状态事务数组中记录这个事务 A 的位置已被新的事务覆盖。为了避免发生这种情况，应该尽量避免系统中的过长事务，同时可以放大 TRX_CMTARR_SIZE 参数。

19.6.4 历史数据获取

当物理记录对当前事务不可见时，根据物理记录和回滚记录的 RPTR 指针，向前回溯一个历史版本记录，通过此历史版本记录的 TID 字段，依据事务可见性原则判断此版本的记录对当前事务是否可见。如可见即获取到了满足当前事务的历史版本数据；如不可见则根据 RPTR 指针继续向前回溯。如果一直不能找到对当前事务的可见版本（例如此记录是一个活动事务插入的新记录），则此记录将不会添加到查询结果集中。

下面以 UPDATE 为例描述多版本的实现；依次执行事务 T1 和 T2：

事务 T1

```
CREATE TABLE TEST_UPDATE (col_1 int PRIMARY KEY, col_2 varchar(10));
INSERT INTO TEST_UPDATE VALUES(1, 'ABCD');
```

事务 T2

```
UPDATE TEST_UPDATE SET col_2='xyz' where col_1 = 1;
```

执行以上两个事务以后，表 TEST_UPDATE 记录了两个版本

物理记录：

物理记录，字段值为（1，‘XYZ’）	TID（T2 的 ID）	RPTR（回滚记录地址）

回滚记录：

回滚记录，保存了老记录的值（1，‘ABCD’）	TID(T1 的 ID)	RPTR

19.6.5 回滚段自动清理

由于需要根据回滚记录回溯、还原物理记录的历史版本信息，因此不能在事务提交时立即清除当前事务产生的回滚记录。但是，如果不及时清理回滚段，可能造成回滚段空间的不断膨胀，占用大量磁盘空间。

DM 提供了自动清理、回收回滚段空间的机制。系统定时（缺省是每间隔 1 秒）扫描回滚段，根据回滚记录的 TID，判断是否需要保留回滚记录，清除那些对所有活动事务可见的回滚记录空间。

19.7 事务隔离级

在关系型数据库中，事务的隔离性分为四个隔离级别，在解读这四个级别前先介绍几个关于读数据的概念。

1. **脏读（DirtyRead）**

所谓脏读就是对脏数据的读取，而脏数据所指的就是未提交的已修改数据。也就是说，一个事务正在对一条记录做修改，在这个事务完成并提交之前，这条数据是处于待定状态的（可能提交也可能回滚），这时，第二个事务来读取这条没有提交的数据，并据此做进一步的处理，就会产生未提交的数据依赖关系，这种现象被称为脏读。如果一个事务在提交操作结果之前，另一个事务可以看到该结果，就会发生脏读。

2. **不可重复读（Non-RepeatableRead）**

一个事务先后读取同一条记录，但两次读取的数据不同，我们称之为不可重复读。如果一个事务在读取了一条记录后，另一个事务修改了这条记录并且提交了事务，再次读取记录

时如果获取到的是修改后的数据，这就发生了不可重复读情况。

3. 幻读（PhantomRead）

一个事务按相同的查询条件重新读取以前检索过的数据，却发现其他事务插入了满足其查询条件的新数据，这种现象就称为幻读。

在 SQL-92 标准中，定义了四种隔离级别：读未提交、读提交、可重复读和串行化。每种隔离级别下对于读数据有不同的要求，表 19.2 中列出四种隔离级别下系统允许/禁止哪些类型的读数据现象。其中“Y”表示允许，“N”表示禁止。

表 19.2 隔离级别表

解决 / 级别	脏读	不可重复读	幻读
读未提交	Y	Y	Y
读提交	N	Y	Y
可重复读	N	N	Y
串行化	N	N	N

在只有单一用户的数据库中，用户可以任意修改数据，而无需考虑同时有其他用户正在修改相同的数据。但在一个多用户数据库中，多个并发事务中包含的语句可能会修改相同的数据。数据库中并发执行的事务最终应产生有意义且具备一致性的结果。因此在多用户数据库中，对数据并发访问及数据一致性进行控制是两项极为重要的工作。

为了描述同时执行的多个事务如何实现数据一致性，数据库研究人员定义了被称为串行化处理的事务隔离模型。当所有事务都采取串行化模式执行时，我们可以认为同一时间只有一个事务在运行（串行的），而非并发的。

DM 数据库支持三种事务隔离级别：读未提交、读提交和串行化。其中，读提交是 DM 数据库默认使用的事务隔离级别。可重复读升级为更严格的串行化隔离级。

19.7.1 读提交隔离级

DM 数据库的读提交隔离可以确保只访问到已提交事务修改的数据，保证数据处于一致性状态，能够满足大多数应用的要求，并最大限度的保证系统并发性能，但可能会出现不可重复读取和幻读。

用户可以在事务开始时使用以下语句设定事务为读提交隔离级：

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

19.7.2 串行化隔离级

在要求消除不可重复读取或幻读的情况下，我们可以设置事务隔离级为串行化。跟读提交隔离级相比，串行化事务的查询本身不会增加任何代价，但修改数据可能引发“串行化事务被打断”错误。

具体来说，当一个串行化事务试图更新或删除数据时，而这些数据在此事务开始后被其他事务修改并提交时，DM 数据库将报“串行化事务被打断”错误。应用开发者应该充分考虑串行化事务带来的回滚及重做事务的开销，从应用逻辑上避免对相同数据行的激烈竞争导致产生大量事务回滚。并结合应用逻辑，捕获“串行化事务被打断”错误，进行事务重做等相应处理。如果系统中存在长时间运行的写事务，并且该长事务所操作的数据还会被其他短事务频繁更新的话，最好避免使用串行化事务。

用户可以在事务开始时使用以下语句设定事务为串行化隔离级：

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

19.7.3 读未提交隔离级

DM 数据库除了支持读提交、串行化两种隔离级之外，还支持读未提交这种隔离级。

读未提交隔离级别是最不严格的隔离级别。实际上，在使用这个隔离级别时，有可能发

生脏读、不可重复读和幻读。一般来说，读未提交隔离级别通常只用于访问只读表和只读视图，以消除可见性判断带来的系统开销，提升查询性能。

用户可以在事务开始时使用以下语句，设定事务为读未提交隔离级：

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

此外，DM 还允许用户在 SELECT 语句的末尾加上 WITH UR 以指定当前查询语句的隔离级为读未提交，即允许脏读，并在该语句结束时自动恢复为原来的隔离级。需要注意的，DPC 环境下不支持该功能。

19.7.4 只读事务

除了前面所述的各种标准特性外，DM 数据库还支持只读事务，只读事务只能访问数据，但不能修改数据。并且只读事务不会改变事务原有的隔离级。

用户可以在事务开始时使用以下语句，设定事务为只读事务：

```
SET TRANSACTION READ ONLY;
```

19.8 锁等待与死锁检测

锁等待（或称阻塞）和死锁是会与并发事务一起发生的两个事件，它们都与锁相关。当一个事务正在占用某个资源的锁，此时另一个事务正在请求这个资源上与第一个锁相冲突的锁类型时，就会导致锁等待。发生锁等待的事务将一直挂起，直到持有锁的事务放弃锁定的资源为止。死锁与锁等待的不同之处在于死锁包括两个或两个以上发生锁等待的事务，它们之间形成了等待环，每个都等待其他事务释放锁。例如事务 1 给表 T1 上了排他锁，第二个事务给表 T2 上了排他锁，此时事务 1 请求 T2 的排他锁，就会处于等待状态，导致了锁等待。若此时 T2 再请求表 T1 的排他锁，则 T2 也处于锁等待状态。此时这两个事务发生死锁，DM 数据库会选择牺牲掉其中一个事务。

在 DM 数据库中，INSERT、UPDATE、DELETE 是最常见的会产生锁等待和死锁的语句。

INSERT 语句导致锁等待的唯一情况是，当多个事务同时试图向有主键或 UNIQUE 约束的表中插入相同的数据时，其中的一个事务将陷入锁等待，直到另外一个事务提交或回滚。一个事务提交时，另一个事务将收到唯一性冲突的错误；一个事务回滚时，陷入锁等待的事务可以继续执行。

当 UPDATE 和 DELETE 语句所修改的记录，已经被另外的事务修改过，也将导致锁等待，直到另一个事务提交或回滚。

19.9 闪回

当用户操作不慎导致错误的删改数据时，非常希望有一种简单快捷的方式可以恢复数据。闪回技术，就是为了用户可以迅速处理这种数据逻辑损坏的情况而产生的。

闪回技术主要是通过回滚段存储的 UNDO 记录来完成历史记录的还原。设置 ENABLE_FLASHBACK 为 1 后，开启闪回功能。DM 会保留回滚段一段时间，回滚段保留的时间代表着可以闪回的时间长度。由 UNDO_RETENTION 参数指定。

开启闪回功能后，DM 会在内存中记录下每个事务的起始时间和提交时间。通过用户指定的时刻，查询到该时刻的 LSN，结合当前记录和回滚段中的 UNDO 记录，就可以还原出特定 LSN 的记录。即指定时刻的记录状态。从而完成闪回查询。闪回查询功能完全依赖于回滚段管理，对于 DROP 等误操作不能恢复。闪回特性可应用在以下方面：

1. 自我维护过程中的修复：当一些重要的记录被意外删除，用户可以向后移动到一个时间点，查看丢失的行并把它们重新插入现在的表内恢复；
2. 用于分析数据变化：可以对同一张表的不同闪回时刻进行链接查询，以此查看变化的数据。

19.10 XA 事务

XA 协议是由 X/Open 组织提出的分布式事务处理规范，主要定义了事务管理器 TM 和

资源管理器 RM 之间的编程接口。因此，使用了 XA 协议的事务被称为 XA 事务。

一个 XA 事务由一个或多个资源管理器 RM、一个事务管理器 TM 和一个应用程序 AP 组成。一个 XA 事务中的多个 RM 可以是不同类型的数据库。例如：达梦、Oracle、Mysql 等。

达梦提供了两种使用 XA 事务的方法：一是 XA 编程接口，供 TM 调用，详情请参考《DM8 程序员手册》中 DM XA 编程指南；二是 DBMS_XA 包，供用户在 Disql 工具或 DMSQL 程序中使用，详情请参考《DM8 系统包手册》中 DBMS_XA 包。

第四部分 故障排除和性能优化

第20章 问题跟踪和解决

问题跟踪和解决是最复杂的脑力劳动之一，其复杂主要体现在问题的无法预知性和问题根源的多样性。本章的目的是提供一种相对比较系统的方法来查找并解决使用 DM 数据库过程中可能遇到的各种难以跟踪、定位的问题，为 DBA 分析、解决问题提供帮助。

20.1 问题分析

当系统出现问题，无法及时响应用户/应用请求时，可能的原因是多方面的。一般来说，DBA 应该查看和分析的内容包括：

1. **网络是否正常**

DBA 可以直接使用各种工具/软件来排除网络问题。如果远程操作有问题，但是本地操作没有问题，则有可能是网络出现故障或者网络带宽耗尽。但是如果本地也有问题，不能说明网络一定没有问题，此时还是需要用其他方式确定网络是否有故障，同时进一步分析本地问题产生的原因。

2. **内存使用量**

用户可以通过操作系统提供的内存检测工具/命令来查看数据库占用内存情况，看是否数据库占用了过多内存，并且开始大量使用页面文件（Windows）/交换分区（Linux/UNIX），如果数据库占用内存过多，则需要进一步分析可能的原因：是否数据库的内存相关参数设置错误，是否客户端请求的资源过多并且一直没有释放（如不断打开连接/游标，并且一直不关闭）等等。对于参数设置错误，DBA 可以通过修改参数加以解决。如果是客户端请求资源过多，DBA 可以通过查询运行时动态视图（参考附录 2 动态性能视图）来检查资源使用情

况，明确问题产生的原因并予以纠正。若排除其他原因后，发现数据库内存仍在不断增长，此时可以联系达梦公司技术服务人员协助解决。

3. CPU 使用率

当发现系统响应很慢甚至无法响应时，CPU 的使用率也是一个重要的观察指标。如果 CPU 使用率一直持续 90%以上，甚至 100%，则说明 CPU 使用率过高，此时需要分析导致 CPU 使用率过高的原因。可能的原因包括：写了错误的存储过程/函数死循环逻辑；某条 SQL 语句执行计划不好导致 CPU 使用率过高（如没有建立合适的索引等）；系统内部 SQL 语句都执行正常，只是实际应用负载过大等。针对上述原因，DBA 可以分别考虑改正存储过程/函数的死循环逻辑、建立合适的索引以及提供更高配置的软/硬件环境等措施分别予以解决。

4. I/O 是否正常

I/O 性能没有满足要求是导致很多系统性能低下的原因。通常情况下，主要是两个方面的原因导致 I/O 性能瓶颈：在系统规划时没有对 I/O 性能进行估算或者估算偏差太大，导致存储的 I/O 性能无法满足要求；其次是没有利用好数据库特性，如没有建立合适的索引，导致经常要做全表扫描，消耗大量 I/O 带宽，这可以通过查看 SQL 语句执行计划来加以分析。

5. 系统日志和 SQL 日志

最后，DBA 还可以通过查看系统日志来辅助分析问题。在 DM 数据库运行过程中，会将一些关键信息记录到安装目录下一级 log 目录下的名称为 dm_实例名_YYYYMM.log 的系统日志文件中，其中 YYYY 表示年份，MM 表示月份。例如，dm_DMSERVER_202102.log。该文件会记录下数据库服务启动/关闭的时间、系统关键错误如打开文件失败等。

另外，如果将 DM 数据库配置文件中的参数 SVR_LOG 设置为打开，则系统还会在刚才的 log 目录下生成名为 dmsql_实例名_日期_时间.log 的 SQL 日志文件。例如，dmsql_DMSERVER_20210226_102712.log。在该文件中记录了启用 SVR_LOG 之后数据

库接收到的所有 SQL 语句等信息，DBA 也可以通过分析该文件来帮助解决问题。

20.2 监控系统性能(V$)

在 DM 数据库中，定义了一系列以 V$为前缀的系统动态视图（参考附录 2：动态性能视图），这些表只有表结构信息，没有数据，有时也称之为虚视图。查询动态视图时，服务器动态加载数据。在 DM 数据库运行过程中，系统动态视图提供了大量系统内部信息，便于数据库管理员监视服务器的运行状况，并根据这些信息对数据库进行调优，达到提高数据库性能的目的。用户可以通过普通 SQL 查询语句来查询动态视图信息，也可以通过图形化客户端工具 Monitor 来进行查看。

某些动态性能视图(如 V$SYSSTAT)需要 ENABLE_MONITOR、MONITOR_SQL_EXEC、DSC_ENABLE_MONITOR 参数打开时才会进行相关信息的收集。

20.3 检查数据物理一致性

DM 数据库提供了用于检查数据物理一致性的工具 dmdbchk。在数据库服务器正常关闭的情况下，可以使用 dmdbchk 对数据文件完整性进行校验，包括校验 DM 数据库内部的物理存储结构是否正常，对象信息是否合法等。除此之外，dmdbchk 工具还可以检测并删除操作系统中残留的信号量和共享内存，避免达到操作系统上限后数据库服务无法启动。在检验完毕后，dmdbchk 会在当前目录下生成一个名为 dbchk_err.txt 的检查报告，供用户查看。

dmdbchk 的使用必须指定必要的执行参数，其调用格式为：

```
./dmdbchk help

dmdbchk V8

version: 03134284585-20260210-314028-20151

书写格式: ./dmdbchk KEYWORD=<value> {KEYWORD=<value>}
```

其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中

使用示例: ./dmdbchk path=/opt/dmdbms/bin/dm.ini

关键词（KEYWORD） 说明 缺省值 是否必选项

PATH dm.ini 绝对路径或者当前目录的 dm.ini 无 否

DCR_INI dmdcr.ini 的路径 默认为当前目录下 dmdcr.ini 否

START_INDEXID 最小检查索引号 无 否

END_INDEXID 最大检查索引号 无 否

CHECK_SEMA 检查当前系统信号量使用情况（只适用于 LINUX，1：只做检查 2：检查并删除残留信号量） 无 否

CHECK_SHM 检查当前系统共享内存使用情况（只适用于 LINUX，1：只做检查 2：检查并删除残留共享内存） 无 否

CHECK_REC 检查数据库中 B 树叶子节点的记录（0：不做检查 1：检查 2：同时检查 BLOB 字段，仅用于 B 树索引检测，在 FILE_PATH 参数下不起作用） 默认为 0 否

PARALLEL 用于指定检测的过程中所使用的线程数目(1-100) 默认为 8 否

FILE_PATH 单独检测指定的 dbf 文件的路径地址 无 否

TABLESPACEID 单独检测指定的表空间号 无 否

FEEDBACK 是否显示检测百分比进度（0：不显示 1：显示 2：只显示简略信息） 默认为 0 否

OUTPUT_FIL 用于指定生成的检测报告的路径 默认为当前目录下的 dbchk_err.txt 否

PAGES_FILE 单独检测指定的数据页页号的路径地址 无 否

HELP 打印帮助信息 无 否

```
--------------------------------------------------------------------------------
```

dmdbchk 的详细说明可参考《DM8_dmdbchk 使用手册》。

20.4 调整配置参数

在 DM 数据库中，很多参数都是动态的，会自动响应各种负载，但是 DBA 仍然可以调用系统过程来改变 DM 实例的运行参数，从而获得更佳的性能体验。DBA 可以在 DM 数据库运行过程中执行 SF_GET_PARA_VALUE、SF_GET_PARA_DOUBLE_VALUE 和 SF_GET_PARA_STRING_VALUE 这三个函数来获取系统的当前配置参数，并且可以使用 SP_SET_PARA_VALUE、SP_SET_PARA_DOUBLE_VALUE、SF_SET_SYSTEM_PARA_VALUE 和 SP_SET_INI_PARA_VALUE 过程来修改静态/动态配置参数:

1. SF_GET_PARA_VALUE (scope int, paraname varchar(256))

配置参数的值类型为数值类型时使用该函数来获取当前值。SCOPE 参数为 1 表示获取 INI 文件中配置参数的值，为 2 表示获取内存中配置参数的值。

2. SF_GET_PARA_DOUBLE_VALUE(scope int, paraname varchar(256))

配置参数的值类型为浮点型时使用该函数来获取当前值。SCOPE 参数为 1 表示获取 INI 文件中配置参数的值，为 2 表示获取内存中配置参数的值。

3. SF_GET_PARA_STRING_VALUE (scope int, paraname varchar(256))

配置参数的值为字符串类型时用该系统函数来获取当前值。SCOPE 参数为 1 表示获取 INI 文件中配置参数的值，为 2 表示获取内存中配置参数的值。

4. SP_SET_PARA_VALUE (scope int, paraname varchar(256), value bigint)
   或 SP_SET_PARA_VALUE(inst_name varchar(128), scope int, paraname varchar(256),value bigint)

该过程用于修改整型静态配置参数和动态配置参数。SCOPE 参数为 0 表示修改内存中

的动态配置参数值；参数为 1 表示修改内存和 INI 文件中的动态配置参数值；参数为 2 表示只在 INI 文件中修改配置参数，此时可修改静态配置参数和动态配置参数。当 SCOPE 等于 0 或 1，试图修改静态配置参数时服务器会返回错误信息。只有具有 DBA 角色的用户才有权限调用 SP_SET_PARA_VALUE。

inst_name 用于 DMDPC 环境下指定实例名，表示修改特定实例名下的整型静态配置参数和动态配置参数，此时被修改的配置参数不能是必须同步的参数。

5. SP_SET_PARA_DOUBLE_VALUE (scope int, paraname varchar(256), value double) 或 SP_SET_PARA_DOUBLE_VALUE(inst_name varchar(128), scope int, paraname varchar(256),value double)

该过程用于修改浮点型静态配置参数和动态配置参数。SCOPE 参数为 0 表示修改内存中的动态配置参数值；参数为 1 表示修改内存和 INI 文件中的动态配置参数值；参数为 2 表示只在 INI 文件中修改配置参数，此时可修改静态配置参数和动态配置参数。当 SCOPE 等于 0 或 1，试图修改静态配置参数时服务器会返回错误信息。只有具有 DBA 角色的用户才有权限调用 SP_SET_PARA_DOUBLE_VALUE。

inst_name 用于 DMDPC 环境下指定实例名，表示修改特定实例名下的浮点型静态配置参数和动态配置参数，此时被修改的配置参数不能是必须同步的参数。

6. SP_SET_PARA_STRING_VALUE(scope int, paraname varchar(256),value varchar(256)) 或 SP_SET_PARA_STRING_VALUE(inst_name varchar(128), scope int, paraname varchar(256),value varchar(256))

该过程用于修改字符串类型静态配置参数和动态配置参数。SCOPE 参数为 0 表示修改内存中的动态配置参数值；参数为 1 表示修改内存和 INI 文件中的动态配置参数值；参数为 2 表示只在 INI 文件中修改配置参数，此时可修改静态配置参数和动态配置参数。当 SCOPE 等于 0 或 1，试图修改静态配置参数时服务器会返回错误信息。只有具有 DBA 角色的用户

才有权限调用 SP_SET_PARA_DOUBLE_VALUE。

inst_name 用于 DMDPC 环境下指定实例名，表示修改特定实例名下的字符串类型静态配置参数和动态配置参数，此时被修改的配置参数不能是必须同步的参数。

7. SF_SET_SYSTEM_PARA_VALUE (paraname varchar(256), value bigint\double\varchar (256), deferred int, scope int) 或 SF_SET_SYSTEM_PARA_VALUE(inst_name varchar(128),paraname varchar(256), value bigint\double\varchar（256）, deferred int, scope int)

该过程用于修改系统整型、double、varchar 的静态配置参数或动态配置参数。DEFERRED 参数，为 0 表示当前 session 修改的参数立即生效，为 1 表示当前 session 不生效，后续再生效，默认为 0。SCOPE 参数为 0 表示修改内存中的动态配置参数值；参数为 1 表示修改内存和 INI 文件中的动态配置参数值；参数为 2 表示只在 INI 文件中修改配置参数，此时可修改静态配置参数和动态配置参数。当 SCOPE 等于 0 或 1，试图修改静态配置参数时服务器会返回错误信息。只有具有 DBA 角色的用户才有权限调用 SF_SET_SYSTEM_PARA_VALUE。

inst_name 用于 DMDPC 环境下指定实例名，表示修改特定实例名下的系统整型、double、varchar 的静态配置参数或动态配置参数，此时被修改的配置参数不能是必须同步的参数。

8. SP_SET_INI_PARA_VALUE (scope int, paraname varchar(256), value varchar(32767))

该过程用于修改整型，浮点型，字符串静态配置参数和动态配置参数。SCOPE 参数为 0 表示修改内存中的动态配置参数值；参数为 1 表示修改内存和 INI 文件中的动态配置参数值;参数为 2 表示只在 INI 文件中修改配置参数,此时可修改静态配置参数和动态配置参数。

当 SCOPE 等于 0 或 1，试图修改静态配置参数时服务器会返回错误信息。只有具有 DBA 角色的用户才有权限调用 SP_SET_INI_PARA_VALUE。

DM 的动态 INI 参数分为系统级和会话级两种级别。会话级参数在服务器运行过程中被修改时，之前创建的会话不受影响，只有当前会话和新创建的会话使用新的参数值。

1. SF_SET_SESSION_PARA_VALUE (paraname varchar(256), value bigint) 或者 SF_SET_SESSION_PARA_VALUE (paraname varchar(256),value double) 或者 SF_SET_SESSION_PARA_VALUE (paraname varchar(256),value varchar(128))

设置某个会话级 INI 参数的值，设置的参数值只对本会话有效。

2. SP_RESET_SESSION_PARA_VALUE (paraname varchar(256))

重置某个会话级 INI 参数的值，使得这个 INI 参数的值和系统 INI 参数的值保持一致。

3. SF_GET_SESSION_PARA_VALUE (paraname varchar(256))

获得当前会话的某个会话级 INI 参数的值。

需要注意的是，在对参数进行调整前，DBA 应该深刻理解配置参数中每个参数的含义和对系统的影响，避免由于错误的调整导致影响整个系统对外提供正常服务。对于一些关键业务，在实际调整前，建议在测试系统上先进行试验，验证通过后再在生产系统上进行调整。

20.5 优化数据库布局

数据库的布局直接影响整个系统的 I/O 性能。通常情况下，DBA 应该遵循下述原则：

1. 日志文件放在独立的物理磁盘上，保持与数据文件分开存储；
2. 预先估算并分配好磁盘空间，避免运行过程中频繁扩充数据文件；
3. 系统中不同表空间尽量分布在不同的磁盘上，这样当数据分布在多个表空间时，可以充分利用不同磁盘的并行 I/O 能力；
4. 对于分区表，尽可能将不同的分区放到不同的表空间；

5. 对于分析型应用，数据库的页大小和簇大小都可以考虑取最大值，并且在采用列存储的情况下，应该尽可能让每列存放在独立的表。

第21章 动态管理和性能视图

21.1 理解动态管理视图

达梦数据库中的动态性能视图能自动收集数据库中的一些活动信息，系统管理员根据这些信息可以了解数据库运行的基本情况，为数据库的维护和优化提供依据。动态性能视图信息也是数据库中数据字典的一部分，与我们平常所说的数据字典不同的是，平常意义上的数据字典是指静态数据字典信息，也即用户访问数据字典信息时，内容不会发生改变，而动态视图信息是随着数据库的运行随时更改，具有一定的即时性。

系统管理员为了更好地了解数据库的一些运行时信息，可以查询动态视图表。首先系统管理员需要知道达梦数据库中提供了多少动态视图，有哪些类型动态视图，以及这些动态视图的用途是什么。关于这些内容可以参考附录 2，也可以通过查询 V$DYNAMIC_TABLES 获得所有的动态性能视图名称。

动态视图表与静态字典信息表命名方式不同，静态字典表一般以 SYS 为前缀，如系统表 SYSINDEXES，而动态视图则以 V$为前缀，如 V$DM_INI。

21.2 使用动态管理视图

在 DM8 中，动态视图提供的系统信息主要分为以下几个方面：

1. **系统信息**

包括数据库版本、实例统计信息、资源限制信息、进程信息、全局索引 IID 信息、事件信息；涉及的动态视图有 V$SESSIONS、V$INSTANCE、V$RESOURCE_LIMIT、V$PROCESS、V$IID、V$SYSSTAT 等。

例 查看数据库中实例信息

```
SELECT * FROM V$INSTANCE;
```

查询结果如下:

```
行号 NAME INSTANCE_NAME INSTANCE_NUMBER HOST_NAME SVR_VERSION
DB_VERSION START_TIME STATUS$ MODE$ OGUID DSC_SEQNO
DSC_ROLE BUILD_VERSION BUILD_TIME
---------- -------- ------------ --------------- --------- ----------------------
----- ------------------ ------------------- ------- ------ ----------- --------
--- -------- ---------------------------------- --------------------
1 DMSERVER DMSERVER 1 test121 DM Database Server x64 V8
DB Version: 0x7000d 2024-11-26 17:10:40 OPEN NORMAL 0 0 NULL
03134284347-20241121-250764-10000 Nov 26 2024 15:41:48
```

2. 存储信息

包括数据库信息、表空间信息、数据文件信息、日志相关信息；涉及的动态视图有 V$DATAFILE、V$DATABASE、V$TABLESPACE、V$HUGE_TABLESPACE、V$RLOGFILE 等。

例 查询表空间信息

```
SELECT * FROM V$TABLESPACE;
```

查询结果如下:

```
ID NAME CACHE TYPE$ STATUS$ MAX_SIZE TOTAL_SIZE FILE_NUM
ENCRYPT_NAME ENCRYPTED_KEY COPY_NUM SIZE_MODE OPT_NODE
……
7 TABLESPACE_03 NORMAL 1 0 0 32768 1 null null null null NULL
9 TS_001 NORMAL 1 0 0 16384 1 NULL NULL NULL NULL NULL
```

3. 内存管理信息

包括内存池使用情况、缓冲区信息、虚拟机信息、虚拟机栈帧信息；涉及的动态视图

有 V$MEM_POOL、V$VMS、V$STKFRM、V$BUFFERPOOL、V$BUFFER_LRU_FIRST、V$BUFFER_UPD_FIRST、V$BUFFER_LRU_LAST、V$BUFFER_UPD_LAST 等。

例 查询内存池 BUFFERPOOL 的页数、读取页数和命中率信息

```
SELECT NAME,N_PAGES,N_LOGIC_READS,RAT_HIT FROM V$BUFFERPOOL;
```

查询结果如下:

```
行号     NAME          N_PAGES            N_LOGIC_READS              RAT_HIT

------ ---- ------------------- ----------- -----------------

1        NORMAL     1024               0                    1.0000000000E+000

2        RECYCLE    8192               52                   9.8113207547E-001

3        NORMAL     1280               1772                 9.2726321298E-001

4        NORMAL     8960               7                      7.0000000000E-001
```

4. 事务信息

包括所有事务信息、当前事务可见的事务信息、事务锁信息（TID 锁、对象锁）、回滚段信息、事务等待信息; 涉及的动态视图有 V$TRX、V$TRXWAIT、V$LOCK、V$PURGE 等。

例 查询系统中上锁的事务、锁类型，以及表 ID 信息

```
SELECT TRX_ID,LTYPE,LMODE,TABLE_ID FROM V$LOCK;
```

查询结果如下:

```
行号          TRX_ID                  LTYPE    LMODE TABLE_ID

---------- ------------------- ------ ----- -----------

1             1436                    OBJECT IX      1311

2             1433                    OBJECT IX      1310

3             1436                    TID      X       -1

4             1433                    TID      X       -1
```

5. 线程信息

包括所有活动线程信息、线程作业信息、线程锁信息、线程的资源等待信息；涉及的动态视图有 V$THREADS 等。

例 查看系统中所有活动的线程信息

```
SELECT TOP 2* FROM V$THREADS;
```

查询结果如下:

```
行号 ID NAME START_TIME THREAD_DESC
SESS_ID PROCESSOR_ID WAIT_SEQ WAIT_STATUS WAIT_TIME
---------- -------------------- ---------- ------------------------- ----------
--------- -------------------- ------------ ----------- --------------- --------
---
1 84540 dm_sql_thd 2025-12-03 09:47:44.000000 User session
thread 140199769256688 65535 0 NOTHING TO WAIT -1
2 84541 dm_sql_thd 2025-12-03 09:47:44.000000 User session
thread 140198870257504 65535 27 NOTHING TO WAIT -1
```

6. 历史模块

包括 SQL 历史信息、SQL 执行节点历史信息、检查点历史信息、命令行历史信息、线程等待历史信息、死锁历史信息、回滚段历史信息、运行时错误历史信息、DMSQL 程序中执行 DDL 语句的历史信息、返回大数据量结果集的历史信息、所有活动过线程的历史信息；涉及的动态视图有 V$CKPT_HISTORY、V$CMD_HISTORY、V$DEADLOCK_HISTORY、V$PLSQL_DDL_HISTORY、V$PRE_RETURN_HISTORY、V$RUNTIME_ERR_HISTORY、V$WAIT_HISTORY、V$SQL_HISTORY、V$SQL_NODE_HISTORY、V$SQL_NODE_NAME 等。

例 查询系统执行的 SQL 历史信息

```
SELECT SESS_ID,TOP_SQL_TEXT,TIME_USED FROM V$SQL_HISTORY;
```

查询结果如下:

```
行号      SESS_ID            TOP_SQL_TEXT                    TIME_USED

--------  -----------------  ----------------------

1         187744368          INSERT INTO T1 VALUES(5,0);      21707
```

7. 缓存信息

包括 SQL 语句缓存、执行计划缓存、结果集缓存、字典缓存信息、字典缓存中的对象信息、代价信息；涉及的动态视图有 V$CACHEITEM、V$SQL_PLAN、V$CACHERS、V$CACHESQL、V$DICT_CACHE_ITEM、V$DICT_CACHE 等。

例 查看字典缓存的信息

```
SELECT * FROM V$DICT_CACHE;
```

查询结果如下:

```
行号     ADDR          POOL_ID        TOTAL_SIZE          USED_SIZE             DICT_NUM

---------- ------------ -------- ----------------- --------------------

1        0x0B56F070        1             5242880              113530                 36
```

8. 会话信息

包括连接信息、会话信息；涉及的动态视图有 V$CONNECT、V$STMTS、V$SESSIONS 等。

例 查看会话信息

```
SELECT SESS_ID,SQL_TEXT,STATE,CREATE_TIME,CLNT_HOST FROM V$SESSIONS;
```

查询结果如下:

```
行号      SESS_ID            SQL_TEXT          STATE              CREATE_TIME            CLNT_HOST

--------  -----------------  ----------------------

1         187744368          SELECT SESS_ID,SQL_TEXT,STATE,CREATE_TIME,CLNT_HOST FROM

V$SESSIONS;       ACTIVE       2011-09-19 19:20:38.000000          FREESKYC-FB8846
```

查看动态视图可以不单单只查询一个动态视图表，还可利用动态视图表之间的联系得到更多想要的信息。

例如，系统管理员如果要对一条 SQL 语句进行调优，需要知道每个执行节点花费了多少时间，查询 V$SQL_NODE_NAME 可以知道执行节点的名字，查询 V$SQL_NODE_HISTORY 可以查询到每个执行节点的时间，通过两个动态视图表的执行节点类型 TYPE$字段做等值连接。

如执行一条 SQL 语句，然后查询其执行节点所花费时间，假设其执行 ID（EXEC_ID）为 4。

```
SELECT * FROM t1 WHERE c1 = (SELECT d1 FROM t2 WHERE c2 = d2);
```

关联查询动态视图 V$SQL_NODE_NAME 和 V$SQL_NODE_HISTORY 获取节点执行时间：

```
SELECT N.NAME, TIME_USED, N_ENTER FROM V$SQL_NODE_NAME N, V$SQL_NODE_HISTORY H
WHERE N.TYPE$ = H.TYPE$ AND EXEC_ID = 4;
```

查询结果如下：

```
行号    NAME    TIME_USED    N_ENTER
1        CSCN2     381        6
2        CSCN2     250        3
3        NLI2      52         14
4        SLCT2     102        8
5        HAGR2     11831      6
6        PRJT2     32         6
7        CSCN2     272        3
8        HI3       19309      9
```

```
9          PRJT2     29          6
10         NSET2     120         4
11         DLCK      23          2
```

根据结果可以看到执行计划中各执行节点花费的时间，进而对 SQL 语句进行分析和改写。

第22章 查询优化

数据库执行一条语句有多种方式，为了选择最优的执行方式，产生了查询优化器。查询优化器分析语句运行时的所有因素，选择最优的方式去执行，提高了查询效率。因此，查询优化是数据库执行 SQL 语句的重要过程，决定了数据库的查询性能。

22.1 优化目标

达梦数据库查询优化器的优化目标为最快响应时间。通过设置参数 FIRST_ROWS 来决定优先返回多少条记录给用户，而不需要等待全部结果确定后再输出，FIRST_ROWS 设置范围为 1~1000000，单位为行。例如：FIRST_ROWS = 10，意思是查询出 10 条结果就立即返回给用户。可以根据实际情况，调整参数值。

22.2 查询优化器

查询优化器通过分析可用的执行方式和查询所涉及的对象统计信息来生成最优的执行计划。此外，如果存在 HINT 优化提示，优化器还需要考虑优化提示的因素。

查询优化器的处理过程包括：

1. 优化器生成所有可能的执行计划集合；
2. 优化器基于字典信息的数据分布统计值、执行语句涉及到的表、索引和分区的存储特点来估算每个执行计划的代价。代价是指 SQL 语句使用某种执行方式所消耗的系统资源的估算值。其中，系统资源消耗包括 I/O、CPU 使用情况、内存消耗等；
3. 优化器选择代价最小的执行方式作为该条语句的最终执行计划。

优化器所做的操作有：查询转换、估算代价、生成计划。

22.2.1 查询转换

查询转换是指把经过语法、语义分析的查询块之间的连接类型、嵌套关系进行调整，生成一个更好的查询计划。常用的查询转换技术包括过滤条件的下放、相关子查询的去相关性。

1. 过滤条件下放：在连接查询中，把部分表的过滤条件下移，在连接之前先过滤，可以减少连接操作的数据量，提升语句性能；
2. 相关子查询的去相关性：把与子查询相关的外表与内表采用半连接的方式执行，放弃默认采取的嵌套连接方式，对性能有较大提升。

22.2.2 估算代价

估算代价是指对执行计划的成本进行估算。执行节点之间的代价值相关性较强，一个执行节点的代价包括该节点包含的子节点代价。代价衡量指标包括选择率、基数、代价。

1. 选择率

选择率是指满足条件的记录占总记录数的百分比。记录集可以是基表、视图、连接或分组操作的结果集。选择率和查询谓词相关，如 name = '韩梅梅'；或者是谓词的连接，如 name ='韩梅梅' and no ='0123'。一个谓词可以看作是一个过滤器，过滤掉结果集中不满足条件的记录。选择率的范围从 0 到 1。其中，0 表示没有记录被选中，1 表示行集中所有记录都被选中。

如果没有统计信息，则优化器依据过滤条件的类型来设置对应的选择率。例如，等值条件的选择率低于范围条件选择率。这些假定是根据经验值，认为等值条件返回的结果集最少。

如果有统计信息，则可以使用统计信息来估算选择率。例如，对于等值谓词 (name ='韩梅梅')，如果 name 列有 N 个不同值，那么，选择率是 N 分之一。

2. 基数

基数是指整个行集的行数，该行集可以是基表、视图、连接或分组操作的结果集。

3. 代价

代价表示资源的使用情况。查询优化器使用磁盘 I/O、CPU 占用和内存使用作为代价计算的依据，所以代价可以用 I/O 数、CPU 使用率和内存使用一组值来表示。所有操作都可以进行代价计算，例如扫描基表、索引扫描、连接操作或者对结果集排序等。

访问路径决定了从一个基表中获取数据所需要的代价。访问路径可以是基表扫描、索引扫描等。在进行基表扫描或索引扫描时，一次 I/O 读多个页，所以，基表扫描或索引全扫描的代价依赖于表的数据页数和多页读的参数值。二级索引扫描的代价依赖于 B 树的层次、需扫描的叶子块树以及根据 ROWID 访问聚集索引的记录数。

连接代价是指访问两个连接的结果集代价与连接操作的代价之和。

22.2.3 生成计划

生成计划指计划生成器对给定的查询按照连接方式、连接顺序、访问路径生成不同的执行计划，选择代价最小的一个作为最终的执行计划。

连接顺序指不同连接项的处理顺序。连接项可以是基表、视图、或者是一个中间结果集。例如表 t1、t2、t3 的连接顺序是先访问 t1，再访问 t2，然后对 t1 与 t2 做连接生成结果集 r1，最后把 t3 与 r1 做连接。一个查询语句可能的计划数量是与 FROM 语句中连接项的数量成正比的，随着连接项的数量增加而增加。

22.3 数据访问路径

访问路径指从数据库中检索数据的方法。一般情况下，索引访问用于检索表的小部分数据，全表扫描用于访问表的大部分数据。OLTP 应用中，一般使用索引访问路径，因为 OLTP 中包含了许多高选择率的 SQL 语句。而决策支持系统则倾向于执行全表扫描来获取数据。从数据库中定位和检索数据的方法有：全表扫描、聚集索引扫描、二级索引扫描等。

全表扫描是指从基表中检索数据时，扫描该表中所有的数据。全表扫描方式适合检索表

中大部分数据，这时比索引扫描更加有效率。

索引扫描是指通过指定语句中的索引列进行遍历来检索表中的数据。索引扫描是从基于一列或多列的索引中检索数据。索引不仅包含索引值，还包含对应表中数据的 ROWID。如果需要访问的不是索引列，这时需要通过 ROWID 或聚集索引来找到表中的数据行。

索引扫描包含聚集索引扫描和二级索引扫描。由于在聚集索引中，包含了表中所有的列值，所以检索数据时只需要扫描这一个索引就可以得到所有需要的数据。如果是二级索引，由于只包含索引列以及对应的 ROWID，如果查询列不在二级索引中则还需要扫描聚集索引来得到所需要的数据。

查询优化器选择访问路径基于以下几个因素：

1. 执行语句中可能的访问路径；
2. 估算每条执行路径的代价。为了选择一个访问路径，优化器首先会通过检查语句中的 FROM 子句和 WHERE 子句中的条件表达式来决定哪一个访问路径可以使用。优化器会根据可用的访问路径生成可能的执行计划集合，然后使用索引、列和表的统计信息来估算每个计划的代价。最后，优化器选择最小代价的那个执行计划。

影响优化器选择访问路径的因素有语句中的提示（HINT）和统计信息。用户可以在执行的语句中使用 HINT 来指定访问路径。而统计信息会根据表中数据的分布情况决定采用哪个访问路径会产生最小的代价。

22.4 连接

查询语句中 FROM 子句包含多个表时，我们称为连接查询。如 SELECT * FROM t1,t2 就是连接查询。

生成连接查询的执行计划，需要考虑三方面因素：

1. 访问路径

对于每张表采用何种方式来获取数据。例如：全表扫描、索引扫描等。

查询优化器会估算每种扫描方式的代价，选择代价较小的访问路径。

2. 连接方式

确定两张表之间采用哪种连接方式。例如：哈希连接、嵌套连接、归并连接、外连接。

等值连接条件一般会选择哈希连接；非等值连接条件会采用嵌套连接；连接列均为索引列时，会采用归并连接。

1） 嵌套连接：两张表进行非等值连接时会选择嵌套连接。相当于两张表进行笛卡尔集操作。此时，优化器会选择一张代价较小的表作为外表(驱动表)，另一张表作为内表，外表的每条记录与内表进行一次连接操作。

2） 哈希连接：两张表进行等值连接时会选择哈希连接。以一张表的连接列为哈希键，构造哈希表，另张表的连接列进行哈希探测，找到满足条件的记录。由于哈希命中率高，因此，在大数据量情况下，哈希连接的效率较高。哈希连接的代价是建立哈希表和哈希探测的代价。

3） 归并连接：两张表的连接列均为索引列，则可以按照索引顺序进行归并，一趟归并就可以找出满足条件的记录。如果查询列也属于索引列的子集，则归并连接只需扫描索引，会有更好的性能表现。在两表连接条件不是等值（如<，<=，>，>=）情况下时，归并排序连接很有用。

4） 外连接：外连接分为左外连接、右外连接、全外连接。作为外表的数据会全部返回，如果没有与外表匹配的记录，则填充 NULL 值。右外连接与左外连接的处理过程类似，只是外表不同，一个是左表，一个是右表。全外连接是进行左外连接和右外连接，返回两次外连接的 union 结果集。

例 1 左外连接

```
SELECT * FROM t1 LEFT OUTER JOIN t2 ON t1.c1=t2.d1;
```

例 1 中 t1 表为外表(左表)，如果 t2 表中不存在与 t1.c1 相等的记录，则 t2 表的该行记

录用 NULL 填充。右外连接与左外类似。

例 2 全外连接

```
SELECT * FROM t1 FULL OUTER JOIN t2 ON t1.c1=t2.d1;
```

例 2 中分别以 t1 为左表进行左外和右外连接，两次结果进行 union，返回最终结果。

子查询会转换成半连接。共有四种半连接方式：哈希半连接、索引半连接、嵌套半连接、归并半连接。等值连接条件会选择哈希\索引\归并半连接，非等值连接条件会选择嵌套半连接。

1） 哈希半连接：以外表的连接列为 KEY 构造哈希表，内表的连接列进行探测来查找满足连接条件的记录；

2） 索引半连接：如果子查询的连接列为索引前导列，可采用索引半连接。处理过程为外表的数据对子查询使用索引查找，返回满足条件的记录；

3） 归并半连接：如果相关子查询的连接条件列均为索引列，可采用归并半连接。按照索引顺序，对外表、内表进行同步扫描，返回满足条件的记录；

4） 嵌套半连接：如果连接条件为非等值，可转换为嵌套半连接。处理过程为外表的每条记录去遍历内表，返回满足条件的记录。

3. 连接顺序

当超过 2 张表进行连接时，就需要考虑表之间的连接顺序。不合适的连接顺序对执行效率有较大影响。一般原则是，经过连接可以产生较小结果集的表优先处理。

一个连接查询通常会对应多个执行计划，查询优化器会根据优化规则、代价估算挑选最优的执行计划。

22.5 统计信息

对象统计信息描述了对象数据的分布特征。统计信息是优化器的代价计算的依据，可以帮助优化器较精确地估算成本，对执行计划的选择起着至关重要的作用。统计信息的收集频

率是一把双刃剑，频率太低导致统计信息滞后，频率太高又影响查询性能，因此，系统管理员需要根据实际情况，合理安排统计信息收集的频率。

22.5.1 统计信息简介

22.5.1.1 种类

达梦数据库的统计数据对象分三种：表统计信息、列统计信息和索引统计信息。统计信息生成过程分三个步骤：

1. 确定采样的对象：根据数据对象，确定需要分析哪些数据。

1） 表：计算表的行数、所占的页数目。

2） 列：统计列数据的分布特征。

3） 索引：统计索引列的数据分布特征。

2. 确定采样率。用户根据统计数据对象的大小，确定采样率。如缺省，则默认通过内部算法确定数据的采样率。

3. 生成统计信息。根据不同的数据对象生成不同的统计信息。

1）表：表的行数、所占的页数目等汇总数据。

2）列和索引：将采样的数据按照不同的分布特征生成相应的直方图。有两种类型的直方图：频率直方图和等高直方图。根据算法分析表的数据分布特征（以不同值的数据量 1 万个为分界线），确定直方图的类型。频率直方图的每个桶(保存统计信息的对象)的高度不同，等高直方图每个桶的高度相同。生成直方图时，如果不同值少于 1 万个则用频率直方图，否则用等高直方图。

22.5.1.2 作用

在执行查询时，如果数据对象存在统计信息，代价算法可以根据统计信息中的数据，比

较精确地计算出操作所需花费的成本，以此来确定连接方式、对象访问路径、连接顺序，选择最优的执行计划。

22.5.1.3 收集时机

收集统计信息的时机有两种：一是在查询之前进行静态收集；二是在查询的同时进行动态收集。

静态收集是在查询之前完成。和查询操作互不干涉，因此不影响查询性能。

动态收集是在查询的过程中完成。具体为在构造查询计划阶段进行，统计信息收集完成之后再继续构造计划，因此会影响计划阶段性能，特别是在高并发场景中。

从性能角度考虑，推荐用户使用静态收集。

22.5.2 统计信息用法

统计信息的使用方法有：收集统计信息（静态或动态）、查看统计信息和监控统计信息等。

22.5.2.1 收集统计信息

22.5.2.1.1 静态收集

静态收集统计信息有两种方式。一是自动收集；二是手动收集。采用任意一种即可。静态收集的结果均会保存到系统表中。

22.5.2.1.1.1 自动收集

在 INI 参数 AUTO_STAT_OBJ 为 1 或 2 时，执行 SP_CREATE_AUTO_STAT_TRIGGER 过程实现自动收集。

22.5.2.1.1.2 手动收集

手动收集有三种可选方式。下面分别介绍：

1. 通过《DM8 系统包使用手册》里 DBMS_STATS 包中的方法（GATHER_TABLE_STATS、GATHER_INDEX_STATS 或 GATHER_SCHEMA_STATS）进行生成并收集统计信息。

2. 通过《DM8_SQL 语言使用手册》中设置 STAT 语法为表、列、索引生成统计信息。

例 对 SYSOBJECTS 表上 ID 列生成统计信息，采样率的百分比为 30%。使用按新比例收集的列统计信息覆盖之前老的统计信息。

```
STAT 30 ON SYS.SYSOBJECTS (ID);
```

3. 通过《DM8_SQL 语言使用手册》附录 3 中统计信息章节的系统存储过程和函数进行生成。

例 对 SYSOBJECTS 表上所有的索引生成统计信息。

```
CALL SP_TAB_INDEX_STAT_INIT ('SYS', 'SYSOBJECTS');
```

22.5.2.1.2 动态收集

当已收集的统计信息失效或未曾收集时，用户可通过配置 DM.INI 参数 OPTIMIZER_DYNAMIC_SAMPLING，开启动态收集统计信息。

OPTIMIZER_DYNAMIC_SAMPLING 为动态会话级参数，取值范围：0~12。0：不启用；1~10：启用，采用率 10%~100%，收集动态统计信息，但收集结果不会保存到系统表；11：启用，由优化器自动确定采样率（0.1%~99.9%），收集结果也不会保存到系统表；12：和 11 相同，但收集的结果会保存到系统表。

22.5.2.2 查看统计信息

执行过生成统计信息操作后，可通过下面多种方式查看到具体的统计信息。其中，动态收集中只有 OPTIMIZER_DYNAMIC_SAMPLING=12 可以被查询到。

1. 系统表 SYSSTATS

用于执行收集统计信息操作之后，记录系统中的统计信息。

2. 系统表 SYSMSTATS

记录多维统计信息的内容。

3. 系统表 SYSSTATPREFS

记录指定模式下表的统计信息的静态参数对应的值。

4. 系统表 SYSSTATTABLEIDU

记录用户表上一次收集统计信息时的总行数和增删改的影响行数以及是否有过 TRUNCATE 操作。

5. 通过 DBMS_STATS 包中 COLUMN_STATS_SHOW、TABLE_STATS_SHOW 和 INDEX_STATS_SHOW 查看表、列、索引的统计信息。只有在生成统计信息之后才可以查看，可以用 DBMS_STATS 包中 GATHER_TABLE_STATS、GATHER_INDEX_STATS 或 GATHER_SCHEMA_STATS 生成统计信息，具体参考 22.5.2.1 收集统计信息。

22.5.2.3 监控统计信息

支持对自动收集统计信息的过程进行监控。

通过创建确定的过程 SYSDBA.GET_AUTO_STAT_INFO_FUNC，用以接收服务器在自动收集统计信息时的相关信息。该过程的模块体为开放式，用户可以根据自己的需求来编写如何使用统计信息的代码。

语法如下：

```
CREATE OR REPLACE PROCEDURE  SYSDBA.GET_AUTO_STAT_INFO_FUNC(
TASK_ID INT,
TOTAL_STAT INT,
TABLE_ID INT,
SCH_NAME VARCHAR(24),
TABLE_NAME VARCHAR(24),
CURR_GATH_TAB_ID INT,
CURR_GATH_SCH_NAME VARCHAR(24),
CURR_GATH_TAB_NAME VARCHAR(24),
SUCCESS_STAT INT,
FAIL_STAT INT,
TASK_START_TIME DATETIME,
TASK_END_TIME DATETIME,
GATHER_TBL_START_TIME DATETIME,
GATHER_TBL_END_TIME DATETIME
) AS
BEGIN
        /*用户自定义如何使用统计信息的代码*/
END;
/
```

参数详解

- task_id 任务 id，同一个任务的 task_id 相同。
- total_stat 一次任务需要收集的表的总个数。

- table_id 收集完成的一个表 table 的 id，每收集完一个表的统计信息，就会调用一次 SYSDBA.GET_AUTO_STAT_INFO_FUNC，传出一次数据，用户可自定义该过程，自定义处理接收到的数据。
- sch_name 对应 table_id 的模式名。
- table_name 对应 table_id 的表名。
- curr_gath_tab_id 当前正在收集的表 id。收集完一个表 table 后，传出数据时，该字段为接下来要收集的表 id，也是服务器当前正要或正在收集的表。
- curr_gath_sch_name 对应 curr_gath_tab_id 的模式名。
- curr_gath_tab_name 对应 curr_gath_tab_id 的表名。
- success_stat 截止到目前这次收集任务一共成功收集了多少张表。
- fail_stat 截止到目前这次收集任务一共失败收集了多少张表。
- task_start_time 这次任务的开始时间。
- task_end_time 这次任务的结束时间，未结束为 NULL。
- gather_tbl_start_time 收集完一个表 table 时，该表收集的开始时间。
- gather_tbl_end_time 收集完一个表 table 时，该表收集的结束时间，收集失败，则结束时间为 NULL。

22.5.3 应用实例

22.5.3.1 自动收集

在打开 INI 监控参数 AUTO_STAT_OBJ（为 1 或 2）的前提下，可使用 SP_CREATE_AUTO_STAT_TRIGGER 过程对表的监控信息进行自动收集。

下面详细介绍使用 SP_CREATE_AUTO_STAT_TRIGGER 过程自动收集用户表的统计信息的完整过程。

第一步，打开监控。

设置 INI 参数 AUTO_STAT_OBJ 为 1 或 2。1：对所有表进行监控；2：只对用户通过 DBMS_STATS.SET_TABLE_PREFS 设置过 STALE_PERCENT 属性的表对象进行监控。AUTO_STAT_OBJ=1 时，STALE_PERCENT 为缺省值 10，即数据变化率>=10%时需要更新统计信息。如果 AUTO_STAT_OBJ=2，需进一步使用 DBMS_STATS.SET_TABLE_PREFS 设置 STALE_PERCENT 属性。

```
//用 AUTO_STAT_OBJ=1 打开对 T 表的监控。
CREATE TABLE T(A INT);
SP_SET_PARA_VALUE(1,'AUTO_STAT_OBJ',1);


//用 AUTO_STAT_OBJ=2 打开对 T 表的监控。
CREATE TABLE T(A INT);
SP_SET_PARA_VALUE(1,'AUTO_STAT_OBJ',2);
DBMS_STATS.SET_TABLE_PREFS('SYSDBA','T','STALE_PERCENT',15);//对修改行数占总行数的比达到
STALE_PERCENT 要求的对象和总行数为 0 的对象收集统计信息
```

下面以 AUTO_STAT_OBJ=2 为例。

第二步，执行统计信息收集操作。自动收集统计信息使用 SP_CREATE_AUTO_STAT_TRIGGER 设置触发器。

```
SP_CREATE_AUTO_STAT_TRIGGER(1, 1, 1, 1,'11:20', '2021/1/11',0,1);
```

第三步，对用户表 T 进行增删改操作。

```
INSERT INTO T SELECT LEVEL CONNECT BY LEVEL< =20;
COMMIT;
```

第四步，通过系统表 SYSSTATTABLEIDU 和动态视图 V$AUTO_STAT_TABLE_IDU 查

看监控信息，通过系统表 SYSSTATS 查看统计信息。

当打开监控（设置 AUTO_STAT_OBJ 为 1 或 2）并执行统计信息收集操作之后可以查看到 SYSSTATTABLEIDU 和 SYSSTATS 的变化。SYSSTATTABLEIDU 中的记录为对上一次的统计信息的监控。V$AUTO_STAT_TABLE_IDU 中的记录为对统计信息的实时监控，不需要执行收集统计信息操作也能查看。

本例中 AUTO_STAT_OBJ=2。查看监控信息：

```
//在触发器触发之后查看
SELECT * FROM SYSSTATTABLEIDU;


行号          ID              TOTAL_ROWS              INSERT_ROWS             DELETE_ROWS
UPDATE_ROWS         LAST_STAT_DT            MONITOR_FLAG RSVD1         TRUNCATED
---------- ----------- -------------------- --------------------
1             1061          20                      0                       0                       0
2024-12-19 14:09:16.978000 2               0XFFFFFFFF 0
```

查看统计信息：

```
SELECT * FROM SYSSTATS WHERE ID=1061;
行号     ID      COLID     T_FLAG T_TOTAL    N_SAMPLE     N_DISTINCT     N_NULL     V_MIN
V_MAX    BLEVEL     N_LEAF_PAGES    N_LEAF_USED_PAGES    CLUSTER_FACTOR N_BUCKETS
DATA     COL_AVG_LEN LAST_GATHERED    INFO1         INFO2
------------------------------- ------------------------ ------------------
-
1             1061          -1             T         20                      0                       0
0                       NULL          NULL          0             1                       1
              0                0
```

```
            0X0000FFFF00000700

            -1              2024-12-19 14:09:17.001081 NULL          NULL


2           1061          0             C        20                    20                    20

0                     0X01000000 0X14000000 0            1                     1

            0                 20

0X140001000800070001000000010000000100000002000000010000000300000001000000040

000000100000005000000010000000600000001000000070000000100000008000000010000

000900000001000000A000000010000000B000000010000000C000000010000000D000000

010000000E000000010000000F0000000100000010000000010000001100000001000000120000

000100000013000000010000001400000 0
            4               2024-12-19 14:09:16.998000 NULL          NULL
```

第五步，监控统计信息收集过程。接着上一步的例子，展示监控的完整过程。

首先，创建一个用户表 AUTO_STAT_INFO，用以保存自动收集过程的相关信息。

```
create table AUTO_STAT_INFO(
task_id INT,
total_stat INT,
table_id INT,
sch_name varchar(24),
table_name varchar(24),
curr_gath_tab_id INT,
curr_gath_sch_name varchar(24),
curr_gath_tab_name varchar(24),
```

```
success_stat INT,
fail_stat INT,
task_start_time DATETIME,
task_end_time DATETIME,
gather_tbl_start_time DATETIME,
gather_tbl_end_time DATETIME
);
```

其次，创建过程 SYSDBA.GET_AUTO_STAT_INFO_FUNC，接收服务器在自动收集统计信息时的过程信息。并在模块体编写用户代码，将过程收集的统计信息写入 AUTO_STAT_INFO 中。

```
CREATE OR REPLACE PROCEDURE  SYSDBA.GET_AUTO_STAT_INFO_FUNC(task_id INT,total_stat
INT,table_id INT, sch_name varchar(24), table_name varchar(24),curr_gath_tab_id INT,
curr_gath_sch_name varchar(24), curr_gath_tab_name varchar(24),success_stat
INT,fail_stat INT,task_start_time DATETIME, task_end_time DATETIME,gather_tbl_start_time
DATETIME,gather_tbl_end_time DATETIME) as
BEGIN
//下面是用户自定义的代码，将 SYSDBA.GET_AUTO_STAT_INFO_FUNC 过程的信息插入到用户表
AUTO_STAT_INFO 中
    INSERT INTO AUTO_STAT_INFO VALUES(task_id,total_stat,table_id,
sch_name,table_name,curr_gath_tab_id, curr_gath_sch_name,
curr_gath_tab_name,success_stat ,fail_stat,task_start_time,task_end_time,gather_tbl_st
art_time,gather_tbl_end_time);
    commit;
EXCEPTION
```

```
    WHEN OTHERS THEN
        NULL;
END;
/
```

最后，解读表 AUTO_STAT_INFO，介绍一次自动收集统计信息任务的相关过程信息。如果 SP_CREATE_AUTO_STAT_TRIGGER 触发一次自动收集统计信息，在任务开始时，会先记录当前任务的开始时间，当前任务的待收集表的总个数，以及接下来待收集的表的 id。之后每收集完成一个表的统计信息，就会记录该表 table 的 id 及收集该表的开始和结束时间，和截至目前收集成功失败的表个数情况，以及接下来待收集的表的 id。其中“接下来待收集的表的 id”即 current_gather_tab_id，为当前服务器正在收集的表 id。

22.5.3.2 手动收集

例 使用 DBMS_STATS 包中 GATHER_TABLE_STATS 方法手动收集用户表的统计信息的完整过程。

使用包内的过程和函数之前，如果还未创建过系统包。请先调用系统过程创建系统包。

```
SP_CREATE_SYSTEM_PACKAGES (1,'DBMS_STATS');
SET SERVEROUTPUT ON;  //PRINT 需要设置这条语句，才能打印出消息
```

收集模式 PERSON 下表 ADDRESS 的统计信息，并打印收集的表信息。

```
BEGIN
    DECLARE
     OBJTAB DBMS_STATS.OBJECTTAB;
     OBJ_FILTER_LIST DBMS_STATS.OBJECTTAB;
    BEGIN
        OBJ_FILTER_LIST(0).OWNNAME = 'PERSON';
```

```
        OBJ_FILTER_LIST(0).OBJTYPE = 'TABLE';
        OBJ_FILTER_LIST(0).OBJNAME = 'ADDRESS';
DBMS_STATS.GATHER_SCHEMA_STATS(
     'PERSON',
      1.0,
      FALSE,
       'FOR ALL COLUMNS SIZE AUTO',
       1,
       'AUTO',
       TRUE,
       NULL,
       NULL,
       'GATHER',
       OBJTAB,
       NULL,
       TRUE,
       TRUE,
       OBJ_FILTER_LIST
);
PRINT OBJTAB.COUNT;
       FOR I IN 0..OBJTAB.COUNT-1 LOOP
             PRINT OBJTAB(I).OWNNAME;
             PRINT OBJTAB(I).OBJTYPE;
             PRINT OBJTAB(I).OBJNAME;
```

```
                PRINT OBJTAB(I).PARTNAME;
                PRINT OBJTAB(I).SUBPARTNAME;
                PRINT '-------- ';
            END LOOP;
        END;
END;
/
```

打印结果：

```
1
PERSON
TABLE
ADDRESS
NULL
NULL
```

22.6 执行计划

执行计划是 SQL 语句的执行方式，由查询优化器为语句设计的执行方式，交给执行器去执行。

（一）在 SQL 命令行使用 EXPLAIN 可以打印出语句的执行计划。

例如：

建表和建索引语句：

```
CREATE TABLE T1(C1 INT,C2 CHAR);
CREATE TABLE T2(D1 INT,D2 CHAR);
CREATE  INDEX  IDX_T1_C1  ON  T1(C1);
```

```
INSERT INTO T1 VALUES(1,'A');
INSERT INTO T1 VALUES(2,'B');
INSERT INTO T1 VALUES(3,'C');
INSERT INTO T1 VALUES(4,'D');
INSERT INTO T2 VALUES(1,'A');
INSERT INTO T2 VALUES(2,'B');
INSERT INTO T2 VALUES(5,'C');
INSERT INTO T2 VALUES(6,'D');
```

打印执行计划：

```
EXPLAIN SELECT A.C1+1,B.D2 FROM T1 A, T2 B WHERE A.C1 = B.D1;
```

执行计划如下：

```
1    #NSET2: [1, 12, 56]
2        #PRJT2: [1, 12, 56]; exp_num(2), is_atom(FALSE); INFO_BITS(0); prjt_batch_exec(1); spl_info(NULL)
3            #NEST LOOP INDEX JOIN2: [1, 12, 56]
4                #CSCN2: [1, 4, 52]; INDEX33555537(T2 as B); btr_scan(1); need_slct(0); prejudge_iescn(0)
5                #SSEK2: [1, 3, 4]; scan_type(ASC), IDX_T1_C1(T1 as A), scan_range[B.D1,B.D1], is_global(0)
```

这个执行计划看起来就像一棵树，执行过程为：控制流从上向下传递，数据流从下向上传递。其中，类似[0, 16, 9]这样的三个数字，分别表示估算的操作符代价、处理的记录行数和每行记录的字节数。同一层次中的操作符，如本例中的 CSCN2 和 SSEK2，由父节点 NEST LOOP INDEX JOIN2 控制它们的执行顺序。

该计划的大致执行流程如下：

1) CSCN2：扫描 T2 表的聚集索引，数据传递给父节点索引连接；

2) NEST LOOP INDEX JOIN2：当左孩子有数据返回时取右侧数据；

3) SSEK2：利用 T2 表当前的 D1 值作为二级索引 IDX_T1_C1 定位查找的 KEY，返回结果给父节点；

4) NEST LOOP INDEX JOIN2：如果右孩子有数据则将结果传递给父节点 PRJT2，否则继续取左孩子的下一条记录；

5) PRJT2：进行表达式计算 C1+1, D2；

6) NSET2：输出最后结果；

7) 重复过程 1) ~ 4)直至左侧 CSCN2 数据全部取完。

用户如果想了解更多关于操作符的知识，请查看动态视图 V$SQL_NODE_NAME，手册的附录 4 执行计划操作符给出了常用操作符的说明。

在执行计划的输出结果中，单独展示谓词信息（Predicate Information），用于说明各执行计划节点对应的条件表达式。谓词信息位于执行计划主体信息之后，以 Predicate Information 的形式集中列出。标准结构如下：

```
Predicate Information (identified by operation id):
---------------------------------------------------------------------------
---
<operation_id> - <predicate_type> (<predicate_expression>)
```

参数说明:

1．<operation_id>：谓词信息显示的操作编号，对应执行计划中的节点编号，指明该谓词隶属于执行计划中的哪一个节点，每个谓词只对应一个节点，一个节点可以包含多个谓词。

2．<predicate_type>：谓词类型，有 access 和 filter 两类。连接条件使用 access

类型，过滤条件、连接的 join condition 使用 filter 类型。

3．<predicate_expres`sion>：优化器内部解析后的条件表达式。

例如：

打印执行计划：

```
EXPLAIN SELECT name, schid FROM sysobjects WHERE subtype$='STAB' AND name NOT IN
(SELECT name FROM sysobjects WHERE name in (SELECT name FROM sysobjects WHERE
SUBTYPE$='STAB') AND TYPE$='DSYNOM');
```

执行计划如下：

```
1  #NSET2: [1, 28, 100]
2    #PRJT2: [1, 28, 100]; exp_num(2), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3      #HASH LEFT SEMI JOIN2: [1, 28, 100]; (ANTI), KEY_NUM(1), KEY(SYSOBJECTS.NAME =
DMTEMPVIEW_889215893.colname), KEY_NULL_EQU(0)
4        #SLCT2: [1, 28, 100]; SYSOBJECTS.SUBTYPE$ = 'STAB'; slct_pushdown(1);
spl_info(NULL); flt_batch_exec(1)
5          #CSCN2: [1, 1130, 100]; SYSINDEXSYSOBJECTS(SYSOBJECTS as SYSOBJECTS);
btr_scan(1); need_slct(1); prejudge_iescn(0)
6      #PRJT2: [1, 1, 96]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
7        #INDEX JOIN SEMI JOIN2: [1, 1, 96]; join condition(SYSOBJECTS.SUBTYPE$ = 'STAB'),
flt_batch_exec(0)
8          #CSEK2: [1, 28, 96]; scan_type(ASC), SYSINDEXSYSOBJECTS(SYSOBJECTS as
SYSOBJECTS), scan_range[('DSYNOM',min,min),('DSYNOM',max,max))
9        #BLKUP2: [1, 3, 96]; SYSINDEXNAMESYSOBJECTS(SYSOBJECTS); use_clu_addr(0)
10         #SSEK2: [1, 3, 96]; scan_type(ASC), SYSINDEXNAMESYSOBJECTS(SYSOBJECTS
as SYSOBJECTS), scan_range[SYSOBJECTS.NAME,SYSOBJECTS.NAME], is_global(0)
```

```
Predicate Information (identified by operation id):
------------------------------------------------
3 - access(SYSOBJECTS.NAME = DMTEMPVIEW_889215893.colname)
4 - filter(SYSOBJECTS.SUBTYPE$ = 'STAB')
7 - filter(SYSOBJECTS.SUBTYPE$ = 'STAB')
```

其中，以下谓词信息对应执行计划中 3 #HASH LEFT SEMI JOIN2 的连接条件 KEY(SYSOBJECTS.NAME = DMTEMPVIEW_889215893.colname)。

```
3 - access(SYSOBJECTS.NAME = DMTEMPVIEW_889215893.colname)
```

以下谓词信息对应执行计划中 4 #SCLT2 中的过滤条件 SYSOBJECTS.SUBTYPE$ = 'STAB'。

```
4 - filter(SYSOBJECTS.SUBTYPE$ = 'STAB')
```

以下谓词信息对应执行计划中 7 #INDEX JOIN SEMI JOIN2 中连接的 join condition(SYSOBJECTS.SUBTYPE$ = 'STAB')。

```
7 - filter(SYSOBJECTS.SUBTYPE$ = 'STAB')
```

（二）可以使用 EXPLAIN FOR 进行执行计划的查看。

例如：

```
EXPLAIN FOR SELECT A.C1+1,B.D2 FROM T1 A, T2 B WHERE A.C1 = B.D1;
```

执行结果如下：

```
LINEID      PLAN_ID     PLAN_NAME  CREATE_TIME                    LEVEL_ID    OPERATION
TAB_NAME
---------- ----------- --------- -------------------------- ----------- ---------
------------ --------
            IDX_NAME                                SCAN_TYPE   SCAN_RANGE     ROW_NUMS
```

```
BYTES          COST
            ---------------------- --------- ----------- -------------------- ----
------- --------------------
            CPU_COST               IO_COST                FILTER JOIN_COND ADVICE_INFO
PSTART         PSTOP
            ------------------- ------------------- ------ --------- ----------- -
---------- -----------
1              1                NULL           2026-04-10 10:48:39.000000 0                NSET2
NULL
            NULL                       NULL         NULL          12                    56            1
            0                          0                          NULL     NULL         NULL          0
0
LINEID         PLAN_ID        PLAN_NAME CREATE_TIME                LEVEL_ID       OPERATION
TAB_NAME
---------- ----------- --------- -------------------------- ----------- ---------
------------ --------
            IDX_NAME                               SCAN_TYPE   SCAN_RANGE     ROW_NUMS
BYTES          COST
            ---------------------- --------- ----------- -------------------- ----
------- --------------------
            CPU_COST               IO_COST                FILTER JOIN_COND ADVICE_INFO
PSTART         PSTOP
            ------------------- ------------------- ------ --------- ----------- -
---------- -----------
```

```
2              1              NULL         2026-04-10 10:48:39.000000 1              PRJT2
NULL
           NULL                    NULL       NULL         12                   56           1
           0                       0                      NULL    NULL         NULL         0
0
LINEID     PLAN_ID     PLAN_NAME CREATE_TIME                LEVEL_ID    OPERATION
TAB_NAME
---------- ----------- --------- -------------------------- ----------- ---------
------------ --------
           IDX_NAME                           SCAN_TYPE  SCAN_RANGE    ROW_NUMS
BYTES        COST
           ---------------------- --------- ----------- ------------------- ----
------- ---------------------
           CPU_COST               IO_COST                FILTER JOIN_COND ADVICE_INFO
PSTART       PSTOP
           ---------------------- ---------------------- ------ --------- ----------- -
---------- -----------
3          1            NULL       2026-04-10 10:48:39.000000 2            NEST LOOP INDEX
JOIN2 NULL
           NULL                    NULL       NULL         12                   56           1
           0                       0                      NULL    NULL         NULL         0
0
LINEID     PLAN_ID     PLAN_NAME CREATE_TIME                LEVEL_ID    OPERATION
TAB_NAME
```

```
---------- ---------- --------- ------------------------- ---------- ---------
------------ --------
          IDX_NAME                              SCAN_TYPE  SCAN_RANGE    ROW_NUMS
BYTES         COST
          ---------------------- --------- ---------- -------------------- ----
------- --------------------
          CPU_COST               IO_COST                FILTER JOIN_COND ADVICE_INFO
PSTART        PSTOP
          ---------------------- ---------------------- ------ --------- ---------- -
---------- -----------
4             1              NULL         2026-04-10 10:48:39.000000 3              CSCN2
T2
          INDEX33555479              NULL         NULL           4                        52
1
          0                      0                      NULL   NULL        NULL          0
0
LINEID        PLAN_ID        PLAN_NAME CREATE_TIME                LEVEL_ID      OPERATION
TAB_NAME
---------- ---------- --------- ------------------------- ---------- ---------
------------ --------
          IDX_NAME                              SCAN_TYPE  SCAN_RANGE    ROW_NUMS
BYTES         COST
          ---------------------- --------- ---------- -------------------- ----
------- --------------------
```

```
           CPU_COST                IO_COST                FILTER  JOIN_COND  ADVICE_INFO
PSTART       PSTOP
           ---------------------- ---------------------- ------ --------- ----------- -
---------- -----------
5              1                NULL          2026-04-10  10:48:39.000000  3                SSEK2
T1
           IDX_T1_C1[IS_GLOBAL(0)]  ASC            [B.D1,B.D1]  3                          4
1
           0                      0                      NULL     NULL       NULL         0
0
```

EXPLAIN FOR 的每一列定义如下:

序号	列	说明
1	PLAN_ID	执行计划 ID
2	PLAN_NAME	执行计划名称
3	CREATE_TIME	执行计划生成时间
4	LEVEL_ID	执行计划操作符在计划树第几层
5	OPERATION	执行计划操作符名称
6	TAB_NAME	表名
7	IDX_NAME	索引名
8	SCAN_TYPE	扫描类型
9	SCAN_RANGE	扫描范围
10	ROW_NUMS	执行计划操作符结果集预测行数
11	BYTES	执行计划操作符行数据处理长度

12	COST	代价
13	CPU_COST	CPU 代价
14	IO_COST	IO 代价
15	FILTER	执行计划操作符上过滤条件
16	JOIN_COND	连接条件
17	ADVICE_INFO	给用户的优化建议
18	PSTART	分区表起始分区
19	PSTOP	分区表结束分区

22.6.1 自适应计划

在 DM 优化器进行代价估算时，如果子节点是一个复杂查询，可能使得对于子节点的代价估算不准确，导致最终选择的计划在执行时并非最优计划。因此，DM 引入了自适应计划机制。

将 INI 参数 OPTIMIZER_MODE 和 ADAPTIVE_NPLN_FLAG 都置为 1，启用自适应计划机制。此时，优化器会自动判断在某些情况下，复杂查询的子节点可能导致代价估算不准确，则会为该节点生成一个备用计划节点。在实际执行到该节点时，根据准确的代价信息确定是否需要采用备用计划。

例如：

建表、插入数据并建索引：

```
DROP TABLE T1;

DROP TABLE T2;



CREATE TABLE T1(C1 INT,C2 INT);

CREATE TABLE T2(D1 INT,D2 INT);
```

```
INSERT INTO T1 VALUES(1,1);
INSERT INTO T1 VALUES(1,2);
INSERT INTO T2 VALUES(1,1);
INSERT INTO T2 VALUES(2,1);
INSERT INTO T2 VALUES(3,1);
INSERT INTO T2 VALUES(4,1);
INSERT INTO T2 VALUES(5,1);
COMMIT;

CREATE INDEX IND1 ON T1(C2);
```

打印执行计划：

```
EXPLAIN SELECT * FROM T1 ,T2 ,T1 T3 WHERE T2.D1= T3.C1 AND T1.C2=T2.D2;
```

执行计划如下：

```
1    #NSET2: [1, 5, 24]
2       #PRJT2: [1, 5, 24]; exp_num(6), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3          #HASH2 INNER JOIN: [1, 5, 24];   KEY_NUM(1); KEY(T2.D2=T1.C2) KEY_NULL_EQU(0)
4             #NEST LOOP INDEX JOIN2: [1, 5, 24]
5                #ACTRL: [1, 5, 24]
6                   #HASH2 INNER JOIN: [1, 5, 16];   KEY_NUM(1); KEY(T3.C1=T2.D1) KEY_NULL_EQU(0)
7                      #CSCN2: [1, 2, 8]; INDEX33555539(T1 as T3); btr_scan(1); need_slct(0);
prejudge_iescn(0)
8                         #CSCN2: [1, 5, 8]; INDEX33555540(T2); btr_scan(1); need_slct(0);
prejudge_iescn(0)
```

```
9                #BLKUP2: [1, 1, 4]; IND1(T1); use_clu_addr(0)

10                        #SSEK2: [1, 1, 4]; scan_type(ASC), IND1(T1), scan_range[T2.D2,T2.D2],
is_global(0)

11            #CSCN2: [1, 2, 8]; INDEX33555539(T1); btr_scan(1); need_slct(0); prejudge_iescn(0)


Predicate Information (identified by operation id):

---------------------------------------------------

3 - access(T2.D2 = T1.C2)

6 - access(T3.C1 = T2.D1)
```

可以看到执行计划中有一个 ACTRL 操作符，它说明优化器为这一条 SQL 语句生成了备用计划。ACTRL 是控制备用计划转换的操作符，其上面一层 NEST LOOP INDEX JOIN2 为默认的主计划，再上面一层 HASH2 INNER JOIN 则为备用计划。ACTRL 操作符计算下层孩子节点的代价，决定采用默认主计划还是备用计划。

需要说明的是，MPP 和并行查询不支持自适应计划。

22.7 使用索引

为了提高查询效率，用户一般会在表中创建索引。查询中的条件列为索引列时，如果索引扫描代价最小，优化器就会采用索引扫描。索引扫描有多种方式，例如，索引等值查询、索引范围查询。如果查询列属于索引列的子集，则通过索引扫描就可以获得数据，否则，还需要根据 ROWID 或者 PK 在聚集索引中定位记录。

常用的索引类型有唯一索引、组合索引、函数索引。各自有不同的使用场景。

1. 条件列具有 UNIQUE 约束，则可以创建唯一索引，减少索引扫描次数；
2. 条件列是多个列，而且可以过滤掉大部分数据，可以在多个列上创建组合索引，把等值条件列作为组合索引的首列；

3. 条件列使用确定性函数(同样环境下多次执行得到相同的结果)，可以创建函数索引，会把函数值进行存储，使用方式与普通索引一样；

22.8 并行查询

并行查询（Parallel Query，PQ）是一种多个线程或进程间协作、共同完成 SQL 计划的优化技术。

根据参与执行的 DM 服务器实例个数，并行查询可以划分为多机并行查询和单机并行查询。单机并行查询仅利用了单机的 CPU、磁盘、内存等机器资源，因此又称为本地并行查询。

DM 支持本地并行和多机并行，并行查询具体实现上采用对称并行技术，即每一个执行者（进程或线程）都执行相同的计划。主要执行者负责向其他并行执行者分发计划、搜集数据并向客户返回最终结果。不管是主执行者，还是从执行者，它们执行的计划都是完全相同的。

22.8.1 多机并行查询

多机并行查询是指在分布式数据库系统中，利用集群中的多台机器同时完成用户请求。

例如，DM MPP 就是一个支持多机并行的数据库系统。用户登录到某个 EP 执行查询操作时，查询计划会被自动打包发送给集群中其它 EP，多个 EP 协同合作完成用户请求。为了使得数据能够在多个 EP 间流动，MPP 引入了专门的数据分发操作符如 MPP GATHER（MGAT）、MPP COLLECT（MCLCT）、MPP BROADCAST（MBRO）和 MPP SCATTER（MSCT）。详情请参考《DM8 大规模并行处理 MPP》。

22.8.2 本地并行查询

本地并行查询（Local Parallel Query，LPQ）是指利用一台数据库服务器的多个线程同

时执行查询操作。只有当主机具有多个 CPU 内核时，开启 LPQ 才能提高查询任务的性能。

22.8.2.1 本地并行步骤

达梦数据库通过三个步骤来完成本地并行查询：首先，启动本地并行查询；其次，确定并行度；再次，确定并行工作线程总个数；最后，执行查询。

并行查询相关参数见下表：

表 22.1 并行查询相关参数

参数名	缺省值	属性	说明
MAX_PARALLEL_DEGREE	1	动态， 会话级	用于设置并行度（即最大并行线程数）。取值范围：1~128，同时要小于等于 PARALLEL_THRD_NUM。1 表示不并行。
PARALLEL_POLICY	0	动态， 会话级	用来设置并行策略。取值范围：0、1 和 2。 0：关闭并行； 1：开启自动并行模式，使用 INI 参数 MAX_PARALLEL_DEGREE 作为并行度。如果同时使用了 HINT 参数 PARALLEL 指定并行度，则 HINT 优先； 2：开启手动并行模式，使用 HINT 参数 PARALLEL 指定并行度，不指定表示不并行。在 DMDPC 环境下，取值 2 等同于取值 1。 不支持 HINT 方式指定该参数
PARALLEL_THRD_NUM	10	静态	用来设置并行工作线程总个数。取值范围 1~65535

22.8.2.1.1 启用本地并行查询

使用 PARALLEL_POLICY 开启或关闭 LPQ。0 表示禁止 LPQ。1 采用自动并行，即系统对任何一个 SQL 都试图使用 LPQ，除非使用了 HINT 限制并行任务个数。2 采用手动并行，即除非使用了 HINT 指定了并行任务个数，否则都不采用 LPQ。

22.8.2.1.2 确定并行度

当 PARALLEL_POLICY=1 时，表示自动并行，使用 INI 参数 MAX_PARALLEL_DEGREE 作为并行度。如果同时使用了 HINT 参数 PARALLEL 指定并行度，则 HINT 优先。

当 PARALLEL_POLICY=2 时，表示手动并行，只能使用 HINT 参数 PARALLEL 指定并行度。在 DMDPC 环境下，取值 2 等同于取值 1。

1． 在 INI 参数中设置默认值

当 PARALLEL_POLICY=1 时，使用 MAX_PARALLEL_DEGREE 设置单个查询或并行创建索引等操作的并行度。

例 在 INI 参数中将 MAX_PARALLEL_DEGREE 设置为 3 的格式如下：

```
MAX_PARALLEL_DEGREE          3
```

然后，使用一般的 SQL 语句查询即可执行并行查询，不需要使用 HINT。如：

```
SELECT * FROM SYSOBJECTS;
```

2． 在 SQL 语句中使用"PARALLEL"关键字特别指定

当 PARALLEL_POLICY=1 时，也可在 SQL 语句中通过 HINT PARALLEL 指定并行度。当 INI 参数 MAX_PARALLEL_DEGREE 和 HINIT PARALLEL 同时指定并行度时，HINT 优先。

当 PARALLEL_POLICY=2 时，只能在 SQL 语句中通过 HINT PARALLEL 指定并行度，否则不并行。

"PARALLEL"关键字的用法是在数据查询语句的 SELECT 关键字后，增加 HINT 子句来实现。

语法格式

```
/*+ PARALLEL([<表名>] <并行度>) */
```

参数说明

<并行度>：用于设置并行度，即最大并行线程数。取值范围：1~

MAX_PARALLEL_DEGREE。1 表示不并行。

另外，每个语句中仅能设置一次并行任务个数，如果设置了多次，则以第一次设置为准，而且任务个数在全语句中生效。

例 下面的例子中，使用的并行任务个数为 4。

```
SELECT /*+ PARALLEL(4) *//*+ PARALLEL(5) */ * FROM SYSOBJECTS;
```

这种方式能够为单条查询语句设置额外的并行任务个数，以此来提高某些特殊查询任务的性能。

在 DM 分布计算集群（DMDPC）环境中，还支持另一种并行度调整的语法。

语法格式

```
/*+ PARALLEL(#<子任务序号> <并行度>) */
```

参数说明

<子任务序号>：子任务（又称子计划）的序号。如果给定的子任务序号不存在，或者所给定子任务的并行度不支持调整（如使用了 USE_SQC_NO 访问粒度的子任务），则该 HINT 会被忽略。另外，DPC_OPT_FLAG 优化取值 1024 会对各子任务并行度进行调整从而覆盖该 HINT 的作用。

<并行度>：用于设置单个子任务的并行度（即最大并行线程数）。取值范围：1~MAX_PARALLEL_DEGREE。1 表示不并行。

22.8.2.1.3 确定并行工作线程总个数

在执行并行查询任务之前，您需要指定完成并行查询的并行工作线程总个数。并行工作线程总数是系统启动时，创建的并行工作线程的总个数，一般不超过 CPU 内核个数的 2 倍。

1. 并行工作线程数，在 INI 参数中设定

首先，使用 PARALLEL_POLICY 参数来设置并行策略。

当开启本地并行（PARALLEL_POLICY>0）时，需要指定 PARALLEL_THRD_NUM。

PARALLEL_THRD_NUM 重启生效。需要注意的是，若 PARALLEL_POLICY=1，如果 PARALLEL_THRD_NUM=1，则按照 CPU 个数创建并行线程。

例 设置并行策略 PARALLEL_POLICY 为 2，即手动设置并行工作线程数；同时设置 PARALLEL_THRD_NUM 为 4。

```
PARALLEL_POLICY          2
PARALLEL_THRD_NUM        4
```

当然，并非所有的查询都适合使用并行查询。大量占用 CPU 周期的查询最适合采用并行查询的功能。例如，大型表的连接查询、大量数据的聚合和大型结果集的排序等都很适合采用并行查询。对于简单查询（常用于事务处理应用程序）而言，执行并行查询所需的额外协调工作会大于潜在的性能提升。所以，数据库管理员在确定是否需要使用并行策略的时候，需要慎重。

2. 实际使用的线程数，达梦数据库会根据每个并行查询操作自动检测

实际使用线程数是执行计划准备执行之前，系统根据并行任务数和实际空闲的并行工作线程数来自动确定的，不需要用户去干预。此操作所依据的条件如下：首先，检测达梦数据库是否运行在具有多个CPU的计算机上。只有具有多个 CPU 的计算机才能使用并行查询。这是一个硬性的限制条件。其次，检测可用的空闲工作线程是否足够。并行查询到底采用多少线程数，除了跟操作的复杂程度相关外，还跟当时的服务器状态相关，如是否有足够的可用的空闲工作线程数量等。每个并行查询操作都要求一定的工作线程数量才能够执行；而且执行并行计划比执行串行计划需要更多的线程，所需要的线程数量也会随着任务个数的提高而增加。当无法满足特定并行查询执行的线程要求时，数据库引擎就会自动减少任务个数，甚至会放弃并行查询而改为串行计划。所以，即使同一个操作在不同时候可能会采用不同的线程数。

例如，即使设置并行度为4。而实际使用的线程数可能只有3个，或者更少。

22.8.2.1.4 执行查询

当以上内容确定好之后，数据库就会执行具体的查询任务。例如，当开启了自动并行模式 PARALLEL_POLICY=1，并行度为 16，并行线程总个数为 32。在这种搭配下，当以 16 路并行时，最多可以有 2 个 SQL 采用并行查询，并行线程总数不超过 32，此时，如果有其他 SQL，只能采用非并行查询。

22.8.2.2 并行操作符

为了使得数据在同一进程的不同线程间流动，引入了下述用于本地数据分发的操作符。

表 22.2 LPQ 并行执行计划通讯操作符

操作符名称	功能
LOCAL GATHER (LGAT)	主线程收集所有线程数据，从线程将数据发送到主线程
LOCAL DISTRIBUTE (LDIS)	各线程间相互分发数据，按照分发列计算行数据的目标线程并发送过去，目标线程负责接收
LOCAL BROADCAST (LBRO)	各个线程将本线程数据发送到其他所有线程
LOCAL SCATTER (LSCT)	主线程发送完整数据到所有从线程，保证每个线程数据都完整，一般和 LGAT 配合使用
LOCAL COLLECT (LCLCT)	数据收集到主线程处理，用于替换顶层的 LOCAL GATHER。相较于 LOCAL GATHER 有额外的处理如数据同步、处理邮件堆积等

22.8.2.3 使用场景

使用手动并行模式时，只需要在 INI 参数中设置好如下 2 个参数，然后执行并行 SQL 查

询语句时，需手动指定并行度。若不指定，将不使用并行。设置的 2 个参数如下:

```
PARALLEL_POLICY          2
PARALLEL_THRD_NUM        4
```

使用自动并行模式时，一般指定如下三个参数:

```
MAX_PARALLEL_DEGREE              3
PARALLEL_POLICY                  1
PARALLEL_THRD_NUM          10
```

另外，当 PARALLEL_POLICY 为 0 时，即使有并行任务，也不支持并行。

然后，执行语法格式类似“SELECT * FROM SYSOBJECTS;”的并行 SQL 语句即可，本条语句使用的并行度为 3。

当然,如果单条查询语句不想使用默认并行任务数,可以通过在 SQL 语句中增加 HINT，通过“PARALLEL”关键字来特别指定。此时，执行的并行 SQL 语句格式为“SELECT /*+ PARALLEL(SYSOBJECTS 4) */ * FROM SYSOBJECTS;”，本条语句使用的并行任务数为 4。

22.8.3 多机和本地并行查询

在 DM MPP 系统中开启 LPQ，计划执行时就会用到多机并行和本地并行。如下面 TPCH 的 Q1 查询语句。

```
--##建表脚本
CREATE HUGE TABLE LINEITEM
(
L_ORDERKEY                BIGINT NOT NULL,
L_PARTKEY                 INT NOT NULL,
L_SUPPKEY                 INT NOT NULL,
L_LINENUMBER              INT NOT NULL,
```

```
L_QUANTITY                    FLOAT NOT NULL,
L_EXTENDEDPRICE               FLOAT NOT NULL,
L_DISCOUNT                    FLOAT NOT NULL,
L_TAX                         FLOAT NOT NULL,
L_RETURNFLAG                  CHAR(1) NOT NULL,
L_LINESTATUS                  CHAR(1) NOT NULL,
L_SHIPDATE                    DATE NOT NULL,
L_COMMITDATE                  DATE NOT NULL,
L_RECEIPTDATE                 DATE NOT NULL,
L_SHIPINSTRUCT                CHAR(25) NOT NULL,
L_SHIPMODE                    CHAR(10) NOT NULL,
L_COMMENT                     VARCHAR(44) NOT NULL,
PRIMARY KEY(L_ORDERKEY, L_LINENUMBER)
)
DISTRIBUTED BY (L_ORDERKEY);
--执行查询
SELECT
l_returnflag,
l_linestatus,
sum(l_quantity) as sum_qty,
sum(l_extendedprice) as sum_base_price,
sum(l_extendedprice*(1-l_discount)) as sum_disc_price,
sum(l_extendedprice*(1-l_discount)*(1+l_tax)) as sum_charge,
avg(l_quantity) as avg_qty,
```

```
avg(l_extendedprice) as avg_price,
avg(l_discount) as avg_disc,
count(*) as count_order
FROM lineitem
WHERE l_shipdate <= date' 1998-12-01' - interval '90' day
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

该查询为单表过滤后的分组和排序。一个可能的计划形态如下：

```
1    #NSET2: [5, 1, 141]
2      #PRJT2: [5, 1, 141]; exp_num(10), is_atom(FALSE); INFO_BITS(0)
3              #SORT3: [5, 1, 141]; key_num(2), partition_key_num(0), is_distinct(FALSE),
top_flag(0), is_adaptive(0)
4               #HAGR2: [4, 1, 141]; grp_num(2), sfun_num(8), distinct_flag[0,0,0,0,0,0,0,0];
slave_empty(0) keys(DMTEMPVIEW_889214018.TMPCOL0, DMTEMPVIEW_889214018.TMPCOL1)
5                #MPP COLLECT: [1, 4, 141]; op_id(2) n_grp_by (0) n_cols(0) n_keys(0)
for_sync(FALSE)
6                 #LOCAL COLLECT: [1, 1, 141]; op_id(1) n_grp_by (0) n_cols(0) n_keys(0)
for_sync(FALSE)
7                  #PRJT2: [1, 1, 141]; exp_num(7), is_atom(FALSE); INFO_BITS(0)
8                    #HFLKUP2: [1, 1, 141]; (LINEITEM)
9                      #SLCT2: [1, 1, 141]; LINEITEM.L_SHIPDATE <= var2; slct_pushdown(0)
10                       #HFSEK2: [1, 1, 141]; (LINEITEM), scan_type(LE)(null2,1998-12-01-exp12]


Predicate Information (identified by operation id):
```

```
------------------------------------------------

9 - filter(LINEITEM.L_SHIPDATE <= var2)
```

假定上述计划执行在 EP1~EP4 共四个 EP 上执行，每个 EP 设置的并行度为 8，那么该计划一共有 8 * 4= 32 个线程协作完成。执行的大体流程是，首先每个 EP 的 8 路线程各自获取 LIMEITEM 表数据，经过查询条件过滤、检索和投影等运算后得到各自线程的最终结果，各自线程的结果经过本地收集（#6），交由各 EP 内的主线程 x，每个 EP 的主线程再将结果汇总至主 EP（#5），主 EP 再进行分组聚合等操作，得到最终结果。

22.9 查询计划重用

如果同一条语句执行频率较高，或者每次执行的语句仅仅是常量值不同，则可以考虑使用计划重用机制。避免每次执行都需要优化器进行分析处理，可以直接从计划缓存中获取已有的执行计划，减少了分析优化过程，提高执行率。

对于计划重用，达梦数据库提供了 INI 参数 USE_PLN_POOL 来控制，当置为非 0 时，会启用计划重用。

22.10 结果集重用

执行计划的生成与优化是一个非常依赖 CPU 的操作，而执行一个查询获得结果集也是一个非常消耗资源的操作。当系统连续执行两个完全相同的 SQL 语句，其执行计划和结果集很有可能是相同的，如果重新生成和执行计划，会大大浪费系统资源。这时如果使用计划重用和结果集重用，系统的响应速度可以大大提升。

结果集重用是基于计划重用的，如果查询的计划不能缓存，则其查询结果集必然不能缓存。此外，当语句的游标属性为 FORWARD ONLY 时，默认查询不会生成结果集。而参数 BUILD_FORWARD_RS 可以强制在此类查询中生成结果集，以便进行结果集重用。

可通过设置 INI 参数 RS_CAN_CACHE 来控制结果集重用。当置为 0 时表示禁止重用

结果集；置为 1 时表示强制模式（FORCE），在此模式下默认缓存所有可缓存结果集，但是 DBA 也可以通过新增的配置参数以及语句提示等方法取消某些不合适的结果集缓存；置为 2 时表示手动模式（MANUAL），在此模式下默认不缓存查询结果集，但是 DBA 可以通过语句提示等方法指示系统对必要的查询结果集进行缓存。

当 RS_CAN_CACHE 为 1 时，还可以通过设置 INI 参数 RS_CACHE_TABLES 和 RS_CACHE_MIN_TIME 对缓存的结果集进行限制和过滤。RS_CACHE_TABLES 指定可以缓存结果集的基表清单，只有查询涉及的所有基表全部在参数指定范围内，此查询才会缓存结果集。RS_CACHE_MIN_TIME 则指定了缓存结果集的查询语句执行时间的下限，只有实际执行时间不少于指定值的查询结果集才会缓存。

DBA 可以通过在 SQL 语句中设置“RESULT_CACHE”或“NO_RESULT_CACHE” HINT 手动指示查询的结果集是否缓存。如：

```
select /*+ RESULT_CACHE */ id, name from sysobjects;
```

或者

```
SELECT /*+ NO_RESULT_CACHE */ id, name from sysobjects;
```

在语句中使用 HINT 指定结果集缓存的优先级要高于 INI 中相关参数的设置。

还可以使用系统过程 SP_SET_PLN_RS_CACHE 来强制设置指定计划结果集缓存的生效及失效。这个系统过程对结果集缓存的指定高于其它所有结果集缓存的设置。

在以下情况下，DM 不支持结果缓存：

1. 必须是单纯的查询语句计划，PL 脚本中包含查询语句也不能缓存结果集。
2. 查询语句的计划本身必须是缓存的。
3. 守护环境中的备库不支持结果集缓存。
4. MPP 等集群环境下不支持结果集缓存（3、4 两点限制都是因为无法精确控制基表的数据更新时戳）。
5. 查询语句中包含以下任意一项，其结果集都不能缓存：

1） 包含临时表；

2） 包含序列的 CURVAL 或 NEXTVAL；

3） 包含非确定的 SQL 函数或包方法(现有逻辑是不支持所有 SQL 函数或包方法)；

4） 包含 RAND、SYSDATE 等返回值实时变化的系统函数；

5） 包含其它的一些实时要素。

第23章 SQL 调优

23.1 简介

SQL 调优作为数据库性能调优中的最后一个环节，对查询性能产生着直接的影响。在进行正式的 SQL 调优前，用户首先要关注下列几点：

1．达梦数据库安装时的配置参数是否符合应用场景需求；

2．达梦数据库的 INI 配置文件中各项参数是否已经处于最优配置；

3．应用系统中数据库设计是否合理。

本章将介绍定位高负载的 SQL 语句的方法，利用自动 SQL 调整功能进行优化，以及如何开发有效的 SQL 语句和使用优化器提示来影响执行计划。

23.2 调优目标

SQL 调优的整体目标是使用最优的执行计划，这意味着 IO 以及 CPU 代价最小。具体而言调优主要关注下列方面：

- **表扫描**

如果计划中对某大表使用了全索引扫描，那么用户需要关注是否存在着该表的某个查询条件使得过滤后可以淘汰至少一半的数据量。通过添加相应的索引，全索引扫描可能被转换为范围扫描或等值查找。添加的二级索引可以包含该表上所有被选择项以避免 BLKUP2 操作符的查找操作带来的第二次 IO 开销，但无疑这会增加二级索引的大小。用户需权衡二者的利弊以选择正确的处理方式。

- **连接操作的顺序和类型**

多表连接时，不同的连接顺序会影响中间结果集数量的大小，这时调优的目标就是要找到一种能使中间结果保持最小的连接顺序。

对于给定的一个连接或半连接，DM 可以用 HASH 连接、嵌套循环连接、索引连接或者是归并连接实现。通过分析表的数据量大小和索引信息，SQL 调优目标是选择最适宜的操作符。

对半连接而言，HASH 连接还可细分为左半 HASH 和右半 HASH。用户可以通过始终对数据量小的一侧建立 HASH 来进行调优。

■ **分组操作**

分组操作往往要求缓存所有数据以找到属于同一组的所有数据，在大数据量情况下这会带来大量的 IO。用户应该检查 SQL 查询和表上索引信息，如果可以利用包含分组列的索引，那么执行计划就会使用排序分组从而不用缓存中间结果。

23.3 确定高负载的 SQL

在打开监控开关（ENABLE_MONITOR=1）后，可以通过查询动态视图 V$LONG_EXEC_SQLS 或 V$SYSTEM_LONG_EXEC_SQLS 来确定高负载的 SQL 语句。前者显示最近 1000 条执行时间较长的 SQL 语句，后者显示服务器启动以来执行时间最长的 20 条 SQL 语句。例如：

```
SELECT * FROM V$LONG_EXEC_SQLS;
```

或者

```
SELECT * FROM V$SYSTEM_LONG_EXEC_SQLS;
```

23.4 自动 SQL 调整

使用查询优化向导工具，输入需要进行调整的 SQL 语句，向导工具将在分析完执行计划后给出推荐索引的提示。用户只需按提示建立相应索引即可。

23.5 开发有效的 SQL 语句

SQL 语言是一种相当灵活的结构化查询语言。用户可以利用多种不同形式的查询语句完成相同的查询功能。为了使执行效率达到最优，用户需要参考以下原则以开发出有效的 SQL 语句：

1．避免使用 OR 子句

OR 子句在实际执行中会被转换为类似于 UNION 的查询。如果某一个 OR 子句不能利用上索引则会使用全表扫描造成效率低下，应避免使用。

如果 OR 子句都是对同一列进行过滤，用户可以考虑使用 IN VALUE LIST 的过滤形式。如：

```
SELECT ... WHERE CITY = 'SHANGHAI' OR CITY = 'WUHAN' OR CITY = 'BEIJING';
```

调整为

```
SELECT ... WHERE CITY IN( 'SHANGHAI','WUHAN','BEIJING');
```

2．避免使用困难的正则表达式

在 SQL 语言中，LIKE 关键字支持通配符匹配，含通配符的表达式被称为正则表达式。有的正则表达式可以自动优化为非匹配的。例如：a LIKE 'L%'可以优化为 a>='L' AND a <'M',这样就可以用到 a 上的索引。即使没有索引，转换后的比较也更快。再如：a LIKE 'LM_'可以转化为 a>='LM' AND a<'LN' AND a LIKE 'LM_'。虽然仍然包含着通配符匹配，但大大缩小了匹配的范围。

所谓困难的正则表达式是指开头和结尾都为通配符的正则表达式，如'_L%'、'%L_'，优化器没办法缩小它们的匹配范围，也不可能用到索引而必须使用全表扫描。因此要尽可能避免这样的正则表达式。

如果仅仅是开头为通配符，用户可以在列 a 上建立 REVERSE(a)这样一个函数索引，利用函数索引反转待匹配项从而使用函数索引进行范围扫描。

3．灵活使用伪表(SYSDUAL)

首先可以利用伪表进行科学计算，执行语句 SELECT 3*4 FROM SYSDUAL，则可以得到结果 12;

其次，在某些方面使用 SYSDUAL 可提高效率。例如：查询过程中要判断表 t1 中是否有满足 condition1 条件的记录存在，可执行以下语句:

```
SELECT COUNT(*) INTO x FROM t1 WHERE condition1;
```

然后根据变量 x 的取值来判断。但是当 t1 非常大时该语句执行速度很慢，而且由于不知道 SELECT 返回的个数，不能用 SELECT *代替。事实上这个查询可以利用伪表来完成:

```
SELECT 'A' INTO y FROM SYSDUAL WHERE EXISTS (SELECT 1 FROM t1 WHERE condition1);
```

判断 y 值，如等于'A'则 T1 中有记录。调整后的语句执行速度明显比上一句高。

另外，在 DM 的语法里是可以省略 FROM 子句的，这时系统会自动加上 FROM SYSDUAL。因此前面的科学计算例子可以简化为 SELECT 3*4;

4．SELECT 项避免'*'

除非用户确实要选择表中所有列，否则 SELECT *这种写法将让执行器背上沉重的负荷。因为每一列的数据不得不自下往上层层向上传递。不仅仅如此，如果用户查询的是列存储表，那么列存储所带来的 IO 优势将损耗殆尽。

任何时候，用户都要了解表结构和业务需求，小心地选择需要的列并一一给出名称，避免直接用 SELECT *。

5．避免功能相似的重复索引

索引并非越多越好。抛开优化器面对众多索引逐一试探所耗费的时间不谈，如果表上增删改操作频繁，那么索引的维护将会成为大麻烦，尤其是函数索引的计算开销更不能忽略。

6．使用 COUNT(*)统计结果行数

如果对单表查询 COUNT(*)且没有过滤条件，那么 DM 优化器会直接读取相关索引中存储的行数信息，加以回滚段中其他事务插入或删除元组的行数修正，迅速地给出最终结果而

避免对实际数据的读取。相比之下，COUNT(列名)会对数据进行读操作，执行效率远低于COUNT(*)。

即使查询中含有过滤条件，由于 DM 特有的批处理方式，COUNT(*)依旧快于其他写法。这是因为 COUNT(*)无需取得行的具体值而仅仅需要行数这一信息。

需要额外说明的是，COUNT(*)会将 NULL 值计算在内而 COUNT(列名)是不包含 NULL 值的，因此用户要结合应用场景决定是否可以使用 COUNT(*)。

7．使用 EXPLAIN 来查看执行计划

在查询语句或者插入、删除、更新语句前增加 EXPLAIN 关键字，DM 将显示其执行计划而无需实际执行它。查阅 V$SQL_NODE_NAME 动态视图中每个操作符的含义，用户可以很方便且直观地了解数据如何被处理及传递。如果启用了统计信息收集，那么对照执行计划和对动态视图 V$SQL_NODE_HISTORY，V$SQL_NODE_NAME 的查询结果，用户就可以知道在实际执行中每一个操作符执行的时间，进而找出性能瓶颈。

8．UNION 和 UNION ALL 的选择

UNION 和 UNION ALL 的区别是前者会过滤掉值完全相同的元组，为此 UNION 操作符需要建立 HASH 表缓存所有数据并去除重复，当 HASH 表大小超过了 INI 参数指定的限制时还会做刷盘。

因此如果应用场景并不关心重复元组或者不可能出现重复，那么 UNION ALL 无疑优于 UNION。

9．优化 GROUP BY … HAVING

GROUP BY 最常见的实现有 HASH 分组（HAGR）和排序分组（SAGR）。前者需要缓存中间结果；如果用户在 GROUP BY 的列上建立索引，那么优化器就会判断并可能使用上该索引，这时的 GROUP BY 就会变为 SAGR。

HAVING 是分组后对结果集进行的过滤，如果过滤条件无关集函数操作，用户可以考虑

将过滤条件放在WHERE而不是HAVING中。DM优化器会判断并自动转换部分等效于WHERE的 HAVING 子句，但显式地给出最佳 SQL 语句会让优化器工作得更好。

10．使用优化器提示（HINT）

利用经验对优化器的计划选择进行调整，HINT 是 SQL 调整不可或缺的一步。我们将在下一节给出详细说明。

23.6 使用优化器提示

DM 查询优化器采用基于代价的方法。在估计代价时，主要以统计信息或者普遍的数据分布为依据。在大多数情况下，估计的代价都是准确的。但在一些比较特殊的场合，例如缺少统计信息，或统计信息陈旧，或抽样数据不能很好地反映数据分布时，优化器选择的执行计划不是“最优”的，甚至可能是很差的执行计划。

DBA 对于数据分布是很清楚的，他们知道 SQL 语句按照哪种方法执行会最快。在这种情况下，DBA 可以主动进行人工干预，指示优化器按照指定的方法去选择 SQL 的执行计划。

DM 把这种人工干预优化器的方法称为 HINT，它使优化器根据 DBA 的 HINT 提示来生成指定的执行计划。如果优化器无法生成相应的执行计划，该 HINT 将会被忽略。需要注意的是，如果 HINT 的语法有误，DM 并不会报错，而是直接忽略有误的部分 HINT，其他部分继续执行。

HINT 的常见格式如下所示：

```
SELECT /*+ HINT1 [HINT2]*/  列名 FROM 表名 WHERE_CLAUSE ;
UPDATE 表名 /*+ HINT1 [HINT2]*/ SET 列名 =变量 WHERE_CLAUSE ;
DELETE FROM 表名  /*+ HINT1 [HINT2]*/ WHERE_CLAUSE ;
```

23.6.1 INI 参数提示

DBA 可以通过 HINT 方式对 INI 参数的值进行语句级的指定。语句中的 HINT 对 INI 参数

值的设置优先级高于 INI 文件中参数值的设置。通过 HINT 方式只会修改 INI 参数的在本会话中的值，不会改变它在 INI 文件中的值。当通过 HINT 指定 INI 参数时，若 INI 参数名有误，则认为该部分 HINT 存在语法错误，直接忽略该部分 HINT；若设置值与参数类型不兼容，则参数实际取值为默认值；若设置值小于参数取值范围的最小值，则实际取值为最小值；若设置值大于参数取值范围的最大值，则实际取值为最大值。

支持使用 HINT 的 INI 参数可通过 V$HINT_INI_INFO 动态视图查询。支持 HINT 的 INI 参数分为两类：一是 HINT_TYPE 为“OPT”，表示分析阶段使用的参数；二是 HINT_TYPE 为“EXEC”，表示运行阶段使用的参数，运行阶段使用的参数对于视图无效。

例如：

```
SELECT /*+ENABLE_HASH_JOIN(1)*/ * FROM T1,T2 WHERE C1=D1;
```

上面的语句中使用了 HINT，指明在执行此 SQL 时参数 ENABLE_HASH_JOIN 被置为 1。

23.6.2 索引提示

1. 使用索引

目前 DM 提供的 HINT 为表索引的选择 HINT，它指示使用指定索引进行数据检索。

语法：

```
表名 + INDEX + 索引名

或

/*+ INDEX (表名[,] 索引名) {INDEX (表名[,] 索引名)} */
```

一个语句中最多指定 8 个索引。在后一种语法格式中，如果查询中给出了表的别名那么必须使用别名。

假设表 t1 上 id 和 name 列上都存在着单列索引。

```
//数据准备

DROP TABLE T1 CASCADE;
```

```
CREATE TABLE T1 (ID INTEGER,NAME VARCHAR(128));

CREATE INDEX IDX_T1_ID ON T1(ID);

CREATE INDEX IDX_T1_NAME ON T1(NAME);
```

例 1 对于如下查询语句，在查询语句中指定索引。

```
SELECT * FROM T1 WHERE ID > 2011 AND NAME < 'XXX';
```

如果 ID 列上能过滤更多数据，建议指示用索引 IDX_T1_ID。

```
SELECT * FROM T1 INDEX IDX_T1_ID WHERE ID > 2011 AND NAME < 'XXX';
```

或

```
SELECT /*+INDEX(T1, IDX_T1_ID) */ * FROM T1 WHERE ID > 2011 AND NAME < 'XXX';
```

执行计划如下：

```
1    #NSET2: [1, 1, 64]

2       #PRJT2: [1, 1, 64]; exp_num(3), is_atom(FALSE); INFO_BITS(0)

3          #SLCT2: [1, 1, 64]; T1.NAME < 'XXX'; slct_pushdown(0)

4             #BLKUP2: [1, 1, 64]; IDX_T1_ID(T1); use_clu_addr(0)

5                #SSEK2: [1, 1, 64]; scan_type(ASC), IDX_T1_ID(T1), scan_range(2011,max],
is_global(0)



Predicate Information (identified by operation id):

---------------------------------------------------

3 - filter(T1.NAME < 'XXX')
```

例 2 对于如下查询语句，当有多个索引时，要指定使执行计划最优的。

```
SELECT * FROM T1 WHERE ID > 2011 AND NAME < 'XXX' ORDER BY NAME;
```

考虑到后面的 NAME 列排序操作，建议指示使用 NAME 列的索引 IDX_T1_NAME，因为

这样可以在执行过程中省略掉排序操作(执行计划中可以看出来), 比使用 ID 列索引代价小。

```
SELECT * FROM T1 INDEX IDX_T1_NAME WHERE ID > 2011 AND NAME < 'XXX' ORDER BY NAME;
```

或

```
SELECT /*+ INDEX(A IDX_T1_NAME)*/ * FROM T1 A WHERE ID > 2011 AND NAME < 'XXX'

ORDER BY NAME;
```

执行计划如下:

```
1    #NSET2: [1, 1, 64]

2       #PRJT2: [1, 1, 64]; exp_num(3), is_atom(FALSE) ; INFO_BITS(0)

3          #SLCT2: [1, 1, 64]; A.ID > 2011; slct_pushdown(0)

4             #BLKUP2: [1, 1, 64]; IDX_T1_NAME(A); use_clu_addr(0)

5                #SSEK2: [1, 1, 64]; scan_type(ASC), IDX_T1_NAME(T1 as A),

scan_range(null2,'XXX'), is_global(0)



Predicate Information (identified by operation id):

---------------------------------------------------

3 - filter(A.ID > 2011)
```

2. 不使用索引

语法:

```
/*+ NO_INDEX (表名[,] 索引名) { NO_INDEX (表名[,] 索引名)} */
```

可以指定多个索引，则这些索引都不能被使用。一个语句中最多指定 8 个索引。

23.6.3 连接方法提示

DBA 可以通过指定两个表间的连接方法来检测不同连接方式的查询效率，指定的连接可能由于无法实现或代价过高而被忽略。如果连接方法提示中的表名(别名)或索引名无效也会

被自动忽略。

```
//数据准备
DROP TABLE T1 CASCADE;
DROP TABLE T2 CASCADE;

CREATE TABLE T1 (ID INTEGER,NAME VARCHAR(128));
CREATE TABLE T2 (ID INTEGER,NAME VARCHAR(128));

begin
for i in 1..1000 loop
insert into T1 values(i,'dameng'||i);
insert into T2 values(i+500,'damengsh'||i);
end loop;
end;
```

1. USE_HASH

强制两个或多个表间使用哈希连接，最多支持指定八个表名。例如：

```
1    #NSET2: [1, 9800, 104]
2        #PRJT2: [1, 9800, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3            #HASH2 INNER JOIN: [1, 9800, 104];   KEY_NUM(1); KEY(T1.ID=T2.ID) KEY_NULL_EQU(0)
4                    #CSCN2:  [1,  1000,  52];  INDEX33555496(T1);  btr_scan(1);  need_slct(0);
prejudge_iescn(0)
5                    #CSCN2:  [1,  1000,  52];  INDEX33555497(T2);  btr_scan(1);  need_slct(0);
prejudge_iescn(0)
```

```
Predicate Information (identified by operation id):

--------------------------------------------------

3 - access(T1.ID = T2.ID)
```

2. NO_USE_HASH

强制两个或多个表间不能使用哈希连接，最多支持指定八个表名。例如：

```
EXPLAIN SELECT /*+ NO_USE_HASH(T1, T2) */ * FROM T1, T2 WHERE T1.ID = T2.ID;

1    #NSET2: [24685, 9800, 104]

2      #PRJT2: [24685, 9800, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3            #SLCT2: [24685, 9800, 104]; T1.ID = T2.ID; slct_pushdown(0); spl_info(NULL);
flt_batch_exec(1)

4          #NEST LOOP INNER JOIN2: [24685, 9800, 104]

5                  #CSCN2: [1, 1000, 52]; INDEX33555496(T1); btr_scan(1); need_slct(0);
prejudge_iescn(0)

6                  #CSCN2: [1, 1000, 52]; INDEX33555497(T2); btr_scan(1); need_slct(0);
prejudge_iescn(0)

Predicate Information (identified by operation id):

--------------------------------------------------

3 - filter(T1.ID = T2.ID)
```

3. USE_NL

强制两个或多个表间使用嵌套循环连接或嵌套索引连接，最多支持指定八个表名。例如：

```
EXPLAIN SELECT /*+ USE_NL(A, B) */ * FROM T1 A, T2 B WHERE A.ID = B.ID;

1    #NSET2: [24685, 9800, 104]

2      #PRJT2: [24685, 9800, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3            #SLCT2: [24685, 9800, 104]; A.ID = B.ID; slct_pushdown(0); spl_info(NULL);
```

```
flt_batch_exec(1)
4            #NEST LOOP INNER JOIN2: [24685, 9800, 104]
5                #CSCN2: [1, 1000, 52]; INDEX33555496(T1 as A); btr_scan(1); need_slct(0);
prejudge_iescn(0)
6                #CSCN2: [1, 1000, 52]; INDEX33555497(T2 as B); btr_scan(1); need_slct(0);
prejudge_iescn(0)
Predicate Information (identified by operation id):
------------------------------------------------
3 - filter(A.ID = B.ID)
```

4. NO_USE_NL

强制两个或多个表间不能使用嵌套循环连接或嵌套索引连接，最多支持指定八个表名。例如：

```
EXPLAIN SELECT /*+ NO_USE_NL(A, B) */ * FROM T1 A, T2 B WHERE A.ID = B.ID;
1    #NSET2: [1, 9800, 104]
2       #PRJT2: [1, 9800, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3          #HASH2 INNER JOIN: [1, 9800, 104];   KEY_NUM(1); KEY(A.ID=B.ID) KEY_NULL_EQU(0)
4                #CSCN2: [1, 1000, 52]; INDEX33555496(T1 as A); btr_scan(1); need_slct(0);
prejudge_iescn(0)
5                #CSCN2: [1, 1000, 52]; INDEX33555497(T2 as B); btr_scan(1); need_slct(0);
prejudge_iescn(0)

Predicate Information (identified by operation id):
------------------------------------------------
3 - access(A.ID = B.ID)
```

5. USE_NL_WITH_INDEX

当连接情况为左表+右表索引时，强制两个表间使用索引连接，例如：

```
//数据准备
CREATE INDEX IDX_T2_ID ON T2(ID);
//执行 EXPLAIN
EXPLAIN SELECT /*+ USE_NL_WITH_INDEX(T1, IDX_T2_ID) */ * FROM T1, T2 WHERE T1.ID = T2.ID;
1    #NSET2: [6, 3000, 104]
2      #PRJT2: [6, 3000, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3        #NEST LOOP INDEX JOIN2: [6, 3000, 104]
4                  #CSCN2: [1, 1000, 52]; INDEX33555496(T1); btr_scan(1); need_slct(0);
prejudge_iescn(0)
5            #BLKUP2: [6, 3, 4]; IDX_T2_ID(T2); use_clu_addr(0)
6                  #SSEK2: [6, 3, 4]; scan_type(ASC), IDX_T2_ID(T2), scan_range[T1.ID,T1.ID],
is_global(0)
```

6. NO_USE_NL_WITH_INDEX

当连接情况为左表+右表索引时，强制两个表间不能使用索引连接，例如：

```
EXPLAIN SELECT /*+ NO_USE_NL_WITH_INDEX(T1, IDX_T2_ID) */ * FROM T1, T2 WHERE T1.ID = T2.ID;
1    #NSET2: [1, 9800, 104]
2      #PRJT2: [1, 9800, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3        #HASH2 INNER JOIN: [1, 9800, 104];  KEY_NUM(1); KEY(T1.ID=T2.ID) KEY_NULL_EQU(0)
4                  #CSCN2: [1, 1000, 52]; INDEX33555496(T1); btr_scan(1); need_slct(0);
prejudge_iescn(0)
5                  #CSCN2: [1, 1000, 52]; INDEX33555497(T2); btr_scan(1); need_slct(0);
prejudge_iescn(0)
```

```
Predicate Information (identified by operation id):
---------------------------------------------------
3 - access(T1.ID = T2.ID)
//使用完毕，请删除索引
DROP INDEX IDX_T2_ID;
```

7. USE_MERGE

强制两个或多个表间使用归并连接，最多支持指定八个表名。归并连接所用的两个列都必须是索引列。例如：

```
//数据准备
CREATE INDEX IDX_T1_ID ON T1(ID);
CREATE INDEX IDX_T2_ID ON T2(ID);
STAT 100 ON T1(ID);
STAT 100 ON T2(ID);
//执行 EXPLAIN
EXPLAIN SELECT /*+ USE_MERGE(T1,T2) */ * FROM T1, T2 WHERE T1.ID = T2.ID AND T1.ID < 1 AND T2.ID
< 1;
1    #NSET2: [1, 1, 104]
2        #PRJT2: [1, 1, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3            #MERGE INNER JOIN3: [1, 1, 104]; KEY_NUM(1); KEY(T1.ID = T2.ID), KEY_NULL_EQU(0)
4                #BLKUP2: [1, 1, 52]; IDX_T1_ID(T1); use_clu_addr(0)
5                    #SSEK2: [1, 1, 52]; scan_type(ASC), IDX_T1_ID(T1), scan_range(null2,1), is_global(0)
6                #BLKUP2: [1, 1, 52]; IDX_T2_ID(T2); use_clu_addr(0)
7                        #SSEK2: [1, 1, 52]; scan_type(ASC), IDX_T2_ID(T2), scan_range(null2,1),
```

```
is_global(0)


Predicate Information (identified by operation id):
---------------------------------------------------
3 - access(T1.ID = T2.ID)
```

8. NO_USE_MERGE

强制两个或多个表间不能使用归并连接，最多支持指定八个表名。例如：

```
EXPLAIN SELECT /*+ NO_USE_MERGE(T1,T2) */ * FROM T1, T2 WHERE T1.ID = T2.ID AND T1.ID > 1 AND
T2.ID > 1;
1    #NSET2: [1, 999, 104]
2      #PRJT2: [1, 999, 104]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3        #SLCT2: [1, 999, 104]; T2.ID > 1; slct_pushdown(0); spl_info(NULL); flt_batch_exec(1)
4          #HASH2 INNER JOIN: [1, 999, 104];   KEY_NUM(1); KEY(T1.ID=T2.ID) KEY_NULL_EQU(0)
5            #SLCT2: [1, 999, 52]; T1.ID > 1; slct_pushdown(1); spl_info(NULL); flt_batch_exec(1)
6                      #CSCN2:  [1,  1000,  52];  INDEX33555500(T1);  btr_scan(1);  need_slct(1);
prejudge_iescn(0)
7                    #CSCN2:  [1,  1000,  52];  INDEX33555501(T2);  btr_scan(1);  need_slct(0);
prejudge_iescn(0)


Predicate Information (identified by operation id):
---------------------------------------------------
3 - filter(T2.ID > 1)
4 - access(T1.ID = T2.ID)
5 - filter(T1.ID > 1)
```

```
//使用完毕，请删除索引

DROP INDEX IDX_T1_ID;

DROP INDEX IDX_T2_ID;
```

9. SEMI_GEN_CROSS

优先采用半连接转换为等价的内连接，仅 OPTIMIZER_MODE=1 有效。例如：

```
EXPLAIN SELECT /*+ SEMI_GEN_CROSS  OPTIMIZER_MODE(1) */ COUNT(*) FROM T1 WHERE T1.ID
IN (SELECT T2.ID FROM T2);

1    #NSET2: [2, 1, 8]

2      #PRJT2: [2, 1, 8]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3        #AAGR2: [2, 1, 8]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)

4                          #HASH2 INNER JOIN: [2, 1000, 8]; LKEY_UNIQUE KEY_NUM(1);
KEY(DMTEMPVIEW_889216439.colname=T1.ID) KEY_NULL_EQU(0)

5              #DISTINCT: [1, 1000, 4]

6                #PRJT2: [1, 1000, 4]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

7                  #SSCN: [1, 1000, 4]; IDX_T2_ID(T2); btr_scan(1); is_global(0)

8              #SSCN: [1, 1000, 4]; IDX_T1_ID(T1); btr_scan(1); is_global(0)


Predicate Information (identified by operation id):

---------------------------------------------------

4 - access(DMTEMPVIEW_889216439.colname = T1.ID)
```

10. NO_SEMI_GEN_CROSS

不采用半连接转换为等价的内连接，仅 OPTIMIZER_MODE=1 有效。例如：

```
EXPLAIN SELECT /*+ NO_SEMI_GEN_CROSS  OPTIMIZER_MODE(1) */ COUNT(*) FROM T1 WHERE
T1.ID IN (SELECT T2.ID FROM T2);
```

```
1    #NSET2: [1, 1, 4]
2      #PRJT2: [1, 1, 4]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3        #AAGR2: [1, 1, 4]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)
4          #HASH LEFT SEMI JOIN2: [1, 1000, 4]; KEY_NUM(1), KEY(T1.ID = T2.ID), KEY_NULL_EQU(0)
5                    #CSCN2: [1, 1000, 4]; INDEX33555476(T1); btr_scan(1); need_slct(0);
prejudge_iescn(0)
6                    #CSCN2: [1, 1000, 4]; INDEX33555477(T2); btr_scan(1); need_slct(0);
prejudge_iescn(0)

Predicate Information (identified by operation id):
---------------------------------------------------
4 - access(T1.ID = T2.ID)
```

11. USE_CVT_VAR

优先采用变量改写方式实现连接，适合驱动表数据量少而另一侧计划较复杂的场景，目前支持变量改写的连接方式有 NEST LOOP INNER JOIN2、NEST LOOP LEFT JOIN2、NEST LOOP SEMI JOIN2，仅 OPTIMIZER_MODE=1 有效。例如：

```
EXPLAIN SELECT /*+ USE_CVT_VAR OPTIMIZER_MODE(1) */ COUNT(*) FROM T1 A WHERE A.id =
1001 AND EXISTS (SELECT 1 FROM T1 B, T1 C WHERE B.id = C.id AND A.name= B.name);
1    #NSET2: [1, 1, 52]
2      #PRJT2: [1, 1, 52]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3        #AAGR2: [1, 1, 52]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)
4                  #NEST LOOP SEMI JOIN2: [1, 1, 52];  join condition(A.NAME =
DMTEMPVIEW_889216444.colname), flt_batch_exec(0)[with var]
5              #BLKUP2: [1, 1, 52]; IDX_T1_ID(A); use_clu_addr(0)
```

```
6                #SSEK2: [1, 1, 52]; scan_type(ASC), IDX_T1_ID(T1 as A), scan_range[1001,1001],
is_global(0)
7              #PRJT2: [1, 25, 56]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
8                #HASH2 INNER JOIN: [1, 25, 56];   KEY_NUM(1); KEY(B.ID=C.ID) KEY_NULL_EQU(0)
9                      #SLCT2: [1, 25, 52]; B.NAME = var1; slct_pushdown(1); spl_info(NULL);
flt_batch_exec(1)
10                   #CSCN2: [1, 1000, 52]; INDEX33555500(T1 as B); btr_scan(1); need_slct(1);
prejudge_iescn(0)
11                 #SSCN: [1, 1000, 4]; IDX_T1_ID(T1 as C); btr_scan(1); is_global(0)


Predicate Information (identified by operation id):
---------------------------------------------------
4 - filter(A.NAME = DMTEMPVIEW_889216444.colname)
8 - access(B.ID = C.ID)
9 - filter(B.NAME = var1)
```

12. NO_USE_CVT_VAR

不考虑变量改写方式实现连接，仅 OPTIMIZER_MODE=1 有效。例如：

```
EXPLAIN SELECT /*+ NO_USE_CVT_VAR OPTIMIZER_MODE(1) */ COUNT(*) FROM   T1 A WHERE A.id
= 1001 AND EXISTS (SELECT 1 FROM T1 B, T1 C WHERE B.id = C.id AND A.name= B.name);
1    #NSET2: [2, 1, 52]
2       #PRJT2: [2, 1, 52]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3         #AAGR2: [2, 1, 52]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)
4                      #HASH LEFT SEMI JOIN2: [2, 1, 52]; KEY_NUM(1), KEY(A.NAME =
DMTEMPVIEW_889216448.colname), KEY_NULL_EQU(0)
```

```
5               #BLKUP2: [1, 1, 52]; IDX_T1_ID(A); use_clu_addr(0)
6                 #SSEK2: [1, 1, 52]; scan_type(ASC), IDX_T1_ID(T1 as A), scan_range[1001,1001],
is_global(0)
7               #PRJT2: [1, 1000, 56]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
8                 #HASH2 INNER JOIN: [1, 1000, 56];   KEY_NUM(1); KEY(C.ID=B.ID) KEY_NULL_EQU(0)
9                   #SSCN: [1, 1000, 4]; IDX_T1_ID(T1 as C); btr_scan(1); is_global(0)
10                  #CSCN2: [1, 1000, 52]; INDEX33555500(T1 as B); btr_scan(1); need_slct(0);
prejudge_iescn(0)


Predicate Information (identified by operation id):
---------------------------------------------------
4 - access(A.NAME = DMTEMPVIEW_889216448.colname)
8 - access(C.ID = B.ID)
```

13. ENHANCED_MERGE_JOIN

一般情况下，归并连接需要左右孩子的数据按照连接列有序，使用此优化器提示时，优化器将考虑通过插入排序操作符的方式实现归并连接，仅 OPTIMIZER_MODE=1 有效。例如：

```
//不加 HINT 时计划，使用了哈希连接
EXPLAIN SELECT /*+ stat(T1 1M) stat(T2 1M) */COUNT(*) FROM T1, T2 WHERE T1.NAME=T2.NAME AND
T1.ID=T2.ID;
1    #NSET2: [442, 1, 104]
2       #PRJT2: [442, 1, 104]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3         #AAGR2: [442, 1, 104]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)
4               #HASH2 INNER JOIN: [442, 2, 104];   KEY_NUM(2); KEY(T1.NAME=T2.NAME AND
T1.ID=T2.ID) KEY_NULL_EQU(0, 0)
```

```
5                #CSCN2: [115, 1000000, 52]; INDEX33555500(T1); btr_scan(1); need_slct(0);
prejudge_iescn(0)
6                #CSCN2: [115, 1000000, 52]; INDEX33555501(T2); btr_scan(1); need_slct(0);
prejudge_iescn(0)


Predicate Information (identified by operation id):
---------------------------------------------------
4 - access(T1.NAME = T2.NAME AND T1.ID = T2.ID)

// 加 HINT 后计划，通过增加排序以使用了归并连接
EXPLAIN SELECT /*+ENHANCED_MERGE_JOIN OPTIMIZER_MODE(1) stat(T1 1M) stat(T2 1M)
*/COUNT(*) FROM T1, T2 WHERE T1.NAME=T2.NAME AND T1.ID=T2.ID;
1    #NSET2: [436, 1, 104]
2       #PRJT2: [436, 1, 104]; exp_num(1), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3         #AAGR2: [436, 1, 104]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)
4           #MERGE INNER JOIN3: [436, 1471621, 104]; KEY_NUM(2); KEY(T1.NAME = T2.NAME AND
T1.ID = T2.ID), KEY_NULL_EQU(0, 0)
5                        #SORT3: [122, 1000000, 52]; key_num(2), partition_key_num(0),
is_distinct(FALSE), top_flag(0), is_adaptive(0)
6                #CSCN2: [115, 1000000, 52]; INDEX33555476(T1); btr_scan(1); need_slct(0);
prejudge_iescn(0)
7                        #SORT3: [122, 1000000, 52]; key_num(2), partition_key_num(0),
is_distinct(FALSE), top_flag(0), is_adaptive(0)
8                #CSCN2: [115, 1000000, 52]; INDEX33555477(T2); btr_scan(1); need_slct(0);
```

```
prejudge_iescn(0)



Predicate Information (identified by operation id):

---------------------------------------------------

4 - access(T1.NAME = T2.NAME AND T1.ID = T2.ID)
```

23.6.4 连接顺序提示

多表连接时优化器会考虑各种可能的排列组合顺序。使用 ORDER HINT 指定连接顺序提示可以缩小优化器试探的排列空间，进而得到接近 DBA 所期望的查询计划。如果连接顺序和连接方法提示同时指定且二者间存在自相矛盾，优化器会以连接顺序提示为准。

语法：

```
/*+ ORDER (T1, T2 , T3, … tn ) */
```

本节中的例子用到 4 个表 T1, T2 , T3, T4。

```
CREATE TABLE T1(C1 INT,C2 VARCHAR);

CREATE TABLE T2(D1 INT,D2 VARCHAR);

CREATE TABLE T3(E1 INT,E2 VARCHAR);

CREATE TABLE T4(F1 INT,F2 VARCHAR);
```

例如：

```
SELECT * FROM t1, T2 , T3, T4 WHERE …
```

如果期望表的连接顺序是 T1, T2, T3，那么可以加入这样的提示：

```
SELECT /*+ ORDER(T1, T2, T3 )*/* FROM T1, T2 , T3, T4 WHERE …
```

在指定上述连接顺序后，类似 T4,T1,T2,T3 或 T1,T2,T4,T3 的顺序会被考虑；而诸如 T3,T1,T2 或 T1,T3,T2 的顺序不被考虑。

连接顺序也可以和连接方法同时指定用于得到更特定的执行计划。

例如：

```
EXPLAIN SELECT /*+ OPTIMIZER_MODE(1), ORDER(T1,T2,T3,T4) ,USE_HASH(T1,T2),
USE_HASH(T2,T3), USE_HASH(T3,T4)*/* FROM T1,T2,T3,T4 WHERE T1.c1=T2.d1 AND T2.d2 = T3.e2
AND T3.e1 = T4.f1;
```

此时的执行计划如下：

```
1    #NSET2: [2, 1, 208]
2       #PRJT2: [2, 1, 208]; exp_num(8), is_atom(FALSE); INFO_BITS(0); ...; spl_info(NULL)
3          #HASH2 INNER JOIN: [2, 1, 208];   KEY_NUM(1); KEY(T3.E1=T4.F1) KEY_NULL_EQU(0)
4             #HASH2 INNER JOIN: [1, 1, 156];   KEY_NUM(1); KEY(T2.D2=T3.E2) KEY_NULL_EQU(0)
5                #HASH2 INNER JOIN: [1, 1, 104];   KEY_NUM(1); KEY(T1.C1=T2.D1) KEY_NULL_EQU(0)
6                            #CSCN2:  [1,  1,  52];  INDEX33555506(T1);  btr_scan(1);  need_slct(0);
prejudge_iescn(0)
7                            #CSCN2:  [1,  1,  52];  INDEX33555507(T2);  btr_scan(1);  need_slct(0);
prejudge_iescn(0)
8                          #CSCN2:  [1,  1,  52];  INDEX33555508(T3);  btr_scan(1);  need_slct(0);
prejudge_iescn(0)
9                        #CSCN2:  [1,  1,  52];  INDEX33555509(T4);  btr_scan(1);  need_slct(0);
prejudge_iescn(0)


Predicate Information (identified by operation id):
---------------------------------------------------
3 - access(T3.E1 = T4.F1)
4 - access(T2.D2 = T3.E2)
5 - access(T1.C1 = T2.D1)
```

23.6.5 统计信息提示

优化器在计划优化阶段会自动获取基表的行数。但是一些特殊类型的表行数估算并不准确，或者 DBA 希望了解表大小对计划影响的时候，需要手动设置表的行数。

语法：

```
/*+ STAT (表名, 行数) */
```

统计信息提示只能针对基表设置，视图和派生表等对象设置无效。如果表对象存在别名则必须使用别名。行数只能使用整数，或者整数+K（千），整数+M（百万），整数+G（十亿）。行数提示设置后，统计信息的其它内容也会做相应的调整。

例如：

```
CREATE TABLE T_S(C1 INT);

INSERT INTO T_S SELECT LEVEL FROM DUAL CONNECT BY LEVEL<= 100;

COMMIT;

STAT 100 ON T_S(C1);

EXPLAIN SELECT /*+ STAT(T_S,1M) */ * FROM T_S WHERE C1 <= 10;
```

计划如下：

```
1    #NSET2: [117, 100000, 16]

2       #PRJT2: [117, 100000, 16]; exp_num(2), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3                #SLCT2: [117, 100000, 16]; T_S.C1 <= 10; slct_pushdown(1); spl_info(NULL);
flt_batch_exec(1)

4                 #CSCN2: [117, 1000000, 16]; INDEX33555510(T_S); btr_scan(1); need_slct(1);
prejudge_iescn(0)



Predicate Information (identified by operation id):
```

```
------------------------------------------------

3 - filter(T_S.C1 <= 10)
```

不使用 HINT 时计划：

```
EXPLAIN SELECT * FROM T_S WHERE C1 <= 10;

1    #NSET2: [1, 10, 16]

2       #PRJT2: [1, 10, 16]; exp_num(2), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3          #SLCT2: [1, 10, 16]; T_S.C1 <= 10; slct_pushdown(1); spl_info(NULL); flt_batch_exec(1)

4                      #CSCN2:  [1,  100,  16];  INDEX33555510(T_S);  btr_scan(1);  need_slct(1);
prejudge_iescn(0)


Predicate Information (identified by operation id):

------------------------------------------------

3 - filter(T_S.C1 <= 10)
```

23.6.6 MPP 本地对象提示

MPP 环境下，提供一种将用户表或动态视图作为本地对象处理的方法，通过指示符 LOCAL_OBJECT(对象名/别名）进行处理。

语法：

```
/*+LOCAL_OBJECT(对象名/别名)*/
```

对于系统表当做本地对象的处理，本方法不适用，系统表只能在主站点才能做本地对象处理。

例 MPP 环境下，执行下面测例：

```
drop table t1;

create table t1(c1 int) distributed by(c1);
```

```
insert into t1 values(0);

insert into t1 values(1);

commit;
```

将表 t1 作为本地对象进行处理：

```
select /*+local_object(t1)*/ * from t1;
```

将表 t1 作为本地对象进行处理，支持使用 t1 的别名 X。

```
select /*+local_object(x)*/ * from t1 x;
```

23.6.7 忽略重复键值提示

当执行 INSERT 操作时，如果存在 UNIQUE 索引，那么可能会发生重复键值冲突。使用 HINT IGNORE_ROW_ON_DUPKEY_INDEX 则可以忽略该冲突，冲突数据既不进行插入也不会报错，其他非冲突插入正常进行。

每一个 HINT 仅对应一个索引，如需忽略多个索引上的重复键值冲突，请指定多个 HINT。

语法：

```
/*+IGNORE_ROW_ON_DUPKEY_INDEX(<表名>[(<列名>{,<列名>})])*/
```

若 INSERT 的目标表具有别名，则只有<表名>与别名一致时 HINT 才生效。

支持使用该 HINT 时不指定<列名>，此时忽略表上除函数索引外的所有 UNIQUE 索引的报错。当指定<列名>时，该 HINT 指定的<列名>必须是某个 UNIQUE 索引的前导列或者全部列，否则报错；当作为前导列时，不能出现其它也将这些列作为前导列的 UNIQUE 索引，否则报错。

指定该 HINT 时，不支持 insert 操作的批量优化处理。

例 指定 IGNORE_ROW_ON_DUPKEY_INDEX 不会报错。

```
drop table t1;

create table t1(c1 int, c2 int, c3 int, c4 int);
```

```
create unique index t1i1 on t1(c1,c2,c3);

insert  into t1 values (1,2,1,1);

insert  into t1 values (1,2,1,1); //报错

insert  /*+IGNORE_ROW_ON_DUPKEY_INDEX(t1(c1,c2,c3))*/ into t1 values (1,2,1,1);  //不报错

insert  /*+IGNORE_ROW_ON_DUPKEY_INDEX(t1)*/ into t1 values (1,2,1,1);  //不报错
```

23.6.8 禁用计划缓存提示

使用 HINT PLAN_NO_CACHE 禁用计划缓存，当前语句的执行计划将不会被缓存。

语法：

```
/*+PLAN_NO_CACHE*/
```

例 下面三条 SELECT 语句中，第一条 SELECT 指定了 PLAN_NO_CACHE，它的执行计划生成后将不会被缓存；第二条 SELECT 未指定 PLAN_NO_CACHE，所以它的执行计划被缓存了；第三条 SELECT 将不需要重新生成执行计划，可重用缓存的第二条 select 的执行计划。

```
drop table t1;

create table t1(c1 int);

select /*+PLAN_NO_CACHE*/ * from t1;

select * from t1;

select * from t1;
```

23.6.9 DMDPC 数据分发方式提示

DMDPC 环境下提供了一种专门针对指定的连接、分组、排序、去重和分析函数操作符数据分发方式进行人工干预的优化器提示。该提示被采纳的前提是指定的分发路径有效，因为代价原因没有被优化器选中。

语法:

```
/*+DPC(分发方式探测序号 分发方式字符串)*/
```

其中,分发方式探测序号请在 10053 trace event 生成的 TRACE 文件中查看,以 nth_try 标识;分发方式字符串请参考 23.6.9.1 二元操作符和 23.6.9.2 一元操作符。

例 在 SQL 语句中使用 DMDPC 数据分发方式提示。

```
select /*+ DPC(1 x_merge) */ para_name from v$dm_ini order by para_name;
```

23.6.9.1 二元操作符

对于连接和集合运算二元操作符,可供选择的分发方式字符串参见下表。

表 23.1 二元操作符的分发方式字符串

分发方式字符串	含义
L_NO_R_NO	二元操作符两侧都不添加任何数据分发操作符
L_BRO	二元操作符左侧增加广播操作
R_BRO	二元操作符右侧增加广播操作
L_DIS_R_DIS	分别对二元操作符两侧数据进行分发
L_DIS	对二元操作符左侧数据进行分发
R_DIS	对二元操作符右侧数据进行分发
L_GAT_R_GAT	二元操作符两侧数据汇总到一个线程
ONE_GAT_ONLY	二元操作符一侧为指令,另一侧为非指令。非指令的一侧将数据发送到一个 SP 站点执行
ONE_PART_ONLY	二元操作符一侧为普通表,另一侧将数据发送到此普通表所在的站点执行

例 下面一段 trace 片段展示了采用优化器提示中的二元操作符分发方式的情况。

```
//连接片段,正在试探第 5 个需要分发的操作符 hash left semi join(anti)。
```

```
*** probe best distribute method for hash left semi join(anti) (000000005089E500), DPC
motion nth_try[5], restricts[0x12], self cost 1.000000
    left   motion cost: broadcast = 0.000011, dis = 0.000011, gather = 0.000011
    right motion cost: broadcast = 0.000023, dis = 0.000011, gather = 0.000011
    > try          R_BRO(404), cost 0.062523, n_parallel 2, n_sites 0, best*
    > try L_DIS_R_DIS(405), cost 0.062523, n_parallel 2, n_sites 0, best*
    > try L_GAT_R_GAT(408), cost 1.000023, n_parallel 1, n_sites 0
```

23.6.9.2 一元操作符

对于分组（group）、排序（order）、去重（distinct）和分析函数（analytic function）一元操作符，可供选择的分发方式字符串参见下表。

下表中的 X 代表 group、order、distinct 或 analytic function。例如，当按照 group 进行分发时，X 即表示 group。

表 23.2 一元操作符的分发方式字符串

分发方式字符串	含义
X	直接进行 X
X_DIS_X	本地先 X，在数据分发后进行二次 X
DIS_X	数据分发后进行 X
DIS_SORT_X	数据分发后先进行排序，接下来执行 X
X_GAT_X	本地先 X，数据汇总到一个线程后进行二次 X
GAT_X	数据汇总到一个线程后进行 X
GAT_SORT_X	数据汇总到一个线程后排序，接下来执行 X
X_MERGE	每个线程先分别排序，汇总后由最终的线程进行归并排序

DIS_X_GAT_X	对无分组项且含有 DISTINCT 的集函数，先按集函数参数进行数据分发，然后再对分发的数据进行 X_GAT_X。仅当 INI 参数 GROUP_OPT_FLAG 取值包含 64 时考虑该方式
DIST_DIS_X_GAT_X	和 DIS_X_GAT_X 类似。区别为，在对集函数参数进行数据分发前先做一次去重处理。仅当 INI 参数 GROUP_OPT_FLAG 取值包含 64 时考虑该方式

例 下面一段 trace 展示了采用优化器提示中的二元操作符分发方式的情况。

```
//分组 group 片段，正要探测第 2 个需要分发的 group。
*** probe best distribute method for group(000000000070E1F8), DPC motion nth_try[2],
restricts[0xd], self cost 1.000000
    left   motion cost: broadcast = 236.980132, dis = 236.980132, gather = 236.980132
    self   motion cost: broadcast = 0.000006, dis = 0.000006, gather = 0.000006
    > try              X(420), cost 1.100000, n_parallel 0, n_sites 0, best*
    > try      X_GAT_X(424), cost 3.000006, n_parallel 0, n_sites 0
    > try        GAT_X(425), cost 239.980132, n_parallel 0, n_sites 0


//排序 order 片段，正要探测第 7 个需要分发的 order。
*** probe best distribute method for order(0000000050897128), DPC motion nth_try[7],
restricts[0x45], self cost 1.000000
    left   motion cost: broadcast = 0.000011, dis = 0.000011, gather = 0.000011
    self   motion cost: broadcast = 0.000011, dis = 0.000011, gather = 0.000011
    > try              X(420), cost 1.100000, n_parallel 0, n_sites 0, best*
    > try        GAT_X(425), cost 3.000011, n_parallel 0, n_sites 0
    > try      X_MERGE(427), cost 12.000011, n_parallel 0, n_sites 0
```

```
//排序去重 distinct 片段，正要探测第 1 个需要分发的 distinct。
*** probe best distribute method for distinct(00000000007112E8), DPC motion nth_try[1],
restricts[0x1f], self cost 21.828696
      left   motion cost: broadcast = 3.638299, dis = 1.819149, gather = 1.819149
      self   motion cost: broadcast = 0.036383, dis = 0.018191, gather = 0.018191
      > try              X(420), not available
      > try       X_DIS_X(421), cost 114.344539, n_parallel 1, n_sites 1, best*
      > try         DIS_X(422), cost 2464.784487, n_parallel 1, n_sites 1
      > try       X_GAT_X(424), cost 14.150826, n_parallel 0, n_sites 0, best*
      > try         GAT_X(425), cost 26.647845, n_parallel 0, n_sites 0

//分析函数 analytic function 片段，正要探测第 6 个需要分发的 analytic function。
*** probe best distribute method for analytic function(0000000050899AC0), DPC motion
nth_try[6], restricts[0x181], self cost 1.000000
      left   motion cost: broadcast = 0.000011, dis = 0.000011, gather = 0.000011
      self   motion cost: broadcast = 0.000011, dis = 0.000011, gather = 0.000011
      > try              X(420), not available
      > try   DIS_SORT_X(423), cost 20.000112, n_parallel 1, n_sites 1, best*
      > try   GAT_SORT_X(426), cost 4.000011, n_parallel 0, n_sites 0
```

23.6.10 分区表遍历操作符 PLL 提示

使用 HINT COLLECT_ALL_SUB_TAB 或 HINT NO_COLLECT_ALL_SUB_TAB 可以实现提前收集或者提前不收集分区表的所有叶子表的字典信息。

语法:

```
/*+ COLLECT_ALL_SUB_TAB*/

/*+ NO_COLLECT_ALL_SUB_TAB*/
```

例 下面的两条SELECT语句中,第一条SELECT指定了HINT COLLECT_ALL_SUB_TAB,表明分区表 TEST 的遍历操作符 PLL（或者 DPC 下包含 PLL 的操作符 GI）会将 TEST 的多个子表 TEST_P1、TEST_P2、TEST_P3 的字典信息缓存在 HASH 表中（HASH 表也在计划中），执行时不需要再从字典缓存加载字典信息。

第二条 SELECT 指定了 HINT NO_COLLECT_ALL_SUB_TAB，它的执行计划生成时，将不会有子表的字典信息被缓存。第二条 SELECT 语句会重新生成计划，所以要求计划重用未打开。

```
drop table TEST;

CREATE TABLE TEST(C1 INT,C2 INT,C3 INT)

PARTITION BY HASH(C1)

(

PARTITION p1,

PARTITION p2,

PARTITION p3);

select /*+COLLECT_ALL_SUB_TAB*/ * from TEST;

select /*+NO_COLLECT_ALL_SUB_TAB*/ * from TEST;
```

23.6.11 单节点登录提示

在 DMMPP 和 DMDPC 环境下支持单节点登录，可以使用 HINT MPP_LOCAL 或 HINT DPC_LOCAL 指定为以单节点模式登录。HINT DPC_LOCAL 仅支持单条语句，不支持在语句块中使用。在单节点登录模式下，功能存在限制，节点退化为单机模式，查询只能查到当

前节点的信息。

语法：

```
/*+ MPP_LOCAL */

/*+ DPC_LOCAL*/
```

例 1 在 DMDPC 环境下，创建分区表分布在不同的节点，正常查询时可以得到分区表上的所有数据；如果查询时指定了 HINT DPC_LOCAL，那么计划会退化为单机形式，只能得到当前节点的数据（如果连接的是 SP，那么不会返回数据；如果连接的是 RAFT_1，可以得到 RAFT_1 上的数据）。

```
//创建表空间

CREATE TABLESPACE TS1 DATAFILE 'opt/ts/ts01.dbf' SIZE 128 STORAGE (ON RAFT_1);

CREATE TABLESPACE TS2 DATAFILE 'opt/ts/ts02.dbf' SIZE 128 STORAGE (ON RAFT_2);

//创建表

DROP TABLE TEST;

CREATE TABLE "SYSDBA"."TEST"

(

"C1" INT,

"C2" INT,

"C3" INT)

PARTITION BY HASH("C1")

(

PARTITION  "P1"   STORAGE(ON "TS1", CLUSTERBTR),

PARTITION  "P2"   STORAGE(ON "TS2", CLUSTERBTR),

PARTITION  "P3"   STORAGE(ON "TS1", CLUSTERBTR)

) STORAGE(HASHPARTMAP(2), ON "TS2", CLUSTERBTR);
```

```
INSERT INTO TEST SELECT LEVEL,LEVEL,LEVEL FROM DUAL CONNECT BY LEVEL <8;

COMMIT;
```

正常查询时的执行计划：

```
EXPLAIN SELECT * FROM TEST;

1    #NSET2: [1, 7, 24]

2      #ERECV: [1, 7, 24]; stask_no(-1), l_stask_no(0), n_key(0), trig(0)

3        #ESEND: [1, 7, 24]; stask_no(0), type(DIRECT), sites(1:1,2:1), sql_invoke(0), pwj_opt(0),

table(-); INFO_BITS(0xc)

4          #GI: [1, 7, 24]; policy(PART_UNIT), gi_unit[0..0], scan_type[0](FULL)

5            #PRJT2: [1, 7, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

6                      #CSCN2: [1, 7, 24]; INDEX33555511(TEST); btr_scan(1); need_slct(0);

prejudge_iescn(0)
```

查询时通过 HINT 指定 DMDPC 单节点登录 DPC_LOCAL 时的执行计划：

```
EXPLAIN SELECT /*+DPC_LOCAL*/ * FROM TEST;

1    #NSET2: [1, 7, 24]

2      #PRJT2: [1, 7, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)

3        #PARALLEL: [1, 7, 24]; scan_type(FULL), key_num(0, 0, 0), simple(0), range_sfun_opt(0)

4                    #CSCN2: [1, 7, 24]; INDEX33555511(TEST); btr_scan(1); need_slct(0);

prejudge_iescn(0)
```

例 2 在 DMMPP 环境下，创建分区表分布在不同的节点，正常查询时可以得到分区表上的所有数据；如果查询时指定了 HINT MPP_LOCAL，那么计划会退化为单机形式，只能得到当前节点的数据。

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT,C2 INT,C3 INT)
```

```
DISTRIBUTED BY HASH(C1) ;
insert into test select level,level,level from dual connect by level <8;
commit;
```

正常查询时的执行计划：

```
explain select * from TEST;
1    #NSET2: [1, 6, 24]
2      #MPP COLLECT: [1, 6, 24]; op_id(1) n_grp_by (0) n_cols(0) n_keys(0) for_sync(FALSE)
3        #PRJT2: [1, 3, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
4                  #CSCN2: [1, 3, 24]; INDEX33555482(TEST); btr_scan(1); need_slct(0);
prejudge_iescn(0)
```

查询时通过 HINT 指定 DMMPP 单节点登录 MPP_LOCAL 时的执行计划：

```
explain select /*+ MPP_LOCAL*/ * from TEST;
1    #NSET2: [1, 3, 24]
2      #PRJT2: [1, 3, 24]; exp_num(4), is_atom(FALSE); INFO_BITS(0); spl_info(NULL)
3                #CSCN2: [1, 3, 24]; INDEX33555482(TEST); btr_scan(1); need_slct(0);
prejudge_iescn(0)
```

23.6.12 视图不替换 SQL 记录提示

可以使用 HINT SQL_NODE_KEEP_CACHE 来保证当前 SQL 查询语句在视图 V$SQL_NODE_HISTORY 中不被替换。需要同时配置 ini 参数 MONITOR_SQL_EXEC=1, ENABLE_MONITOR=1，视图 V$SQL_NODE_HISTORY 中才会有 SQL 的执行信息记录。

语法：

```
/*+ SQL_NODE_KEEP_CACHE */
```

因为视图的容量存在限制，在并发场景下，单个查询语句的执行信息内容很容易被覆盖，

使用该 HINT 可以使 SQL 语句的执行信息保存更长时间，以便分析语句中存在的问题。

例 通过 HINT 指定不替换 V$SQL_NODE_HISTORY 中当前查询语句的执行记录。

```
select /*+ SQL_NODE_KEEP_CACHE*/ * from TEST;
```

HINT 指定后，可以调用系统过程 SP_RESET_SQL_NODE_KEEP_CACHE_FLAG（）将视图中的所有项都设置为可替换。

23.6.13 DMDPC 读写分离设置事务的数据一致性级别提示

在 DMDPC 读写分离环境中，可以使用 HINT READ_POLICY LEADER 和 READ_POLICY FOLLOWER 来设置当前事务的数据一致性级别。指定 HINT READ_POLICY LEADER 表示使用强一致性级别来执行语句；指定 HINT READ_POLICY FOLLOWER 表示使用弱一致性级别来执行语句。

语法：

```
/*+ READ_POLICY (LEADER)*/
/*+ READ_POLICY (FOLLOWER)*/
```

该 HINT 仅在打开了 DPC 读写分离的环境中有效，而且会受到当前的事务状态及语句类型的限制。仅在当前事务尚未确定其数据一致性级别时，即事务的第一个语句中设置该 HINT 时才会有效；如果当前事务已经确定了自己的事务一致性级别，则该 HINT 无效。

该 HINT 还受到语句类型的限制，仅查询语句使用 READ_POLICY FOLLOWER 才有效；插入、删除和更新语句即使设置了 READ_POLICY FOLLOWER，也是无效的，这些需要修改数据的语句必须要使用强一致性级别。

当 HINT 指定的数据一致性级别与当前事务或语句的一致性级别冲突时，忽略该 HINT，而且不会报错。

例 在 DMDPC 读写分离环境中，事务的第一个查询语句使用 READ_POLICY FOLLOWER 指定使用弱一致性级别进行查询。

```
select /*+ READ_POLICY (FOLLOWER)*/ * from TEST;
```

第五部分 数据高可用性

第24章 故障恢复

24.1 简介

数据库系统在运行过程中可能会发生一些故障。造成故障的原因多种多样，包括磁盘崩溃、电源故障、软件错误，甚至人为破坏。故障情况一旦发生，就可能导致丢失数据，数据库系统将无法正常运行。因此，故障恢复是数据库系统必不可少的组成部分，即数据库系统必须保证即使发生故障，也可以保障数据的完整性和一致性。

目前支持故障恢复的主要技术是日志。日志以一种安全的方式记录了数据库系统变更的历史信息，一旦系统出现故障，数据库系统可以根据日志将系统恢复至故障发生前的某个时刻。数据库系统的日志分为两种类型：一是 REDO 日志（即重做日志），每当在数据库中指定 DDL 及 DML 操作后，DM 都会按照特定的格式，生成一条 REDO 日志记录登记该操作，并记录操作后的数据；另一种是 UNDO 日志，UNDO 日志中记录进行 DML 操作之前的原始数据信息。

另外，当服务器处于归档模式时，如果数据库发生故障，通过备份文件和归档日志可以恢复到指定时间点。

24.2 REDO 日志

在恢复操作中最重要的结构是联机 REDO 日志。REDO 日志存放在联机日志文件中，这些文件存储数据库所做的所有物理更改信息。达梦数据库每一个实例都默认会生成两个相关联的联机日志文件，通过 REDO 日志可以保证数据库的完整性和一致性。

DMServer 的 REDO 日志用于存储被 DML 或 DDL 操作后数据的新值，包括事务对数据

文件和回滚段的修改。REDO 日志每次被修改后，都会自动生成一个新的日志序列值 LSN（Log Sequence Number）。LSN 取值范围 0~正无穷大，新建的库 LSN 为 0，以后日志每被写入一次，LSN 值增加 1。

REDO 日志里包含有一种特殊的记录，叫 PWR（Page Written Record）日志。PWR 包括（ts_id, fil_id, page_no, page_lsn）信息。每个数据页刷盘时，都会生成一条对应的 PWR 日志。PWR 日志的 LSN 与上一个日志的 LSN 相同。在以下情况下可以利用 PWR 日志：

1. DSC 故障恢复时：利用 PWR 日志减少加载磁盘数据页的次数；

2. 故障恢复时：利用 PWR 日志提升 redo 速度；

3. 增量备份时：利用 PWR 日志提升备份速度。

日志文件由日志表空间（RLOG）管理。DM 支持增加日志文件和扩展日志文件大小。

例 1 增加大小为 300MB 的日志文件 dameng_003.log

```
ALTER DATABASE ADD LOGFILE 'dameng_003.log' size 300;
```

例 2 将日志文件 dameng_003.log 大小更改为 400MB

```
ALTER DATABASE RESIZE LOGFILE 'dameng_003.log' to 400;
```

在 MOUNT 状态下，支持对日志文件的重命名操作。

例 3 将日志文件 dameng_003.log 重命名为 dameng_004.log

```
ALTER DATABASE RENAME LOGFILE 'dameng_003.log' to 'dameng_004.log';
```

24.3 重做日志归档

DM 服务器可以运行在两种模式下，即归档模式和非归档模式，这两种模式可以由用户进行设置，系统在归档模式下运行时，会产生归档日志文件，此时系统管理员应该事先预留出足够的磁盘空间以便存储归档日志文件。通过调整 dm.ini 和 dmarch.ini 文件参数可以配置归档，参见第 2 章 DM 物理存储结构。DM 支持以下八种归档类型：

24.3.1 本地归档

本地归档就是将 REDO 日志写入到本地归档文件。本地归档在 REDO 日志写入联机日志文件后触发，由归档线程完成本地归档动作，最多可以设置 8 个本地归档。

与联机 Redo 日志文件可以被覆盖重用不同，本地归档日志文件不能被覆盖，写入其中的 Redo 日志信息会一直保留，直到用户主动删除；如果配置了归档日志空间上限，系统会自动删除最早生成的归档 Redo 日志文件，腾出空间。

为了最大限度地保护数据，当磁盘空间不足导致归档写入失败，系统将自动挂起等待，直到用户主动释放出足够的空间后继续运行。在设置多路本地归档的情况下，如果第一路本地归档的磁盘空间不足，则系统会被强制挂起，直到磁盘空间释放后继续运行。其它本地归档可以通过在 dmarch.ini 中设置 ARCH_HANG_FLAG 参数表示本地归档写入失败系统是否挂起，缺省值为 1，表示本地归档写入失败系统挂起。此外，系统会定期尝试恢复 INVALID 的本地归档状态为 VALID。恢复 VALID 状态之后，如果正好剩余磁盘空间充足，那么可以继续归档。但是其它路（第一路归档之外的）本地归档中缺失的归档就需要人工手动进行修复了。当磁盘损坏导致归档日志写入失败时，系统会强制 HALT。

24.3.2 实时归档

实时归档的执行流程为：主库在写入 REDO 日志到联机日志文件之前，通过 MAL 系统发送 REDO 日志到备库，备库收到 REDO 日志后，不需要等待 REDO 日志重演完成，马上响应主库，返回确认消息。主库收到确认消息后，执行后续操作，将 REDO 日志写入联机日志文件；若发送 REDO 日志失败，或从备库返回的数据库模式不是 STANDBY，将数据库切换为 SUSPEND 状态，阻塞所有 REDO 日志的写入操作。一个主库可以配置 1~8 个实时备库。

根据备库重演 REDO 日志和响应主库时机的不同，实时归档可以分为两种模式：事务一致模式和高性能模式。实时归档默认采用高性能模式。

对事务一致模式与高性能模式的详细说明，请查看《DM8 数据守护与读写分离集群 V4.0》。

24.3.3 即时归档

即时归档在主库将 REDO 日志写入联机日志文件后，再通过 MAL 系统将 REDO 日志发送到备库，在默认模式下备库会在收到 REDO 日志并重演完成后，再响应主库。即时归档与实时归档的主要区别是发送 REDO 日志的时机不同，即时归档在主库 REDO 日志写入联机日志文件之后发送，实时归档在主库 REDO 日志写入联机日志文件之前发送。即时归档是读写分离集群的实现基础。一个主库可以配置 1~8 个即时备库。

根据备库重演 REDO 日志和响应主库时机的不同，即时归档也可以分为两种模式：事务一致模式和高性能模式。即时归档默认采用事务一致模式。

对事务一致模式与高性能模式的详细说明，请查看《DM8 数据守护与读写分离集群 V4.0》。

24.3.4 异步归档

异步归档由在主、备库上配置的定时器触发，在设定的时间点或者每隔设定时间，启动归档 REDO 日志发送。设置异步归档，必须确保至少有一个本地归档。根据异步备库的 KEEP LSN 信息，扫描本地归档目录获取 REDO 日志，并通过 MAL 系统将 REDO 日志发送到异步备库，备库收到 REDO 日志后，将其加入日志重演任务系统，并马上响应主库，不需要等待 REDO 日志重演结束后再响应主库。最多可以设置 8 个异步归档。

24.3.5 同步归档

主库在归档日志刷盘后，通过 MAL 系统将 REDO 日志发送到备库，备库收到 REDO 日志后，将其加入日志重演任务系统，并马上响应主库，不需要等待 REDO 日志重演结束后再

响应主库。同步归档与异步归档的区别是，同步归档不依赖定时器，主库归档日志刷盘后立即将归档日志发送到备库。一个主库可以配置 1~8 个同步备库。

24.3.6 远程归档

远程归档专门用于 DMDSC 环境中，是将当前节点的远程归档目录配置为另一节点的本地归档目录，以此来共享它的本地归档日志文件。远程归档采用双向配置的方式，即两个节点将自己的远程归档相互配置在对方机器上。DMDSC 集群配置远程归档后，集群中任意实例都可以像访问本地归档一样，访问所有实例产生的归档日志。

24.3.7 RAFT 归档

RAFT 归档是 DMDPC 多副本系统的实现基础，仅用于 DMDPC 集群。DMDPC 多副本系统中的主库通过 RAFT 归档方式向备库同步数据，RAFT 归档将主库产生的 REDO 日志通过 XMAL 模块传递到备库，并且 RAFT 归档只在主库生效，一个主库可以配置 2~8 个 RAFT 备库，归档目标个数必须是偶数（确保总的实例个数是奇数）。

RAFT 归档的执行流程是主库在 REDO 日志写入联机日志文件前，将 REDO 日志发送到备库，并且不需要等待备库的响应消息，主库继续往下正常执行。备库收到 REDO 日志后，将日志包加入日志重演任务系统，在日志包写入本地日志文件后，发送日志刷盘消息给主库，主库根据此消息确定是否需要推进 C_SEQNO 和 C_LSN。

24.3.8 LEARNER 归档

在多副本系统中，若需动态新增副本，将采用 LEARNER 归档机制进行处理。LEARNER 归档仅用于 DMDPC 集群，且仅在动态增删副本节点时使用。

新副本节点首先以 LEARNER 归档模式加入 RAFT 组，成为 LEARNER 副本，待其与主库数据同步一致后，可以选择继续作为 LEARNER 副本运行，相当于只读备库，不参与主库选举

和日志提交；也可以选择转变为 RAFT 归档模式，成为 RAFT 副本，参与主库选举和日志提交。LEARNER 副本支持动态转换为 RAFT 副本，无需重启原有副本。

24.4 检查点

创建数据库时，联机日志文件通常被扩展至一定长度，其内容则被初始化为空，当系统运行时，该文件逐渐被产生的日志所填充。为了达到循环利用日志系统空间的目的，必须在所有日志文件空间将被占满时，系统能够自动清空一部分日志，以便重用日志文件的空间，为了保证被清空的日志所“保护”的数据在磁盘上是安全的，需要引入一个关键的数据库概念——检查点。当系统中产生检查点时，系统会自动将日志缓冲区中的日志和数据缓冲区中的脏数据页全部写入磁盘，以保证当前日志所“保护”的数据页都已安全写入磁盘，这样日志文件即可被安全重用。

当服务器启动和关闭时，系统都会产生检查点。服务器运行过程中，系统会自动判断是否需要执行检查点；当空闲日志空间不足时，系统自动产生一个检查点；系统调度线程也会定时产生检查点；还可以通过调用系统函数 CHECKPOINT 主动生成检查点。

系统调度线程根据 dm.ini 的参数配置，产生检查点，下面给出具体的例子和说明。

```
CKPT_INTERVAL          = 1800      #每间隔 1800 秒，产生检查点间隔
CKPT_FLUSH_RATE        = 5         #检查点的刷盘比例为 5%，将系统中所有脏页的 5%写入磁盘
CKPT_FLUSH_PAGES       = 1000      #一个检查点最少写入 1000 个脏页
```

检查点系统函数 CHECKPOINT（FLUSH_RATE），其参数 FLUSH_RATE 为 INTEGER 类型或 DOUBLE 类型，指示刷盘比例，替代 dm.ini 中的 CKPT_FLUSH_RATE，同时 CKPT_FLUSH_PAGES 参数同时生效。例如，执行 SELECT CHECKPOINT(10);会将系统中 10%脏页写入磁盘。

24.5 回滚段与回滚记录

DMServer 采用回滚段机制来处理 UNDO 日志。回滚段由一定数量的回滚页组成，回滚页存放的是一批回滚记录。回滚记录存放被修改数据的旧值，并有专属的格式，与物理记录格式不同。回滚段的管理同一般数据文件一样，其回滚数据页的分配、淘汰和释放也交由数据缓冲区完成。回滚文件属于回滚表空间。回滚段的数据并不会永久保留，事务结束后，由系统的 PURGE 模块释放回滚页。

REDO 日志记录了所有对数据库数据进行修改的数据，当然也包括回滚段的修改数据。因此，REDO 日志同样也会保护回滚数据。当使用重做日志恢复数据库时，系统将重做回滚段中修改了的数据，并将这些修改写回到回滚段中，从而形成回滚记录。回滚记录与重做记录类似，只不过回滚记录记录被修改数据的旧值，以达到数据回滚到原来状态的目的。

另外，回滚表空间不允许修改和删除操作。

24.6 系统故障恢复

系统故障也就是我们通常所说的系统崩溃。系统崩溃的原因比较复杂，有硬件故障，或者是数据库软件或操作系统的漏洞等。它导致整个系统停止运行，内存中的数据全部丢失，但磁盘上存储的数据仍然有效。

当系统故障发生时，系统管理员可以通过查看系统日志文件以了解系统故障发生的原因或故障发生之前系统的运行状况。多数情况下，系统在故障发生之前会自动记录产生故障的原因，但也不全是这样，如系统掉电时，发生故障的原因可能来不及记录。总之，在处理系统故障之前，系统管理员需要完全了解系统故障发生的原因，并采取相应的措施，如更换硬件，升级操作系统或数据库软件等，必要时还可以联系达梦公司的技术支持工程师，以获得必要的支持。

系统故障恢复的最后一步则是重新启动 DM 服务器。此时，系统将从最近一个检查点处

开始扫描联机日志，并重做日志记录的内容，这样系统即恢复到了故障发生前的某个很近的时刻，然后将系统中仍然活动的事务再依次利用回滚段进行回滚。由此可以看出，系统故障恢复可以保证事务的原子性和持久性，即系统故障发生时，仍然活动的事务可以被安全回滚，而已经提交的事务则保证其所修改数据的持久性。

24.7 介质故障恢复

介质故障指的是由于各种原因导致数据库系统存储在磁盘上的数据被损坏，如磁盘损坏等。介质故障是数据库系统最为严重的故障，此时系统已经无法重新启动，磁盘上的数据也无法复原。

出现介质故障后，系统管理员首先需要分析介质故障发生的原因，并采取措施，如磁盘损坏则更换磁盘等。由于系统的数据已经全部或部分丢失，此时只能依赖以前建立的备份和系统产生的归档日志文件进行恢复。由于涉及到备份，具体的介质恢复策略将在《DM8 备份与还原.docx》中详细介绍。

需要说明的是，为了避免系统在出现故障时丢失数据，应关闭磁盘缓存（若存在的话），具体事项请咨询操作系统管理员。

第25章 版本升级

达梦数据库产品注重向下兼容性，虽然版本不断地更新升级，但在绝大多数情况下，使用高版本的执行码启动低版本的数据库时，会自动执行一系列更新升级动作。对于待升级的数据库，用户只需要保证它之前是正常退出的即可。

在数据库升降级操作前，建议先进行数据备份。

本章节中的升级步骤，是基于待升级的老库版本号等于或者高于 V7.1.6.11 的前提来写的。需要注意的是，DM 进行的两项较大的功能改进（V8.1.1.15 开始支持 RLOG_PKG 日志格式、V8.1.1.101 开始支持回滚管理段），使得待升级的库在跨这两个版本进行升级时，需要进行额外的升级操作，下面进行详细的说明。

25.1 日志格式升级

从 V8.1.1.15 版本开始，DM 对 REDO 日志格式进行了升级，与老的日志格式不兼容，因此要求数据库在执行升级操作前，必须是使用之前版本的执行码执行正常退出后的库，以此来保证所有数据都已经刷盘，否则使用新版本的执行码启动数据库时，无法根据老的 REDO 日志对故障重启的数据库执行重做 REDO 日志、归档文件修复等动作，无法正常完成数据库升级。

25.1.1 版本说明

支持 DM7、DM8 老版本的库升级到 V8.1.1.15 或更高的数据库版本。

DM8 老版本的库正常退出后，允许直接启动升级到 V8.1.1.15 或者更高版本的 DM8，但必须要严格遵守 26.1.2 节中的升级步骤。

DM7 老版本的库，针对不同版本有一些升级限制，具体说明如下：

- V7.1.6.11 之后的版本（包括 V7.1.6.11）

允许直接启动升级到 V8.1.1.15 或者更高版本的 DM8，同样也需要严格遵守升级步骤。

- 低于 V7.1.6.11 的版本

需要先升级到 V7.6.0.183 或者更高版本的 DM7，对日志版本号进行升级，然后才允许升级到 V8.1.1.15 或者更高版本的 DM8。需要注意的是，V7.1.5.125 以及更低版本的库，如果打开了日志加密，则不再支持升级到 V8.1.1.15 或更高的数据库版本。

低于 V7.1.6.11 的版本需要升级两次，这两次版本升级的步骤是完全相同的，“升级到 V7.6.0.183 或者更高版本的 DM7”的步骤可以参考下文中描述的升级步骤，只是将升级步骤中的版本号替换为“V7.6.0.183 或者更高版本的 DM7”。

25.1.2 升级步骤

配置有本地归档的情况下，由于升级后老的归档日志文件不再可用，因此必须在执行升级前先将归档日志文件全部从归档目录中移走，避免升级后再次降级时，误判归档日志文件不连续。

具体的升级步骤如下：

正常退出低版本的数据库之后，直接使用 V8.1.1.15 或者更高版本的执行码启动数据库即可，在启动过程中会自动完成系统表和动态视图等字典信息的升级。

单节点可以正常启动到 Open 状态，就表示升级成功，升级成功的情况下，一定是可以查到 SYSOPENHISTORY 系统表的，可通过查询此系统表确认升级结果。

25.2 回滚管理段升级

DM 从 V8.1.1.101 版本开始支持回滚管理段，用于存放事务信息，在系统故障重启或者 DSC 故障处理时，通过扫描回滚管理段收集事务信息，以达到缩减事务收集时长的目的。

此版本增加了一个建库参数（PSEG_MGR_FLAG）表示是否仅使用回滚管理段记录事务

信息，取值 0、1，默认为 0。取值含义说明如下：

- 0：除了使用回滚管理段记录事务信息外，在事务的首个回滚页上也记录事务信息。
- 1：仅使用回滚管理段记录事务信息，事务的首个回滚页上不再记录。

老版本的库升级时，一律采用 PSEG_MGR_FLAG=0 的方式进行升级，老版本库在升级时会自动创建回滚管理段，升级成功后，不允许再使用老执行码启动升级后的库。

25.2.1 版本说明

具体的升级版本限制说明如下：

- 升级前的老版本库是 DM8，版本号等于或者高于 V8.1.1.15

老版本库已支持升级后的日志格式，本次升级仅需要升级回滚管理段。

- 升级前的老版本库是 DM8，版本号低于 V8.1.1.15；或者升级前的老版本库是 DM7

老版本库不支持升级后的日志格式，本次升级需要先升级日志格式，再升级回滚管理段。

对这个版本范围的老版本库，升级前首先需要满足“26.1 日志格式升级”的各项升级条件，然后还要满足回滚管理段的升级条件才能升级。

25.2.2 升级步骤

回滚管理段的升级必须满足以下两个条件：

1. 老库升级前必须正常退出。

2. 老库如果是新初始化库，则必须用老执行码正常启动过并正常退出，才允许升级。

如果老库已支持升级后的日志格式（版本号等于或者高于 V8.1.1.15），在满足这两个条件之后，直接使用 V8.1.1.101 或者更高版本的执行码启动老库到 Open 状态即可完成升级。

如果老库还未支持升级后的日志格式，则需要同时满足日志格式和回滚管理段的升级条件，具体执行升级时，要根据老库当前的版本号，按照“26.1 日志格式升级”中的方式进行

升级，只需要将其中的 V8.1.1.15 替换为 V8.1.1.101 即可，V8.1.1.101 可以同时完成日志格式和回滚管理段的升级。

第六部分 附录

附录 1 数据字典

DM 提供非常多的数据字典，供用户更好的了解系统的详细信息。数据字典表的模式名为 SYS。例如：SYS.SYSOBJECTS。

例 1 查看系统表 SYS.SYSOBJECTS。

```
SELECT count(*) FROM SYS.SYSOBJECTS;

行号          COUNT(*)

---------- --------------------

1             1126
```

同时，DM 为每个数据字典表建立了同名的全局同义词，因此用户也可以直接使用同义词对数据字典表进行访问。例如：数据字典 SYS.SYSOBJECTS 的同义词即为 SYSOBJECTS。

例 2 通过使用同义词 SYSOBJECTS 进行查询，查询的结果和 SYS.SYSOBJECTS 一致。

```
SELECT count(*) FROM SYSOBJECTS;

行号          COUNT(*)

---------- --------------------

1             1126
```

但是，如果启用了 SVI 角色，则同义词会指向本数据字典的 V 视图，而不再是数据字典。

例 3 先启用 SVI 角色，再使用同义词 SYSOBJECTS 进行查询，查询的结果和 SYS.VSYSOBJECTS 一致。

```
SP_INIT_SVI_SYS(1); //创建 V 视图

SP_SWITCH_SVI(1);//启用 SVI 角色
```

```
SELECT count(*) FROM SYS.VSYSOBJECTS;

行号          COUNT(*)

---------- --------------------

1             722


SELECT count(*) FROM SYSOBJECTS;

行号          COUNT(*)

---------- --------------------

1             722
```

下面详细介绍各数据字典的含义：

1. SYSOBJECTS

记录系统中所有对象的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	对象名称
2	ID	INTEGER	对象 ID
3	SCHID	INTEGER	TYPE$=SCHOBJ 或者 TYPE$=TABOBJ 时表示对象所属的模式 ID，否则为 0
4	TYPE$	VARCHAR(10)	对象的主类型。分为三种： 1）库级： UR 用户，具体类型看 SUBTYPE$、 SCH 模式、GDBLINK 全局 DBLINK、DSYNOM 全局同义词、DIR 目录、PROFILE 对象

			2）模式级： SCHOBJ 模式内对象，具体类型看 SUBTYPE$；DMNOBJ 域对象，具体类型可为 CONS 约束，为创建域时指定约束由系统生成 3）表级： TABOBJ 表的下级对象，具体类型看 SUBTYPE$
5	SUBTYPE$	VARCHAR(10)	对象的子类型。分为三种： 1）用户对象： USER 用户、ROLE 角色 2）模式对象： UTAB 用户表、STAB 系统表、VIEW 视图、PROC 过程、SEQ 序列、PKG 包、TRIG 触发器、DBLINK 外部链接、SYNOM 同义词、 CLASS 对象类型（类）、TYPE 数据类型、JCLASS JAVA 类、DOMAIN 域、CLLT 集合、CONTEXT 上下文、PGRP 分区组、OPERATOR 运算符 3）表对象： INDEX 索引、CNTIND 全文索引、CONS 约束
6	PID	INTEGER	对象的父对象 ID，为-1 表示当前行 PID 列无意义
7	VERSION	INTEGER	对象的版本
8	CRTDATE	DATETIME(6)	对象的创建时间
9	INFO1	INTEGER	表对象： 表数据所在的缓冲区 ID(0xFF000000)，数据页填充因子(0x00F00000)， BRANCH(0x000FF000)，NOBARNCH(0x00000FF0)，

			BRANCHTYPE(0x0000000F) 用户对象：BYTE(4)用户类型 视图对象：BIT(0) CHECK，BIT(1) CHECK CASCADE， BIT(2)是否加密，BIT(4)SYSTEM 触发器对象： BIT(1) TV|EVENT FLAG，BIT(2,3)执行类型(前或后)，BIT(4)是否加密, BIT(5)是否系统级，BIT(13)是否启用 对于 TV 触发器：BIT(6) RSFLAG，BIT(7) NEW REFED FLAG， BIT(8) OLD REFED FLAG，BIT(9) ALL NEW MDF FLAG 对于事件触发器：BIT(6,7) SCOPE， BIT(8,11) SCHEDUAL TYPE 约束对象：列数 存储过程：BIT(0)是否存储过程，BIT(1)是否加密，BIT(2)是否系统级，BIT(5)是否为 AUTHID CURRENT_USER 类型，BIT(8)外部函数是否是标准 C 角色：角色类型 序列：BYTE(0)是否循环，BYTE(1)是否排序，BYTE(2)是否有缓存，BYTE(3)是否 MPP 环境下为本地，BIT(1)是否扩展 同义词：是否带系统标识 包：BIT(1)文本是否加密，BIT(2)是否带系统标识，BIT(5)是否为 AUTHID CURRENT_USER 类型 自定义对象类型：BIT(6)是否为 create type as object 方式创建
10	INFO2	INTEGER	表/用户/数据库/表空间：BYTE(4)空间限制值，单位为数据页页

			数 视图：基表 ID
11	INFO3	BIGINT	序列：起始值，扩展类型序列写入-1 触发器：BYTE(0-3)EVENTS ● TV 触发器，BYTE(4)更新操作可触发的字段， BYTE(5)行前触发器中可被触发器修改值的新行字段，BYTE(6)元组级触发器中引用的字段 ● 事件触发器，BYTE(4)间隔，BYTE(5)子间隔，BYTE(6,7)分间隔 表：BYTE(0)表类型或临时表类型；BYTE(1)日志类型、错误响应、不可用标识；bit(17)是否临时表会话级；bit(22)是否允许行迁移；bit(34)表是否带高级日志表；BYTE(5)标记分布表；bit(50)表的超长记录是否可以转换为行外数据存储；bit(51)表是否开启记录物理逻辑日志；bit(57)表是否为物化视图预建表 用户：BYTE(0-1)默认表空间 ID，BYTE(2-3)用户索引默认表空间 ID 视图：BYTE(0-3)预建表 ID；bit(32)物化视图表列与定义查询列的精度是否一致；bit(33)物化视图完全刷新时使用 TRUNCATE 方式删除老数据；bit(34)物化视图完全刷新时使用 DELETE 方式删除老数据 约束：BIT(0)check 约束中是否包含 is json 表达式；BIT(1)约束是否是检查老数据
12	INFO4	BIGINT	序列：增量

			表：低 4 字节表示表版本，当表字典对象发生变化时，值加 1; 高 4 字节表示大字段数据版本，当大字段数据发生变化时，值加 1 子索引：上级子表或主表索引 ID 子表约束：上级子表或主表约束 ID 触发器：触发器执行节点，低 2 字节保存 dpc 下指定触发的 raft_id
13	INFO5	VARBINARY(128)	表：BYTE(10)BLOB 数据段头 序列：BYTE(8)序列最大值，BYTE(8)序列最小值，BYTE(2)文件 ID，BYTE(4)页号，BYTE(2)序列当前位置 用户：BYTE(4)HASH 散列算法 id，BYTE(1)HASH 加盐选项 视图：BIT(0-3)物化视图最近一次刷新使用的刷新方法， BIT(4-7)物化视图的分类，BYTE(1-8)物化视图最近一次刷新开始时间，BYTE(9-16)物化视图最近一次刷新结束时间
14	INFO6	VARBINARY(2048)	视图：BYTE(4)表或视图 ID 触发器: TV 触发器，BYTE(2)更新操作可触发字段， BYTE(2)元组级触发器前可能被触发器修改值的字段, BYTE(2)元组级触发器中引用的字段， 事件触发器，BYTE(8)开始/ BYTE(8)结束日期、BYTE(5)开始/ BYTE(5)结束时间 约束对象：(BYTE(4)ID)表列链表 同义词：BYTE(2)模式名和 BYTE(2)对象名

			表： BYTE(1-8)普通表 IDENTITY 自增列的 SEED 选项；BYTE(9-16)普通表 IDENTITY 自增列的 INCREMENT 选项；BYTE(17-20)WITH/WITHOUT COUNTER 选项；BYTE(21-24)水平分区根表使用的分区组 ID；BYTE(25-26)普通表的自增列类型，0：无自增列，1：IDENTITY，2：AUTO_INCREMENT；BYTE(27-30)对象表模板对象的 clsid；BYTE(31)对象表是否允许子类型代替父类型；BYTE(32)对象表 oid 生成信息，0：SYSTEM GENERATED，系统默认生成，1：PRIMARY KEY，基于主键生成；BYTE(33-36)水平分区根表的分区表字典版本；BYTE(37-40)水平分区根表的自增列预分配的序列 ID；BYTE(41-42)水平分区根表的自增列类型，0：无自增列，1：IDENTITY，2：AUTO_INCREMENT；BYTE(43-44)水平分区根表的自增列的列 ID；BYTE(45-48)建表时指定存储的表空间组 ID 序列：仅扩展类型序列使用 BYTE(0-15) FOR MAXVALUE, BYTE(16-31) FOR MINVALUE, BYTE(32-47) FOR STARTVALUE
15	INFO7	BIGINT	BYTE(4)自动重写标记，0：不允许重写；1：允许重写
16	INFO8	VARBINARY(1024)	外部表：记录控制文件路径 上下文 context：记录访问方式（ACCESSED LOCALLY，即只允许当前会话访问，不允许其他会话访问） 物化视图日志表：记录刷新时间 触发器：BYTE(2)触发器类型，BYTE(4)FOLLOWS 或 PRECEDES 指定的触发器数量，BYTE(4)FOLLOWS 或 PRECEDES 指定的触

			发器 ID
17	VALID	CHAR(1)	对象是否有效，'Y'表示有效，'N'表示失效

2. SYSINDEXES

记录系统中所有索引定义信息。

序号	列	数据类型	说明
1	ID	INTEGER	索引 ID
2	ISUNIQUE	CHAR(1)	是否为唯一索引
3	GROUPID	SMALLINT	所在表空间的 ID
4	ROOTFILE	SMALLINT	存放根的文件号。取值为 0 表示文件号为 0，是正常取值。 也可能取特殊值-1 或-2，特殊值说明如下: 取值-1：不需要创建 B 树。 通常使用场景: a)临时表，其地址由会话维护，不需要写入元数据 b)分区表普通索引，在多级分区中间层子表上不创建 B 树 c)分区表全局索引，在子表上的虚索引不创建 B 树 d)非 B 树索引 取值-2：因为延迟段分配，暂未实际分配空间
5	ROOTPAGE	INTEGER	存放根的页号 特殊值说明: 取值-1：不需要创建 B 树。 通常使用场景:

			a)临时表，其地址由会话维护，不需要写入元数据 b)分区表普通索引，在多级分区中间层子表上不创建 B 树 c)分区表全局索引，在子表上的虚索引不创建 B 树 d)非 B 树索引 取值-2：因为延迟段分配，暂未实际分配空间
6	TYPE$	CHAR(2)	类型。BT：B 树，BM：位图，AR：数组，IF：IVFFLAT 向量，HS：HNSW 向量，MP：BMP 向量
7	XTYPE	INTEGER	标识索引类型。按位取值，10 进制取值如下： 0：聚集索引 1：二级索引 2：标识函数索引 4：全局索引在水平分区子表上标识 8：全局索引在水平分区主表上标识 16：标识唯一索引 32：标识扁平索引 64：标识数组索引 2048：表示该位图索引是由改造后创建 4096：位图索引 8192：位图连接索引 16384：位图连接索引虚索引 65536：标识索引是否可见 131072：标识 DPC 全局索引的根索引

			262144：标识 DPC 全局索引的子索引 1048576：向量 IVFFLAT 索引 2097152：向量 HNSW 索引 4194304：向量 BMP 索引 支持使用上述有效值的组合值，如 9 表示同时符合 1 和 8
8	FLAG	INTEGER	索引标记。按位取值，10 进制取值如下： 0：1~32 以外的情况均为 0 1：系统索引 2：虚索引 4：PK 8：在临时表上 32：fast pool 支持使用上述有效值的组合值，如 9 表示同时符合 1 和 8
9	KEYNUM	SMALLINT	索引包含的键值数目
10	KEYINFO	VARBINARY(816)	索引的键值信息
11	INIT_EXTENTS	SMALLINT	初始簇数目
12	BATCH_ALLOC	SMALLINT	下次分配簇数目
13	MIN_EXTENTS	SMALLINT	最小簇数

3. SYSCOLUMNS

记录系统中所有列定义的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	列名

2	ID	INTEGER	父对象 ID
3	COLID	SMALLINT	列 ID
4	TYPE$	VARCHAR(128)	列数据类型
5	LENGTH$	INTEGER	列定义长度
6	SCALE	SMALLINT	列定义刻度。对于数值数据类型，SCALE 字段值为标度；对于 VARCHAR(<长度> CHAR)数据类型，SCALE 字段值为 7；对于 CHAR(<长度> CHAR)数据类型，SCALE 字段值为 8；对于时间数据类型，SCALE 字段值为秒精度，其中针对本地时区类型，SCALE 字段两个字节中的高字节表示 LOCAL 掩码，低字节表示秒精度，针对时间间隔数据类型，SCALE 字段两个字节中的高字节表示时间间隔类型，低字节中的高 4 位表示引导精度，低 4 位表示秒精度；对于其他数据类型，SCALE 字段值为 0
7	NULLABLE$	CHAR(1)	是否允许为空
8	DEFVAL	VARCHAR(2048)	缺省值
9	INFO1	SMALLINT	列存储表: BIT(0)压缩标记；BIT(1-12)区大小；BIT(13)列存储的区上是否做最大最小值统计；BIT(14)是否加密列 其他表：BIT(0)压缩标记 视图：BYTE(2)多层视图中在直接上级视图中的列 ID，不支持连接视图 存储过程：BYTE(2)参数类型
10	INFO2	SMALLINT	普通表：BIT(0)是否自增列； BIT(14)是否加密列 视图：BYTE(2)多层视图中在最原始表中的列 ID，不支持连接视

			图 列存储表：group_id

4. SYSCONS

记录系统中除 NOT NULL 约束、NOT VISIBLE 约束以及 CLUSTER KEY 约束之外的所有约束信息。

序号	列	数据类型	说明
1	ID	INTEGER	约束 ID
2	TABLEID	INTEGER	所属表 ID
3	COLID	SMALLINT	列 ID。暂时不支持，无意义。全部为-1
4	TYPE$	CHAR(1)	约束类型。 P：PRIMARY KEY 约束、[NOT] CLUSTER PRIMARY KEY 约束 U：UNIQUE 约束、CLUSTER UNIQUE KEY 约束 F：FOREIGN KEY 约束 C：CHECK 约束
5	VALID	CHAR(1)	约束是否有效
6	INDEXID	INTEGER	索引 ID
7	CHECKINFO	VARCHAR(2048)	CHECK 约束的文本
8	FINDEXID	INTEGER	外键所引用的索引 ID
9	FACTION	CHAR(2)	前一字符对应外键的更新动作，后一字符对应外键的删除动作
10	TRIGID	INTEGER	动作触发器 ID

5. SYSSTATS

用于执行收集统计信息操作之后，记录系统中的统计信息。

序号	列	数据类型	说明

1	ID	INTEGER	对象 ID
2	COLID	SMALLINT	列 ID，表级统计为-1
3	T_FLAG	CHAR(1)	对象的标记。标记为：T 表、C 列、I 索引
4	T_TOTAL	BIGINT	总行数
5	N_SAMPLE	BIGINT	采样个数
6	N_DISTINCT	BIGINT	不同值的个数
7	N_NULL	BIGINT	空值个数
8	V_MIN	VARBINARY(255)	列的采样数据中的最小值
9	V_MAX	VARBINARY(255)	列的采样数据中的最大值
10	BLEVEL	TINYINT	B 树层次
11	N_LEAF_PAGES	BIGINT	叶子段总页数
12	N_LEAF_USED_PAGES	BIGINT	叶子占用的页数
13	CLUSTER_FACTOR	INTEGER	索引的 cluster_factor
14	N_BUCKETS	SMALLINT	直方图桶数目
15	DATA	BLOB	直方图数据
16	COL_AVG_LEN	INTEGER	平均行长
17	LAST_GATHERED	DATETIME(6)	最后收集时间
18	INFO1	VARBINARY(255)	预留列
19	INFO2	VARBINARY(255)	预留列

注：COL_AVG_LEN 和 LAST_GATHERED 两个字段在 V7.1.5.173 版本和之后的版本都能看到。如果使用了该版本及以后的版本服务器后，需要再退回到之前版本的服务器，那么需要在新版本上执行 SP_UPDATE_SYSSTATS(0)并正常退出之后，才能使用老版本的服务器。另外，SP_UPDATE_SYSSTATS (99)可以在 SYSSTATS 表上增加这两个列，对老库进行升

级。SP_UPDATE_SYSSTATS 详细使用方法请参考《DM8_SQL 使用手册》。

6. SYSDUAL

为不带表名的查询而设，用户一般不需查看。

序号	列	数据类型	说明
1	ID	INTEGER	始终为 1

7. SYSTEXTS

存放字典对象的文本信息。使用时，系统内部临时解析的字典对象才会放入此处。

序号	列	数据类型	说明
1	ID	INTEGER	所属对象的 ID。对象为：VIEW 视图、 PROC 过程、SEQ 序列、PKG 包、TRIG 触发器、SYNOM 同义词、 CLASS 对象类型（类）、TYPE 数据类型、JCLASS JAVA 类、DOMAIN 域、CLLT 集合、CONTEXT 上下文、USER 用户
2	SEQNO	INTEGER	视图:0 表示视图定义，1 表示视图的查询子句; 包：0 表示包规范，1 表示包体; 用户：0 表示 ALLOW_ADDR，1 表示 NOT_ALLOW_ADDR; 其它：均为 0
3	TXT	CLOB	文本信息

8. SYSGRANTS

记录系统中权限信息。

序号	列	数据类型	说明
1	URID	INTEGER	被授权用户/角色 ID
2	OBJID	INTEGER	授权对象 ID，对于数据库权限为-1
3	COLID	INTEGER	表/视图列 ID，非列权限为-1

4	PRIVID	INTEGER	权限 ID。如果是角色授权为-1
5	GRANTOR	INTEGER	授权者 ID。如果是角色授权为-1
6	GRANTABLE	CHAR(1)	权限是否可转授，Y 可转授，N 不可转授

9. SYSAUDIT

记录系统中的审计设置。

序号	列	数据类型	说明
1	LEVEL	SMALLINT	审计级别
2	UID	INTEGER	用户 ID
3	TVPID	INTEGER	表/视图/触发器/存储过程函数 ID
4	COLID	SMALLINT	列 ID
5	TYPE	SMALLINT	审计类型
6	WHENEVER	SMALLINT	审计情况

10. SYSAUDITRULES

记录系统中审计规则的信息。

序号	列	数据类型	说明
1	ID	INTEGER	规则 ID
2	RULENAME	VARCHAR(128)	规则名
3	USERID	INTEGER	用户 ID
4	SCHID	INTEGER	模式 ID
5	OBJID	INTEGER	操作对象 ID
6	COLID	SMALLINT	列 ID
7	OPTYPE	SMALLINT	操作类型
8	WHENEVER$	SMALLINT	审计情况

9	ALLOW_IP	VARCHAR(1024)	允许的 IP
10	ALLOW_DT	VARCHAR(1024)	时间段
11	INTERVAL$	INTEGER	时间间隔
12	TIMES	INTEGER	操作次数

11. SYSAUDITSQLSEQ

记录系统中审计语句序列的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	语句序列审计规则名
2	SQLSEQ	INTEGER	语句序列中的 SQL 语句序号
3	SQLSTR	VARCHAR(8188)	语句序列中的 SQL 语句

12. SYSHPARTTABLEINFO

记录系统中分区表的信息。

序号	列	数据类型	说明
1	BASE_TABLE_ID	INTEGER	基表 ID
2	PART_TABLE_ID	INTEGER	分区表 ID
3	PARTITION_TYPE	VARCHAR(10)	分区类型
4	PARTITION_NAME	VARCHAR(128)	分区名
5	HIGH_VALUE	VARBINARY(8188)	LIST 分区的临界值; 范围分区的分区值; 哈希分区此值为 NULL
6	INCLUDE_HIGH_VALUE	CHAR(1)	对 LIST 分区表示分区是否包含临界值; 对范围分区始终为 1；对哈希分区此值为 0
7	RESVD1	INTEGER	对子表记录：同层的第一个(最左边)子表 id

			对模板记录：该模板中的子分区个数
8	RESVD2	INTEGER	保留
9	RESVD3	INTEGER	保留
10	RESVD4	VARCHAR(128)	保留
11	RESVD5	VARCHAR(2000)	保留

13. SYSDISTABLEINFO

记录 MPP 下范围分布表和 LIST 分布表的信息。

序号	列	数据类型	说明
1	TABLE_ID	INTEGER	表 ID
2	SEQNO	INTEGER	分布列序号
3	DIS_TYPE	VARCHAR(10)	分布类型，取值：LIST、RANGE
4	HIGH_VALUE	VARBINARY(8188)	LIST 分布的分布值； 范围分布的临界值
5	INCLUDE_HIGH_VALUE	CHAR(1)	对范围分布表示分布是否包含临界值； 对 LIST 分布始终为 1

14. SYSMACPLYS

记录策略定义。

序号	列	数据类型	说明
1	ID	INTEGER	策略 ID
2	NAME	VARCHAR(128)	策略名

15. SYSMACLVLS

记录策略的等级。

序号	列	数据类型	说明

1	PID	INTEGER	策略 ID
2	ID	SMALLINT	等级 ID
3	NAME	VARCHAR(128)	等级名

16. SYSMACCOMPS

记录策略的范围。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	SMALLINT	范围 ID
3	NAME	VARCHAR(128)	范围名

17. SYSMACGRPS

记录策略所在组的信息。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	SMALLINT	组 ID
3	PARENTID	SMALLINT	父节点 ID
4	NAME	VARCHAR(128)	组名

18. SYSMACLABELS

记录策略的标记信息。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	INTEGER	标记 ID
3	LABEL	VARCHAR(4000)	标记信息

19. SYSMACTABPLY

记录表策略信息。

序号	列	数据类型	说明
1	TID	INTEGER	表 ID
2	PID	INTEGER	策略 ID
3	COLID	SMALLINT	列 ID
4	OPTIONS	TINYINT	可见性

20. SYSMACUSRPLY

记录用户的策略信息。

序号	列	数据类型	说明
1	UID	INTEGER	用户 ID
2	PID	INTEGER	策略 ID
3	MAXREAD	INTEGER	最大读标记 ID
4	MINWRITE	INTEGER	最小写标记 ID
5	DEFTAG	INTEGER	默认标记 ID
6	ROWTAG	INTEGER	行级标记 ID
7	PRIVS	TINYINT	特权

21. SYSMACOBJ

记录扩展客体标记信息。

序号	列	数据类型	说明
1	OBJID	INTEGER	对象 ID
2	COLID	SMALLINT	列 ID
3	PID	INTEGER	策略 ID
4	TAG	INTEGER	标记 ID

22. SYSCOLCYT

记录列的加密信息。

序号	列	数据类型	说明
1	TID	INTEGER	表 ID
2	CID	SMALLINT	列 ID
3	ENC_ID	INTEGER	加密类型 ID
4	ENC_TYPE	CHAR(1)	加密类型
5	HASH_ID	INTEGER	哈希算法 ID
6	HASH_TYPE	CHAR(1)	是否加盐
7	CIPHER	VARCHAR(1024)	密钥

23. SYSACCHISTORIES

记录登录失败的历史信息。用户登录成功后，会清理当前用户成功登录前的登录失败记录，仅保留其他用户的登录失败信息以便查看。

序号	列	数据类型	说明
1	LOGINID	INTEGER	登录 ID
2	LOGINNAME	VARCHAR(128)	登录名
3	TYPE$	INTEGER	登录用户类型。0：普通用户；1：AUIDT 审计员；2：SSO 安全员；3：DBO 操作员
4	ACCPIP	VARCHAR(128)	访问 IP
5	ACCDT	DATETIME(6)	访问时间

24. SYSPWDCHGS

记录密码的修改信息。

序号	列	数据类型	说明
1	LOGINID	INTEGER	登录 ID
2	OLD_PWD	VARCHAR(512)	旧密码
3	NEW_PWD	VARCHAR(512)	新密码
4	MODIFIED_TIME	DATETIME(6)	修改日期

25. SYSCONTEXTINDEXES

记录全文索引的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	索引名
2	ID	INTEGER	索引号
3	TABLEID	INTEGER	基表号
4	COLID	SMALLINT	列号
5	UPD_TIMESTAMP	DATETIME(6)	索引相关 DDL 的更新时间。例如：CREATE、ALTER 全文索引之后该时间均会发生变化
6	TIID	INTEGER	CTI$INDEX_NAME$I 表 ID
7	TDID	INTEGER	CTI$INDEX_NAME$D 表 ID
8	TPID	INTEGER	CTI$INDEX_NAME$P 表 ID
9	TNID	INTEGER	CTI$INDEX_NAME$N 表 ID
10	TRID	INTEGER	CTI$INDEX_NAME$R 表 ID
11	WSEG_TYPE	SMALLINT	分词参数类型。 0：中文分词，生成尽量少的分词词条； 1：中文分词，生成尽量多的分词词条；

			2：英文分词； 3：中文分词，以双字为单位机械式分词，分词结果可能存在无意义的分词
12	RESVD	VARBINARY(100)	保留字段

26. SYSTABLECOMMENTS

记录表或视图的注释信息。

序号	列	数据类型	说明
1	SCHNAME	VARCHAR(128)	模式名
2	TVNAME	VARCHAR(128)	表/视图名
3	TABLE_TYPE	VARCHAR(10)	对象类型
4	COMMENT$	VARCHAR(4000)	注释信息

27. SYSCOLUMNCOMMENTS

记录列的注释信息。

序号	列	数据类型	说明
1	SCHNAME	VARCHAR(128)	模式名
2	TVNAME	VARCHAR(128)	表/视图名
3	COLNAME	VARCHAR(128)	列名
4	TABLE_TYPE	VARCHAR(10)	对象类型
5	COMMENT$	VARCHAR(4000)	注释信息

28. SYSUSERS

记录系统中用户信息。

序号	列	数据类型	说明
1	ID	INTEGER	用户 ID

2	PASSWORD	VARCHAR(512)	用户口令
3	AUTHENT_TYPE	INTEGER	用户身份验证模式: 1：数据库身份验证; 2：基于操作系统的身份验证、LDAP 身份验证、SSL 双因子身份验证、KERBEROS 身份验证、UKEY 身份验证; 4：RADIUS 身份验证; 0：其他情况
4	SESS_PER_USER	INTEGER	高位 2 字节表示在所有 DSC 实例中，一个用户可以同时拥有的会话数量，低位 2 字节表示在一个实例中，一个用户可以同时拥有的会话数量
5	CONN_IDLE_TIME	INTEGER	用户会话的最大空闲时间。单位由 dm.ini 中安全参数 RESOURCE_FLAG 决定，取值 1 表示秒，0 表示分钟。
6	FAILED_NUM	INTEGER	用户登录失败次数限制
7	LIFE_TIME	INTEGER	口令有效期。单位：天
8	REUSE_TIME	INTEGER	重用同一个口令的时间间隔。单位：天
9	REUSE_MAX	INTEGER	重用同一个口令之前，需要修改口令的次数
10	LOCK_TIME	INTEGER	用户口令锁定时间，单位：分钟
11	GRACE_TIME	INTEGER	过期口令宽限期。单位：天
12	LOCKED_STATUS	SMALLINT	用户登录是否锁定:

			1：手动锁定 2：未锁定 3：自动锁定（登录失败次数超过上限） 4：自动锁定（超过指定天数未登录）
13	LASTEST_LOCKED	DATETIME(0)	用户最后一次的锁定时间
14	PWD_POLICY	INTEGER	用户口令策略。取值意义请参考 INI 参数 PWD_POLICY
15	RN_FLAG	INTEGER	是否只读，1：只读，0：非只读
16	ALLOW_ADDR	VARCHAR(500)	允许的 IP 地址，长度超过 500 转入 SYSTEXTS 系统表存储
17	NOT_ALLOW_ADDR	VARCHAR(500)	不允许的 IP 地址，长度超过 500 转入 SYSTEXTS 系统表存储
18	ALLOW_DT	VARCHAR(500)	允许登录的时间段
19	NOT_ALLOW_DT	VARCHAR(500)	不允许登录的时间段
20	LAST_LOGIN_DTID	VARCHAR(128)	上次登录时间(首次设置非活跃时间时或解锁非活跃用户锁时，LAST_LOGIN_DTID 更新为本次操作时间)
21	LAST_LOGIN_IP	VARCHAR(128)	上次登录 IP 地址
22	FAILED_ATTEMPS	INTEGER	用户自上一次成功登录以来的失败登录次数
23	ENCRYPT_KEY	VARCHAR(256)	用户登录的存储加密密钥
24	OLD_PASSWORD	VARCHAR(512)	用户辅助口令

29. SYSOBJINFOS

记录对象的依赖信息。

序号	列	数据类型	说明
1	ID	INTEGER	被依赖类的 ID
2	TYPE$	VARCHAR(100)	对象依赖类型
3	INT_VALUE	INTEGER	对象类型对应的值
4	STR_VALUE	VARCHAR(2048)	如果是域对象，表示 DOMAIN+域 ID。其他对象暂未利用
5	BIN_VALUE	VARBINARY(2048)	暂未利用

30. SYSRESOURCES

记录用户使用系统资源的限制信息。

序号	列	数据类型	说明
1	ID	INTEGER	用户 ID
2	CPU_PER_CALL	INTEGER	用户的一个请求能够使用的 CPU 时间上限，单位秒
3	CPU_PER_SESSION	INTEGER	一个会话允许使用的 CPU 时间上限，单位秒
4	MEM_SPACE	INTEGER	会话占有的私有内存空间上限，单位 MB
5	READ_PER_CALL	INTEGER	每个请求能够读取的数据页数
6	READ_PER_SESSION	INTEGER	一个会话能够读取的总数据页数上限
7	INFO1	VARCHAR(256)	取值格式为: CONNCET_TIME . INACTIVE_ACCOUNT_TIME CONNCET_TIME 表示：一个会话连接、访问和操

			作数据库服务器的时间上限，单位：10 分钟； INACTIVE_ACCOUNT_TIME 表示：设定的用户的非活跃时间，单位：天

31. SYSCOLINFOS

记录列的附加信息。

序号	列	数据类型	说明
1	ID	INTEGER	表 ID
2	COLID	SMALLINT	列 ID
3	INFO1	INTEGER	共计 7 位，从右侧起按位取值，含义如下： bit(0)：是否是虚拟列 bit(1)：是否是定长 DEC bit(2)：NUMBER_MODE=1 时标记是否为 FLOAT bit(3)：是否兼容 ORACLE 的 DATE 类型格式 bit(4)：设置列缺省值时是否指定了 ON NULL 选项 bit(5)：列上是否隐式添加了 NOT NULL 约束 bit(6)：该列是否设置了 ON UPDATE 自动更新
4	INFO2	INTEGER	第一字节：NUMBER_MODE=1 时记录 FLOAT 类型列的精度
5	INFO3	INTEGER	备用

32. SYSUSERINI

记录定制的 INI 参数。

序号	列	数据类型	说明
1	USER_NAME	VARCHAR(128)	用户名

2	USER_ID	INTEGER	用户 ID
3	PARA_NAME	VARCHAR(256)	INI 参数名
4	INT_VALUE	BIGINT	整型参数的值
5	DOUBLE_VALUE	DOUBLE	浮点类型参数值
6	STRING_VALUE	VARCHAR(4000)	字符类型参数的值。

33. SYSDEPENDENCIES

记录对象间的依赖关系。

序号	列	数据类型	说明
1	ID	INTEGER	对象 ID
2	TYPE$	VARCHAR(17)	对象类型，包括：TABLE、VIEW、MATERIALIZED VIEW、INDEX、PROCEDURE、FUNCTION、TRIGGER、SEQUENCE、CLASS、CLASS_BODY、JCLASS、TYPE、PACKAGE、PACKAGE_BODY、SYNONYM、DOMAIN
3	REFED_ID	INTEGER	被引用对象 ID
4	REFED_TYPE$	VARCHAR(17)	被引用对象类型，包括类型同 TYPE$，其中不包含 CLASS_BODY 与 PACKAGE_BODY
5	DEPEND_TYPE	VARCHAR(4)	默认为"HARD"，当 TYPE$为 MATERIALIZED VIEW 和 INDEX 时，其值为"REF"

34. SYSINJECTHINT

SQL 语句注入的 HINT 规则。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	规则名称

2	DESCRIPTION	VARCHAR(256)	规则的详细描述
3	VALIDATE	VARCHAR(5)	规则是否生效，取值 TRUE/FALSE
4	SQL_TEXT	TEXT	规则中的 SQL 语句
5	HINT_TEXT	TEXT	为 SQL 语句注入的 HINT 规则内容
6	CREATOR	VARCHAR(128)	规则创建人
7	CRTDATE	DATETIME(6)	规则创建时间
8	INFO1	INTEGER	SQL 的匹配规则为精准匹配或模糊匹配。0: 精确匹配; 1: 模糊匹配; 2 通过 sql_text_id 精准匹配
9	INFO2	VARBINARY(128)	HINT 规则的生效类别。可使用 BINTOCHAR 函数将该字段转换为 VARCHAR 类型显示
10	INFO3	VARBINARY(1024)	保留字段

35. SYSMSTATS

记录多维统计信息的内容。

序号	列	数据类型	说明
1	ID	INTEGER	ID 号
2	MCOLID	VARBINARY(4096)	多维的维度信息
3	T_FLAG	CHAR(1)	保留字段，目前一定是列 'C'
4	N_DIMENSION	INTEGER	维度
5	N_TOTAL	BIGINT	总的行数
6	N_SAMPLE	BIGINT	采样数

7	N_DISTINCT	BIGINT	不同值的个数
8	N_NULL	BIGINT	NULL 值的个数
9	BLEVEL	TINYINT	保留字段
10	N_LEAF_PAGES	BIGINT	保留字段
11	N_LEAF_USED_PAGES	BIGINT	保留字段
12	CLUSTER_FACTOR	INTEGER	保留字段
13	N_BUCKETS	SMALLINT	桶数
14	DATA	BLOB	直方图的信息
15	COL_AVG_LEN	INTEGER	保留字段
16	LAST_GATHERED	DATETIME(6)	最后收集的时间

36. SYSSTATPREFS

记录指定模式下表的统计信息的静态参数对应的值。

序号	列	数据类型	说明
1	ID	INTEGER	ID 号
2	PNAME	VARCHAR(256)	参数名
3	VALUENUM	DOUBLE	参数值
4	VALUECHAR	VARCHAR(256)	参数值
5	CHGTIME	DATETIME(6)	修改时间
6	RSVD1	VARBINARY(2000)	保留信息

37. SYSSTATTABLEIDU

记录用户表上一次收集统计信息时的总行数和增删改的影响行数以及是否有过 TRUNCATE 操作。

当 INI 参数 AUTO_STAT_OBJ 为 1 或 2 时，执行统计信息收集操作之后可以查看到

SYSSTATTABLEIDU 表中数据的变化。当用户通过 DBMS_STATS.SET_TABLE_PREFS 系统包方法设置表 STALE_PERCENT 属性时，SYSSTATTABLEIDU 表中数据也可能发生变化，具体说明请参见《DM8 系统包使用手册》。

序号	列	数据类型	说明
1	ID	INTEGER	ID 号
2	TOTAL_ROWS	BIGINT	表中数据的总行数。只有发生统计的时候，TOTAL_ROWS 才会更新
3	INSERT_ROWS	BIGINT	插入行数。只有系统重启后，才会更新
4	DELETE_ROWS	BIGINT	删除行数。只有系统重启后，才会更新
5	UPDATE_ROWS	BIGINT	更新行数。只有系统重启后，才会更新
6	LAST_STAT_DT	DATETIME(6)	上一次更新时间
7	MONITOR_FLAG	INTEGER	监控级别。1：监控级别为所有表；2：监控级别为由 SET_TABLE_PREFS 的指定过的表
8	RSVD1	VARBINARY(2000)	保留信息
9	TRUNCATED	INTEGER	所在表是否有过 TRUNCATE 操作

38. SYSMODIFICATIONS

监控功能专用。记录所有用户表对象的数据变化情况。其中，是否受统计信息影响表示是否在每次收集统计信息时将之前的监控数据清零。

仅当 INI 参数 MONITOR_MODIFICATIONS 开启时 SYSMODIFICATIONS 表才会收集数据，根据 INI 参数 MON_CHECK_INTERVAL 设置的时间间隔进行收集，也可使用过程 SP_FLUSH_MODIFICATIONS_INFO()手动进行收集。

序	列	数据类型	说明

号			
1	ID	INTEGER	表对象 ID
2	ROOT_ID	INTEGER	根表 ID，仅对分区表子表有效，默认值为-1，表示无根表
3	BASE_ID	INTEGER	父表 ID，仅对分区表子表有效，默认值为-1，表示无父表
4	INSERTS	BIGINT	插入行数，不受统计信息影响
5	DELETES	BIGINT	删除行数，不受统计信息影响
6	UPDATES	BIGINT	更新行数，不受统计信息影响
7	TRUNCATED	INTEGER	是否执行过 TRUNCATE 操作，不受统计信息影响
8	PART_INSERTS	BIGINT	插入行数，受统计信息影响
9	PART_DELETES	BIGINT	删除行数，受统计信息影响
10	PART_UPDATES	BIGINT	更新行数，受统计信息影响
11	PART_TRUNCATED	INTEGER	是否执行过 TRUNCATE 操作，受统计信息影响
12	LAST_MOD_TIME	DATETIME(6)	上次修改的时间
13	DROP_SUB_TABLES	INTEGER	删除的分区子表的个数
14	MONITOR_FLAG	INTEGER	是否为被指定监控的对象
15	CHANGED_AFTER_STAT	INTEGER	上次收集统计信息后，数据是否发生过变化
16	RSVD1	VARBINARY(2000)	保留信息

39. SYSFCOLDEFAULT

记录表添加列时追加列的默认值。

序号	列	数据类型	说明

1	TABID	INTEGER	表 ID
2	COLID	SMALLINT	列 ID
3	DEFDATA	VARBINARY(4096)	追加列默认值，默认值数据最长不允许超过 4000 字节

40. SYS.PROXY_INFO$

记录所有代理用户 ID 的信息。只有角色 DBA、DB_AUDIT_ADMIN、DB_POLICY_ADMIN 以及 DB_OBJECT_ADMIN 才能访问该系统表。

序号	列	数据类型	说明
1	CLIENT#	INTEGER	目标登录用户 ID
2	PROXY#	INTEGER	代理用户 ID
3	CREDENTIAL_TYPE#	TINYINT	认证方式，只支持 0（不认证）
4	FLAGS	TINYINT	代理范围标记，只支持 1（全部角色）

41. SYSTUNINGTASK

记录 SQL 语句调优任务相关信息。

序号	列	数据类型	说明
1	TASK_ID	INTEGER	调优任务 ID
2	USER_ID	INTEGER	任务所属用户 ID
3	TASK_NAME	VARCHAR(128)	任务名称
4	SQL_TEXT	TEXT	待调优 SQL 语句
5	BIND_LIST	VARCHAR(256)	绑定参数
6	DESCRIPTION	VARCHAR(256)	任务描述
7	CREATE_TIME	DATETIME(6)	任务创建时间
8	SCOPE	INTEGER	调优范围，取值：

			1：LIMITED; 2：COMPREHENSIVE
9	TIME_LIMIT	INTEGER	任务执行的最大时间，单位秒
10	RESVD1	INTEGER	保留字段 1
11	RESVD2	VARBINARY(128)	保留字段 2
12	RESVD3	VARBINARY(512)	保留字段 3

42. SYSTUNINGEXECUTION

记录 SQL 语句调优任务每次执行的相关信息。

序号	列	数据类型	说明
1	EXEC_ID	INTEGER	调优任务执行 ID
2	TASK_ID	INTEGER	任务 ID
3	EXEC_NAME	VARCHAR(128)	执行名称
4	DESCRIPTION	VARCHAR(256)	执行描述
5	START_TIME	DATETIME(6)	执行开始时间
6	END_TIME	DATETIME(6)	执行结束时间
7	STATUS	INTEGER	执行状态，取值: 1：INITIAL; 2：EXECUTING; 3：COMPLETED; 4：INTERRUPTED; 5：CANCELLED; 6：FATAL ERROR
8	RESVD1	INTEGER	保留字段 1

9	RESVD2	VARBINARY(128)	保留字段 2
10	RESVD3	VARBINARY(512)	保留字段 3

43. SYSTUNINGREPORT

记录 SQL 语句调优任务的调优报告相关信息。

序号	列	数据类型	说明
1	EXEC_ID	INTEGER	调优任务执行 ID
2	SECTION	INTEGER	显示章节，取值: 1：SUMMARY; 2：FINDING; 3：PLAN; 4：INFORMATION; 5：ERROR; 6：ALL
3	FINDING_NO	INTEGER	统计信息或索引信息。仅当 SECTION=2 时有效
4	OBJ_NO	INTEGER	涉及多个对象时的对象编号
5	BASIC_MSG	TEXT	调优报告等级为 BASIC 时的报告内容
6	TYPICAL_MSG	TEXT	调优报告等级为 TYPICAL 时的报告内容
7	ALL_MSG	TEXT	调优报告等级为 ALL 时的报告内容
8	RESVD1	INTEGER	保留字段 1
9	RESVD2	VARBINARY(128)	保留字段 2
10	RESVD3	VARBINARY(512)	保留字段 3

44. SYSPROFILES

记录 PROFILE 配置的资源和密码限制项。

序号	列	数据类型	说明
1	ID	INTEGER	PROFILE ID
2	CPU_PER_CALL	INTEGER	用户的一个请求能够使用的 CPU 时间上限，单位秒
3	CPU_PER_SESSION	INTEGER	一个会话允许使用的 CPU 时间上限，单位秒
4	MEM_SPACE	INTEGER	会话占有的私有内存空间上限，单位 MB
5	READ_PER_CALL	INTEGER	每个请求能够读取的数据页数
6	READ_PER_SESSION	INTEGER	一个会话能够读取的总数据页数上限
7	CONNECT_TIME	INTEGER	一个会话连接、访问和操作数据库服务器的时间上限，单位：10 分钟
8	SESS_PER_USER	INTEGER	高位 2 字节表示在所有 DSC 实例中，一个用户可以同时拥有的会话数量，低位 2 字节表示在一个实例中，一个用户可以同时拥有的会话数量
9	CONN_IDLE_TIME	INTEGER	用户会话的最大空闲时间，单位由 dm.ini 中安全参数 RESOURCE_FLAG 决定，取值 1 表示秒，0 表示分钟。
10	FAILED_NUM	INTEGER	用户登录失败次数限制
11	LIFE_TIME	INTEGER	口令有效期。单位：天
12	REUSE_TIME	INTEGER	重用同一个口令的时间间隔。单位：天
13	REUSE_MAX	INTEGER	重用同一个口令之前，需要修改口令的次数
14	LOCK_TIME	INTEGER	用户口令锁定时间，单位：分钟
15	GRACE_TIME	INTEGER	过期口令宽限期。单位：天
16	INFO1	VARCHAR(256)	INACTIVE_ACCOUNT_TIME

			设定的用户非活跃时间，单位：天

45. SYSUSERPROFILES

记录用户关联的 PROFILE。

序号	列	数据类型	说明
1	UID	INTEGER	用户 ID
2	PID	INTEGER	PROFILE ID

46. SYSPKGPROCINFOS

记录包中的方法信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	方法名
2	PKGID	INTEGER	包 ID
3	MTDID	INTEGER	方法 ID，每个包中方法从 0 开始编号（每个包额外存在一条 MTDID 字段值为 NULL 的记录）
4	OVERLOAD	INTEGER	方法在包中第几个重载
5	AGGREGATE	VARCHAR(10)	是否是聚合函数，兼容保留字段
6	PIPELINED	VARCHAR(10)	是否是管道化表函数，取值 YES/NO
7	IMPLTYPEOWNER	VARCHAR(128)	实现类所属的模式名，兼容保留字段
8	IMPLTYPENAME	VARCHAR(128)	实现类名，兼容保留字段
9	PARALLEL	VARCHAR(10)	是否可以并行，取值 YES/NO
10	INTERFACE	VARCHAR(10)	是否是使用 DCI 接口实现的表函数，兼容保留字段
11	DETERMINISTIC	VARCHAR(10)	是否是确定性函数，取值 YES/NO
12	IS_PROC	CHAR(1)	是否是过程，'Y'表示是过程，'N'表示是函数

13	INFO1	VARBINARY(1024)	表示方法是否支持批量

47. SYSPKGPROCPARAMS

记录包中方法的参数信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	参数名
2	PKGID	INTEGER	包 ID
3	MTDID	INTEGER	方法 ID，每个包中方法从 0 开始编号
4	PARAMID	SMALLINT	参数 ID，每个方法的参数从 1 开始编号，0 表示返回值
5	TYPE$	VARCHAR(128)	参数类型
6	LENGTH$	INTEGER	参数长度
7	SCALE	SMALLINT	参数精度
8	NULLABLE	CHAR(1)	参数是否可为空，取值 Y/N
9	IOFLAG	CHAR(2)	参数输入输出标记，'I'表示输入，'O'表示输出，'IO'表示输入输出，'R'表示返回值
10	DEFVAL	VARCHAR(2000)	参数缺省值
11	COMB_INFO	VARBINARY(1024)	参数的复合类型信息
12	INFO1	VARBINARY(1024)	保留字段

48. SYSCONTEXTLIBS

记录全文索引所需要的词库信息。

序号	列	数据类型	说明
1	LIB_ID	INTEGER	词库 ID
2	LIB_NAME	VARCHAR(32)	词库名

3	LIB_PATH	VARCHAR(256)	词库路径
4	DEF_LIB	CHAR(1)	是否为默认词库。Y 是，N 否
5	RESV1	VARCHAR(10)	保留字段
6	RESV2	INTEGER	保留字段
7	RESV3	INTEGER	保留字段
8	RESV4	INTEGER	保留字段

49. SYSTSGROUPS

记录表空间组相关信息。仅在 DMDPC 环境下有效。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	表空间组名
2	ID	INTEGER	表空间组 ID
3	DESCRIPTION	VARCHAR(256)	描述信息
4	TS_NUM	INTEGER	表空间组包含的表空间个数，每个表空间组最多包含 1536 个表空间
5	TS_INFO	VARBINARY(8192)	表空间组包含的表空间 ID 信息
6	INFO1	INTEGER	保留字段
7	INFO2	BIGINT	保留字段
8	INFO3	VARBINARY(1024)	保留字段

50. SYSPLNINFO

记录系统中绑定执行计划持久化的信息。

序号	列	数据类型	说明
1	PLN_ID	BIGINT	绑定执行计划 ID

2	HASH_VALUE	INTEGER	执行计划哈希值
3	SCHID	INTEGER	模式 ID
4	STMT_TYPE	INTEGER	SQL 语句类型
5	SQLSTR	VARBINARY(1000)	执行计划 SQL 语句
6	PLN_INFO	BLOB	执行计划信息
7	FMT_VERSION	INTEGER	格式版本号
8	CMD_VERSION	BIGINT	指令版本号
9	DISABLED	CHAR(1)	是否禁用

51. SYSPLNOBJID

记录系统中绑定执行计划对应字典对象的信息。

序号	列	数据类型	说明
1	PLN_ID	BIGINT	绑定执行计划 ID
2	OBJID	INTEGER	字典对象 ID

52. SYSOPARGS

记录自定义运算符的重载信息。

序号	列	数据类型	说明
1	ID	INTEGER	重载对象 ID
2	PID	INTEGER	重载对象的父 ID，对应 SYSOBJECTS 中被重载的运算符对象 ID
3	LEFTARG_TYPE	VARCHAR(128)	左运算对象的数据类型
4	RIGHTARG_TYPE	VARCHAR(128)	右运算对象的数据类型
5	CRTDATE	DATETIME(6)	重载对象的创建时间
6	VERSION	INTEGER	版本号

7	VALID	CHAR(1)	是否有效，Y：有效；N：无效
8	INFO1	VARBINARY(2048)	记录模式名、函数名等信息
9	INFO2	INTEGER	是否为系统自定义运算符，0：否；1：是
10	INFO3	BIGINT	保留字段
11	INFO4	VARBINARY(2048)	保留字段

53. SYSCRASHTRXS

记录回滚过程中访问到损坏数据页的事务信息。

序号	列	数据类型	说明
1	TS_ID	INTEGER	数据页所属表空间 ID
2	FIL_ID	INTEGER	数据页所属文件 ID
3	PAGE_NO	INTEGER	数据页号
4	TRX_ID	BIGINT	事务 ID
5	TRX_START_LSN	BIGINT	事务启动时系统 LSN 值
6	ERRCODE	INTEGER	错误码
7	RECORD_TIME	DATETIME(6)	登记时间
8	DB_MAGIC	BIGINT	数据库 DB_MAGIC 值

54. SYSPLANHIST

记录历史执行计划中每个节点的信息。仅当 USE_PLN_POOL!=0 和 ENABLE_MONITOR_PLNHIST 开启时才将历史执行计划记录到该系统表中。不支持 DMDPC 环境下记录执行计划历史信息。

系统会自动将历史执行计划登记到 SYSPLANHIST 中。对于系统尚未来得及登记的计划信息，用户可执行 SP_FLUSH_HIST_PLAN();进行手动登记。DSC 环境中，普通节点执行 SP_FLUSH_HIST_PLAN();时，普通节点的计划信息发送给控制节点。控制节点并不会直接

将计划刷盘至 SYSPLANHIST，需要等待控制节点达到刷盘条件或者执行 SP_FLUSH_HIST_PLAN();时，才会将计划一起记录进 SYSPLANHIST。

序号	列	数据类型	说明
1	PLAN_HASH_VALUE	INTEGER	SQL 哈希值
2	PLAN_ADDR	VARBINARY(8)	计划地址
3	SESS_ID	BIGINT	会话 ID
4	SCH_ID	INTEGER	模式 ID
5	SQL_STR	VARCHAR(1000)	SQL 语句
6	PLN_STR	CLOB	计划语句
7	ID	INTEGER	标识执行计划的每一步（从 0 开始）
8	OPERATION	VARCHAR(30)	操作符
9	OPTIONS	VARCHAR(20)	扫描类型
10	TABLE#	INTEGER	表 ID
11	TABLE_NAME	VARCHAR(128)	表名
12	TABLE_ALIAS	VARCHAR(128)	表别名
13	INDEX_NAME	VARCHAR(128)	索引名称
14	DEPTH	INTEGER	层数（从 0 开始）
15	COST	BIGINT	代价
16	CARDINALITY	BIGINT	行数
17	BYTES	INTEGER	字节数
18	PARTITION_START	INTEGER	分区起始编号（仅兼容）
19	PARTITION_STOP	INTEGER	分区结束编号（仅兼容）
20	CPU_COST	BIGINT	CPU 代价

21	IO_COST	BIGINT	IO 代价，暂不支持，始终为 0
22	ACCESS_PREDICATES	VATCHAR(128)	扫描范围
23	FILTER_PREDICATES	VARCHAR(1000)	过滤条件
24	JOIN_CONDITION	VARCHAR(1000)	连接条件
25	KEY_CONDITION	VARCHAR(1000)	键值条件
26	REMARKS	VARCHAR(1000)	评论（仅兼容）
27	TIMECREATE	DATETIME(6)	创建时间

55. SYSTYPEATTRINFO

记录对象的属性信息。

该表不具有 PUBLIC 权限，三权分立下具有 SOI，DB_AUDIT_SOI，DB_POLICY_SOI 权限，如果是四权分立则额外拥有 DB_OBJECT_SOI 权限。

序号	列	数据类型	说明
1	OID	INTEGER	存放当前对象的 id
2	ATTR_NAME	VARCHAR2(128)	存放当前对象的属性名
3	ATTR_TYPE_OID	INTEGER	属性的类型 id(仅属性为对象类型时有效)
4	ATTR_TYPE_NAME	VARCHAR2(128)	对象类型属性名
5	ATTR_LENGTH	INTEGER	属性的长度，仅字符串类型有效
6	ATTR_PRECISION	INTEGER	对象类型属性精度
7	ATTR_SCALE	INTEGER	对象类型属性范围
8	ATTR_NO	INTEGER	属性的序号，即当前属性在对象中是第几个
9	ATTR_CHARSET	INTEGER	对象类型属性所属字符集，暂时无用
10	ATTR_IS_INHERI	INTEGER	对象类型属性是否继承自父类型
11	ATTR_PROPERTIES	NUMBER	保存当前属性的类型，暂时无用

12	ATTR_INFO1	BIGINT	对象类型附加信息 1，暂时无用
13	ATTR_INOF2	VARBINARY(1024)	对象类型附加信息 2，暂时无用

56. SYSOPENHISTORY

记录数据库的 Open 历史记录。在 Primary 或 Normal 模式库 Open 时，向系统表写入一条记录（称为 Open 记录）。SYSOPENHISTORY 表最多只维护 128 条 Open 记录，达到 128 条以后会从第一条记录开始自动覆盖，并依次往后循环使用。

DMDPC 多副本系统中的节点以及 DMDPC 中的 BP 节点不会生成 Open 记录。

序号	列	数据类型	说明
1	RGUID	VARCHAR(48)	唯一 GUID 串，随机生成
2	OPEN_TIME	DATETIME(6)	实例 OPEN 时间
3	SYS_MODE	VARCHAR(32)	实例模式（PRIMARY/STANDBY/NORMAL）
4	PRIMARY_INST_NAME	VARCHAR(32)	主库实例名
5	CUR_INST_NAME	VARCHAR(32)	当前实例名
6	PRIMARY_DB_MAGIC	BIGINT	主库 DB_MAGIC 值
7	CUR_DB_MAGIC	BIGINT	当前库 DB_MAGIC 值
8	N_EP	SMALLINT	集群节点个数
9	PKG_SEQNO_0	BIGINT	0 号节点最大重演的日志包序号
10	PKG_SEQNO_1	BIGINT	1 号节点最大重演的日志包序号
11	PKG_SEQNO_2	BIGINT	2 号节点最大重演的日志包序号
12	PKG_SEQNO_3	BIGINT	3 号节点最大重演的日志包序号
13	PKG_SEQNO_4	BIGINT	4 号节点最大重演的日志包序号
14	PKG_SEQNO_5	BIGINT	5 号节点最大重演的日志包序号
15	PKG_SEQNO_6	BIGINT	6 号节点最大重演的日志包序号

16	PKG_SEQNO_7	BIGINT	7 号节点最大重演的日志包序号
17	PKG_SEQNO_8	BIGINT	8 号节点最大重演的日志包序号
18	PKG_SEQNO_9	BIGINT	9 号节点最大重演的日志包序号
19	PKG_SEQNO_10	BIGINT	10 号节点最大重演的日志包序号
20	PKG_SEQNO_11	BIGINT	11 号节点最大重演的日志包序号
21	PKG_SEQNO_12	BIGINT	12 号节点最大重演的日志包序号
22	PKG_SEQNO_13	BIGINT	13 号节点最大重演的日志包序号
23	PKG_SEQNO_14	BIGINT	14 号节点最大重演的日志包序号
24	PKG_SEQNO_15	BIGINT	15 号节点最大重演的日志包序号
25	APPLY_LSN_0	BIGINT	0 号节点最大重演的 LSN 值
26	APPLY_LSN_1	BIGINT	1 号节点最大重演的 LSN 值
27	APPLY_LSN_2	BIGINT	2 号节点最大重演的 LSN 值
28	APPLY_LSN_3	BIGINT	3 号节点最大重演的 LSN 值
29	APPLY_LSN_4	BIGINT	4 号节点最大重演的 LSN 值
30	APPLY_LSN_5	BIGINT	5 号节点最大重演的 LSN 值
31	APPLY_LSN_6	BIGINT	6 号节点最大重演的 LSN 值
32	APPLY_LSN_7	BIGINT	7 号节点最大重演的 LSN 值
33	APPLY_LSN_8	BIGINT	8 号节点最大重演的 LSN 值
34	APPLY_LSN_9	BIGINT	9 号节点最大重演的 LSN 值
35	APPLY_LSN_10	BIGINT	10 号节点最大重演的 LSN 值
36	APPLY_LSN_11	BIGINT	11 号节点最大重演的 LSN 值
37	APPLY_LSN_12	BIGINT	12 号节点最大重演的 LSN 值
38	APPLY_LSN_13	BIGINT	13 号节点最大重演的 LSN 值

39	APPLY_LSN_14	BIGINT	14 号节点最大重演的 LSN 值
40	APPLY_LSN_15	BIGINT	15 号节点最大重演的 LSN 值

57. SYSDTYPEALIAS

记录系统中所有类型别名的信息。该表在执行 SP_INIT_DTYPE_SYS(1);后创建。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	类型别名的名称
2	BASE_NAME	VARCHAR(128)	基础数据类型名
3	LENGTH	SMALLINT	基础数据类型长度
4	SCALE	SMALLINT	见 SYSCOLUMNS 中的 SCALE 列

附录 2 动态性能视图

动态视图存储着数据库的配置及数据库中变化的活动状态信息，可以通过查询V$DYNAMIC_TABLES获得所有的动态性能视图名称，V$DYNAMIC_TABLES各列介绍如下：

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	动态视图名
2	ID	INTEGER	动态视图 ID
3	SCHNAME	VARCHAR(128)	动态视图所属模式
4	SYNONYMS	VARCHAR(512)	动态视图的同义词

动态视图的模式名为 SYS，例如：SYS. V$DICT_CACHE_ITEM。不过 DM 为每个动态视图建立了同名的全局同义词，因此用户也可以省略模式名对动态视图进行访问。

关于动态视图的几点说明如下：

一．普通用户查询某动态视图需要获得系统管理员的授权才能进行；

二．在 MPP 环境下，查询结果会显示所有节点的动态视图信息；

三．与备份还原相关的动态视图此处不作介绍，请参考《DM8 备份与还原》；

四．与 DM MPP 相关的动态视图此处不作介绍，请参考《DM8 大规模并行处理 MPP》；

五．与 DMDSC 相关的动态视图此处不作介绍，请参考《DM8 数据共享集群》；

六．与数据守护相关的动态视图此处不作介绍，请参考《DM8 数据守护与读写分离集群V4.0》。

1. 资源管理

1) V$DICT_CACHE_ITEM

显示字典缓存中的字典对象信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(128)	字典对象的类型。类型如下：DB、TABLE、VIEW、INDEX、USER、ROLE、PROCEDURE、TRIGGER、CONSTRAINT、SCHEMA、SEQUENCE、DBLINK、PACKAGE、OBJECT、SYNOM、CRYPT、CIPHER、IDENTITY、SYS PRIVILEGE、OBJ PRIVILEGE、POLICY、RULE、COLUMN、DOMAIN、COLLATION、CONTEXT INDEX、REGEXP REWRITE、NORMAL REWRITE、CONTEXT、DIRECTORY、PROFILE、OPERATOR
2	ID	INTEGER	字典对象 ID
3	NAME	VARCHAR(128)	字典对象的名称
4	SCHID	INTEGER	字典对象所属模式
5	PID	INTEGER	父 ID
6	HASH_VALUE	BIGINT	缓存的 HASH 值
7	SIZE	INTEGER	字典对象内存大小，单位 BYTE

2) V$DICT_CACHE

显示字典缓存信息。

序号	列	数据类型	说明
1	ADDR	VARCHAR(20)	地址
2	POOL_ID	INTEGER	缓冲区 ID
3	TOTAL_SIZE	BIGINT	总大小，单位 BYTE
4	USED_SIZE	BIGINT	实际使用大小，单位 BYTE

5	DICT_NUM	INTEGER	字典对象总数

3) V$BUFFERPOOL

页面缓冲区动态性能表，用来记录页面缓冲区结构的信息。

其中，（一）N_DISCARD、N_PHY_READS、N_PHY_M_READS、N_UPD_REMOVE、N_PHY_WRITE, N_UPD_PUT、N_UPD_SEARCH 几个字段为 INTEGER 类型；（二）N_DISCARD64、N_PHY_READS64、N_PHY_M_READS64、N_UPD_REMOVE64 、N_PHY_WRITE64 、N_UPD_PUT64、N_UPD_SEARCH64 为 BIGINT64 类型。（一）和（二）中几个字段的意义相同，只是数据类型不同。正常情况下，（一）和（二）数据结果是一样的。如果（一）的字段有溢出，那么以（二）的结果为准。

序号	列	数据类型	说明
1	ID	INTEGER	缓冲区 ID
2	NAME	VARCHAR(20)	缓冲区名称　NORMAL/RECYCLE/FAST
3	PAGE_SIZE	INTEGER	基缓冲区页大小，不包括扩展池页，单位 BYTE
4	N_PAGES	INTEGER	页数
5	N_FIXED	INTEGER	数据页被引用的次数
6	FREE	INTEGER	空闲页数目
7	N_DIRTY	INTEGER	脏页数目
8	N_CLEAR	INTEGER	非空闲页数目
9	N_TOTAL_PAGES	INTEGER	页总数，包括扩展池页
10	N_MAX_PAGES	INTEGER	最多的页数
11	N_LOGIC_READS	BIGINT	READ 命中的次数
12	N_DISCARD	BIGINT	淘汰的页数
13	N_PHY_READS	BIGINT	READ 未命中的次数

14	N_PHY_M_READS	BIGINT	READ 未命中，批量读的次数
15	RAT_HIT	DOUBLE	命中率
16	N_EXP_BUFFERPOOL	INTEGER	扩展缓冲区个数
17	N_UPD_REMOVE	BIGINT	从 update 链表删除页总数
18	N_PHY_WRITE	BIGINT	物理写入磁盘总次数
19	N_UPD_PUT	BIGINT	DSC 远程读取数据后，加入 update 链表总数
20	N_UPD_SEARCH	BIGINT	DSC 远程读取数据后，查找 update 链表插入位置扫描总数
21	N_DISCARD64	BIGINT	淘汰的页数
22	N_PHY_READS64	BIGINT	READ 未命中的次数
23	N_PHY_M_READS64	BIGINT	READ 未命中，批量读的次数
24	N_UPD_REMOVE64	BIGINT	从 update 链表删除页总数
25	N_PHY_WRITE64	BIGINT	物理写入磁盘总数
26	N_UPD_PUT64	BIGINT	DSC 远程读取数据后，加入 update 链表总数
27	N_UPD_SEARCH64	BIGINT	DSC 远程读取数据后，查找 update 链表插入位置扫描总数
28	N_BUFMEM_ALLOCED	INTEGER	已分配出去的缓冲区页数
29	N_PTX_BI_ALLOCED	INTEGER	分配给 PTX 前镜像页的缓冲区页数

4） V$BUFFER_LRU_FIRST

显示所有缓冲区 LRU 链首页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲区 ID
2	SPACE_ID	INTEGER	表空间 ID

3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态。1：空闲；2：使用；4:正在读；5：正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

5） V$BUFFER_UPD_FIRST

显示所有缓冲区 UPDATE 链首页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲区 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号

5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态。1：空闲；2：使用；4:正在读；5：正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

6） V$BUFFER_LRU_LAST

显示所有缓冲区 LRU 链末页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲区 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址

7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN
9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态。1：空闲；2：使用；4:正在读；5：正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

7） V$BUFFER_UPD_LAST

显示所有缓冲区 UPDATE 链末页信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	缓冲区 ID
2	SPACE_ID	INTEGER	表空间 ID
3	FILE_ID	INTEGER	文件 ID
4	PAGE_NO	INTEGER	数据在文件中的页号
5	RWLOCK	BIGINT	线程锁地址
6	PAGE	BIGINT	数据页对应的内存块地址
7	HASH	BIGINT	HASH 地址
8	LSN	BIGINT	页的当前 LSN

9	LRU_NEXT	BIGINT	下一个非空闲页地址
10	LRU_PREV	BIGINT	上一个非空闲页地址
11	UPD_NEXT	BIGINT	下一个脏页地址
12	UPD_PREV	BIGINT	上一个脏页地址
13	N_FIXED	INTEGER	数据页被引用的次数
14	STATUS	INTEGER	页状态。1：空闲；2：使用；4:正在读；5：正在写
15	ACCESS_CNT	INTEGER	正在访问的页号
16	FIRST_LSN	BIGINT	第一次被修改时对应的日志文件号
17	FIRST_FIL	INTEGER	第一次被修改时对应的已经刷盘的最新文件号
18	FIRST_OFF	BIGINT	第一次被修改时对应的已经刷盘的最新文件偏移

8） V$CACHEITEM

显示 SQL 缓冲区中缓存项的相关信息。

序号	列	数据类型	说明
1	ADDRESS	BIGINT	CACHE 项的地址
2	TYPE$	VARCHAR(8188)	CACHE 项的类型：SQL，计划（PLN），结果集（RS），包信息（PKG_INFO）
3	OVERFLOW	CHAR(1)	是否溢出
4	IN_POOL	CHAR(1)	是否在内存池中
5	DISABLED	CHAR(1)	是否可用
6	N_FIXED	INTEGER	此缓冲项被引用的次数
7	TS_VALUE	INTEGER	时间戳
8	ITEM_SIZE	BIGINT	缓存节点大小（字节数）

9	N_HIT	INTEGER	节点命中次数
10	N_DIS_FIXED	INTEGER	内部保留字段

9） V$CACHERS

显示 SQL 缓冲区中结果集缓存的相关信息。在 INI 参数 USE_PLN_POOL !=0 时才统计。

序号	列	数据类型	说明
1	CACHE_ITEM	BIGINT	结果集在缓冲区中的地址，对应 V$CACHEITEM 中的 ADDRESS
2	PLN	BIGINT	结果集对应查询计划，对应 V$SQL_PLAN 的 PLN_ADDR
3	N_TABLES	INTEGER	结果集涉及的表的数目
4	TABLEID	VARCHAR(256)	结果集涉及的表的 ID，用空格隔开
5	MEM_SIZE	BIGINT	缓存节点大小（字节数）
6	EXEC_TIME	INTEGER	结果集生成所消耗的时间，单位毫秒

10） V$CACHESQL

显示 SQL 缓冲区中 SQL 语句缓存的相关信息。

序号	列	数据类型	说明
1	CACHE_ITEM	BIGINT	在内存中的位置，对应 V$CACHEITEM 中的 ADDRESS
2	SQL	VARCHAR(4096)	SQL 文本
3	LEN	INTEGER	SQL 文本长度
4	HASH_VALUE	INTEGER	HASH 值

11） V$CACHEPLN

显示 SQL 缓冲区中的计划缓存的相关信息。在 INI 参数 USE_PLN_POOL !=0 时才统计。

序	列	数据类型	说明

号			
1	CACHE_ITEM	BIGINT	CACHE 项的地址
2	TYPE$	VARCHAR(8188)	CACHE 项的类型: SQL, 过程（SP）, 触发器（TRIGGER）, 动态执行（DYNAMIC SQL）, 动态游标（DYNAMIC CURSOR）, 其它类型（UNKNOWN）
3	HASH_VALUE	INTEGER	CACHE 项的 HASH 值
4	SCHID	INTEGER	模式 ID
5	USER_ID	INTEGER	用户 ID
6	OBJ_ID	INTEGER	对象 ID
7	RS_CAN_CACHED	CHAR(1)	结果集是否缓存
8	N_RS_CACHED	INTEGER	无意义，均显示 0
9	N_TABLE	INTEGER	涉及的表个数
10	TABLEID	VARCHAR(256)	计划涉及的表的 ID，用空格隔开
11	SQLSTR	VARCHAR(4096)	语句内容
12	SQLCACHE	BIGINT	对应 V$CACHESQL 中的 CACHE_ITEM 地址
13	DDLSCRIPT	VARCHAR(4096)	无意义，均显示 NULL
14	RET_CMD	INTEGER	返回响应命令字
15	STMT_TYPE	INTEGER	SQL 语句类型

16	N_LIT_PARA	INTEGER	常量参数个数
17	N_CLNT_PARA	INTEGER	客户端绑定参数个数
18	N_COLS	INTEGER	涉及到的列个数
19	SEL_UPDATABLE	CHAR(1)	是否为查询更新
20	MEM_SIZE	BIGINT	缓存节点大小，单位为字节数
21	RS_CAN_CACHED_IN_RULE	CHAR(1)	依据基本规则是否可以缓存结果集（'Y'或'N'），实际是否缓存还受结果集缓存相关参数影响，或者被手工强制设置不缓存
22	BINDED	CHAR(1)	用于表示执行计划的绑定情况，'N'：未绑定；'M'：内存中绑定；'P'：持久化绑定；'B'：内存中绑定与持久化绑定同时生效
23	INDEPENDENT	CHAR(1)	用于表示执行计划是否为独立计划。'Y'：独立计划；'N'：非独立计划
24	BPARAM_FOR_OPT	CHAR(1)	用于表示执行计划是否是批量参数优化的计划。'Y'：是；'N'：否
25	HAS_VPD_POLICY	CHAR(1)	用于表示执行计划是否含有 VPD 策略，'Y'：有；'N'：没有

12） V$SQLTEXT

显示 SQL 缓冲区中的 SQL 语句的相关信息。

序号	列	数据类型	说明
1	SQL_ADDR	VARBINARY(8)	语句在缓存中的地址
2	SQL_ID	INTEGER	SQL 语句 ID（唯一标识）
3	N_EXEC	INTEGER	语句执行次数

4	HASH_VALUE	INTEGER	语句 HASH 值
5	CMD_TYPE	VARCHAR(32)	语句类型（来自语句类型，例如：DRO_STMT_CTAB）
6	SQL_TEXT	VARCHAR(7168)	SQL 语句内容（如果超过 7168 字符，则分段存储）
7	SQL_NTH	INTEGER	SQL 语句段号（从 0 开始）
8	HASH	VARBINARY(8)	SQL 语句的哈希节点地址
9	LINK_ADDR	VARBINARY(8)	下一个 SQL 语句地址
10	SQL_TEXT_ID	VARCHAR(13)	SQL 语句对应的固定散列字符，用于标记
11	USER_ID	INTEGER	用户 ID
12	SCH_ID	INTEGER	模式 ID

13) V$SQL_PLAN

显示 SQL 缓冲区中的执行计划的相关信息。在 INI 参数 USE_PLN_POOL!=0 时才统计。

序号	列	数据类型	说明
1	PLN_ADDR	VARBINARY(8)	计划在缓存中的地址
2	HASH_VALUE	INTEGER	计划 HASH 值
3	SQL_ID	INTEGER	SQL 语句 ID（唯一标识）
4	PLN_TYPE	VARCHAR(16)	计划类型
5	SQLSTR	VARCHAR(1000)	语句内容
6	RT_METHOD	VARBINARY(8)	计划的运行时方法
7	SVPNT	VARCHAR(128)	计划的保存点名称
8	N_LIT_PARAS	INTEGER	常量参数个数
9	N_CLNT_PARAS	INTEGER	客户端绑定参数个数
10	N_COLS	INTEGER	涉及到的列个数

11	SEL_UPDATABLE	CHAR(1)	是否为查询更新
12	N_NDCTS	INTEGER	涉及到的字典对象个数
13	N_SUBPLNS	INTEGER	子计划个数
14	N_SUBPGS	INTEGER	子过程个数
15	PRE_COMMIT	CHAR(1)	执行 DDL 之前先提交事务
16	IS_RECURSIVE	CHAR(1)	是否为递归调用
17	BPARAM_CAN_OPT	CHAR(1)	参数是否可以优化
18	NDCT_VERSION	INTEGER	字典对象版本号
19	CAN_REUSE	CHAR(1)	计划是否可重用
20	HAS_SQL	CHAR(1)	是否有对应的 SQL 语句
21	HASH	VARBINARY(8)	计划哈希节点地址
22	SCHID	INTEGER	模式 ID
23	USER_ID	INTEGER	用户 ID
24	OBJ_ID	INTEGER	对象 ID
25	RS_CAN_CACHE	CHAR(1)	结果集是否缓存
26	RS_CAN_CLT_CACHE	CHAR(1)	客户端是否缓存结果集
27	RS_MUTEX	VARBINARY(8)	互斥量地址
28	N_TABLES	INTEGER	涉及的表个数
29	LINK_ADDR	VARBINARY(8)	下一个计划地址
30	PHD_TIME	DATETIME(6)	生成计划时间
31	OPTIMIZER	VARCHAR(128)	优化方式
32	TABLEID	VARCHAR(256)	计划涉及的表的 ID，用空格隔开
33	SQLCACHE	BIGINT	对应 V$CACHESQL 中的 CACHE_ITEM

34	RET_CMD	SMALLINT	返回响应命令字
35	STMT_TYPE	INTEGER	SQL 语句类型号，具体对应类型名称可查看 V$SQLCOMMAND
36	MEM_SIZE	BIGINT	缓存节点大小，单位为字节数
37	RS_CAN_CACHED_IN_RULE	CHAR(1)	依据基本规则是否可以缓存结果集（'Y'或 'N'），实际是否缓存还受结果集缓存相关参数影响，或者被手工强制设置不缓存
38	NLS_SORT_TYPE	INTEGER	执行计划的自然语言的排序方式。0：按默认字符集二进制编码排序；1：按中文拼音排序；2：按中文笔画排序；3：按中文部首排序；4：按泰文排序；5：按韩文排序
39	BINDED	CHAR(1)	用于表示执行计划的绑定情况，'N'：未绑定；'M'：内存中绑定；'P'：持久化绑定；'B'：内存中绑定与持久化绑定同时生效
40	SQL_TEXT_ID	VARCHAR(13)	语句固定的散列字符；相同的 SQL 散列字符相同

14) V$MEM_POOL

显示所有的内存池信息。

序号	列	数据类型	说明
1	ADDR	BIGINT	内存结构地址
2	NAME	VARCHAR(128)	内存池名称
3	IS_SHARED	CHAR(1)	是否是共享的

4	CHK_MAGIC	CHAR(1)	是否打开了内存校验
5	CHK_LEAK	CHAR(1)	是否打开了泄漏检查
6	IS_OVERFLOW	CHAR(1)	是否已经触发 BAK_POOL 的分配
7	IS_DSA_ITEM	CHAR(1)	是否是 DSA(Dameng Share Area)项目， 目前一律为 N
8	ORG_SIZE	BIGINT	初始大小，单位为字节数
9	TOTAL_SIZE	BIGINT	当前总大小，单位为字节数
10	RESERVED_SIZE	BIGINT	当前分配出去的大小，单位为字节数
11	DATA_SIZE	BIGINT	当前分配出去的数据占用大小，单位为字节数
12	EXTEND_SIZE	BIGINT	每次扩展的块大小，单位为字节数
13	TARGET_SIZE	BIGINT	可以扩展到的大小，单位为字节数。当 TARGET_SIZE 为 0 时，不限制此内存池的扩展；对于共享内存池，即使 TARGET_SIZE 不为 0，也不限制其扩展，TARGET_SIZE 用于提示系统尽快把内存占用释放到 TARGET_SIZE 以下
14	EXTEND_LEN	INTEGER	扩展链长度（即扩展块数量）
15	N_ALLOC	BIGINT	累计分配了几次
16	N_EXTEND_NORMAL	BIGINT	TARGET 范围内累计扩展次数
17	N_EXTEND_EXCLUSIVE	BIGINT	超过 TARGET 累计扩展次数
18	N_FREE	BIGINT	累计释放次数
19	MAX_EXTEND_SIZE	BIGINT	保留字段，始终返回 NULL
20	MIN_EXTEND_SIZE	BIGINT	保留字段，始终返回 NULL
21	FILE_NAME	VARCHAR(256)	本池创建点所在的源文件名

22	FILE_LINE	INTEGER	创建点所在的代码行
23	CREATOR	INTEGER	创建者线程号
24	EXTEND_MODE	CHAR(1)	内存池的扩展模式，O：扩展时从操作系统（OS）申请扩展块；S：扩展时从共享池（Share Pool）申请扩展块
25	PEAK_SIZE	BIGINT	该内存池使用内存的峰值，单位为字节数
26	PARENT	BIGINT	该父池的内存结构地址

15） V$MEM_REGINFO

显示系统当前已分配并未释放的内存信息，当 MEMORY_LEAK_CHECK 为 1 时才会在此动态视图注册信息。

序号	列	数据类型	说明
1	POOL	VARCHAR(128)	注册项来源内存池名称
2	FNO	INTEGER	文件编号
3	LINENO	INTEGER	文件中的代码行编号
4	REFNUM	INTEGER	引用次数
5	RESERVED_SIZE	BIGINT	调用点累计分配未释放的内存量，单位为字节数
6	DATA_SIZE	BIGINT	调用点累计分配未释放的净数据量，单位为字节数
7	FNAME	VARCHAR(256)	源文件名
8	ADDR	BIGINT	POOL 的内存地址
9	N_ENTER	INTEGER	用于监测该文件/行位置内存申请调用/释放的次数

16） V$GSA

显示全局 SORT 内存缓冲区的使用情况。

序号	列	数据类型	说明
1	ADDR	BIGINT	缓冲块的内存地址
2	SIZE	BIGINT	缓冲块大小，单位为字节数

17) V$MEM_HEAP

显示系统当前内存堆的信息，仅当系统启动时 MEMORY_LEAK_CHECK 为 1 时有效。

序号	列	数据类型	说明
1	ADDR	BIGINT	内存堆对象的地址
2	FILE_NAME	VARCHAR(256)	本内存堆创建点所在的源文件名
3	FILE_LINE	INTEGER	创建点所在的代码行
4	DATA_LEN	BIGINT	内存堆占用空间大小，单位为字节
5	BLK_LEN	INTEGER	内存堆包含的 block 个数

18) V$LARGE_MEM_SQLS

最近 1000 条使用大内存的 sql 语句。一条 sql 语句使用的内存值超过 ini 参数 LARGE_MEM_THRESHOLD，就认为使用了大内存。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID，系统内部标识
2	SQL_ID	INTEGER	SQL 语句 ID
3	SQL_TEXT	VARCHAR(1024)	SQL 文本。最多选取 1024 个字符。如果 SQL 语句大于 1024 个字符，则从 SQL 语句的首尾各取一半，中间部分省略
4	MEM_USED_BY_K	BIGINT	使用的内存数，单位 KB
5	FINISH_TIME	DATETIME(0)	执行结束时间

6	N_RUNS	INTEGER	执行次数
7	SEQNO	INTEGER	编号
8	TRX_ID	BIGINT	事务号
9	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号

19) V$SYSTEM_LARGE_MEM_SQLS

系统中使用大内存最多的 20 条 sql 语句。字段定义与 v$large_mem_sqls 相同。

20) V$SCP_CACHE

显示缓存池信息。

序号	列	数据类型	说明
1	SCP_ADDR	VARBINARY(8)	缓存池地址
2	PLN_CNT	BIGINT	计划缓存总数
3	RS_CNT	BIGINT	结果集缓存总数
4	SQL_CNT	BIGINT	SQL 缓存总数
5	PKGINFO_CNT	BIGINT	包信息缓存总数
6	LRU_SIZE	INTEGER	LRU 链表的节点数
7	DISABLE_SIZE	INTEGER	失效缓存节点数
8	DISCARD	BIGINT	缓存对象淘汰次数
9	DISCARD_MEM	BIGINT	缓存对象淘汰的大小，单位 BYTE
10	N_ADD_FAIL	BIGINT	添加缓存失败次数

21) V$DB_SYSPRIV_CACHE

系统权限缓存信息。

序号	列	数据类型	说明

1	ID	INTEGER	字典对象的 ID
2	UID	INTEGER	用户 ID
3	DDL_PRIV	VARBINARY(512)	DDL 权限
4	DML_PRIV	VARBINARY(16)	DML 权限，由 4 个 INT 值组成。 0~3 字节：is_any&is_tab 4~7 字节：!is_any&is_tab 8~11 字节：is_any&!is_tab 12~15 字节：!is_any&!is_tab

22) V$DB_OBJPRIV_CACHE

对象权限缓存信息。

序号	列	数据类型	说明
1	ID	INTEGER	字典对象的 ID
2	UID	INTEGER	用户 ID
3	COLID	INTEGER	列 ID
4	OBJ_PRIV	VARBINARY(8)	对象权限
5	COL_PRIV	VARBINARY(8)	列权限

23) V$SQL_STAT

语句级资源监控内容。记录当前正在执行的 SQL 语句的资源开销。需要 ENABLE_MONITOR=1 才开始监控。针对 63~68 列中的监控项，还需开启参数 MONITOR_SQL_EXEC=1、ENABLE_MONITOR_DMSQL=1 才开始监控。

序号	列	数据类型	说明
1	SESSID	BIGINT	会话 ID，系统内部标识

2	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
3	SQL_TXT	VARCHAR(1024)	语句
4	SQL_ID	INTEGER	SQL 语句 ID
5	EXEC_TIME	BIGINT	以系统时间统计的执行时间，单位毫秒
6	PARSE_CNT	BIGINT	解析次数
7	PARSE_TIME	BIGINT	以系统时间统计的解析时间，单位毫秒
8	HARD_PARSE_CNT	BIGINT	硬解析次数
9	HARD_PARSE_TIME	BIGINT	硬解析时间，单位毫秒
10	SEL_SQL_CNT	BIGINT	执行的查询语句总数
11	INS_SQL_CNT	BIGINT	执行的插入语句总数
12	DEL_SQL_CNT	BIGINT	执行的删除语句总数
13	UPD_SQL_CNT	BIGINT	执行的更新语句总数
14	DDL_SQL_CNT	BIGINT	执行的 DDL 语句总数
15	SEL_IN_PL_CNT	BIGINT	执行的语句块中的查询语句总数
16	INS_IN_PL_CNT	BIGINT	执行的语句块中的插入语句总数
17	DEL_IN_PL_CNT	BIGINT	执行的语句块中的删除语句总数
18	UPD_IN_PL_CNT	BIGINT	执行的语句块中的更新语句总数
19	DYN_EXEC_CNT	BIGINT	执行的语句块中的动态执行语句总数
20	DDL_EVT_TRG_CNT	BIGINT	DDL 事件触发器触发次数
21	STMT_BF_TRG_CNT	BIGINT	语句级 BEFORE 触发器触发次数
22	STMT_AF_TRG_CNT	BIGINT	语句级 AFTER 触发器触发次数

23	ROW_BF_TRG_CNT	BIGINT	元组级 BEFORE 触发器触发次数
24	ROW_AF_TRG_CNT	BIGINT	元组级 AFTER 触发器触发次数
25	INSTEAD_OF_TRG_CNT	BIGINT	INSTEAD OF 触发器触发次数
26	OPTIMIZED_SORT_CNT	BIGINT	最优排序次数
27	ONE_WAY_SORT_CNT	BIGINT	单路排序次数
28	MULTI_WAY_SORT_CNT	BIGINT	多路排序次数
29	RUNTIME_OBJ_ALLOC_CNT	BIGINT	运行时对象创建次数
30	RUNTIME_OBJ_SIZE_CNT	BIGINT	运行时对象占用空间大小，单位 BYTE
31	RUNTIME_OBJ_RECLAIM_CNT	BIGINT	运行时对象回收次数
32	LONG_ROW_CVT_CNT	BIGINT	超长记录字段压缩次数
33	LOGIC_READ_CNT	BIGINT	逻辑读页次数
34	PHY_READ_CNT	BIGINT	物理读页次数
35	PHY_MULTI_READ_CNT	BIGINT	物理读多页次数
36	RECYCLE_LOGIC_READ_CNT	BIGINT	临时表空间逻辑读次数
37	RECYCLE_PHY_READ_CNT	BIGINT	临时表空间物理读次数
38	HBUF_LOGIC_READ_CNT	BIGINT	HBUF 逻辑读次数
39	HBUF_PHY_READ_CNT	BIGINT	HBUF 物理读次数
40	HBUF_PHY_WRITE_CNT	BIGINT	HBUF 物理写次数
41	HBUF_PHY_READ_SIZE	BIGINT	HBUF 物理读总大小，单位 BYTE
42	HBUF_PHY_WRITE_SIZE	BIGINT	HBUF 物理写总大小，单位 BYTE
43	UNDO_PAGE_CHANGES_CNT	BIGINT	undo 页变化次数
44	RECYCLE_PAGE_CHANGES_CNT	BIGINT	临时页变化次数
45	DATA_PAGE_CHANGES_CNT	BIGINT	数据页变化次数

46	IO_WAIT_TIME	BIGINT	I/O 等待时间(MS)
47	TAB_SCAN_CNT	BIGINT	统计全表扫描次数
48	HASH_JOIN_CNT	BIGINT	统计哈希连接的次数
49	BTR_SPLIT_CNT	BIGINT	B 树分裂次数
50	BTR_PAGE_DISCARD_CNT	BIGINT	数据页丢弃次数
51	BTR_LEVEL_DISCARD_CNT	BIGINT	B 树层丢弃次数
52	BTR_LEFT_TRY_CNT	BIGINT	B 树左移次数
53	BTR_DIRECT_UPDATE_CNT	BIGINT	B 树直接更新次数
54	BTR_INSDEL_UPDATE_CNT	BIGINT	B 树插入删除更新次数
55	BTR_UPDATE_2ND_CONFLICT_CNT	BIGINT	二级索引更新冲突次数
56	UPDATE_MVCC_RETRY_CNT	BIGINT	多版本更新重试次数
57	DELETE_MVCC_RETRY_CNT	BIGINT	多版本删除重试次数
58	MAX_MEM_USED	BIGINT	内存使用峰值(KB)
59	VIS_RECS_CNT	BIGINT	可见数据获取次数
60	VIS_RECS_TIME	BIGINT	可见数据获取时间，单位微秒
61	INVIS_RECS_CNT	BIGINT	不可见数据获取次数
62	INVIS_RECS_TIME	BIGINT	不可见数据获取时间，单位微秒
63	NET_BYTES_RECV	BIGINT	从客户端接收的数据字节数
64	NET_BYTES_SEND	BIGINT	向客户端发送的数据字节数
65	NET_ROUNDTRIPS	BIGINT	和客户端之间的数据往返传送次数
66	REDO_SIZE	BIGINT	REDO 日志大小，单位字节
67	PARSE_ELAPSD	BIGINT	以 CPU 时钟统计的解析时间，单位毫秒

68	EXEC_CPU	BIGINT	以 CPU 时钟统计的执行时间，单位毫秒
69	PLN_ADDR	BIGINT	执行计划地址，与 V$CACHEPLN 的 CACHE_ITEM 列对应
70	START_TIME	DATETIME(6)	SQL 执行的开始时间
71	END_TIME	DATETIME(6)	SQL 执行的结束时间
72	BTR_SPLIT_OPT_CNT	BIGINT	BTR_SPLIT_MODE 开启插入封锁优化时，B 树分裂优化成功的次数
73	BTR_SPLIT_OPT_FAIL_CNT	BIGINT	BTR_SPLIT_MODE 开启插入封锁优化时，B 树分裂优化失败的次数
74	EXEC_ID	BIGINT	SQL 执行 ID，取值为-1 表示没有到计划执行阶段
75	SQL_TEXT_ID	VARCHAR(13)	语句固定的散列字符；相同的 SQL 散列字符相同

24) V$SQL_STAT_HISTORY

语句级资源监控内容。记录历史 SQL 语句（不含存储过程中的 SQL）执行的资源开销。需要 ENABLE_MONITOR=1 才开始监控。单机最大行数为 10000。

其中 5~62、64 和 66 列中的监控项，可以通过 SP_SET_SQL_STAT_THRESHOLD() 设置监控阈值，当资源超过设置的阈值才开始统计并记录该项，具体使用参考《DM8_SQL 语言使用手册》。当前已设置的监控阈值内容，可以通过查询视图

V$SQL_STAT_THRESHOLD 进行查看。

序号	列	数据类型	说明
1~75	1~75 行和 V$SQL_STAT 结构相同。请参考 V$SQL_STAT	……	……

25） V$HLDR_TABLE

记录当前系统中所有 HLDR 使用 HLDR_BUF 的情况。

序号	列	数据类型	说明
1	TABLE_ID	INTEGER	装载表 ID
2	N_ALLOC	INTEGER	申请 HLDR_BUF 的次数
3	N_FAIL	INTEGER	申请 HLDR_BUF 失败的次数
4	N_WAIT	INTEGER	申请 HLDR_BUF 时等待的次数
5	WAIT_TIME	BIGINT	申请 HLDR_BUF 等待的总时长
6	N_BUF	INTEGER	申请的 HLDR_BUF 的个数

26） V$HBUF_STAT

显示当前系统中 HUGE BUFFER 的使用信息。

序号	列	数据类型	说明
1	SIZE_LEVEL	INTEGER	空间大小的层级，按照页数是 2 的幂数划分层级
2	MIN_PAGE_NUM	INTEGER	当前层级理论上的最小页数
3	MAX_PAGE_NUM	INTEGER	当前层级理论上的最大页数
4	N_FREE_FRAGMENTS	INTEGER	当前层级 HUGE BUFFER 中空闲碎片的个数
5	N_FREE_PAGES	INTEGER	当前层级所有空闲碎片的页数之和
6	N_MEM_USE_PAGES	INTEGER	内存一次使用的空间大小属于当前层级的页数总和

7	N_MEM_ALLOC	BIGINT	内存一次申请的空间大小属于当前层级的次数
8	N_MEM_RELEASE	BIGINT	内存一次释放的空间大小属于当前层级的次数
9	N_CTL_USE_PAGES	INTEGER	HUGE BUFFER 一次使用的空间大小属于当前层级的页数总和
10	N_CTL_ALLOC	BIGINT	HUGE BUFFER 一次申请的空间大小属于当前层级的次数
11	N_CTL_RELEASE	BIGINT	HUGE BUFFER 一次释放的空间大小属于当前层级的次数
12	N_CTL_RESERVE	BIGINT	HUGE BUFFER 一次 RESERVE 的空间大小属于当前层级的次数

27) V$CACHE_ID

显示缓存中可分配 ID 的相关信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(100)	对象类型名
2	MIN_ID	INTEGER	最小可用 ID 值
3	MAX_ID	INTEGER	最大可用 ID 值
4	COUNT	INTEGER	可用 ID 数量

28) V$IDBITMAP

显示系统中已分配字典 ID 的位图项信息，仅当 INI 参数 ID_RECYCLE_FLAG 为 1 时有效。

序号	列	数据类型	说明
1	NTH	INTEGER	序号
2	DICT_TYPE	VARCHAR(32)	对应字典类型
3	MIN_ID	INTEGER	起始 ID
4	MAX_ID	INTEGER	最大 ID
5	REAL_MAX_ID	INTEGER	已分配的最大 ID
6	N_ALLOC	INTEGER	已分配 ID 数

7	SCAN_START	INTEGER	下次分配时起始扫描 ID

29） V$IDPURGE

显示系统中已登记待延迟处理的事务 ID 信息，仅当 INI 参数 ID_RECYCLE_FLAG 为 1 时有效。

序号	列	数据类型	说明
1	TRID	BIGINT	事务号
2	ID_NUM	INTEGER	登记的字典 ID 数目
3	ARR_CNT	INTEGER	登记数组项数

30） V$HOTPAGE

显示缓存中的热点页信息。输出系统缓存中访问最频繁的前 1000 页信息，按 ACCESS_CNT 排序。

序号	列	数据类型	说明
1	TS_ID	INTEGER	所属的表空间 ID
2	FILE_ID	INTEGER	数据库文件 ID
3	PAGE_NO	INTEGER	页号
4	ACCESS_CNT	BIGINT	自启动以后页的访问计数
5	INDEXID	INTEGER	索引 ID，仅对索引/数据页有意义
6	PAGETYPE	VARCHAR(16)	页类型，取值如下： PSEG_MGR：回滚记录管理页 BLOB_REC_CTL：大字段记录管理页 BLOB_REC：大字段页 MTAB_PAGE：临时数据页 BLOB_PAGE：大字段页

			PSEG_SLOT_CTL：回滚段管理页 PSEG_UREC：回滚记录页 INDEX_INNER：索引内节点页 INDEX_LEAF：索引叶节点页 DATA_INNER：数据内节点页 DATA_LEAF：数据页节点页 TS_HDR：表空间/文件头页 INODE：段描述页 BTR_CONTROL：B 树控制页 EXTENT_DESC：簇描述页 OTHER：其他页
7	FIRST_LSN	BIGINT	缓存中首次修改时的 LSN，0 表示没有修改过
8	LSN	BIGINT	当前的 LSN，0 表示没有被修改过
9	POOL_ID	INTEGER	页所在的缓存池编号

31） V$OS_MEM_INFO

显示操作系统内存的分配释放信息。

序号	列	数据类型	说明
1	N_2_LOG	INTEGER	2 的 n 次方的 n
2	N_ALLOC	BIGINT	分配的次数
3	N_FREE	BIGINT	释放的次数

32） V$XBOX

获取并显示系统中所有 XBOX 的槽信息。

序号	列	数据类型	说明

1	ID	INTEGER	XBOX ID
2	SLOT_NO	INTEGER	XBOX 内的槽号
3	LETTER_NUM	INTEGER	槽内收到的邮件数
4	FILE_NAME	VARCHAR(256)	创建 XBOX 的文件名
5	FILE_LINE	INTEGER	创建 XBOX 的文件行号
6	EID	BIGINT	使用 XBOX 的 ESESS_ID
7	SIO	INTEGER	使用 XBOX 的 ERECV 操作符在计划中的行序号
8	STASK_NO	INTEGER	使用 XBOX 的 ERECV 操作符在计划中的子任务号
9	NO_IN_STASK	INTEGER	使用 XBOX 的 ERECV 操作符在子任务中的序号
10	DUMP_LEN_BYTES	BIGINT	XBOX 上使用临时文件保存的邮件总量，单位 BYTE
11	LONG_BYTES	BIGINT	槽内缓存的大于 32KB 的消息空间大小
12	SHORT_NUM	INTEGER	槽内缓存的 1KB 大小的消息数
13	MIDDLE_NUM	INTEGER	槽内缓存的 32KB 大小的消息数
14	LONG_NUM	INTEGER	槽内缓存的大于 32KB 大小的消息数
15	DUMP_NUM	INTEGER	槽内缓存的需转储的消息数

33） V$PREBUILD_PLNS

此视图获取并显示计划缓存池中所有预生成计划信息，在 ini 参数 USE_PLN_POOL !=0 且 ENABLE_PLN_PRESEARCH=1 时，才有预生成计划。

序号	列	数据类型	说明
1	SCHID	INTEGER	预生成计划模式 ID
2	UID	INTEGER	预生成计划用户 ID
3	SQLSTR	VARCHAR(4096)	预生成计划 SQL 语句内容
4	OBJID	INTEGER	若预生成计划是过程子计划，其对象 ID

5	N_WAIT	INTEGER	相同 SQL 正在等待的线程数
6	THRD_ID	INTEGER	创建预生成计划的线程 ID
7	MPP_LOGIN_TYPE	INTEGER	会话 MPP 登录类型： 0：全局登录； 1：本地登录，限制模式； 2：本地登录，非限制模式，可以进行 DDL 操作
8	MPP_DDL_FLAG	CHAR(1)	会话 MPP 下 DDL 标识
9	NLS_SORT_TYPE	INTEGER	会话执行计划的自然语言的排序方式
10	CASE_SENSITIVE	INTEGER	会话标识符大小写是否敏感
11	PARSE_TYPE	INTEGER	会话解析器类型
12	PARALLEL_POLICY	INTEGER	会话默认并行任务个数
13	LANGUAGE	INTEGER	会话语言

34） V$NOT_FREED_CACHEITEMS

此视图显示不在缓冲池中且未释放的缓存项信息，便于定位内存泄漏。在 INI 参数 CACHEITEM_ALLOC_MONITOR 非 0 时才统计对应的未释放缓存项。

其中 HEAP_ADDR 列即为视图 V$MEM_HEAP 的 ADDR 列，结合本视图可以查看计划堆的相关信息，如堆的创建文件名、创建文件行等，方便定位未释放计划的来源。计划、结果集的 SQLSTR 列均为其对应的 SQL 语句，包信息的 SQLSTR 列为其包名。

存在使用注意事项如下：

1. 此动态性能视图的主要目的为便于定位内存泄漏位置来确定泄漏原因，INI 参数 CACHEITEM_ALLOC_MONITOR 默认为 0，不开启统计功能。当 SQL 缓冲区有内存膨胀等异常现象时，若用户怀疑产生了内存泄漏，可以动态打开 CACHEITEM_ALLOC_MONITOR 参数来监控后续加入的缓存项，打开参数前已生成的缓存项无法记录。当查看完相关信息后，

需要及时关闭 CACHEITEM_ALLOC_MONITOR 参数，避免影响性能；关闭参数后，将不再记录后续缓存项，当前的记录会随缓存项释放自然淘汰。

2. 由于一般情况下，内存泄漏最有可能由计划缓存引起，建议将 INI 参数 CACHEITEM_ALLOC_MONITOR 设置为 1 或 3，用于收集监控不在缓冲池中未释放的计划缓存、包信息。因为 SQL 语句缓存一定加入缓冲池，所以不必记录。

序号	列	数据类型	说明
1	ADDRESS	BIGINT	未释放的缓存项的地址
2	TYPE$	VARCHAR(10)	未释放的缓存项的类型：计划（PLN），结果集（RS），包信息（PKG_INFO）
3	IN_POOL	CHAR(1)	未释放缓存项是否在 SQL 缓冲区中
4	DISABLED	CHAR(1)	未释放缓存项是否失效
5	HEAP_ADDR	BIGINT	未释放缓存项堆地址，即 V$MEM_HEAP 的 ADDR
6	SQLSTR	VARCHAR(4096)	未释放缓存项的 SQL 语句/包名
7	MEM_SIZE	BIGINT	未释放缓存项的缓存节点大小，单位为字节数

35） V$MEM_POOL_EXTEND_INFO

此视图显示内存池的扩展信息。视图中的每一行数据表示内存池中的一个扩展大块的信息。扩展大块可以进一步划分为扩展小块，扩展小块的信息请通过视图 V$MEM_POOL_EXTEND_BLOCK_INFO 获取。

序号	列	数据类型	说明
1	POOL_NAME	VARCHAR(33)	内存池名
2	POOL_ADDR	BIGINT	内存池地址
3	NTH	INTEGER	扩展大块的序号
4	SIZE	BIGINT	扩展大块的总大小，单位字节
5	USED_SIZE	BIGINT	扩展大块的已使用的大小，单位字节
6	ADDR	VARBINARY(8)	扩展大块的地址
7	BLOCK_COUNT	INTEGER	扩展大块包含的扩展小块的数量

8	USED_BLOCK	INTEGER	扩展大块包含的已使用扩展小块的数量

36) V$MEM_POOL_EXTEND_BLOCK_INFO

此视图显示内存池的各个扩展块的具体信息。视图中的每一行数据表示内存池的一个扩展大块的其中一个扩展小块的信息。

支持此动态视图的主要目的是便于用户定位内存中未释放的扩展内存位置，来确定扩展内存未释放原因。

查询该视图时，一定要使用带 WHERE 条件的 SQL 语句，并且同时指定 POOL_ADDR 和 NTH，且 WHERE 条件必须是等值条件。例如：SELECT * FROM V$MEM_POOL_EXTEND_BLOCK_INFO WHERE POOL_ADDR=197578644 AND NTH=1;

其中 POOL_ADDR 可通过视图 V$MEM_POOL_EXTEND_INFO 获取。

序号	列	数据类型	说明
1	POOL_NAME	VARCHAR(33)	内存池名
2	POOL_ADDR	BIGINT	内存池地址
3	NTH	INTEGER	扩展大块的序号
4	EXTEND_SIZE	BIGINT	扩展大块的总大小，单位字节
5	BLOCK_SIZE	BIGINT	扩展小块的大小，单位字节
6	BLOCK_ADDR	VARBINARY(8)	扩展小块的地址
7	IS_USED	CHAR	扩展小块的是否正在被使用
8	FILE_NAME	VARCHAR(256)	使用该扩展小块的文件名
9	FILE_LINE	INTEGER	使用该扩展小块的文件行号

37) V$MEM_FREE_LIST

此视图显示系统当前空闲链表和空闲对象的信息。

可以通过系统过程 SP_CLEAR_MEM_FREE_LIST 清理指定模块的可清理的空闲对象，或清理所有模块的可清理的空闲对象。

序号	列	数据类型	说明
1	MODULE_NAME	VARCHAR(128)	空闲链表所属模块名
2	FREE_OBJ_COUNT	INT	空闲对象数目

2. 段簇页

1) V$SEGMENT_INFOS

显示所有的段信息。

序号	列	数据类型	说明
1	TS_ID	INTEGER	表空间 ID
2	SEG_ID	INTEGER	段 ID
3	TYPE	VARCHAR(256)	段的类型（INNER 表示内节点段，LEAF 表示叶子节点段，LIST 表示堆表段，BLOB 表示 BLOB 段，SEQ 表示序列段，REP 表示复制段）
4	OBJ_ID	INTEGER	若段为内节点段或叶子节点段，则为索引 ID；若为 BLOB 段，则为 BLOB 所在的表 ID
5	INODE_FILE_ID	INTEGER	段的 inode 所在的页的文件号
6	INODE_PAGE_NO	INTEGER	段的 inode 所在的页的页号
7	INODE_OFFSET	INTEGER	段的 inode 所在的页的页内的偏移
8	N_FULL_EXTENT	INTEGER	段中的满簇数
9	N_FREE_EXTENT	INTEGER	段中的空闲簇数
10	N_FRAG_EXTENT	INTEGER	段中的半满簇数
11	N_FRAG_PAGE	INTEGER	段中的半满簇中总页数，为估算值

2） V$EXTENTS

显示所有的簇信息。

序号	列	数据类型	说明
1	TS_ID	INTEGER	表空间 ID
2	FILE_ID	INTEGER	文件 ID

3	SEG_ID	INTEGER	段 ID
4	EXTENT_ID	INTEGER	簇 ID
5	STATE	VARCHAR(8)	簇状态，其中 FULL 代表全满簇，FRAG 代表半满簇，FREE 代表空闲簇，NOUSE 代表未使用簇
6	USED	INTEGER	簇中已分配数据页个数

3） V$SEGMENTINFO

索引叶子段信息视图。查询该视图时，一定要带 WHERE 条件，并且必须是等值条件。同时，查询条件必须且只能指定 1 个，且只能是该视图的首列，即 INDEX_ID。

序号	列	数据类型	说明
1	INDEX_ID	INTEGER	索引 ID
2	SEG_ID	INTEGER	段 ID
3	LOGIC_PAGE_NO	INTEGER	逻辑页号
4	PHY_PAGE_NO	INTEGER	物理页号
5	PHY_FIRST_PAGE_IN_EXTENT	INTEGER	物理页所属的 extent 的第一个物理页
6	FIL_ID	INTEGER	文件 ID
7	USE_BYTES	BIGINT	已使用的字节数
8	FREE_BYTES	BIGINT	空闲的字节数

4） V$BTREE_INNER_PAGES/V$BTREE_LEAF_PAGES

索引的叶子段/内节点段的页信息视图。查询该视图时，一定要带 WHERE 条件，并且必须是等值条件。同时，查询条件必须且只能指定 1 个，且只能是该视图的首列，即 INDEX_ID。如：select * from v$btree_leaf_pages where index_id = *id*; （此处 *id* 表示需要根据实际情况替换为指定的索引 ID 值）

序号	列	数据类型	说明

1	INDEX_ID	INTEGER	索引 ID
2	LEVEL	INTEGER	页所在的层数
3	GROUP_ID	INTEGER	页的表空间号
4	FILE_ID	INTEGER	页的文件号
5	PAGE_NO	INTEGER	页的页号
6	PAGE_LSN	BIGINT	页的版本号
7	N_REC	INTEGER	页的记录数

注意：在 V$BTREE_INNER_PAGES 视图中，索引 BTREE 的控制页的 level 指定为-1。

5） V$LOB_SEG

用于查看各表大字段的段首页中记录的信息。

列号	列名	列数据类型	说明
1	SEG_ID	INTEGER	段 ID
2	TAB_ID	INTEGER	表 ID
3	USER_ID	INTEGER	用户 ID
4	N_EXTENT	BIGINT	占用簇的个数
5	N_PAGE	BIGINT	空闲页的个数

6） V$RESOURCE_LIMIT

显示表的空间限制信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	表名称
2	ID	INTEGER	表 ID
3	TYPE	VARCHAR(6)	表类型
4	SPACE_LIMIT	INTEGER	空间限制大小，以页为单位

5	SPACE_USED	BIGINT	空间实际使用大小，以页为单位

7) V$SEGMENT_PAGES

段中数据页的信息视图。查询该视图时，一定要带 WHERE 条件并且同时指定 GROUP_ID 和 SEG_ID，并且必须是等值条件。例如：select * from v$segment_pages where group_id=1 and seg_id = 200;

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	表空间 ID
2	SEG_ID	INTEGER	段编号
3	FILE_ID	INTEGER	页的文件号
4	PAGE_NO	INTEGER	页的页号

8) V$PSEG_SYS

显示当前回滚段信息。

序号	列	数据类型	说明
1	N_ITEM	INTEGER	系统中存在回滚项目的个数
2	EXTENT_SIZE	INTEGER	每次新申请的回滚页数目
3	ALLOC_PAGES	BIGINT	回滚页分配的空间大小，单位页数
4	EXTEND_NUM	BIGINT	回滚页分配的簇大小 ，单位页数
5	RECLAIM_PAGES	BIGINT	回滚段重新回收的页数
6	TAB_ITEMS	BIGINT	回滚段缓存表的个数（有 storage 选项不计）
7	TAB_HASH_SIZE	BIGINT	回滚段缓存的 HASH 表中节点的个数
8	OBJ_COUNT	BIGINT	purge 涉及对象的个数
9	OBJ_HASH_SIZE	BIGINT	purge 涉及对象的 HASH SIZE（哈希表中元素个数）

9） V$PSEG_ITEMS

显示回滚系统中当前回滚项信息。

序号	列	数据类型	说明
1	NTH	INTEGER	ITEM 序号
2	N_PAGES	INTEGER	回滚页数
3	N_EXTEND	INTEGER	扩展回滚段次数
4	N_PURGED_PAGES	INTEGER	已 PURGE 的页数
5	N_USED_PAGES	INTEGER	正在使用的页数
6	N_UREC_BYTES	BIGINT	生成回滚记录的总字节数
7	N_COMMIT_TRX	INTEGER	已经提交未 PURGE 的事务数
8	RESERVE_TIME	INTEGER	事务提交后，最长的保留时间，单位秒
9	STACK_SIZE	INTEGER	空闲回滚页堆栈空间大小，单位页数
10	STACK_SP	INTEGER	空闲回滚页堆栈中空闲页数
11	MAX_COMMIT_TRX	INTEGER	已提交未 PURGE 的最大事务数
12	N_FREE_NODE	INTEGER	提交队列的空闲个数
13	N_QUEUE_ITEM	INTEGER	队列项的个数，每个项管理一组提交队列
14	FIRST_COMMIT_TIME	INTEGER	第一个提交事务的时间到当前时间的间隔，单位：秒
15	N_PURGING_TRX	INTEGER	正在 PURGE 的事务数
16	N_TRX	BIGINT	申请 ITEM 对应的事务数
17	TUNED_UNDO_RETENTION	DOUBLE	调整后的回滚页保留时长，单位秒
18	N_PURGE_DEL_PAGE_DIRECT	BIGINT	PURGE 时根据记录地址直接数据页内定位的次

			数
19	N_PURGE_DEL_PAGE_SEARCH	BIGINT	PURGE 时根据记录地址在数据页内查找的次数
20	N_PURGE_DEL_BTR_SEARCH	BIGINT	PURGE 时通过 B 树进行查找的次数

10） V$PSEG_COMMIT_TRX

显示回滚项中已提交但未 PURGE 的事务信息。

序号	列	数据类型	说明
1	ITEM_NTH	INTEGER	隶属回滚项的序号
2	TRX_ID	BIGINT	事务 ID
3	CMT_TIME	TIMESTAMP(6)	事务提交时间
4	FPA_FILE_ID	INTEGER	起始页地址文件号。-1 表示该事务没有涉及回滚记录的删除与更新，在事务提交时会直接释放掉涉及的所有回滚页，并清理事务对应的回滚页统计信息，所以回滚文件号为空
5	FPA_PAGE_NO	INTEGER	起始页地址页 ID。-1 表示该事务没有涉及回滚记录的删除与更新，在事务提交时会直接释放掉涉及的所有回滚页，并清理事务对应的回滚页统计信息，所以回滚页为空
6	N_PAGES	INTEGER	事务实际使用的回滚页数量
7	FPA_OFFSET	INTEGER	回滚记录在起始页内的起始偏移位置。-1 表示该事务没有修改数据，所以没有回滚记录
8	NEED_PURGE	INTEGER	是否需要 purge：1 表示需要，0 表示不需要
9	CRASH_FLAG	INTEGER	是否为故障前提交的事务：1 表示是，0 表示不是

11） V$PURGE

显示当前 PURGE 回滚段信息。

序号	列	数据类型	说明
1	OBJ_NUM	INTEGER	待 PURGE 事务个数
2	IS_RUNNING	CHAR(1)	是否有 PURGE 线程正在执行清理回滚段任务(Y/N)
3	PURG_FOR_TS	CHAR(1)	是否正在 PURGE 表空间（Y/N）

12） V$PURGE_PSEG_OBJ

显示 PURGE 系统中，待 PURGE 的所有 PSEG 对象信息。

序号	列	数据类型	说明
1	TRX_ID	BIGINT	事务 ID
2	NEED_PURGE	CHAR(1)	是否需要 PURGE（Y/N）
3	TAB_NUM	INTEGER	涉及表数

13） V$PURGE_PSEG_TAB

显示待 PURGE 表信息。

序号	列	数据类型	说明
1	TRX_ID	BIGINT	事务 ID
2	TAB_ID	INTEGER	表 ID
3	GROUP_ID	INTEGER	控制页所在表空间 ID
4	FILE_ID	INTEGER	控制页文件 ID
5	PAGE_NO	INTEGER	控制页编号
6	TAB_TYPE	INTEGER	表类型：0 普通表、1 全局临时表、6 范围分区表主表、7 范围分区表子表、8HASH 分区表主表、9HASH 分区表子表、10 位图连接索引表、11LIST 分区表主表、12LIST 分区表子表、13 外部表、15 DBLINK 远程表、24HUGE 表范围分区表主表、25HUGE

			表范围分区表子表、26HUGE 表 HASH 分区表主表、27HUGE 表 HASH 分区表子表、28HUGE 表 LIST 分区表主表、29HUGE 表 LIST 分区表子表、32 位图索引表、33HUGE 表
7	ROW_COUNT	BIGINT	insert/delete 操作影响表行数。每插入一行加 1，每删除一行减 1

14) V$TMP_FREE_EXTENTS

显示开启临时表簇管理优化策略后，存储在全局 FREE_EXTENTS 堆栈中的空闲簇信息。

序号	列	数据类型	说明
1	NTH	INTEGER	堆栈序号
2	FREE_EXTENTS_NUM	INTEGER	当前堆栈中保存的空闲簇数目
3	GLOBAL_EXTENT_SIZE	INTEGER	一个空闲簇所占用的内存大小，单位 KB
4	FREE_EXTENTS_SIZE	BIGINT	当前堆栈中保存的空闲簇所占用的内存空间，单位 KB

15) V$PURGING_TRX

显示当前正在 PURGE 的事务信息。

序号	列	数据类型	说明
1	ITEM_NTH	INTEGER	隶属回滚项的序号
2	TRX_ID	BIGINT	事务 ID
3	CMT_TIME	TIMESTAMP(6)	事务提交时间
4	FPA_FILE_ID	INTEGER	起始页地址文件号。-1 表示该事务没有涉及回滚记录的删除与更新，在事务提交时会直接释放掉涉及的所有回滚页，并清理事务对应的回滚页统计信息，所以回滚文件号为空
5	FPA_PAGE_NO	INTEGER	起始页地址页 ID。-1 表示该事务没有涉及回滚记录的删除与更新，在事务提交时会直接释放掉涉及的所有回滚页，并清理事务对应的回滚页统计信息，所以回滚页为空
6	N_PAGES	INTEGER	事务实际使用的回滚页数量

7	CRASH_FLAG	INTEGER	是否为故障前提交的事务：1 表示是，0 表示不是
8	PURGE_STIME	TIMESTAMP(6)	事务开始 purge 的时间
9	N_PAGE_PURGED	INTEGER	事务已经 purge 掉的回滚页数量

3. 数据库信息

1) V$LICENSE

显示 LICENSE 信息，用来查询当前系统的 LICENSE 信息。

序号	列	数据类型	说明
1	LIC_VERSION	VARCHAR(256)	许可证版本号
2	SERIES_NO	VARCHAR(256)	LICENSE 文件序列号
3	SERVER_SERIES	VARCHAR(256)	服务器版本类型。P：单机版、S：标准版、E：企业版、A：安全版、T:可信版、X：定制版
4	SERVER_TYPE	VARCHAR(256)	服务器发布类型。1：正式版、2：测试版、3：试用版、4：其他
5	SERVER_VER	VARCHAR(256)	服务器版本号
6	EXPIRED_DATE	DATE	有效日期
7	AUTHORIZED_CUSTOMER	VARCHAR(256)	用户名称
8	AUTHORIZED_USER_NUMBER	BIGINT	授权用户数
9	CONCURRENCY_USER_NUMBER	BIGINT	并发连接数
10	MAX_CPU_NUM	INTEGER	最大 CPU 数目
11	NOACTIVE_DEADLINE	DATE	未激活状态截止日期
12	HARDWARE_ID	VARCHAR(256)	绑定的硬件编码，根据 HARDWARE_TYPE

			不同，此项内容对应的获取和检测方式不同
13	CHECK_CODE	VARCHAR(16)	校验码
14	PRODUCT_TYPE	VARCHAR(8)	产品类型：内容为：DM8/DM7/DM6/DMETLV4/DMETLV3/DMHSV3
15	PROJECT_NAME	VARCHAR(128)	项目名称
16	CPU_TYPE	VARCHAR(24)	授权运行的 CPU 类型
17	OS_TYPE	VARCHAR(24)	授权运行的操作系统
18	MAX_CORE_NUM	INTEGER	授权最大 CPU 核个数，0 表示无限制
19	HARDWARE_TYPE	VARCHAR(24)	硬件绑定类型。1：MAC 地址、 2：cpu id、 3：harddriver id
20	CLUSTER_TYPE	VARCHAR(24)	授权使用的集群类型，格式为字符串“XXXX”，每一位上 0 表示禁止，1 表示授权使用 第 1 个字符：表示数据守护 第 2 个字符：表示 MPP 第 3 个字符：表示读写分离 第 4 个字符：表示 DSC 例如：“0010”表示授权该可作为读写分离集群的节点使用
21	DATE_GEN	DATE	KEY 的生成日期

22	SERVER_SERIES_NAME	VARCHAR(128)	定制版版本名称
23	TABLE_RECORD_NUMBER_LIMIT	INTEGER	单表最大数据行数限制，以万为单位，0 表示无限制
24	TOTAL_SPACE_LIMIT	INTEGER	数据总空间大小限制，以 GB 为单位，0 表示无限制

2） V$VERSION

显示版本信息。

序号	列	数据类型	说明
1	BANNER	VARCHAR(80)	版本标识。 DM Database Server 64/空：服务器版本号。其中，64/空表示 64/32 操作系统。 DB Version：数据库版本号。 DMDSC Version：DMDSC 版本号。 Msg Version：服务器通信版本号。 Gsu level(5) cnt：服务器 5 级不兼容修改版本计数

3） GV$VERSION

显示版本信息，包括其他 raft 节点、实时归档目标、即时归档目标、dsc 其他节点。

序号	列	数据类型	说明
1	INSTANCE_NAME	VARCHAR(128)	数据库实例名
2	BANNER	VARCHAR(128)	版本标识。 DM Database Server [64]：服务器版本号。

			DB Version：库版本号。 DSC Version：DMDSC 版本号。 Msg Version：服务器通信版本号。 Gsu level(5) cnt：服务器 5 级不兼容修改版本计数

4） V$DATAFILE

显示数据文件信息，不包括 RLOG 表空间信息。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	所属的表空间 ID
2	ID	INTEGER	数据库文件 ID
3	PATH	VARCHAR(256)	数据库文件路径
4	CLIENT_PATH	VARCHAR(256)	数据库文件路径，专门提供给客户端
5	CREATE_TIME	DATETIME(0)	数据库文件创建时间
6	STATUS$	TINYINT	状态，标识数据文件当前的状态，取值 0/1，取值为 0 表示数据文件状态异常（被删除或者处于恢复中），取值为 1 表示数据文件状态正常 对于 windows 平台，正在使用的文件不支持直接在操作系统中删除，因此 windows 平台下，status 取值恒为 1 对于其他平台，当数据库检查到数据文件被删除或者数据文件正在执行恢复，则 status 取值为 0，否则取值为 1
7	RW_STATUS	TINYINT	读写状态：1 读，2 写
8	LAST_CKPT_TIME	DATETIME(0)	最后一次检查点时间。只对 online 的 DB 做统计，否则就是空

9	MODIFY_TIME	DATETIME(0)	文件修改时间
10	MODIFY_TRX	BIGINT	修改事务
11	TOTAL_SIZE	BIGINT	总大小。单位：页数
12	FREE_SIZE	BIGINT	未分配的空闲大小，单位：页数
13	FREE_PAGE_NO	BIGINT	数据文件中尚未格式化的最小页号。文件系统以簇为单位格式化数据页，数据文件创建时首先格式化第一个簇的数据页，后续分配使用数据页时再继续格式化数据页，数据页一旦被格式化便无法再恢复到未被格式化时的状态
14	PAGES_READ	BIGINT	累计读页数量
15	PAGES_WRITE	BIGINT	累计写页数量
16	PAGE_SIZE	INTEGER	页大小，单位 BYTE
17	READ_REQUEST	INTEGER	累计读请求次数
18	WRITE_REQUEST	INTEGER	累计写请求次数
19	AUTO_EXTEND	INTEGER	是否支持自动扩展：1 支持，0 不支持
20	MAX_SIZE	INTEGER	文件最大大小，单位 MB
21	NEXT_SIZE	INTEGER	文件每次扩展大小，单位 MB，若创建文件时没有指定，则由 INI 参数 TS_AUTO_EXTEND_SIZE 决定
22	MIRROR_PATH	VARCHAR(256)	镜像文件路径
23	REAL_FREE_SIZE	BIGINT	未分配的空闲大小，单位：页数

5) V$DATABASE

显示数据库信息。

序号	列	数据类型	说明

1	NAME	VARCHAR(128)	数据库名称
2	CREATE_TIME	DATETIME(0)	数据库创建时间
3	ARCH_MODE	CHAR(1)	归档模式：归档或不归档。默认为不归档
4	LAST_CKPT_TIME	DATETIME(0)	最后一次检查点时间。只对 ONLINE 的 DB 做统计，否则就是空
5	STATUS$	TINYINT	状态。1：启动；2：启动，redo 完成；3：MOUNT；4：打开；5：挂起；6：关闭
6	ROLE$	TINYINT	角色。0：普通；1：主库；2：备库
7	MAX_SIZE	BIGINT	最大大小，单位：页数。0 代表只受操作系统限制
8	TOTAL_SIZE	BIGINT	除回滚表空间外所有表空间的文件大小之和，单位：页数
9	DSC_NODES	INTEGER	DSC集群系统中的实例总数
10	OPEN_COUNT	INTEGER	数据库open次数
11	STARTUP_COUNT	BIGINT	数据库启动次数
12	LAST_STARTUP_TIME	DATETIME(0)	数据库最近一次启动时间

6) V$IID

显示下一个创建的数据库对象的 ID。该视图提供用户可以查询下一个创建对象的 ID 的值，可以方便用户查询预知自己所要建立对象的信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(100)	对象名称
2	VALUE	BIGINT	对象值

7) V$INSTANCE

显示实例信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	实例名称
2	INSTANCE_NAME	VARCHAR(128)	实例名称
3	INSTANCE_NUMBER	INTEGER	实例ID，单节点默认1，MPP或DSC环境下为实例序号加1
4	HOST_NAME	VARCHAR(128)	主机名称
5	SVR_VERSION	VARCHAR(128)	服务器版本号。详见 V$VERSION 介绍
6	DB_VERSION	VARCHAR(40)	数据库版本号
7	START_TIME	DATETIME(0)	服务器启动时间
8	STATUS$	VARCHAR(128)	系统状态
9	MODE$	VARCHAR(128)	模式
10	OGUID	INTEGER	控制文件的 OGUID
11	DSC_SEQNO	INTEGER	DSC 序号
12	DSC_ROLE	VARCHAR(32)	非 DSC 环境显示 NULL；DSC 环境下表示 DSC 系统角色：Control node（控制节点）/Normal node（普通节点）
13	BUILD_VERSION	VARCHAR(128)	编译版本号
14	BUILD_TIME	VARCHAR(32)	编译时间

8） GV$INSTANCE

显示全局实例信息。在 DPC 环境的备机上查询该视图，将只返回 RAFT 内的主备信息。

GV$INSTANCE 结构和 V$INSTANCE 结构一样。

9） V$RESERVED_WORDS

显示关键字和保留字的分类信息。

对于 RESERVED=Y 且 RES_FIXED=N 的保留字，可通过 ini 参数 EXCLUDE_RESERVED_WORDS 去除保留字身份。去除后的保留字将变为普通字符串，本视图不再记录。例如：若想去除保留字 AUTO_INCREMENT，可设置 EXCLUDE_RESERVED_WORDS=AUTO_INCREMENT，重启服务器后，查询此视图时，则不会出现保留字 AUTO_INCREMENT 的相关信息。

序号	列	数据类型	说明
1	KEYWORD	VARCHAR(30)	关键字名字
2	LENGTH	INTEGER	关键字长度
3	RESERVED	VARCHAR(1)	是否是保留字。Y 是； N 否
4	RES_SQL	VARCHAR(1)	是否是 SQL 保留字。Y 是； N 否。是则不能作 SQL 中的标识符
5	RES_PL	VARCHAR(1)	是否是 DMSQL 程序保留字。Y 是； N 否。是则不能作 DMSQL 程序语句块中的标识符
6	RES_SCHEMA	VARCHAR(1)	是否是模式保留字。Y 是； N 否。是则不能作模式标识符
7	RES_VARIABLE	VARCHAR(1)	是否是变量保留字。Y 是； N 否。是则不能作变量标识符
8	RES_ALIAS	VARCHAR(1)	是否是别名保留字。Y 是； N 否。是则不能作别名标识符
9	RES_FIXED	VARCHAR(1)	表示当前保留字是否被系统预定。Y 是，N 否。只有 RES_FIXED 为 N 时，才可以通过 ini 参数 EXCLUDE_RESERVED_WORDS 去除保留字身份

10) V$ERR_INFO

显示系统中的错误码信息。

序号	列	数据类型	说明
1	CODE	INTEGER	错误码

2	ERRINFO	VARCHAR(512)	错误码描述

11) V$HINT_INI_INFO

显示支持的 HINT 参数信息。

序号	列	数据类型	说明
1	PARA_NAME	VARCHAR(128)	参数名称
2	HINT_TYPE	VARCHAR(16)	HINT 类型，OPT：优化分析阶段使用；EXEC：执行阶段使用

12) V$DYNAMIC_TABLES

显示当前数据库中的动态视图。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	动态视图名称
2	ID	INTEGER	动态视图 ID
3	SCHNAME	VARCHAR(128)	动态视图所属模式名
4	SYNONYMS	VARCHAR(512)	动态视图的其他同义词，若有多个用逗号分隔，没有为 NULL

13) V$DYNAMIC_TABLE_COLUMNS

显示当前数据库中所有动态视图的列信息。

序号	列	数据类型	说明
1	TABNAME	VARCHAR(128)	列所属动态视图名称
2	COLNAME	VARCHAR(128)	列名
3	COLID	SMALLINT	列 ID
4	TYPE$	VARCHAR(128)	列数据类型
5	LENGTH$	INTEGER	列定义长度
6	SCALE	SMALLINT	列定义精度
7	NULLABLE$	CHAR(1)	是否可以为空，Y:是，N：否

8	DEFVAL	VARCHAR(2048)	默认值

14) V$AUTHORITIES

显示当前数据库中的所有权限信息。

序号	列	数据类型	说明
1	ID	INTEGER	权限 ID
2	NAME	VARCHAR(256)	权限名称
3	TYPE	INTEGER	权限类型。1 表示数据库权限；2 表示对象权限；3 表示黑名单权限。黑名单权限表示关闭指定的对象权限

4. 数据库对象相关

数据库对象包括：表空间、序列、包、索引和函数等。

1) V$TABLESPACE

显示表空间信息，不包括 RLOG 表空间信息。

在集群环境中，V$TABLESPACE会汇总所有站点的表空间信息。在集群环境中，为了区分不同站点的表空间信息，建议使用select SF_GET_PARA_STRING_VALUE(1, 'INSTANCE_NAME'),* from V$TABLESPACE;加入实例名称一起查看。

序号	列	数据类型	说明
1	ID	INTEGER	表空间 ID
2	NAME	VARCHAR(128)	表空间名称
3	CACHE	VARCHAR(20)	CACHE 名，早期用于区分表空间使用的缓冲区。后因系统调整，实际已无意义，目前为空或 NORMAL

4	TYPE$	TINYINT	表空间类型：1 DB 类型，2 临时表空间
5	STATUS$	TINYINT	状态。0 ONLINE，1 OFFLINE，2 RES_OFFLINE 3 CORRUPT
6	MAX_SIZE	BIGINT	最大大小，以页为单位。0 代表只受操作系统限制（暂无实际意义）
7	TOTAL_SIZE	BIGINT	总大小，以页为单位
8	FILE_NUM	INTEGER	包含的文件数
9	ENCRYPT_NAME	VARCHAR(128)	加密算法名
10	ENCRYPTED_KEY	VARCHAR(500)	暂无意义，均显示 NULL
11	COPY_NUM	INTEGER	保留字段
12	SIZE_MODE	VARCHAR(128)	保留字段
13	OPT_NODE	INTEGER	指定的 DSC 集群表空间负载均衡优化节点号
14	USED_SIZE	BIGINT	表示已使用的簇所占的大小，以页为单位
15	SPACE_LIMIT_CNT	BIGINT	表示达到表空间限制的次数
16	EXTEND_FAILED_CNT	BIGINT	扩展失败的次数
17	N_RESERVE_EXTENTS	INTEGER	期望保留的簇个数
18	FREE_EXTENTS	INTEGER	当前表空间剩余空间簇的个数
19	MDF_FREE_COUNT	INTEGER	执行 MODIFY FREE EXTENTS 的次数，每 100 次重新计算一次，保证 FREE_EXTENTS 准确

2) V$HUGE_TABLESPACE

显示混合表空间的 HUGE 数据文件副本策略和路径统计信息。

序号	列	数据类型	说明
1	ID	INTEGER	混合表空间 ID

2	NAME	VARCHAR(128)	混合表空间名称
3	PATHNAME	VARCHAR(256)	混合表空间的 HUGE 数据文件路径
4	DIR_NUM	INTEGER	混合表空间的 HUGE 数据文件路径数
5	COPY_NUM	INTEGER	保留字段
6	SIZE_MODE	VARCHAR(128)	保留字段

3) V$HUGE_TABLESPACE_PATH

显示混合表空间的 HUGE 数据文件路径信息。

序号	列	数据类型	说明
1	ID	INTEGER	混合表空间 ID
2	PATHNAME	VARCHAR(256)	混合表空间的 HUGE 数据文件路径

4) V$SEQCACHE

显示当前系统中缓存的序列的信息。

序号	列	数据类型	说明
1	ID	INTEGER	序列 ID
2	NAME	VARCHAR(128)	序列名
3	NEXTVAL	DEC(38)	序列的值
4	CACHE_NUM	INTEGER	创建序列时指定的缓存值的个数
5	N_USED	INTEGER	已经使用的缓存值的个数
6	OVERFLOW	INTEGER	当前序列是否已溢出的标识

5) V$PKGPROCS

显示当前用户拥有执行权限的包中的方法。

序号	列	数据类型	说明
1	PKGID	INTEGER	包 ID

2	PKGNAME	VARCHAR(128)	包名
3	MTDID	INTEGER	方法 ID，每个包中方法从 0 开始编号
4	MTDNAME	VARCHAR(128)	方法名
5	IS_PROC	CHAR(1)	是否是过程，'Y'表示是过程，'N'表示是函数

6） V$PKGPROCINFOS

显示当前用户拥有执行权限的包中方法的相关信息。

序号	列	数据类型	说明
1	OWNER	VARCHAR(128)	包所属用户名
2	OBJECT_NAME	VARCHAR(128)	包名
3	PROCEDURE_NAME	VARCHAR(128)	方法名
4	OBJECT_ID	INTEGER	包 ID
5	SUBPROGRAM_ID	INTEGER	方法 ID，每个包中方法从 1 开始编号
6	OVERLOAD	INTEGER	方法在包中第几个重载
7	OBJECT_TYPE	VARCHAR(7)	对象类型，始终为 PACKAGE
8	AGGREGATE	VARCHAR(10)	是否是聚合函数，兼容保留字段
9	PIPELINED	VARCHAR(10)	是否是管道化表函数，'YES'：是；'NO'：否
10	IMPLTYPEOWNER	VARCHAR(128)	实现类所属的模式名，兼容保留字段
11	IMPLTYPENAME	VARCHAR(128)	实现类名，兼容保留字段
12	PARALLEL	VARCHAR(10)	是否可以并行，'YES'：是；'NO'：否
13	INTERFACE	VARCHAR(10)	是否是使用 DCI 接口实现的表函数，兼容保留字段
14	DETERMINISTIC	VARCHAR(10)	是否是确定性函数，'YES'：是；'NO'：否
15	AUTHID	VARCHAR(12)	标识定义者权限和调用者权限，DEFINER：定义者权限；

			CURRENT_USER：调用者权限
16	IS_PRIV	INTEGER	当前用户是否具有执行权限，1：是；0：否

7） V$PKGPROCPARAMS

显示当前用户拥有执行权限的包中方法的参数信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	参数名
2	PKGID	INTEGER	包 ID
3	MTDID	INTEGER	方法 ID
4	PARAMID	SMALLINT	参数 ID，每个方法的参数从 1 开始编号，0 表示返回值
5	TYPE$	VARCHAR(128)	参数类型
6	LENGTH$	INTEGER	参数长度
7	SCALE	SMALLINT	参数精度
8	NULLABLE	CHAR(1)	参数是否可为空
9	IOFLAG	CHAR(2)	参数输入输出标记，'I'表示输入，'O'表示输出，'IO'表示输入输出，'R'表示返回值
10	DEFVAL	VARCHAR(2000)	参数缺省值

8） V$DB_CACHE

数据字典缓存表，用于记录数据字典的实时信息。

序号	列	数据类型	说明
1	DB_ADDR	VARBINARY(8)	数据字典地址
2	POOL_ID	INTEGER	缓存池 ID
3	TOTAL_SIZE	BIGINT	缓存池总空间，单位字节

4	USED_SIZE	BIGINT	实际使用的空间，单位字节
5	DICT_NUM	INTEGER	缓存池中字典对象的总数
6	SIZE_LRU_DISCARD	BIGINT	由于缓存池已满导致字典对象被淘汰的空间的总和，单位字节
7	LRU_DISCARD	INTEGER	由于缓存池已满导致字典对象被淘汰的次数
8	DDL_DISCARD	INTEGER	DDL 操作导致字典对象被淘汰的次数
9	DISABLED_SIZE	BIGINT	被淘汰字典对象的空间，单位字节
10	DISABLED_DICT_NUM	INTEGER	缓存池中被淘汰字典对象的总数

9） V$DB_OBJECT_CACHE

数据字典对象缓存表，用于记录数据字典中每个对象的信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(32)	字典对象类型
2	ID	INTEGER	字典对象的 ID
3	NAME	VARCHAR(128)	字典对象的名称
4	SCHID	INTEGER	所在模式的 ID
5	PID	INTEGER	所属对象的 ID
6	STATUS	VARCHAR(16)	状态
7	T_SIZE	INTEGER	对象结构所占空间，单位字节
8	R_SIZE	INTEGER	对象在缓存中实际的空间，单位字节
9	VERSION	INTEGER	对象的版本号
10	ID_HASH	VARBINARY(8)	对象 ID 的 HASH 值
11	NAME_HASH	VARBINARY(8)	对象 NAME 的 HASH 值
12	LOAD_TIME	DATETIME(6)	对象被加载的时间

13	N_DECODE	INTEGER	对象 DECODE 的次数
14	N_SEARCH	INTEGER	对象 SEARCH 的次数

10) V$OBJECT_USAGE

记录索引监控信息。

序号	列	数据类型	说明
1	INDEX_NAME	VARCHAR(128)	索引名
2	SCH_NAME	VARCHAR(128)	模式名
3	TABLE_NAME	VARCHAR(128)	表名
4	MONITORING	VARCHAR(3)	是否被监控(YES\NO)，仅在 MONITOR_INDEX_FLAG=0 时有效
5	USED	VARCHAR(3)	是否被使用(YES\NO)
6	START_MONITORING	VARCHAR(19)	开始监控时间，仅在 MONITOR_INDEX_FLAG=0 时有效
7	END_MONITORING	VARCHAR(19)	停止监控时间，仅在 MONITOR_INDEX_FLAG=0 时有效

11) V$IFUN

显示数据库提供的所有函数。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	函数名
2	ID	INTEGER	函数 ID
3	ARG_NUM	INTEGER	参数个数
4	HAS_RT_FACT	CHAR(1)	是否存在运行时因素，'Y'：是；'N'：否

5	IS_READONLY	CHAR(1)	是否只读
6	IS_MPP_BRO	CHAR(1)	MPP 下是否需要广播
7	IS_MPP_FORBIDEN	CHAR(1)	MPP 下是否禁用该函数
8	IS_MOUNT_ONLY	CHAR(1)	系统 MOUNT 状态下是否可以使用该函数
9	IS_INCLUDE_SQL	CHAR(1)	函数中是否会执行 SQL 语句
10	EXTRA_ATTR	VARCHAR(1024)	预留字段，无实际意义，全为 NULL
11	CLASS$	VARCHAR(128)	函数所属的类别
12	COMMENT$	VARCHAR(1024)	预留字段，无实际意义，全为 NULL
13	IS_LPQ_FORBIDDEN	CHAR(1)	此函数是否禁止并行执行

12) V$IFUN_ARG

显示数据库提供的所有函数的参数。

序号	列	数据类型	说明
1	ID	INTEGER	函数 ID
2	NAME	VARCHAR(128)	函数参数名
3	SEQ	INTEGER	参数序号，从 0 开始依次递增，-1 表示返回值
4	DATA_TYPE	VARCHAR(64)	参数数据类型
5	LEN	INTEGER	参数数据长度
6	PREC	INTEGER	参数数据精度
7	IO_TYPE	VARCHAR(8)	参数输入输出类型
8	COMMENT$	VARCHAR(1024)	参数说明。预留字段，无实际意义，全为 NULL。

13) V$SYSSTAT

显示系统中统计对象的信息，支持通过系统过程 SP_DYNAMIC_VIEW_DATA_CLEAR 来清空此视图的历史信息。

序号	列	数据类型	说明
1	ID	INTEGER	统计对象 ID
2	CLASSID	INTEGER	统计对象所属类别 ID。1：字典信息；2：SQL；3：事务； 4：检查点；5：RLOG；6：UNDO；7：IO；8：B 树；9：网络；10：文件；11：内存；12：CPU；13：OS；14：缓冲区；15：限流控制；16：DMDPC；20：其它
3	NAME	VARCHAR(128)	统计对象名
4	STAT_VAL	BIGINT	统计值

14) V$JOBS_RUNNING

显示系统中正在执行的作业信息。

序号	列	数据类型	说明
1	SID	BIGINT	作业执行的 SESSION 的 ID
2	JOBID	INTEGER	作业号
3	FAILURES	INTEGER	作业自上一次成功以来的失败次数，暂不支持
4	LAST_DATE	DATETIME(6)	最后一次成功运行工作的时间
5	THIS_DATE	DATETIME(6)	本次运行的开始时间
6	INSTANCE	INTEGER	本次运行所在实例的序号

15) V$AUTO_STAT_TABLE_IDU

记录用户表中增删改数据的实时变化信息。在 INI 参数 AUTO_STAT_OBJ =1 或 2 时才统计。

序号	列	数据类型	说明

1	ID	INTEGER	表 ID
2	TOTAL_ROWS	BIGINT	表中数据的总行数。TOTAL_ROWS 为系统启动时的总行数，不是实时数据。只有发生统计的时候，TOTAL_ROWS 才会更新
3	INSERT_ROWS	BIGINT	插入数据的行数，为实时数据
4	DELETE_ROWS	BIGINT	删除数据的行数，为实时数据
5	UPDATE_ROWS	BIGINT	更新数据的行数，为实时数据

16) V$HFILE

显示当前系统打开的 HUGE 表列中存储的文件信息。

序号	列	数据类型	说明
1	TSID	INTEGER	HUGE 表所属表空间 ID
2	SCHID	INTEGER	HUGE 表所属模式 ID
3	TABLEID	INTEGER	HUGE 表 ID
4	COLID	SMALLINT	列 ID
5	FILEID	INTEGER	列中存储的文件 ID
6	HANDLE	BIGINT	文件句柄
7	N_FIX	INTEGER	引用计数，表示有 N_FIX 处正在使用该文件

17) V$POLICY_CACHE

记录缓存中的对象策略信息。

序号	列	数据类型	说明
1	ID	INTEGER	对象 ID
2	SCHID	INTEGER	对象所在模式 ID
3	CONTEXT_NUM	INTEGER	对象关联的上下文数目
4	GROUP_NUM	INTEGER	对象关联的策略组数目

18）V$POLICY_CACHE_ITEM

记录指定对象的具体策略信息。查询该视图时，一定要带 WHERE 条件。同时，查询条件必须且只能指定 1 个，且只能是该视图的首列，即 ID。

序号	列	数据类型	说明
1	ID	INTEGER	对象 ID
2	SCHID	INTEGER	对象所在模式 ID
3	CONTEXT	VARCHAR(128)	上下文名
4	ATTRIBUTE	VARCHAR(128)	上下文属性值
5	GROUP	VARCHAR(128)	策略组名
6	POLICY	VARCHAR(128)	策略组内的策略名

19）V$SYSINFO

显示系统中全局量的信息。

序号	列	数据类型	说明
1	ID	INTEGER	全局量 ID
2	NAME	VARCHAR(128)	全局量名
3	INFO	VARCHAR(2048)	全局量值

20）V$IO_TIMEOUT

监控耗时较长的 IO 操作。仅保留最新的 10000 条记录。

TYPE 为 RLOG 时，GROUP_ID、FILE_ID、PAGE_NO 为 NULL，记录 IO 延迟超过 1 秒的记录。DSC 环境下，TYPE 为 RLOG 时，记录 IO 延迟超过 IO_TIMEOUT/2 的记录，并且会根据 DSC_SLOT_WAIT_TIMEOUT 调整，但保证最小为 15 秒。

TYPE 为 PAGE 时，记录 IO 延迟超过 3 秒的记录。

序号	列	数据类型	说明

1	TYPE	VARCHAR(8)	PAGE/RLOG
2	CHECK_TIME	TIMESTAMP(0)	检查时间
3	WAIT_TIME	INT	IO 延迟时间，单位：秒
4	TIMEOUT	INT	超时时间，单位：秒
5	GROUP_ID	INT	页的表空间号
6	FILE_ID	INT	页的文件号
7	PAGE_NO	INT	页号

5. 数据库配置参数

1) V$PARAMETER

显示 ini 参数和 dminit 建库参数的类型及参数值信息（当前会话值、系统值及 dm.ini 文件中的值）。

序号	字段	类型	说明
1	ID	INTEGER	ID 号
2	NAME	VARCHAR(80)	参数名字
3	TYPE	VARCHAR(200)	参数类型。 READ ONLY：手动参数，表示服务器运行过程中不可修改； IN FILE：静态参数，只可修改 ini 文件； SYS 和 SESSION：动态参数，ini 文件和内存同时可修改，其中，SYS 为系统级参数；SESSION 为会话级参数
4	VALUE	VARCHAR(4000)	参数的值（当前会话）
5	SYS_VALUE	VARCHAR(4000)	参数的值（系统）

6	FILE_VALUE	VARCHAR(4000)	参数的值（ini 文件）
7	DESCRIPTION	VARCHAR2(255)	参数描述
8	DEFAULT_VALUE	VARCHAR(256)	参数默认值
9	ISDEFAULT	INTEGER	dm.ini 中参数的值和默认值是否相同，1 表示相同，0 表示不同。系统默认初始化后，某些静态建库参数在系统启动过程中会根据环境自动调整数值，导致与默认值不同

2） V$DM_INI

显示 ini 参数和 dminit 建库参数信息。

序号	列	数据类型	说明
1	PARA_NAME	VARCHAR (128)	参数名称
2	PARA_VALUE	VARCHAR (256)	系统参数值
3	MIN_VALUE	VARCHAR (256)	最小值
4	MAX_VALUE	VARCHAR (256)	最大值
5	DEFAULT_VALUE	VARCHAR (256)	默认值
6	MPP_CHK	CHAR(1)	是否检查 MPP 节点间参数一致性。Y 是，N 否
7	SESS_VALUE	VARCHAR(256)	会话参数值
8	FILE_VALUE	VARCHAR(256)	INI 文件中参数值
9	DESCRIPTION	VARCHAR(256)	参数描述
10	PARA_TYPE	VARCHAR(200)	参数类型
11	SYNC_FLAG	VARCHAR(16)	已废弃，取值 UNKNOWN。
12	SYNC_LEVEL	VARCHAR(16)	已废弃，取值 UNKNOWN。
13	PARA_MODULE	VARCHAR(32)	标识参数所属模块，有如下可能取值:

			files location：控制文件相关 instance name：实例名 memory pool and buffer：内存池和缓冲区相关 thread：线程相关 query：查询相关 select rate：查询效率相关 checkpoint：检查点相关 IO：IO 相关 database：数据库相关 pre-load table：预先装载表相关 redo log：REDO 日志生成相关 redo：REDO 重做/重演相关 transaction：事务相关 security：安全相关，储存在 dm.ini 中 security2：安全相关，储存在系统数据文件中，仅能通过系统过程调整，未列在 dm.ini 中 compatibility：兼容性相关 drop table：删除表相关 fast insert：快速插入 request trace：用户请求跟踪相关 system trace：系统跟踪相关 monitor：MONITOR 监控相关 data watch：数据守护相关

			for test only：测试专用 for context index：全文索引相关 rdma：RDMA 通讯相关 configuration file：配置文件相关 DSC：DMDSC 相关 mem only：内存相关 backup and restore：备份还原相关
14	PRIMARY_SYNC	CHAR(1)	参数是否会进行主备同步，取值：Y/N。Y：会进行主备同步，N：不会主备同步。
15	DSC_SYNC	CHAR(1)	参数是否进行 DSC 节点间同步，取值：Y/N。Y：会进行 DSC 节点间同步，N：不会进行 DSC 节点间同步。

3） GV$DM_INI

DPC 环境下查询所有正常打开实例的 INI 参数。

序号	列	数据类型	说明
1	INSTANCE_NAME	VARCHAR (128)	实例名
2	PARA_NAME	VARCHAR (128)	参数名称
3	PARA_VALUE	VARCHAR(256)	系统参数值
4	MIN_VALUE	VARCHAR(256)	最小值
5	MAX_VALUE	VARCHAR(256)	最大值
6	DEFAULT_VALUE	VARCHAR(256)	默认值
7	MPP_CHK	CHAR(1)	是否检查 MPP 节点间参数一致性。Y 是，N 否
8	SESS_VALUE	VARCHAR(256)	会话参数值

9	FILE_VALUE	VARCHAR(256)	INI 文件中参数值
10	DESCRIPTION	VARCHAR(256)	参数描述
11	PARA_TYPE	VARCHAR(200)	参数类型
12	SYNC_FLAG	VARCHAR(16)	已废弃，取值 UNKNOWN。
13	SYNC_LEVEL	VARCHAR(16)	已废弃，取值 UNKNOWN。
14	PARA_MODULE	VARCHAR(32)	标识参数所属模块，有如下可能取值: files location、instance name、memory pool and buffer、thread、query、select rate、cpu、checkpoint、IO、database、pre-load table、redo log、redo、transaction、security、security2、compatibility、drop table、fast insert、request trace、system trace、ecs、monitor、data watch、for test only、for context index、rdma、configuration file、DSC、mem only、backup and restore
15	PRIMARY_SYNC	CHAR(1)	参数是否会进行主备同步，取值：Y/N。Y：会进行主备同步，N：不会主备同步。
16	DSC_SYNC	CHAR(1)	参数是否进行 DSC 节点间同步，取值:Y/N。Y：会进行 DSC 节点间同步，N：不会进行 DSC 节点间同步。

4) V$DM_INI_DESC

所有查询相关 INI 参数的具体描述信息。

序号	列	数据类型	说明
1	PARA_NAME	VARCHAR(128)	参数名称

2	DESCRIPTION	VARCHAR(4096)	参数具体描述

5) V$DM_ARCH_INI

归档参数信息。

序号	列	数据类型	说明
1	ARCH_NAME	VARCHAR (128)	归档名称
2	ARCH_TYPE	VARCHAR (128)	归档类型
3	ARCH_DEST	VARCHAR (512)	对于本地归档，表示归档路径；对于远程归档，表示远程节点实例名；对于其余类型的归档，表示归档目标实例名
4	ARCH_FILE_SIZE	INTEGER	归档文件大小，单位 MB
5	ARCH_SPACE_LIMIT	INTEGER	归档文件的磁盘空间限制，单位 MB
6	ARCH_HANG_FLAG	INTEGER	如果写入本地归档时磁盘空间不够，是否让服务器挂起。仅对本地归档有效，其他归档类型显示为NULL
7	ARCH_TIMER_NAME	VARCHAR (128)	对于异步归档，表示定时器名称；其余类型归档显示 NULL
8	ARCH_IS_VALID	CHAR(1)	归档状态，是否有效
9	ARCH_WAIT_APPLY	INTEGER	在主备系统中，主库发送归档日志给备库之前，是否需要等待备库将上一批日志重演完成。 取值 0：高性能模式，不用等待重演完成； 1：事务一致模式，需等待重演完成。除即时

			归档和实时归档外，其他归档取值均为 NULL
10	ARCH_INCOMING_PATH	VARCHAR(256)	对于远程归档，表示远程节点的本地归档日志文件存放路径；其余类型归档显示 NULL
11	ARCH_CURR_DEST	VARCHAR(256)	当前归档目标实例名，如果备库是 DMDSC 集群，则归档目标是备库控制节点，该字段表示当前被选定为归档目标的节点实例名。备库是单机的情况下，就是指的备库实例名。
12	ARCH_FLUSH_BUF_SIZE	INTEGER	归档刷盘缓存大小，单位 MB，取值范围 0~128。0 表示不使用归档刷盘缓存机制。缺省为 2
13	ARCH_RESERVE_TIME	INTEGER	归档日志保留时间，单位分钟。只对本地归档有效。服务器每隔 5 分钟检查是否存在超过保留时间的归档并删除。缺省为 0，表示不删除归档。
14	ARCH_LOCAL_SHARE	INTEGER	预留字段
15	ARCH_LOCAL_SHARE_CHECK	INTEGER	预留字段
16	ARCH_SEND_DELAY	INTEGER	源库到异步备库的归档延时发送时间。单位分钟。0 表示未启用归档延时发送功能
17	ARCH_DEST_IP	VARCHAR(64)	RAFT 归档目标 IP
18	ARCH_DEST_PORT	INTEGER	RAFT 归档目标端口
19	ARCH_DEST_ID	INTEGER	归档目标多副本节点编号，用于标识多副本 RAFT 组中不同的节点

20	ARCH_ASM_MIRROR	SMALLINT	归档文件配置在 DMASM 上的镜像类型，DMASM 镜像专用，取值：1：单副本，表示外部冗余，不提供任何镜像，假设磁盘高度可靠；2：双副本，表示正常冗余，提供双向镜像，可较好的保证数据完整性；3：三副本，表示高度冗余，提供三向镜像，能够更好的保证数据的完整性。未配置显示 NULL。 仅在 DMASM 镜像环境下有效
21	ARCH_ASM_STRIPING	SMALLINT	归档文件配置在 DMASM 上的条带化粒度，DMASM 镜像专用，取值 0 表示粗粒度，32/64/128/256 表示细粒度条带化大小，单位 KB。未配置显示 NULL。仅在 DMASM 镜像环境下有效
22	ARCH_RECOVER_TIME	INTEGER	主库向归档状态为 Invalid 的同步备库发起异步恢复的时间间隔，单位为秒

6） V$DM_MAL_INI

MAL 参数信息。

序号	列	数据类型	说明
1	MAL_NAME	VARCHAR (128)	MAL 名称
2	MAL_INST_NAME	VARCHAR (256)	实例名称
3	MAL_HOST	VARCHAR (256)	IP 地址

4	MAL_PORT	INTEGER	端口号
5	MAL_INST_HOST	VARCHAR (256)	对应实例 IP 地址
6	MAL_INST_PORT	INTEGER	对应实例的端口号
7	MAL_DW_PORT	INTEGER	MAL_INST_NAME 实例的守护进程监听远程守护进程或监视器连接请求的端口
8	MAL_LINK_MAGIC	INTEGER	MAL 链路网段标识
9	MAL_INST_DW_PORT	INTEGER	MAL_INST_NAME 实例监听守护进程连接请求的端口
10	MAL_RDMA_MAGIC	INTEGER	MAL 链路通信方式标识，取值范围 0~65534，缺省为 0，属于为每个实例配置的局部变量，为了支持 RDMA/TCP 混合通信。在 MAL_USE_RDMA 开启的情况下，配置相同 MAL_RDMA_MAGIC 的节点间使用 RDMA 通信，不同则使用 TCP 通信。若不配置 MAL_RDMA_MAGIC，缺省为 0，可以和 MAL_RDMA_MAGIC 为 0 的节点进行 RDMA 通信

7) V$DM_TIMER_INI

定时器参数信息。

序号	列	数据类型	说明
1	TIMER_NAME	VARCHAR(128)	定时器名称
2	TYPE	TINYINT	类型。1: 执行一次; 2: 按日执行; 3: 按周执行; 4: 按月执行的第几天; 5: 按月执

			行的第一周；6：按月执行的第二周；7：按月执行的第三周；8：按月执行的第四周；9：按月执行的最后一周；10：日历表达式
3	FREQ_MONTH_WEEK_INTERVAL	INTEGER	间隔的月/周/天（调度类型决定）数
4	FREQ_SUB_INTERVAL	INTEGER	第几天/星期几/一个星期中的某些天
5	FREQ_MINUTE_INTERVAL	INTEGER	分钟间隔
6	START_TIME	TIME(0)	开始时间
7	END_TIME	TIME(0)	结束时间
8	DURING_START_DATE	DATETIME(6)	有效时间段的开始日期时间
9	DURING_END_DATE	DATETIME(6)	有效时间段的结束日期时间
10	NO_END_DATE_FLAG	CHAR(1)	无结束日期标识，Y 无结束日期；N 有结束日期。对应 dmtimer.ini 中的 NO_END_DATA_FLAG 配置项。Y 对应 1；N 对应 0
11	DESCRIBE	VARCHAR(256)	描述
12	IS_VALID	CHAR(1)	是否有效，Y 是；N 否。对应 dmtimer.ini 中的 IS_VALID 配置项。Y 对应 1；N 对应 0

13	REPEAT_INTERVAL	VARCHAR(4000)	日历表达式

8) V$OBSOLETE_PARAMETER

已作废的 INI 信息。

序号	列	数据类型	说明
1	PARA_NAME	VARCHAR (128)	参数名称

9) V$OPTION

安装数据库时的参数设置。

序号	列	数据类型	说明
1	PARA_NAME	VARCHAR (128)	参数名称
2	PARA_VALUE	VARCHAR (256)	系统参数值

10) V$DM_SQLLOG_INI

显示 sqllog.ini 中的配置参数信息。

序号	列	数据类型	说明
1	MODE_NAME	VARCHAR(128)	SQL 日志模式名。值为<PUBLIC>时，表示当前参数为全局配置参数
2	PARA_NAME	VARCHAR(128)	SQL 日志中的参数名
3	PARA_VALUE	VARCHAR(32767)	SQL 日志中的参数值
4	IS_DEFAULT	CHAR(1)	PARA_VALUE 是否与<DEFAULT>模式中对应参数值相等

11) V$DM_SQLLOG_CONFIG

显示内存中的 SQL 日志配置参数信息，即当前生效的配置参数信息。

序号	列	数据类型	说明
1	MODE_NAME	VARCHAR(128)	SQL 日志模式名。值为<PUBLIC>时，表示当前参数

			为全局配置参数；值为<DEFAULT>时，表示当前使用系统默认配置
2	PARA_NAME	VARCHAR(128)	SQL 日志中的参数名
3	PARA_VALUE	VARCHAR(32767)	SQL 日志中的参数值

6. 日志管理

1) V$RLOG

显示日志的总体信息。通过该视图可以了解系统当前日志事务号 LSN 的情况、检查点的执行情况等。

序号	列	数据类型	说明
1	CKPT_LSN	BIGINT	最近一次检查点 LSN
2	FILE_LSN	BIGINT	已经到盘上的 LSN
3	FLUSH_LSN	BIGINT	当前准备刷盘的 LSN
4	CUR_LSN	BIGINT	当前的 LSN
5	NEXT_SEQ	BIGINT	下一个日志包序号
6	N_MAGIC	INTEGER	本次运行，日志的 MAGIC
7	DB_MAGIC	BIGINT	数据库的 MAGIC
8	FLUSH_PAGES	INTEGER	Flush 链表中的总页数
9	FLUSHING_PAGES	INTEGER	正在刷盘的总页数
10	CUR_FILE	INTEGER	记录刷文件前当前文件的 ID
11	CUR_OFFSET	BIGINT	记录刷文件前 cur_file 的 free
12	CKPT_FILE	INTEGER	最近一次检查点对应的当时的文件号

13	CKPT_OFFSET	BIGINT	最近一次检查点对应的当时的文件偏移
14	FREE_SPACE	BIGINT	目前可用的日志空间，单位为字节
15	TOTAL_SPACE	BIGINT	日志总空间，单位为字节
16	SUSPEND_TIME	DATETIME(6)	挂起时间戳
17	UPD_CTL_LSN	BIGINT	系统修改控制文件的 RLOG_RESERVE_THRESHOLD ->lsn
18	N_RESERVE_WAIT	INTEGER	日志空间预申请等待的个数
19	TOTAL_FLUSH_PAGES	BIGINT	系统启动后累计刷盘的总页数
20	TOTAL_FLUSH_TIMES	BIGINT	系统启动后累计刷盘的总次数
21	TOTAL_ECPR_FLUSH_PAGES	BIGINT	系统启动后累计刷盘的日志总页数，非加密压缩情况下等于 TOTAL_FLUSH_PAGES
22	GLOBAL_NEXT_SEQ	BIGINT	下一个全局日志包序号
23	N_PRIMARY_EP	INTEGER	主库节点数
24	PRIMARY_DB_MAGIC	BIGINT	主库 DB_MAGIC 值
25	CKPT_N_PRIMARY_EP	INTEGER	主库节点数,刷盘时调整
26	CKPT_PRIMARY_DB_MAGIC	BIGINT	主库 DB_MAGIC，刷盘时调整
27	MIN_EXEC_VER	VARCHAR(64)	日志文件允许访问的最小执行码版本号
28	MIN_DCT_VER	INTEGER	日志文件允许访问的最小字典版本号
29	CLSN_OPEN_ID	INTEGER	当前实例检查点偏移对应的 DB_OPEN_ID
30	DB_OPEN_ID	INTEGER	当前实例 OPEN 次数。DMDSC 集群推进检查点时，从 DB_OPEN_ID 值最小的节点开始依次推进检查点，以保证故障处理时严格按照日志包产生顺序重做 REDO 日志

31	ALMOST_FULL_STATUS	VARCHAR(16)	日志空间使用情况，SAFE：剩余空间足够，UNSAFE：剩余空间不足
32	PTX_RESERVED	BIGINT	当前所有物理事务 ptx 预留的日志总量，单位：字节
33	RESERVE_THRESHOLD	INTEGER	启动日志空间预留机制的日志量门槛，单位：MB
34	RESERVE_SIZE	INTEGER	启动日志空间预留机制后，每个物理事务 ptx 操作预留的日志空间大小，单位为日志页个数，参考 2.1.1.1.10 中 RLOG_RESERVE_SIZE

2) V$RLOG_PKG

显示日志包信息。通过该视图可以查询日志系统中当前日志包的使用情况，如包的长度、最大 LSN、最小 LSN 等。通过该视图还可以查询当前实例日志系统中等待刷盘的链表上的日志包信息。

序号	列	数据类型	说明
1	PKG_LEN	BIGINT	日志包的长度，单位 BYTE
2	DATA_OFF	BIGINT	日志包中有效数据长度，单位 BYTE
3	ENC_LEN	BIGINT	日志包加密后数据长度，单位 BYTE
4	CMPR_LEN	BIGINT	日志包压缩后数据长度，单位 BYTE
5	HDR_SIZE	INTEGER	日志包包头长度，单位 BYTE
6	N_MAGIC	BIGINT	日志包固定魔数
7	DB_MAGIC	BIGINT	数据库魔数
8	N_DSC	INTEGER	DSC 集群节点数

9	DSC_SEQNO	INTEGER	DSC 集群节点号
10	FROM_PRIMARY	INTEGER	是否为主库生成的包，1：是主库生成的包；0：不是主库生成的包
11	PKG_TYPE	INTEGER	日志包包类型，不同取值分别对应类型如下：0：RLOG；1：PLOG；2：CKPT；3：UPD_KEY，联机日志密码更新
12	ENC_CMPR	INTEGER	加密压缩标记，0：CMPR_NONE；1：ENC_ONLY；2：CMPR_ONLY；3：ENC_CMPR。
13	L_SEQNO	BIGINT	本地包序号
14	G_SEQNO	BIGINT	全局包序号
15	MIN_LSN	BIGINT	包内日志最小 LSN
16	MAX_LSN	BIGINT	包内日志最大 LSN
17	PREV_LSN	BIGINT	前一个包的日志最大 LSN
18	N_FIXED	INTEGER	包的日志填充次数
19	N_FILLED	INTEGER	包的填充完成的次数
20	PKG_STAT	INTEGER	日志包状态，0 表示正在填充的串行包，1 表示正在填充的并行包，2 表示已填充完成待刷盘的日志包

3） V$RLOG_PKG_STAT

显示当前实例日志系统中日志包使用的统计信息。

序号	列	数据类型	说明
1	N_FLUSH_TO	BIGINT	请求刷盘的次数
2	N_FLUSH_WAIT	BIGINT	刷盘等待的次数
3	N_PUSHED	BIGINT	拷贝到 flush_lst 次数
4	N_COPY_WAIT	BIGINT	拷贝到 flush_lst 过程中产生等待的次数

5	N_FOR_CKPT	BIGINT	检查点类型的日志包个数
6	N_DISCARDED	BIGINT	日志刷盘线程中丢弃的日志包个数
7	N_FLUSHED	BIGINT	刷盘日志包个数
8	BYTES_FLUSHED	BIGINT	刷盘日志包长度总和，单位 BYTE
9	PKG_AVG_BYTES	INTEGER	刷盘日志包平均长度，单位 BYTE
10	NEXT_SEQNO	BIGINT	下一个日志包序号
11	N_ALLOC_WAIT	BIGINT	分配日志包过程中产生等待的次数
12	GLOBAL_NEXT_SEQNO	BIGINT	下一个全局日志包序号
13	PKG_AVG_PLL	INTEGER	日志包的平均并行度
14	N_FLUSH_LST	INTEGER	待刷盘的日志包个数

4) V$RLOGFILE

显示日志文件的具体信息。包括文件号、完整路径、文件的状态、文件大小等等。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	表空间 ID
2	FILE_ID	INTEGER	文件 ID
3	PATH	VARCHAR(256)	文件路径
4	CLIENT_PATH	VARCHAR(256)	文件路径，专门提供给客户端
5	CREATE_TIME	DATETIME(6)	创建时间
6	RLOG_SIZE	BIGINT	文件大小，单位 BYTE
7	MIN_EXEC_VER	VARCHAR(64)	此文件允许访问的最小执行码版本号
8	MIN_DCT_VER	INTEGER	此文件允许访问的最小字典版本号

5) V$ARCHIVED_LOG

显示当前实例的所有归档日志文件信息。此动态视图中有一些保留列，查询时均显示

NULL。

序号	列	数据类型	说明
1	RECID	INTEGER	保留列
2	STAMP	INTEGER	保留列
3	NAME	VARCHAR(513)	归档日志文件名
4	DEST_ID	INTEGER	保留列
5	THREAD#	BIGINT	保留列。均为 0
6	SEQUENCE#	INTEGER	日志文件的序号；若序号 SEQUENCE#连续，则表示不存在归档缺失；若相邻两行的序号 SEQUENCE#之间跳过一个数，则表示这两个序号代表的归档文件中间有归档缺失
7	RESETLOGS_CHANGE#	INTEGER	保留列
8	RESETLOGS_TIME	DATETIME(6)	保留列
9	RESETLOGS_ID	INTEGER	保留列
10	FIRST_CHANGE#	BIGINT	日志文件所记录日志的最小 LSN 值
11	FIRST_TIME	DATETIME(6)	日志文件所记录的日志的起始时间
12	NEXT_CHANGE#	BIGINT	日志文件所记录日志的最大 LSN 值
13	NEXT_TIME	DATETIME(6)	日志文件所记录的日志的截止时间
14	BLOCKS	INTEGER	保留列
15	BLOCK_SIZE	INTEGER	保留列
16	CREATOR	VARCHAR(7)	保留列
17	REGISTRAR	VARCHAR(7)	保留列
18	STANDBY_DEST	VARCHAR(3)	保留列

19	ARCHIVED	VARCHAR(3)	保留列。均为 YES
20	APPLIED	VARCHAR(9)	归档文件日志包是否全部重演完成。NO：归档文件中日志包还没有重演完成；YES：归档文件中日志包全部重演完成 对于 DMDSC 备库非控制节点，不会重演，此字段结果为 NULL；对于 RAFT 下的影子备库，不会重演，此字段结果为 NO
21	DELETED	VARCHAR(3)	保留列。均为 NO
22	STATUS	VARCHAR(1)	保留列。均为 A
23	COMPLETION_TIME	DATETIME(6)	保留列
24	DICTIONARY_BEGIN	VARCHAR(3)	保留列
25	DICTIONARY_END	VARCHAR(3)	保留列
26	END_OF_REDO	VARCHAR(3)	保留列
27	BACKUP_COUNT	INTEGER	保留列
28	ARCHIVAL_THREAD#	INTEGER	保留列
29	ACTIVATION#	INTEGER	保留列
30	IS_RECOVERY_DEST_FILE	VARCHAR(3)	保留列。均为 NO
31	COMPRESSED	VARCHAR(3)	保留列
32	FAL	VARCHAR(3)	保留列
33	END_OF_REDO_TYPE	VARCHAR(10)	保留列
34	BACKED_BY_VSS	VARCHAR(3)	保留列
35	ARCH_TYPE	VARCHAR(128)	归档类型
36	EP_SEQNO	INTEGER	DSC 节点号

37	ARCH_SEQ	BIGINT	起始包序号
38	NEXT_SEQ	BIGINT	最大包序号
39	VERSION	BIGINT	库版本

6) V$LOGMNR_LOGS

显示当前会话添加的需要分析的归档日志文件。

序号	列	数据类型	说明
1	LOG_ID	INTEGER	日志文件 ID 号
2	FILENAME	VARCHAR(512)	日志文件 ID 名
3	LOW_TIME	DATETIME(6)	日志文件创建时间
4	HIGH_TIME	DATETIME(6)	日志文件最后修改时间
5	DB_ID	INTEGER	日志文件的实例 ID 号
6	DB_NAME	VARCHAR(128)	日志文件的数据库名称
7	RESET_SCN	BIGINT	保留列。暂无意义
8	RESET_SCN_TIME	DATETIME(6)	保留列。暂无意义
9	COMPATIBLE	VARCHAR(17)	保留列。暂无意义
10	THREAD_ID	BIGINT	保留列。均为 0
11	THREAD_SQN	INTEGER	日志文件的序号
12	LOW_SCN	BIGINT	日志文件最小 LSN
13	NEXT_SCN	BIGINT	日志文件最大 LSN
14	DICTIONARY_BEGIN	VARCHAR(3)	保留列。均为 NO
15	DICTIONARY_END	VARCHAR(3)	保留列。均为 NO
16	TYPE	VARCHAR(8)	保留列。均为 ARCHIVED

17	BLOCKSIZE	INTEGER	保留列。均为 512
18	FILESIZE	BIGINT	日志文件的大小，单位 BYTE
19	INFO	VARCHAR(32)	保留列。暂无意义
20	STATUS	INTEGER	保留列。均为 0

7) V$LOGMNR_PARAMETERS

显示当前会话 START_LOGMNR 启动日志文件分析的参数。

序号	列	数据类型	说明
1	START_DATE	DATETIME(6)	过滤分析归档日志的起始时间
2	REQUIRED_START_DATE	DATETIME(6)	保留列，返回 NULL
3	END_DATE	DATETIME(6)	过滤分析归档日志的截止时间
4	START_SCN	BIGINT	过滤分析归档日志的起始 LSN
5	REQUIRED_START_SCN	BIGINT	保留列，返回 0
6	END_SCN	BIGINT	过滤分析归档日志的截止 LSN
7	OPTIONS	INTEGER	分析归档日志的选项
8	INFO	VARCHAR(32)	保留列，返回 NULL
9	STATUS	INTEGER	保留列，返回 0

8) V$LOGMNR_CONTENTS

显示当前会话日志分析的内容。

序号	列	数据类型	说明
1	SCN	BIGINT	当前记录的 LSN
2	START_SCN	BIGINT	当前事务的起始 LSN

3	COMMIT_SCN	BIGINT	当前事务的截止 LSN
4	TIMESTAMP	DATETIME(6)	当前记录的创建时间
5	START_TIMESTAMP	DATETIME(6)	当前事务的起始时间
6	COMMIT_TIMESTAMP	DATETIME(6)	当前事务的截止时间
7	XIDUSN	BIGINT	保留列，暂无意义，显示为 NULL
8	XIDSLT	BIGINT	保留列，暂无意义，显示为 NULL
9	XIDSQN	BIGINT	保留列，暂无意义，显示为 NULL
10	XID	BINARY(8)	当前记录的事务 ID 号，为 BIGINT 类型的事务 ID 直接转换为 16 进制，并在前面补 0
11	PXIDUSN	BIGINT	保留列，暂无意义，显示为 NULL
12	PXIDSLT	BIGINT	保留列，暂无意义，显示为 NULL
13	PXIDSQN	BIGINT	保留列，暂无意义，显示为 NULL
14	PXID	BINARY(8)	保留列，暂无意义，显示为 NULL
15	TX_NAME	VARCHAR(256)	保留列，暂无意义，显示为 NULL
16	OPERATION	VARCHAR(32)	操作类型：INSERT、DELETE、UPDATE、 BATCH_UPDATE、DDL、START、COMMIT、 LOB_NEW、LOB_DATA、PRE_COMMIT、 ROLLBACK、SEQ MODIFY、XA_COMMIT、 UNSUPPORTED
17	OPERATION_CODE	INTEGER	操作类型编号： 1 - INSERT 2 - DELETE 3 - UPDATE

			4 - BATCH_UPDATE 5 - DDL 6 - START 7 – COMMIT 13 - LOB_NEW 14 - LOB_DATA 15 - PRE_COMMIT 36 - ROLLBACK 37 - SEQ MODIFY 38 - XA_COMMIT 255 - UNSUPPORTED
18	ROLL_BACK	INTEGER	当前记录是否被回滚，1：是，0：否
19	SEG_OWNER	VARCHAR(128)	操作的模式名
20	SEG_NAME	VARCHAR(256)	保留列，暂无意义，显示为 NULL
21	TABLE_NAME	VARCHAR(128)	操作的表名
22	SEG_TYPE	INTEGER	保留列，暂无意义，显示为 NULL
23	SEG_TYPE_NAME	VARCHAR(32)	保留列，暂无意义，显示为 NULL
24	TABLE_SPACE	VARCHAR(32)	保留列，暂无意义，显示为 NULL
25	ROW_ID	VARCHAR(20)	对应记录的行号
26	USERNAME	VARCHAR(128)	执行这条语句的用户名
27	OS_USERNAME	VARCHAR(4000)	保留列，暂无意义，显示为 NULL
28	MACHINE_NAME	VARCHAR(4000)	保留列，暂无意义，显示为 NULL
29	AUDIT_SESSIONID	BIGINT	保留列，暂无意义，显示为 NULL

30	SESSION#	BIGINT	当前记录的会话 ID 信息
31	SERIAL#	BIGINT	保留列，暂无意义，显示为 NULL
32	SESSION_INFO	VARCHAR(4000)	语句来源机器的 IP 地址
33	THREAD#	BIGINT	保留列，暂无意义，显示为 NULL
34	SEQUENCE#	INTEGER	保留列，暂无意义，显示为 NULL
35	RBASQN	INTEGER	对应的归档日志文件号
36	RBABLK	INTEGER	RBASQN 所指日志文件的块号，从 0 开始
37	RBABYTE	INTEGER	RBABLK 所指块号的块内偏移
38	UBAFIL	BIGINT	保留列，暂无意义，显示为 NULL
39	UBABLK	BIGINT	保留列，暂无意义，显示为 NULL
40	UBAREC	BIGINT	保留列，暂无意义，显示为 NULL
41	UBASQN	BIGINT	保留列，暂无意义，显示为 NULL
42	ABS_FILE#	INTEGER	保留列，暂无意义，显示为 NULL
43	REL_FILE#	INTEGER	保留列，暂无意义，显示为 NULL
44	DATA_BLK#	INTEGER	保留列，暂无意义，显示为 NULL
45	DATA_OBJ#	INTEGER	对象 ID 号
46	DATA_OBJV#	INTEGER	对象版本号
47	DATA_OBJD#	INTEGER	保留列，暂无意义，显示为 NULL
48	SQL_REDO	VARCHAR(4000)	客户端发送给数据库的 SQL 语句

49	SQL_UNDO	VARCHAR(4000)	回滚 SQL 语句。该列结果受 INI 参数 RLOG_APPEND_LOGIC 影响。 当 RLOG_APPEND_LOGIC 为 1 时，支持挖掘 INSERT 类型的 UNDO 语句，如果表含有主键列，则不能挖掘 DELETE、UPDATE 类型的语句，否则可挖掘； 当 RLOG_APPEND_LOGIC 为 2 时，支持挖掘 INSERT、DELETE、UPDATE 类型的 UNDO 语句； 当 RLOG_APPEND_LOGIC 为 3 时，仅支持挖掘 INSERT 类型的 UNDO 语句； 当 RLOG_APPEND_LOGIC 为 4 时，不支持挖掘 INSERT、DELETE、UPDATE 类型的 UNDO 语句
50	RS_ID	VARCHAR(32)	记录集 ID
51	SSN	INTEGER	连续 SQL 标志。如果 SQL 长度超过单个 sql_redo 字段能存储的长度，则 SQL 会被截断成多个 SQL 片段在结果集中“连续”返回
52	CSF	INTEGER	与 SSN 配合，最后一个片段的 csf 值为 0，其余片段的值均为 1。没因超长发生截断的 SQL 该字段值均为 0。
53	INFO	VARCHAR(32)	保留列，暂无意义，显示为 NULL
54	STATUS	INTEGER	保留列，暂无意义，显示为 NULL
55	REDO_VALUE	BIGINT	用于数据挖掘新值
56	UNDO_VALUE	BIGINT	用于数据挖掘旧值

57	SAFE_RESUME_SCN	BIGINT	保留列，暂无意义，显示为 NULL
58	CSCN	BIGINT	与 COMMITSCN 一样，已过时
59	OBJECT_ID	BINARY(16)	保留列，暂无意义，显示为 NULL
60	EDITION_NAME	VARCHAR(30)	保留列，暂无意义，显示为 NULL
61	CLIENT_ID	VARCHAR(64)	保留列，暂无意义，显示为 NULL
62	SUBTAB_ID	INTEGER	子表 ID
63	SRC_RAFT_ID	INTEGER	用于作为事务来源节点号

9） V$ARCH_QUEUE

显示当前归档任务队列信息。

序号	列	数据类型	说明
1	ARCH_TYPE	VARCHAR(32)	归档类型:LOCAL; REALTIME; TIMELY ASYN; REMOTE;
2	WAITING	INTEGER	等待处理任务数
3	READY	BIGINT	已处理任务数
4	CUR_WAIT_TIME	INTEGER	当前任务等待时间，单位毫秒
5	MAX_WAIT_TIME	INTEGER	最大任务等待时间，单位毫秒
6	TOTAL_WAIT_TIME	BIGINT	已处理任务的总等待时间，单位：毫秒
7	AVERAGE_WAIT_TIME	INTEGER	已处理任务的平均等待时间，单位：毫秒

10） V$RLOG_HISTORY

显示日志的历史总体信息。通过该视图可以了解最近一周内系统每过一分钟日志事务号 LSN 的情况、归档日志情况以及检查点的执行情况等。

序号	列	数据类型	说明
1	RECORD_TIME	DATETIME(6)	记录产生的时间

2	G_NEXT_SEQ	BIGINT	全局日志包序号
3	CUR_LSN	BIGINT	记录产生时，当前的 LSN
4	FLUSH_LSN	BIGINT	记录产生时，当前准备刷盘的 LSN
5	FILE_LSN	BIGINT	记录产生时，已经到盘上的 LSN
6	CKPT_LSN	BIGINT	记录产生时，最近一次检查点 LSN
7	FLUSH_PAGES	BIGINT	记录产生时，Flush 链表中的总页数
8	FLUSHING_PAGES	BIGINT	记录产生时，正在刷盘的总页数
9	TOTAL_SPACE	BIGINT	记录产生时，日志总空间，单位字节
10	FREE_SPACE	BIGINT	记录产生时，目前可用的日志空间，单位字节
11	CKPT_FILE	INTEGER	记录产生时，最近一次检查点对应的当时的文件号
12	CKPT_OFF	BIGINT	记录产生时，最近一次检查点对应的当时的文件偏移
13	CUR_FILE	INTEGER	记录产生时，记录刷文件前当前文件的 ID
14	CUR_OFF	BIGINT	记录产生时，记录刷文件前 cur_file 的 free
15	RPKG_MEM_SIZE	BIGINT	记录产生时，RLOG 上当前 PKG 分配的空间
16	RPKG_USED_SIZE	BIGINT	记录产生时，RLOG 上当前 PKG 使用的空间
17	TOTAL_FLUSH_PAGES	BIGINT	记录产生时，系统启动后累计刷盘的总页数
18	TOTAL_FLUSH_TIMES	BIGINT	记录产生时，系统启动后累计刷盘的总次数

11） V$ELOG_FILE

显示 ELOG 日志的记录时间范围、大小、SITE_ID 等信息。通过该视图可以了解当前实例的所有 ELOG 日志文件的信息。

序号	列	数据类型	说明
1	FILE_NAME	VARCHAR(128)	日志文件名，包含文件类型

2	SITE_ID	INTEGER	文件所属的站点的站点号，在 DPC 环境中为 RAFT_ID，在 MPP 环境中为执行站点号 EPSEQ，在 DSC 环境中为节点号
3	FILE_SIZE	INTEGER	日志文件大小，单位 MB，总大小不足 1MB 的按照 1MB 算
4	FIRST_REC_TIME	DATETIME(6)	日志文件内第一条记录的时间
5	LAST_REC_TIME	DATETIME(6)	日志文件内最后一条记录的时间

12) V$RLOG_FREE_LIST

显示日志空闲链表中的空闲日志包信息。

对于主库或 NORMAL 模式的库，如果开启了 RLOG_PARALLEL_ENABLE，则会显示每路并行日志的空闲链表信息，显示行数为 WORKER_THREADS + 1，否则只显示 1 行。

其中 RLOG_BUF_SIZE 和 RLOG_POOL_SIZE 会结合日志并行信息、工作线程个数等因素再进行调整，不一定是 dm.ini 中的配置大小。

日志包内存分配关系说明如下：

（1）默认日志包大小 = RLOG_BUF_SIZE * 512 字节。

（2）FREE_RPKG_LEN = FREE_RPKG_CNT * 默认日志包大小，代表已分配但未使用的空闲空间。

（3）FREE_RPKG_LEN 一定不会超过 RLOG_POOL_SIZE 大小。

（4）已分配出的默认大小的日志包内存空间 = N_ALLOC_PAGE * 512 字节，此空间大小大于等于 FREE_RPKG_LEN，但不会超过 RLOG_POOL_SIZE 大小。

序号	列	数据类型	说明
1	RLOG_BUF_SIZE	INTEGER	单个日志缓冲区大小，以日志页个数为单位
2	RLOG_POOL_SIZE	BIGINT	最大日志缓冲区大小，单位字节
3	N_ALLOC_PAGE	INTEGER	空闲链表中已分配出的日志页个数
4	FREE_RPKG_CNT	INTEGER	空闲链表中存放的日志包个数

5	FREE_RPKG_LEN	BIGINT	空闲链表中存放的日志包总大小，单位字节

13) V$LOG_HISTORY

服务器启动后，记录当前节点联机日志切换的历史信息。只记录最近 128 次切换记录。

序号	列	数据类型	说明
1	RECID	BIGINT	记录 ID，表示联机日志发生的第 N 次切换
2	RECTIME	DATETIME(0)	联机日志切换时间
3	FIL_ID	INTEGER	切换后的联机日志文件号
4	CKPT_LSN	BIGINT	切换时的 CKPT_LSN
5	FILE_LSN	BIGINT	切换时的 FILE_LSN
6	CUR_LSN	BIGINT	切换时的 CUR_LSN

14) V$ARCH_BACKUP_HISTORY

服务器启动后，记录当前节点归档日志备份的历史信息。只记录最近 100 次备份记录。

序号	列	数据类型	说明
1	BEGIN_ARCH_PATH	VARCHAR(256)	本次归档备份开始时的归档文件路径
2	BEGIN_LSN	BIGINT	本次归档备份开始时的归档文件的 BEGIN_LSN
3	BEGIN_SEQ	BIGINT	本次归档备份开始时的归档文件的 BEGIN_SEQ
4	END_ARCH_PATH	VARCHAR(256)	本次归档备份结束时的归档文件路径
5	END_LSN	BIGINT	本次归档备份结束时的归档文件的 END_LSN
6	END_SEQ	BIGINT	本次归档备份结束时的归档文件的 END_SEQ

15) V$ARCH_APPLY_INFO

主库或级联异步备库上记录的备库重演信息，与备库真实的重演信息存在 60s 延迟。

序号	列	数据类型	说明

1	DEST_NAME	VARCHAR(512)	归档目标名，如果备库是 DSC 集群，则打印控制节点实例名，若控制节点不存在，则打印归档对象字符串中首个归档对象实例名。
2	ARCH_STANDBY_APPLY	INTEGER	DMARCH.INI 中归档目标的 ARCH_STANDBY_APPLY 配置
3	DSC_SEQNO	INTEGER	重演信息对应的主库 DSC 节点号
4	APPLY_SEQNO	BIGINT	备库刷盘到的日志包序号
5	APPLY_LSN	BIGINT	备库刷盘到的最大 LSN
6	KEEP_SEQNO	BIGINT	备库保持不重演的最大日志包序号
7	KEEP_LSN	BIGINT	备库保持不重演的最大 LSN
8	STANDBY_SEQNO	BIGINT	备库已接收到的最大日志包序号
9	STANDBY_LSN	BIGINT	备库已接收到的最大 LSN
10	RPKG_SEQNO	BIGINT	备库已重演到的日志包序号
11	RPKG_LSN	BIGINT	备库已重演到的 LSN
12	C_SEQNO	BIGINT	备库已提交到的日志包序号，仅在 DMDPC 的多副本系统中有效
13	C_LSN	BIGINT	备库已提交到的 LSN，仅在 DMDPC 的多副本系统中有效

16）V$ARCH_FILE

查询本地已经归档的日志信息。对 DMDSC 集群，除了显示本地归档外，也显示远程归档信息。

序号	列	数据类型	说明

1	DB_MAGIC	BIGINT	写入日志的数据库的 MAGIC 值
2	STATUS	VARCHAR(64)	归档日志状态
3	LEN	BIGINT	日志长度，单位为字节
4	FREE	BIGINT	写日志偏移位置
5	ARCH_LSN	BIGINT	归档文件起始 LSN，仅对归档文件有意义
6	CLSN	BIGINT	已归档的最大 LSN
7	ARCH_SEQ	BIGINT	归档文件起始包序号，仅对归档文件有意义
8	NEXT_SEQ	BIGINT	已归档的最大包序号。
9	CREATE_TIME	DATETIME(6)	归档文件创建时间
10	CLOSE_TIME	DATETIME(6)	归档文件关闭时间
11	PATH	VARCHAR(256)	归档文件所在路径，包括归档文件名
12	PMNT_MAGIC	BIGINT	数据库的永久 MAGIC
13	DSC_SEQNO	INTEGER	DSC 节点序号，单节点为 0
14	CRC_CHECK	INTEGER	是否进行 CRC 校验
15	LAST_PKG_OFF	BIGINT	最后一个有效的日志包在文件中的偏移位置
16	PREV_LSN	BIGINT	前一个归档文件已归档的最大 LSN
17	GLOBAL_NEXT_SEQ	BIGINT	已归档的最大包序号
18	SRC_DB_MAGIC	BIGINT	产生日志的源数据库的 MAGIC 值
19	LLOG_FIRST_TIME	DATETIME(6)	归档文件中包含的第一条逻辑日志的生成时间，仅在开启逻辑日志时有效
20	LLOG_LAST_TIME	DATETIME(6)	归档文件中包含的最后一条逻辑日志的生成时间，仅在开启逻辑日志时有效

17) V$ARCH_FILE_HISTORY

当 INI 参数 ENABLE_MONITOR=1 时，该视图显示因归档空间不足而被服务器主动删除的所有归档文件信息。

1. 此视图只能查询到已被服务器主动删除的归档文件信息，对于尚未被删除或者用户手动删除的归档文件则无法查询到；

2. 此视图最多能够查询到最近 10000 个被服务器主动删除的归档文件信息；

3. 若存在多路本地归档，则该视图的查询结果中将包含每一路本地归档下被删除的归档文件信息；

4. 此视图记录的归档文件历史信息是内存值，不会持久化到磁盘，因此服务器重启后将无法再查询到此前被删除的归档文件信息；

5. 可以通过系统过程 SP_DYNAMIC_VIEW_DATA_CLEAR 来清空此视图的历史信息。

序号	列	数据类型	说明
1	DSC_SEQNO	INTEGER	DSC 节点序号，单节点为 0
2	SRC_DB_MAGIC	BIGINT	产生日志的源数据库的 MAGIC 值
3	PMNT_MAGIC	BIGINT	数据库的永久 MAGIC
4	DB_MAGIC	BIGINT	写入日志的数据库的 MAGIC 值
5	CREATE_TIME	DATETIME(6)	归档文件创建时间
6	CLOSE_TIME	DATETIME(6)	归档文件关闭时间
7	DELETE_TIME	DATETIME(6)	归档文件删除时间
8	LEN	BIGINT	日志长度，单位为字节
9	FREE	BIGINT	写日志偏移位置
10	ARCH_LSN	BIGINT	归档文件起始 LSN
11	CLSN	BIGINT	已归档的最大 LSN
12	ARCH_SEQ	BIGINT	归档文件起始包序号

13	NEXT_SEQ	BIGINT	已归档的最大包序号
14	PATH	VARCHAR(256)	归档文件所在路径，包括归档文件名

18) V$ARCH_FILE2

查询本地已经归档的日志信息。对 DMDSC 集群，除了显示本地归档外，也显示远程归档信息，若存在故障节点，查询 V$ARCH_FILE 视图，可以显示故障节点的归档信息，而 V$ARCH_FILE2 中无法显示故障节点的归档信息。

V$ARCH_FILE2 展示的字段与 V$ARCH_FILE 相同。V$ARCH_FILE2 中的信息从缓存中获取，查询速度更快，但是可能存在一定的延时问题，可能会导致处于 ACTIVIE 状态下的某些字段与查询 V$ARCH_FILE 视图展示的结果不一致。若想获得即时的归档日志信息，请查询 V$ARCH_FILE。如果手动删除归档日志文件，目前缓存中还会保留已删除的文件信息，即查询 V$ARCH_FILE2 视图也会展示已经删除的归档日志文件信息，会导致查询 V$ARCH_FILE2 与 V$ARCH_FILE2 结果不一致。

7. 会话

1) V$CONNECT

显示活动连接的所有信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	连接名称
2	SADDR	BIGINT	会话地址
3	CREATE_TIME	DATETIME(0)	会话创建时间
4	STATUS$	VARCHAR(128)	连接状态
5	TYPE$	VARCHAR(128)	连接类型，有如下取值：ODBC、SQL3、

			JDBC、.NET PROVIDER、PYTHON、GO、NODEJS、DCI、DCCI、PHP、UNKNOWN
6	PROTOCOL_TYPE	TINYINT	协议类型，取值 1 表示 TCP
7	IP_ADDR	VARCHAR(192)	IP 地址

2） V$SESSIONS

显示当前数据库中所有会话的信息，如执行的 sql 语句、主机名、当前会话状态、用户名等等。本视图中时间相关的字段均按服务器端进行记录。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID，系统内部标识
2	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
3	SQL_TEXT	VARCHAR(1000)	对于会话当前正在执行的或最近一次执行的 SQL 语句，记录该 SQL 的前 1000 个字符
4	STATE	VARCHAR(8)	会话状态。 CREATE：创建，表示会话对象已创建，但还不能使用； STARTUP：启动，表示会话正在启动中；

			IDLE：空闲，表示会话当前没有执行操作； ACTIVE：活动，表示会话当前正在执行操作； PENDING：限流等待，当 INI 参数 MAX_CONCURRENT_TRX>0 时，会话可能会因为并行事务限流而处于此状态； UNKNOWN：正在释放，表示会话正在被释放
5	N_STMT	INTEGER	语句 STMT 的容量
6	N_USED_STMT	INTEGER	已使用的语句 STMT 数量
7	SEQ_NO	INTEGER	会话上语句的序列号
8	CURR_SCH	VARCHAR(128)	当前模式
9	USER_NAME	VARCHAR(128)	当前用户
10	TRX_ID	BIGINT	事务 id，0 表示事务未开始或事务已结束
11	CREATE_TIME	DATETIME(6)	会话创建时间
12	CLNT_TYPE	VARCHAR(128)	客户端类型,可能有如下取值：ODBC、SQL3、JDBC、.NET PROVIDER、PYTHON、GO、NODEJS、DCI、DCCI、PHP、UNKNOWN
13	TIME_ZONE	VARCHAR(6)	会话使用的时区
14	CHK_CONS	CHAR(1)	会话是否忽略约束检查。'Y'：忽略约束检查，'N'：不忽略
15	CHK_IDENT	CHAR(1)	会话是否忽略自增列指定列表检查。'Y'：是，'N'：否
16	RDONLY	CHAR(1)	是否为只读会话。'Y'：是，'N'：否
17	INS_NULL	CHAR(1)	当列不存在 default 选项时，是否插入空值。'Y'：是，'N'：否

18	COMPILE_FLAG	CHAR(1)	是否忽略编译选项，用于 VIEW、PROCEDURE、TRIGGER 等对象的编译。'Y'：是，'N'：否
19	AUTO_CMT	CHAR(1)	会话是否为自动提交。'Y'：是，'N'：否
20	DDL_AUTOCMT	CHAR(1)	DDL 语句是否自动提交。'Y'：是，'N'：否
21	RS_FOR_QRY	CHAR(1)	是否为非查询语句生成结果集。'Y'：是，'N'：否
22	CHK_NET	CHAR(1)	会话是否检查网络。'Y'：是，'N'：否
23	ISO_LEVEL	INTEGER	隔离级别。0：读未提交；1：读提交；2：可重复读；3：串行化
24	CLNT_HOST	VARCHAR(128)	客户端主机名
25	APPNAME	VARCHAR(128)	应用程序名
26	CLNT_IP	VARCHAR(128)	客户端 IP
27	OSNAME	VARCHAR(128)	客户端操作系统名
28	CONN_TYPE	VARCHAR(20)	连接类型
29	VPOOLADDR	BIGINT	内存池的首地址
30	RUN_STATUS	VARCHAR(20)	记录运行状态，可能值为"IDLE"或"RUNNING"
31	MSG_STATUS	VARCHAR(20)	记录消息处理状态，可能值为"RECEIVE"（已接受）或"SEND"（已发送）
32	LAST_RECV_TIME	DATETIME(6)	会话最近一次接收消息的时间
33	LAST_SEND_TIME	DATETIME(6)	会话最近一次发送消息的时间
34	DCP_FLAG	CHAR(1)	1）是否是通过 DCP_PORT 登录 DCP 服务器的会话，'Y'表示是，'N'表示否； 2）是否是通过 DCP 代理连接到 MPP 的会话，'Y'表示是，'N'表示否

35	THRD_ID	INTEGER	线程 ID
36	CONNECTED	INTEGER	会话连接是否正常：1 表示连接正常；0 表示连接已经关闭
37	PORT_TYPE	INTEGER	连接端口类型： 0 表示 TCP 连接； 1 表示 TCP 和 SSL 连接； 2 表示 NetIPC 连接； 3 表示连接已经关闭； 4 表示 UDP 连接； 5 表示 RDMA 连接； 6 表示 UNIXSOCKET 连接； 7 表示国密 GmSSL 连接
38	SRC_SITE	INTEGER	源节点，65535 表示是本地 SESSION，其他值代表 MPP 集群用户直接登录的节点号
39	MAL_ID	BIGINT	邮箱 ID
40	CONCURRENT_FLAG	INTEGER	是否占用了限流资源，1：是；0：否
41	CUR_LINENO	INTEGER	记录存储过程执行时，当前执行的行号
42	CUR_MTDNAME	VARCHAR(128)	记录存储过程执行时，当前执行的方法名
43	CUR_SQLSTR	VARCHAR(128)	记录存储过程执行时，当前执行的 SQL 的前 100 个字符
44	CLNT_VER	VARCHAR(128)	记录登录会话接口的程序的版本号
45	SQL_ID	INTEGER	SQL_TEXT 对应的语句 ID，为-1 表示会话当前没有执行 SQL，SQL_TEXT 中显示的 SQL 为上次执行的 SQL

			信息
46	EID	BIGINT	在 DPC 环境下会话的全局 ID
47	CLIENT_INFO	VARCHAR(64)	记录通过系统包方法 dbms_application_info.set_client_info('当前会话的客户端信息')设置的当前会话的客户端信息
48	CLIENT_IDENTIFIER	VARCHAR(64)	记录通过系统包方法 dbms_session.set_identifier('当前会话 ID')设置的当前会话 ID
49	MODULE	VARCHAR(64)	记录通过系统包方法 dbms_application_info.set_module()设置的当前会话运行的模块名称
50	ACTION	VARCHAR(64)	记录通过系统包方法 dbms_application_info.set_action('当前模块正在执行的事务名称')设置的当前模块正在执行的事务名称
51	HEART_BEAT_INTERVAL	INTEGER	会话的连接心跳间隔，单位：秒
52	HEART_BEAT_TIMEOUT	INTEGER	会话的连接心跳超时时间，单位：秒
53	TMP_USED_TYPE	VARCHAR(128)	记录会话中使用的临时表空间的类型（如 BTR、BLOB、MTAB、BACKUP 等），存在多个类型时，不同类型用"/"隔开（如 BTR/BLOB）
54	TMP_USED_EXTENT_NUM	INTEGER	记录会话中使用的临时表空间所占用的总簇数

55	SQL_TEXT_ID	VARCHAR(13)	会话当前正在执行的或最近一次执行的 SQL 语句固定的散列字符
56	LOGIN_PWD_FLAG	VARCHAR(8)	会话登录方式。 NORMAL：主密码登登录； OLDPWD：辅助密码登录； OTHERS：其他方式登录
57	VM_COUNT	INTEGER	会话的虚拟机个数

3） GV$SESSIONS

显示全局会话的具体信息，如执行的 sql 语句、主机名、当前会话状态、用户名等等。本视图中时间相关的字段均按服务器端进行记录。

支持的集群有 DPC、DSC、实时主备、读写分离集群、AFC。

序号	列	数据类型	说明
1	INSTANCE_NAME	VARCHAR(128)	实例名
2	SESS_ID	BIGINT	会话 ID，系统内部标识
3	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
4	SQL_TEXT	VARCHAR(1000)	对于会话当前正在执行的或最近一次执行的 SQL 语句，记录该 SQL 的前 1000 个字符
5	STATE	VARCHAR(8)	会话状态。共 6 种状态：CREATE 创建、STARTUP 启动、IDLE 空闲、ACTIVE 活动、PENDING 限流等待、UNKNOWN 正在释放
6	N_STMT	INTEGER	语句 STMT 的容量

7	N_USED_STMT	INTEGER	已使用的语句 STMT 数量
8	SEQ_NO	INTEGER	会话上语句的序列号
9	CURR_SCH	VARCHAR(128)	当前模式
10	USER_NAME	VARCHAR(128)	当前用户
11	TRX_ID	BIGINT	事务 id
12	CREATE_TIME	DATETIME(6)	会话创建时间
13	CLNT_TYPE	VARCHAR(128)	客户端类型，可能有如下取值：ODBC、SQL3、JDBC、.NET PROVIDER、PYTHON、GO、NODEJS、DCI、DCCI、PHP、UNKNOWN
14	TIME_ZONE	VARCHAR(6)	会话使用的时区
15	CHK_CONS	CHAR(1)	会话是否忽略约束检查。'Y'：忽略约束检查，'N'：不忽略
16	CHK_IDENT	CHAR(1)	会话是否忽略自增列指定列表检查。'Y'：忽略约束检查，'N'：不忽略
17	RDONLY	CHAR(1)	是否为只读会话。'Y'：是，'N'：否
18	INS_NULL	CHAR(1)	当列不存在 default 选项，是否插入空值。'Y'：是，'N'：否
19	COMPILE_FLAG	CHAR(1)	是否忽略编译选项，用于 VIEW、PROCEDURE、TRIGGER 等对象的编译。'Y'：是，'N'：否
20	AUTO_CMT	CHAR(1)	会话是否为自动提交。'Y'：是，'N'：否
21	DDL_AUTOCMT	CHAR(1)	DDL 语句是否自动提交。'Y'：是，'N'：否
22	RS_FOR_QRY	CHAR(1)	是否为非查询语句生成结果集。'Y'：是，'N'：否
23	CHK_NET	CHAR(1)	会话是否检查网络。'Y'：是，'N'：否

24	ISO_LEVEL	INTEGER	隔离级别。0：读未提交；1：读提交；2：可重复读；3：串行化
25	CLNT_HOST	VARCHAR(128)	客户端主机名
26	APPNAME	VARCHAR(128)	应用程序名
27	CLNT_IP	VARCHAR(128)	客户端 IP
28	OSNAME	VARCHAR(128)	客户端操作系统名
29	CONN_TYPE	VARCHAR(20)	连接类型
30	VPOOLADDR	BIGINT	内存池的首地址
31	RUN_STATUS	VARCHAR(20)	记录运行状态，可能值为"IDLE"或"RUNNING"
32	MSG_STATUS	VARCHAR(20)	记录消息处理状态，可能值为"RECEIVE"（已接受）或"SEND"（已发送）
33	LAST_RECV_TIME	DATETIME(6)	会话最近一次接收消息的时间
34	LAST_SEND_TIME	DATETIME(6)	会话最近一次发送消息的时间
35	DCP_FLAG	CHAR(1)	1）是否是通过 DCP_PORT 登录 DCP 服务器的会话，'Y'表示是，'N'表示否； 2）是否是通过 DCP 代理连接到 MPP 的会话，'Y'表示是，'N'表示否
36	THRD_ID	INTEGER	线程 ID
37	CONNECTED	INTEGER	会话连接是否正常：1 表示连接正常，0 表示连接已经关闭
38	PORT_TYPE	INTEGER	连接端口类型：2 表示 TCP 连接，12 表示端口已经关闭，13 表示 UDP 连接
39	SRC_SITE	INTEGER	源节点，65535 表示本地 SESSION，其他值代表 MPP

			集群用户直接登录的节点号
40	MAL_ID	BIGINT	邮箱 ID
41	CONCURRENT_FLAG	INTEGER	是否占用了限流资源，1：是；0：否
42	CUR_LINENO	INTEGER	记录存储过程执行时，当前执行的行号
43	CUR_MTDNAME	VARCHAR(128)	记录存储过程执行时，当前执行的方法名
44	CUR_SQLSTR	VARCHAR(128)	记录存储过程执行时，当前执行的 SQL 的前 100 个字符
45	CLNT_VER	VARCHAR(128)	记录登录会话接口的程序的版本号
46	SYSDATE	DATETIME(0)	数据库服务器所在操作系统的当前日期时间

4） V$SESSION_SYS

显示系统当前会话的一些状态统计信息。

序号	列	数据类型	说明
1	N_SESS	INTEGER	会话个数
2	N_FREEING	INTEGER	正在释放的会话个数
3	ALLOW_NEW_LOGIN	INTEGER	是否允许登录
4	MAX_CONCURRENT_TRX	INTEGER	限流的最大并行数
5	CONCURRENT_TRX_MODE	INTEGER	预留参数，暂无意义
6	CURR_CONCURRENT_TRX	INTEGER	可用资源
7	N_WAIT	INTEGER	等待执行的任务数

5） GV$SESSION_SYS

显示系统全局会话的一些状态统计信息。

支持的集群有 DPC、DSC、实时主备、读写分离集群、AFC。

序号	列	数据类型	说明

1	INSTANCE_NAME	VARCHAR(128)	实例名
2	N_SESS	INTEGER	会话个数
3	N_FREEING	INTEGER	正在释放的会话个数
4	ALLOW_NEW_LOGIN	INTEGER	是否允许登录
5	MAX_CONCURRENT_TRX	INTEGER	限流的最大并行数
6	CONCURRENT_TRX_MODE	INTEGER	预留参数，暂无意义
7	CURR_CONCURRENT_TRX	INTEGER	可用资源
8	N_WAIT	INTEGER	等待执行的任务数

6） V$OPEN_STMT

连接语句句柄表，用于记录 SESSION 上语句句柄的信息。

序号	列	数据类型	说明
1	SESS_ADDR	VARBINARY(8)	SESSION 地址
2	SESSION#	INTEGER	SESSION 的 ID
3	STMT_ID	INTEGER	STMT 的 ID
4	STMT_SEQNO	INTEGER	STMT 的执行序号
5	TYPE	VARCHAR(32)	语句类型
6	REP_TYPE	VARCHAR(32)	复制语句类型
7	EXEC_DIRECT	CHAR(1)	是否直接执行
8	PLN_ADDR	VARBINARY(8)	执行计划的地址
9	ROWCOUNT	BIGINT	影响记录的行数
10	ROWCOUNT_FOR_DML	BIGINT	DML 影响的行数
11	ROWCOUNT_REQUEST	BIGINT	请求的行数
12	BATCH_FLAG	CHAR(1)	批量处理标识

13	N_PARA_ROWS	INTEGER	多行参数时的行数
14	NTH_PARA_ROW	INTEGER	多行参数时的当前行序号
15	CUR_PARA_OFF	INTEGER	参数的当前偏移
16	PARA_BUF	VARBINARY(8)	参数的内存地址
17	SQL_STR	VARCHAR(1000)	执行语句的前 1000 个字符
18	BPARAM_CAN_OPT	CHAR(1)	绑定批量参数时是否可优化
19	CURSOR_FORWARD_ONLY	CHAR(1)	对应游标是否 FORWARD_ONLY
20	CURSOR_NAME	VARCHAR(128)	对应游标名
21	CURSOR_KEEP_NAME	CHAR(1)	对应游标是否保存名字
22	CURSOR_REF_ID	INTEGER	引用游标语句句柄 ID
23	CURSOR_REF_SEQNO	INTEGER	引用语句的执行序号
24	MPP_EXEC_ID	BIGINT	MPP 下的执行 ID
25	VM_ADDR	VARBINARY(8)	VM 地址
26	DBG_FLAG	CHAR(1)	是否 DEBUG
27	STMT_DBG	VARBINARY(8)	DEBUG 的语句句柄地址
28	MPP_DDL_FLAG	CHAR(1)	MPP 下 DDL 标识
29	REMOTE_OPERATION	CHAR(1)	是否是 DBLINK 的远程操作
30	RS_BDTA_FLAG	CHAR(1)	返回的结果集是否直接将 BDTA 打包
31	RS_BDTA_SIZE	INTEGER	返回的打包结果集时最大的行数
32	STMT_LINK	VARBINARY(8)	用于串连语句的链表地址
33	LPQ	VARBINARY(8)	并行查询地址
34	RS_INFO	VARBINARY(8)	结果集地址
35	MPLN_ADDR	VARBINARY(8)	从 EP 上保留 MPLN

36	BIND_PARAM	VARBINARY(8)	绑定参数地址
37	BAK_SYS	VARBINARY(8)	备份链表信息
38	RT_STACK	VARBINARY(8)	MPP 从 EP 的运行时信息堆栈

7) V$SESSION_HISTORY

显示当前会话历史的记录信息，如主机名、用户名等，与 V$SESSIONS 的区别在于会话历史记录只记录了会话一部分信息，对于一些动态改变的信息没有记录，如执行的 SQL 语句等。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID，系统内部标识
2	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
3	CURR_SCH	VARCHAR(128)	当前模式
4	USER_NAME	VARCHAR(128)	当前用户
5	CREATE_TIME	DATETIME(6)	会话创建时间
6	CLNT_TYPE	VARCHAR(128)	客户类型，可能有如下取值：ODBC、SQL3、JDBC、.NET PROVIDER、PYTHON、GO、NODEJS、DCI、DCCI、PHP、UNKNOWN
7	TIME_ZONE	VARCHAR(6)	时区
8	RDONLY	CHAR(1)	是否是只读会话
9	DDL_AUTOCMT	CHAR(1)	DDL 语句是否自动提交
10	RS_FOR_QRY	CHAR(1)	非查询语句生成结果集标记
11	CHK_NET	CHAR(1)	是否检查网络

12	CLNT_HOST	VARCHAR(128)	客户主机名
13	APPNAME	VARCHAR(128)	应用程序名
14	CLNT_IP	VARCHAR(128)	客户端 IP
15	OSNAME	VARCHAR(128)	客户端操作系统名
16	CONN_TYPE	VARCHAR(20)	连接类型

8) GV$SESSION_HISTORY

显示全局会话历史的记录信息，如主机名、用户名等，与 V$SESSIONS 的区别在于会话历史记录只记录了会话一部分信息，对于一些动态改变的信息没有记录，如执行的 SQL 语句等。

支持的集群有 DPC、DSC、实时主备、读写分离集群、AFC。

序号	列	数据类型	说明
1	INSTANCE_NAME	VARCHAR(128)	实例名
2	SESS_ID	BIGINT	会话 ID，系统内部标识
3	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
4	CURR_SCH	VARCHAR(128)	当前模式
5	USER_NAME	VARCHAR(128)	当前用户
6	CREATE_TIME	DATETIME(6)	会话创建时间
7	CLNT_TYPE	VARCHAR(128)	客户类型，可能有如下取值：ODBC、SQL3、JDBC、.NET PROVIDER、PYTHON、GO、NODEJS、DCI、DCCI、PHP、UNKNOWN
8	TIME_ZONE	VARCHAR(6)	时区

9	RDONLY	CHAR(1)	是否是只读会话
10	DDL_AUTOCMT	CHAR(1)	DDL 语句是否自动提交
11	RS_FOR_QRY	CHAR(1)	非查询语句生成结果集标记
12	CHK_NET	CHAR(1)	是否检查网络
13	CLNT_HOST	VARCHAR(128)	客户主机名
14	APPNAME	VARCHAR(128)	应用程序名
15	CLNT_IP	VARCHAR(128)	客户端 IP
16	OSNAME	VARCHAR(128)	客户端操作系统名
17	CONN_TYPE	VARCHAR(20)	连接类型

9) V$CONTEXT

显示当前会话所有上下文的名字空间、属性和值。

序号	列	数据类型	说明
1	NAMESPACE	VARCHAR(30)	上下文名字空间
2	ATTRIBUTE	VARCHAR(30)	名字空间属性
3	VALUE	VARCHAR(4000)	属性值

10) V$NLS_PARAMETERS

显示当前会话的日期时间格式、日期时间语言以及自然语言的排序方式。

序号	列	数据类型	说明
1	PARAMETER	VARCHAR(128)	当前会话的属性：目前只支持日期串语言、日期串格式以及自然语言的排序方式
2	VALUE	VARCHAR(128)	当前会话的属性值

在没有设置 NLS_*_FORMAT 情况下，显示为 NULL，则说明使用默认格式（无会话格

式、会话格式没有默认格式）。DM 默认格式为：DATE 格式是"YYYY-MM-DD"；TIME 格式是"HH24:MI:SS.FF"；TIMESTAMP 格式是"YYYY-MM-DD HH24:MI:SS.FF"；TIMESTAMP WITH TIME ZONE 格式是"YYYY-MM-DD HH24:MI:SS.FF TZH:TZM"；TIME WITH TIME ZONE 格式是"HH24:MI:SS.FF TZH:TZM"。

默认格式的解析逻辑为：上述格式中的分、秒、毫秒、时区分量都会依次尝试解析，都允许从后往前完整省略后续分量，直到最后一个分量“时区”或整个源串解析完成。分钟及其之后的分量可以一起省略。若源串解析完对应分量后，仍存在未解析的冗余字符，则根据 COMPATIBLE_MODE 的值判断是否进行报错：当 COMPATIBLE_MODE=0（达梦默认）或者 2（兼容 ORACLE）时直接报错；当 COMPATIBLE_MODE=4（兼容 MYSQL）或者 7（兼容 PG）时成功。

11） V$ACTIVE_SESSION_HISTORY

当 INI 参数 ENABLE_MONITOR=1 时，显示当前活动会话的等待信息，保留 1 万条数据。

序号	列	数据类型	说明
1	SAMPLE_ID	BIGINT	采样ID
2	SAMPLE_TIME	DATETIME(3)	采样时戳
3	IS_AWR_SAMPLE	VARCHAR(1)	是否已经刷盘或者即将刷盘(Y/N)
4	SESSION_ID	BIGINT	会话ID
5	SESSION_SERIAL#	INTEGER	会话序列号，用来唯一标识会话
6	SESSION_TYPE	VARCHAR(10)	FOREGROUND/BACKGROUND FOREGROUND表示用户创建的会话; BACKGROUND表示系统内部创建的会话
7	USER_ID	INTEGER	当前用户ID

8	SQL_ID	INTEGER	采样时正在执行的SQL语句ID
9	IS_SQLID_CURRENT	VARCHAR(1)	SQL_ID列所标识的语句是否正在执行(Y/N)
10	SQL_OPCODE	INTEGER	SQL语句类型
11	SQL_OPNAME	VARCHAR(64)	SQL语句类型名
12	TOP_LEVEL_SQL_ID	INTEGER	顶层SQL语句ID
13	TOP_LEVEL_SQL_OPCODE	INTEGER	顶层SQL语句类型
14	SQL_EXEC_ID	BIGINT	SQL执行ID
15	EVENT	VARCHAR(64)	如果SESSION_STATE=WAITING,则表示会话在采样时的等待事件。否则为NULL
16	EVENT_ID	SMALLINT	采样时等待事件的 ID
17	EVENT#	SMALLINT	会话正在等待或者会话上次等待的资源或事件编号
18	P1TEXT	VARCHAR(64)	等待事件对应的参数 1 说明
19	P1	VARCHAR(256)	等待事件对应的参数 1 值
20	P2TEXT	VARCHAR(64)	等待事件对应的参数 2 说明
21	P2	BIGINT	等待事件对应的参数 2 值
22	P3TEXT	VARCHAR(64)	等待事件对应的参数 3 说明
23	P3	BIGINT	等待事件对应的参数 3 值
24	P4TEXT	VARCHAR(64)	等待事件对应的参数 4 说明
25	P4	BIGINT	等待事件对应的参数 4 值
26	WAIT_CLASS	VARCHAR(64)	等待会话在采样时等待的事件的类名
27	WAIT_CLASS_ID	INTEGER	等待会话在采样时等待的事件的类 ID
28	WAIT_TIME	BIGINT	如果是在 SESSION_STATE=ON CPU 采样，则

			表示会话上次等待的事件的总等待时间，单位微妙，否则为 0
29	SESSION_STATE	VARCHAR(7)	WAITING/ON CPU
30	TIME_WAITED	BIGINT	如果 SESSION_STATE=WAITING，则表示会话实际花费在等待该事件的时间，单位微秒
31	TRX_ID	BIGINT	采样时的事务 ID
32	PROGRAM	VARCHAR(128)	程序名
33	MODULE	VARCHAR(64)	采样时执行模块的名称（DBMS_APPLICATION_INFO.SET_MODULE）
34	ACTION	VARCHAR(64)	采样时执行模块的动作（DBMS_APPLICATION_INFO.SET_ACTION）
35	CLIENT_ID	VARCHAR(64)	客户端标识
36	MACHINE	VARCHAR(128)	客户端的机器名
37	PORT	INTEGER	客户端的端口号

8. SQL 执行相关

1) V$SQL_HISTORY

当 INI 参数 ENABLE_MONITOR=1 时，显示执行 SQL 的历史记录信息；可以方便用户经常使用的记录进行保存。

序号	列	数据类型	说明
1	SEQ_NO	INTEGER	序列号
2	SQL_ID	INTEGER	SQL 语句 ID

3	SESS_ID	BIGINT	会话 ID，系统内部标识
4	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
5	TRX_ID	BIGINT	事务 ID
6	THREAD_ID	BIGINT	线程 ID
7	TOP_SQL_TEXT	VARCHAR(1000)	栈帧中第一个 SQL
8	SEC_SQL_TEXT	VARCHAR(1000)	栈帧中第二个 SQL
9	THRD_SQL_TEXT	VARCHAR(1000)	栈帧中第三个 SQL
10	START_TIME	DATETIME(6)	SQL 执行的起始时间
11	TIME_USED	BIGINT	SQL 执行所使用时间（usec），单位为微秒（us）
12	IS_OVER	CHAR(1)	是否结束
13	EXEC_ID	BIGINT	SQL 执行 ID
14	VM	BIGINT	执行 SQL 的虚拟机地址
15	STKFRM	BIGINT	当前的栈帧
16	STK_LEVEL	INTEGER	当前栈帧的级别
17	BYTES_DYNAMIC_ALLOCED	BIGINT	动态分配字节数
18	BYTES_DYNAMIC_FREED	BIGINT	动态释放字节数
19	CUR_SQL_NODE	BIGINT	当前的 SQL 节点地址
20	MAL_ID	BIGINT	邮件标识号
21	N_LOGIC_READ	INTEGER	语句逻辑读的次数
22	N_PHY_READ	INTEGER	语句物理读的次数
23	AFFECTED_ROWS	INTEGER	语句影响的行数

24	HARD_PARSE_FLAG	INTEGER	语句硬解析标记，0：软解析；1：语义解析；2：硬解析
25	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同
26	COMMAND_TYPE	INTEGER	SQL 语句的类型
27	IS_SQC	INTEGER	是否为 DPC 子任务，0：否；1：是

2) V$SQL_NODE_HISTORY

通过该视图既可以查询 SQL 执行节点信息，包括 SQL 节点的类型、进入次数和使用时间等等；又可以查询所有执行的 SQL 节点执行情况，如哪些使用最频繁、耗时多少等。

当 INI 参数 ENABLE_MONITOR 和 MONITOR_SQL_EXEC 都开启时，才会记录 SQL 执行节点信息。

序号	列	数据类型	说明
1	SEQ_NO	INTEGER	操作符在计划中的序号，即 explain 显示执行计划时的行号
2	EXEC_ID	BIGINT	执行 ID
3	NODE	BIGINT	节点 ID
4	TYPE$	INTEGER	节点类型
5	BYTES_DYNAMIC_ALLOCED	BIGINT	动态分配字节数
6	BYTES_DYNAMIC_FREED	BIGINT	动态释放字节数
7	N_ENTER	INTEGER	节点进入次数
8	TIME_USED	BIGINT	节点执行所耗时，单位为微秒
9	PLN_OP_ID	INTEGER	MPP 模式下，节点所属通讯操作符中的序号

10	BYTES_SEND	BIGINT	发送的字节数，仅在 MPP 环境下有效
11	BYTES_RECV	BIGINT	接收的字节数，仅在 MPP 环境下有效
12	ROWS_SEND	BIGINT	发送的行数，仅在 MPP 环境下有效
13	ROWS_RECV	BIGINT	接收的行数，仅在 MPP 环境下有效
14	BDTA_SEND	BIGINT	发送 BDTA 的次数，仅在 MPP 环境下有效
15	BDTA_RECV	BIGINT	接收 BDTA 的次数，仅在 MPP 环境下有效
16	MAL_ID	BIGINT	邮件标识号
17	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同
18	HASH_USED_CELLS	BIGINT	哈希表上使用的槽数
19	HASH_CONFLICT	BIGINT	哈希表上每个槽的冲突的个数
20	STASK_NO	INTEGER	节点所在的子任务号
21	THRD_NO	INTEGER	节点所在的线程号
22	MEM_USED	BIGINT	操作符使用的内存空间，单位 KB
23	DISK_USED	BIGINT	操作符预期使用的磁盘空间，单位 KB
24	DHASH3_USED_CELLS	VARCHAR(500)	动态哈希表中使用的槽数
25	DHASH3_CONFLICT	VARCHAR(300)	动态哈希表中的冲突情况
26	HASH_SAME_VALUE	BIGINT	开启哈希相同值挂链优化时，记录哈希表中相同值的总数，唯一值不纳入计数。例如：哈希表中存在一个 0 和两个相同值链 1—1—1、3—3，此时 HASH_SAME_VALUE=5
27	SORT_BUF_ROWS	BIGINT	排序操作符的排序缓冲区存放的总行数（关闭大内存排序时）或每路排序的行数（开启大内

			存排序时）
28	SORT_BUF_AVG_ROWSIZE	BIGINT	排序操作符的排序缓冲区存放数据的预估平均行长，单位 Byte

3） V$SQL_PARSE_HISTORY

通过该视图可以查看 SQL 查询分析过程中的优化信息。

当 INI 参数 ENABLE_MONITOR 和 MONITOR_SQL_PARSE 都开启时，才会记录查询分析过程中的优化信息。

序号	列	数据类型	说明
1	SEQ_NO	INTEGER	序列号
2	SQL_ID	INTEGER	监控语句的 SQL 语句 ID
3	SESS_ID	BIGINT	会话 ID
4	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
5	TRX_ID	BIGINT	事务 ID
6	THREAD_ID	BIGINT	线程 ID
7	SQL_TEXT	VARCHAR(256)	原始 SQL 内容
8	PHASE_FLAG	VARCHAR(10)	优化发生阶段（pha、phb、pha/phb）
9	OPTIMIZE_CLASS	VARCHAR(64)	优化类别
10	OPTIMIZE_TYPE	INTEGER	优化类型
11	OPT_DESCRIPTION_EN	VARCHAR(256)	优化类型的英文描述
12	OPT_DESCRIPTION_CN	VARCHAR(256)	优化类型的中文描述
13	INI_PARA	VARCHAR(256)	相关 INI 参数及决定优化的相关取值
14	OPT_MONITOR_TEXT	VARCHAR(256)	监控优化内容或释义

4） V$SQL_NODE_NAME

显示所有的 sql 节点描述信息，包括 sql 节点类型、名字和详细描述。

序号	列	数据类型	说明
1	TYPE$	INTEGER	节点类型
2	NAME	VARCHAR(24)	节点的名字
3	DESC_CONTENT	VARCHAR(128)	节点的详细描述
4	VERSION	SMALLINT	节点的版本号

5) V$COSTPARA

显示代价估算各项指标的基准值。

序号	列	数据类型	说明
1	RAN_IO_R_COST	DEC(5,3)	随机读代价基准值
2	RAN_IO_W_COST	DEC(5,3)	随机写代价基准值
3	SEQ_IO_R_COST	DEC(5,3)	顺序读代价基准值
4	SEQ_IO_W_COST	DEC(5,3)	顺序写代价基准值
5	SEL_RATE_EQU	DEC(5,3)	等于谓词的选择率基准值
6	SEL_RATE_SINGLE	DEC(5,3)	非等于谓词的选择率基准值
7	INDEX_JOIN_HIT_RATIO	DEC(5,3)	索引连接命中率基准值

6) V$LONG_EXEC_SQLS

当 INI 参数 ENABLE_MONITOR=1 时，显示系统最近 n 条执行时间超过预定值的 SQL 语句。n 由 INI 参数 LONG_EXEC_SQLS_CNT 指定。默认预定值为 1000 毫秒，可通过 SP_SET_LONG_TIME 系统函数修改，通过 SF_GET_LONG_TIME 系统函数查看当前值。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID，系统内部标识

2	SQL_ID	INTEGER	SQL 语句 ID
3	SQL_TEXT	VARCHAR(1024)	SQL 文本
4	EXEC_TIME	BIGINT	执行时间，单位毫秒
5	FINISH_TIME	DATETIME(0)	执行结束时间
6	N_RUNS	INTEGER	执行次数
7	SEQNO	INTEGER	编号
8	TRX_ID	BIGINT	事务号
9	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
10	EXEC_ID	INTEGER	SQL 执行 ID，对应 V$SQL_HISTORY 的 EXEC_ID
11	USER_ID	INTEGER	用户 ID
12	SCH_ID	INTEGER	模式 ID

7) V$SYSTEM_LONG_EXEC_SQLS

当 INI 参数 ENABLE_MONITOR=1 时，显示系统自启动以来执行时间最长的 n 条 SQL 语句，不包括执行时间低于预定值的语句。n 由 INI 参数 SYSTEM_LONG_EXEC_SQLS_CNT 指定。默认预定值为 1000 毫秒，可通过 SP_SET_LONG_TIME 系统函数修改，通过 SF_GET_LONG_TIME 系统函数查看当前值。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID，系统内部标识
2	SQL_ID	INTEGER	SQL 语句 ID
3	SQL_TEXT	VARCHAR(1024)	SQL 文本
4	EXEC_TIME	BIGINT	执行时间，单位毫秒

5	FINISH_TIME	DATETIME(0)	执行结束时间
6	N_RUNS	INTEGER	执行次数
7	SEQNO	INTEGER	编号
8	TRX_ID	BIGINT	事务号
9	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
10	MIN_EXEC_TIME	BIGINT	该 SQL 最小执行时间，单位毫秒
11	MAX_EXEC_TIME	BIGINT	该 SQL 最大执行时间，单位毫秒
12	TOTAL_EXEC_TIME	BIGINT	该 SQL 累计执行时间，单位毫秒
13	EXEC_ID	INTEGER	SQL 执行 ID，对应 V$SQL_HISTORY 的 EXEC_ID
14	USER_ID	INTEGER	用户 ID
15	SCH_ID	INTEGER	模式 ID

8） V$VMS

显示虚拟机信息。

序号	列	数据类型	说明
1	ID	INTEGER	虚拟机 ID
2	TRX_ID	BIGINT	事务 ID
3	STMT_ID	INTEGER	语句 ID
4	EXP_FLAG	CHAR(1)	表达式标识
5	VSTACK_SIZE	INTEGER	栈大小，单位 8*BYTE
6	VSTACK	BIGINT	栈基址
7	VTOP	INTEGER	栈顶离基址的距离
8	VUSED	INTEGER	栈已用空间的大小，单位

			8*BYTE
9	MEMOBJ	BIGINT	内存地址
10	STKFRM_DEPTH	INTEGER	栈深度
11	FREE_STKFRMS	INTEGER	已释放的栈
12	CURR_FRM	BIGINT	当前栈
13	IP	BIGINT	IP 地址
14	RT_HEAP	BIGINT	DMSQL 程序所使用的堆
15	SQL_NO	INTEGER	SQL 语句树的序号
16	RS_SEQ_NO	INTEGER	结果集序号
17	ECPT_CODE	INTEGER	返回的 CODE
18	ERR_DESC	VARCHAR(256)	错误描述
19	ROW_AFFECHED	BIGINT	结果集的行数
20	SQL_TYPE	INTEGER	SQL 语句类型
21	N_FUNS	INTEGER	已使用函数的数目
22	SESS_ID	BIGINT	会话 ID，系统内部标识
23	MEM_ADDR	BIGINT	虚拟机使用的 MEM_POOL 地址

9） V$STKFRM

显示虚拟机栈桢信息。该视图必须在 INI 参数 ENABLE_MONITOR 和 MONITOR_SQL_EXEC 都开启时才有信息。

序号	列	数据类型	说明
1	VM_ID	INTEGER	虚拟机 ID
2	FRAME_LEVEL	INTEGER	栈的层次
3	ADDR	BIGINT	地址

4	RET_IP	BIGINT	返回的指令地址
5	CURR_METHOD	BIGINT	当前方法
6	LOCAL_SPACE	INTEGER	临时变量空间大小，单位是栈帧的个数
7	ARG_SPACE	INTEGER	参数空间大小，单位是栈帧的个数
8	OFF_LOCALS	BIGINT	每个变量基于原栈顶的偏移
9	OFF_ARGS	BIGINT	每个参数基于原栈顶的偏移
10	CURR_BP	BIGINT	新的栈顶
11	IP_BASE	BIGINT	指令基址
12	SQL_VM_NODE	BIGINT	VM 地址
13	SQL_TRX_IDS	BIGINT	事务 ID 地址
14	SYSTEM_FLAG	CHAR(1)	系统标记
15	FRAME_SIZE	INTEGER	栈的大小，单位 BYTE

10）V$STMTS

显示当前活动会话的最近的语句的相关信息。

序号	列	数据类型	说明
1	STMT_ID	INTEGER	语句 ID，从 iid 页取得
2	TYPE$	INTEGER	语句类型，供系统内部使用
3	EXEC_DIRECT	CHAR(1)	是否立即执行标记
4	SESS_ID	BIGINT	对应的会话 ID，系统内部标识
5	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
6	PLN	BIGINT	查询计划地址

7	DML_RCNT	BIGINT	DML 操作所影响的行数
8	REQ_RCNT	BIGINT	prepare 或 execute 查询要求返回的行数
9	BATCH_FLAG	CHAR(1)	是否批量绑定参数
10	N_PARA_ROWS	INTEGER	多行参数时的行数目
11	NTH_PARA_ROW	INTEGER	多行参数时的当前行序号
12	CUR_PARA_OFF	INTEGER	参数的当前偏移
13	PARA_BUF	BIGINT	参数的内存地址
14	SQL_TEXT	VARCHAR(1000)	SQL 语句
15	CURSOR_FORWARD_ONLY	CHAR(1)	是否是只能向前移动的游标
16	CURSOR_NAME	VARCHAR(128)	游标名字
17	CURSOR_KEEP_NAME	CHAR(1)	是否保存游标名字
18	CURSOR_REF_ID	INTEGER	引用的游标语句句柄
19	CURSOR_REF_SEQNO	INTEGER	引用语句的执行序号
20	EXEC_ID	INTEGER	语句的执行号

11) V$SQL_PLAN_NODE

当 INI 参数 ENABLE_MONITOR 和 MONITOR_SQL_EXEC 都开启时，显示执行计划的节点信息。

序号	列	数据类型	说明
1	PLN_ADDR	VARBINARY(8)	计划地址
2	SQL_ID	INTEGER	SQL 语句 ID（唯一标识）
3	NODE_ADDR	VARBINARY(8)	节点地址
4	PARENT	VARBINARY(8)	父节点地址
5	LEFT_CHILD	VARBINARY(8)	左孩子地址

6	RIGHT_CHILD	VARBINARY(8)	右孩子地址
7	HAS_INVOKE	CHAR(1)	是否包含函数
8	OPERATION	VARCHAR(30)	操作符名称
9	OPTIONS	VARCHAR(30)	扫描类型
10	TABLE#	INTEGER	表 ID
11	TABLE_NAME	VARCHAR(128)	表名
12	TABLE_ALIAS	VARCHAR(128)	表别名
13	INDEX_NAME	VARCHAR(128)	索引名称
14	SCHID	INTEGER	模式 ID
15	DEPTH	INTEGER	节点层次
16	CARD	BIGINT	行数
17	BYTES	BIGINT	字节数
18	COST	BIGINT	代价
19	CPU_COST	BIGINT	CPU 代价
20	IO_COST	BIGINT	IO 代价，暂不支持，始终为 0
21	SCAN_RANGE	VARCHAR(128)	扫描范围
22	FILTER	VARCHAR(1000)	过滤表达式
23	JOIN_COND	VARCHAR(1000)	连接表达式

12) V$SQL_SUBPLAN

显示子计划信息。

序号	列	数据类型	说明
1	PLN_ADDR	VARBINARY(8)	计划地址
2	SQL_ID	INTEGER	SQL 语句 ID(唯一标识)

3	SUBPLN	VARBINARY(8)	子计划地址(SUBPLN 在 V$SQL_PLAN_DCTREF 中即为 PLN_ADDR)

13) V$SQL_PLAN_DCTREF

显示所有执行计划相关的详细字典对象信息。

序号	列	数据类型	说明
1	PLN_ADDR	VARBINARY(8)	计划地址
2	SQL_ID	INTEGER	SQL 语句 ID（唯一标识）
3	TYPE	VARCHAR(16)	字典对象类型（DATABASE、TABLE、VIEW、INDEX、USER、ROLE、PROCEDURE、TRIGGER、CONSTRAINT、SCHEMA、SEQUENCE、LOGIN、DBLINK、SYSROLE、PACKAGE、OBJECT、SYNOM、CRYPTION、CIPHER、IDENTITY、SYS PRIVILEGE、OBJ PRIVILEGE、POLICY、RULE、COLUMN、DOMAIN、CHARSET、COLLATION、CONTEXT INDEX、USER LOGIN、REGEXP REWRITE、NORMAL REWRITE、UNKNOWN)
4	STATUS	VARCHAR(16)	字典对象状态。共四种状态(NORMAL、CREATING、ALTERING、DROPPING)，本视图中的执行计划只涉及 DML 语句的 NORMAL 状态
5	R_SIZE	INTEGER	字典对象大小，单位字节
6	VERSION	INTEGER	字典对象版本号
7	NAME	VARCHAR(128)	字典对象名称
8	SCHID	INTEGER	模式 ID
9	ID	INTEGER	字典对象 ID

10	UID	INTEGER	用户 ID
11	TS_ID	INTEGER	表空间 ID

14) V$MTAB_USED_HISTORY

显示系统自启动以来使用 MTAB 空间最多的 500 个操作符信息。当操作符使用 MTAB 空间大于等于 1MB 时才会记录到该视图中。

序号	列	数据类型	说明
1	EXEC_ID	BIGINT	语句的执行 ID
2	SQL_TEXT	VARCHAR(1024)	SQL 文本
3	STK_LEVEL	INTEGER	当前栈帧的级别
4	CUR_SQL_NODE	BIGINT	当前的 SQL 节点
5	OP_SEQ_NO	INTEGER	操作符在执行计划中序号
6	OP_TYPE$	INTEGER	操作符节点类型
7	MTAB_USED_BY_M	INTEGER	操作符使用的 MTAB 空间，单位 MB
8	MEM_USED_BY_M	INTEGER	MTAB 使用的内存空间，单位 MB
9	MMT_FILE_ALLOC	INTEGER	操作符使用 MMT 时以 MMT_SIZE 为大小分配的 MMT 文件个数
10	MMT_PAGES_USED	INTEGER	操作符使用 MMT 时实际使用的页数
11	MTAB_TYPE	VARCHAR(10)	MTAB 类型: FLUSH: 将 ESEND 操作符，TOP 操作符，HUGE 表插入，分析函数，层次查询，闪回查询，哈希连接，归并连接，嵌套连接，哈希表，临时表，动态表、动态视图、等待事件，排序相关操作使用的数据缓冲区写入

			MTAB 空间; DESTORY: 释放 MTAB 空间; SORT: 排序空间不足时将排序缓冲区数据写入 MTAB 空间
12	MAL_ID	BIGINT	邮件标识号
13	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同
14	START_TIME	DATETIME(0)	操作符信息创建的时间(精确到秒)
15	END_TIME	DATETIME(0)	操作符信息最后一次更新的时间(精确到秒)

15) V$SORT_HISTORY

当 INI 参数 ENABLE_MONITOR=1 都打开时，显示系统自启动以来使用排序页数最多的 50 个操作符信息。

序号	列	数据类型	说明
1	EXEC_ID	BIGINT	语句的执行 ID
2	SQL_TEXT	VARCHAR(1024)	SQL 文本
3	STK_LEVEL	INTEGER	当前栈帧的级别
4	CUR_SQL_NODE	BIGINT	当前的 SQL 节点
5	OP_SEQ_NO	INTEGER	操作符在执行计划中序号
6	OP_TYPE$	INTEGER	操作符节点类型
7	N_PAGES	INTEGER	排序使用的页数
8	N_MERGE_SORT	INTEGER	归并排序的趟数
9	SORT_BYTES	INTEGER	参与排序的每行字节数（估算值）

10	SORT_ROWS	BIGINT	参与排序的行数
11	MAL_ID	BIGINT	邮件标识号
12	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同

16）V$HASH_MERGE_USED_HISTORY

HASH MERGE 连接操作符使用的缓存信息。

序号	列	数据类型	说明
1	TYPE$	INTEGER	节点类型
2	SEQNO	INTEGER	序号
3	EXEC_ID	BIGINT	执行 ID
4	SQL_TEXT	VARCHAR(1000)	执行 SQL 文本
5	MERGE_USED	INTEGER	HASH MERGE 使用的缓存空间，单位 MB
6	MAL_ID	BIGINT	邮件标识号
7	MPP_EXEC_ID	BIGINT	MPP 环境 SQL 执行号

17）V$PLSQL_DDL_HISTORY

记录 DMSQL 程序中执行的 DDL 语句，主要监控 truncate table 和 Execute immediate DDL 语句的情况。

序号	列	数据类型	说明
1	SEQNO	INTEGER	该类语句的执行编号
2	SESS_ID	BIGINT	执行会话 ID，系统内部标识
3	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号

4	TRX_ID	BIGINT	执行事务 ID
5	EXEC_ID	BIGINT	虚拟机的执行序号
6	VM_ID	INTEGER	虚拟机 ID
7	STKFRM_LEVEL	INTEGER	当前栈帧在虚拟机的层次
8	DDL_FROM	VARCHAR(10)	若为 TRUNCATE TABLE，则为'TRUNC'，否则为'EXEC'
9	SQL_TEXT	VARCHAR(1000)	获得该层栈帧执行的 SQL 语句，若字符长度超过 1000，则取前 1000 个字符。
10	DDL_TIME	DATETIME(6)	语句执行时间
11	MAIL_ID	BIGINT	邮件标识号
12	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同

18）V$PRE_RETURN_HISTORY

记录大量数据返回结果集的历史信息（查询大量数据产生）。当 INI 参数 ENABLE_MONITOR 开启时，显示该视图信息；当数据返回的结果集大于等于 INI 参数 FIRST_ROWS 时，才会记录到该视图中。

序号	列	数据类型	说明
1	SEQNO	INTEGER	编号
2	TRX_ID	BIGINT	事务 ID
3	VM_ID	INTEGER	虚拟机 ID
4	EXEC_ID	BIGINT	执行 ID，记录执行的次数
5	SESS_ID	BIGINT	会话 ID，系统内部标识
6	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局

			递增的序列号
7	SQL_TEXT	VARCHAR(1000)	返回的 SQL 语句
8	N_ROWS	BIGINT	返回的行数
9	IS_FIRST	CHAR(1)	是否第一次返回
10	RETURN_TIME	DATETIME（6）	记录返回的时间
11	MAL_ID	BIGINT	邮件标识号
12	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同
13	SQL_ID	INTEGER	SQL 语句 ID(唯一标识)

19）V$DMSQL_EXEC_TIME

记录动态监控的 SQL 语句执行时间。当 INI 参数 ENABLE_MONITOR_DMSQL=1 时才会记录监控的 SQL 语句，仅监控多条或复杂 SQL，例如包含引用包、嵌套子过程、子方法、动态 SQL 的 SQL 语句。

V$DMSQL_EXEC_TIME 的容量（最大记录数）由 INI 参数 DMSQL_ET_CNT 指定。

序号	列	数据类型	说明
1	EXEC_ID	BIGINT	语句执行号
2	TYPE$	VARCHAR(32)	语句类型：SQL、DYNAMIC SQL、METHOD
3	SEQ	INTEGER	相对序号：动态 SQL 的指令偏移、SQL 语句序号、方法的指令偏移
4	LEVEL	INTEGER	执行层次
5	CALLER	VARCHAR(256)	调用方法名
6	METHOD	VARCHAR(256)	方法名
7	EXEC_CNT	INTEGER	执行次数
8	LAST_EXEC_TICK	BIGINT	最近一次执行时戳 ，单位微秒
9	TIME_USED	BIGINT	执行时间 ，单位毫秒
10	MAL_ID	BIGINT	邮件标识号
11	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同
12	SQL_TEXT	VARCHAR(1000)	语句脚本
13	SQL_ID	INTEGER	SQL 语句 ID。本字段仅对语句块中的动态 SQL 有效，其

			它情况输出值均为 NULL
14	LINE_NO	INTEGER	该行所在脚本的行号， 方便定位
15	FIRST_START_TIME	DATETIME(6)	首次执行该行的时间

20) V$VIRTUAL_MACHINE

显示活动的虚拟机信息。

序号	列	数据类型	说明
1	ID	INTEGER	虚拟机 ID
2	SESSID	BIGINT	会话 ID
3	STMT_ID	INTEGER	语句 ID
4	EXP_FLAG	CHAR(1)	表达式标识
5	VSTACK_SIZE	INTEGER	栈大小，单位是栈帧的个数
6	VSTACK	BIGINT	栈基址
7	VTOP	INTEGER	栈顶离基址的距离
8	VUSED	INTEGER	栈已用空间的大小，单位是栈帧的个数
9	STKFRM_DEPTH	INTEGER	栈深度
10	FREE_STKFRMS	INTEGER	已释放栈帧的数目
11	CURR_FRM	BIGINT	当前栈的地址
12	IP	BIGINT	IP 地址
13	SQL_NO	INTEGER	SQL 语句树的序号
14	REUSE_CNT	INTEGER	虚拟机重用次数

21) V$SQLCOMMAND

显示 SQL 语句的类型。

序号	列	数据类型	说明

1	ID	INTEGER	SQL 语句的类型 ID
2	COMMAND_TYPE	INTEGER	SQL 语句的类型号
3	COMMAND_NAME	VARCHAR(24)	SQL 语句的类型名称
4	CON_ID	INTEGER	保留字段

22) V$SQL_BINDDATA_HISTORY

当 INI 参数 ENABLE_MONITOR=1、ENABLE_MONITOR_BP=1 时，记录数据库所有语句级绑定参数的类型和值。

该视图可对接口发送至服务器的带参数的方法/过程、INSERT/UPDATE/DELETE/SELECT 语句进行监控记录，但不对 PL/SQL 中动态执行的带参数语句进行监控记录。

序号	列	数据类型	说明
1	EXEC_ID	BIGINT	语句执行号
2	BINDDATA	VARBINARY(8188) 或 BLOB	绑定参数的信息。该列的数据类型与 INI 参数 BINDDATA_COLTYPE 相关，该参数缺省值为 0。当 BINDDATA_COLTYPE=0 时，数据类型为 VARBINARY，最大长度为 8188 个字节，若绑定参数信息的总长度超过 8188 个字节，则将对 OBJECT（名称/值对）以及 BINARY 两种类型的参数值进行缩减处理；当 BINDDATA_COLTYPE=1 时，数据类型为 BLOB，最大长度限制为 2GB

23) V$PLN_HISTORY

显示近期执行的 SQL 语句的执行计划。在 ini 参数 USE_PLN_POOL !=0 时才统计。

序号	列	数据类型	说明

1	SQL_ID	INTEGER	SQL 语句 ID，可与 V$SQL_HISTORY 关联查看更多信息
2	TOP_SQL_TEXT	VARCHAR(1000)	SQL 语句
3	SQL_PLAN	CLOB	SQL 语句对应的执行计划，与 explain SQL 结果相同；执行计划大小超过 10M 的部分不显示
4	SQL_TEXT_ID	VARCHAR(13)	语句固定的散列字符；相同的 SQL 散列字符相同

24) V$SQL_STAT_THRESHOLD

显示当前各个监控项的阈值。各个监控项的阈值默认为 0，表示未设置和未限制。可以通过 SP_SET_SQL_STAT_THRESHOLD()设置监控阈值，超过阈值才开始监控。具体使用参考《DM8_SQL 语言使用手册》。

除监控项 EXEC_TIME、PARSE_TIME、HARD_PARSE_TIME、IO_WAIT_TIME 在本视图中的单位为毫秒外，其余各监控项的单位与 V$SQL_STAT 中相同。

序号	列	数据类型	说明
1	ID	INTEGER	监控项的序号
2	STAT_NAME	VARCHAR(256)	监控项的名称
3	THRESHOLD_VALUE	BIGINT	监控项的阈值
4	COMMENT	VARCHAR(256)	监控项的使用描述

25) V$BUSY_PAGE_INFO

记录服务器启动后，最新的 20480 条 DSC LBS 等待信息，仅记录等待不少于 1 秒的信息，每秒更新一次记录。当打开数据页损坏报错功能后，额外记录数据页读超时和 DSC LBS 超时的错误信息。

序号	列	数据类型	说明
1	SESS_ID	BIGINT	会话 ID
2	TRX_ID	BIGINT	事务 ID

3	TS_ID	INTEGER	数据页表空间 ID
4	FILE_ID	INTEGER	数据页文件 ID
5	PAGE_NO	INTEGER	数据页号
6	ERR_CODE	INTEGER	日志 ID
7	ERR_TIME	DATETIME(0)	报错时间
8	WAIT_TIME	INTEGER	等待时长，单位 ms
9	DB_MAGIC	INTEGER	数据库魔数
10	SQLS	VARCHAR(1024)	空字符串""
11	EXEC_ID	INTEGER	SQL 执行 ID，对应 V$SQL_HISTORY 的 EXEC_ID

26) V$INJECT_HINT_INFO

记录服务器启动后 INJECT HINT 的使用情况。需要设置 INI 参数 ENABLE_INJECT_HINT=1，并使用 SF_INJECT_HINT 为 SQL 指定 HINT。对于模糊匹配，每条 HINT 的监控个数上限为 100，超出时摘除最早记录。当 INI 参数 USE_PLN_POOL 取值包含 1 时（即启用执行计划的重用功能），只有在 SQL 语句需要生成执行计划的时候才会记录 INJECT HINT 的使用情况。可以通过系统过程 SP_DYNAMIC_VIEW_DATA_CLEAR 来清空此视图的历史信息。

序号	列	数据类型	说明
1	HASH_VALUE	INTEGER	SQL 语句的 HASH 值
2	SQL_TEXT	VARCHAR(1000)	实际执行的 SQL 语句(最多只展示 1000 个字符)
3	HINT_NAME	VARCHAR(128)	HINT 规则名称
4	HINT_TEXT	VARCHAR(1000)	为 SQL 语句指定的 HINT 内容(最多只展示 1000 个字符)

5	FIRST_TIME	DATETIME(0)	第一次使用时间
6	LAST_TIME	DATETIME(0)	最近一次使用时间
7	COUNT	BIGINT	使用次数
8	IS_FUZZY	CHAR(1)	是否是模糊匹配
9	VALIDATE	VARCHAR(5)	规则是否生效，取值 TRUE/FALSE
10	SQL_TEXT_ID	VARCHAR(13)	语句固定的散列字符；相同的 SQL 语句，散列字符相同

9. 事务和检查点

1) V$TRX

显示所有活动事务的信息。通过该视图可以查看系统中所有的事务以及相关信息，如锁信息等。

序号	列	数据类型	说明
1	ID	BIGINT	当前活动事务的 ID 号
2	NEXTID	BIGINT	下一个事务 ID 号
3	MIN_ACTIVE_ID	BIGINT	所有活动事务 ID 号最小者
4	STATUS	VARCHAR(20)	当前事务的状态。 NOT START 未开始任何操作； ACTIVE 活动； LOCK WAIT 锁等待； ROLLING 正在回滚； ROLLING_SUSPEND 回滚挂起； PRE_COMMIT 两阶段事务的预提交状态； TO_RELEASE DPC 环境下分布式事务的等待释放状态。

			分布式事务完成第二阶段提交后转入 TO_RELEASE 状态，分布式事务在所有节点都转入 TO_RELEASE 状态后才允许释放；
5	ISOLATION	INTEGER	隔离级。0：读未提交；1：读提交；2：可重复读；3：串行化
6	READ_ONLY	CHAR(1)	是否是一个只读事务
7	SESS_ID	BIGINT	当前事务的所在会话 ID，系统内部标识
8	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
9	INS_CNT	INTEGER	插入回滚记录个数
10	DEL_CNT	INTEGER	删除回滚记录个数
11	UPD_CNT	INTEGER	更新回滚记录个数
12	UPD_INS_CNT	INTEGER	更新插入回滚记录个数
13	UREC_SEQNO	INTEGER	当前 UNDO 记录的递增序列号
14	WAITING	BIGINT	事务等待的锁地址，对应 V$LOCK 的 ADDR 字段
15	START_LSN	BIGINT	事务的起始 LSN
16	ROLLBACK_FLAG	INTEGER	是否在回滚
17	XID	VARBINARY(140)	XA 事务唯一标识
18	LOCK_CNT	INTEGER	使用中的事务锁个数
19	IGN_LOCK_CNT	INTEGER	IGNORABLE 锁个数
20	SAVEPOINT_CNT	INTEGER	保存点个数
21	THRD_ID	INTEGER	当前事务对应的线程 ID
22	SRC_SITE	INTEGER	分布式事务的来源站点 ID

23	N_BRANCHES	INTEGER	DPC 下事务涉及的节点数
24	N_WRITES	INTEGER	DPC 下事务涉及的非只读的节点数
25	N_PAGES	INTEGER	事务实际使用的回滚页数量
26	SNAP_CMTSEQ	BIGINT	获取可见性视图时的提交序号快照
27	N_LONG	INTEGER	获取可见性视图时的长事务个数
28	CRASH_FLAG	INTEGER	是否为故障前的事务：1 表示是，0 表示不是
29	START_TIME	DATETIME(0)	事务开始时间
30	ROLL_STIME	DATETIME(0)	事务最近一次回滚的开始时间
31	N_PAGE_ROLLED	INTEGER	事务已经回滚的回滚页数量

2) V$TRXWAIT

显示事务等待信息。

序号	列	数据类型	说明
1	ID	BIGINT	事务 ID
2	WAIT_FOR_ID	BIGINT	所等待的事务 ID
3	WAIT_TIME	INTEGER	当前等待时间。单位：毫秒
4	THRD_ID	INTEGER	所等待事务对应的线程 ID
5	LOCK	BIGINT	当前事务的锁地址
6	WAIT_FOR_LOCK	BIGINT	所等待事务的锁地址

3) V$RECV_ROLLBACK_TRX

显示数据库启动时回滚的所有事务信息。

序号	列	数据类型	说明

1	TRX_ID	BIGINT	事务号
2	BEGIN_LSN	BIGINT	事务启动逻辑日志的 LSN 值，只在 INI 参数开启逻辑日志情况下有效
3	BEGIN_TIME	DATETIME(6)	事务启动时间，只在 INI 参数开启逻辑日志情况下有效
4	N_UPAGES	INTEGER	回滚页总数
5	N_URECS	INTEGER	回滚记录总数

4) V$LOCK

显示活动的事务锁信息。

序号	列	数据类型	说明
1	ADDR	BIGINT	本地封锁对象的地址
2	TRX_ID	BIGINT	所属事务 ID
3	LTYPE	VARCHAR(10)	锁类型：TID 锁、对象锁
4	LMODE	CHAR(2)	锁模式：S 锁、X 锁、IX 锁、IS 锁
5	BLOCKED	INTEGER	是否处于上锁等待状态。0 表示已上锁成功；1 表示处于上锁等待状态
6	TABLE_ID	INTEGER	对于对象锁，表示表对象或字典对象的 ID；对于 TID 锁，表示封锁记录对应的表 ID。-1 表示事务启动封锁自身的 TID
7	ROW_IDX	BIGINT	TID 锁封锁记录行信息。-1 表示事务启动封锁自身的 TID
8	TID	BIGINT	TID 锁对象事务 ID
9	IGN_FLAG	INTEGER	锁对象的 IGNORABLE 标记，INI 参数 LOCK_DICT_OPT 开启时有效。 取值 0：表示锁正在使用中，IGN_FLAG 最低位置为 0；

			取值 1：表示事务 TRX1 已经提交，但是锁资源未释放，TRX1 重新封锁时可以直接使用，此时 IGN_FLAG 最低位置为 1; 取值 2：为 0+2 的组合值。当锁正被使用时，另一事务要封锁同一对象，将当前正被使用的锁的 IGN_FLAG 次低位置为 1; 取值 3：为 1+2 的组合值。当 TRX1 已提交，但锁未释放时，另一事务 TRX2 要封锁同一对象，此时 TRX2 可以忽略此 IGN_FLAG=1 的锁，但是要将此锁的 IGN_FLAG 次低位置为 1
10	HLCK_EP	INTEGER	关联对象 HLCK 所属节点号。DSC 集群环境标识创建 HLCK 对象的节点号，缺省为 255，单机无效
11	SPT_NO	INTEGER	关联的保存点序号
12	THRD_ID	INTEGER	锁的创建者线程号
13	HASH_VALUE	BIGINT	锁的缓存的 HASH 值

5） V$DEADLOCK_HISTORY

记录死锁的历史信息。

需要注意，在 DMDSC 环境中，只有控制节点记录死锁的历史信息。

序号	列	数据类型	说明
1	SEQNO	INTEGER	编号
2	TRX_ID	BIGINT	事务 ID
3	SESS_ID	BIGINT	会话 ID，系统内部标识
4	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
5	SQL_TEXT	VARCHAR(1000)	产生死锁的 SQL 语句

6	HAPPEN_TIME	DATETIME(6)	死锁发生的时间
7	TOO_MANY_CHECKS	CHAR(1)	当事务发生锁等待时，是否检测到等待锁的数量大于等于1024。Y 是，此时视为发生死锁；N 否，表示实际死锁
8	DEADLOCK_CYCLE	VARCHAR(4096)	当 TOO_MANY_CHECKS = 'Y'时，格式为： self ->((tid1, sess_id1), (tid2, sess_id2), ..., (tidn, sess_idn))，表示事务等待的锁和会话 ID 列表； 当 TOO_MANY_CHECKS = 'N'时， 格式为： self -> (tid1, sess_id1) ->(tid2, sess_id2) -> ... -> self，表示发生死锁时的等待环。其中 self 表示被牺牲的事务号，对应视图中本行记录的第 2 个字段 TRX_ID
9	USER_ID	INTEGER	用户 ID
10	SCH_ID	INTEGER	模式 ID
11	SQL_TEXT_ID	VARCHAR(13)	SQL 文本 ID

6） V$FLASHBACK_TRX_INFO

显示闪回信息，无法显示只有 Insert 操作的事务。

序号	列	数据类型	说明
1	START_TRXID	BIGINT	闪回开始事务 ID
2	START_TIMESTAMP	DATETIME(6)	开始时间
3	COMMIT_TRXID	BIGINT	闪回完成事务 ID
4	COMMIT_TIMESTAMP	DATETIME(6)	完成时间
5	LOGIN_USER	VARCHAR(256)	闪回用户名
6	UNDO_CHANGE#	INTEGER	回滚记录序号

7	OPERATION	CHAR(1)	DML 操作类型。 D：删除；U：修改；I：插入；N：更新插入（专门针对 CLUSTER PRIMARY KEY 的插入）； C：事务提交；P：预提交记录；O：default
8	TABLE_NAME	VARCHAR(256)	表名
9	TABLE_OWNER	VARCHAR(256)	表所属用户名
10	ROW_ID	BIGINT	ROWID
11	UNDO_SQL	VARCHAR(3900)	回滚 SQL 语句
12	COMMIT_LSN	BIGINT	闪回完成时的 LSN

7） V$LSN_TIME

显示 LSN/TIME 映射关系。每 3 秒收集一次映射关系，并且仅支持显示最近 5 天的映射关系。每次系统重启或者主备切换都会重新收集映射关系，目前暂不支持显示系统启动之前的历史映射关系。

序号	列	数据类型	说明
1	TIME_MP	BIGINT	当前时间，单位为秒
2	TIME_DP	DATETIME(0)	当前时间，将 TIME_MP 转换为用户可读的 DATETIME 类型
3	LSN	BIGINT	当前 LSN

8） V$CKPT_HISTORY

显示检查点历史信息。该视图需要在开启 ENABLE_MONITOR 后才能监控检查点信息。

对于未结束的检查点，尚未确定的值输出为 0，例如 TIME_USED、PAGE_FLUSHED、FIL、OFF、CKPT_LSN、FREE_SPACE 等。

序号	列	数据类型	说明
1	SEQNO	INTEGER	序列号
2	CKPT_CMD	VARCHAR(24)	触发检查点的原因。分别如下: INI_INTERVAL 表示定时触发，由 INI 参数

			CKPT_INTERVAL 确定; INI_RLOG_SIZE 表示满足 INI 参数 CKPT_RLOG_SIZE 的条件时触发; INI_DIRTY_PAGES 表示满足 INI 参数 CKPT_DIRTY_PAGE 的条件时触发; UNSAFE/FLUSH_PAGE 表示强制定时刷页时检测到日志空间不足; UNSAFE/FLUSH_RLOG 表示日志写入线程检测到日志空间不足; UNSAFE/AFTER_CKPT 表示检查点线程检测到日志空间不足; CKPT_MTTR_TUNE 表示为满足故障恢复时间限制触发; IF_NECESSARY 表示需要时触发; USER_CMD 表示用户发起的手工触发; BY_FORCE 表示由于 DDL 等特殊情况强制触发; OTHER 表示其他情况
3	THREAD_ID	BIGINT	线程 ID
4	START_TIME	DATETIME(6)	开始时间
5	TIME_USED	BIGINT	从开始到结束使用时间，单位为微秒（us）
6	PAGE_FLUSHED	INTEGER	刷盘数
7	FIL	INTEGER	文件号
8	OFF	BIGINT	文件偏移量
9	CKPT_LSN	BIGINT	检查点 LSN
10	FREE_SPACE	BIGINT	检查点完成时刻联机日志文件的空闲空间，单位字节
11	IN_PROCESS	CHAR(1)	当前检查点是否正在进行中。Y 是；N 否

9） V$CKPT

显示系统检查点信息。

序号	列	数据类型	说明
1	CKPT_RLOG_SIZE	INTEGER	检查点日志大小，单位 MB
2	CKPT_DIRTY_PAGES	INTEGER	脏页的数量。产生多少脏页后，才强制产生检查点
3	CKPT_INTERVAL	INTEGER	系统强制产生检查点的时间间隔(s)
4	CKPT_FLUSH_RATE	DEC(5,2)	每次检查点脏页的刷盘比例
5	CKPT_FLUSH_PAGES	INTEGER	每次检查点至少刷盘的脏页数

6	LAST_BEGIN_TIME	DATETIME(6)	最近一次执行的开始时间
7	LAST_END_TIME	DATETIME(6)	最近一次执行的结束时间
8	CKPT_LSN	BIGINT	最近一次检查点 LSN
9	CKPT_FILE	BIGINT	最近一次检查点对应的当时的文件号
10	CKPT_OFFSET	BIGINT	最近一次检查点对应的当时的文件偏移
11	STATE	VARCHAR(128)	检查点状态，只有2种。0 NONE，其他 PROCESSING
12	CKPT_TOTAL_COUNT	BIGINT	检查点已做的个数
13	CKPT_RESERVE_COUNT	BIGINT	预先申请日志空间的次数
14	CKPT_FLUSHED_PAGES	BIGINT	检查点已刷页的个数
15	CKPT_TIME_USED	BIGINT	检查点从开始到结束经历的时间，单位毫秒

10) V$TRX_SAVEPOINT

显示所有活动事务的保存点信息。

序号	列	数据类型	说明
1	TRX_ID	BIGINT	当前活动事务的 ID 号
2	NAME	VARCHAR(128)	保存点名字
3	SEQ_NO	INTEGER	保存点序号
4	MSEQ_NO	INTEGER	MPP 下从 EP 在主 EP 上的对应序号
5	LAST_LOCK	BIGINT	释放锁的起始地址
6	LAST_OLOCK	BIGINT	释放 OLOCK 锁的起始地址

11) V$LOCKED_OBJECT

显示系统中被锁封锁的对象信息及其所属的会话。

序号	列	数据类型	说明

1	XIDUSN	INTEGER	回滚段编号（尚未完善，暂时用 0 代替）
2	XIDSLOT	INTEGER	槽号，使用回滚段中的 nth 替代
3	XIDSQN	INTEGER	序列号
4	OBJECT_ID	INTEGER	被上锁的对象 ID
5	SESSION_ID	BIGINT	会话 ID
6	USERNAME	VARCHAR(128)	用户名
7	OS_USER_NAME	VARCHAR(32)	系统用户名
8	PROCESS	VARCHAR(30)	进程号与线程号，格式为"进程号：线程号"
9	LOCKED_MODE	CHAR(2)	锁模式；可为：S 锁、X 锁、IX 锁、IS 锁
10	LOCKED_TYPE	VARCHAR(10)	锁类型；可为：对象锁

12) V$LONG_TRX

显示所有活动的长事务的信息，通过该视图可以查看系统中所有的长事务及其所属站点号。

长事务，指长时间未提交的事务，由最新提交的事务号和系统可分配的总事务号决定。假设系统可分配的总事务号为 N，最新提交的事务号为 N1，那么所有事务号小于(N1 - N/2)的未提交事务都被定义为长事务。长事务总数不能超过 10240，超过后则不再收集。

例如：可分配的总事务号为 1024，当最新提交的事务号为 600 时，计算差值为(600 - 1024/2)=88，则所有事务号小于 88 的未提交事务会被判断为长事务。

序号	列	数据类型	说明
1	TRX_ID	BIGINT	当前活动的长事务的 ID 号
2	EP_SEQNO	INTEGER	事务所在站点号；在单机环境中为 0，在 DSC 环境中为节点号，在 DPC 环境中为 RAFT_ID

13) V$TRX_VIEW

显示当前事务可见的所有活动事务视图信息。根据达梦多版本规则，通过该视图可以查询系统中自己所见的事务信息；可以通过与 v$trx 表的连接查询它所见事务的具体信息。

序号	列	数据类型	说明
1	TRXID	BIGINT	当前活动事务的 ID 号
2	NEXT_ID	BIGINT	下一个事务 ID 号，大于等于 next_id 的事务不可见
3	MIN_ACTIVE_ID	BIGINT	所有活动事务最小 ID，小于 min_active_id 的事务不可见
4	EGTS_ENO	BIGINT	刷新视图添加 snap 任务时的 egts 执行号
5	TEMP_ENO	BIGINT	获取最小事务号时代替 trx 登记缓存事务提交计数的执行号
6	TEMP_CMTSEQ	BIGINT	获取最小事务号时代替 trx 登记的缓存事务提交计数
7	N_LONG	INTEGER	长事务数目
8	LONG_TIDS	CLOB	长事务事务 ID 数组
9	N_PE	INTEGER	数据页损坏事务信息数目
10	PE_TIDS	CLOB	数据页损坏事务信息数组
11	MVCC_PAGE_OPT	INTEGER	数据页可见性判断优化标记
12	LAST_CMT_LSN	BIGINT	上次事务提交产生的 LSN
13	TRX_START_LSN	BIGINT	第一次分配回滚页时的 LSN
14	MIN_LAST_CMT_LSN	BIGINT	可见视图里事务号最小事务的 last_cmt_lsn
15	N_INHASH_IDS	INTEGER	活动事务 ID 数组已使用数目
16	INHASH_IDS	CLOB	活动事务 ID 数组

14）V$TRX_CMTARR

显示当前已提交事务数组信息。

序号	列	数据类型	说明
1	CMT_SEQ	BIGINT	事务提交计数
2	N_CELL	INTEGER	可配置数组大小
3	N_CMT_CELL	INTEGER	提交事务登记数组大小
4	FILL_POS	BIGINT	提交事务登记数组填充下标
5	LONG_DIFF	INTEGER	用于区分长事务计算
6	FIRST_TID	BIGINT	系统启动后的第一个事务 ID
7	CMTARR_MODE_FLAG	INTEGER	是否启用基于数组的可见性判断机制
8	MIN_ID	BIGINT	不包含长事务的最小活动事务 ID
9	MIN_START_LSN	BIGINT	最小活动事务的启动 LSN
10	N_LONG	INTEGER	长事务数目
11	LONG_ARR	CLOB	长事务事务 ID 数组

10. 事件

1) V$WAIT_HISTORY

通过该视图可以查询等待事件的具体信息，如等待的线程 id，会话 id 等。可以查看具体等待事件的信息，如果某个事务等待时间过长，则可以查询到具体事务信息以及所在的线程和所牵涉的对象，分析原因进行优化等操作。

序号	列	数据类型	说明
1	THREAD_ID	BIGINT	线程 ID
2	TRX_ID	BIGINT	事务 ID

3	WAIT_CLASS	INTEGER	等待类型号
4	WAIT_OBJECT	BIGINT	等待对象
5	WAIT_START	DATETIME(6)	等待事件的开始时间
6	WAIT_TIME	BIGINT	等待时间，单位微秒
7	SPACE_ID	INTEGER	表空间号
8	FILE_ID	INTEGER	文件号
9	PAGE_NO	INTEGER	页号

2） V$EVENT_NAME

显示当前系统所支持的等待事件的类型汇总信息。

序号	列	数据类型	说明
1	EVENT#	INTEGER	事件编号
2	EVENT	VARCHAR(32)	dbfile read wait：读数据页等待； dbfile flush wait：写数据页等待； dbfile remote read：DSC 引起的读磁盘； dbfile remote write：DSC 引起的写磁盘； dbfile discard write：buf 不够引起的写磁盘； dbfile ckpt write：ckpt 引起的写磁盘； dbfile extend：扩库文件引起的写磁盘； shared memory pool busy：共享内存并发使用冲突； buffer busy wait：buffer 并发使用冲突； dict cache busy：字典 cache 并发使用冲突； plan cache busy：计划 cache 并发使用冲突；

			redo log system busy：redo log 系统并发冲突； redo log buffer busy：redo log 缓冲区并发冲突； kernel busy：kernel 并发冲突； table lock busy：表锁系统并发冲突； tid lock busy：行锁系统并发冲突； parallel bdta pool busy：并行 bdta pool 并发冲突； iid system busy：iid 系统并发冲突； pseg queue busy：pseg 队列并发冲突； pseg stack busy：pseg 堆栈并发冲突； page s latch wait：数据页的S闩封锁； page x latch wait:数据页的X闩封锁； rlog flush wait:日志刷盘； commit wait:事务提交； table lock wait：事务间表锁等待发生； trxid lock wait：事务间行锁等待发生； scp cache busy：scp缓存并发冲突； trx system s busy：事务系统s封锁并发冲突； trx system x busy ：事务系统x封锁并发冲突； trx view s busy：事务可见性s封锁并发冲突； trx view x busy：事务可见性x封锁并发冲突； dscids s busy：dscids s封锁并发冲突； dscids x busy：dscids x封锁并发冲突； slog single busy：SQL日志个体并发冲突；

			network idle wait：网络处理等待； preload wait：数据页预加载任务； session system s latch wait：SESSION系统的S闩封锁； session system x latch wait：SESSION系统的X闩封锁； file system s latch wait：文件系统的S闩封锁； file system x latch wait：文件系统的X闩封锁； gtv system s latch wait：DSC全局事务系统的S闩封锁； gtv system x latch wait：DSC全局事务系统的X闩封锁； aud system s latch wait：审计缓冲池的S闩封锁； aud system x latch wait：审计缓冲池的X闩封锁； purge tinfo s latch wait：purge表信息的S闩封锁； purge tinfo x latch wait：purge表信息的X闩封锁； lbs pool s latch wait：LBS线程池的S闩封锁； lbs pool x latch wait：LBS线程池的X闩封锁； fback system s latch wait：可闪回系统的S闩封锁； fback system x latch wait：可闪回系统的X闩封锁； march/rt arch s latch wait：实时归档的S闩封锁； march/rt arch x latch wait：实时归档的X闩封锁； timely arch s latch wait：即时归档的S闩封锁； timely arch x latch wait：即时归档的X闩封锁； bct s latch wait：BCT的S闩封锁； bct x latch wait：BCT的X闩封锁； buffer s latch wait：BUFFER的S闩封锁；

			buffer x latch wait：BUFFER的X闩封锁； dict cache s latch wait：字典缓存的S闩封锁； dict cache x latch wait：字典缓存的X闩封锁； pln_id mapping s latch wait：BP节点缓存的S闩映射封锁； pln_id mapping x latch wait：BP节点缓存的X闩映射封锁； next trxid flush wait：事务id刷盘等待； defer seg create flush wait：延迟段创建刷盘等待； cmtseq flush wait：提交序号刷盘等待； lbs ctl s latch wait：DSC局部缓冲区的S闩封锁； lbs ctl x latch wait：DSC局部缓冲区的X闩封锁； async_arch s latch wait：异步归档的S闩封锁； async_arch x latch wait：异步归档的X闩封锁； sync_arch s latch wait：同步归档的S闩封锁； sync_arch x latch wait：同步归档的X闩封锁
3	WAIT_CLASS#	INTEGER	事件类别编号：0(User I/O)、1(SystemI/O)、 2(Concurrency)、3(Rlog)、4(Transaction)、 5(Memory)、6(Network)、7(Preload)、8(Idle)、 9(Cluster)、10(Commit)、11(Arch)、66(Slog)
4	WAIT_CLASS	VARCHAR(32)	事件类别名称：User I/O、System I/O、Concurrency、 Rlog、Transaction、Memory、Network、Preload、Idle、 Cluster、Commit、arch、Slog

3) V$SYSTEM_EVENT

显示自系统启动以来所有等待事件的详细信息。

序号	列	数据类型	说明
1	EVENT#	INTEGER	事件编号
2	EVENT	VARCHAR(32)	事件名称。
3	TOTAL_WAITS	BIGINT	等待次数
4	TIME_WAITED	BIGINT	等待时间，单位：1/100 秒
5	TIME_WAITED_MICRO	BIGINT	等待时间，单位微秒
6	AVERAGE_WAIT_MICRO	INTEGER	平均等待时间，单位微秒
7	SMAX_TIME	BIGINT	最长等待时间，单位微秒
8	SMIN_TIME	INTEGER	最短等待时间，单位微秒
9	WAIT_CLASS#	INTEGER	等待事件类别编号
10	WAIT_CLASS	VARCHAR(32)	等待事件类别名称

4) V$SESSION_EVENT

显示当前会话等待事件的所有信息。

序号	列	数据类型	说明
1	SESSADDR	VARBINARY(8)	会话地址
2	SESSION#	INTEGER	会话 ID
3	EVENT#	INTEGER	事件编号
4	EVENT	VARCHAR(32)	事件名称
5	TOTAL_WAITS	BIGINT	等待次数
6	TIME_WAITED	BIGINT	等待时间，单位：1/100 秒
7	TIME_WAITED_MICRO	BIGINT	等待时间，单位微秒
8	AVERAGE_WAIT_MICRO	INTEGER	平均等待时间，单位微秒
9	SMAX_TIME	BIGINT	最长等待时间，单位微秒

10	SMIN_TIME	INTEGER	最短等待时间，单位微秒
11	WAIT_CLASS#	INTEGER	等待事件类别编号
12	WAIT_CLASS	VARCHAR(32)	等待事件类别名称

5) GV$SESSION_EVENT

显示全局会话等待事件的所有信息。

支持的集群有 DPC、DSC、实时主备、读写分离集群、AFC。

序号	列	数据类型	说明
1	INSTANCE_NAME	VARCHAR(128)	实例名
2	SESSADDR	VARBINARY(8)	会话地址
3	SESSION#	INTEGER	会话 ID
4	EVENT#	INTEGER	事件编号
5	EVENT	VARCHAR(32)	事件名称
6	TOTAL_WAITS	BIGINT	等待次数
7	TIME_WAITED	BIGINT	等待时间，单位：1/100 秒
8	TIME_WAITED_MICRO	BIGINT	等待时间，单位微秒
9	AVERAGE_WAIT_MICRO	INTEGER	平均等待时间，单位微秒
10	SMAX_TIME	BIGINT	最长等待时间，单位微秒
11	SMIN_TIME	INTEGER	最短等待时间，单位微秒
12	WAIT_CLASS#	INTEGER	等待事件类别编号
13	WAIT_CLASS	VARCHAR(32)	等待事件类别名称

6) V$SESSION_WAIT_HISTORY

显示当前会话等待事件的历史信息。

序	列	数据类型	说明

号			
1	SESSADDR	VARBINARY(8)	会话地址
2	SESSION#	INTEGER	会话 ID
3	SQL_ID	INTEGER	SQL 语句 ID
4	EVENT#	INTEGER	事件编号
5	EVENT	VARCHAR(32)	事件名称
6	P1TEXT	VARCHAR(32)	等待事件对应的参数 1 说明(如果没有足够参数，则为 NULL，以下参数一样)
7	P1	VARCHAR(256)	等待事件对应的参数 1 值
8	P2TEXT	VARCHAR(32)	等待事件对应的参数 2 说明
9	P2	BIGINT	等待事件对应的参数 2 值
10	P3TEXT	VARCHAR(32)	等待事件对应的参数 3 说明
11	P3	BIGINT	等待事件对应的参数 3 值
12	P4TEXT	VARCHAR(32)	等待事件对应的参数 4 说明
13	P4	BIGINT	等待事件对应的参数 4 值
14	TIME_WAITED	INTEGER	等待时间，单位：1/100 秒
15	TIME_WAITED_MICRO	INTEGER	等待时间，单位微秒
16	WAIT_CLASS#	INTEGER	等待事件类别编号
17	WAIT_CLASS	VARCHAR(32)	等待事件类别名称
18	P5TEXT	VARCHAR(32)	等待事件对应的参数 5 说明
19	P5	VARCHAR(1024)	等待事件对应的参数 5 值，目前仅针对 table lock wait/trxid lock wait 有值，表示等待事务的 trxid 和会话 id

7) GV$SESSION_WAIT_HISTORY

显示全局会话等待事件的历史信息。

支持的集群有 DPC、DSC、实时主备、读写分离集群、AFC。

序号	列	数据类型	说明
1	INSTANCE_NAME	VARCHAR(128)	实例名
2	SESSADDR	VARBINARY(8)	会话地址
3	SESSION#	INTEGER	会话 ID
4	SQL_ID	INTEGER	SQL 语句 ID
5	EVENT#	INTEGER	事件编号
6	EVENT	VARCHAR(32)	事件名称
7	P1TEXT	VARCHAR(32)	等待事件对应的参数 1 说明(如果没有足够参数，则为 NULL，以下参数一样)
8	P1	VARCHAR(256)	等待事件对应的参数 1 值
9	P2TEXT	VARCHAR(32)	等待事件对应的参数 2 说明
10	P2	BIGINT	等待事件对应的参数 2 值
11	P3TEXT	VARCHAR(32)	等待事件对应的参数 3 说明
12	P3	BIGINT	等待事件对应的参数 3 值
13	P4TEXT	VARCHAR(32)	等待事件对应的参数 4 说明
14	P4	BIGINT	等待事件对应的参数 4 值
15	TIME_WAITED	INTEGER	等待时间，单位：1/100 秒
16	TIME_WAITED_MICRO	INTEGER	等待时间，单位微秒
17	WAIT_CLASS#	INTEGER	等待事件类别编号

18	WAIT_CLASS	VARCHAR(32)	等待事件类别名称
19	P5TEXT	VARCHAR(32)	等待事件对应的参数 5 说明
20	P5	VARCHAR(1024)	等待事件对应的参数 5 值，目前仅针对 table lock wait/trxid lock wait 有值，表示等待事务的 trxid 和会话 id

8) V$DANGER_EVENT

记录数据库重要事件或行为信息视图。重要的事件或行为是指 ALTER_DATABASE、创建表空间、重命名表空间、重命名数据文件和增加数据文件。

当 INI 参数 ENABLE_MONITOR=1 才有效。

序号	列	数据类型	说明
1	OPTIME	DATETIME(6)	重要事件或行为发生的时间
2	OPERATION	VARCHAR(1024)	重要事件或行为执行的语句
3	OPUSER	VARCHAR(128)	执行重要事件或行为的用户

9) V$TASK_QUEUE

任务系统中的任务队列信息。

序号	列	数据类型	说明
1	READY	BIGINT	已处理任务数
2	TOTAL_WAIT_TIME	BIGINT	已处理任务的总等待时间，单位毫秒
3	AVERAGE_WAIT_TIME	INTEGER	已处理任务的平均等待时间，单位毫秒
4	STATE	INTEGER	任务系统状态。0：开启；1：准备关闭；2：关闭
5	N_OPEN_TASK	INTEGER	OPEN 状态下执行的任务数
6	N_TRIGGER	INTEGER	正在运行的时间触发器数量

10) V$TRACE_QUEUE

跟踪系统中的任务队列信息。

序号	列	数据类型	说明
1	WAITING	INTEGER	等待处理任务数
2	READY	BIGINT	已处理任务数
3	TOTAL_WAIT	BIGINT	已处理任务的总等待时间，单位毫秒
4	AVERAGE_WAIT	INTEGER	已处理任务的平均等待时间，单位毫秒

11) V$TASK_WAITING

任务系统中的所有等待任务信息。

序号	列	数据类型	说明
1	CMD	VARCHAR(128)	任务类型
2	CREATE_TIME	DATETIME(0)	创建时间

11. 进程和线程

1) V$PROCESS

显示当前进程信息。

序号	列	数据类型	说明
1	PID	INTEGER	进程 ID
2	PNAME	VARCHAR(256)	进程名
3	TRACE_NAME	VARCHAR(256)	SQL 日志路径，若 INI 参数 SVR_LOG 为 0，则值为空串
4	TYPE$	TINYINT	暂不支持，值始终为 1

2) V$THREADS

显示系统中所有活动线程的信息。

序号	列	数据类型	说明

1	ID	BIGINT	线程 ID
2	NAME	VARCHAR(128)	线程名
3	START_TIME	DATETIME(6)	线程开始时间
4	THREAD_DESC	VARCHAR(1024)	线程描述
5	SESS_ID	BIGINT	与本线程关联的会话 ID。NULL 表示无关联的会话
6	PROCESSOR_ID	INTEGER	线程绑定的 CPU 处理器 ID。65535 表示未绑定 CPU 处理器
7	WAIT_SEQ	INTEGER	线程等待序号
8	WAIT_STATUS	VARCHAR(128)	线程等待状态
9	WAIT_TIME	INTEGER	线程等待时间，单位毫秒，取值-1 表明线程不在等待状态

3) V$THRDS_POOL

底层线程池信息。

序号	列	数据类型	说明
1	N_THRDS	INTEGER	总线程数
2	N_USED_THRDS	INTEGER	已使用的线程数
3	N_TEMP_THRDS	INTEGER	临时线程数
4	MAX_THRDS	INTEGER	最大线程数
5	MEM_USED	INTEGER	内存使用。单位：字节

4) V$THRD_POOL_GRP

上层特定模块中线程池组的信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(128)	线程池类型

2	ID	INTEGER	线程池 ID
3	N_THRDS	INTEGER	总线程数
4	N_USED_THRDS	INTEGER	使用的线程数
5	N_WAIT_THRDS	INTEGER	锁等待的线程数
6	N_SUSPEND_THRDS	INTEGER	挂起的线程数
7	N_TASK_TODO	INTEGER	等待的任务数

12. 系统信息

1) V$SYSTEMINFO

系统信息视图。

序号	列	数据类型	说明
1	N_CPU	INTEGER	CPU 个数
2	TOTAL_PHY_SIZE	BIGINT	物理内存总大小，单位 BYTE
3	FREE_PHY_SIZE	BIGINT	剩余物理内存大小，单位 BYTE
4	TOTAL_VIR_SIZE	BIGINT	虚拟内存总大小，单位 BYTE
5	FREE_VIR_SIZE	BIGINT	剩余虚拟内存大小，单位 BYTE
6	TOTAL_DISK_SIZE	BIGINT	磁盘总大小，单位 BYTE
7	FREE_DISK_SIZE	BIGINT	剩余磁盘大小，单位 BYTE
8	DRIVER_NAME	VARCHAR(5)	驱动器名称
9	DRIVER_TOTAL_SIZE	BIGINT	驱动器总空间大小，单位 BYTE
10	DRIVER_FREE_SIZE	BIGINT	驱动器剩余空间大小，单位 BYTE
11	LOAD_ONE_AVERAGE	DOUBLE	每分钟平均负载，仅 LINUX 环境有效

12	LOAD_FIVE_AVERAGE	DOUBLE	每五分钟平均负载，仅 LINUX 环境有效
13	LOAD_FIFTEEN_AVERAGE	DOUBLE	每十五分钟平均负载，仅 LINUX 环境有效
14	CPU_USER_RATE	DOUBLE	用户模式占用 CPU 使用率，仅 LINUX 环境有效
15	CPU_SYSTEM_RATE	DOUBLE	系统模式占用的 CPU 使用率，仅 LINUX 环境有效
16	CPU_IDLE_RATE	DOUBLE	空闲 CPU 百分比，仅 LINUX 环境有效
17	SEND_BYTES_TOTAL	BIGINT	发送的总字节数，仅 LINUX 环境有效
18	RECEIVE_BYTES_TOTAL	BIGINT	接收的总字节数，仅 LINUX 环境有效
19	SEND_BYTES_PER_SECOND	BIGINT	当前每秒发送字节数，仅 LINUX 环境有效
20	RECEIVE_BYTES_PER_SECOND	BIGINT	当前每秒接收字节数，仅 LINUX 环境有效
21	SEND_PACKAGES_PER_SECOND	BIGINT	当前每秒发送数据包数，仅 LINUX 环境有效
22	RECEIVE_PACKAGES_PER_SECOND	BIGINT	当前每秒接收数据包数，仅 LINUX 环境有效
23	USED_PHY_SIZE	BIGINT	当前系统使用的内存大小，单位 BYTE

2) V$NETINFO

网络信息视图。

序号	列	数据类型	说明
1	RECV_BYTES	BIGINT	网络接收的字节数
2	SEND_BYTES	BIGINT	网络发送的字节数

3) V$CMD_HISTORY

通过本视图可以观察系统的一些命令的历史信息。其中 cmd 指的是 SESS_ALLOC，

SESS_FREE, CKPT, TIMER_TRIG, SERERR_TRIG, LOG_REP, MAL_LETTER, CMD_LOGIN 等。

序号	列	数据类型	说明
1	CMD	VARCHAR(24)	命令
2	THREAD_ID	BIGINT	线程 ID
3	SESS_ID	BIGINT	会话 ID，系统内部标识
4	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
5	TRX_ID	BIGINT	事务 ID
6	STMT_ID	INTEGER	语句 ID
7	START_TIME	DATETIME(6)	命令开始时间
8	TIME_USED	BIGINT	命令从开始执行到执行结束花费的时间，单位为微秒（us）

4) V$RUNTIME_ERR_HISTORY

监控运行时错误历史。异常分为三种：一种是系统异常，用户没有捕获，由 vm_raise_runtime_error 产生；第二种是用户异常，用户捕获错误，并抛出自定义异常，由 nthrow_exec 产生；第三种是语法异常，语法未通过，由 nsvr_build_npar_cop_out 产生。

该视图仅在异常被抛出时进行记录，并不会跟踪记录异常是否被处理。对于 DMSQL 程序，即便使用 EXCEPTION 语句对异常进行了处理，仍能在该视图中查询到该异常。

V$RUNTIME_ERR_HISTORY 视图中各个列的含义如下：

序号	列	数据类型	说明

1	SEQNO	INTEGER	该类语句的执行编号
2	SESS_ID	BIGINT	执行会话 ID，系统内部标识
3	SESS_SEQ	INTEGER	会话序列号，每创建一个会话，系统自动为其生成一个全局递增的序列号
4	TRX_ID	BIGINT	执行事务 ID
5	EXEC_ID	BIGINT	虚拟机的执行序号
6	VM_ID	INTEGER	虚拟机 ID
7	STKFRM_LEVEL	INTEGER	当前栈帧在虚拟机的层次
8	SQL_TEXT	VARCHAR(1000)	获得该层栈帧执行的 SQL 语句，若字符长度超过 1000，则取前 1000 个字符
9	SU_FLAG	CHAR(1)	U =用户异常，S =系统异常，P =语法异常
10	ECPT_CODE	INTEGER	异常错误号
11	ECPT_DESC	VARCHAR(256)	异常描述
12	MAL_ID	BIGINT	邮件标识号
13	ERR_TIME	DATETIME(3)	错误产生的时间
14	MPP_EXEC_ID	BIGINT	MPP 会话句柄上的执行序号，同一个会话上的每个节点上值相同
15	OBJECT_NAME	VARCHAR(128)	出错的对象名，当前仅若干特定的错误中获取出错对象名；其他的为 NULL
16	USER_NAME	VARCHAR(128)	记录生成错误时 SESSION 对应的用户名
17	APPNAME	VARCHAR(128)	记录生成错误时 SESSION 所在的应用名
18	CLNT_IP	VARCHAR(128)	记录生成错误时 SESSION 对应的客户端的地址信息
19	SQL_ID	INTEGER	记录生成错误时 SESSION 中当前 SQL 语句 ID；为-1 表示错

			误是在语法分析阶段登记的

5） V$INSTANCE_LOG_HISTORY

用于查询服务器实例运行期间生成的最近 1 万条事件日志。

序号	列	数据类型	说明
1	SEQNO	INTEGER	日志序号，从 0 开始递增。每次服务器重启后清零
2	LOG_TIME	DATETIME(6)	日志生成时间，精确到秒
3	PID	BIGINT	生成日志的进程 ID
4	THREAD_NAME	VARCHAR(128)	生成日志的线程名
5	LEVEL$	VARCHAR(20)	日志级别描述。可取值为：INFO、WARN、ERROR、FATAL
6	TXT	VARCHAR(2048)	日志详细内容

6） V$INSTANCE_RECOVERY

用于监控提供给用户的用于限制数据库故障恢复 IO 的相关机制。监控的具体相关机制如下：

（1）设置 FAST_START_MTTR_TARGET；

（2）设置 CKPT_INTERVAL；

（3）设置 CKPT_RLOG_SIZE；

（4）设置 CKPT_DIRTY_PAGES

序号	列	数据类型	说明
1	RECOVERY_ESTIMATED_IOS	BIGINT	缓存中的脏页数量
2	ACTUAL_REDO_BLKS	BIGINT	实际需要恢复的联机日志块数

3	TARGET_REDO_BLKS	INTEGER	当前检查点需要处理的联机日志块数，等于 CKPT_RLOG_SIZE 参数对应的块个数
4	LOG_FILE_SIZE_REDO_BLKS	INTEGER	检查点完成前，保证日志不会发生切换所需要的联机日志块数最大值，等于 CKPT_RLOG_SIZE 参数对应的块个数
5	LOG_CHKPT_TIMEOUT_REDO_BLKS	INTEGER	暂不支持
6	LOG_CHKPT_INTERVAL_REDO_BLKS	INTEGER	检查点间隔，等于 CKPT_INTERVAL 参数值
7	FAST_START_IO_TARGET_REDO_BLKS	INTEGER	暂不支持
8	TARGET_MTTR	INTEGER	有效的故障恢复平均时间，单位秒。若设置了 FAST_START_MTTR_TARGET，等于该值。未设置则为预估的故障恢复平均时间
9	ESTIMATED_MTTR	INTEGER	预估的故障恢复平均时间，单位秒
10	CKPT_BLOCK_WRITES	BIGINT	检查点发起的块写入次数
11	OPTIMAL_LOGFILE_SIZE	INTEGER	根据联机日志故障恢复所需要的时间推荐的联机日志大小，单位 MB
12	ESTD_CLUSTER_AVAILABLE_TIME	INTEGER	共享集群环境下当出现节点故障后可恢复使用的时间。等于预估故障处理需要重演日志的时间
13	WRITES_MTTR	INTEGER	FAST_START_MTTR_TARGET 参数引起的写入次数。等于达到 RLOG_RESERVE_THRESHOLD 而触发的检查点次数
14	WRITES_LOGFILE_SIZE	INTEGER	日志量达到 CKPT_LOG_SIZE 所触发的检查

			点刷盘次数
15	WRITES_LOG_CHECKPOINT_SETTINGS	INTEGER	达到检查点间隔发起的检查点刷盘次数
16	WRITES_OTHER_SETTINGS	INTEGER	达到其他设置条件触发检查点刷盘次数，主要为 CKPT_DIRTY_PAGE 达到设定值触发
17	WRITES_AUTOTUNE	INTEGER	打开 FAST_START_MTTR_TARGET 后，由于 RLOG_RESERVE_THRESHOLD 自动调整时所产生的检查点刷盘次数
18	WRITES_FULL_THREAD_CKPT	INTEGER	完全检查点次数。DSC 环境下只要控制节点上发生

7） V$INI_SYNC_INFO

记录最后一次INI参数同步的时间及错误码。

需要注意：

1） INI同步动作由备库触发，动态视图在对应备库的查询有效；

2） LAST_SYNC_TIME记录的是真正进行了INI同步的时间，而不是触发INI_SYNC的时间。当备库检测到主库INI参数没有修改时，备库为避免无效同步动作产生通信资源浪费，即使此时达到INI_SYNC设置的同步间隔，也不会真正进行同步动作，LAST_SYNC_TIME不会变化；

3） INI_SYNC参数说明：

开启INI_SYNC参数同步后，参数的同步原则如下：

1. 静态 INI 参数：仅同步 dm.ini 文件值，不同步内存值，备库需要重启才能使用新的参数值；

2. 系统级动态 INI 参数：同时同步 dm.ini 文件值与内存值；

3. 会话级动态 INI 参数：同步全局内存值，不同步会话上的 INI 参数值；

4. 若是在 INI 同步改进前后的两个版本间执行，不会真正同步，而是直接返回成功；

5. INI_FILE 类型的参数，只会写入 INI 文件，需要重启才能生效；

6. 不论是否开启 INI_SYNC 参数，对于参数 ENABLE_AUDIT、PWD_POLICY、PWD_MIN_LEN、MAC_LABEL_OPTION、ENABLE_DDL_ANY_PRIV，主库修改后，备库会随重演同步调整 INI 参数值。

序号	列	数据类型	说明
1	LAST_SYNC_TIME	VARCHAR(50)	最后一次 INI 同步时间，没有进行过同步时，此字段为空
2	LAST_SYNC_CODE	INTEGER	最后一次 INI 同步的错误码 CODE，没有进行过同步时，此字段值为 0

13. MAL 系统

1) V$MAL_SYS

MAL 系统信息视图。如果是数据守护环境，则只显示主库的 MAL 系统信息。

序号	列	数据类型	说明
1	SYS_STATUS	INTEGER	MAL 系统状态: 0:OPEN,1:PRE_SHUTDOWN,2:SHUTDOWN
2	STMT_ID	INTEGER	MAL 系统当前 stmtid
3	NEXT_MAL_ID	BIGINT	下一个 MAL_ID
4	MAL_PORT	INTEGER	MAL 监听端口
5	N_SITE	INTEGER	MAL 配置的站点数目
6	MAL_NUM	INTEGER	MAL 系统邮箱数目

7	MAL_SEQ_NO	INTEGER	站点本身的 MAL 序号
8	EMPTY_LET_NUM	INTEGER	空邮件数目
9	MAL_CHECK_INTERVAL	INTEGER	链路检测间隔。单位：秒
10	MAL_CONN_FAIL_INTERVAL	INTEGER	认定链路断开的时间间隔。单位：秒
11	MAL_COMPRESS_LEVEL	INTEGER	邮件压缩级别。取值范围 0~10。0 代表不进行消息压缩，为默认值。 1~9 表示采用 zip 算法，从 1 到 9 表示压缩速度依次递减，压缩率依次递增。 10 表示采用 snappy 算法，压缩速度高于 zip 算法，压缩率相对低。
12	MAL_BUF_SIZE	INTEGER	单个 MAL 缓存大小限制，单位 MB。当 MAL 的缓存邮件超过此大小，会将邮件存储到文件中。取值范围 0~500000，缺省为 100，配置为 0 表示无限制
13	MAL_SYS_BUF_SIZE	INTEGER	MAL 系统总内存大小限制，单位 MB。取值范围 0~500000，缺省为 0，表示无限制
14	MAL_VPOOL_SIZE	INTEGER	MAL 配置的总的 POOL 大小，单位 MB。取值范围 1~500000，缺省为 128
15	MAL_TEMP_PATH	VARCHAR(256)	指定临时文件的目录。当邮件使用的内存超过 MAL_BUF_SIZE 或者 MAL_SYS_BUF_SIZE 时，将新产生的邮件保存到临时文件中。如果缺省，则新产生的邮件保存到 temp.dbf 文件中

16	MAL_CHECK_TIMEOUT	INTEGER	单个实例的 MAL 网络最大延迟时间，0 表示不进行 MAL 网络延迟时间检测。需要和 MAL_CHECK_IP 配合使用。仅当 MAL_CHECK_INTERVAL 不为 0 时有效。单位毫秒（ms），取值范围 0~65534，缺省为 0
17	MAL_CHECK_IP	VARCHAR(128)	第三方确认机器的 IP，用于检测各个实例的 MAL 网络延迟时间。需要和 MAL_CHECK_TIMEOUT 配合使用
18	MAL_SEND_THRESHOLD	INTEGER	最大待发送邮件数，用于控制待发送邮件总数，防止发送邮件堆积。取值范围 1024~10000，缺省为 2048
19	MAL_LEAK_CHECK	INTEGER	是否打开 MAL 内存泄露检查。0：关闭；1：打开。缺省为 0
20	MAL_LOGIN_TIMEOUT	INTEGER	MPP/DBLINK 等实例间登录时的超时检测间隔 3~1800，单位秒，缺省为 15
21	MAL_COMBIN_BUF_SIZE	INTEGER	指定 MAL 消息合并发送的缓冲区大小，单位 KB。MAL 发送消息时进行合并，减少网络发送次数。取值范围 0~131072。0 表示不进行消息合并，缺省为 0。最小缓冲区大小 128KB，当设置值大于 0 小于 128 时，自动调整为 128
22	MAL_MESSAGE_CHECK	INTEGER	是否对 MAL 通信消息启用消息体校验（只有当消息的发送端和接收端都配置为 1 才启用

			通信体校验）。0：不启用；1：启用。缺省为 1
23	MAL_MSG_ENCRYPT	INTEGER	MAL 系统是否启用 ssl 通讯加密。0：不启用，1：启用。缺省为 0
24	MAL_USE_RDMA	INTEGER	Linux 环境下，MAL 是否使用 RDMA 通讯方式。0：否；1：是。缺省为 0

2） V$MAL_INFO

MAL 邮箱信息视图。

序号	列	数据类型	说明
1	MAL_ID	BIGINT	MAL 标识号
2	ORG_SITE_NO	INTEGER	创建 MAL 的原始站点号
3	DEST_SITE_NO	INTEGER	目标站点号
4	MAX_BUF_SIZE	BIGINT	保存邮件的最大缓存大小，单位 BYTE
5	USED_BUF_SIZE	BIGINT	已经使用的缓存大小，单位 BYTE
6	RECEIVE_NUM	INTEGER	收到的邮件数。该统计字段在参数 ENABLE_MONITOR=1 的条件下生效
7	DISCARD_NUM	INTEGER	废弃的邮件数
8	LETTER_NUM	INTEGER	普通邮件数目
9	PLN_LET_NUM	INTEGER	执行计划中的邮件总数目
10	WAIT_FLAG	INTEGER	等待状态。1：等待 0：非等待
11	MCPR_FLAG	INTEGER	MAL 任务处理情况。0：RESUME； 1：MPLN；2：CACEL；4：PAUSE

12	SESS_ID	BIGINT	MAL 对应的会话 ID，和 V$SESSIONS 中的 SESS_ID 对应
13	DONE_NUM	INTEGER	处理的邮件数
14	TOTAL_TIME	BIGINT	邮件处理的总用时，单位秒
15	SEND_LETTER_NUM	INTEGER	发送的邮件数。该统计字段在参数 ENABLE_MONITOR=1 的条件下生效
16	SEND_BYTES	BIGINT	发送的字节总数。该统计字段在参数 ENABLE_MONITOR=1 的条件下生效
17	RECV_LETTER_NUM	INTEGER	接收的邮件数。该统计字段在参数 ENABLE_MONITOR=1 的条件下生效
18	RECV_BYTES	BIGINT	接收的字节总数。该统计字段在参数 ENABLE_MONITOR=1 的条件下生效
19	CURR_TIME	DATETIME(0)	当前系统时间

3) V$MAL_LETTER_INFO

MAL 上的信件信息视图。

序号	列	数据类型	说明
1	MAL_ID	BIGINT	MAL 标识号
2	IS_IN_MEM	INTEGER	是否在缓存中
3	FILE_ID	INTEGER	所在临时表空间的文件号
4	PAGE_NO	INTEGER	所在临时表空间的页号
5	STMT_ID	INTEGER	句柄 ID
6	PLN_OP_ID	INTEGER	通讯操作符的计划序号
7	ORG_SITE	INTEGER	原始站点号
8	MPP_EXEC_ID	BIGINT	操作符的执行序号

9	BUILD_TIME	BIGINT	邮件发送序号
10	P_SRC_SITE	INTEGER	邮件的物理站点号
11	L_SRC_SITE	INTEGER	邮件的逻辑站点号
12	MSG_LEN	INTEGER	信件的消息长度，单位 BYTE
13	TMP_USED_EXTENT_NUM	INTEGER	邮件所占用临时表空间簇的个数

4) V$MAL_USING_LETTERS

服务器中正在使用或者使用过但是没有释放的邮件信息，用于检查 MAL 系统潜在的内存泄露，INI 参数 MAL_LEAK_CHECK 为 1 时有效。

序号	列	数据类型	说明
1	MAL_ID	BIGINT	MAL 标识号
2	STMT_ID	INTEGER	句柄 ID
3	PLN_OP_ID	INTEGER	通讯操作符的计划序号，仅在 MPP 环境下有效
4	ORG_SITE	INTEGER	原始站点号
5	SRC_SITE	INTEGER	源站点号
6	DEST_SITE	INTEGER	目标站点号
7	MPP_EXEC_ID	BIGINT	操作符的执行序号
8	BUILD_TIME	BIGINT	邮件发送序号
9	P_SRC_SITE	INTEGER	邮件的物理站点号
10	L_SRC_SITE	INTEGER	邮件的逻辑站点号
11	CMD_TYPE	VARCHAR(128)	邮件的消息类型
12	MSG_LEN	INTEGER	信件的消息长度，单位 BYTE
13	FILE_NAME	VARCHAR(256)	分配邮件代码的文件名

14	LINENO	INTEGER	分配邮件代码的行号

5） V$ASM_MAL_SYS

ASVRMAL 系统信息视图。仅 DSC 环境可使用。

序号	列	数据类型	说明
1	MAL_CHECK_INTERVAL	INTEGER	链路检测间隔，单位：秒
2	MAL_CONN_FAIL_INTERVAL	INTEGER	认定链路断开的时间间隔，单位：秒
3	MAL_CHECK_TIMEOUT	INTEGER	单个实例的 MAL 网络最大延迟时间，表示不进行 MAL 网络延迟时间检测。需要和 MAL_CHECK_IP 配合使用。仅当 MAL_CHECK_INTERVAL 不为 0 时有效。单位毫秒（ms），取值范围 0~65535，缺省为 0
4	MAL_CHECK_IP	VARCHAR(128)	第三方确认机器的 IP，用于检测各个实例的 MAL 网络延迟时间。需要和 MAL_CHECK_TIMEOUT 配合使用
5	MAL_LOGIN_TIMEOUT	INTEGER	MPP/DBLINK 等实例间登录时的超时检测间隔 3~1800，单位秒，缺省为 15
6	MAL_BUF_SIZE	INTEGER	单个 MAL 缓存大小限制，单位 MB。当 MAL 的缓存邮件超过此大小，会将邮件存储到文件中。取值范围 0~500000，缺省为 100，配置为 0 表示无限制
7	MAL_SYS_BUF_SIZE	INTEGER	MAL 系统总内存大小限制，单位 MB。取值

			范围 0~500000，缺省为 0，表示无限制
8	MAL_VPOOL_SIZE	INTEGER	MAL 配置的总的 POOL 大小，单位 MB。取值范围 1~500000，缺省为 128
9	MAL_MESSAGE_CHECK	INTEGER	是否对 MAL 通信消息启用消息体校验（只有当消息的发送端和接收端都配置为 1 才启用通信体校验）。0：不启用；1：启用。缺省为 1
10	MAL_COMPRESS_LEVEL	INTEGER	邮件压缩级别，取值范围 0~10。0 代表不进行消息压缩，为默认值 1~9 表示采用 zip 算法，从 1 到 9 表示压缩速度依次递减，压缩率依次递增。 10 表示采用 snappy 算法，压缩速度高于 zip 算法，压缩率相对低
11	MAL_USE_RDMA	INTEGER	Linux 环境下，MAL 是否使用 RDMA 通讯方式。0：否；1：是。缺省为 0

6） V$ASM_MAL_INI

ASVRMAL 参数信息。仅 DSC 环境可使用。

序号	列	数据类型	说明
1	MAL_NAME	VARCHAR (128)	MAL 名称
2	MAL_INST_NAME	VARCHAR (256)	实例名称
3	MAL_HOST	VARCHAR (256)	IP 地址
4	MAL_PORT	INTEGER	端口号
5	MAL_INST_HOST	VARCHAR (256)	对应实例 IP 地址

6	MAL_INST_PORT	INTEGER	对应实例的端口号
7	MAL_LINK_MAGIC	INTEGER	MAL 链路网段标识

7) V$MAL_SITE_INFO

MAL 站点信息视图。MPP 模式下，自动收集 MPP 各个站点的信息。

序号	列	数据类型	说明
1	SRC_SITE_SEQ	INTEGER	发送邮件的源站点
2	DEST_SITE_SEQ	INTEGER	目标站点序号
3	MAL_PORT_NUM	INTEGER	目标站点到本站点的 MAL 链路数
4	BUILD_TIME	BIGINT	下一个邮件发送序号
5	LBTAPFDS	BIGINT	收到的邮件中已处理的最后一个邮件的序号
6	CUR_LETTER_NUM	INTEGER	当前保存不连续邮件的个数
7	MAX_LETTER_NUM	INTEGER	目前为止不连续邮件的最大个数
8	TOTAL_LINK_NUM	INTEGER	已创建到目标站点的 MAL_LINK 数
9	FREE_LINK_NUM	INTEGER	当前空闲的 MAL_LINK 数
10	SEND_LETTER_NUM	INTEGER	当前发送的邮件数
11	SEND_BYTES	BIGINT	当前发送的字节总数
12	RECV_LETTER_NUM	INTEGER	当前接收的邮件数
13	RECV_BYTES	BIGINT	当前接收的字节总数
14	CURR_TIME	DATETIME(0)	当前系统时间

8) V$MAL_LINK_STATUS

查询本地实例到远程实例的 MAL 链路连接状态。

序号	列	数据类型	说明

1	SRC_SITE	VARCHAR(256)	源站点实例名
2	DEST_SITE	VARCHAR(256)	目的站点实例名
3	CTL_LINK_STATUS	VARCHAR(256)	控制链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开
4	DATA_LINK_STATUS	VARCHAR(256)	数据链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开

9） V$RDMA_CONN_INFO

MAL 系统使用 RDMA 通信的连接信息视图。

序号	列	数据类型	说明
1	SEND_SIGNAL_CNT	INTEGER	发送带有 SIGNAL 的工作请求个数
2	SEND_CQ_POLL_CNT	INTEGER	发送端 POLL_CQ(轮询完成队列)次数
3	MBLK_SIZE	INTEGER	每次接收消息缓存大小
4	MAX_SEND_WR	INTEGER	允许发送的最大请求数
5	CQ_MOD	INTEGER	发送端生成 CQE(完成队列条目)的批量次数，连续执行 CQ_MOD 次 POST_SEND 生成一个 CQE
6	SEND_NUM	BIGINT	POST_SEND 执行次数，CQ_MOD 配合决定是否生成 CQE，与 MAX_SEND_WR 配置决定是否等待 ACK 响应
7	MAX_INLINE_DATA	INTEGER	发送队列上内联数据最大大小
8	MAX_RECV_WR	INTEGER	允许接收的最大请求数
9	POLL_RCNT	BIGINT	接收操作 POLL_CQ(轮询完成队列)返回总次数
10	COPY_RCNT	BIGINT	已经拷贝走的消息块个数
11	IS_CONNECTED	CHAR(1)	是否成功连接
12	MAL_PORT	INTEGER	MAL 监听端口，用于数据守护、DSC、MPP 等环境中各节点实例之间 MAL 链路配置，监听端端口配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配
13	MAL_HOST	VARCHAR(20)	节点实例
14	CONN_TYPE	VARCHAR(25)	连接链路类型，有三种类型：控制链路、数据链路、DSC 链路；DSC 连接链路只有在 DSC 集群环境下才会出现

14. 通讯

1) V$DBLINK

动态使用到的数据库链接信息视图。

序号	列	数据类型	说明
1	LINK_CONN	BIGINT	DBLINK 连接句柄
2	LINK_ID	INTEGER	DBLINK 的 ID
3	LINK_NAME	VARCHAR(128)	DBLINK 的名称
4	SCH_ID	INTEGER	DBLINK 模式 ID
5	OWNER_ID	INTEGER	DBLINK 用户 ID
6	IS_PUBLIC	VARCHAR(3)	是否为 PUBLIC
7	LOGIN_NAME	VARCHAR(128)	登录名
8	HOST_NAME	VARCHAR(1024)	主机名
9	PORT_NUM	INTEGER	端口号
10	LOGGED_ON	VARCHAR(3)	DBLINK 当前是否已链接
11	HETEROGENEOUS	VARCHAR(3)	DBLINK 同步链接为 YES，异步链接为 NO
12	PROTOCOL	VARCHAR(6)	DBLINK 通信协议
13	IN_USE	VARCHAR(3)	当前 DBLINK 句柄是否正在被使用

15. 系统包

1) V$CACHEPKG

显示当前系统中的包的使用信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	包名

2	N_FIXED	INTEGER	包被引用的次数
3	DISABLED	CHAR(1)	包是否被禁用。"Y"：是，"N"：否
4	MEM_SIZE	INTEGER	包的内存大小，单位 BYTE
5	ADDRESS	VARBINARY(8)	包的地址
6	ID	INTEGER	包 ID
7	CUR_SCHID	INTEGER	当前的模式 ID
8	CUR_USERID	INTEGER	当前的用户 ID
9	IS_DEF_INVOKE	CHAR(1)	是否已定义者权限调用

2) V$DBMS_LOCKS

显示当前系统中的申请的 DBMS_LOCK 包封锁情况。

序号	列	数据类型	说明
1	HANDLE	VARCHAR(128)	封锁的锁对象句柄
2	ID	INTEGER	封锁的锁对象 ID
3	MODE	VARCHAR(10)	封锁的模式
4	OWN_SESS	BIGINT	申请封锁的 session
5	WAIT_SESS	BIGINT	封锁等待的 session，无等待为 NULL

3) V$DB_PIPES

记录使用 DBMS_PIPE 包创建的管道的相关信息。

序号	列名	类型	说明
1	NAME	VARCHAR(128)	管道的名字
2	OWNERID	BIGINT	管道拥有者 ID

3	TYPE	VARCHAR(7)	管道类型:PUBLIC/PRIVATE
4	PIPE_SIZE	BIGINT	管道的长度，单位 BYTE

4) V$SCHEDULER_JOBS_RUNNING

记录数据库中所有由 DBMS_SCHEDULER 包创建且正在执行的作业。

序号	列名	类型	说明
1	SID	BIGINT	作业执行的会话 ID
2	OWNER	VARCHAR(128)	作业执行的所有者
3	JOB_NAME	VARCHAR(128)	作业名称
4	THIS_DATE	DATETIME(6)	作业此次开始执的时间
5	INSTANCE	INTEGER	作业执行的实例（节点号）

5) V$SYSTEM_PACKAGES

显示当前 DM 支持的系统包的有效状态。相比于 SF_CHECK_SYSTEM_PACKAGES，通过本视图能更清晰地看到各系统包的有效状态。部分特殊系统包无法通过该视图查看有效状态，针对视图中无法查到的包，可以查看《DM8 系统包使用手册》获取相应系统包有效状态的查询方法。

序号	列名	类型	说明
1	NAME	VARCHAR(128)	包名
2	ID	INTEGER	包 ID，若尚未创建则为 NULL
3	SCHNAME	VARCHAR(128)	包所属的模式
4	IS_VALID	SMALLINT	包是否有效

16. 审计与加密

1) V$AUDITRECORDS

显示审计文件路径下的审计文件的审计记录，审计文件路径由 INI 参数 AUD_PATH 指定，若未指定，则默认为 SYSTEM_PATH 路径。

此动态性能视图只有在审计开关打开时才有内容，且只有审计用户可以查询。

序号	列	数据类型	说明
1	USERID	INTEGER	用户 ID
2	USERNAME	VARCHAR(128)	用户名
3	ROLEID	INTEGER	角色 ID。 没有具体角色的用户和 SQL 序列审计，没用角色信息
4	ROLENAME	VARCHAR(128)	角色名。 没有具体角色的用户和 SQL 序列审计，没用角色信息
5	IP	VARCHAR(64)	IP 地址
6	SCHID	INTEGER	模式 ID
7	SCHNAME	VARCHAR(128)	模式名
8	OBJID	INTEGER	对象 ID
9	OBJNAME	VARCHAR(128)	对象名
10	OPERATION	VARCHAR(128)	操作类型名
11	SUCC_FLAG	CHAR(1)	成功标记
12	SQL_TEXT	VARCHAR(8188)	SQL 文本
13	DESCRIPTION	VARCHAR(8188)	描述信息
14	OPTIME	DATETIME(6)	操作时间
15	MAC	VARCHAR(25)	操作的 MAC 地址
16	SEQNO	TINYINT	DMDSC 环境下表示生成审计记录的节点号，非 DMDSC 环境下始终 0

17	BIND_INFO	VARCHAR(8188)	绑定参数具体值，如果绑定参数是复杂类型，仅记录类型名，不记录具体值，记录的格式为[第几个参数，参数类型，参数值]

2) V$AUDIT_SPACE

显示审计文件存储空间信息。此动态性能视图只有审计用户可以查询。

序号	列	数据类型	说明
1	AUD_PATH	VARCHAR(256)	审计文件存放路径
2	AUDIT_FILE_FULL_MODE	INTEGER	剩余空间不足时的处理方式: 1：审计文件创建失败，卡住 2：审计文件创建失败，不再审计 3：结合 1 和 2，先尝试删老审计文件，再创建审计文件，失败则不再审计
3	AUDIT_MAX_FILE_SIZE	INTEGER	单个审计文件的最大大小，单位 MB
4	AUD_SPACE_LIMIT	INTEGER	审计文件总存储空间大小限制，单位 MB
5	AUD_SPACE_FREE	INTEGER	可用审计文件存储空间大小，单位 MB
6	AUDIT_IS_NORMAL	CHAR(1)	审计是否正常。Y:是；N:否，不再审计

3) V$CIPHERS

显示系统加密算法和散列算法信息。

序号	列	数据类型	说明
1	CYT_ID	INTEGER	算法 ID
2	CYT_NAME	VARCHAR(256)	算法名
3	CYT_TYPE	INTEGER	算法类型。 1：分组加密算法；

			2：流加密算法； 3：非对称加密算法，保留； 4：散列算法
4	BLOCK_SIZE	INTEGER	分组长度，单位 BYTE
5	KH_SIZE	INTEGER	密钥长度或 HASH 长度，单位 BYTE
6	EXTEND_SIZE	INTEGER	算法扩展长度，单位 BYTE
7	WORK_MODE	VARCHAR(128)	算法模式。 ECB：电子密码本模式； CBC：密码块连接模式； CFB：密文反馈模式。CFB 后缀的数值代表每次迭代处理的位数，默认 CFB 为 CFB128； OFB：输出反馈模式； ECB_NOPAD：非填充 ECB 模式； CBC_NOPAD：非填充 CBC 模式； DEFAULT：未指定或不存在； 其他（EXTKEY、STORE、TRANS、KID、AUTH、LOG）：特殊版本专用

4） V$EXTERNAL_CIPHERS

显示系统中所有的第三方加密算法信息。

序号	列	数据类型	说明
1	ID	INTEGER	算法 ID
2	NAME	VARCHAR(128)	算法名

3	LIB	VARCHAR(300)	算法所在的 lib 库文件名
4	VALID	CHAR(1)	算法是否有效。'Y'：是；'N'：否

5） V$EXTERNAL_CIPHER_LIBS

显示加密引擎的相关信息。

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	加密引擎的名字
2	TYPE	INTEGER	获取加密引擎类型。0：第三方软加密；1：第三方硬件加密；2：UKEY
3	LIB_PATH	VARCHAR(256)	加密引擎所在的 lib 库文件名

6） V$AUDIT_FILE

显示系统中的所有审计文件路径。此动态性能视图只有审计用户可以查询。

序号	列	数据类型	说明
1	FILEPATH	VARCHAR(256)	审计文件的完整路径
2	SGUID	INTEGER	服务器的唯一标识，仅用于判断审计文件与库是否匹配
3	RT_FLAG	INTEGER	是否实时审计
4	HPC_SEQNO	INTEGER	DSC 节点号
5	VERSION	INTEGER	版本号
6	DBNAME	VARCHAR(128)	库名称
7	IS_FULL	INTEGER	审计文件是否已满。1：已满，0：未满。
8	BEGIN_TIME	DATETIME(6)	审计文件的创建时间
9	LENGTH	BIGINT	审计文件的长度，单位 BYTE

7） V$DBMSAUDIT_FILE

显示 DBMS_AUDIT 系统包中的添加的审计文件路径。此动态性能视图只有审计用户可

以查询。

序号	列	数据类型	说明
1	FILEPATH	VARCHAR(256)	审计文件的完整路径
2	SGUID	INTEGER	服务器的唯一标识，仅用于判断审计文件与库是否匹配
3	RT_FLAG	INTEGER	是否实时审计
4	HPC_SEQNO	INTEGER	DSC 节点号
5	VERSION	INTEGER	版本号
6	DBNAME	VARCHAR(128)	库名称
7	IS_FULL	INTEGER	审计文件是否已满。1：已满，0：未满。
8	BEGIN_TIME	DATETIME(6)	审计文件的创建时间
9	LENGTH	BIGINT	审计文件的长度，单位 BYTE

附录 3 V 视图

本节介绍的视图名称均以 V 开头，因此统称为 V 视图。V 视图均为基于系统数据字典而建立的视图，命名方式为：V+数据字典表名。

V 视图的结构和相关数据字典表的结构完全相同。但因为权限不同，拥有不同角色的用户查看到的内容会各不相同。

V 视图的模式名为 SYS。例如：SYS.VSYSOBJECTS。

例 查看 V 视图 VSYSOBJECTS。

```
SELECT * FROM SYS.VSYSOBJECTS;
```

如果系统升级，可使用 SP_INIT_SVI_SYS(1);重建所有 V 视图。

1. 基础 V 视图

1）与原表内容不同的 V 视图

管理员/审计员/安全员/操作员查看本节的 V 视图可获得和原数据字典表相同的内容。而普通用户在 V 视图中仅能查看到自己相关的信息。

序号	视图	说明
1	VSYSACCHISTORIES	结构与用途和 SYSACCHISTORIES 一样
2	VSYSRESOURCES	结构与用途和 SYSRESOURCES 一样
3	VSYSUSERPROFILES	结构与用途和 SYSUSERPROFILES 一样
4	VSYSPWDCHGS	结构与用途和 SYSPWDCHGS 一样
5	VSYSPROFILES	结构与用途和 SYSPROFILES 一样
6	VSYSOBJECTS	结构与用途和 SYSOBJECTS 一样
7	VSYSOPARGS	结构与用途和 SYSOPARGS 一样

2）与原表内容相同的 V 视图

本节的 V 视图和原数据字典表查看到的内容相同。

序号	视图	说明
1	VSYSDEPENDENCIES	结构与用途和 SYSDEPENDENCIES 一样
2	VSYSOBJINFOS	结构与用途和 SYSOBJINFOS 一样
3	VSYSGRANTS	结构与用途和 SYSGRANTS 一样
4	VSYSTEXTS	结构与用途和 SYSTEXTS 一样
5	VSYSCOLUMNS	结构与用途和 SYSCOLUMNS 一样
6	VSYSCOLINFOS	结构与用途和 SYSCOLINFOS 一样
7	VSYSCOLCYT	结构与用途和 SYSCOLCYT 一样
8	VSYSFCOLDEFAULT	结构与用途和 SYSFCOLDEFAULT 一样
9	VSYSCONTEXTINDEXES	结构与用途和 SYSCONTEXTINDEXES 一样
10	VSYSHPARTTABLEINFO	结构与用途和 SYSHPARTTABLEINFO 一样
11	VSYSDISTABLEINFO	结构与用途和 SYSDISTABLEINFO 一样
12	VSYSCONS	结构与用途和 SYSCONS 一样
13	VSYSINDEXES	结构与用途和 SYSINDEXES 一样
14	VSYSTABLECOMMENTS	结构与用途和 SYSTABLECOMMENTS 一样
15	VSYSCOLUMNCOMMENTS	结构与用途和 SYSCOLUMNCOMMENTS 一样
16	VSYSCONTEXTLIBS	结构与用途和 SYSCONTEXTLIBS 一样
17	VSYSINJECTHINT	结构与用途和 SYSINJECTHINT 一样
18	VSYSOPENHISTORY	结构与用途和 SYSOPENHISTORY 一样
19	VSYSMSTATS	结构与用途和 SYSMSTATS 一样
20	VSYSSTATPREFS	结构与用途和 SYSSTATPREFS 一样

21	VSYSSTATS	结构与用途和 SYSSTATS 一样
22	VSYSSTATTABLEIDU	结构与用途和 SYSSTATTABLEIDU 一样
23	VSYSTUNINGEXECUTION	结构与用途和 SYSTUNINGEXECUTION 一样
24	VSYSTUNINGREPORT	结构与用途和 SYSTUNINGREPORT 一样
25	VSYSTUNINGTASK	结构与用途和 SYSTUNINGTASK 一样
26	VSYS_REWRITE_EQUIVALENCES	结构与用途和 SYS_REWRITE_EQUIVALENCES 一样。SYS_REWRITE_EQUIVALENCES 请参考《DM8 系统包使用手册》的 DBMS_ADVANCED_REWRITE 包
27	VSYSMODIFICATIONS	结构与用途和 SYSMODIFICATIONS 一样
28	VSYSPKGPROCINFOS	结构与用途和 SYSPKGPROCINFOS 一样
29	VSYSPKGPROCPARAMS	结构与用途和 SYSPKGPROCPARAMS 一样

3）与原表内容不同的 V 视图

本节的 V 视图和数据字典 SYSOBJECTS 结构完全相同。

管理员/审计员/安全员/操作员查看本节的 SVI 视图可获得和 SYSOBJECTS 一样的内容。而普通用户在 V 视图中仅能查看到自己相关的信息。

序号	视图	说明
1	VSYSOBJTABLES	普通用户有权限访问的 SYSOBJECTS 中的表对象
2	VSYSOBJVIEWS	普通用户有权限访问的 SYSOBJECTS 中的视图对象
3	VSYSOBJINDEXES	普通用户有权限访问的 SYSOBJECTS 中的索引对象
4	VSYSOBJCONS	普通用户有权限访问的 SYSOBJECTS 中的约束对象
5	VSYSOBJPROCS	普通用户有权限访问的 SYSOBJECTS 中的过程对象
6	VSYSOBJPKGS	普通用户有权限访问的 SYSOBJECTS 中的包对象
7	VSYSOBJTRIGS	普通用户有权限访问的 SYSOBJECTS 中的触发器对象

2. 安全 V 视图

拥有 DB_POLICY_SVI 角色才可以查看的安全视图。

本节的 V 视图和原数据字典表查看到的内容相同。

序号	视图	说明
1	VSYSMACCOMPS	结构与用途和 SYSMACCOMPS 一样
2	VSYSMACGRPS	结构与用途和 SYSMACGRPS 一样
3	VSYSMACLABELS	结构与用途和 SYSMACLABELS 一样
4	VSYSMACLVLS	结构与用途和 SYSMACL 结构与用途 LS 一样
5	VSYSMACOBJ	结构与用途和 SYSMACOBJ 一样
6	VSYSMACPLYS	结构与用途和 SYSMACPLYS 一样
7	VSYSMACTABPLY	结构与用途和 SYSMACTABPLY 一样
8	VSYSMACUSRPLY	结构与用途和 SYSMACUSRPLY 一样

3. 审计 V 视图

拥有 DB_AUDIT_SVI 角色权限才可以查看的审计视图。

本节的 V 视图和原数据字典表查看到的内容相同。

序号	视图	说明
1	VSYSAUDIT	结构与用途和 SYSAUDIT 一样
2	VSYSAUDITRULES	结构与用途和 SYSAUDITRULES 一样
3	VSYSAUDITSQLSEQ	结构与用途和 SYSAUDITSQLSEQ 一样

附录 4 执行计划操作符

操作符名称	参数说明	操作说明
AAGR2	grp_num：分组项个数	简单聚集。如果没有分组，则直接计算集函数
	sfun_num：返回上层操作符的集函数个数	
	distinct_flag：集函数参数是否去重	
	slave_empty: 是否限制 MPP 从节点向主节点传送数据, 或本地并行的从线程向主线程传送数据。取值：0：不限制；1：不允许 MPP 从节点向主节点传送数据；2：不允许本地并行的从线程向主线程传送数据	
	dist_arg_opt 参数：标记是否做 distinct 优化（分组项+该 distinct 参数 刚好组成 aagr/sagr 使用索引的前导列，那么计算该参数的 dist 时无需使用 hash 表，直接相邻行比较即可(分组项组内参数已有序)），dist_arg_opt(a1,a2,...an)，a 对应每个 distinct 参数，取值 0 代表此 distinct 参数不优化计算，反之则为 1	
ACTRL	无参数	使用自适应计划机制时，控制备用计划转换
AFUN	afun_num：分析函数个数	分析函数计算
	partition_num：分区项个数	

	[colnamelist]：分区项	
	order_num：排序项个数	
	[colnamelist]：排序项	
ASCN	tabname：表的别名	数组当作表来扫描
	used_in_sss3：是否出现在 sss3(subquery set search，子查询集合查找)指令中。值为 TRUE 时，则每次扫描都重新开始	
ASSERT	assert_condition：触发断言的布尔表达式条件	约束检查
BLKUP2	idxname(tabname)：索引名（表名）。 use_clu_addr：是否从二级索引记录读取对应的聚集索引记录地址对定位聚集索引记录进行优化	定位查找
BMAND	无参数	位图索引的与运算
BMCNT	is_match_out：是否直接根据孩子节点的 MATCH-COUNT 值计算 COUNT(*)	位图索引的行数计算
BMCVT	无参数	位图索引的 ROWID 转换
BMMG	无参数	位图索引归并
BMOR	无参数	位图索引的或运算
BMSEK	scan_type：扫描类型	位图索引的范围查找
	idxname(tabname)：索引名（表名）	
	scan_range：扫描范围	
CONST VALUE LIST	row_num：常量列表行数	常量列表

	col_num：常量列表列数	
CONSTC	N_PRE_VAR：复合索引跳跃扫描中跳跃扫描的索引前导列个数	用于复合索引跳跃扫描
	(out_var)：外部 var 变量序列	
CSCN2	idxname(tabname)：索引名（表名）	聚集索引扫描
	need_slct(1)：是否进行过滤条件下推的优化	
	btr_scan：等于 1 表示使用 B 树扫描，等于 0 表示使用簇游标扫描	
	SAMPLE 标记：用于标记节点上是否存在 sample 表达式，存在则带有此标记，否则无	
CSEK2	scan_type：扫描类型	聚集索引数据定位
	idxname(tabname)：索引名（表名）	
	scan_range：扫描范围	
CTE_SCN	exp_num：映射列数	递归 WITH 查询
	(queryname)：查询名	
CTNS	idxname(tabname)：索引名（表名）	用于实现全文索引的 CONTAINS
	KEY：全文索引的关键词	
DELETE	(partition_type)：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	删除数据
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	
	type：删除类型，取值：SELECT、CURSOR	

	mpp_opt: 是否通过发送整个计划实现对 MPP 处理的优化	
	hp_opt：是否进行分区列等值过滤优化	
DELETE_REMOTE	(partition_type)：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	DBLINK 删除操作
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	
	type：删除类型，取值：SELECT、CURSOR	
	mpp_opt: 是否通过发送整个计划实现对 MPP 处理的优化	
	hp_opt：是否进行分区列等值过滤优化	
DISTINCT	keys：MPP 模式下需要分发的列的信息（仅在 MPP 模式下生效）	去重
	L_FULL_OPT_SEMI 标记：semi 半连接支持 l_full 优化	
	L_FULL_OPT_ANTISEMI 标记：anti semi 连接支持 l_full 优化	
	L_FULL_OPT_JOIN 标记：其他非 semi 连接支持 l_full 优化	
DSCN	idxname(tabname) ：索引名（表名）	动态视图表扫描
DSSEK	scan_type：扫描类型	DISTINCT 列上索引跳跃扫描(单列索引或复合索引)
	idxname(tabname) ：索引名（表名）	

	scan_range：扫描范围	
ESCN	无参数	外部表扫描
EXCEPT	无参数	集合的差运算，且取差集后删除重复项
EXCEPT ALL	无参数	集合的差运算，且取差集后不删除重复项
EHFD	mpp_opt：数据操作方式，1：数据直接发送给 BP 端进行删除；0：通过 SP 端分发	DPC 下事务型 HUGE 表的数据删除
EHFI	(tabname)：表名	DPC 下事务型 HUGE 表的数据插入
	mpp_opt：数据操作方式，1：数据直接发送给 BP 端进行插入；0：通过 SP 端分发	
EHFINS	(tabname)：表名	DPC 下非事务型 HUGE 表的数据插入
	mpp_opt：数据操作方式，1：数据直接发送给 BP 端进行插入；0：通过 SP 端分发	
EHFU	mpp_opt：数据操作方式，1：数据直接发送给 BP 端进行更新；0：通过 SP 端分发	DPC 下事务型 HUGE 表的数据更新
ERECV	stask_no：自身所处的子任务的序号，-1 表示 root 子任务	DPC 下用于接收数据，和 ESEND 对应
	l_stask_no：孩子子计划的序号	
	n_keys：归并排序的 key 的个数	
	trig：是否在 IDU（增删改操作）上存在触发器	
	PV_FLAG：为 1 时开启限流，严格一发一收	

	topN 标记：存在此标记则代表 erecv 节点上含有 top 语句	
ESEND	stask_no：自身所处的子任务的序号，-1 表示 root 子任务	DPC 下用于发送数据，和 ERECV 对应
	type：发送类型，用于确定基于何种数据特征对数据进行分发。详细介绍请参考《DM8 分布计算集群》5.4 小节“数据交换与数据迭代操作符”	
	sites：站点的 RAFT ID 和并行度值，“-”表示空。例如(1:3,2:4)表示 RAFT ID 1 的并行度为 3，RAFT ID 2 的并行度为 4	
	sql_invoke：当前子任务是否包含了含 sql 的函数调用	
	pwj_opt：是否使用了分区智能连接（Partition Wise Join）优化	
	table：表名，“-”表示空，不为空时表示按照该表的分区定义进行数据分发	
	empty_type：当哈希连接相关优化的分发计算无合适接收端时的处理策略，取值：INIT：未指定策略，效果等同于 ERROR；PRUNE：直接抛弃；ANY：给任意线程处理；ERROR：报错；NOT_EMPTY：通知所有线程右孩子不为空集	
	keys：表的分发列(仅当 table 不为空时显示)	

	INFO_BITS：相关优化信息的标志位，6 个标志位自低向高含义如下：bit0：是否在哈希连接左表为广播数据时共享哈希表；bit1：是否使用 UNIONALL 优化；bit2：是否选择开始执行任务；bit3：是否进行链路发送优化；bit4：是否进行归并排序的链路发送优化；bit5：用于指示当前 ESEND 发送的目标进行归并排序	
	PV_FLAG：为 1 时开启限流，严格一发一收	
	PI 标记：存在此标记则代表 esend 是并行插入的 esend	
FAGR2	sfun_num：返回上层操作符的集函数个数	快速聚集，如果没有 where 条件，且取 count(*)，或者基于索引的 MAX/MIN 值，则可以快速取得集函数的值
	idxname：索引名	
FILL BTR	index_id：用于填充 B 树的索引 ID	填充 B 树
FTTS	for_mdis：是否从 MPP DISTRIBUTE 获得数据	MPP\LPQ 下，对临时表的优化
GI	policy：指示表的数据访问粒度，各个取值和含义请参考《DM8 分布计算集群》中 5.4 小节"数据交换与数据迭代操作符"	Granule Iterator，在 DMDPC 集群中，控制各工作线程的数据访问粒度和分区表裁剪
	gi_unit：当前 GI 迭代器控制的数据扫描操作符起止序号	

	scan_type[0]：扫描相关信息，格式为：[扫描类型，反向扫描信息（可选），末尾 KEY 优化信息（可选）]	
	dynamic_pll：是否包含动态裁剪	
	has_hp_col_filter 标记：用于标记下放是否包含有 slct，且 slct 仅包含范围分区列的过滤（is_hp_col_filter == TRUE）。has_hp_col_filter(n)中的 n 表示 GI 节点下所有叶子操作符属性 has_hp_col_filter==TRUE 的数目	
HAGR2	grp_num：分组项个数	HASH 分组，并计算集函数
	sfun_num：返回上层操作符的集函数个数	
	distinct_flag：集函数参数是否去重	
	top_num：是否只返回前 N 组	
	slave_empty：是否限制 MPP 从节点向主节点传送数据，或本地并行的从线程向主线程传送数据。取值：0：不限制；1：不允许 MPP 从节点向主节点传送数据；2：不允许本地并行的从线程向主线程传送数据	
	keys：分组项	
	real_key：依赖分组优化中实际使用的分组列	
HASH FULL JOIN2	key_num：等值连接条件数	HASH 全外连接

	mix_aggr：是否将分组操作下放到此步骤进行，仅支持 COUNT 和 SUM 函数	
	mix_dist：是否存在对分组操作和哈希左外连接的优化（即把分组操作下放到哈希左外连接中）	
	join_condition: 除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
HASH LEFT JOIN2	key_num：等值连接条件数	HASH 左外连接
	partition_keys_num: paritition outer join 的 KEY 个数	
	ret_null：是否仅返回未匹配的行	
	mix：需要计算的单个右表列的 COUNT 函数个数	
	join_condition: 除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	

HASH LEFT SEMI JOIN12	(ANTI)：是否为反连接	HASH 左半连接，其与 HASH LEFT_SEMI_JOIN2 的区别是该计划为 semi join with rowid key，后者是 semi join with exp key
	join_condition: 除等值连接条件以外的连接条件	
HASH LEFT SEMI JOIN2	(ANTI)：是否为反连接	HASH 左半连接
	key_num：等值连接条件数	
	join_condition: 除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL 相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
HASH LEFT SEMI MULTIPLE JOIN	(ANTI)：是否为反连接	实现多列 IN/多列 NOT IN
	KEY：等值连接条件	
	join_condition: 除等值连接条件以外的连接条件	
HASH RIGHT JOIN2	key_num：等值连接条件数	HASH 右外连接
	ret_null：是否仅返回未匹配的行	

	join_condition: 除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
HASH RIGHT SEMI JOIN2	key_num：等值连接条件数	HASH 右半连接
	(ANTI)：是否为反连接	
	join_condition: 除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL 相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
	INFO_BITS：相关优化信息的标志位，其中 bit0 指明是否在左表为广播数据时共享 0 号工作节点哈希表	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	

HASH RIGHT SEMI JOIN32	(any_options)：取值包括："= all"、"< > any"、"op all"、"op any"	用于 OP SOME/ANY/ALL 的 HASH 右半连接
	key_num：等值连接条件数	
	join_condition：除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL 相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
HASH2 INNER JOIN	(UNIQUE_FLAG)：该连接中左右表数据的唯一性策略。取值：LRKEY_UNIQUE：左右表数据分别根据各自 KEY 值唯一；LKEY_UNIQUE：左表数据根据 KEY 值唯一，右表则不唯一；RKEY_UNIQUE:右表数据根据 KEY 值唯一，左表则不唯一	HASH 内连接
	key_num：等值连接条件数	
	join_condition：除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL	

	相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
	INFO_BITS：相关优化信息的标志位，6 个标志位自低向高含义如下：bit0：是否在哈希连接左表为广播数据时共享 0 号工作节点的哈希表；bit1：是否使用 UNIONALL 优化；bit2：是否选择开始执行任务；bit3：是否进行链路发送优化；bit4：是否进行链路发送优化，且需要进行归并；bit5：用于指示当前 ESEND 发送的目标进行归并排序	
HEAP TABLE	stask_no：子任务的序号，-1 表示 root 子任务	临时结果表
	sql_invoke：是否已在所属子计划进行含 sql 的函数调用	
	table_no：临时结果表编号	
	full：并行场景数据完整性，取值：0：局部数据；1：完整数据	
	mpp_full：是否在 MPP/DPC 模式下表示完整数据	
	autoid：是否生成 autoid 列	
	sites：站点的 ID 和并行度序列，“-”表示空。例如：(1:3,2:4)表示 1 号站点并行度 3，2 号站点并行度 4	

HEAP TABLE SCAN	table_no：临时结果表编号	临时结果表扫描
HFD	mpp_opt: 是否通过发送整个计划实现对 MPP 处理的优化	删除事务型 HUGE 表数据
HFDEL	mpp_opt: 是否通过发送整个计划实现对 MPP 处理的优化	删除非事务型 HUGE 表数据
HFI	(tabname)：表名 mpp_opt：MPP 模式下插入优化选项，取值：-1：不优化，DPC 环境下取该值；0：不优化，无优化或非 MPP 环境下取该值；1：在随机插入目标表时分发整个插入计划；2：基于 1 的特性，省略 MDIS 分发	事务型 HUGE 表插入记录
HFI2	(tabname)：表名 mpp_opt：MPP 模式下插入优化选项，取值：-1：不优化，DPC 环境下取该值；0：不优化，无优化或非 MPP 环境下取该值；1：在随机插入目标表时分发整个插入计划；2：基于 1 的特性，省略 MDIS 分发	MPP 模式下优化的事务型 HUGE 表插入记录
HFINS2	mpp_opt：MPP 模式下插入优化选项，取值：-1：不优化，DPC 环境下取该值；0：不优化，无优化或非 MPP 环境下取该值；1：在随机插入目标表时分发整个插入计划；2：基于 1 的特性，省略 MDIS 分发	非事务型 HUGE 表插入记录

HFINS3	无参数	MPP 模式下优化的非事务型 HUGE 表插入记录
HFINS4	无参数	非 MPP 模式下，针对非事务型 HUGE 水平分区主表的插入优化，需要参数 HFINS_PARALLEL_FLAG=2
HFLKUP	(tabname)：表名	根据 ROWID 检索非事务型 HUGE 表数据
HFLKUP2	(tabname)：表名	根据 ROWID 检索事务型 HUGE 表数据
HFS UPDATE	table：表名	更新非事务型 HUGE 表数据
	type：类型，取值：SELECT、CURSOR	
	mpp_opt: 是否通过发送整个计划实现对 MPP 处理的优化	
HFSCN	(tabname)：表名	非事务型 HUGE 表的逐行扫描
	SAMPLE 标记：用于标记节点上是否存在 sample 表达式，存在则带有此标记，否则无	
HFSCN2	(tabname)：表名	事务型 HUGE 表的逐行扫描
	need_slct(1)：是否进行过滤条件下推的优化	
	SAMPLE 标记：用于标记节点上是否存在 sample 表达式，存在则带有此标记，否则无	
HFSEK	(tabname)：表名	根据 KEY 检索非事务型 HUGE 表数据
	scan_type：扫描类型	

	scan_range：扫描范围	
	SAMPLE 标记：用于标记节点上是否存在 sample 表达式，存在则带有此标记，否则无	
HFSEK2	(tabname)：表名	根据 KEY 检索事务型 HUGE 表数据
	scan_type：扫描类型	
	scan_range：扫描范围	
	SAMPLE 标记：用于标记节点上是否存在 sample 表达式，存在则带有此标记，否则无	
HFU	table：表名	更新事务型 HUGE 表数据
	type：类型，取值：SELECT、CURSOR	
	mpp_opt: 是否通过发送整个计划实现对 MPP 处理的优化	
HIERARCHICAL QUERY	(UNIQUE)：结果集是否去重	层次查询
	key_num：等值连接条件数	
	connect_by_condition：连接条件	
HPM	order_keys：排序项	水平分区表归并排序
	is_distinct：是否对排序结果进行去重操作	
	top_flag：是否有 TOP 子句需要处理	
	pll_scan_type: 并行处理时扫描相关信息，格式为：[扫描类型，反向扫描信息（可选），末尾 KEY 优化信息（可选）]	
	pll_keys: 并行处理时分区列计算表达式相关信息，格式为：（分区列的计算表达式中相等的个	

	数，分区列的计算表达式中最后一列起始的个数，分区列的计算表达式中最后一列终止的个数）	
INDEX JOIN LEFT JOIN2	join_condition：连接条件	索引左连接
	ret_null：是否仅返回未匹配的行	
INDEX JOIN SEMI JOIN2	（ANTI）：是否为反连接	索引半连接
	join_condition：连接条件	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
INSERT	（partition_type）：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	插入记录
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	
	type：插入类型，取值：UNKNOWN、VALUES、SELECT、MVALUE	
	hp_opt：是否进行分区列等值过滤优化	
	mpp_opt：MPP 模式下插入优化选项，取值：-1：不优化，DPC 环境下取该值；0：不优化，无优化或非 MPP 环境下取该值；1：在随机插入对目标表时分发整个插入计划；2：基于 1 的特性，省略 MDIS 分发	

INSERT_LIST	(partition_type)：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	堆表插入
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	
	type：插入类型，取值：UNKNOWN、VALUES、SELECT、MVALUE	
	hp_opt：是否进行分区列等值过滤优化	
	mpp_opt：MPP 模式下插入优化选项，取值：-1：不优化，DPC 环境下取该值；0：不优化，无优化或非 MPP 环境下取该值；1：在随机插入目标表时分发整个插入计划；2：基于 1 的特性，省略 MDIS 分发	
INSERT_REMOTE	(partition_type)：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	DBLINK 插入操作
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	
	type：插入类型，取值：UNKNOWN、VALUES、SELECT、MVALUE	
	hp_opt：是否进行分区列等值过滤优化	
	mpp_opt：MPP 模式下插入优化选项，取值：-1：不优化，DPC 环境下取该值；0：不优化，	

	无优化或非 MPP 环境下取该值；1：在随机插入目标表时分发整个插入计划；2：基于 1 的特性，省略 MDIS 分发	
INSERT3	(partition_type)：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	MPP 模式下查询插入优化处理
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	
	type：插入类型，取值：UNKNOWN、VALUES、SELECT、MVALUE	
	hp_opt：是否进行分区列等值过滤优化	
	mpp_opt：MPP 模式下插入优化选项，取值：-1：不优化，DPC 环境下取该值；0：不优化，无优化或非 MPP 环境下取该值；1：在随机插入目标表时分发整个插入计划；2：基于 1 的特性，省略 MDIS 分发	
INTERSECT	无参数	集合的交运算，且取交集后删除重复项
INTERSECT ALL	无参数	集合的交运算，且取交集后不删除重复项
LOCAL BROADCAST	op_id: 本地并行模式下节点所属通讯操作符中的序号	本地并行模式下，消息广播到各线程，包含必要的集函数合并计算

LOCAL COLLECT	op_id: 本地并行模式下节点所属通讯操作符中的序号	本地并行模式下数据收集处理，代替 LOCAL GATHER
	n_grp_by：分组列个数	
	n_cols：输出列个数	
	n_keys：分发列的个数	
	for_sync：该操作符是否仅用于同步	
LOCAL DISTRIBUTE	op_id: 本地并行模式下节点所属通讯操作符中的序号	本地并行模式下，消息各线程的相互重分发
	n_keys：分发列的个数	
	n_grp：分组列个数	
	flt_only：是否只需过滤非当前站点。各站点均拥有完整数据时，分发时只需要过滤掉非当前站点的数据	
	flt_site_data：是否需要过滤站点数据，仅当参数 flt_only 为 TRUE 时生效	
	pipe_mode(TRUE)：是否可优化分区表	
	n：本地并行工作的线程数	
	fbtr_flag：是否用于 fbtr(fast insert b tree)。是则为 fbtr_flag(1)，否则为 fbtr_flag(0)	
LOCAL GATHER	op_id: 本地并行模式下节点所属通讯操作符中的序号	本地并行模式下，消息收集到主线程
	n_grp_by：分组列个数	
	n_cols：输出列个数	

	n_keys：分发列的个数	
	invoke_flag：是否在父亲节点存在函数调用的表达式计算。值为 TRUE 时从 EP 不执行该计算且直接丢弃儿子节点相关数据	
	top_flag：是否有 TOP 子句需要处理	
LOCAL SCATTER	op_id：本地并行模式下节点所属通讯操作符中的序号	本地并行模式下，主线程向各从线程广播消息
	notify_only：邮件是否携带通知标记	
LOCK TID	无参数	上锁
LSET	无参数	DBLINK 查询结果集
MERGE INNER JOIN3	key_num：等值连接条件数	归并内连接
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL 相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
MERGE FULL JOIN	key_num：等值连接条件数	归并全外连接
	join_condition：除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0：NULL 与 NULL 不相等；1：NULL 与 NULL	

	相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
MERGE LEFT JOIN	key_num：等值连接条件数	归并左外连接
	join_condition: 除等值连接条件以外的连接条件	
	ret_null：是否仅返回未匹配的行	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL 相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
MERGE RIGHT JOIN	key_num：等值连接条件数	归并右外连接
	join_condition: 除等值连接条件以外的连接条件	

	ret_null：是否仅返回未匹配的行	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL 相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
MERGE SEMI JOIN3	(ANTI)：是否为反连接	归并半连接
	join_condition: 除等值连接条件以外的连接条件	
	KEY：等值连接条件	
	KEY_NULL_EQU：连接列的 NULL 值比较策略。取值：0:NULL 与 NULL 不相等；1：NULL 与 NULL 相等。例如：(0,1,0)表示第 1 个和第 3 个连接条件中 NULL 值视为不等，第 2 个连接条件中 NULL 值视为相等	
MPP BROADCAST	op_id：MPP 模式下节点所属通讯操作符中的序号	MPP 模式下，消息广播到各站点，包含必要的集函数合并计算
MPP COLLECT	op_id：MPP 模式下节点所属通讯操作符中的序号	用于替换顶层 MPP GATHER，除了收集数据到主节点，还增加主从节

	n_grp_by：分组列个数	点间的同步执行功能，防止从节点不断发送数据到主节点造成邮件堆积
	n_cols：输出列个数	
	n_keys：分发列的个数	
	for_sync：该操作符是否仅用于同步	
MPP DISTRIBUTE	op_id：MPP 模式下节点所属通讯操作符中的序号	MPP 模式下，消息各站点的相互重分发
	n_keys：分发列的个数	
	n_grp：分组列个数	
	filter：是否只需过滤非当前站点。各站点均拥有完整数据时，分发时只需要过滤掉非当前站点的数据	
	rowid_flag：是否根据 rowid 信息获取站点号进行分发	
	n：参与计算的站点数，值为 0 时表示其与 EP 个数一致	
	slave_empty：是否限制 MPP 从节点向主节点传送数据，或本地并行的从线程向主线程传送数据。取值：0：不限制；1：不允许 MPP 从节点向主节点传送数据；2：不允许本地并行的从线程向主线程传送数据	
	fbtr_flag：是否用于 fbtr(fast insert b tree)。是则为 fbtr_flag(1)，否则为 fbtr_flag(0)	

MPP GATHER	op_id：MPP 模式下节点所属通讯操作符中的序号	MPP 模式下，消息收集到主站点
	n_grp_by：分组列个数	
	n_cols：输出列个数	
	n_keys：分发列的个数	
	invoke_flag：是否在父亲节点存在函数调用的表达式计算。值为 TRUE 时，从 EP 不执行该计算且直接丢弃儿子节点相关数据	
	top_flag：是否有 TOP 子句需要处理	
MPP SCATTER	op_id：MPP 模式下节点所属通讯操作符中的序号	MPP 模式下，主站点向各从站点广播消息
MSYNC	op_id：MPP 模式下节点所属通讯操作符中的序号	MPP 模式下，数据同步处理
MVCC CHECK	无参数	多版本检查
NCUR2	cursorname：游标名	游标操作
NEST LOOP FULL JOIN2	join_condition：连接条件	嵌套循环全外连接
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
NEST LOOP INDEX JOIN2	join_condition：连接条件	索引内连接
NEST LOOP INNER JOIN2	join_condition：连接条件	嵌套循环内连接
	[with_var]：嵌套连接是否使用 var 方式	

NEST LOOP LEFT JOIN2	join_condition：连接条件	嵌套循环左外连接
	[with_var]：嵌套连接是否使用 var 方式	
	partition_keys_num: paritition outer join 的 KEY 个数	
	ret_null：是否仅返回未匹配的行	
NEST LOOP SEMI JOIN2	(ANTI)：是否为反连接	嵌套循环半连接
	join_condition：连接条件	
	[with_var]：嵌套连接是否使用 var 方式	
	(colidlist)：与 var 变量相关的左表列序号序列。例如(0,1,3)表示三个 var 变量依次来自左表的第 1、2、4 列	
	flt_batch_exec 标记：节点上是否存在 filter 的批量指令，存在则为 flt_batch_exec(1)，否则为 flt_batch_exec(0)	
NSET2	无参数	结果集收集，一般是查询计划的顶层节点
NTTS2	for_mdis：是否从 MPP DISTRIBUTE 获得数据	临时表，临时存放数据
	is_atom：是否要求单行数据	
PARALLEL	scan_type：分区表子分区裁剪方式。其返回值为 FULL 表示所有子分区裁剪，EQU 表示等值子分区裁剪，NULL EQU 表示空值子分区裁剪，GE_LE 表示闭区间裁剪，G_LE 表示左开右闭区间裁剪，GE_L 表示左闭右开区间裁	控制水平分区子表的扫描

	剪，G_L 表示开区间裁剪，GE 表示左闭区间裁剪，LE 表示右闭区间裁剪，G 表示左开区间裁剪，L 表示右开区间裁剪，CONTAIN 表示等值全文索引扫描，NOT CONTAIN 表示空值全文索引扫描 V_AND 表示列存储表上的多列 rowid AND 后查找，REMOTE 表示 dblink 的远程查询，BITMAP 表示位图索引扫描，IN LIST 表示分区表首列 in list 优化扫描，BITMAP JOIN 表示位图连接索引扫描，INDEX SKIP 表示索引跳读（用于复合索引,过滤条件缺少前导列情况），-表示扫描条件缺失	
	key_num：分区列计算表达式相关信息，格式为：（分区列的计算表达式中相等的个数，分区列的计算表达式中最后一列起始的个数, 分区列的计算表达式中最后一列终止的个数）	
	simple：是否对包含单分区列的一级分区表进行等值查询优化	
	range_sfun_opt：aagr+pll+cscn 对于 range 分区列求 min/max 时的优化标记，若做了优化则为 range_sfun_opt(1)，否则为 range_sfun_opt(0)	
PIPE2	无参数	管道。PIPE2 操作符仅对左孩子的数据进行过滤处理，在左孩子的处

		理过程中，会触发 PIPE2 操作符的右孩子节点执行，并用左孩子数据与之进行匹配过滤
PRJT2	exp_num：映射列数	关系的“投影”(project)运算，用于选择表达式项的计算
	is_atom：是否要求单行数据	
	INFO_BITS：相关优化信息的标志位，标志位含义如下： bit0：PRJT2 在执行时是否可以使用分组 TOP 跳过优化	
	spl_info：spl 节点编号	
	prjt_batch_exec：从前向后依次展示当前投影表达式中每个表达式对应的批量信息。 目前 PRJT2 中仅展示前五个表达式是否进行批量执行，prjt_batch_exec(1)：是；prjt_batch_exec(0)：否。其余表达式的批量信息统一展示为：too much batch info to display ...	
PSCN	无参数	批量参数当作表来扫描
REMOTE SCAN	tabname@dblink_name：表名@dblink 连接名	DBLINK 远程表扫描
	alias_name：表的别名	
	condition 参数：用于展示 dblink 发送给远程的过滤条件	

RN	无参数	实现 ROWNUM 查询
RNSK	(rownum_exp)：与 rownum 相关的过滤条件	ROWNUM 作为过滤条件时的计算处理
SAGR2	grp_num：分组项个数	如果输入流是有序的，则使用流分组，并计算集函数
	sfun_num：返回上层操作符的集函数个数	
	distinct_flag：集函数参数是否去重	
	top_num：是否只返回前 N 组	
	slave_empty: 是否限制 MPP 从节点向主节点传送数据, 或本地并行的从线程向主线程传送数据。取值：0：不限制；1：不允许 MPP 从节点向主节点传送数据；2：不允许本地并行的从线程向主线程传送数据	
	keys：分组项	
	opt_num(n):内层分组项中的前 n 项可以直接代表外层分组项	
	dist_arg_opt 参数: 标记是否做 distinct 优化（分组项 + 该 distinct 参数 刚好组成 aagr/sagr 使用索引的前导列，那么计算该参数的 dist 时无需使用 hash 表，直接相邻行比较即可（分组项组内参数已有序）），dist_arg_opt(a1,a2,...an)， a 对应每个 distinct 参数, 取值 0 代表此 distinct 参数不优化计算，反之则为 1	

SELECT INTO2	无参数	查询插入
SET TRANSACTION	writeable：是否为可更新事务	事务操作(START 除外)
	iso_level：事务隔离级别，取值：READ UNCOMMITTED：读未提交；READ COMMITTED：读提交；SERIALIZABLE：串行化	
SLCT2	(condition)：过滤条件。当优化成涉及 null 的常量表达式时，可能会显示“UNKNOWN”	关系的“选择”（select）运算，用于查询条件的过滤
	slct_pushdown(1)：是否进行过滤条件下推的优化	
	spl_info：spl 节点编号	
	flt_batch_exec：当前 SLCT2 操作符中表达式整体是否批量执行。flt_batch_exec(1)：是；flt_batch_exec(0)：否	
SORT3	key_num：排序列个数	排序
	partition_key_num：已经有序的前导排序列个数	
	is_distinct：排序时是否进行去重操作	
	top_flag：是否返回排序后的 N 行	
	is_adaptive：是否使用排序的自适应优化（即当下层数据有序时，此次排序实际不进行）	
	can_discard：意义为 sort 节点是否能被摘除（排序操作符在存在 can_discard 标记的情况下以管道模式执行，直接上传左孩子数据）	
SPL2	stask_no：子任务的序号，-1 表示 root 子任务	

	key_num：包含 ROWID 在内的 KEY 数	临时表。和 NTTS2 不同的是，它的数据集不向父亲节点传送，而是被编号，用编号和 KEY 来定位访问；而 NTTS2 的数据，主动传递给父亲节点
	spool_num：SPOOL 临时表编号	
	is_atom：是否限制只能产生单行结果	
	has_var：外层传入变量存在性探测结果（相关查询需探测外层传入变量）	
	sites：站点的 ID 和并行度序列，“-”表示空。例如：(1:3,2:4)表示 1 号站点并行度 3，2 号站点并行度 4	
	result_cache：执行结果缓存机制，仅在 has_var=1 时有效，保存 var 和结果的关系，当下次相同 var 传入时，无需执行，直接返回保存的结果即可	
SSCN	idxname(tabname)：索引名（表名）	直接使用二级索引进行扫描
	btr_scan:等于 1 时表示使用 B 树扫描，等于 0 时表示使用簇游标扫描	
	is_global：等于 1 时表示使用全局索引；等于 0 时表示未使用全局索引	
SSEK2	scan_type：扫描类型	二级索引数据定位
	idxname(tabname)：索引名（表名）	
	scan_range：扫描范围	
	is_global：等于 1 时表示使用全局索引；等于 0 时表示未使用全局索引	
	writeable：是否为可更新事务	启动会话

START TRANSACTION	iso_level：事务隔离级别，取值：READ UNCOMMITTED：读未提交；READ COMMITTED：读提交；SERIALIZABLE：串行化	
STAT	无参数	统计信息计算
TOPN2	top_num：TOP 子句的行数表达式	取前 N 条记录
	top_off：top_num 的偏移量。例如：top_off=3、top_num=4 时，从第 3 行后开始，取第 4 到第 7 共 4 行结果	
	top_percent：top_num 的百分比。例如：top_percent=0.5、top_num=100 时，取前 50 行结果（参数 top_off 和 top_percent 不会同时出现）	
UFLT	IS_TOP_1：是否在更新操作时相同 ROWID 只保留 1 条记录，防止碰到重复数据报错。取值：FALSE：重复数据报错；FIRST：重复数据保留第一行；LAST：重复数据保留最后一行	处理 UPDATE FROM 子句
UNION	无参数	UNION 计算
UNION ALL	CTE：递归 WITH 查询名	UNION ALL 运算
UNION ALL(MERGE)	merge_type：排序方式，取值：A：升序排列；D：降序排列；L：升序排列，NULL 值结果排在末尾；1：降序排列，NULL 值结果排在末尾	UNION ALL 运算（使用归并）
	n_merge_keys：表的列数	

UNION FOR OR(MERGE)	merge_type：排序方式，取值：A：升序排列；D：降序排列；L：升序排列，NULL 值结果排在末尾；1：降序排列，NULL 值结果排在末尾	OR 过滤的 UNION 计算(使用归并)
	n_merge_keys：表的列数	
	n_dist_keys：去重的列数	
UNION FOR OR2	key_num：需要进行去重操作的列数	OR 过滤的 UNION 计算
	outer_join：UNION 操作的外连接类型，取值：L：左外连接；R：右外连接；F：全外连接；-：非外连接	
UPDATE	(partition_type)：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	更新数据
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	
	type：更新类型，取值：SELECT、CURSOR	
	mpp_opt：是否通过发送整个计划实现对 MPP 处理的优化	
	hp_opt：是否进行分区列等值过滤优化	
UPDATE_REMOTE	(partition_type)：分区类型，取值：Vertical Partition：垂直分区；Horizon Partition：水平分区；为空表示不分区	DBLINK 更新操作
	table：表名或视图名，“-”表示空，使用远程操作时会增加“@”+“dblink 连接名”作为后缀	

	type：更新类型，取值：SELECT、CURSOR	
	mpp_opt: 是否通过发送整个计划实现对 MPP 处理的优化	
	hp_opt：是否进行分区列等值过滤优化	
MODEL	cell_references:MODEL 所有不同的单元格引用表达式的数量	MODEL 子句
	sorted:当可以使用 key 进行二分搜索或者 model 唯一性模式为：检查维度的所有组合值对应单元格的唯一性时，sorted(1)，否则 sorted(0)	
MERGE INTO	无参数	合并数据

DM 技术支持

如果您在安装或使用 DM 及其相应产品时出现了问题，请首先访问我们的 Web 站点 http://www.dameng.com/。在此站点我们收集整理了安装使用过程中一些常见问题的解决办法，相信会对您有所帮助。

您也可以通过以下途径与我们联系，我们的技术支持工程师会为您提供服务。

武汉达梦数据库股份有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地

电话：(+86)027-87588000

传真：(+86)027-87588810

武汉达梦数据技术有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地 13-14 层

电话：(+86)027-87588000

传真：(+86)027-87588810

北京达梦数据库技术有限公司

地址：北京市海淀区中关村南大街 2 号数码大厦 B 座 1003

电话：(+86)010-51727900

传真：(+86)010-51727983

上海达梦数据库有限公司

地址：上海市张江高科技园区亮秀路 112 号 A 座 401

电话：(+86)021-50111338

四川蜀天梦图数据科技有限公司

地址：成都市天府新区湖畔西路 99 号 B7 栋（天府英才中心）6 层

电话：（+86）028-61458838

达梦数据技术（江苏）有限公司

地址：苏州吴中经济开发区越溪街道吴中大道 1431 号苏州（太湖）软件产业园智慧谷园区 1601 室 03-04 号

电话：（+86）0512-65285955

传真：（+86）0512-65286955

重庆达梦大数据有限公司

地址：重庆市沙坪坝区重庆师范大学（沙坪坝校区）第二教学楼 1002

电话：（+86）023-68165682

达梦数据技术（深圳）有限公司

地址：深圳市龙岗区园山街道荷坳社区龙岗大道 8288 号大运软件小镇 17 栋 101

技术服务：

咨询热线：400-991-6599

技术支持：dmtech@dameng.com

商务咨询：400@dameng.com

手册版本

版本号：V8

发版日期：2026 二季度版-26876

达梦数据

全栈数据产品与解决方案提供商

扫描了解更多资讯

咨询热线：
400-991-6599
技术支持：dmtech@dameng.com
www.dameng.com

武汉达梦数据库股份有限公司